ТЕХНОЛОГИИ
РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

ТРПО

(Программная инженерия)

Часть 4. Унифицированный язык моделирования UML

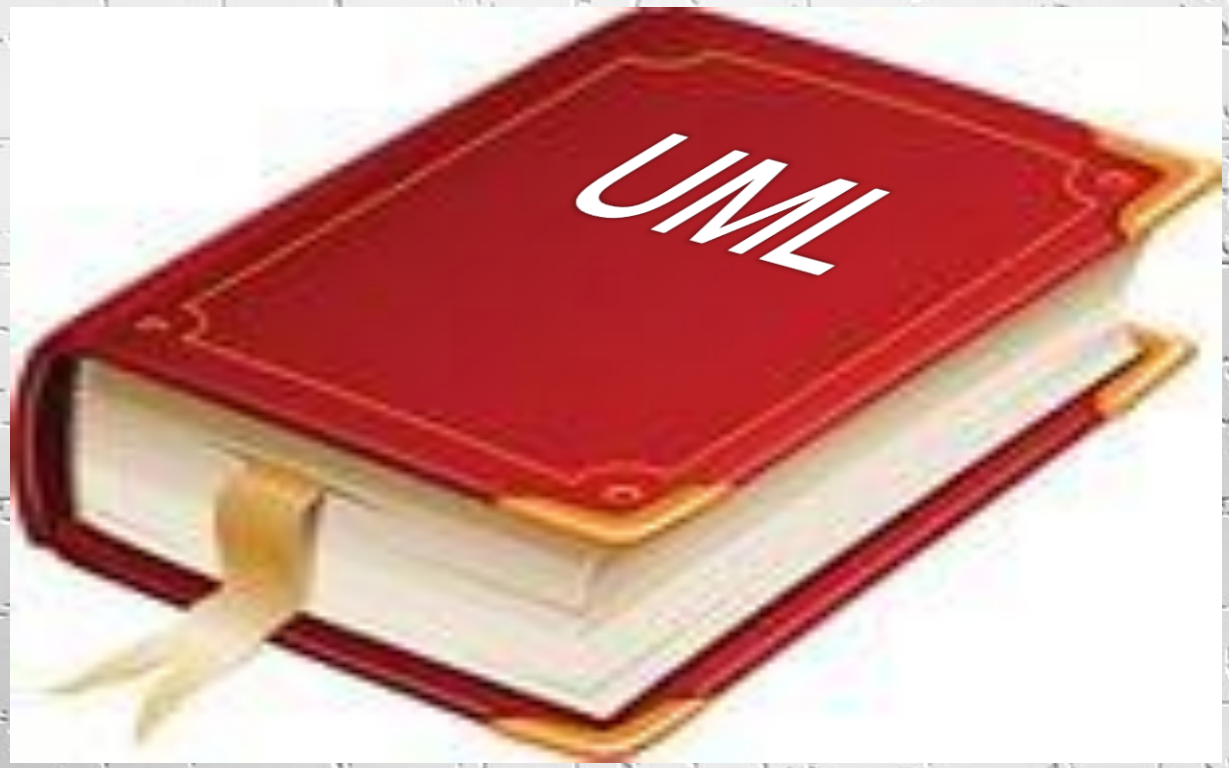
© Ржавин В.В.

ИСТОЧНИКИ

Литература

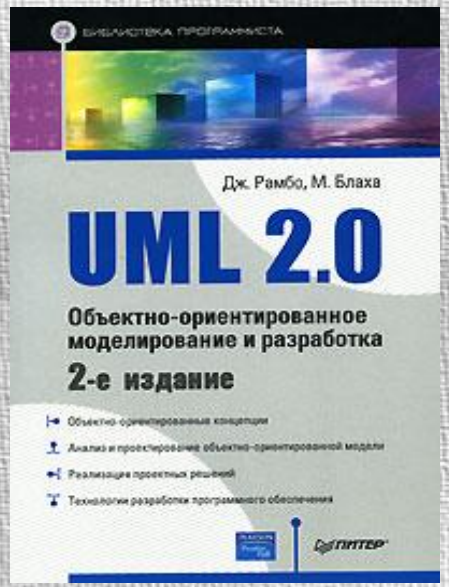
Интернет

Статьи



ЛИТЕРАТУРА

- **Гради Буч, Джеймс Рамбо, Ивар Якобсон**
  Язык UML. Руководство пользователя, 2-е издание
  Издательство: ДМК Пресс , 2007 г.



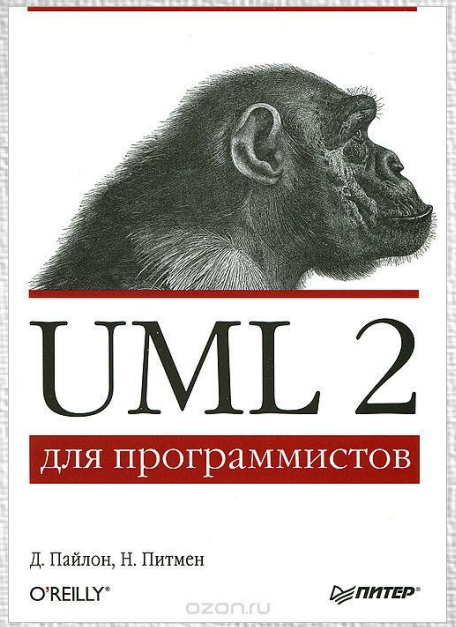
- **Дж. Рамбо, М. Блаха**
  UML 2.0. Объектно-ориентированное моделирование и разработка.
  Издательство: Питер, 2007 г.

- **Джим Арлоу, Айла Нейштадт.**
  UML 2 и Унифицированный процесс. Практический объектно-ориентированный анализ и проектирование, Издательство: Символ-Плюс, 2007 г.

- **Мартин Фаулер**
  UML. Основы, 3-е издание Издательство: Символ-Плюс, 2004 г.

- **Хассан Гома.**
  UML-проектирование систем реального времени параллельных и распределенных приложений.
  Издательство: ДМК Пресс, 2011г.

- *Бабич – А. В.*
  UML: Первое знакомство
  Издательство: Вильямс, 2008 г.

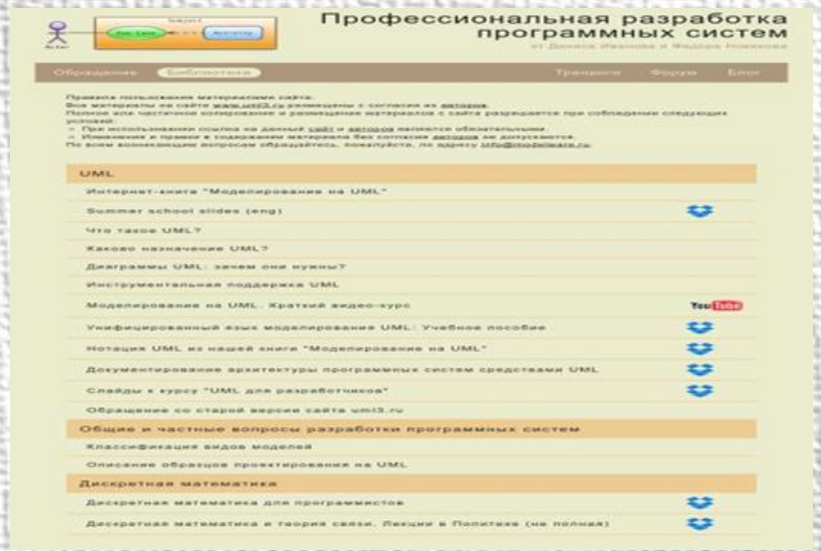


- **Дэн Пайлон, Н. Питмен**
  UML 2 для программистов
  Издательство: Питер, 2012г.

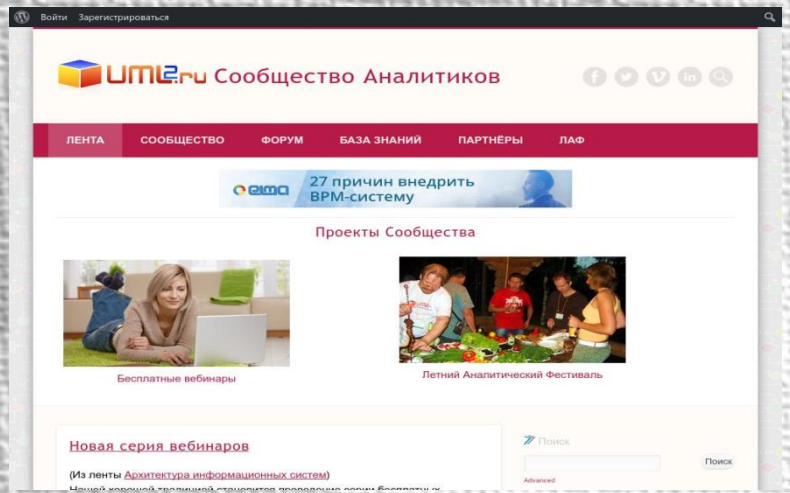
- **Смирнов Михаил**
  UML –быстрый старт
  http://www.msmirnov.ru/public/UML_quick_start.pdf

ИНТЕРНЕТ

ПОРТАЛЫ, БЛОГИ, ФОРУМЫ…

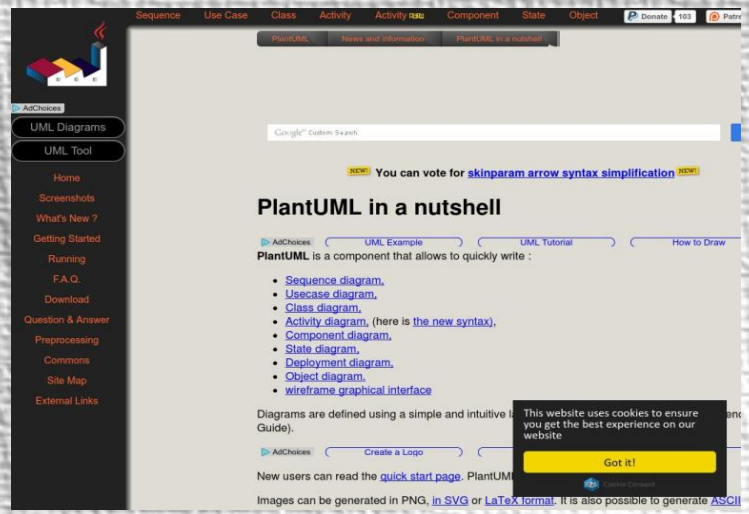


http://uml3.ru/



http://www.uml2.ru/
Сообщество системных аналитиков

http://www.intuit.ru/department/se/intuml

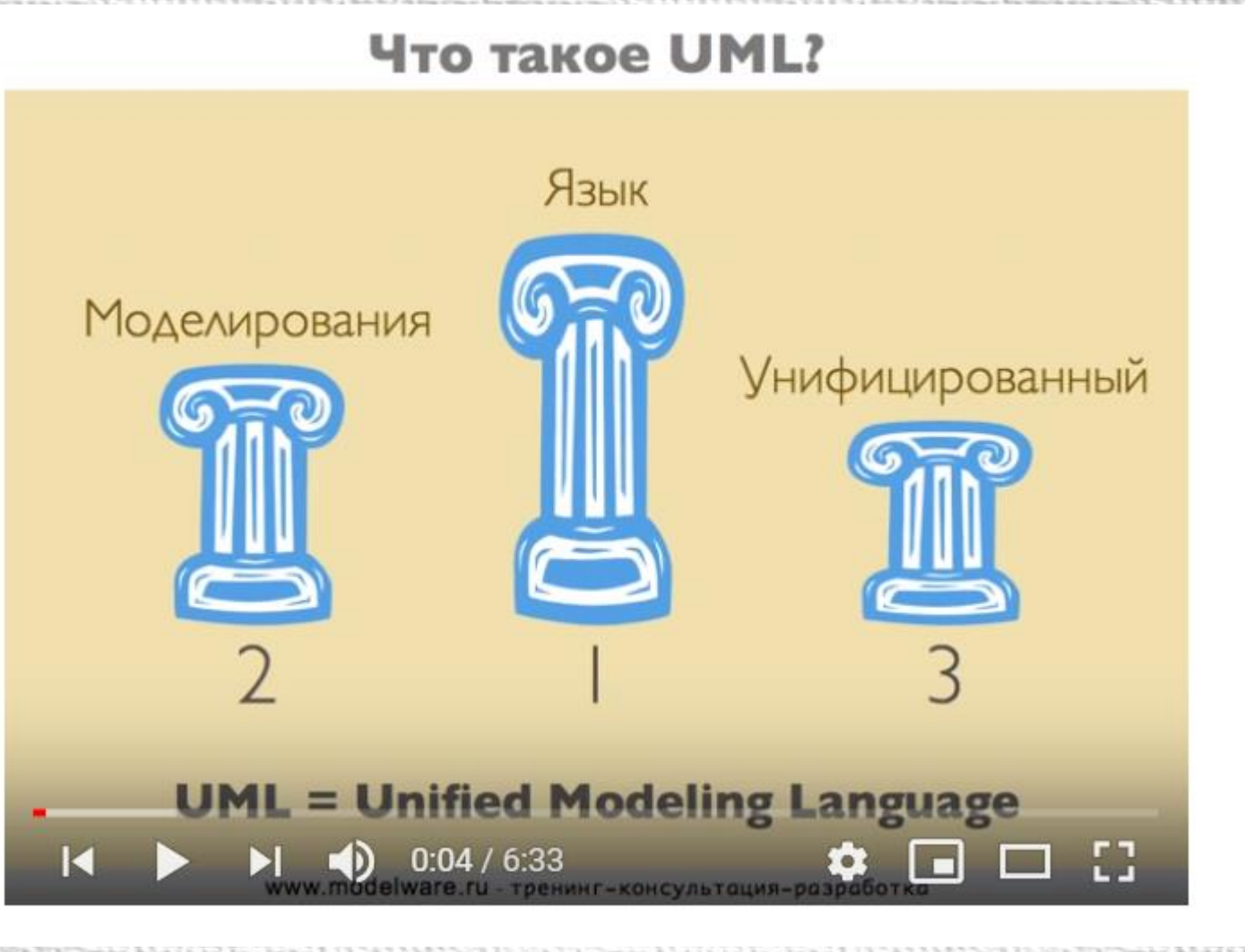


PlantUML - это проект с открытым кодом, позволяющий быстро создавать UML-диаграммы

http://plantuml.com/

ВИДЕО (17 УРОКОВ)

https://www.youtube.com/playlist?list=PL_DKSMPuELblFOEfu98AAtPLJVt7LqtXl



лекции

Историческая справка

- …
- Блок-схемы
- Диаграммы потоков данных (DFD)
- Диаграммы «сущность-связь» (ERD)
- Методология структурного анализа и проектирования (SADT)
- Объектно-ориентированный анализ (OOA)
- …

ОСНОВЫ ЯЗЫКА UML
(UNIFIED MODELING LANGUAGE)

Я слышу и забываю. Я вижу и запоминаю. Я делаю и понимаю.
Конфуций

© Ржавин В.В

ПЛАН

Обзор UML

Концептуальная модель UML

Архитектура программного обеспечения

Процесс разработки программного обеспечения

ОБЗОР UML

ЧТО ТАКОЕ UML

Правильный перевод этого названия на русский язык – **унифицированный язык моделирования.**

1. **UML – это язык**
2. **UML – это язык моделирования**
3. **UML – это унифицированный язык моделирования**

UML – это язык

Язык – это знаковая система для хранения и передачи информации.

Различаются языки **формальные**, правила употребления которых строго и явно определены, и **неформальные**, употребление которых основано на сложившейся практике.

Различаются также языки **естественные**, появляющиеся как бы сами собой в результате неперсонифицированных усилий массы людей, и языки **искусственные**, являющиеся плодом видимых усилий определенных лиц.

Так вот, UML можно охарактеризовать как **формальный искусственный язык**, хотя и не в такой степени, как многие распространенные языки программирования. Признаком искусственности служит наличие трех общепризнанных авторов – Гради Буча, Ивара Якобсона и Джеймса Рамбо.

Для описания формальных искусственных языков (в частности, для описания языков программирования) придумано и используется множество различных способов. Однако на практике сложилась общепринятая структура таких описаний. Считается, что формальный искусственный язык описан должным образом, если это описание содержит, по меньшей мере, следующие части.

1. **Синтаксис** (syntax), то есть определение правил составления конструкций языка.
2. **Семантика** (semantics), то есть определение правил приписывания смысла конструкциям языка.
3. **Прагматика** (pragmatics), то есть определение правил использования конструкций языка для достижения определенных целей.

Как формальный искусственный язык UML имеет синтаксис, семантику и прагматику, хотя эти части названы в некоторых случаях иначе и описаны по другому, нежели это принято в текстовых языках программирования, поскольку, во-первых, **UML язык графический, а не текстовый**, а во-вторых, **UML язык моделирования, а не программирования**.

Синтаксически правильно построенная фраза

«Гло́кая ку́здраште́ко будлану́ла бо́кра и курдя́чит бокрёнка»

Искусственная фраза на основе русского языка, в которой все корневые морфемы заменены на бессмысленные сочетания звуков. Несмотря на это, общий смысл фразы понятен: некоторая определённым образом характеризуемая сущность женского рода что-то сделала определённым образом с другим существом мужского пола, а затем начала (и продолжает до настоящего момента) делать что-то другое с его детёнышем (или более мелким представителем того же вида). Фраза создана для иллюстрации того, что многие семантические признаки слова можно понять из его морфологии.

Пример был предложен академиком Л. В. Щербой в 1930-е годы.

UML – это язык моделирования

Слово "моделирование", входящее в название UML, имеет множество смысловых оттенков и сложившихся способов употребления. В частности, английские слова *modeling* и *simulation* оба переводятся словом "моделирование", хотя означают разные вещи. В первом случае речь идет о составлении модели, которая используется только для описания моделируемого объекта или явления. Во втором случае подразумевается составление модели, которая может быть использована для получения существенной информации о моделируемом объекте или явлении. При этом во втором случае обычно добавляется уточняющее прилагательное: численное, имитационное, математическое моделирование и др. UML является языком моделирования в первом смысле.

В отношении разработки программного обеспечения так сложилось, что результаты фаз анализа и проектирования, оформленные средствами определенного языка, принято называть моделью, что согласуется с общим определением

Модель – абстрактное описание чего-либо

Деятельность по составлению моделей естественно назвать **моделированием**. Именно в этом смысле UML является языком моделирования.

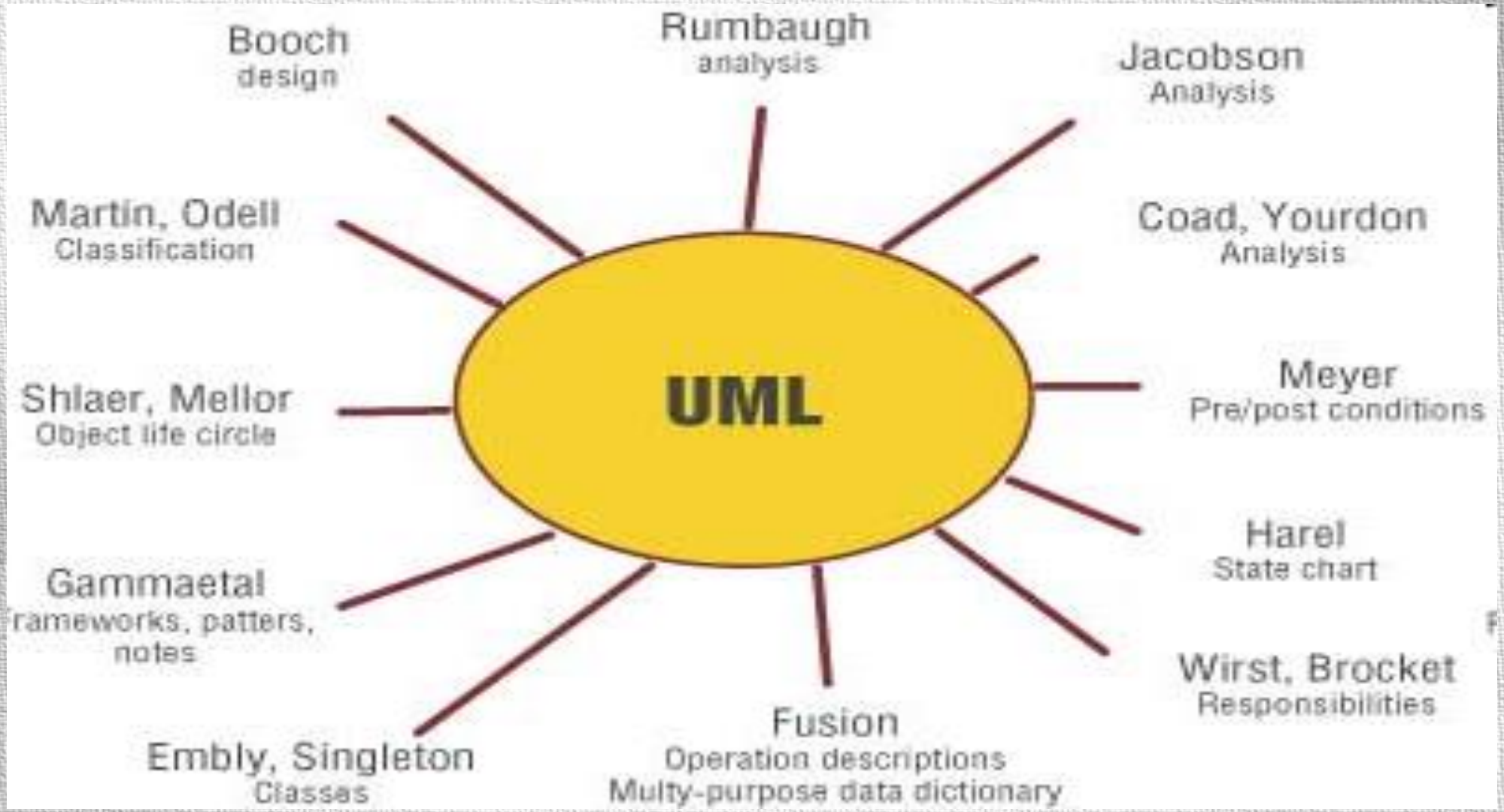
Таким образом, модель UML – это, прежде всего, **описание объекта или явления**.

UML – это унифицированный язык моделирования

Приложив заслуживающие уважения усилия, авторы UML при поддержке и содействии всей международной программистской общественности смогли свести воедино (*унифицировать*) большую часть того, что было известно и до них. **В результате унификации получилась теоретически изящная и практически полезная вещь – UML.**

Если попытаться проследить историю возникновения и развития элементов UML, как на уровне основополагающих идей, так и на уровне технических деталей, то пришлось бы назвать сотни имен и десятки организаций.

UML и его предшественники



К 1994-му существовало 72 метода, или частные методики. Многие из них "перекрывались", т. е. использовали похожие идеи, нотации и т. д. Как уже говорилось выше, чувствовалась острая потребность, "социальный заказ" - закончить "войну методов" и объединить в одном унифицированном средстве все лучшее, что было создано в области моделирования.

В литературе можно встретить описание эры "до *UML*" как "войны методов" моделирования, ни один из которых "не дотягивал" до уровня индустриального стандарта. *UML* как раз и стал таким единым стандартом для *объектно-ориентированного моделирования*, которое во времена его создания как раз "вошло в моду". "Единым" языком моделирования *UML* можно назвать еще и потому, что в его создании, как мы увидим далее, объединились усилия авторов трех наиболее популярных методов моделирования.

Появление ООП требовало удобного инструмента для моделирования, единой нотации для описания сложных программных систем. И вот "три амиго", три крупнейших специалиста, три автора наиболее популярных методов решили объединить свои разработки. В 1991-м каждый из "трех амиго" начал с написания книги, в которой изложил свой метод ООАП. Каждая методология была по-своему хороша, но каждая имела и недостатки. Так, метод Буча был хорош в проектировании, но слабоват в анализе. OMT Румбаха был, наоборот, отличным средством анализа, но плох в проектировании. И наконец, Objectory Якобсона был действительно хорош с точки зрения user experience, на который ни метод Буча, ни OMT не обращали особого внимания. Основной идеей Objectory было то, что анализ должен начинаться с прецедентов, а не с диаграммы классов, которые должны быть производными от них.

Авторы UML

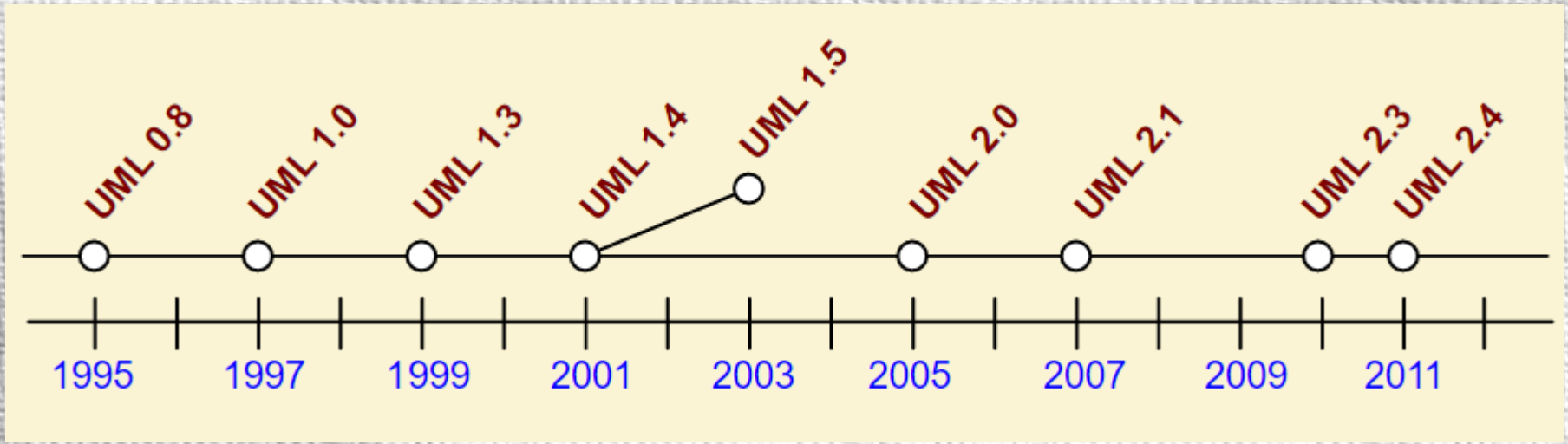
Гради
Буч

Джеймс
Рамбо

Айвар
Якобсон

ИСТОРИЯ РАЗВИТИЯ UML

История развития UML отнюдь не завершена – язык постоянно совершенствуется, обогащается и расширяется. Можно привести картинку, иллюстрирующую историю развития UML.



Последняя версия UML 2.5 опубликована в июне 2015 года, в 2017 – 2.5.1.

.. для моделирования любых систем

Унифицированный язык моделирования (UML) является стандартным инструментом для создания "чертежей" программного обеспечения. С помощью UML можно визуализировать, специфицировать, конструировать и документировать артефакты программных систем.

Авторы *UML* определяют его как графический *язык моделирования* общего назначения (т. е. его можно применять для проектирования чего угодно - от простой качели, как на рисунке, до сложного аппаратно-программного комплекса или даже космического корабля).

UML пригоден для моделирования любых систем: от информационных систем масштаба предприятия до распределенных Web-приложений и даже встроенных систем реального времени. Это очень выразительный язык, позволяющий рассмотреть систему со всех точек зрения, имеющих отношение к ее разработке и последующему развертыванию.

НАЗНАЧЕНИЕ UML

Авторы *UML* определяют его как графический *язык моделирования* общего назначения, т. е. его можно применять для проектирования чего угодно - от простой качели, как на рисунке, до сложного аппаратно-программного комплекса или даже космического корабля.

UML пригоден для моделирования любых систем: от информационных систем масштаба предприятия до распределенных Web-приложений и даже встроенных систем реального времени. Это очень выразительный язык, позволяющий рассмотреть систему со всех точек зрения, имеющих отношение к ее разработке и последующему развертыванию.

Унифицированный язык моделирования (UML) является стандартным инструментом для создания "чертежей" программного обеспечения.

Язык UML – это графический язык моделирования общего назначения, предназначенный для спецификации, визуализации, проектирования и документирования всех артефактов, создаваемых при разработке программных систем.

СПЕЦИФИКАЦИЯ

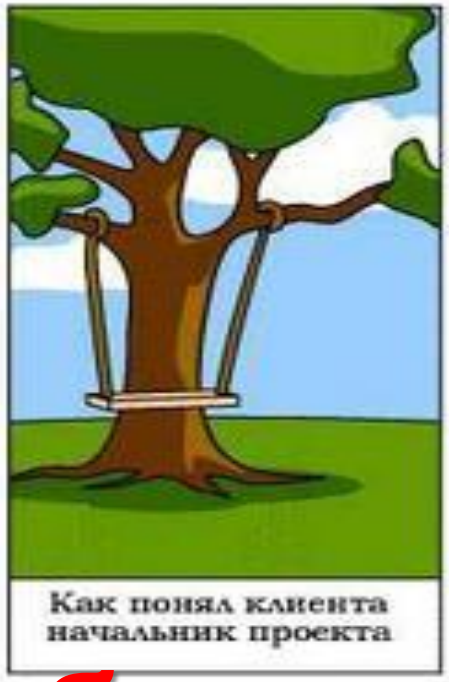
Итак, UML в первую очередь — это спецификации. Заглянем в словарь и обнаружим, что

Спецификация — *подробное описание системы, которое полностью определяет ее цель и функциональные возможности.*

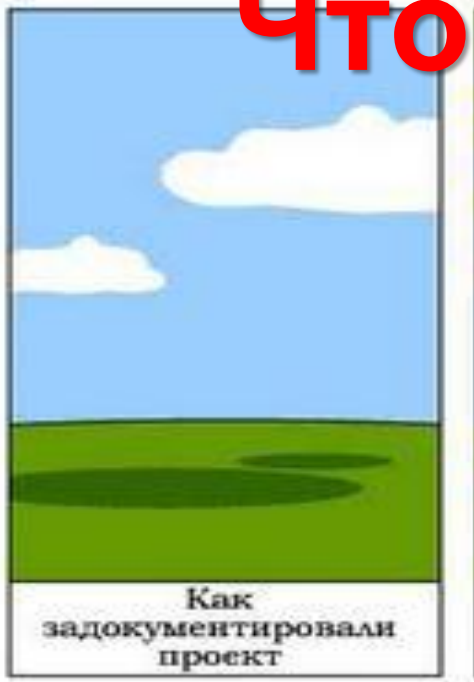
Различают:

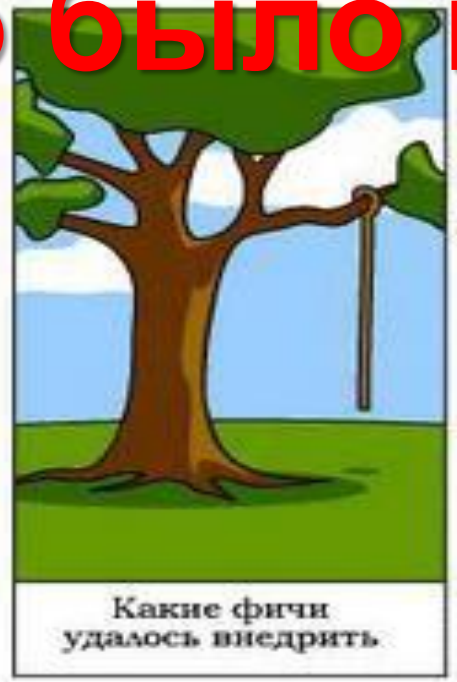
- **словесные спецификации на естественном языке;**
- **формальные спецификации;**
- **модельные спецификации**.

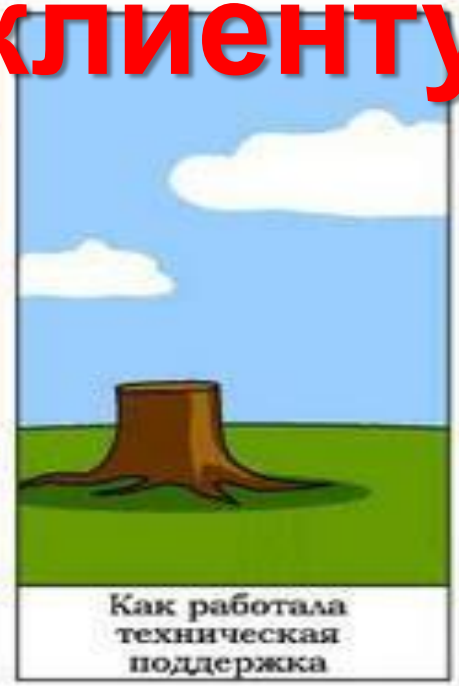
www.oper.ru



Что было нужно клиенту?







Не следует также забывать, что заказчик и разработчик имеют, как правило, абсолютно разное понимание смысла этого артефакта. А ведь кроме этого есть еще аналитики, менеджеры, бизнес-консультанты... Каждый из них называет спецификации по-своему: постановка задачи, требования пользователя, техническое задание, функциональная спецификация, архитектура системы... Причем все эти люди, являясь специалистами в абсолютно разных предметных областях, говорят каждый на своем языке и зачастую просто не понимают друг друга. Вот потому-то и возникает проблема, представленная на рисунке, проблема, которую может решить только наличие **единого, унифицированного средства создания спецификаций, достаточно простого и понятного для всех заинтересованных лиц.**

Словесные спецификации на естественном языке как раз и вызывают массу проблем, поскольку создаются разными специалистами на "их языке".
Другим видом спецификаций являются формальные спецификации. Действительно, описание спецификации с помощью строгого математического языка было бы чудесным решением всех проблем, т. к. сам способ записи исключал бы малейшие неоднозначности. Да, в математике есть, например, алгебра высказываний, с помощью которой можно пытаться создавать технические задания на разработку некоторых приложений. *формальная спецификация* является, *по* сути, математической *моделью задачи* и потому для вычислительных задач все выглядит достаточно просто. Формализация же задач из других областей знаний может оказаться более сложной и трудоемкой проблемой, чем разработка самого приложения ввиду отсутствия четкой математической модели. Один из принципов прикладной "мерфологии" гласит, что лучшей *спецификацией программы* является ее текст.

- Когда мы говорим о том, что UML - это средство визуализации, мы имеем в виду **модельные спецификации**.

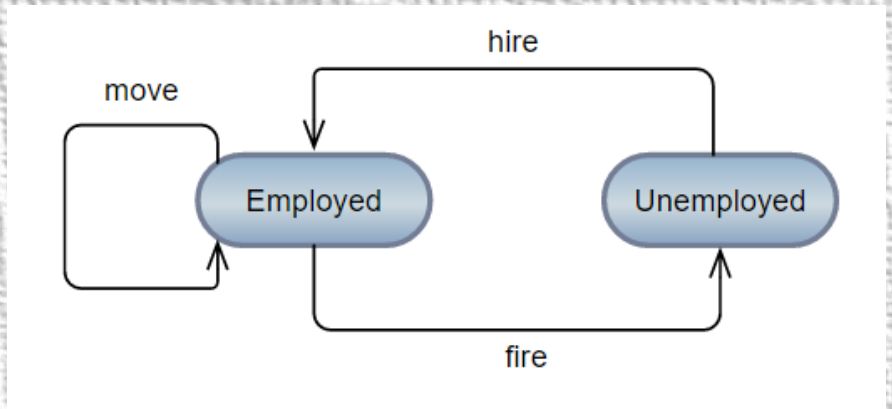
- Все мы знаем, как иногда трудно заставить себя "вникнуть" в суть материала, излагаемого в очередном учебнике или руководстве. Изучение чего-то нового идет гораздо проще, если документ содержит не только текст, а еще и иллюстрации к нему. А если руководство или учебник выглядят как картинки с подписями, то усвоение нового материала происходит еще проще и эффективнее.

ВИЗУАЛИЗАЦИЯ

Известная поговорка гласит, что лучше один раз увидеть, чем сто раз услышать. Особенности человеческого восприятия таковы, что текст с картинками воспринимается легче, чем голый текст. А картинки с текстом (это называется "комиксы") – еще легче. Модели UML допускают представление в форме картинок, причем эти картинки наглядны, интуитивно понятны, практически однозначно интерпретируются и легко составляются.

Разве что-нибудь непонятно?

Таким образом, второе по важности назначение UML состоит в том, чтобы служить адекватным средством коммуникации между людьми. Разумеется, наглядность визуализации моделей UML имеет значение, только если они должны составляться или восприниматься человеком – это назначение UML не имеет отношения к компьютерам.



Жизненный цикл работника на предприятии

Так вот, такие картинки с подписями наглядны и интуитивно понятны, причем почти **однозначно понимаются любыми заинтересованными лицами**, так что могут использоваться в качестве средства общения между людьми. UML позволяет создавать такие простые и понятные картинки (модели), описывающие систему с разных сторон, которые можно показать заказчику и обсудить с ним, т. е. служит средством коммуникации в команде.

ПРОЕКТИРОВАНИЕ

В оригинале данное назначение UML определено с помощью слова construct, которое ближе не к проектированию, а к конструированию ПО, то есть программированию, реализации. Речь идет о том, что UML предназначен не только для описания абстрактных моделей приложений, но и для **создания таких моделей, для которых возможна автоматическая генерация программного кода** (или фрагментов кода) соответствующих приложений. Инструменты, поддерживающие UML, все время совершенствуются, так что в перспективе третье предназначение UML может выйти и на первое место. Хочется надеяться, что когда-нибудь мы сможем создавать приложения визуально, не прибегая к языку программирования, а пользуясь лишь *UML*.

MDA - Model Driven Architecture

ДОКУМЕНТИРОВАНИЕ

Модели UML являются артефактами, которые можно хранить и использовать как в форме электронных документов, так и в виде твердой копии. Кроме этого, как мы еще упомянем далее, модели UML являются XML-документами. Причем любой элемент на любой диаграмме может быть снабжен текстовым комментарием. Т. е. построение набора диаграмм уже является процессом документирования будущей системы. Более того, большинство инструментов UML-проектирования умеют извлекать текстовую информацию из моделей и генерировать относительно удобочитаемые тексты. К сожалению, это замечательная возможность на практике используется меньше, чем она того заслуживает.

Способы использования UML

Рисование картинок. Графические средства UML можно и нужно использовать безотносительно ко всему остальному. Даже рисование диаграмм карандашом на бумаге позволяет упорядочить мысли и зафиксировать для себя существенную информацию о моделируемом приложении или иной системе.

Обмен информацией. Сообщество людей, применяющих и понимающих UML, стремительно растет. Если вы будете использовать UML, то вас будут понимать другие, и вы будете понимать других "с полувзгляда".

Спецификация систем. Это важнейший способ использования UML. И хотя не во всех случаях UML оказывается абсолютно адекватным средством спецификации, надеемся, что по мере развития языка все меньше будет оставаться таких исключений, где UML неприменим.

Повторное использование архитектурных решений. Повторное использование ранее разработанных решений – ключ к повышению эффективности. К сожалению, модели UML пока что повторно используются в весьма ограниченных масштабах.

Генерация кода. Генерировать код нужно и можно, но возможности имеющихся инструментов не стоит переоценивать.

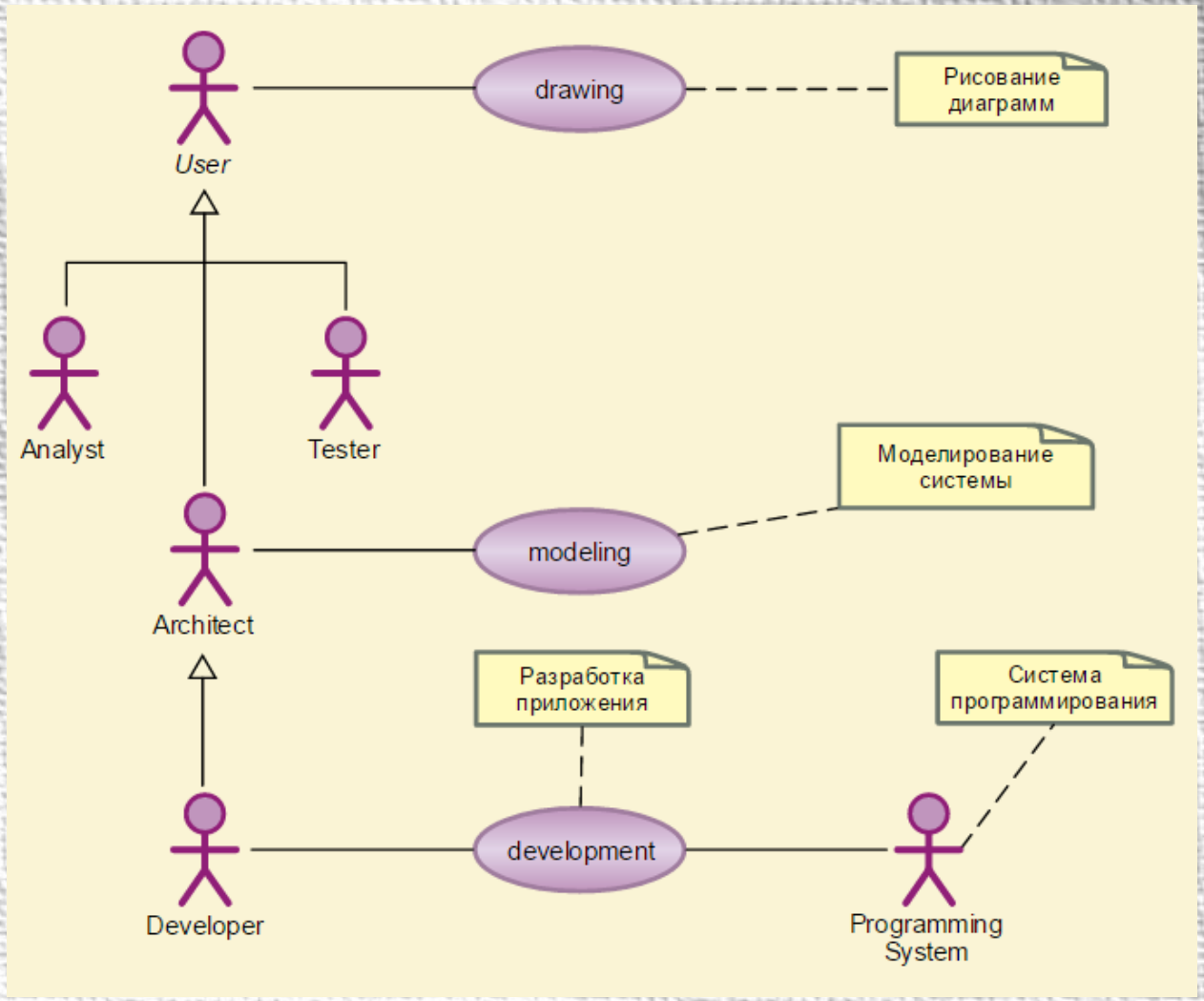
Имитационное моделирование. Возможности построения моделей UML, из которых путем вычислительных экспериментов можно было бы извлекать информацию о моделируемом объекте, пока что уступают возможностям специализированных систем, сконструированных для этих целей.

Верификация моделей. Было бы замечательно, если бы по модели можно было бы делать формальные заключения об ее свойствах: модель непротиворечива, согласована, эффективна и т.п. Кое-что UML позволяет проверить, но, конечно, очень мало. Здесь уместно привести аналогию с традиционными системами программирования: они позволяют быстро и надежно избавиться от синтаксических ошибок, но с логическими ошибками дело обстоит гораздо хуже. Может быть, в будущем...

ИНСТРУМЕНТАЛЬНАЯ ПОДДЕРЖКА

Варианты использования UML:

1. **Рисование диаграмм**
2. **Моделирование систем**
3. **Разработка приложений**



Современные инструменты поддерживают указанные варианты использования далеко не в равной степени. Все инструменты умеют (плохо или хорошо) визуализировать все типы диаграмм UML, некоторые инструменты позволяют построить модель, допускающую какое-то дальнейшее использование, но только немногие инструменты могут генерировать исполняемый код и то, отнюдь, не для всех диаграмм.

ОПРЕДЕЛЕНИЕ UML

Искусственный язык должен быть как-то описан. Искусственный язык, претендующий на массовое использование, должен быть описан так, чтобы притягивать, а не отпугивать потенциальных пользователей. А это сделать не так просто.

В основу описания UML положен метод раскрутки, то есть использование определяемого языка для определения этого языка. Другими словами язык описывает сам себя. А именно, основные конструкции UML формально определены с помощью UML. Конечно, с чего-то раскрутку нужно начать, и это описано в UML неформально, с помощью текстов на естественном (английском) языке.

УРОВНИ ОПИСАНИЯ

В описании UML используются три языковых уровня.

1. **Мета-метамодель**, то есть описание языка, на котором описана метамодель.
2. **Метамодель**, то есть описание языка, на котором описываются модели.
3. **Модель**, то есть описание самой моделируемой предметной области.

В древнегреческом языке предлог μετά (metá) и приставка μετα- имеет значения: «после», «следующее», «за», а также «через», «между».

В эпистемологии приставка «**мета-**» означает «**о себе**». Например, метаданные — данные о данных.

Метафизика описывает принципы на которых основана физика, является наукой более высокого уровня по отношению к ней.

Это обобщение передалось приставке мета-.

ПРИМЕР

Рассмотрим описание какого-либо из обычных языков программирования. Пусть этот язык называется X. Если описание хорошее, то в самом начале указывается язык (иногда его так и называют – метаязык), который используется для описания языка X. Например, приводится фраза такого типа: "синтаксис языка X описан с помощью контекстно-свободной грамматики, правила которой записаны в форме Бэкуса-Наура (БНФ), контекстные условия и семантика описаны на естественном языке". Если описание не очень хорошее, то такая фраза может и отсутствовать, но она все равно неявно подразумевается и от читателя требуется понять используемый метаязык по контексту.

Далее с помощью метаязыка более или менее формально описываются конструкции языка X; все, что не удается описать формально, описывается на естественном языке (при этом зачастую вводится большое количество "новых" терминов и используются трудные для чтения канцеляризмы). Все это, как правило, сопровождается многочисленными конкретными примерами фрагментов текстов на языке X. Чтобы читатель не путался, где текст на языке X, а где текст на метаязыке, или где общее описание, а где конкретный пример, применяются различные полиграфические приемы: изменение гарнитуры и начертания шрифта, отступы, подчеркивание и т.д. В результате получается неплохо: многоязыковая смесь становится вполне удобочитаемой (при минимальном навыке).

Основная идея описания UML вполне традиционна и согласуется с общепринятой практикой:

- **мета-метамодель** – это описание используемого формализма;
- **метамодель** – это и есть собственно описание языка (элементов моделирования);
- там, где формализм не срабатывает, на помощь приходит естественный язык и все это сопровождается примерами фрагментов моделей.

Комплект документации UML

Документ и его содержание	Кол-во страниц
UML Infrastructure Описание базовых механизмов, а также архитектуры, профилей, стереотипов	230
UML Superstructure Описание статических и динамических элементов языка	748
UML Diagram interchange Описание формата обмена дополнительными данными о диаграммах (форматирование, расположение элементов и т.д.) между различными инструментами	86
OCL Specifications Описание объектного языка ограничений OCL (Object Constraint Language)	244
	1308

ТЕРМИНОЛОГИЯ И НОТАЦИЯ

- Проблема терминологии является одной из самых болезненных при обсуждении языка UML.
  - Во-первых, авторы UML старались сделать язык **независимым от конкретных языков программирования**, моделей вычислимости и тому подобного. С этой целью они зачастую вводили новые термины для определяемых понятий, чтобы случайное совпадение со старым термином, уже занятым в какой-либо смежной области программирования, не вводило в заблуждение пользователя.
  - Во-вторых, это проблема перевода английской терминологии на русский язык.

- То, что в других языках называют "синтаксисом« в графическом языке называется "нотация". Само слово "нотация" подчеркивает, что UML - язык графический и модели (а точнее диаграммы) не "записывают", а рисуют.

ЭЛЕМЕНТЫ НОТАЦИИ

При разработке UML были предложены и приняты разумные рекомендации по выбору нотации. Авторы исходили из того, что UML будет использоваться по-разному: начиная от не очень аккуратного рисования от руки на листке бумаги, печати черно-белых изображений в книгах и заканчивая созданием сложных диаграмм с помощью компьютера. Поэтому в качестве основных графических элементов были выбраны такие, которые было бы легко использовать во всех случаях. Типов элементов нотации пять:

1. **фигура (shape);**
2. **линия (line);**
3. **значок (icon);**
4. **текст (text);**
5. **рамка (frame).**

Фигуры (shape)

Фигуры (shape) в UML используются двумерные (т.е. их можно нарисовать на плоскости) и замкнутые (т.е. есть внутренняя и внешняя части). Фигуры могут менять свои размеры и форму, сохраняя при этом свои интуитивные отличительные признаки. Например, среди фигур UML есть прямоугольники и эллипсы. Они могут быть изображены многими способами: разного размера, с разным соотношением длин сторон (или, соответственно, полуосей), по-разному ориентированы относительно границ страницы и т.д., но всех случаях прямоугольник отличен от эллипса и не может быть с ним спутан. Внутри фигур могут помещаться другие элементы нотации: тексты, линии, значки и даже другие фигуры. Единственное требование: должно быть однозначно понятно, что элемент нотации находится внутри фигуры, в частности, его изображение не должно пересекать границу фигуры.

Линии (line)

Линии (line) в UML, естественно, одномерные. Линии всегда присоединяются своими концами к фигурам или значкам, они не могут быть нарисованы сами по себе. Форма линий произвольна: это могут быть прямые, ломаные, плавные кривые – значения это не имеет. Толщина линий также произвольна. А вот стиль линии имеет значение. К счастью, **в UML используется только два стиля линий, которые трудно спутать: сплошные и пунктирные линии**. К линиям могут быть пририсованы различные дополнительные элементы: стрелки на концах, тексты и т.д. Единственное требование: должно быть ясно, что дополнительный элемент относится именно к данной линии. Линии могут пересекаться, и это ничего не значит, но рекомендуется избегать таких случаев, поскольку это затрудняет восприятие.

Значки (icon

Значки (icon) в UML похожи на фигуры тем, что они двумерные, а отличаются тем, что не имеют внутренности, в которую можно что-то поместить, и, как правило, не меняют свою форму и размеры. Впрочем, значки в UML используются очень умеренно, а потому сохраняют свою основную функцию однозначно воспринимаемого иероглифа.

Тексты (text)

Тексты (text) в UML – это, как обычно, последовательности различимых символов некоторого алфавита. Алфавит не фиксирован – он только должен быть понятен читателю модели. Гарнитура, размер и цвет шрифта не имеют значения, а вот начертание шрифта имеет: **в UML различаются прямые, *курсивные* и подчеркнутые тексты**. Предполагается, что читатель сумеет их различить.

Рамки (frame)

Рамки (frame) появились в UML 2. Рамка – это частный случай фигуры, которая используется исключительно как контейнер для других фигур, линий, значков и текстов. Пустые рамки не применяются. **Рамка имеет прямоугольную форму и, как правило, ярлычок в левом верхнем углу, в котором указывается тип и имя рамки.**

КОНЦЕПТУАЛЬНАЯ МОДЕЛЬ UML

КОНЦЕПТУАЛЬНАЯ МОДЕЛЬ UML

Чтобы понять UML, вам необходимо сформировать концептуальную модель языка, а это требует изучения трех основных элементов:

1. **строительных блоков языка,**
2. **правил, определяющих их сочетания,**
3. **некоторых общих для всего языка механизмов.**

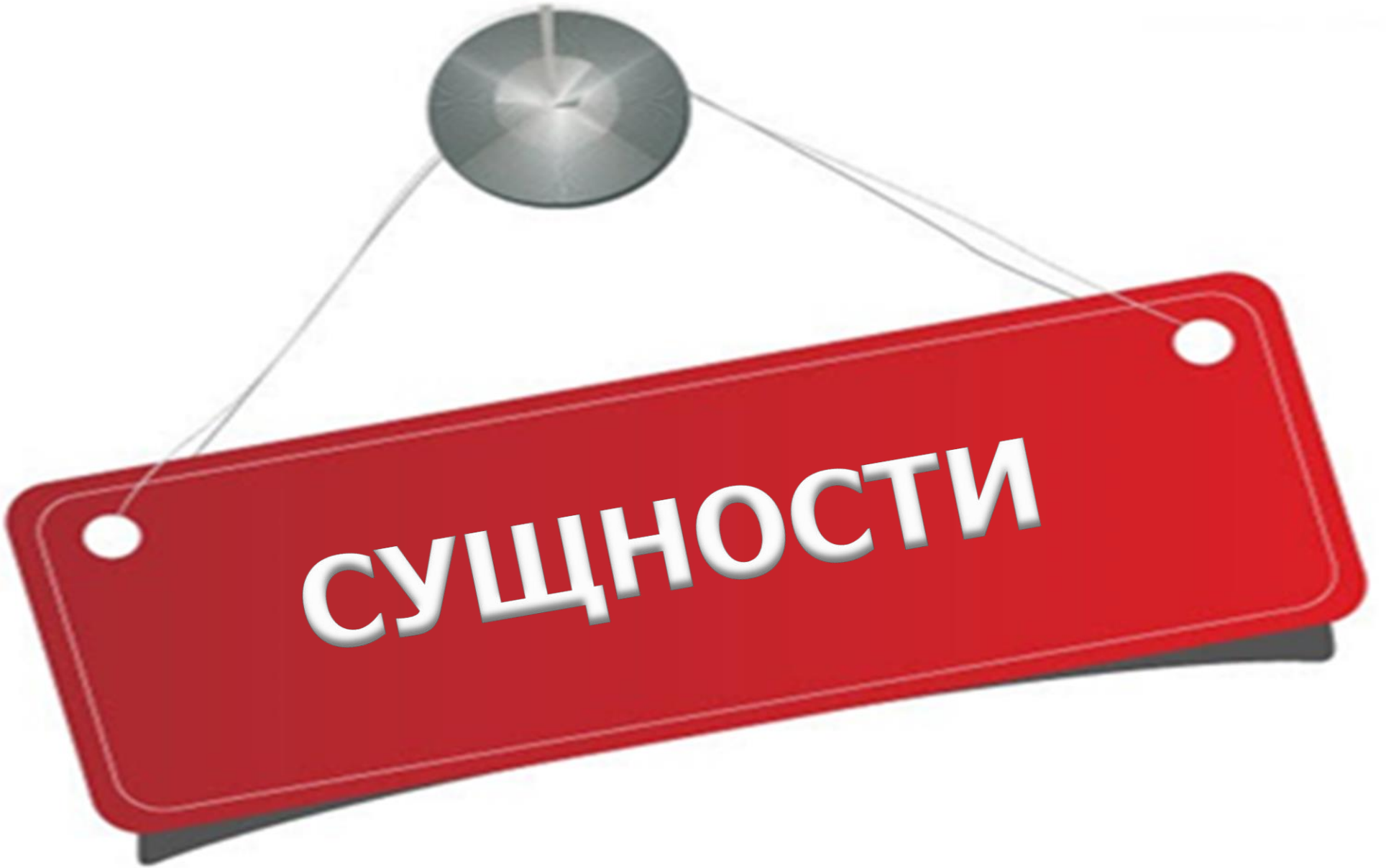
Лишь усвоив эти идеи, вы сможете читать UML-модели и создавать их самостоятельно.

- *Сущности*
- *Отношения*
- *Диаграммы*

СТРОИТЕЛЬНЫЕ БЛОКИ UML

Словарь языка UML включает три вида строительных блоков:

1. **Сущности** - это абстракции, являющиеся основными элементами модели.
2. **Отношения** связывают различные сущности.
3. **Диаграммы** группируют представляющие интерес совокупности сущностей.


СУЩНОСТИ

ТИПЫ СУЩНОСТЕЙ

В UML имеется четыре типа сущностей:

1. **структурные;**
2. **поведенческие;**
3. **группирующие;**
4. **аннотационные**.

Все они представляют собой базовые объектно-ориентированные строительные блоки моделей UML.

1

СТРУКТУРНЫЕ СУЩНОСТИ

СТРУКТУРНЫЕ СУЩНОСТИ

Структурные сущности - это **имена существительные** в моделях на языке UML
Как правило, они представляют собой статические части модели, соответствующие концептуальным или физическим элементам системы.

Существует восемь видов структурных сущностей:

1. **Класс**
2. **Интерфейс**
3. **Кооперация**
4. **Прецедент (Use Case)**
5. **Объект**
6. **Компонент**
7. **Артефакт**
8. **Узел**

В совокупности структурные сущности называются **классификаторами**.

СТРУКТУРНЫЕ СУЩНОСТИ

1. ***Объект*** **(object)** – сущность, обладающая уникальностью и инкапсулирующая в себе состояние и поведение.
2. ***Класс*** **(class)** – описание множества объектов с общими атрибутами, определяющими состояние, и операциями, определяющими поведение.
3. ***Интерфейс*** **(interface)** – именованное множество операций, определяющее набор услуг, которые могут быть запрошены потребителем и предоставлены поставщиком услуг.
4. ***Кооперация*** **(collaboration)** – совокупность объектов, которые взаимодействуют для достижения некоторой цели.

5. ***Действующее лицо*** **(actor)** – сущность, находящаяся вне моделируемой системы и непосредственно взаимодействующая с ней.
6. ***Компонент*** **(component)** – модульная часть системы с четко определенным набором требуемых и предоставляемых интерфейсов.
7. ***Артефакт*** **(artifact)** – элемент информации, который используется или порождается в процессе разработки программного обеспечения. Другими словами, артефакт – это физическая единица реализации, получаемая из элемента модели (например, класса или компонента).
8. ***Узел*** **(node)** – вычислительный ресурс, на котором размещаются и при необходимости выполняются артефакты.

НОТАЦИЯ СТРУКТУРНЫХ СУЩНОСТЕЙ

Объект

Класс

Интерфейс

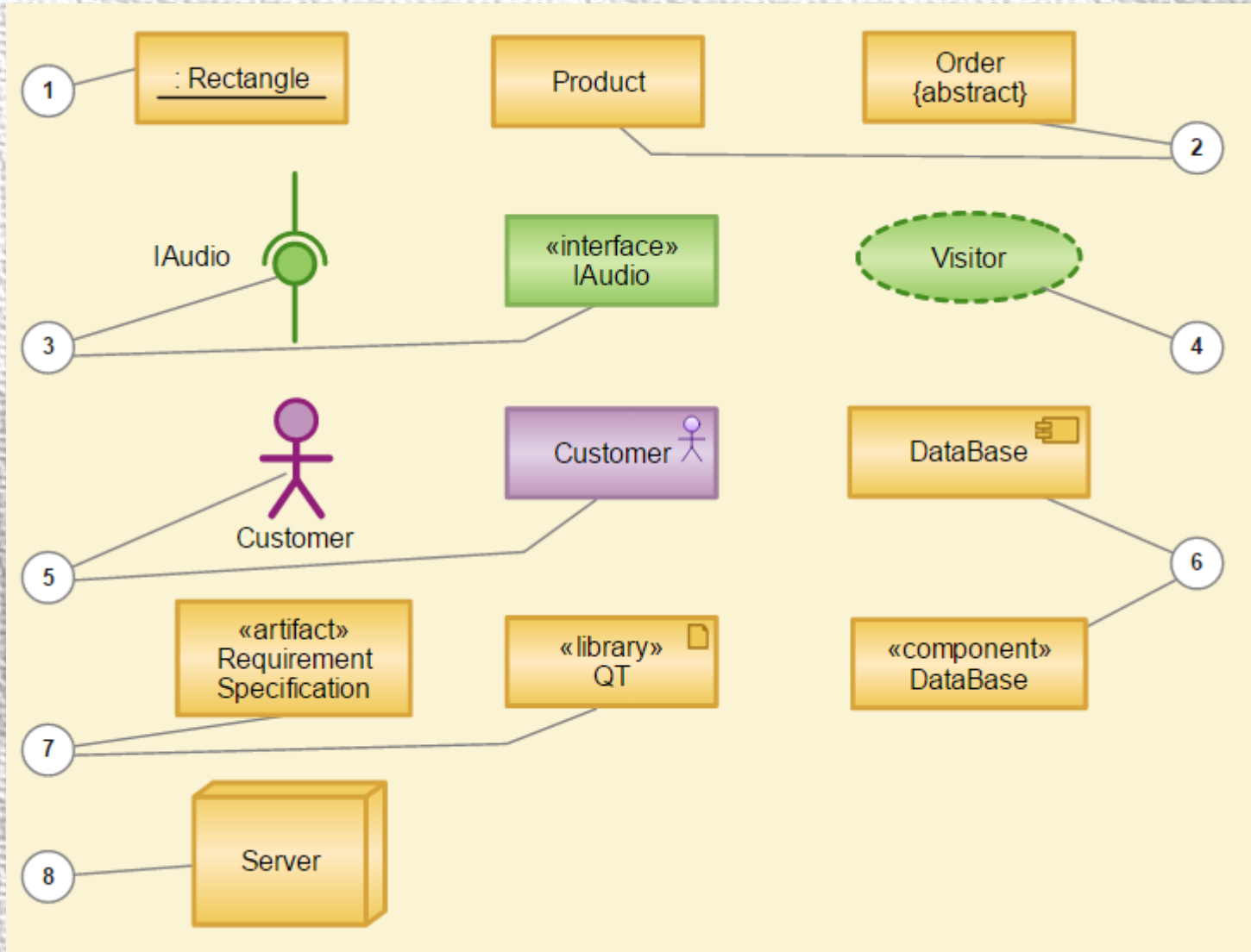
Кооперация

Действующее лицо

Компонент

Артефакт

Узел



Существуют также разновидности этих сущностей:

- актеры,
- сигналы,
- утилиты (виды классов),
- процессы и нити (виды активных классов),
- приложения,
- документы,
- файлы,
- библиотеки,
- страницы и таблицы (виды компонентов).

Дескриптор и классификатор

Дескриптор **(descriptor) – это описание общих свойств множества объектов, включая их структуру, отношения, поведение, ограничения, назначение и т. д.**

Дескриптор имеет две стороны:

- это само описание множества (**intent**)
- и множество значений, описываемых дескриптором (**extent**).

Антонимом для дескриптора является понятие литерала (***literal***). Литерал описывает сам себя. Например, тип данных ***integer*** является дескриптором: он описывает множество целых чисел, потенциально бесконечное (или конечное, но достаточно большое, если речь идет о машинной арифметике). Изображение числа 1 описывает само число "один" и более ничего – это литерал.

Дескриптор и классификатор

Важнейшим типом дескрипторов являются классификаторы.

Классификатор **(classifier) – это дескриптор множества <u>однотипных</u> объектов.**

Из этого определения непосредственно вытекает основное и характеристическое свойство классификатора: **классификатор (прямо или косвенно) может иметь экземпляры**.

ДЕСКРИПТОР И КЛАССИФИКАТОР

- Классификатором называется механизм описания структурных и поведенческих свойств элемента системы.
- У каждой из этих абстракций будут свои экземпляры. Разделение сущностей и экземпляров сущностей – важная часть моделирования.
- Некоторые элементы в UML – например, пакеты или связи обобщения – не имеют экземпляров. Те же элементы моделирования, которые имеют экземпляры, являются классификаторами.
- Но более важный их признак – наличие как структурных свойств (в форме атрибутов), так и поведенческих (в форме операций). Все экземпляры классификатора обладают общими свойствами, но при этом каждый имеет свое собственное значение для каждого атрибута.

СВОЙСТВА КЛАССИФИКАТОРОВ

Все классификаторы имеют некоторые общие свойства, которые используются в дальнейшем изложении. Ниже рассмотрим мы опишем семь наиболее важных свойств классификаторов, которые нам понадобятся прежде всего.

1. Во-первых, **классификаторы** (как и все элементы модели) **имеют имена**. Имя служит для идентификации элемента модели и потому должно быть уникально в данном пространстве имен.
2. Во-вторых, как уже было сказано, **классификатор может иметь экземпляры**. Экземпляры бывают прямые и косвенные.

- .

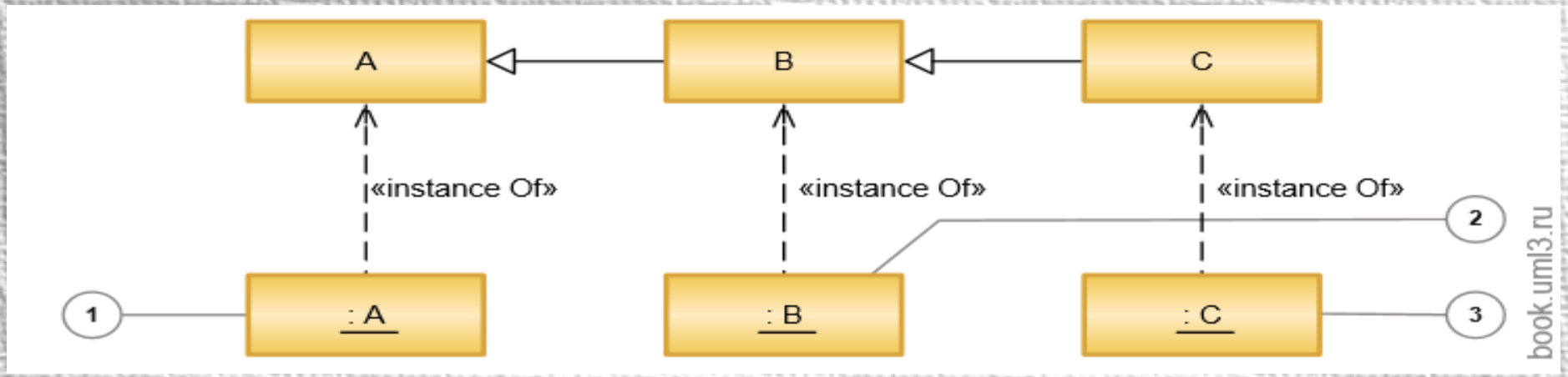
Семь наиболее важных свойств классификаторов

1. **классификаторы** (как и все элементы модели) **имеют имена**
2. **классификатор может иметь экземпляры**
3. **классификатор может быть абстрактным или конкретным**
4. **классификатор** (как и другие элементы модели) **имеет видимость**
5. **все составляющие классификатора имеют область действия**.
6. **классификатор имеет *кратность***
7. **классификаторы** (и только они!) **могут участвовать в отношении *обобщения***

2. Классификатор может иметь экземпляры

Экземпляры бывают прямые и косвенные.

- **Если некоторый объект непосредственно порожден с помощью конструктора классификатора A, то этот объект называется *прямым экземпляром* (direct instance) классификатора A ①.**
- **Если классификатор A является обобщением классификатора B или, что то же самое, классификатор B является специализацией классификатора A, то все экземпляры классификатора B являются *косвенными экземплярами* классификатора A②.**



Прямые и косвенные экземпляры классификатора A

3. Классификатор может быть абстрактным или конкретным

- ***Абстрактный*** **(abstract) классификатор не может иметь прямых экземпляров и в этом случае его имя выделяется курсивом.**
- ***Конкретный*** **(concrete) классификатор может иметь прямые экземпляры и в этом случае его имя записывается прямым шрифтом.**

Абстрактный классификатор – это такой дескриптор множества объектов, в котором нет прямого описания элементов множества, но данный классификатор связан отношением обобщения с другими классификаторами и объединение множеств их экземпляров считается множеством экземпляров данного абстрактного классификатора. Другими словами, множество определяется не прямо, а через совокупность подмножеств. Например, интерфейс, будучи абстрактным классом, не может иметь непосредственных экземпляров, но реализующий его класс может, стало быть, интерфейс является классификатором.

4. **Классификатор** (как и другие элементы модели) **имеет видимость**

Видимость (visibility) определяет, может ли составляющая одного классификатора (в том числе имя) использоваться в другом классификаторе.

Другими словами, если в определенном контексте нечто доступно и может быть как-то использовано, то оно является видимым (в этом контексте). Если же оно не видимо, то и не может быть использовано. Видимость является свойством всех элементов модели (хотя не для всех элементов это свойство является существенным). Видимость может принимать одно из четырех значений:

1. ***открытый*** (обозначается знаком **+** или ключевым словом ***public***);
2. ***защищенный*** (обозначается знаком **#** или ключевым словом ***protected***);
3. ***закрытый*** (обозначается знаком **–** или ключевым словом ***private***).
4. ***пакетный*** (обозначается знаком **~** или ключевым словом ***package***)

5. Все составляющие классификатора имеют область действия

Область действия **(scope) определяет, как проявляет себя составляющая классификатора в экземплярах, т.е. имеют экземпляры свои значения составляющей или совместно используют одно значение.**

Область действия имеет два возможных значения:

- **экземпляр** (instance) — никак специально не обозначается, поскольку подразумевается по умолчанию;
- **классификатор** (classifier) — описание составляющей классификатора подчеркивается.

- Если областью действия составляющей является экземпляр, то каждый экземпляр классификатора имеет свое значение составляющей. Например, областью действия атрибута по умолчанию является экземпляр. Это означает, что каждый объект – экземпляр класса – имеет свое собственное значение атрибута, которое может меняться независимо от значений данного атрибута других объектов, экземпляров этого же класса.
- Если областью действия составляющей является классификатор, то все экземпляры классификатора совместно используют одно значение составляющей. Например, конструктор обычно имеет областью действия классификатор (класс), поскольку является процедурой, общей для всех экземпляров данного класса.

6. Классификатор имеет кратность

Кратность **(multiplicity) множества – это множество чисел, которые задают все допустимые значения мощности для данного множества.**

Классификатор имеет кратность, т.е. ограничение на количество экземпляров классификатора, как множества. Не следует путать кратность с количеством элементов (экземпляров). Множество, указанное в модели, во время выполнения может иметь различное количество элементов, и количество элементов может динамически меняться. Кратность определяет пределы этих изменений.

Синтаксически кратность задается выражением, которое является непустой последовательностью элементов (разделенных запятыми), каждый из которых имеет следующий формат.

Нижняя граница .. Верхняя граница

В качестве верхний и нижней границы используются натуральные числа или ноль. Кроме того, в качестве верхней границы может использоваться символ *****. Если нижняя граница не задана, то она опускается вместе с символом **..** (две точки).

В следующей таблице приведены некоторые примеры выражений кратности.

Выражения кратности

Выражение кратности	Множество может иметь
0..* или *	Произвольное число элементов
1..*	Один или более элементов
0..1	Не более одного элемента
1..10	От одного до десяти элементов
1..3, 5, 7..10	Один, два, три, пять, семь, восемь, девять или десять элементов
3..5	Некорректная кратность. Нижняя граница больше верхней
-1..3	Некорректная кратность. Отрицательные числа недопустимы

Обычно на практике используются следующие варианты кратности классификаторов

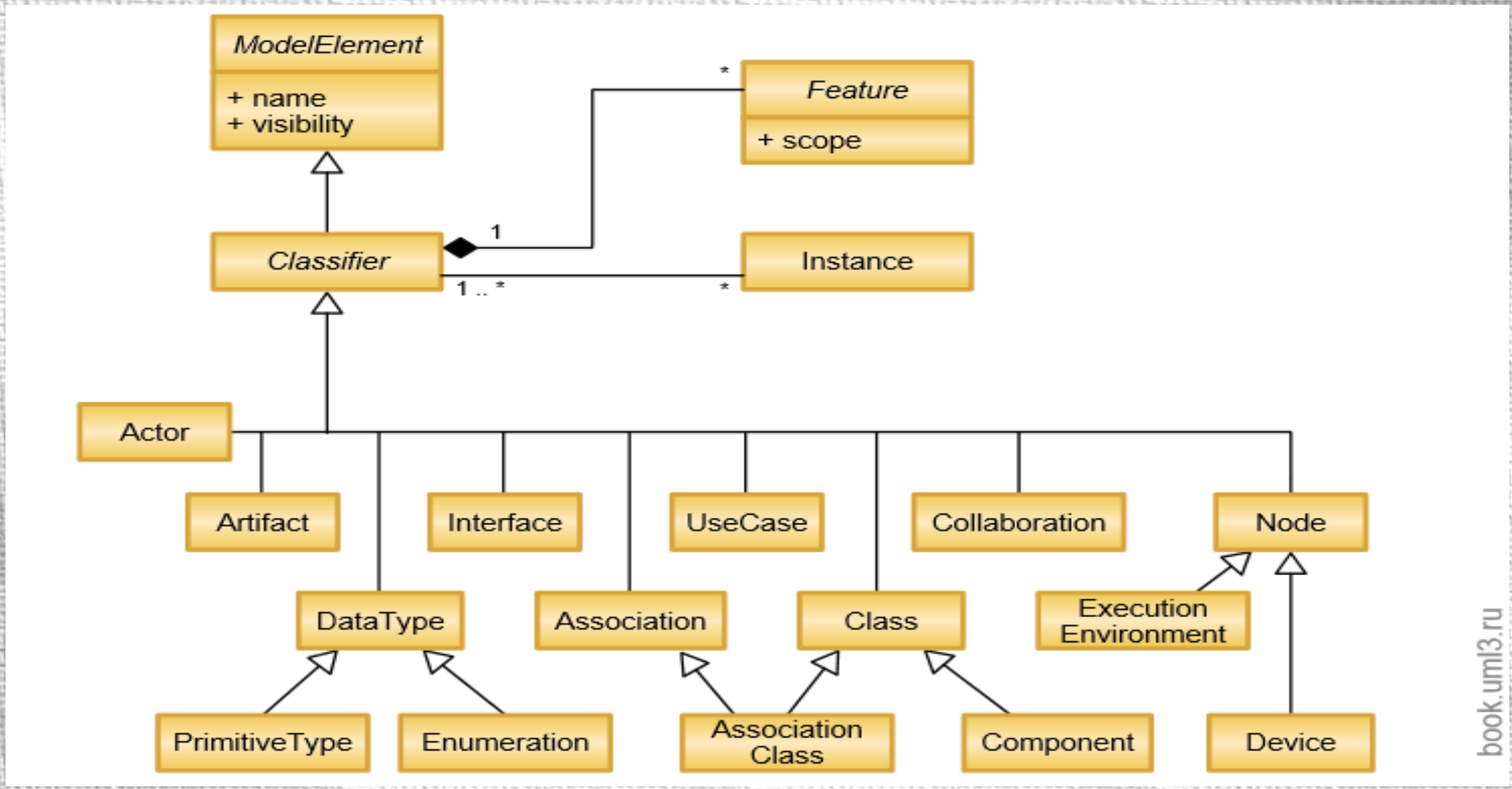
1. Классификатор не имеет экземпляров (кратность 0) – такой классификатор называется **службой** (utility). Все составляющие службы имеют областью действия классификатор. Хранение информации, обрабатываемой службой, обеспечивают объекты, использующие службу. Типичный пример – набор процедур общего пользования, скажем, библиотека математических функций. Службы используются в приложениях достаточно часто, поэтому есть даже стандартный стереотип «utility», определяющий классификатор как службу.

2. **Классификатор имеет ровно один экземпляр** (кратность 1). Такой классификатор называется **одиночкой** (singleton). В сущности, между службой и одиночкой различия незначительны, но иногда одиночку использовать удобнее, например, когда классификатор представляет собой элемент реального мира, существующий в единственном экземпляре. В качестве примера можно привести клавиатуру (существует один экземпляр), которая подключается к компьютеру.
3. **Классификатор имеет фиксированное число экземпляров** (например, кратность 8). Такой вариант не часто, но встречается. Например, моделирование портов в концентраторе.
4. **Классификатор имеет произвольное число экземпляров** (кратность *). Поскольку этот вариант встречается чаще всего, он никак специально не указывается и подразумевается по умолчанию.

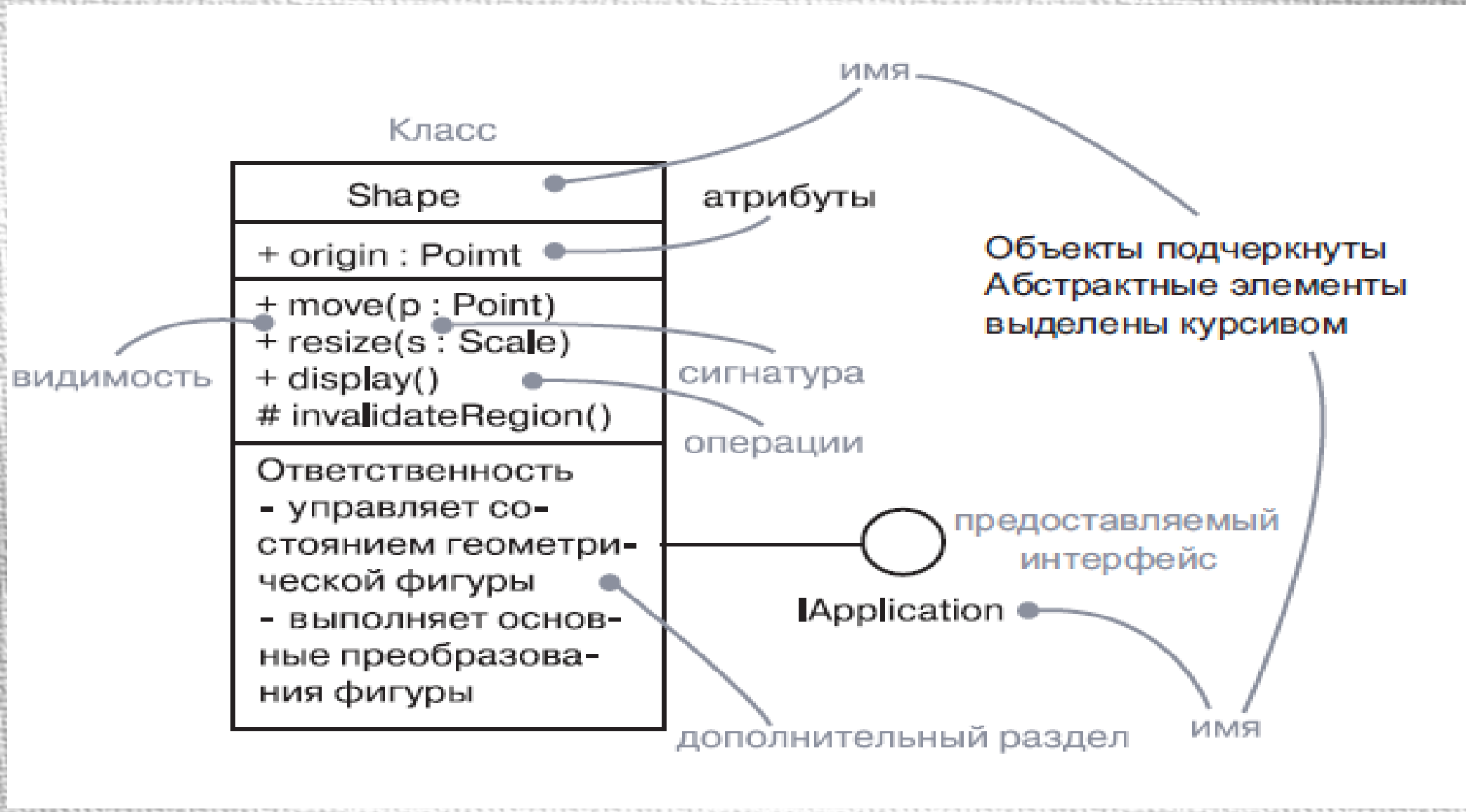
7. Классификаторы (и только они!) могут участвовать в отношении обобщения

Рассмотрим ниже

Часть метамодели классификатора



Структурная сущность КЛАСС

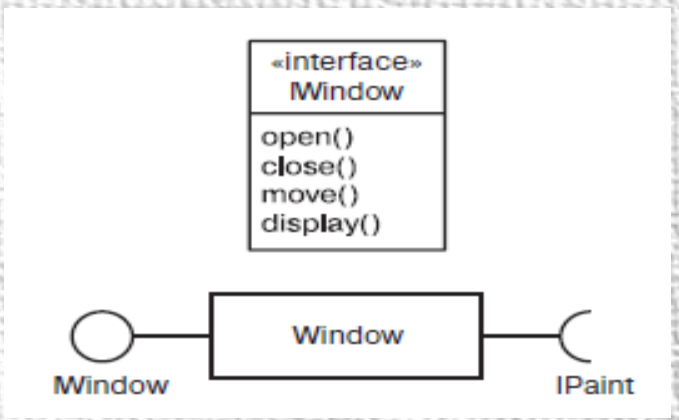


Структурные сущности
КООПЕРАЦИЯ, ПРЕЦЕНДЕНТ, АКТИВНЫЙ КЛАСС, КОМПОНЕНТ,АРТЕФАКТ, УЗЕЛ

Интерфейс

Интерфейс *(interface) – это набор операций, который специфи*цирует сервис (набор услуг) класса или компонента. Таким образом, интерфейс описывает видимое извне поведение элемента. Может представлять полное поведение класса или компонента либо только часть такого поведения. Определяет набор спецификаций операций (то есть их сигнатуру), но никогда не определяет детали их реализации. Объявление интерфейса изображается как класс с ключевым словом «interface» над его именем; атрибуты несущественны, за исключением иногда показываемых констант. Интерфейс, однако, редко существует сам по себе. Интерфейс, представляемый классом для внешнего мира, изображается в виде маленького круга, соединенного линией с рамкой класса.

Интерфейс, запрашиваемый классом от некоторого другого класса, представлен маленьким полукругом, соединенным с рамкой класса линией, как показано на рисунке.



Компонент

С понятием "компонент" часто ассоциируют компонентное или сборочного программирование, идея которого также стара, как и само программирование.

- Действительно, затраты на тиражирование программ любой величины и сложности исчезающе малы по сравнению с затратами на их разработку. . Отсюда возникает естественное желание не создавать каждую новую программу "с нуля", а использовать ранее созданные программы в качестве готовых строительных элементов, благо затраты на копирование столь не велики.

Понятие компонента в UML не соответствует тому понятию, которое имеет в виду сборочное программирование. Компонент UML является частью модели, и описывает логическую сущность, которая существует только во время проектирования (design time), хотя в дальнейшем ее можно связать с физической реализацией (артефактом) времени исполнения (run time).

Компонент (component) – это модульный фрагмент логического представления системы, взаимодействие с которым описывается набором обеспеченных и требуемых интерфейсов.

Компонент *– это модульная часть системы, которая скрывает* свою реализацию за набором внешних интерфейсов. Компоненты системы, разделяющие общие интерфейсы, могут замещать друг друга, сохраняя при этом одинаковое логическое поведение.

Стандартные стереотипы компонентов

Стандартом UML для компонентов предусмотрены стереотипы, приведенные в следующей таблице.

Стереотип	Описание
«buildComponent»	Компонент, служащий для разработки приложения
«entity»	Постоянно хранимый информационный компонент, представляющий некоторое понятие предметной области
«service»	Функциональный компонент без состояния, возвращающий запрашиваемые значения без побочных эффектов
«subsystem»	Единица иерархической декомпозиции большой системы

АРТЕФАКТЫ И УЗЛЫ

Артефакты и узлы – также различаются. Они представляют собой **физические** сущности, в отличие от предыдущих пяти, относящихся к сущностям **логическим** или **концептуальным**.

АРТЕФАКТЫ

К элементам программной системы, а, следовательно, и к артефактам, могут относиться исполняемые файлы, исходные тексты программ, веб-страницы, справочные файлы, сопроводительные документы, файлы с данными, модели и многое другое, являющееся физическими элементами информации. Другими словами, артефактами являются те информационные элементы, которые тем или иным способом используются при работе программной системы и входят в ее состав.

Артефакт *(artifact)* – это физическая и замещаемая часть системы, несущая физическую информацию («биты»). В системе вы можете встретить разные виды артефактов, таких как файлы исходного кода, исполняемые программы и скрипты. Обычно артефакт представляет собой физический пакет с исходным или исполняемым кодом. Изображается как прямоугольник, снабженный ключевым словом «artifact», расположенным над его именем.

2 ПОВЕДЕНЧЕСКИЕ СУЩНОСТИ

ПОВЕДЕНЧЕСКИЕ СУЩНОСТИ
(BEHAVIORAL THINGS)

Поведенческие сущности являются динамическими составляющими модели UML. Это ***<u>глаголы языка</u>***: они описывают поведение модели во времени и пространстве.

Существует всего три основных типа поведенческих сущностей.

1. **Действие**
2. **Состояние**
3. **Деятельность**

Семантически они часто бывают связаны с различными структурными элементами, в первую очередь - классами, кооперациями и объектами.

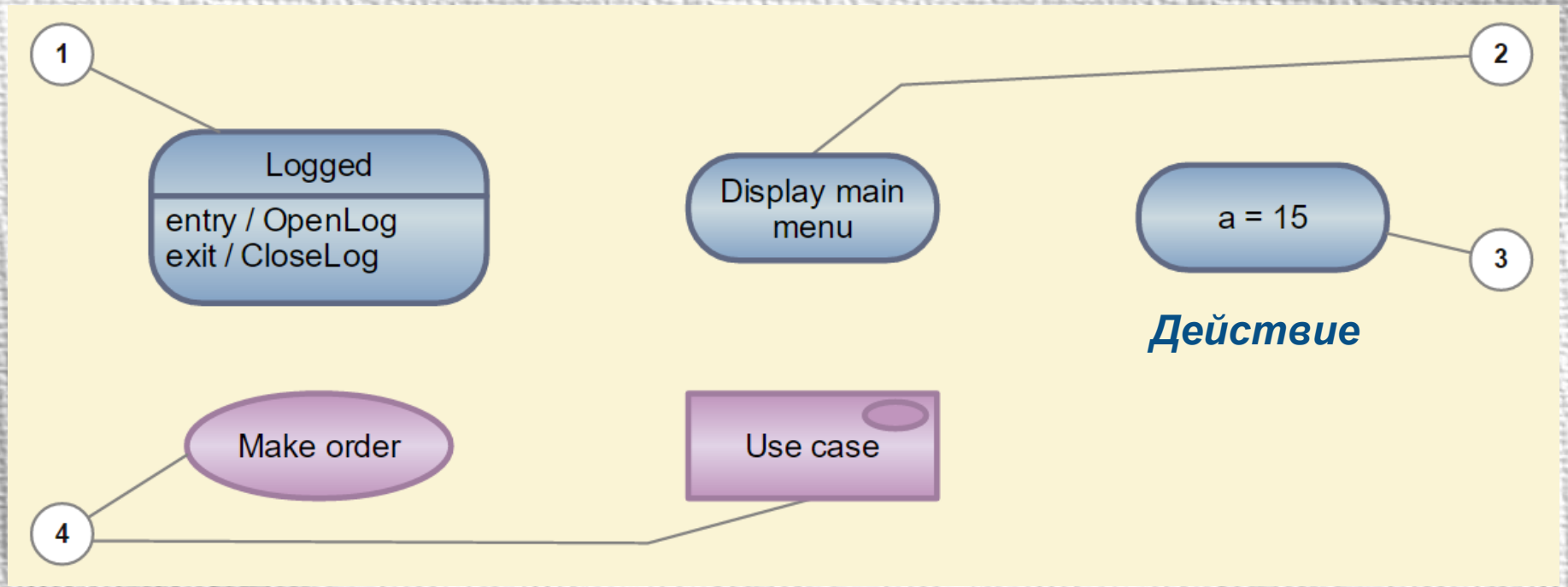
1. **Состояние (state)** – период в жизненном цикле объекта, находясь в котором объект удовлетворяет некоторому условию и осуществляет собственную деятельность или ожидает наступления некоторого события.
2. **Деятельность (activity)** можно считать частным случаем состояния, который характеризуется продолжительными (по времени) не атомарными вычислениями.
3. **Действие (action)** – примитивное атомарное вычисление.

Несколько особняком стоит сущность – вариант использования.

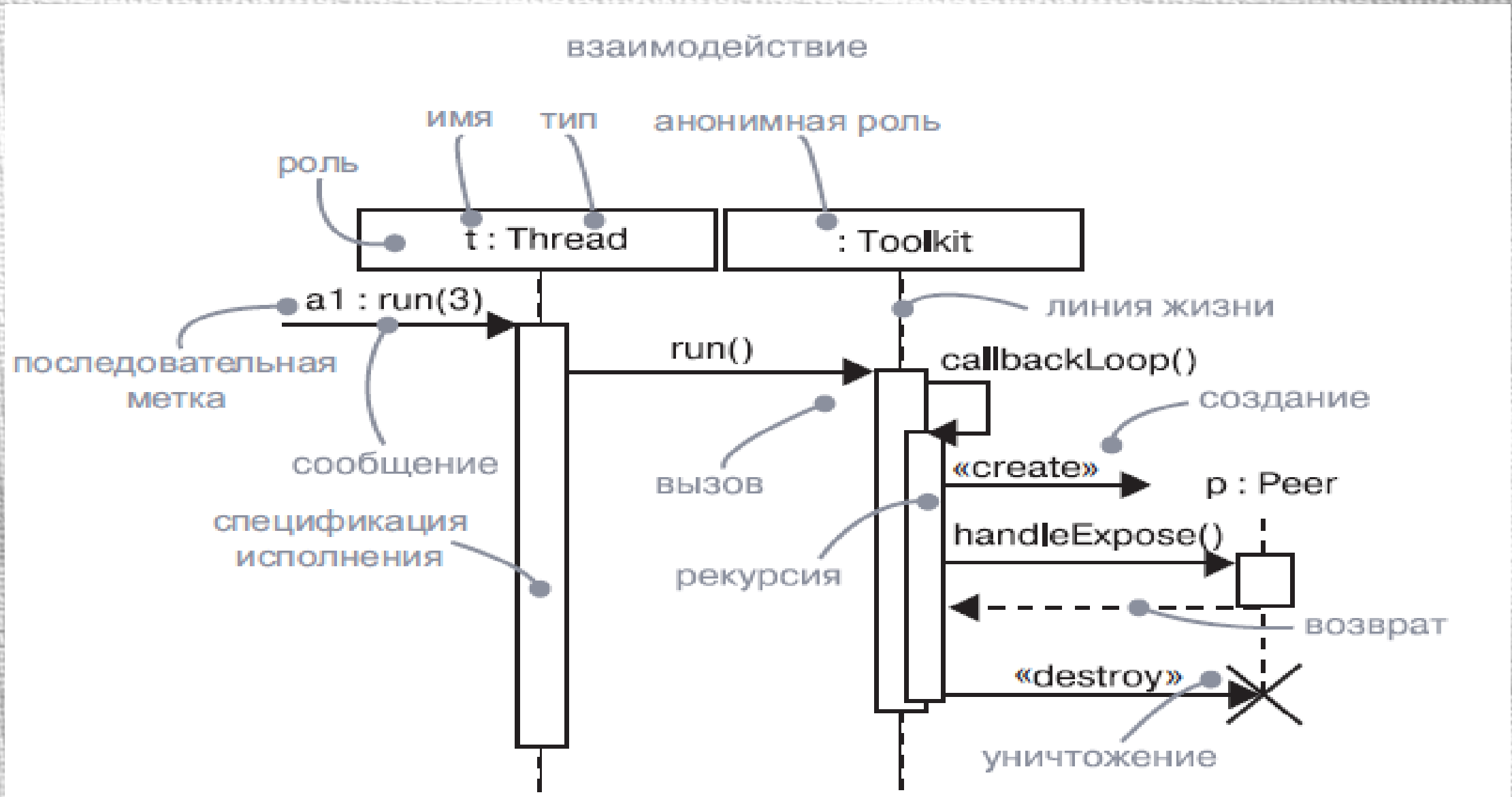
4. ***Вариант использования*** **(use case) – множество сценариев, объединенных по некоторому критерию и описывающих последовательности производимых системой действий, доставляющих значимый для некоторого действующего лица результат.**

НОТАЦИЯ ПОВЕДЕНЧЕСКИХ СУЩНОСТЕЙ

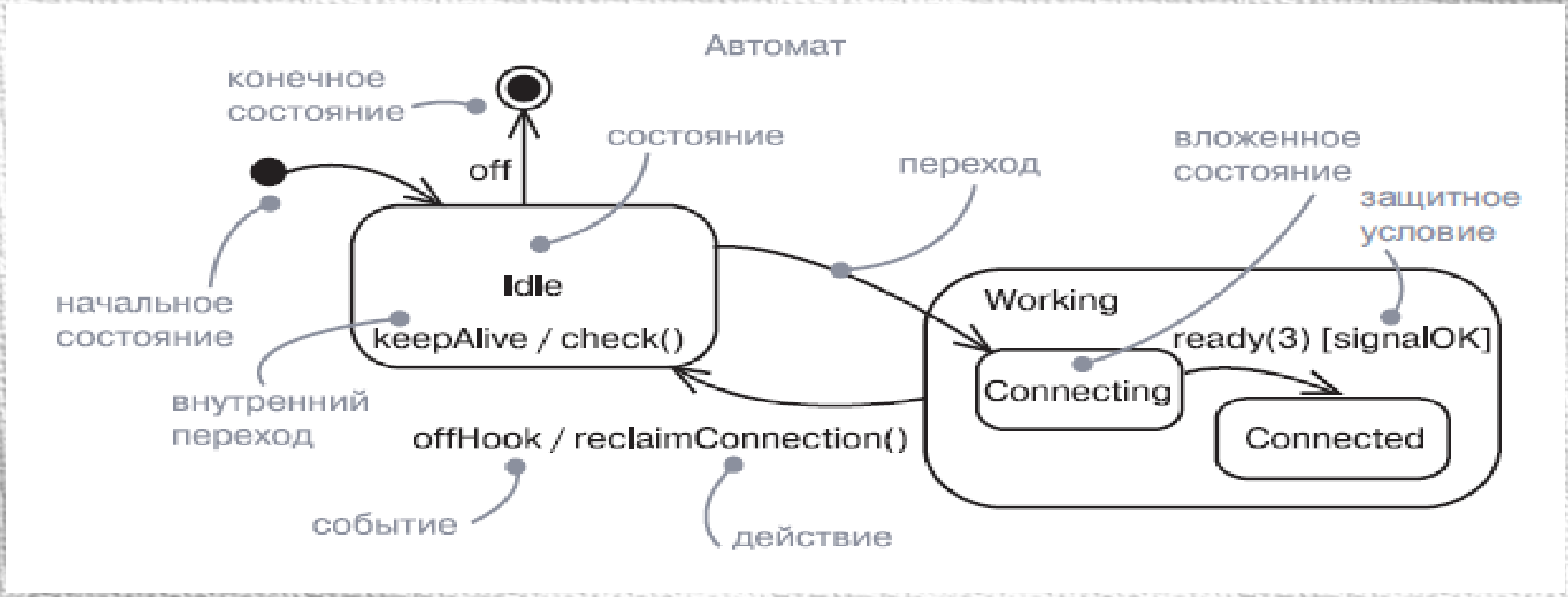
Поведенческие сущности являются динамическими составляющими модели UML.



Взаимодействие



Автомат


Автомат
конечное состояние
off
состояние
переход
вложенное состояние
защитное условие
начальное состояние
Idle
keepAlive / check()
Working
ready(3) [signalOK]
Connecting
Connected
внутренний переход
offHook / reclaimConnection()
событие
действие

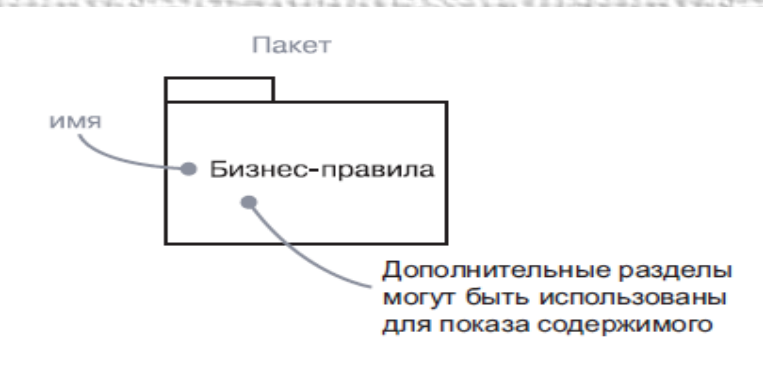
3

ГРУППИРУЮЩИЕ СУЩНОСТИ

Группирующие сущности

Группирующие сущности являются организующими частями модели UML. Это блоки, на которые можно разложить модель. Есть только одна первичная группирующая сущность, а именно пакет.

Пакеты (Packages) представляют собой универсальный механизм организации элементов в группы. В пакет можно поместить структурные, поведенческие и даже другие группирующие сущности. В отличие от компонентов, существующих во время работы программы, пакеты носят чисто концептуальный характер, то есть существуют только во время разработки.



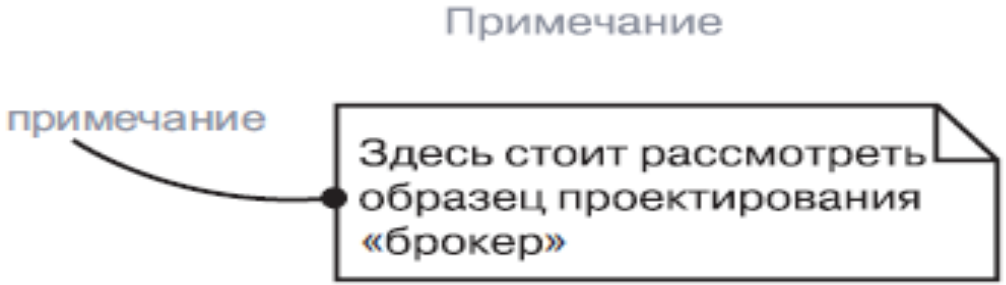
4

АННОТОЦИОННЫЕ СУЩНОСТИ

Аннотационные сущности

Аннотационные сущности - пояснительные части модели UML. Это комментарии для дополнительного описания, разъяснения или замечания к любому элементу модели. **Имеется только один базовый тип аннотационных элементов - примечание (Note).**

Примечание - это просто символ для изображения комментариев или ограничений, присоединенных к элементу или группе элементов. Графически примечание изображается в виде прямоугольника с загнутым краем, содержащим текстовый или графический комментарий.



ОТНОШЕНИЯ

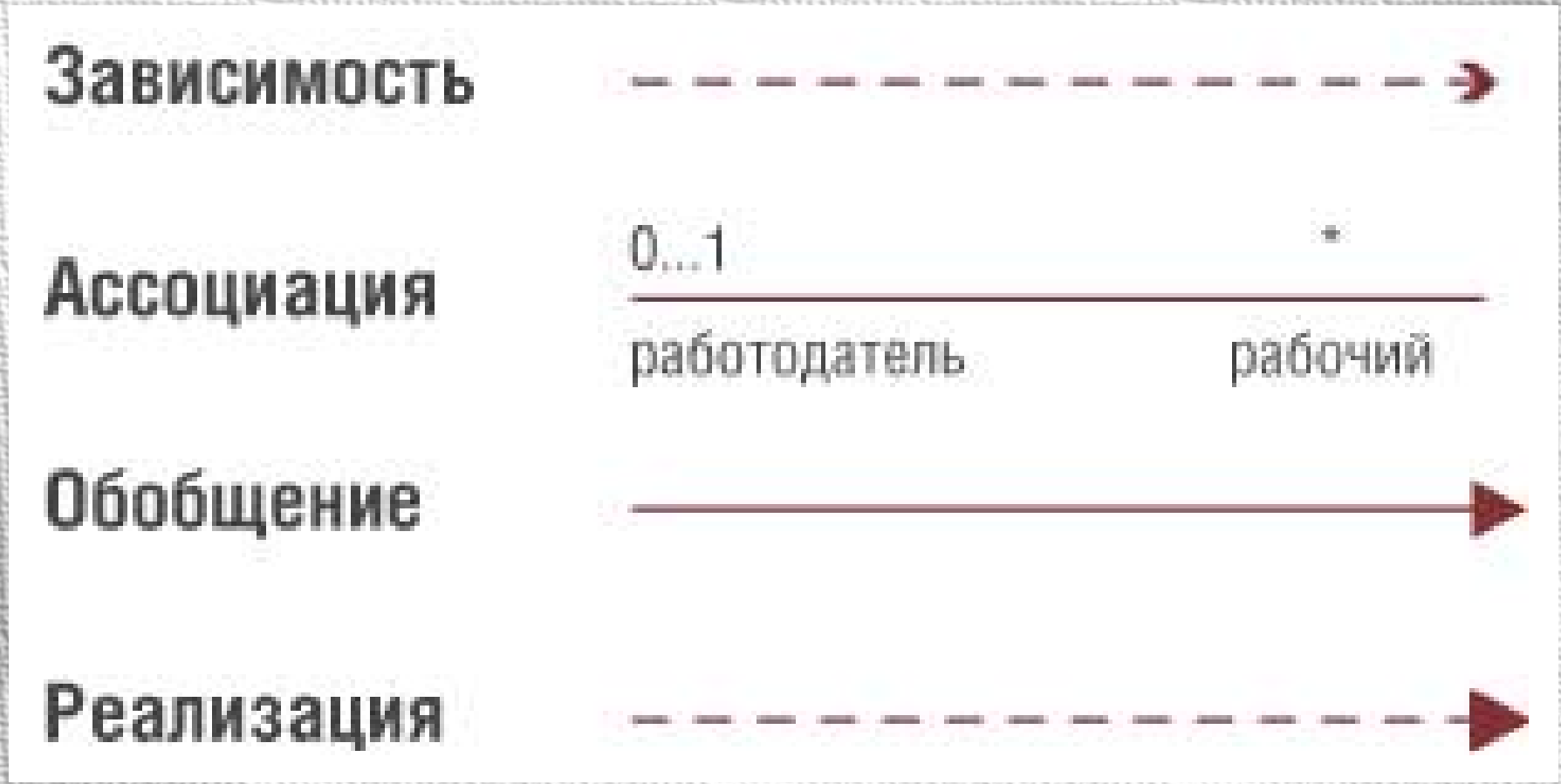
Типы отношений

В языке UML определены четыре типа *отношений*:

- **зависимость;**
- **ассоциация;**
- **обобщение;**
- **реализация.**

Эти отношения являются основными связующими строительными блоками в UML и применяются для создания корректных моделей.

Отношения UML

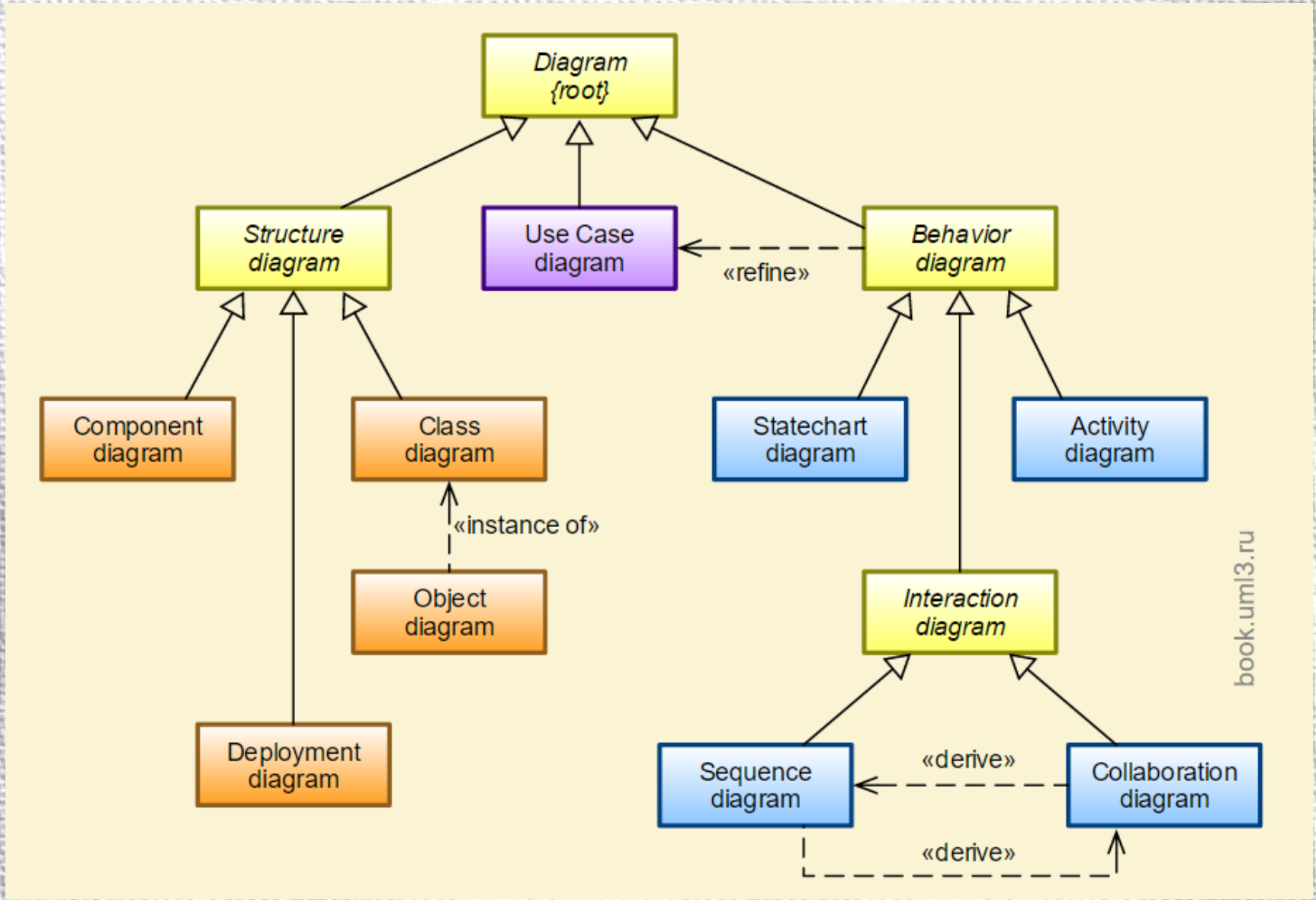


Диаграммы UML 1

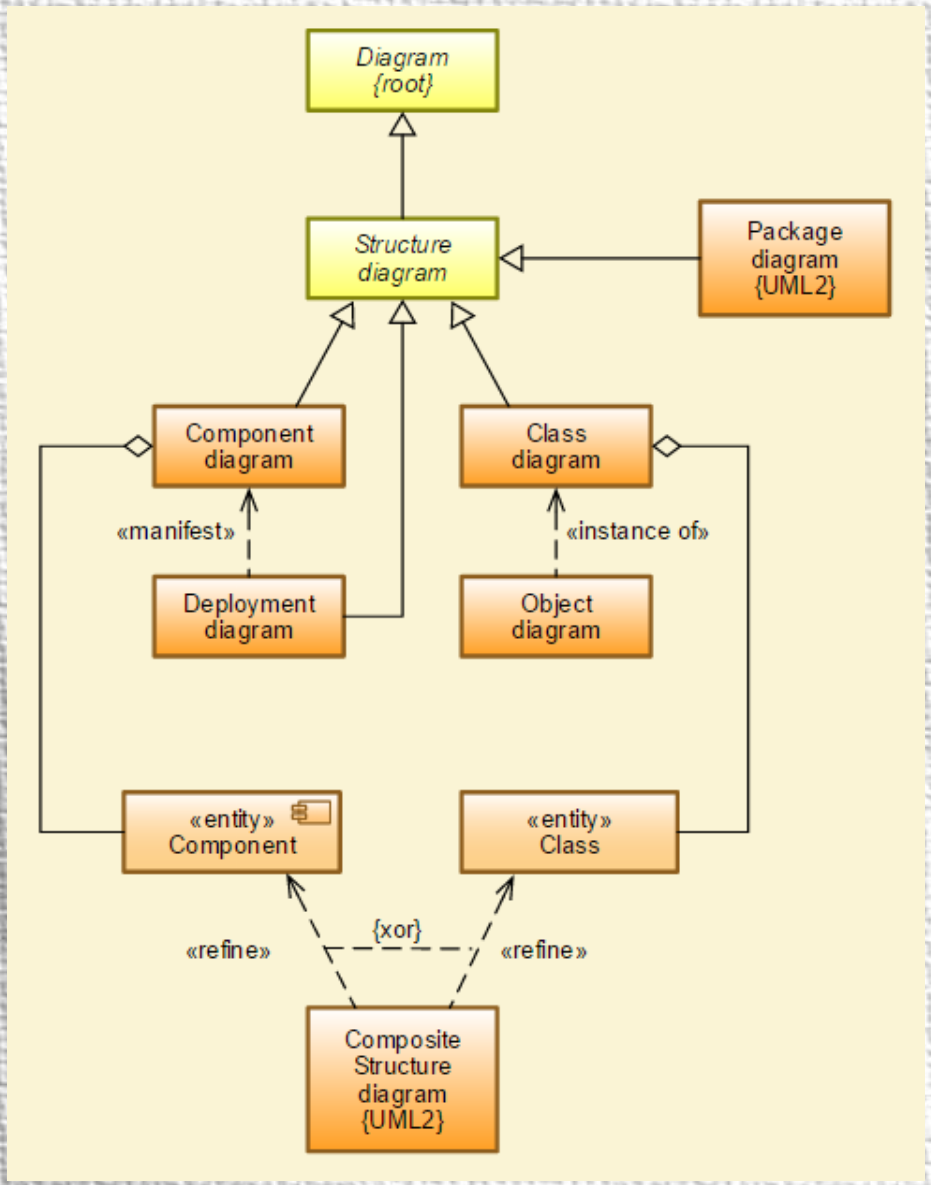
- Диаграмма вариантов использования (use case diagram)
- Диаграмма классов (class diagram)
- Диаграммы поведения (behavior diagrams)
  - Диаграмма состояний (statechart diagram)
  - Диаграмма деятельности (activity diagram)
  - Диаграммы взаимодействия (interaction diagrams)
    - Диаграмма последовательности (sequence diagram)
    - Диаграмма кооперации (collaboration diagram)
- Диаграммы реализации (implementation diagrams)
  - Диаграмма компонентов (component diagram)
  - Диаграмма развертывания (deployment diagram)

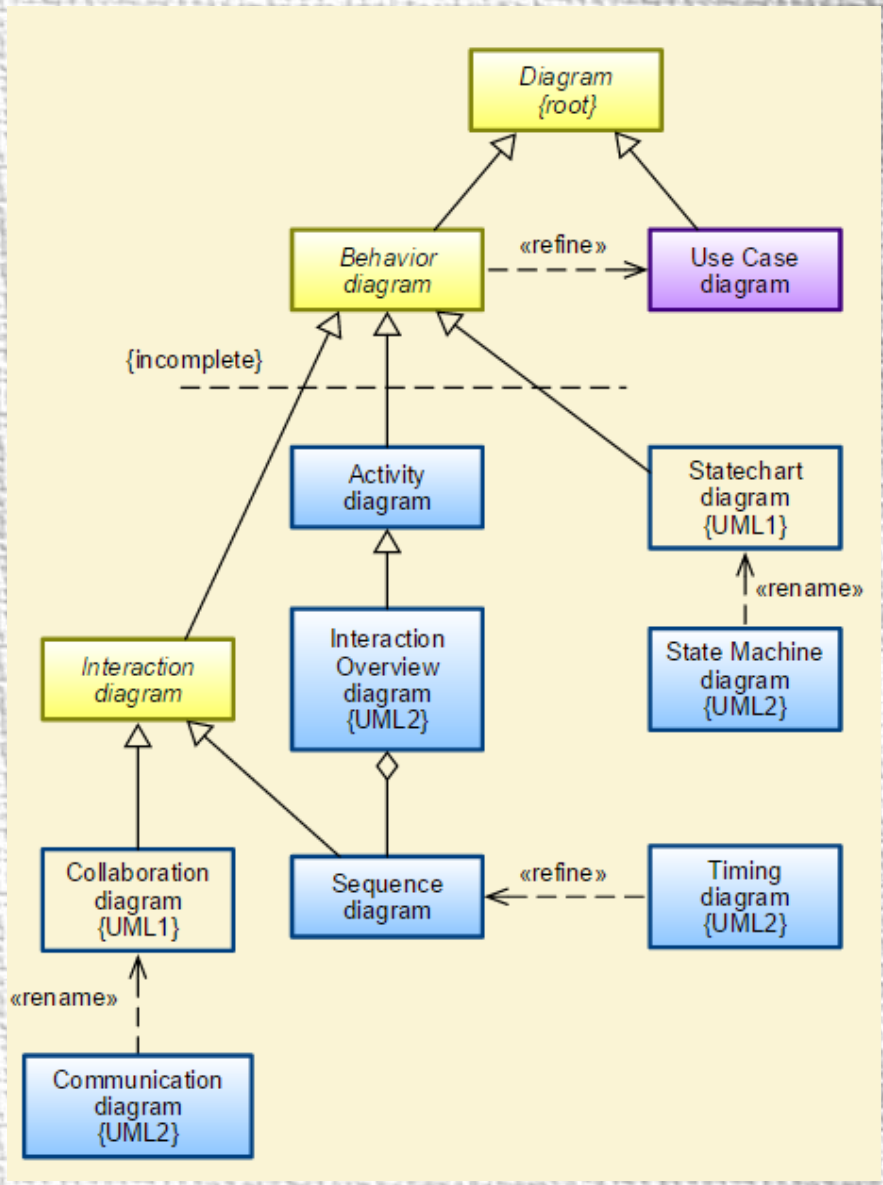
Иерархия типов диаграмм UML 1

ИЕРАРХИЯ ТИПОВ ДИАГРАММ ДЛЯ UML 1



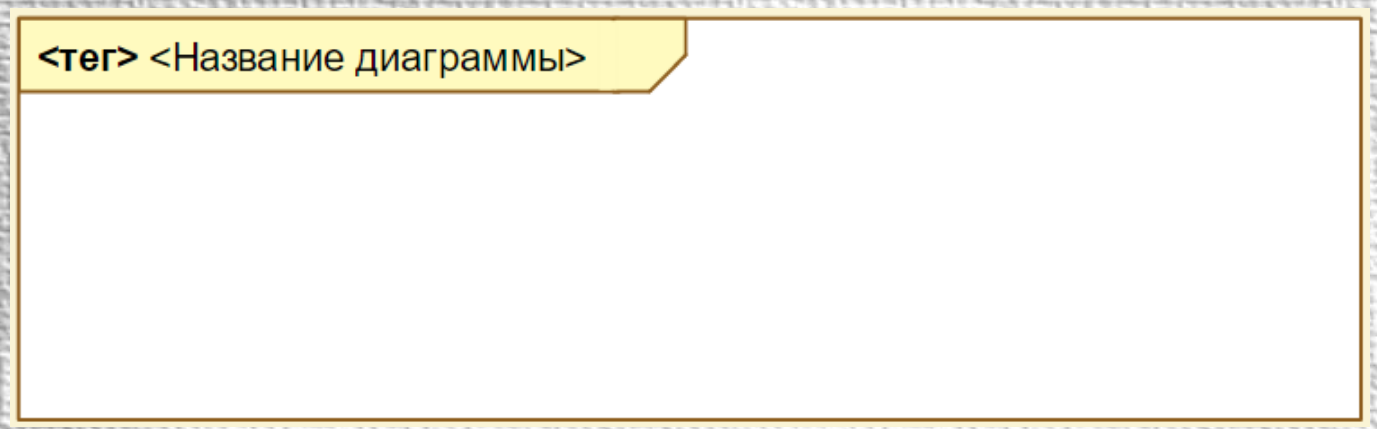
ИЕРАРХИЯ ТИПОВ ДИАГРАММ ДЛЯ UML 2





КАК СТАНДАРТ ТРЕБУЕТ ОФОРМЛЯТЬ ДИАГРАММЫ

Общий шаблон представления диаграммы приведен ниже. Основных элементов оформления два: наружная рамка и ярлычок с названием диаграммы. Название диаграммы записывается в специальном формате, приведенном на рисунке Нотация для диаграмм.



Нотация для диаграмм

ТИПЫ И ТЕГИ ДИАГРАММ

	Название диаграммы	Тег (стандартный)	Тег (предлагаемый)
1	Диаграмма использования	**use case** или **uc**	**use case**
2	Диаграмма классов	**class**	**class**
3	Диаграмма автомата	**state machine** или **stm**	**state machine**
4	Диаграмма деятельности	**activity** или **act**	**activity**
5	Диаграмма последовательности	**interaction** или **sd**	**sd**
6	Диаграмма коммуникации	**interaction** или **sd**	**comm**
7	Диаграмма компонентов	**component** или **cmp**	**component**
8	Диаграмма размещения	не определен	**deployment**
9	Диаграмма объектов	не определен	**object**
10	Диаграмма внутренней структуры	**class**	**class** или **component**
11	Обзорная диаграмма взаимодействия	**interaction** или **sd**	**interaction**
12	Диаграмма синхронизации	**interaction** или **sd**	**timing**
13	Диаграмма пакетов	**package** или **pkg**	**package**

ИНТЕГРИРОВАННАЯ МОДЕЛЬ СЛОЖНОЙ СИСТЕМЫ В НОТАЦИИ UML

Диаграмма использования

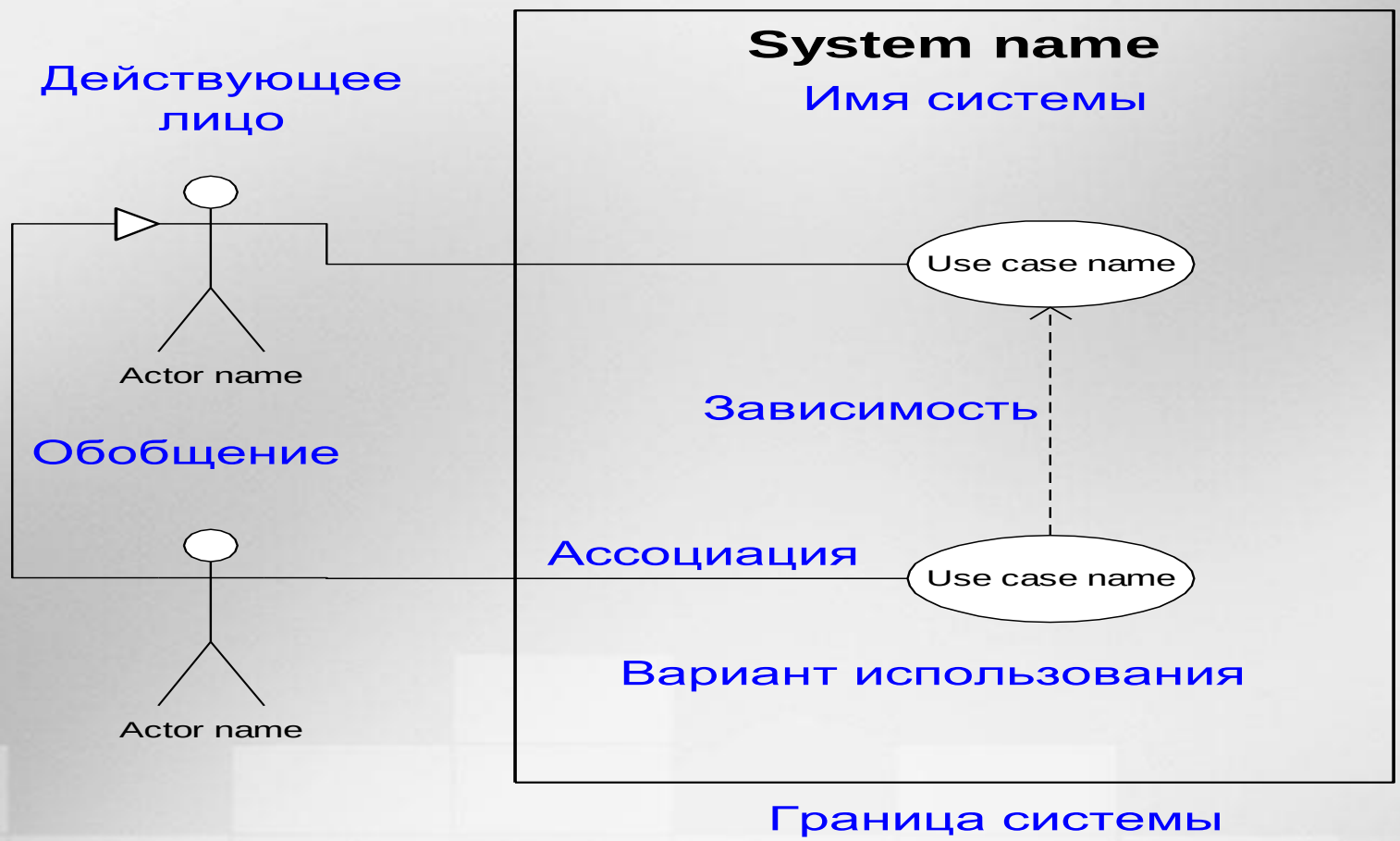


Диаграмма классов

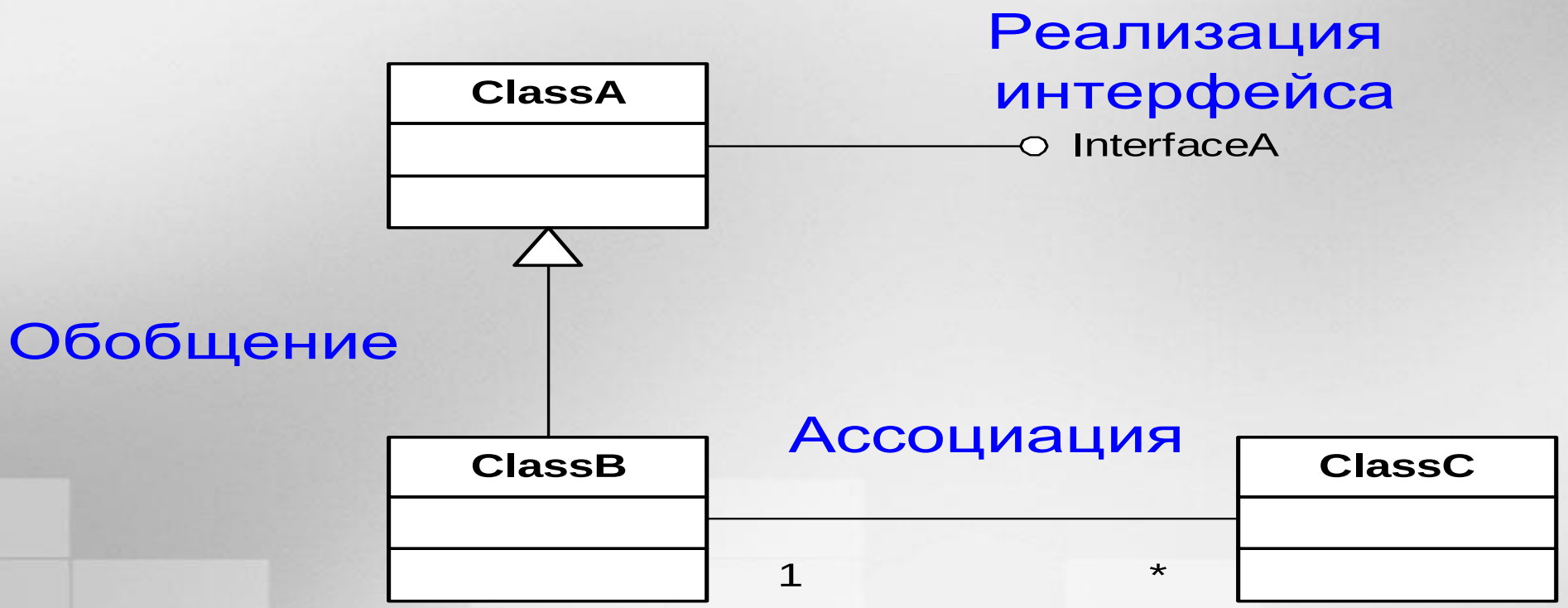


Диаграмма объектов

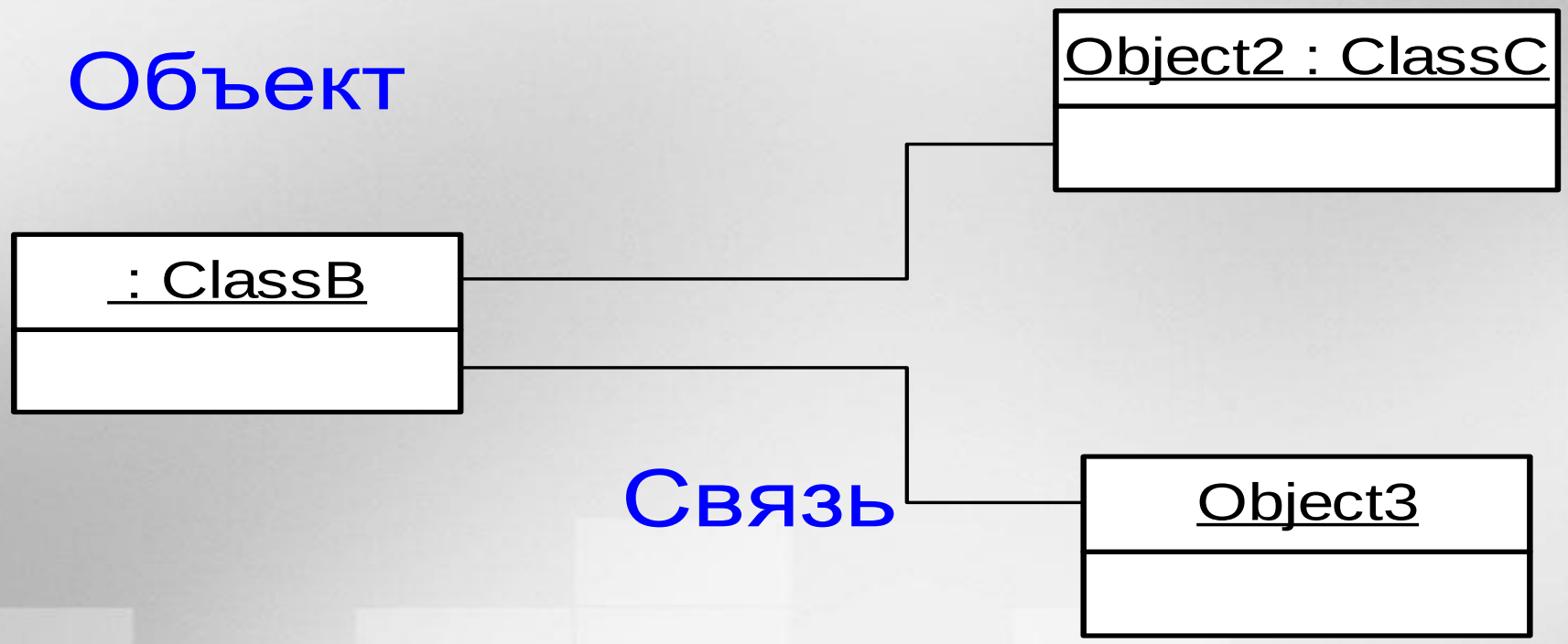


Диаграмма состояний

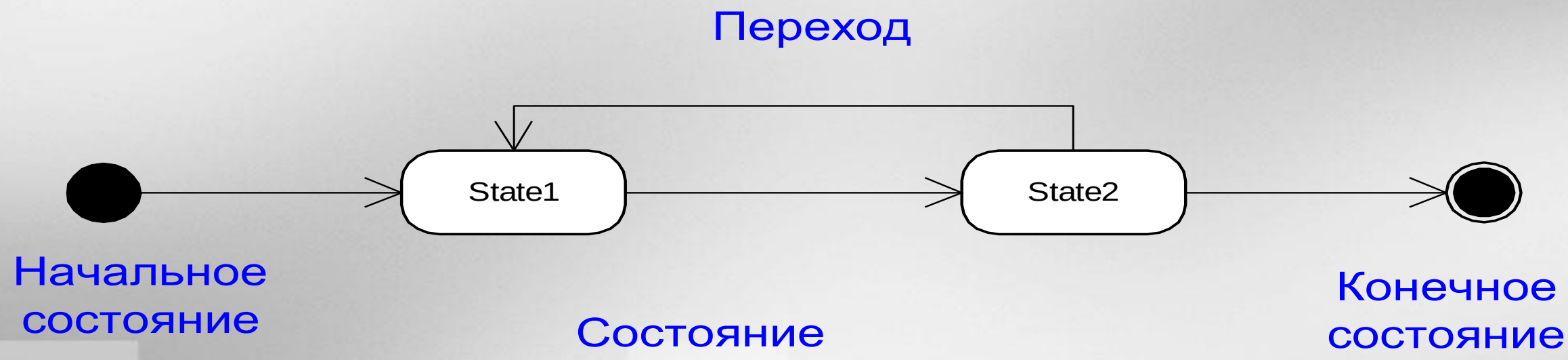


Диаграмма деятельности

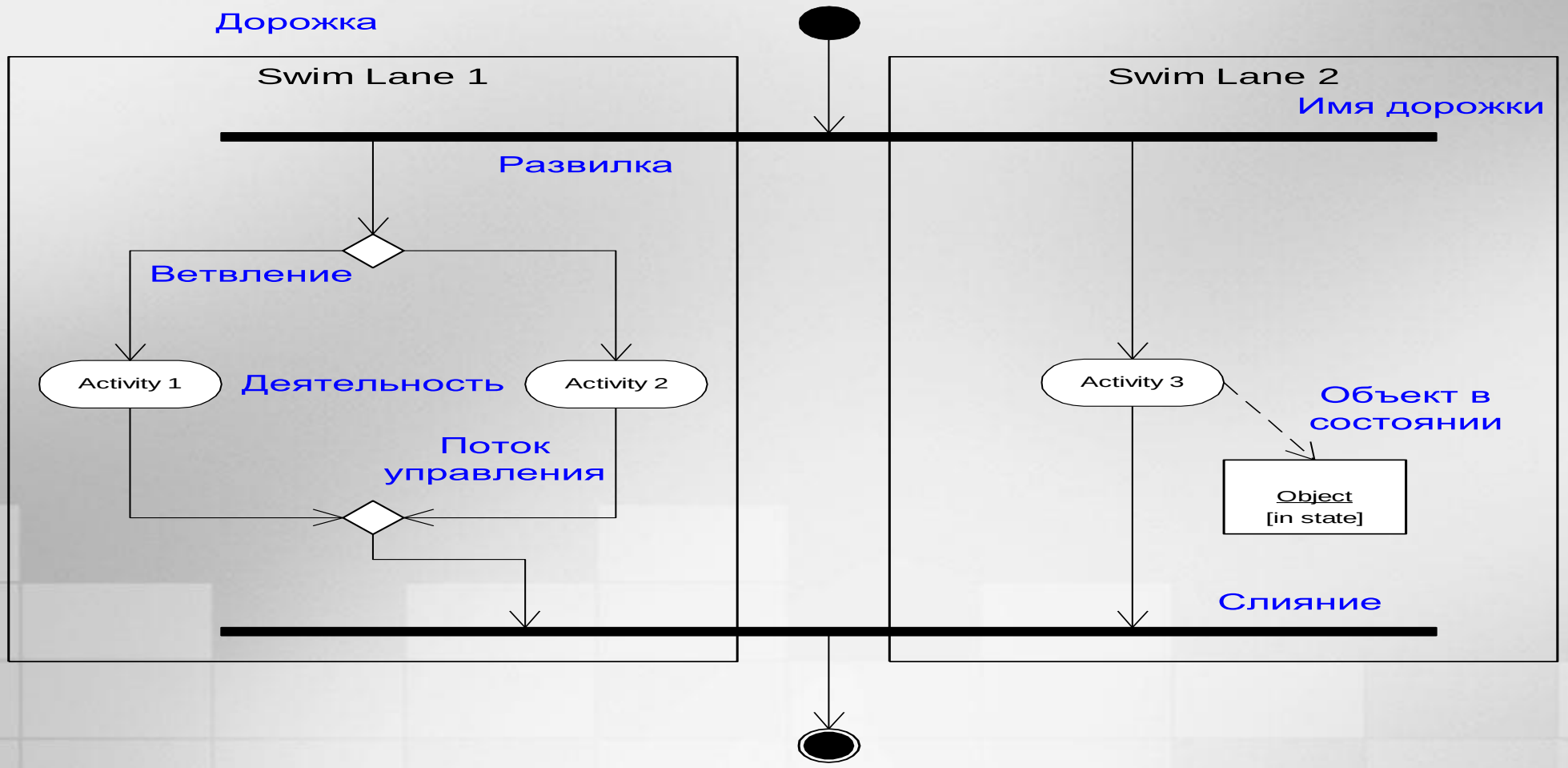


Диаграмма последовательности

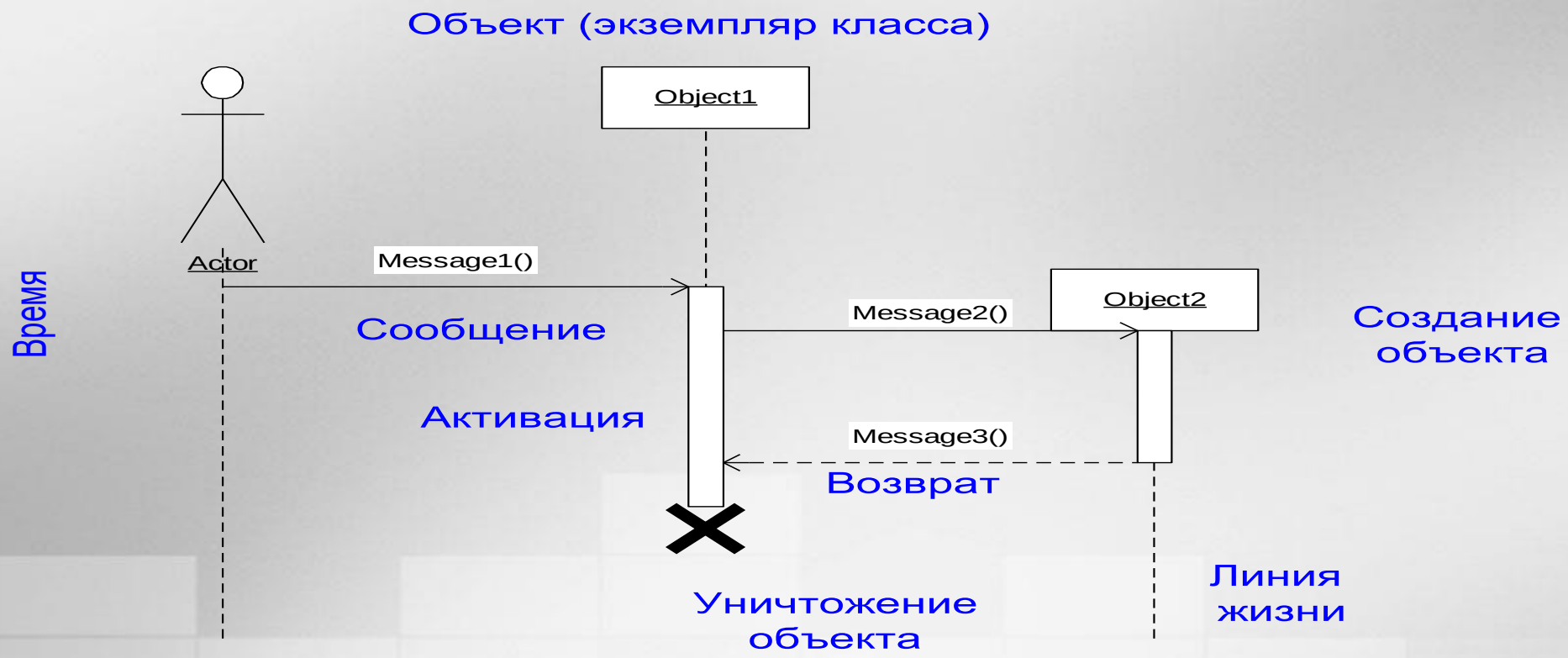


Диаграмма кооперации

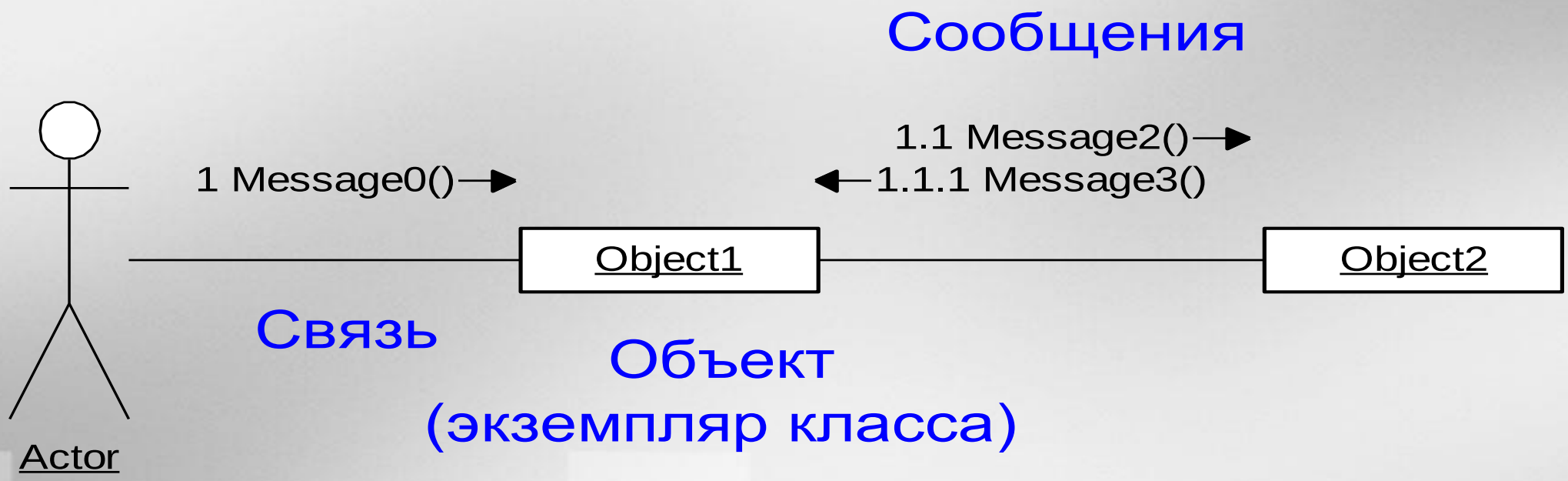
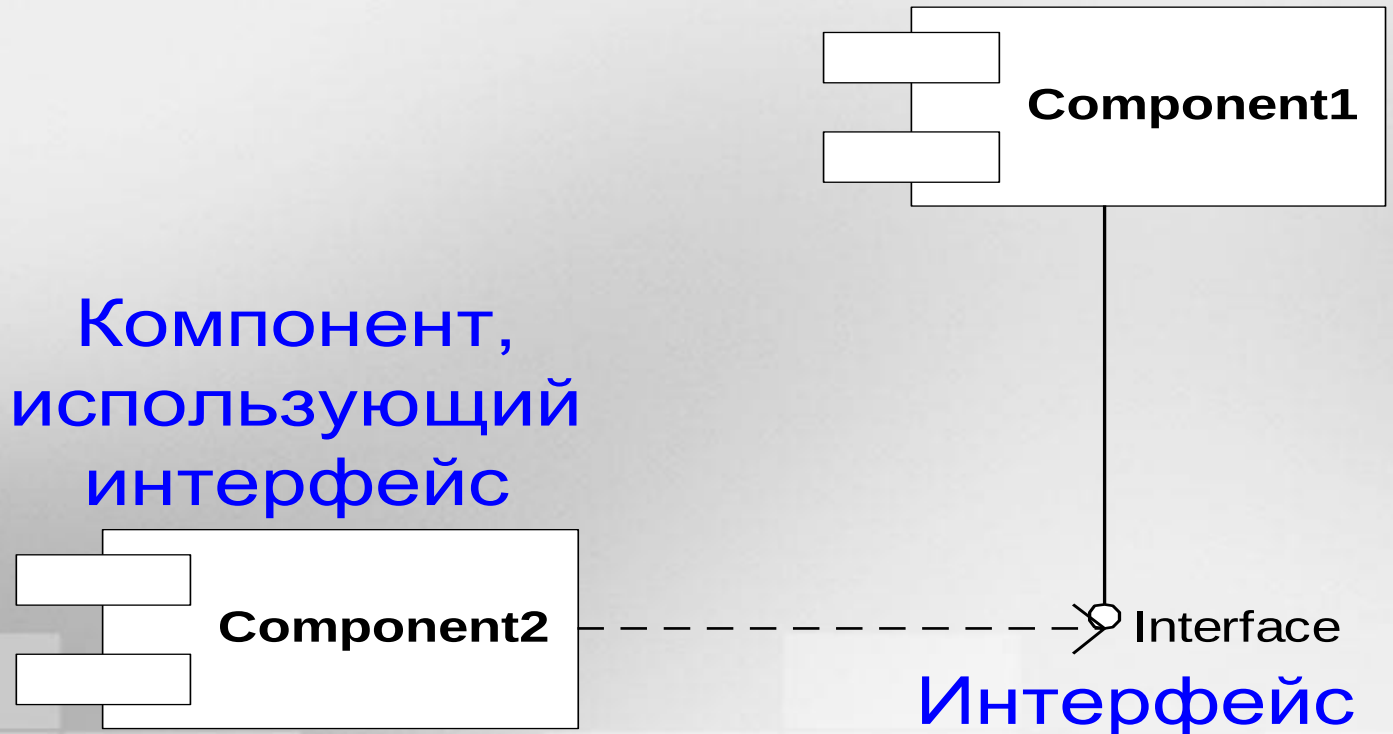


Диаграмма компонентов

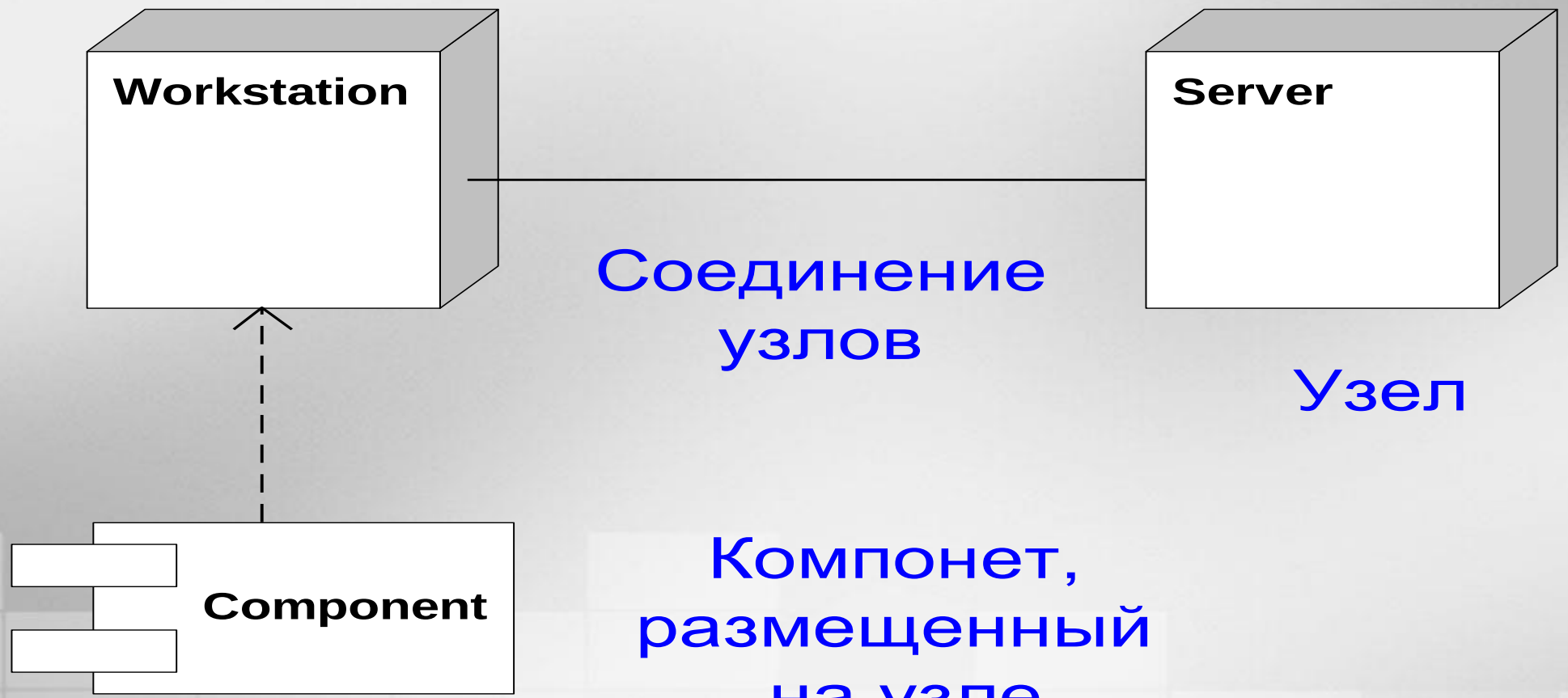


Компонент, реализующий интерфейс

Компонент, использующий интерфейс

Интерфейс

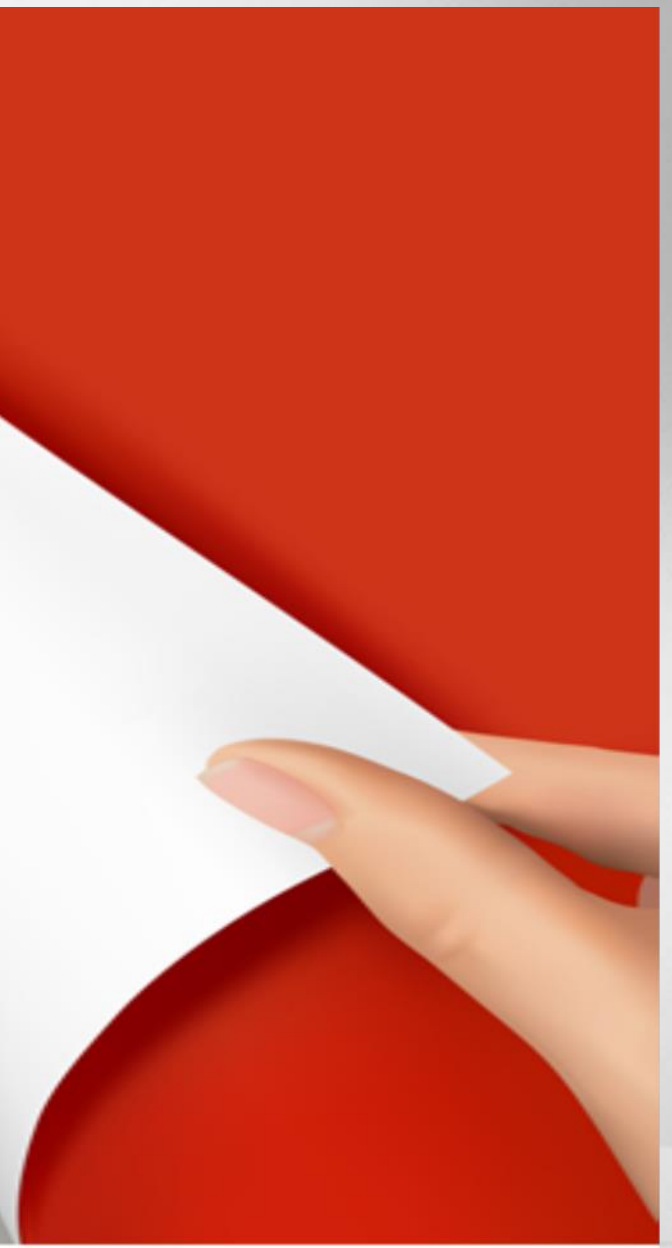
Диаграмма размещения



ПРАВИЛА UML

- спецификации;
- дополнения;
- принятые разделения;
- механизмы расширения.

ОБЩИЕ МЕХАНИЗМЫ UML



ВНУТРЕННЕЕ ПРЕДСТАВЛЕНИЕ МОДЕЛИ

ДОПОЛНЕНИЯ

СТАНДАРТНЫЕ ДИХОТОМИИ

МЕХАНИЗМЫ РАСШИРЕНИЯ

Расширение языка UML

Механизмы расширения – это встроенный в язык способ изменить язык. Авторы UML при унификации различных методов постарались включить в язык как можно больше различных средств, так чтобы язык оказался применимым в разных контекстах и предметных областях.

Но нельзя объять необъятного! Вполне могут возникать и возникают случаи, когда стандартных элементов моделирования не хватает или они не вполне адекватны.

Механизмы расширения позволяют определять новые элементы модели на основе существующих управляемым и унифицированным способом. Таких механизмов три:

1. помеченные значения;
2. ограничения;
3. стереотипы.

Эти механизмы не являются независимыми – они тесно связаны между собой.

Расширение языка UML

Механизмы расширения UML позволяют ввести в рассмотрение дополнительные графические обозначения, ориентированные для решения задач из определенной предметной области. Язык UML содержит два специальных расширения:

- *профиль для бизнес-моделирования (The UML Profile for Business Modeling).*
- *профиль для процесса разработки программного обеспечения (The UML Profile for Software Development Processes);*

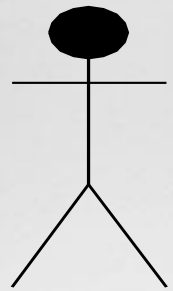
Механизмы расширения моделей

- **Стереотип** (*stereotype*) — новый тип элемента модели, который расширяет семантику метамодели. На диаграммах изображаются в форме текста, заключенного в угловые кавычки.
- **Помеченное значение** (*tagged value*) — явное определение свойства как пары "имя – значение". В помеченном значении само имя называют тегом (tag). На диаграммах изображаются в форме строки текста специального формата, заключенного в фигурные скобки.
- **Ограничение** (*constraint*) — некоторое логическое условие, ограничивающее семантику выбранного элемента модели. Ограничения на диаграммах изображаются в форме строки текста, заключенного в фигурные скобки .

Стереотипы

Ограничение стереотипа

{has full access
to the domain specified}



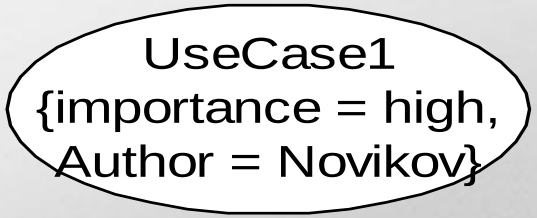
«PowerUser»
Administrator
{domain = LAN}

Измененный значок стереотипа

Имя стереотипа

Помеченное значение стереотипа

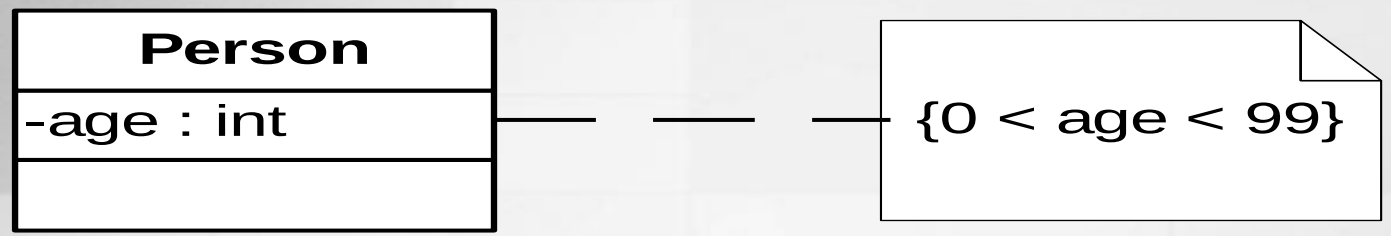
Именованные значения и ограничения



Вариант использования
с помеченными значениями

Класс с
атрибутом

Ограничение
на значение
атрибута



Дополнения

Класс

Ассоциация

Класс с дополнениями

Ассоциация с дополнениями

АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Для визуализации, специфицирования, конструирования и документирования программных систем необходимо рассматривать их с различных точек зрения. Все, кто имеет отношение к проекту, - конечные пользователи, аналитики, разработчики, системные интеграторы, тестировщики, технические писатели и менеджеры проектов - преследуют собственные интересы, и каждый смотрит на создаваемую систему по-разному в различные моменты ее жизни.

Модель «4+1»

Архитектура ПО многомерна, поскольку различные специалисты работают с её различными аспектами. Различные представления архитектуры служат различным целям (модель «4+1»):

1. представление функциональных возможностей ПО (представление вариантов использования, содержащее сценарии взаимодействия системы с внешней средой и роли, которые играют пользователи ПО и внешние системы);
2. представление логической организации ПО (логическое представление, элементами которого являются пакеты, подсистемы, классы, связи между ними);
3. представление физической структуры программных компонент, входящих в состав ПО (представление реализации, элементами которого являются компоненты, связи между ними);
4. представление структуры потоков управления и аспектов параллельной работы ПО (представление процессов, элементы которого: потоки управления, нити, параллелизм, синхронизация);
5. описание физического размещения компонент ПО по узлам вычислительной системы (представление размещения, элементы которого: узлы вычислительной системы, устройства, линии связи, задачи).

Представления архитектуры

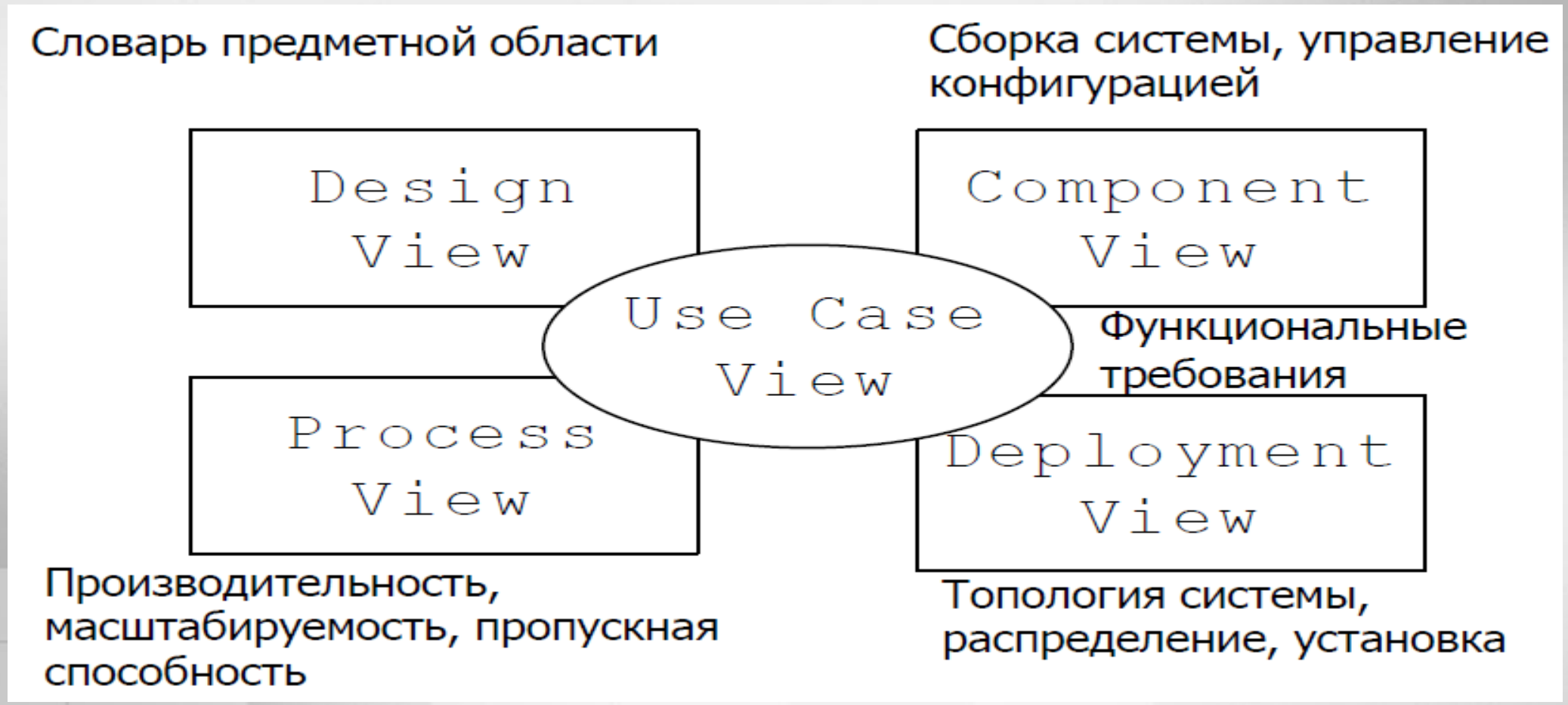
Среди 5-ти представлений особое место занимает представление вариантов использования, поскольку оно используется при управлении разработкой, служит своего рода скелетом проекта.
Каждое архитектурное представление – это модель системы с определенной точки зрения, в которой отражены лишь существенные аспекты и опущено все, что несущественно при данном взгляде на систему.

Представления (views)

Реальные модели сложны, сложную модель невозможно обозреть с одной точки зрения. Представление - проекция графа модели по одному из «измерений», средство логического структурирования модели.

1. Представление использования
   (Use Case View)
2. Логическое представление
   (Design View)
3. Представление процессов
   (Process View)
4. Представление компонентов
   (Component View)
5. Представление размещения
   (Deployment View)

Классические представления



Три представления

1. Представление использования (ЧТО делает система)
   - Диаграммы вариантов использования
2. Представление структуры (ИЗ ЧЕГО состоит система)
   - Диаграммы классов, компонентов и размещения
3. Представление поведения (КАК работает система)
   - Диаграммы состояний, деятельности и взаимодействия (последовательности / кооперации)

ПРОЦЕСС РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Язык UML в основном независим от организации процесса разработки, то есть не привязан к какому-либо конкретному циклу производства программного продукта, но для того, чтобы в максимальной степени воспользоваться его преимуществами, вам стоит рассмотреть процесс, который должен быть:

1. **управляемым вариантами использования;**
2. **сконцентрированным на архитектуре;**
3. **итеративным и пошаговым.**

1. ПРОЦЕСС, УПРАВЛЯЕМЫЙ ВАРИАНТАМИ ИСПОЛЬЗОВАНИЯ (USE CASE DRIVEN)

Процесс, *управляемый вариантами использования* *(use case driven)* – означает, что варианты использования используются в качестве первичных рабочих продуктов, на основании которых определяется желательное поведение системы, проверяется и подтверждается правильность выбранной системной архитектуры, производится тестирование и ведется общение заинтересованных лиц, имеющих отношение к проекту.

2. Процесс, сконцентрированный на архитектуре (architecturecentric)

Процесс, *сконцентрированный на архитектуре (architecturecentric)* – означает, что архитектура системы является первичным рабочим продуктом для процесса концептуализации, конструирования, управления и развития системы во время ее разработки.

Такой управляемый вариантами использования, сконцентрированный на архитектуре, итеративный и пошаговый процесс может быть разбит на **фазы**.

*Фаза – это отрезок времени между двумя важ*ными *контрольными точками (milestones) процесса, в которых до*стигаются четко определенные цели, завершено создание рабочих продуктов и принимается решение о переходе к следующей фазе.

Существуют четыре фазы жизненного цикла разработки: **начальная фаза, разработка, конструирование и внедрение.**

На диаграмме поток работ изображен в контексте этих фаз, в результате чего демонстрируется различный уровень концентрации на выполнении его составных частей.

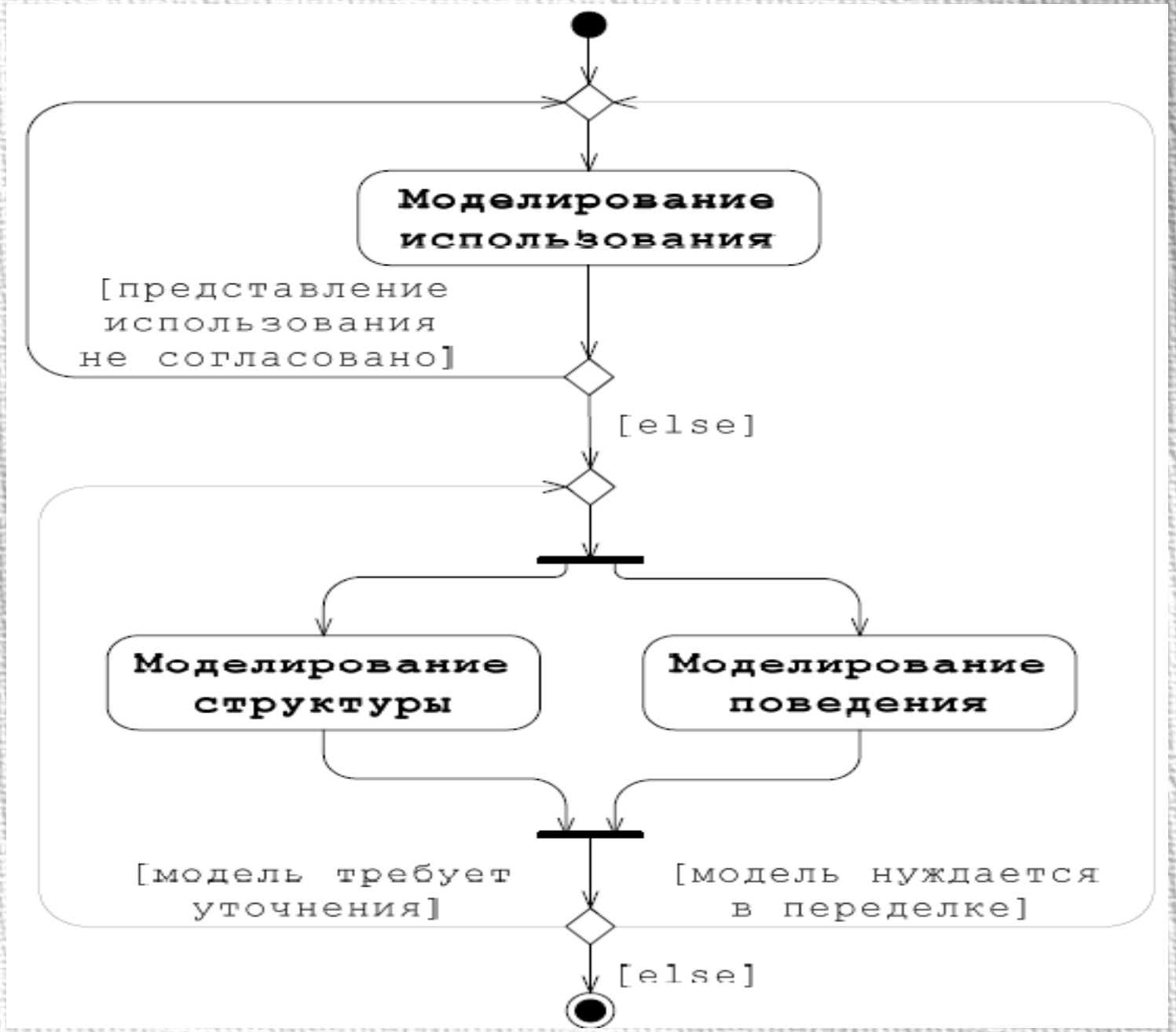
3. ИТЕРАТИВНЫЙ (ITERATIVE) ПРОЦЕСС

Итеративный (iterative) процесс *– тот, который включает управ*ление потоком исполняемых версий системы.

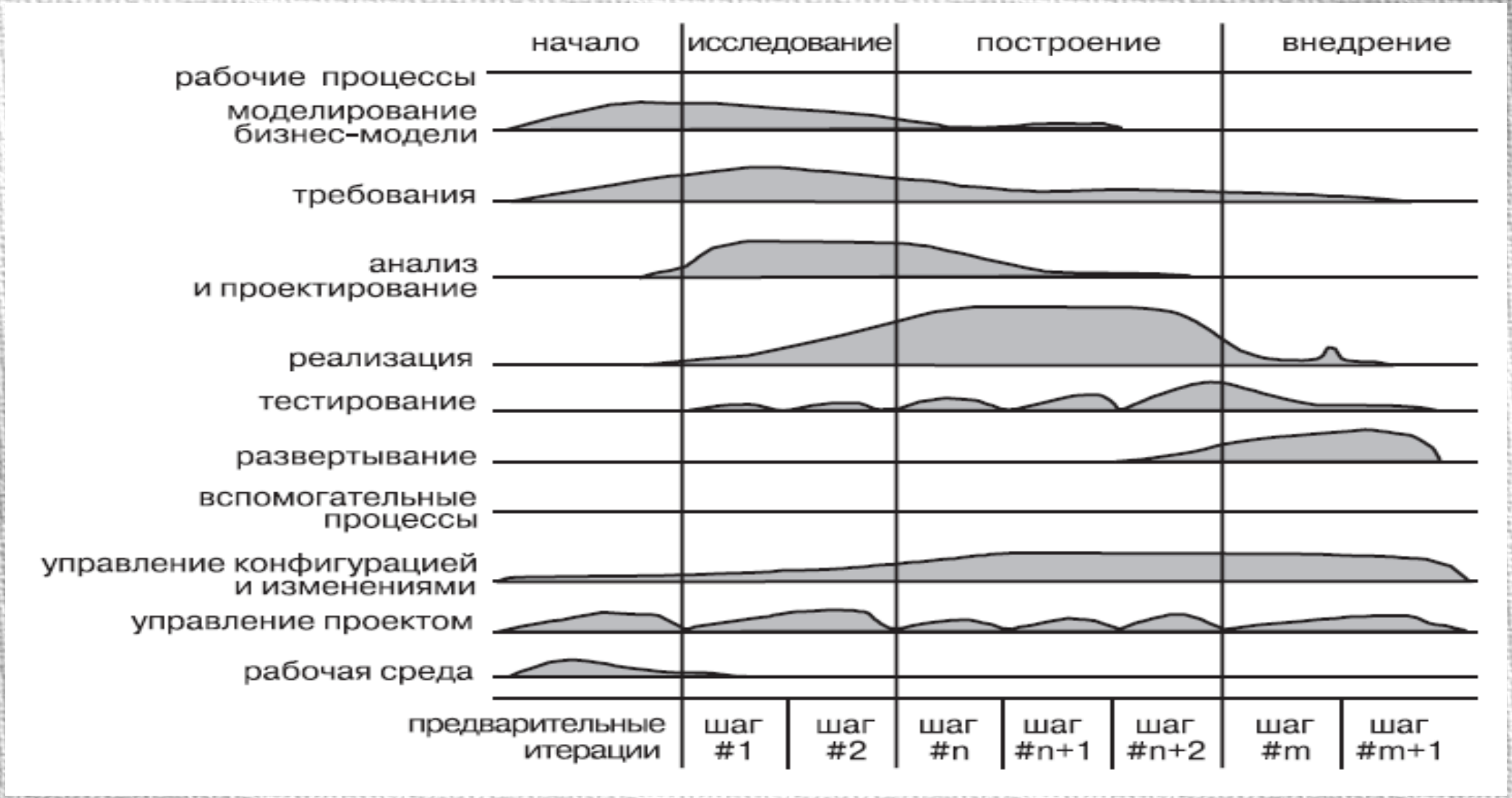
Пошаговый (incremental) процесс подразумевает непрерывную интеграцию системной архитектуры в целях выпуска версий, каждая последующая из которых усовершенствована по сравнению с предыдущей.

Процесс, являющийся итеративным и пошаговым, называется *процессом с управляемым риском (risk-driven), поскольку при выпуске каждой* новой версии (релиза) серьезное внимание уделяется выявлению факторов, представляющих наибольший риск для проекта в целом, и сведения их к минимуму.

ИТЕРАТИВНЫЙ ПРОЦЕСС МОДЕЛИРОВАНИЯ



Жизненный цикл разработки программной системы



Контроль знаний

1. Буква «U» в аббревиатуре «UML» означает:
   - А) United
   - Б) Unified
   - В) Universal

2. Модель UML состоит из (укажите лишнее):
   - А) сущностей
   - Б) отношений
   - В) множеств

3. Сущности UML подразделяются на (укажите лишнее)
   - А) структурные
   - Б) поведенческие
   - В) графические
   - Г) группирующие
   - Д) аннотационные

4. Отношения UML подразделяются на (укажите лишнее)

- А) зависимости
- Б) ассоциации
- В) уточнения
- Г) обобщения
- Д) реализации

5. Структурные сущности UML включают в себя (укажите лишнее)

- А) классы
- Б) узлы
- В) пакеты
- Г) варианты использования
- Д) интерфейсы

6. Поведенческие сущности UML включают в себя (укажите лишнее)

- А) состояния
- Б) деятельности
- В) варианты использования
- Г) интерфейсы

7. Сущностями UML являются (укажите лишнее)

- А) классы
- Б) узлы
- В) зависимости
- Г) примечания
- Д) варианты использования

8. Группирующие сущности UML включают в себя

- А) классы
- Б) узлы
- В) пакеты
- Г) примечания

9. Аннотационные сущности UML включают в себя

- А) классы
- Б) узлы
- В) пакеты
- Г) примечания

10. Отношения зависимости в UML являются

- А) симметричными
- Б) антисимметричными
- В) транзитивными

11. Отношения обобщения в UML являются
- А) симметричными
- Б) антисимметричными
- В) транзитивными

12. Отношения ассоциации (без дополнений) в UML являются
- А) симметричными
- Б) антисимметричными
- В) транзитивными

13. Отношения реализации в UML являются
- А) симметричными
- Б) антисимметричными
- В) транзитивными

14. Множество канонических диаграмм UML
- А) определяется стандартом языка
- Б) является соглашением пользователей языка
- В) определяется производителями инструментов,
- поддерживающих UML

15. Множество канонических структурных диаграмм UML включает в себя (укажите лишнее)
- А) Диаграммы классов

ОСНОВНЫЕ ВОПРОСЫ

При создании программной системы необходимо ответить на вопросы:

- **Что делает система?**

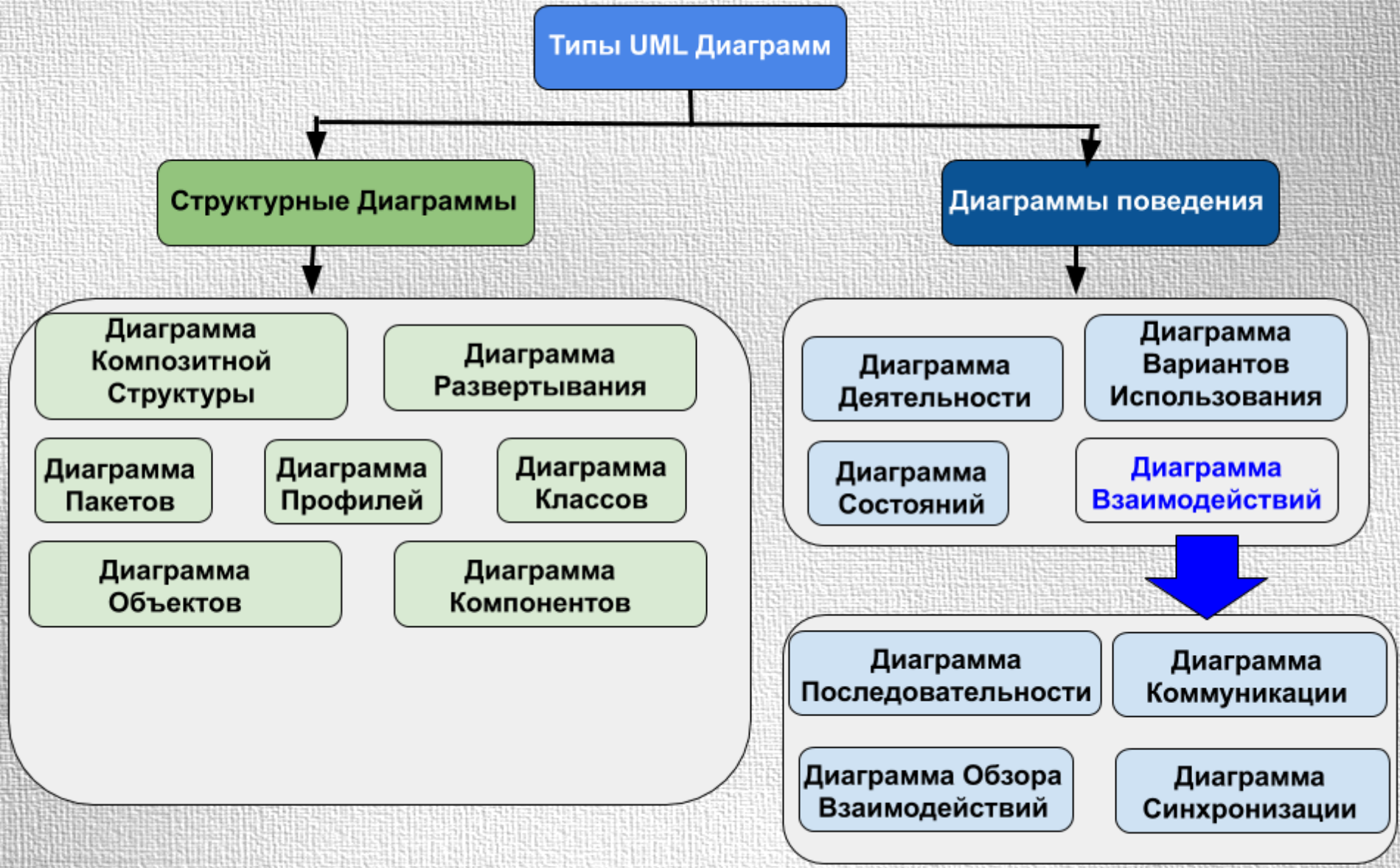
  Моделирование использования

- **Из чего она состоит?**

  Моделирование структуры

- **Как работает система?**

  Моделирование поведения


Типы UML Диаграмм
Структурные Диаграммы
Диаграммы поведения
Диаграмма Композитной Структуры
Диаграмма Развертывания
Диаграмма Пакетов
Диаграмма Профилей
Диаграмма Классов
Диаграмма Объектов
Диаграмма Компонентов
Диаграмма Деятельности
Диаграмма Вариантов Использования
Диаграмма Состояний
Диаграмма Взаимодействий
Диаграмма Последовательности
Диаграмма Коммуникации
Диаграмма Обзора Взаимодействий
Диаграмма Синхронизации

МОДЕЛИРОВАНИЕ ИСПОЛЬЗОВАНИЯ

- Диаграммы вариантов использования (прецедентов)

- Основные элементы ДВИ
  - Пользователи
  - Варианты использования
- Отношения на ДВИ

ДИАГРАММА ПРЕЦЕДЕНТОВ
(USE CASE DIAGRAM)

ОПРЕДЕЛЕНИЕ

Диаграмма использования – это наиболее общее представление функционального назначения системы

Диаграмма вариантов использования (ДВИ) призвана ответить на главный вопрос моделирования:

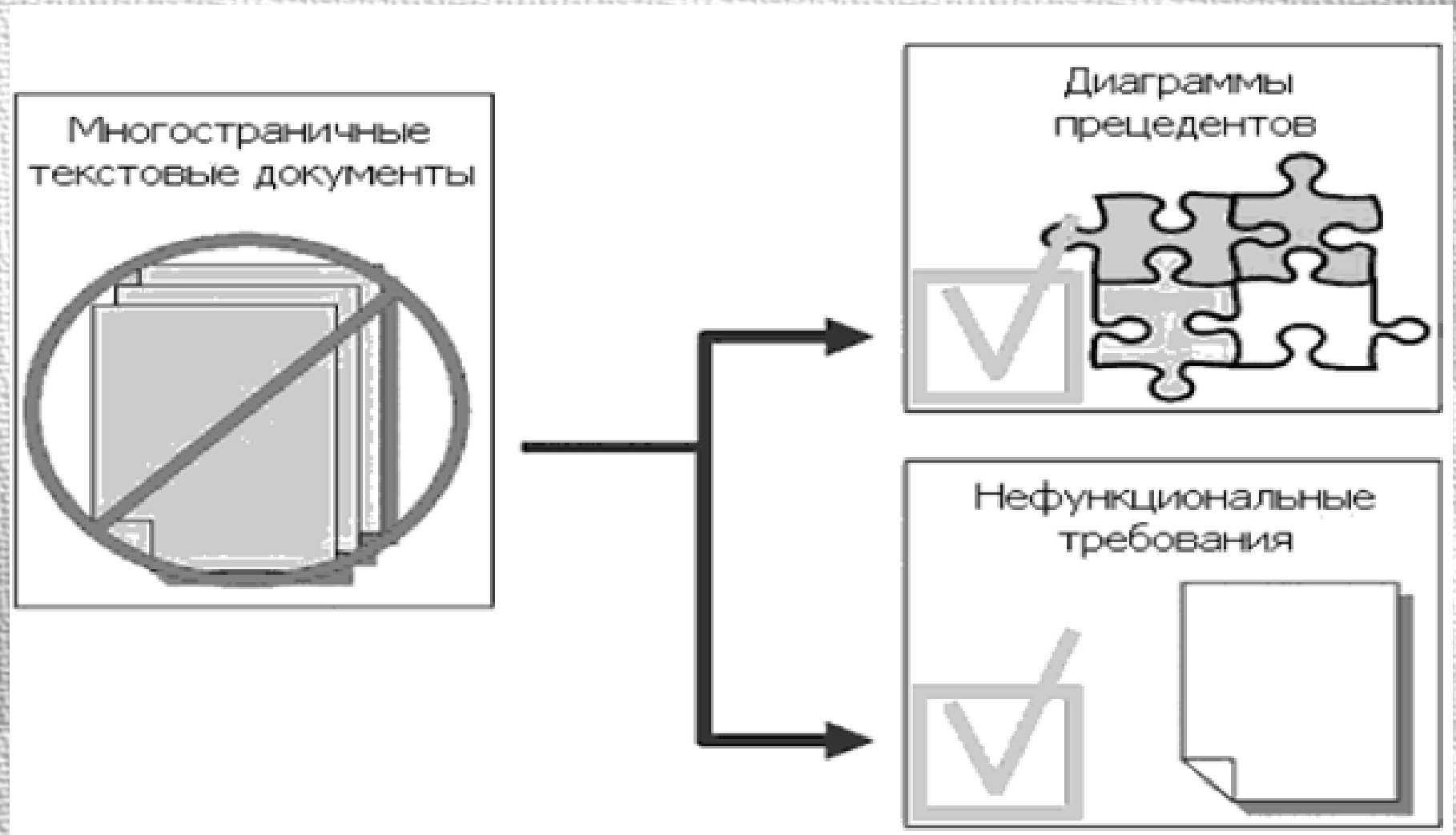
Что делает система во внешнем мире?

От ТЗ к …?

Техническое задание - основной документ, без составления которого не начинался в советские времена ни один проект. Документ это был большой, многостраничный, с четкой структурой, определяемой ГОСТами. И описывал он, *по* сути, не что иное, как требования к создаваемой системе!

Техническое задание - вещь *по*-своему хорошая. Но время шло, менялись стандарты, нотации, способы описания требований. И вот постепенно *техническое задание* уступило *место* набору артефактов, состоящему из документов двух видов:

1. **Диаграммы прецедентов;**
2. **Нефункциональные требования**.


Многостраничные
текстовые документы
Диаграммы
прецедентов
Нефункциональные
требования

ЦЕЛИ РАЗРАБОТКИ ДВИ

Разработка диаграммы вариантов использования преследует цели:

- Определить общие границы и контекст моделируемой предметной области на начальных этапах проектирования системы.
- Сформулировать общие требования к функциональному поведению проектируемой системы.
- Разработать исходную концептуальную модель системы для ее последующей детализации в форме логических и физических моделей.
- Подготовить исходную документацию для взаимодействия разработчиков системы с ее заказчиками и пользователями.

ОСНОВНЫЕ ЭЛЕМЕНТЫ ДИАГРАММЫ

1. Use Case
2. Actor
3. Отношения

Диаграммы имеют самую простую нотацию: всего два основных типа сущностей и три типа отношений (зависимости, ассоциации, обобщения)!

Как переводят USE CASE?

1. Вариант использования (ВИ)
2. Сценарий использования
3. Прецедент
   - **Прецедент** - (от лат. praecedens - род. п. praecedentis – предшествующий) - случай, имевший место ранее и служащий примером или оправданием для последующих случаев подобного рода.

Как переводят ACTOR?

1. Пользователь
2. Субъект
3. Действующее лицо
4. Исполнитель
5. Актер
6. Актант (пришло из лингвистики и означает любой член предложения, обозначающий лицо, предмет, участвующий в процессе, обозначенном глаголом)

Как видите, ACTOR повезло меньше!

ЧТО ТАКОЕ «ВАРИАНТ ИСПОЛЬЗОВАНИЯ»?

Прямого ответа, как пишет Г.Буч, не существует. Но можно попытаться на него ответить, описав вначале сценарий.

Сценарий представляет собой последовательность шагов, описывающих взаимодействие между пользователем и системой. Например, оплата товара кредитной карточкой

1.

2.…

Подобный сценарий описывает только одну ситуацию, которая может иметь место. Если авторизация кредитной карточки окажется неудачной, то сценарий будет другим.

ВИ представляет собой множество сценариев, объединенных вместе некоторой общей целью пользователя.

ВИ={сценарий}

Цель варианта использования заключается в том, чтобы определить законченный аспект или фрагмент поведения некоторой сущности без раскрытия внутренней структуры этой сущности. В качестве такой сущности может выступать исходная система или любой другой элемент модели, который обладает собственным поведением, подобно подсистеме или классу в модели системы.

BИ как сервис

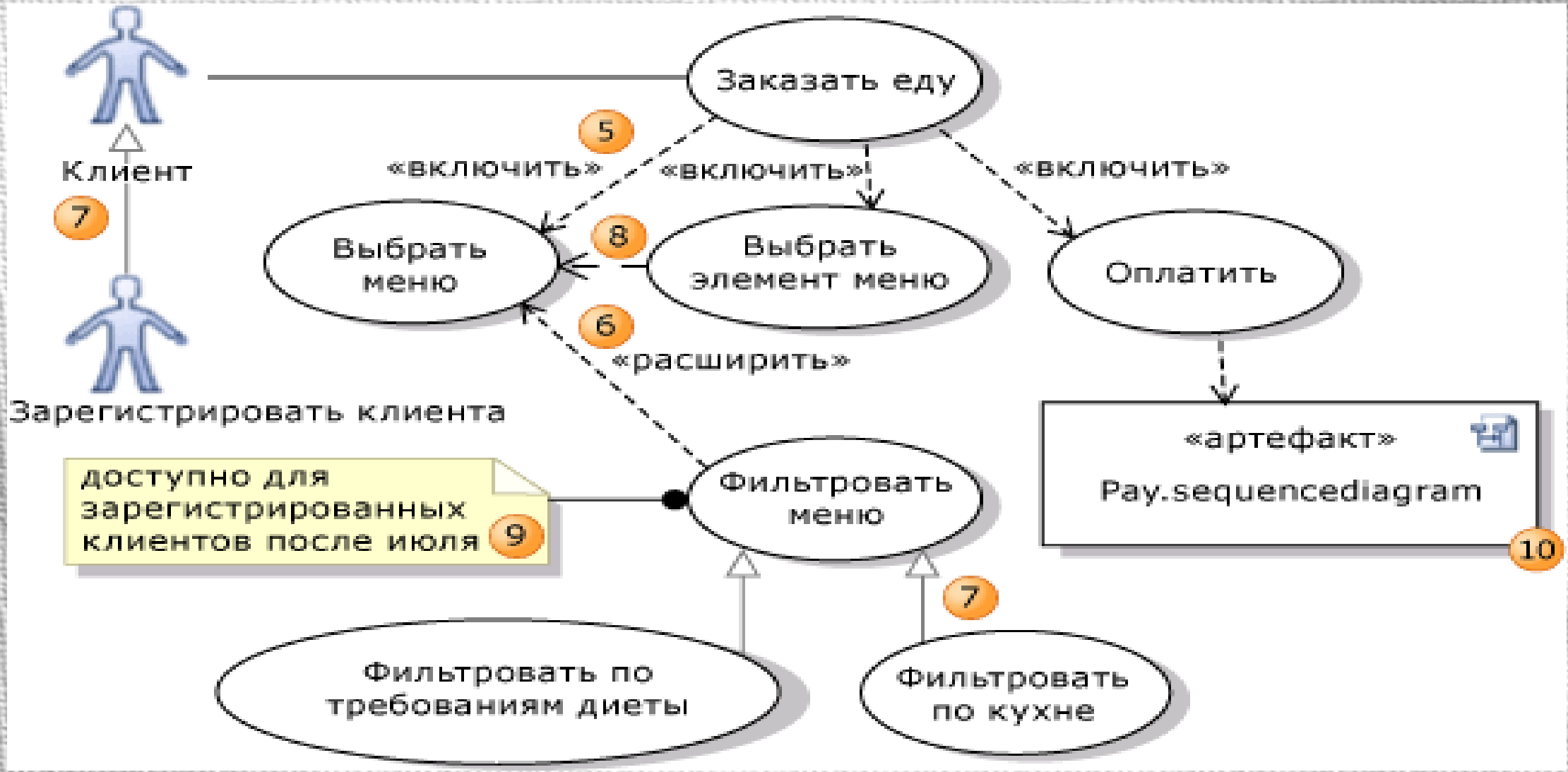
Каждый вариант использования соответствует отдельному сервису, который предоставляет моделируемая сущность или система по запросу пользователя (актера), т. е. определяет способ применения этой сущности. Сервис, который инициализируется по запросу пользователя, представляет собой законченную последовательность действий (**транзакцию**) . Это означает, что после того как система закончит обработку запроса пользователя, она должна возвратиться в исходное состояние, в котором готова к выполнению следующих запросов.

С системно-аналитической точки зрения варианты использования могут применяться как для спецификации внешних требований к проектируемой системе, так и для спецификации функционального поведения уже существующей системы. Кроме этого, варианты использования неявно устанавливают требования, определяющие, как пользователи должны взаимодействовать с системой, чтобы иметь возможность корректно работать с предоставляемыми данной системой сервисами!

Субъекты, варианты использования и подсистемы

Фигура	Элемент	Описание и основные свойства
1	**Субъект**	Представляет пользователя, организацию или внешнюю систему, взаимодействующую с используемым приложением или системой. Субъект — это вид типа.
2	**Вариант использования**	Представляет действия, выполненные одним или несколькими субъектами для достижения конкретной цели. Вариант использования — это вид типа.
3	**Ассоциация**	Указывает, что субъект принимает участие в варианте использования.
4	**Подсистема или компонент**	Система или приложение, с которым ведется работа, либо часть системы или приложения. Может представлять собой что угодно — от крупной сети до одного класса в приложении. Варианты использования, поддерживаемые системой или компонентом, отображаются внутри прямоугольника. Чтобы более ясно очертить область действия системы, рекомендуется показать некоторые варианты использования за пределами прямоугольника. Подсистема на схеме вариантов использования, по сути, имеет тот же тип, что и компонент на схеме компонентов.

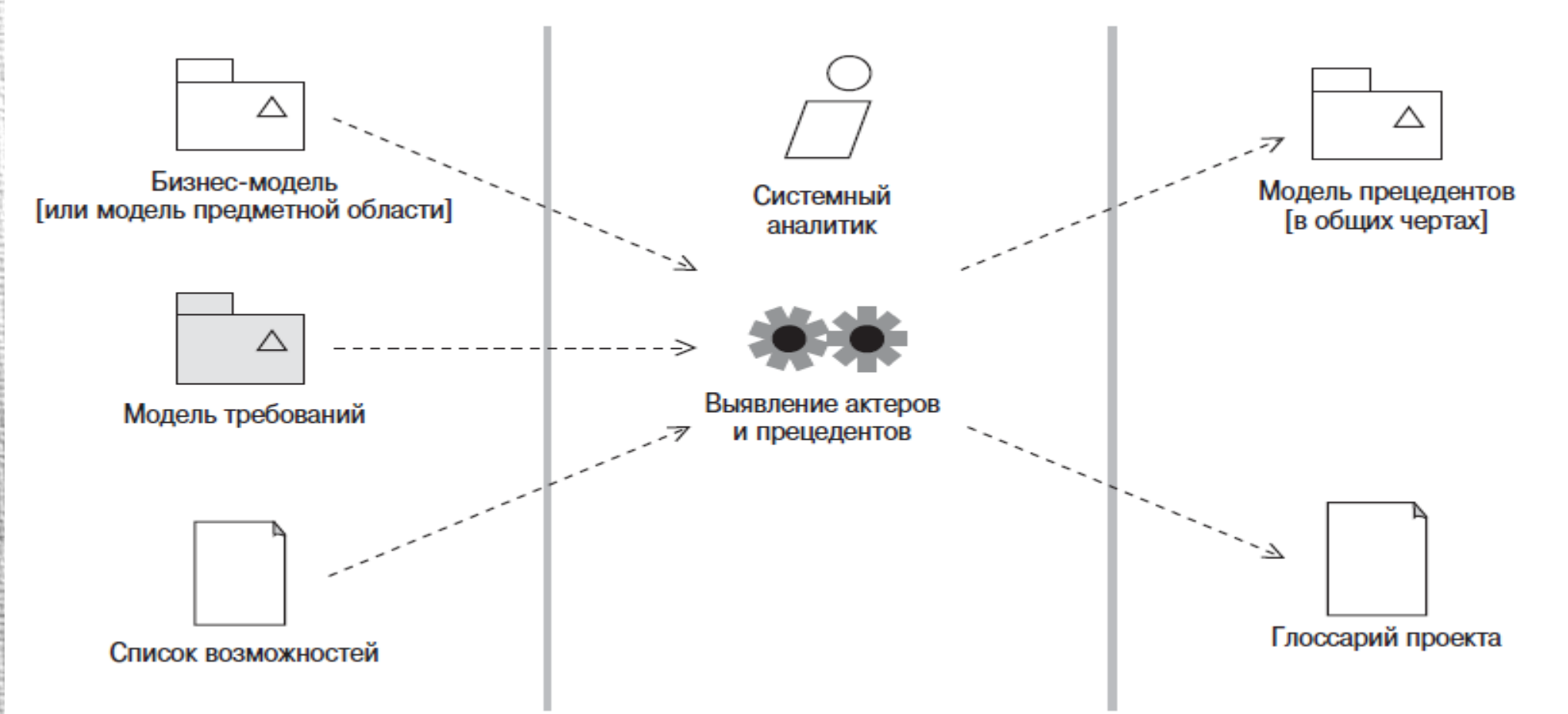
Структурирование вариантов использования



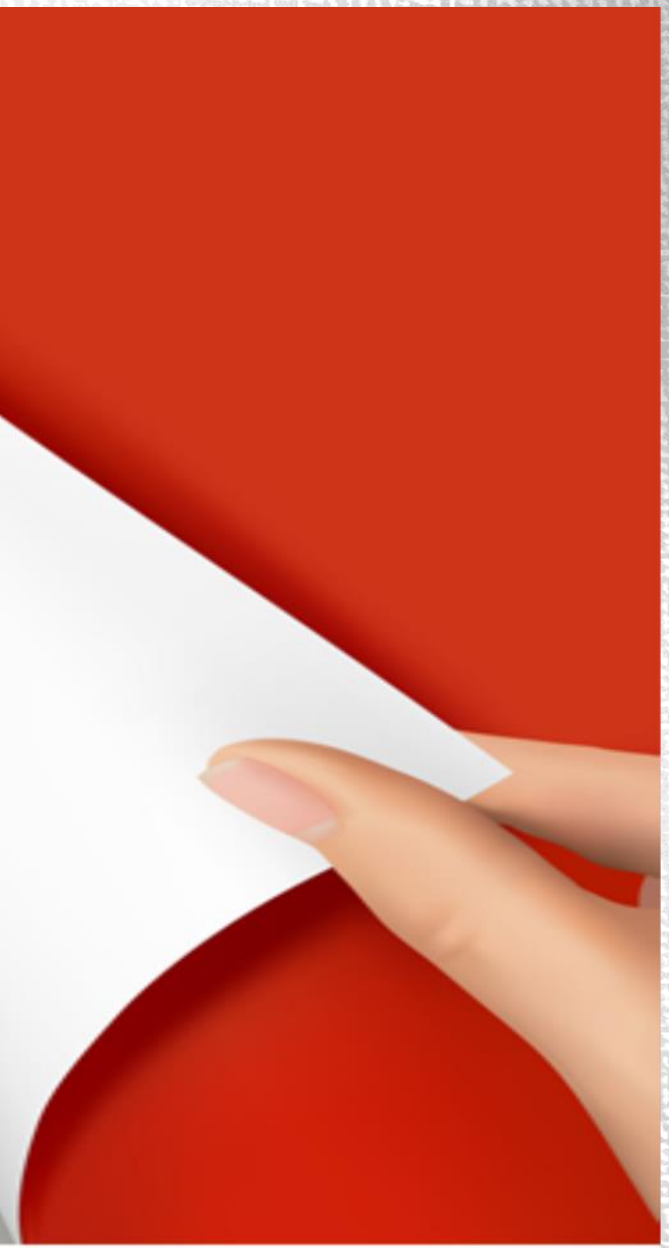
Фигура	**Элемент**	Описание
5	**Включение**	Включающий вариант использования вызывает включенный. Включение используется, чтобы показать, как разбить вариант использования на несколько более мелких шагов. Включенный вариант использования находится на окончании с наконечником стрелки. Обратите внимание, что на схеме не показана последовательность шагов. Для подробного описания этих шагов можно воспользоваться схемой деятельности, схемой последовательностей или другим документом.
6	**Расширение**	Расширяющий вариант использования добавляет цели и шаги в расширяемый вариант использования. Расширения работают только при определенных условиях. Расширенный вариант использования находится на окончании с наконечником стрелки. Обратите внимание, что на схеме не показаны конкретные условия, при которых применяются расширения: их можно записать в комментарии или другом документе.
7	**Наследование**	Устанавливает отношение между специализированным и обобщенным элементом. Обобщенный элемент находится на окончании с наконечником стрелки. Специализированный вариант использования наследует цели и субъекты своего обобщения и может добавлять более конкретные цели и шаги для их достижения. Специализированный субъект наследует варианты использования, атрибуты и ассоциации своего обобщения и может добавлять дополнительные объекты.

8	**Зависимость**	Указывает, что конструкция источника зависит от конструкции целевого объекта.
9	**Комментарий**	Используется для добавления общих примечаний на схеме.
10	**Артефакт**	Артефакт предоставляет ссылку на другую схему или документ. Его можно создать, перетащив файл из Обозревателя решений. С помощью инструмента "Зависимость" артефакт можно связать с любым другим элементом на схеме. Как правило, артефакт используется для связи варианта использования со схемой последовательностей, страницей OneNote, документом Word или презентацией PowerPoint, которая подробно его описывает. Документ может либо представлять собой элемент в решении Visual Studio, либо документ в расположении с общим доступом, например на сайте SharePoint.
11 (не показана)	**Пакеты**	Варианты использования, субъекты и подсистемы могут содержаться внутри пакетов. Фигуры пакетов не отображаются на схеме, но можно задать свойство схемы **LinkedPackage**. Элементы, которые впоследствии будут созданы на схеме, помещаются в этот пакет.

Глоссарий проекта



Действующие лица



Определения

С синтаксической точки зрения *действующее лицо* (actor) – это стереотип классификатора, который обозначается специальным значком. Для действующего лица указывается только имя, идентифицирующее его в системе.

Семантически *действующее лицо* — это множество логически взаимосвязанных ролей.

***Роль* (role) в UML — это *интерфейс* (interface),** поддерживаемый данным *классификатором* (classifier) в данной *ассоциации* (assocation).

С прагматической точки зрения главным является то, что **действующие лица находятся вне проектируемой системы** (или рассматриваемой части системы).

Вопрос о выделении (или идентификации) ДЛ при составлении модели — один из самых болезненных. Неудачный выбор ДЛ может отрицательно повлиять на всю модель в целом.

Здесь легко впасть в крайность: объявить, что имеется одно действующее лицо (внешний мир), взаимодействующее со всеми вариантами использования или, наоборот, придумать искусственных ДЛ лиц для каждого варианта использования.

Оба экстремальных варианта являются, по существу, моделью черного ящика и сводят к нулю преимущества моделирования использования. Формального метода идентификации ДЛ лиц не существует.

Каждое ДЛ должно иметь одну явно выраженную цель. Если объект из реального мира сочетает в себе несколько целей, их следует представлять несколькими ДЛ.

Действующие лица

- Действующие лица находятся ВНЕ проектируемой системы
- Действующее лицо – это множество логически взаимосвязанных РОЛЕЙ
- Действующее лицо – это стереотипный КЛАСС

Следует всегда помнить, что актер – это не конкретный человек или устройство, а роль (должностная обязанность), в которой он выступает по отношению к программной системе. Например, в качестве актера «Бухгалтер» может выступать весь наличный штат бухгалтерии. В то же время один конкретный человек может играть несколько ролей по отношению к системе. Главный бухгалтер может выступать как актер с таким же именем, но может использовать систему так же, как актер «бухгалтер» (то есть, выполнять работу обычного бухгалтера). В то же время актеры – не обязательно люди. Если ПС должна выполнять какие-либо действия в определенные моменты времени, то в качестве актера, инициирующего выполнение этих действий, может быть указан системный таймер.

ОСНОВНЫЕ ТИПЫ

По отношению к системе в целом можно выделить 4 типа актёров:

1. Пользователи;
2. Программные системы (отличные от разрабатываемой);
3. Аппаратные/технические средства, приборы;
4. Время. Время становится действующим лицом, если от него зависит запуск каких-либо событий в системе.

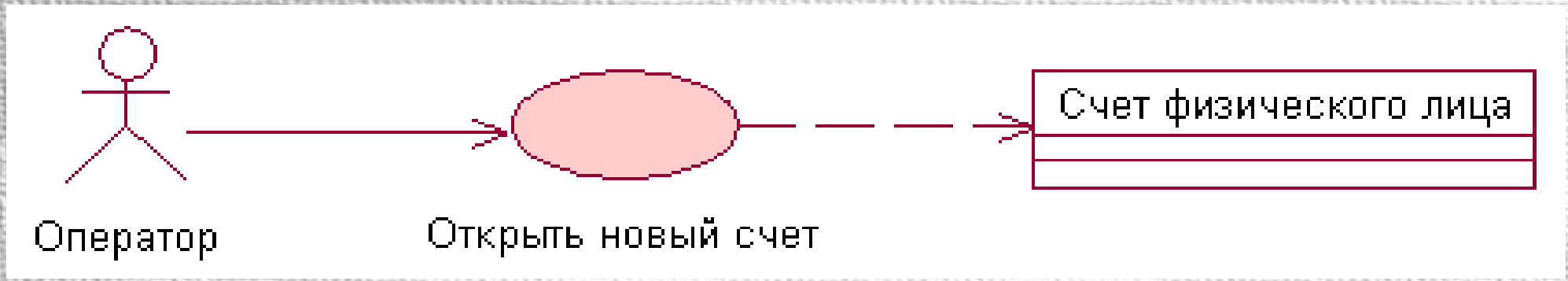
Идентификация актеров

Следующие вопросы помогут идентифицировать актеров.

- **Кто или что использует систему?**
- **Какие роли они играю во взаимодействии?**
- **Кто устанавливает систему?**
- **Кто или что запускает и выключает систему?**
- **Кто обслуживает систему?**
- **Какие системы взаимодействуют с данной системой?**
- **Кто или что получает и предоставляет информацию системе?**
- **Происходит ли что-нибудь в точно установленное время?**

Другие элементы UML

На диаграммах вариантов использования (ВИ) изображаются *актеры и варианты использования*, между которыми существуют *отношения*. Здесь можно показывать и другие элементы UML (например, классы могут показывать, какие сущности порождаются или используются в конкретных ВИ).



ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Информационная система «Отдел кадров» (сокращенно ИС ОК) предназначена для ввода, хранения и обработки информации о сотрудниках и движении кадров. Система должна обеспечивать выполнение следующих основных функций:

1. Прием, перевод и увольнение сотрудников.
2. Создание и ликвидация подразделений.
3. Создание вакансий и сокращение должностей.

Конечно, техническое задание из одного абзаца текста и трех нумерованных пунктов – это не более чем учебный пример

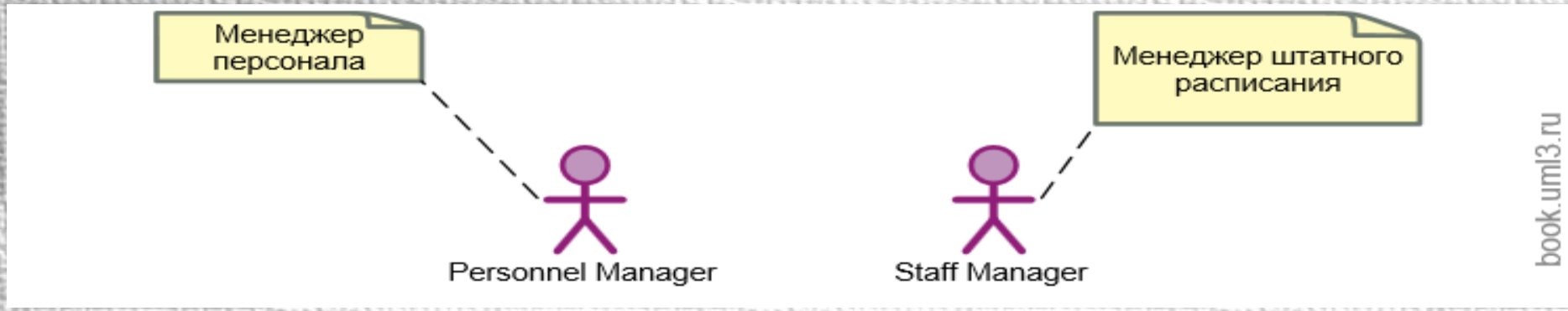
Как выделять пользователей?

Выделение категорий пользователей происходит, как правило, неформально: из соображений здравого смысла и собственного опыта. Тем не менее, несколько советов можно дать. Имеет смысл отнести пользователей к разным категориям, если наблюдаются следующие признаки:

- пользователи участвуют в разных (независимых) бизнес-процессах;
- пользователи имеют различные права на выполнение действий и доступ к информации;
- пользователи взаимодействуют с системой в разных режимах:
  - от случая к случаю,
  - регулярно,
  - постоянно.

Действующие лица ИС ОК

1. **Менеджер персонала**
   - Работает с конкретными людьми
2. **Менеджер штатного расписания**
   - Работает с абстрактными должностями и подразделениями



Обобщение действующих лиц

Обобщение действующих лиц (actors) обеспечивает возможность вынести поведение, общее для двух и более действующих лиц, в действующее лицо-родителя

- ДЛ-родитель является более обобщенным, чем его потомки, а ДЛ-потомки - более специализированными, чем их родитель.
- Дочернее ДЛ везде может заменять ДЛ-родителя - это принцип замещаемости.
- ДЛ-родитель обычно абстрактный - он определяет абстрактную роль.
- Дочерние ДЛ конкретны - они определяют конкретные роли
- Обобщение ДЛ может упрощать диаграммы вариантов использования.

Что дает обобщение

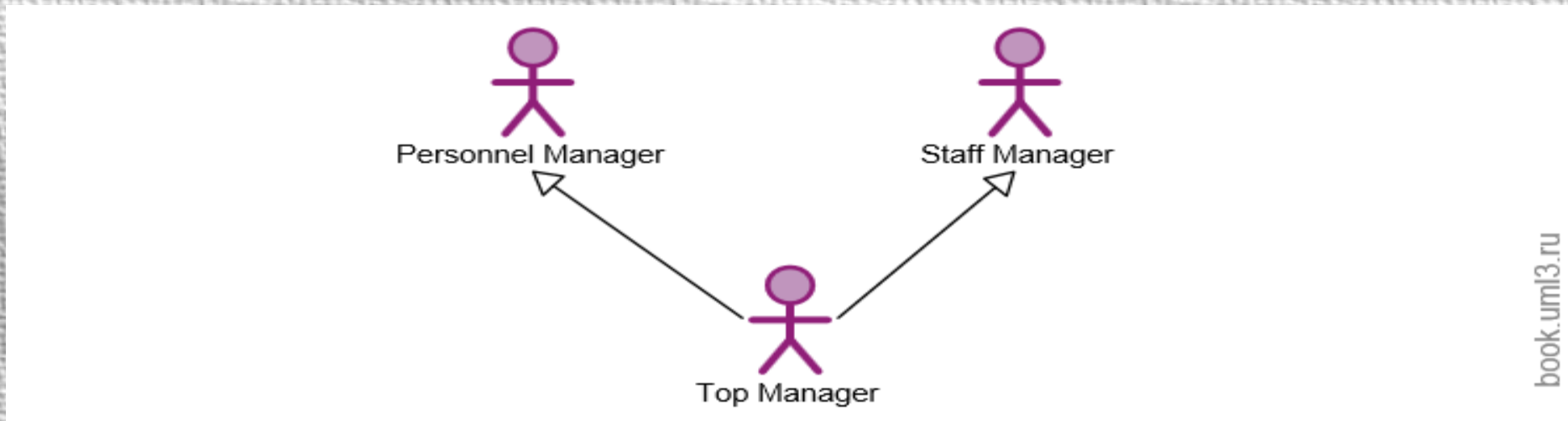
Обобщение между действующими лицами показывает, что одно действующее лицо наследует все свойства (в частности, участие в ассоциациях) другого действующего лица.

Такое обобщение является весьма мощным средством моделирования.

1. **Иерархия категорий пользователей.** Во-первых, с помощью обобщения между действующими лицами легко показать иерархию категорий пользователей системы, в частности, иерархию прав доступа к выполняемым функциям и хранимым данным.
2. **Абстрактное действующее лицо.** Во-вторых, действующее лицо, будучи классификатором, может быть абстрактным классификатором, то есть таким классификатором, который не может иметь непосредственных экземпляров. Введение абстрактных действующих лиц позволяет без потери информации сократить количество непосредственных ассоциаций в модели, сделав ее более лаконичной, а значит более наглядной и полезной.

Иерархия категорий пользователей ИС ОК

ИЗМЕНЕНИЯ В ТЕХНИЧЕСКОМ ЗАДАНИИ. Среди всех пользователей информационной системы следует выделить особую категорию пользователей (высшее руководство), которой разрешен доступ к любым данным и операциям.

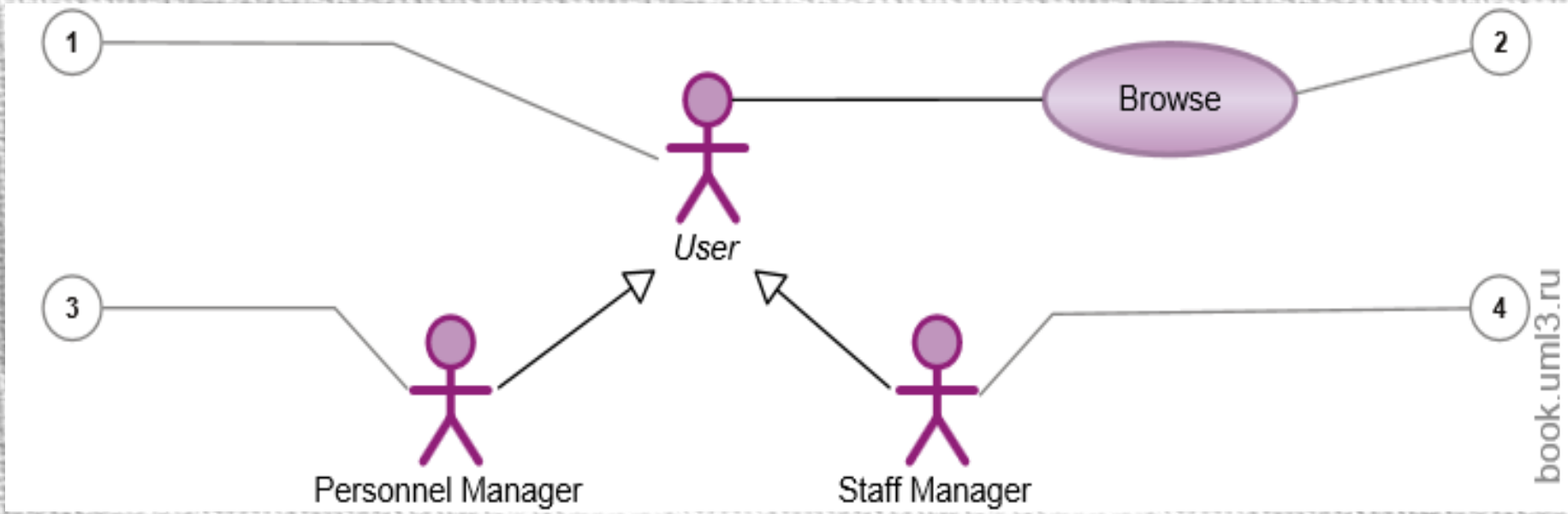


Абстрактное действующее лицо

ИЗМЕНЕНИЯ В ТЕХНИЧЕСКОМ ЗАДАНИИ. Информационная система должна предоставлять возможность просматривать данные без внесения в них каких-либо изменений.

Данное требование следует оформить в виде дополнительного варианта использования – **Browse**. Если мы проектируем систему отдела кадров для обычной организации, а не для государственной секретной службы, то разумно предположить, что просматривать данные могут все категории пользователей. В этом случае можно установить ассоциации между новым вариантом использования и всеми действующими лицами, а можно поступить так, как показано следующем рисунке.

Ввести обобщенного абстрактного пользователя *User* 1, который будет связан ассоциацией с вариантом использования Browse 2. При этом все специализации 3 и 4 обобщенного пользователя автоматически будут связаны ассоциацией с вариантом использования Browse.



Пример

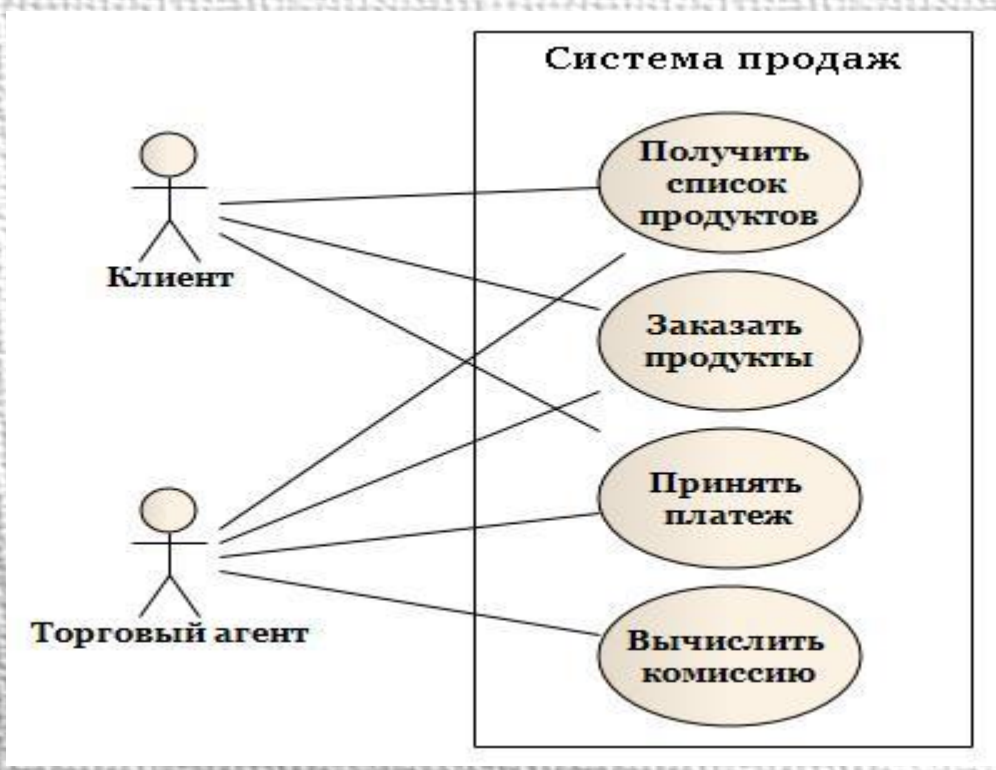


Рис. 1. Пример двух ДЛ с общим поведением

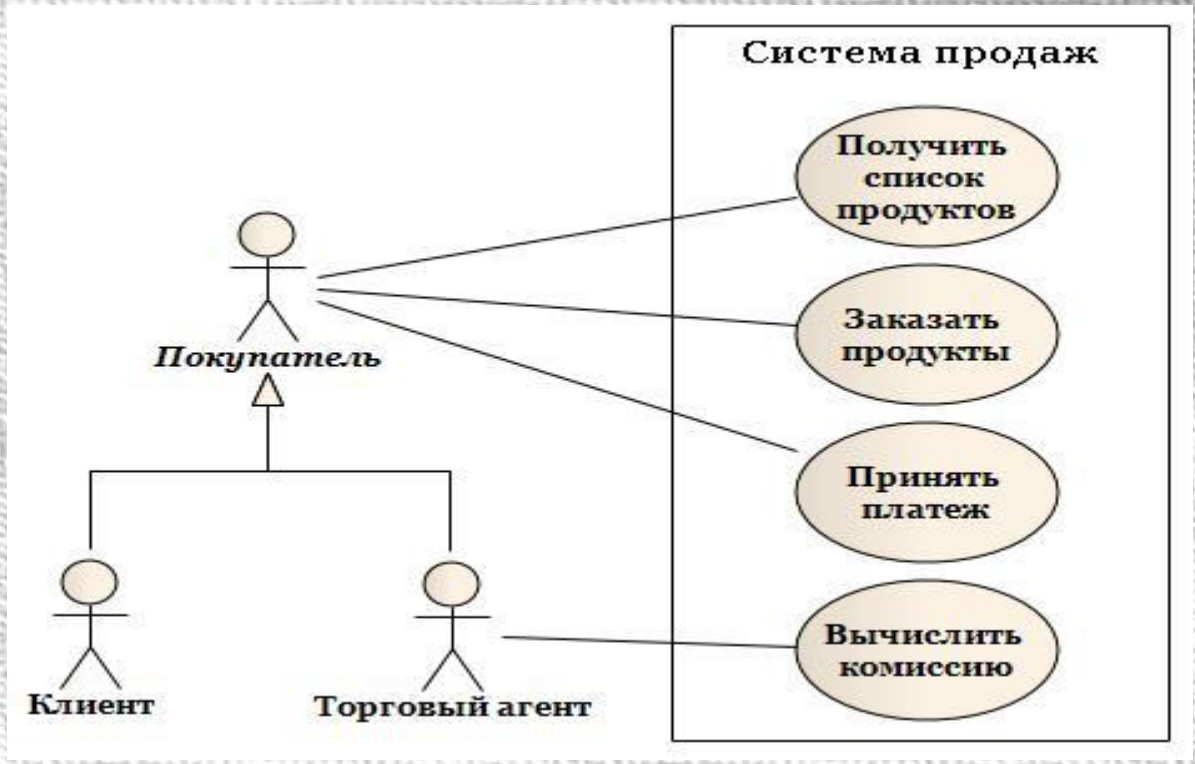


Рис. 2. Общее поведение вынесено в ДЛ-родителя

Имена

Для UML пока что нет достаточно устоявшейся дисциплины имен, но некоторый набор рекомендаций можно найти в литературе. В частности, в качестве имен действующих лиц рекомендуется использовать существительное (возможно, с определяющим словом), а в качестве имен вариантов использования – глагол (возможно, с дополнением). Эти правила основаны на семантике моделирования использования.

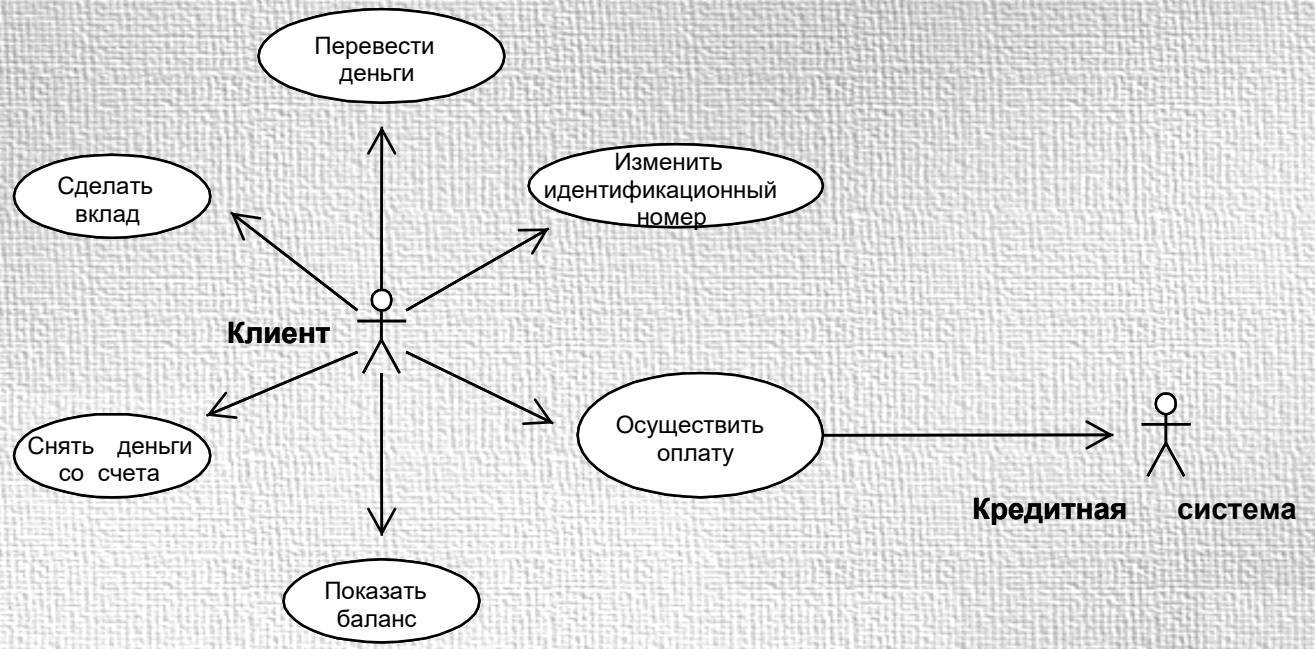
Вариант использования

Вариант использования

Вариант использования представляет собой последовательность действий (транзакций), выполняемых системой в ответ на событие, инициируемое некоторым внешним объектом (действующим лицом).

Вариант использования описывает типичное взаимодействие между пользователем и системой.

В простейшем случае вариант использования определяется в процессе обсуждения с пользователем тех функций, которые он хотел бы реализовать.



Описание прецедента

- Семантика
- Структура
  - Черты
  - Отношения
    - Ассоциация
    - Обобщение
    - Расширение
    - Включение
  - Поведение
  - Реализация
- Нотация

Семантика

Вариант использования служит для определения некой части поведения классификатора (которым можно также считать подсистему и даже всю систему целиком), без указания на его внутреннюю структуру. **Каждый вариант использования описывает некую услугу, которую предоставляет своим пользователям классификатор.**

Иначе говоря, это некоторый способ использования классификатора, который виден со стороны. Вариант использования описывает всю последовательность сообщений, которую начинает пользователь, в терминах взаимодействия между пользователем и классификатором, включая ответы классификатора. К взаимодействию относятся только коммуникации между системой и актантами. Внутреннее поведение и реализация скрыты.

Имя варианта использования

Нет стандарта.

Рекомендуем начинать с глагола или глагольной группы (отглагольного существительного). Например, *Выплатить налог с оборота* или *Выплата налога с оборота*.

Не смешивайте оба стиля в одной модели!

Является уникальным идентификатором в рамках модели ВИ!

Все множество вариантов использования какого-либо классификатора или системы полностью описывает его поведение.

Каждый вариант использования представляет собой некую разумную долю функциональности, которая доступна пользователям. Обратите внимание, что под термином пользователь следует понимать не только людей, но и компьютеры, а также прочие объекты. Актер представляет собой некую идеализацию намерений пользователя, а не самого этого пользователя.

Один реальный пользователь может соответствовать нескольким актерам (может играть несколько ролей), а один актер может представлять одно и то же намерение сразу нескольких пользователей.

Вариант использования включает в себя описание

1) основного поведения, осуществляемого в ответ на запрос пользователя,
2) а также все возможные варианты этого поведения, например
   - альтернативные последовательности,
   - исключительное поведение и
   - обработка ошибок.

Цель создания варианта использования - описать некую часть функциональности во всех ее вариантах, включая ошибочные. Все варианты использования классификатора специфицируют все возможные способы его использования. Для большего удобства варианты использования можно группировать в пакеты.

Вариант использования является дескриптором, поскольку описывает возможное поведение.

При выполнении варианта использования возникает его экземпляр.

Поведение варианта использования определяется при помощи прикрепленного к нему конечного автомата или при помощи текстового кода (что равнозначно конечному автомату). Кроме этого, оно может быть описано в виде неформального текста. Поведение можно проиллюстрировать (но не специфицировать формально) с помощью множества сценариев.

Экземпляр варианта использования - это выполнение варианта использования, которое начинается после получения первого сообщения от экземпляра актанта. В качестве реакции на данное сообщение вариант использования выполняет определенную последовательность действий, например, отправляет сообщения другим экземплярам актанта (не только тому, который инициировал его выполнение). В свою очередь, эти актанты отправляют сообщения данному экземпляру варианта использования, и взаимодействие продолжается до тех пор, пока экземпляр варианта использования не отреагирует на все входящие сообщения. После этого он заканчивается.

Вариант использования является спецификацией поведения системы (или любого другого классификатора) в целом, в процессе его взаимодействия с внешними актантами. **Внутренние взаимодействия между внутренними объектами системы, при помощи которых реализуется поведение, описываются кооперацией, которая реализует вариант использования.**

Большая опасность прецедентов заключается в том, что разработчики делают их очень сложными и застревают на них. Обычно чем меньше вы делаете, тем меньший вред можете нанести. Если у вас немного информации, то получится короткий, легко читаемый документ, который явится отправной точкой для вопросов. Если информации слишком много, то вряд ли кто-то вообще будет ее изучать и пытаться понять.

Структура ВИ

- Структура
  - Черты
  - Отношения
    - Ассоциация
    - Обобщение
    - Расширение
    - Включение
  - Поведение
  - Реализация

Черты

Вариант использования является классификатором, а значит, у него есть атрибуты и операции.

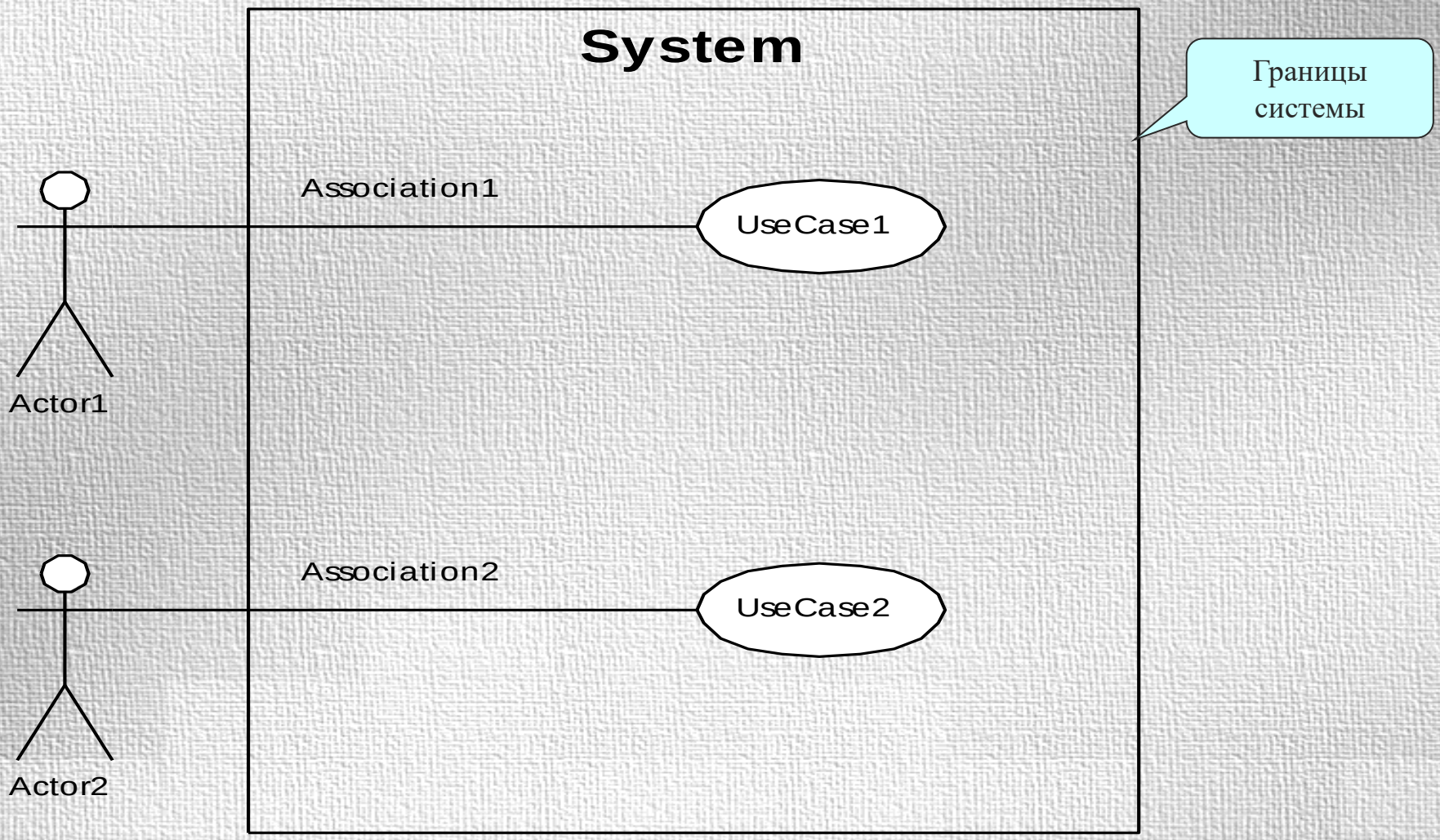
Атрибуты используются для обозначения состояния варианта использования, - иначе говоря, процесс его выполнения.

Операция представляет собой часть работы, которую может выполнить вариант использования. Операцию нельзя вызывать напрямую, однако, ее можно использовать при описании воздействия варианта использования на систему. Выполнение операции можно связать с получением сообщения от ДЛ. Операции воздействуют на атрибуты варианта использования, а также косвенно на атрибуты системы или класса, к которому прикреплен вариант использования.

Для включения в диаграмму выбранные прецеденты должны удовлетворять следующим критериям

- прецедент должен описывать, **ЧТО** нужно делать, а не **КАК**; прецедент должен описывать действия с точки зрения **ИСПОЛНИТЕЛЯ**;
- прецедент должен возвращать исполнителю некоторое **СООБЩЕНИЕ**;
- последовательность действий внутри прецедента должна представлять собой одну **НЕДЕЛИМУЮ** цепочку.

Нотация диаграммы прецедентов



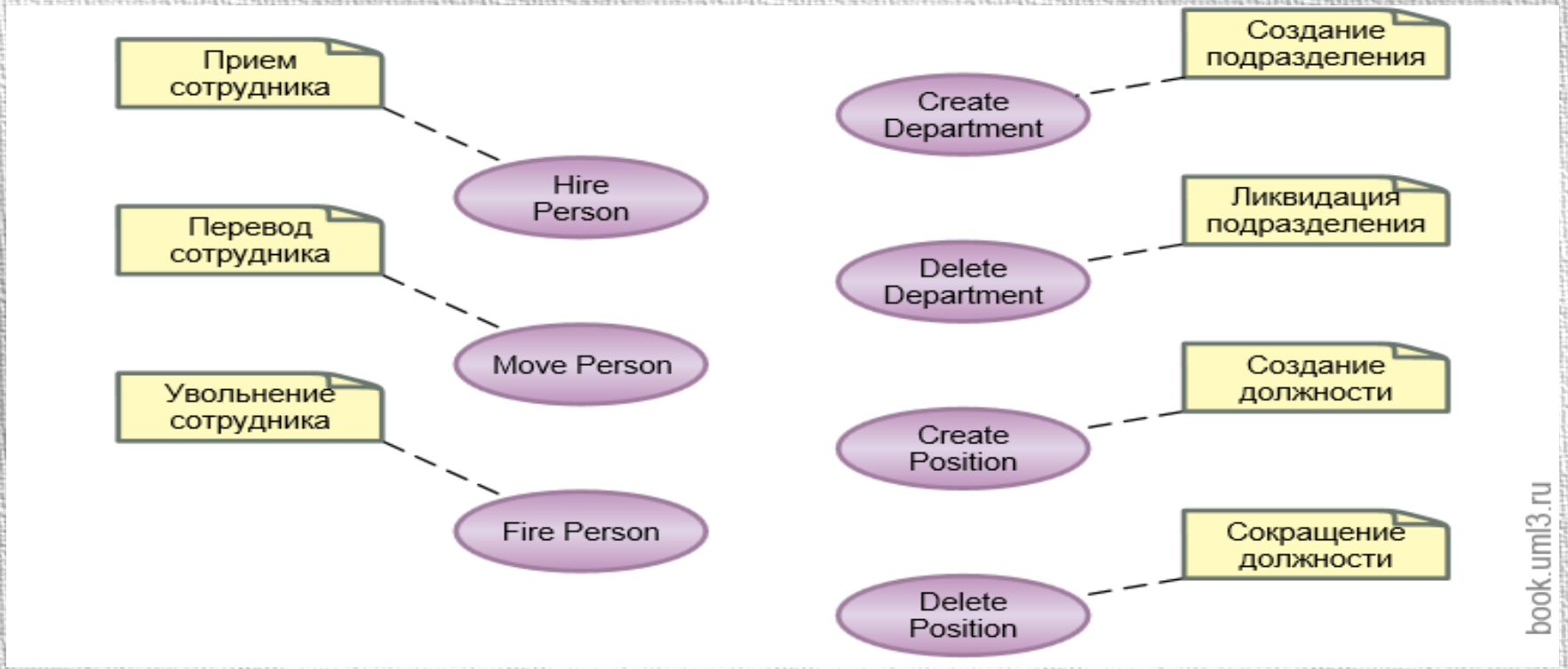
Варианты использования ИС ОК

- Менеджер персонала выполняет действия
  - Прием сотрудника
  - Перевод сотрудника
  - Увольнение сотрудника
- Менеджер штатного расписания выполняет действия
  - Создание подразделения
  - Ликвидация подразделения
  - Создание вакансии
  - Сокращение должности

Варианты использования ИС ОК

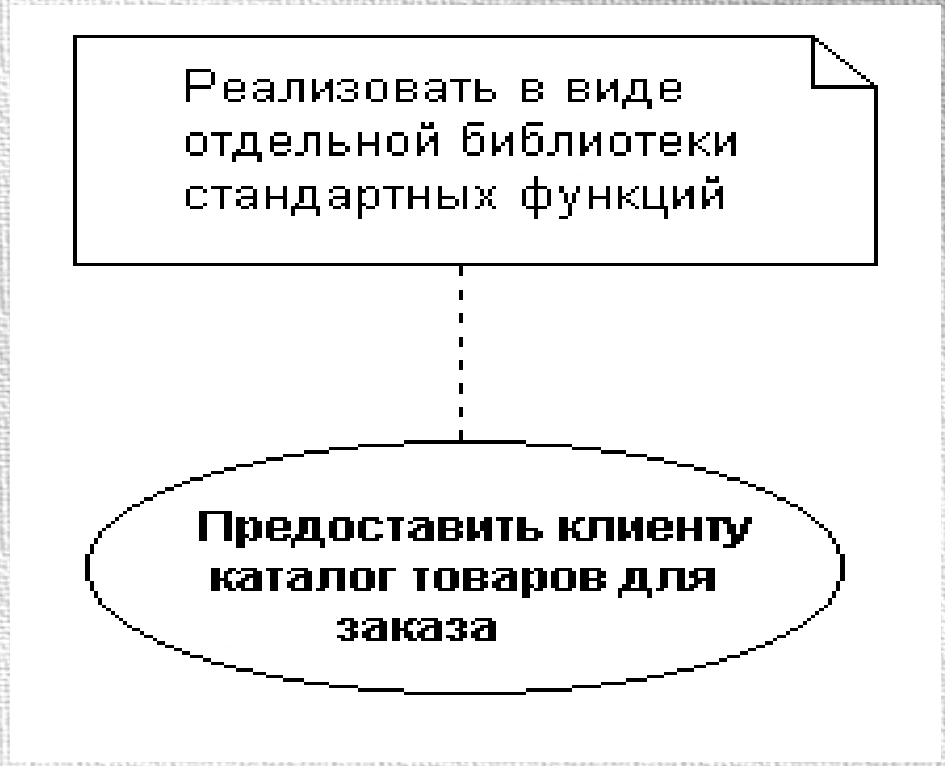
Простой анализ текста технического задания выявляет семь вариантов использования:

1. **прием сотрудника;**
2. **перевод сотрудника;**
3. **увольнение сотрудника;**
4. **создание подразделения;**
5. **ликвидация подразделения;**
6. **создание вакансии;**
7. **сокращение должности;**



Комментарии

Третьим типом сущности, применяемым на диаграмме использования, является **комментарий** (comment). Заметим, что комментарии являются очень важным средством UML, значение которого часто недооценивается начинающими пользователями. Комментарии можно и нужно употреблять на всех типах диаграмм, а не только на диаграммах использования. UML является унифицированным, но никак не универсальным языком — при моделировании проектировщик часто может сказать о моделируемой системе больше, чем это позволяет сделать строгая, но ограниченная нотация UML. В таких случаях наиболее подходящим средством для внесения в модель дополнительной информации является комментарий.



Комментарий имеет свою графическую нотацию – прямоугольник с загнутым уголком ("собачье ухо"), в котором находится текст комментария. Комментарии могут находиться в отношении соответствия с другими сущностями – эти отношения изображаются пунктирной линией без стрелок. Если пунктирная линия отсутствует, то комментарий относится ко всей диаграмме.

Комментарий содержит текст, который вводит пользователь — создатель модели. Это может быть текст в произвольном формате: на естественном языке, на языке программирования, на формальном логическом языке, например, OCL и т. д. Более того, если возможности инструмента это позволяют, в примечаниях можно хранить гиперссылки, вложенные файлы и другие артефакты, внешние по отношению к модели.

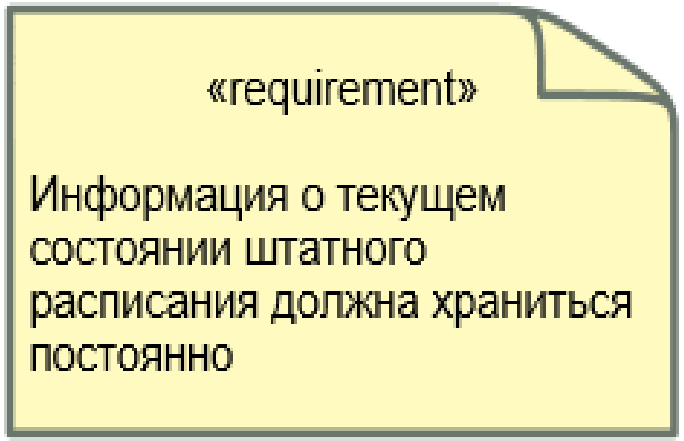
В UML последовательно проводится следующий важный принцип: **вся информация, которую пользователь вносит в модель, должна быть сохранена инструментом во внутреннем представлении модели** и предъявлена по первому требованию, даже если инструмент не умеет обрабатывать эту информацию. Комментарии являются важнейшим примером реализации этого принципа.

Комментарии могут иметь стереотипы. В UML определены два стандартных стереотипа для комментариев:

1. **«requirement»** — описывает общее требование к системе;
2. **«responsibility»** — описывает ответственность сущности (классификатора).

Комментарии первого стереотипа часто присутствуют на диаграммах использования, а комментарии второго стереотипа — на диаграммах классов.

Нефункциональное требование

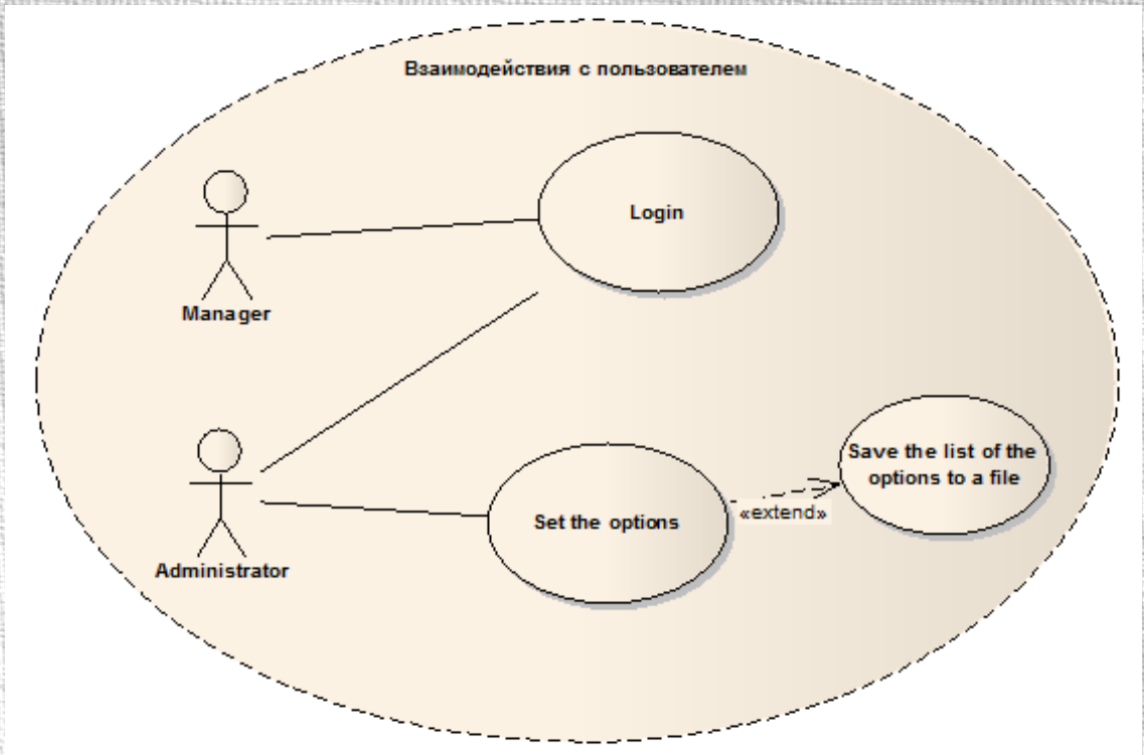


book.uml3.ru

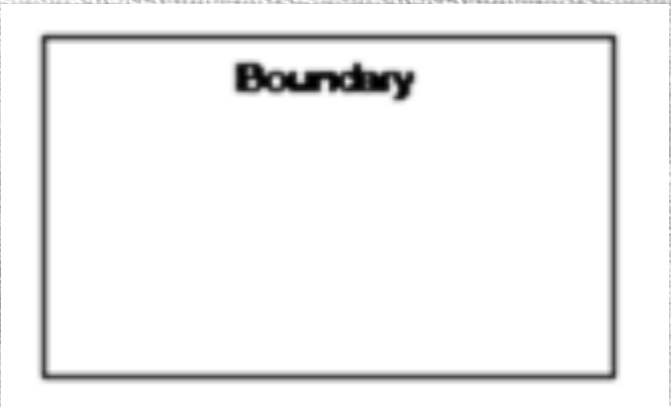
Группировка

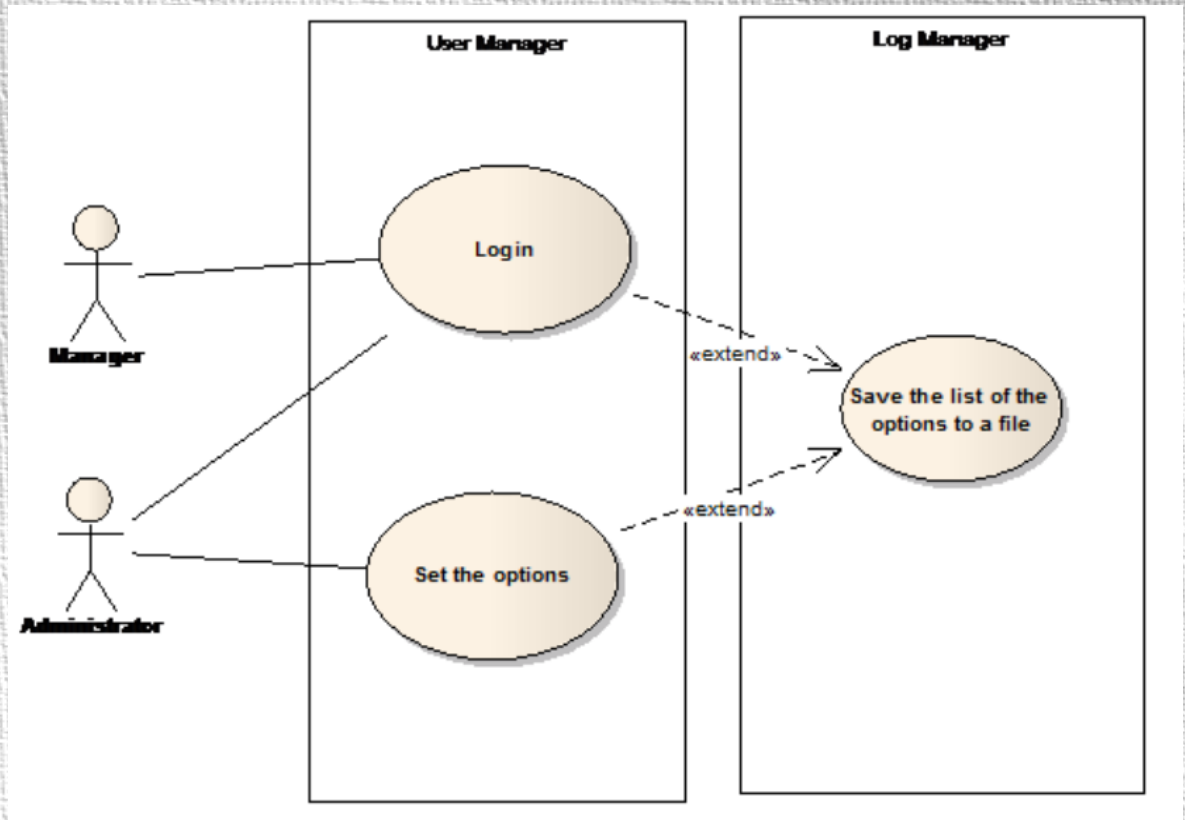
Кроме *Актёров* и *Вариантов использования* на диаграмме также могут находиться следующие элементы:

1. “**Collaboration**” – элемент, предназначенный для визуальной группировки объектов – актёров и вариантов использования – по принципу их совместной работы. Обозначается значком



2. «**Boundary**» - элемент, предназначенный для визуальной группировки объектов – актёров и вариантов использования – по принципу их распределения на подсистемы или компоненты. Обозначается значком





Уровни прецедентов

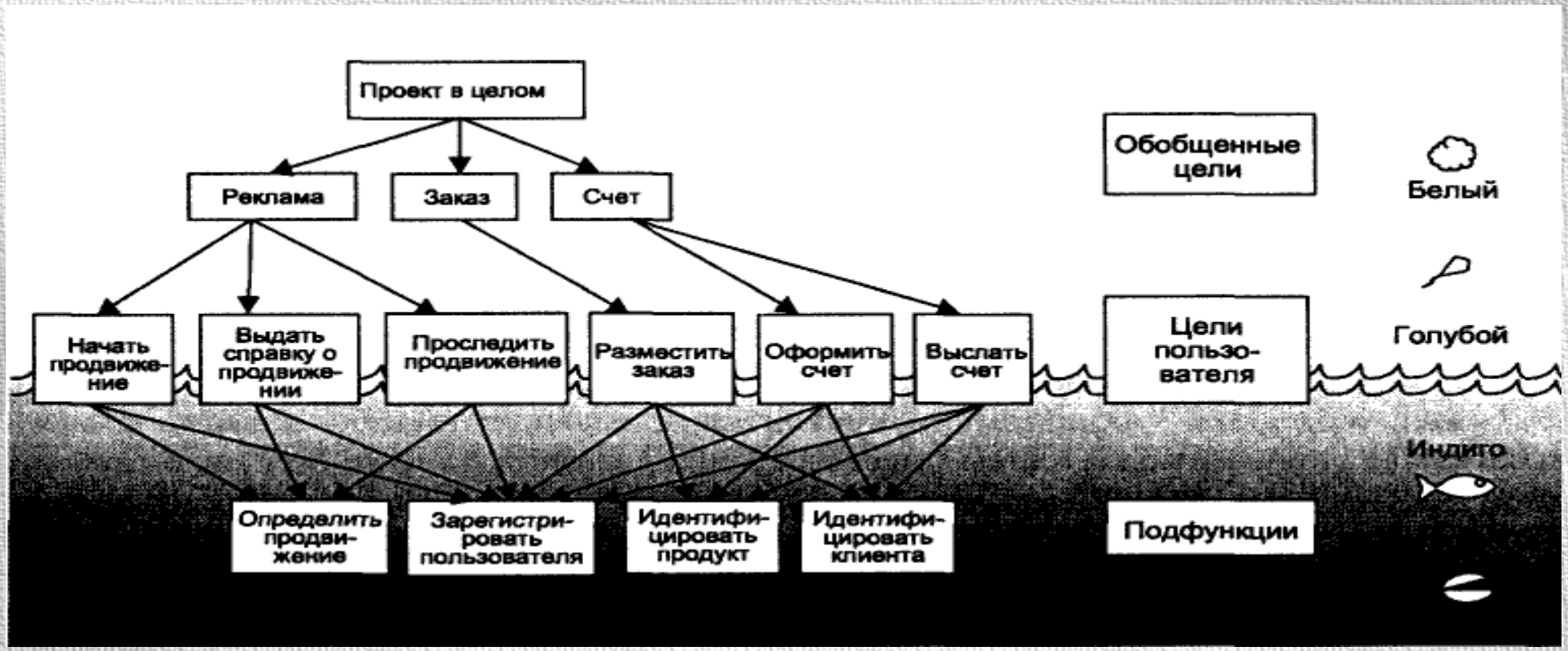
Схема уровней прецедентов

В книге Коберна предлагается схема уровней прецедентов. Базовый прецедент находится на «уровне моря».

- **Прецеденты уровня моря (sea level)** обычно представляют отдельное взаимодействие ведущего актера и системы. Такие прецеденты предоставляют ведущему актеру какой-либо полезный результат и обычно занимают от пары минут до получаса.
- **Прецеденты уровня рыб (fish level).** Прецеденты, которые существуют в системе, только если они включены в прецеденты уровня моря, называются прецедентами уровня рыб.
- **Прецеденты высшего уровня, уровня воздушного змея (kite-level)**, показывают, как прецеденты уровня моря настраиваются на более широкое взаимодействие с бизнес-процессами. Обычно прецеденты уровня воздушного змея являются прецедентами бизнес-процессов, а на уровне моря и на уровне рыб находятся прецеденты системы.

Большинство ваших прецедентов должно принадлежать уровню моря.

Небо находится очень высоко над уровнем моря, а вода опускается ниже уровня моря на большую глубину, но существует только один уровень, где встречаются небо и море, — уровень моря. То же самое справедливо и для целей. Много уровней цели находится над целью пользователя и много под ней, но цели пользователя достаточно важны, чтобы их записать. Поэтому им соответствует уровень моря.



Цель пользователя

Наибольший интерес представляет цель пользователя. Это та цель, которую преследует основное действующее лицо, пытаясь добиться от системы выполнения определенной работы, либо пользователь, работающий с системой. Она соответствует “элементарному бизнес-процессу” в технологии бизнесс-процессов.

- Цель пользователя характеризуется вопросом:

“Уйдет ли основное действующее лицо удовлетворенным, выполнив это?”

- Для служащего этот вопрос звучит так:

“Зависит ли ваша производительность труда от того, сколько вы сегодня сделаете?”

- Цель можно также пропустить через тест “перерыва на кофе”:

“Когда это закончу, сделаю перерыв на кофе”.

- В большинстве случаев цель пользователя проходит тест для одного клиента за один сеанс (2 — 20 мин).

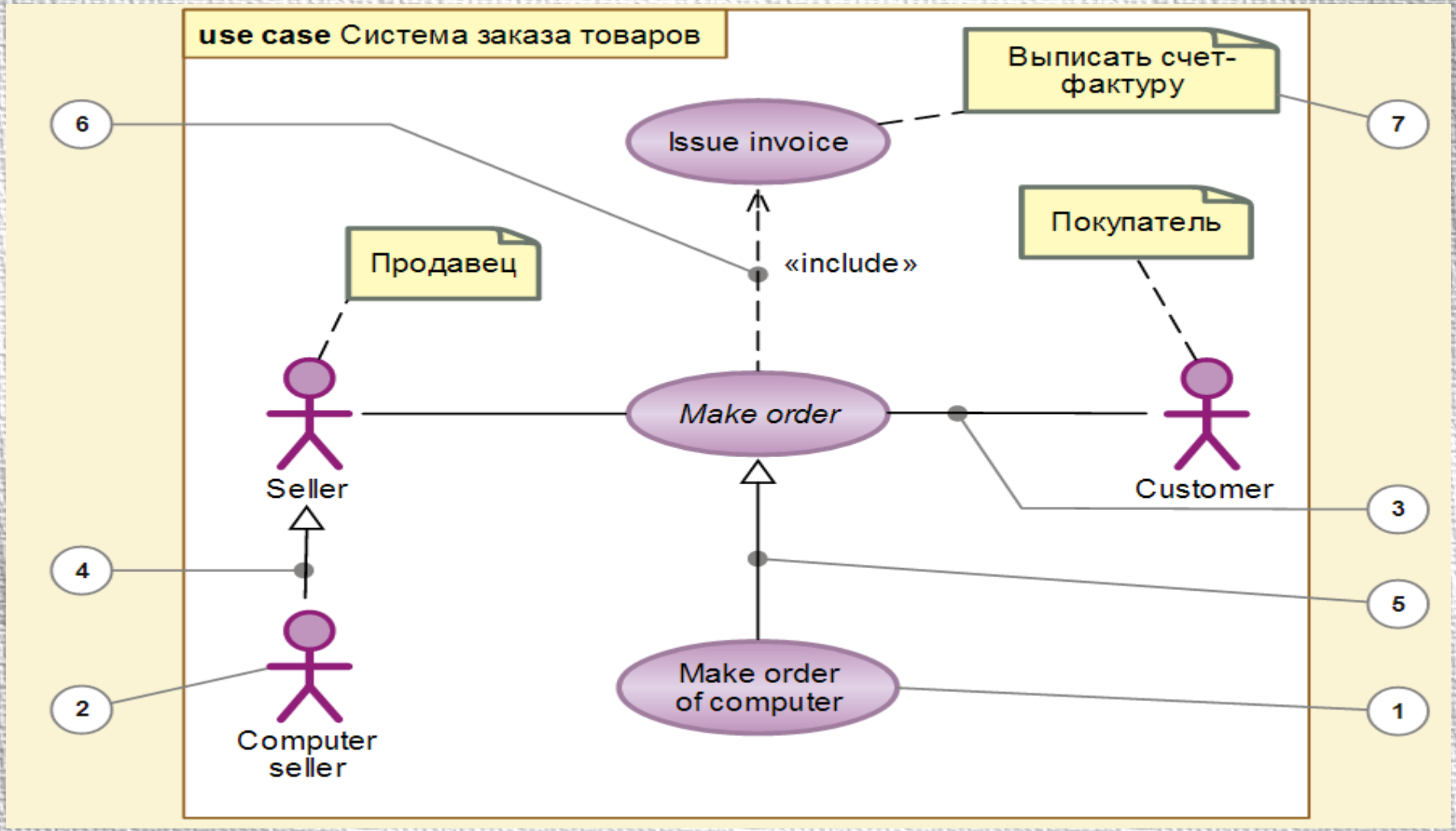
Отношения на диаграммах использования

Диаграмма использования призвана ответить на главный вопрос моделирования: что делает система во внешнем мире?

На диаграмме использования применяются два типа основных сущностей: варианты использования **1** и действующие лица **2**, между которыми устанавливаются следующие основные типы отношений:

- ассоциация между действующим лицом и вариантом использования **3**;
- обобщение между действующими лицами **4**;
- обобщение между вариантами использования **5**;
- зависимости (различных типов) между вариантами использования **6**.

На диаграмме использования, как и на любой другой, могут присутствовать комментарии **7**. Более того, это настоятельно рекомендуется делать для улучшения читаемости диаграмм.

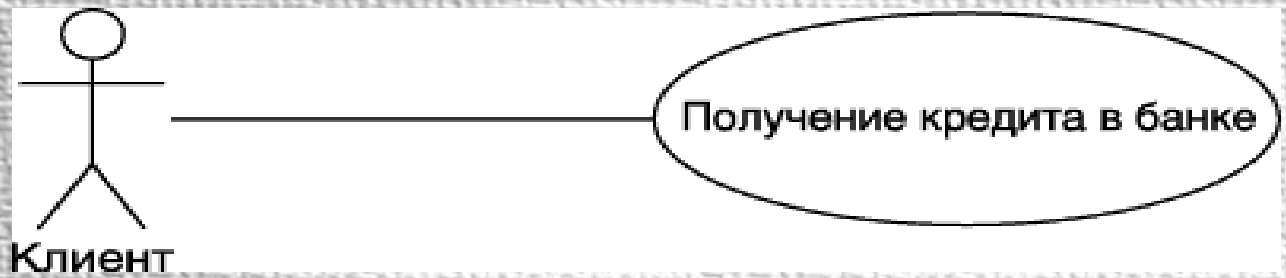

use case Система заказа товаров
Выписать счет-фактуру
Issue invoice
«include»
Покупатель
Продавец
Seller
Make order
Customer
Computer seller
Make order of computer
1
2
3
4
5
6
7

Типы связей между элементами диаграммы

1. Ассоциации (communication)
2. Зависимости
    1. включения (include),
    2. расширения (extend)
3. Обобщения (generalization).
4. Реализации (realization)

Ассоциации (communication)

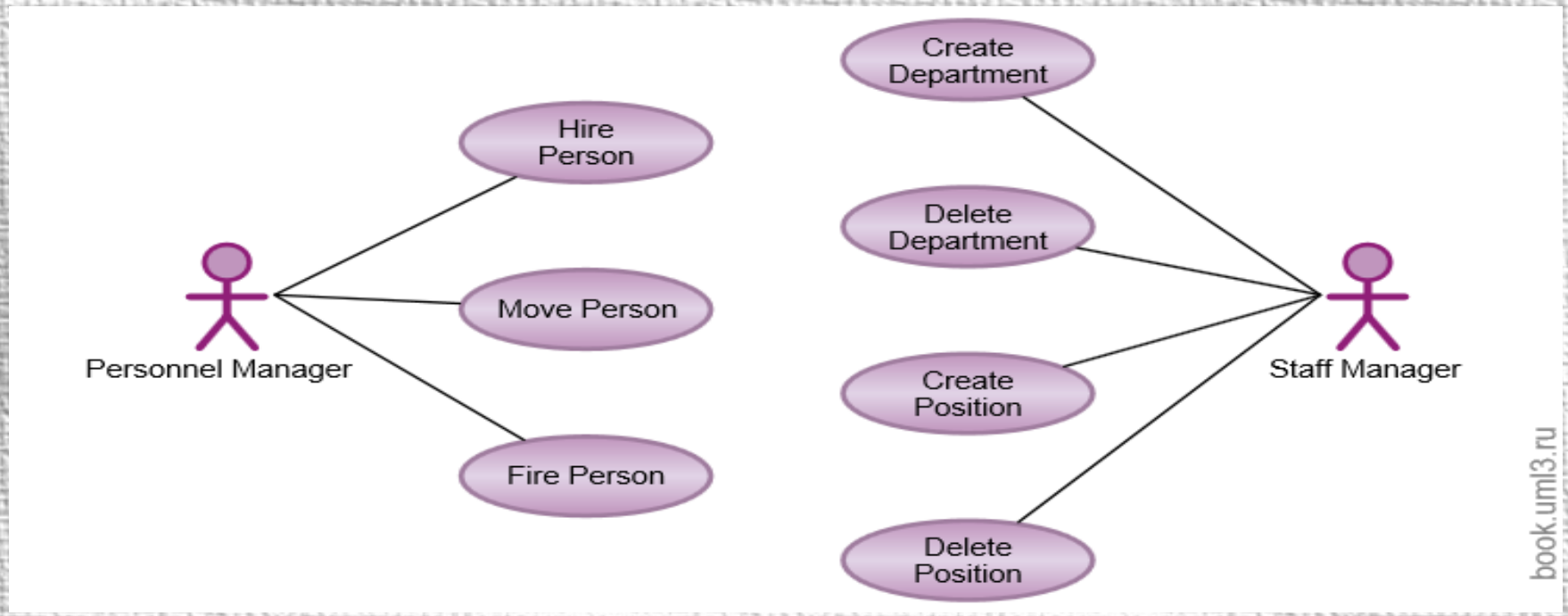
В контексте диаграммы *вариантов использования* отношение *ассоциации* между *актером* и *вариантом использования* может указывать на то, что *актер* инициирует соответствующий *вариант использования*. Такого *актера* называют главным. В других случаях подобная *ассоциация* может указывать на *актера*, которому предоставляется справочная информация о результатах функционирования моделируемой системы. Таких *актеров* часто называют второстепенными.



Пример графического представления отношения ассоциации между актером и вариантом использования

Ассоциации между действующими лицами и вариантами использования

Ассоциация между действующим лицом и вариантом использования показывает, что действующее лицо тем или иным способом взаимодействует (предоставляет исходные данные, получает результат) с вариантом использования.



Задание кратности в ассоциации

- **1** — чтобы указать, что только один экземпляр этой роли может участвовать в каждой связи.
- **1..*** — чтобы указать, что в каждой связи может участвовать один или несколько экземпляров этой роли.
- **0..1** — чтобы указать, что участие не является обязательным.
- * — чтобы указать, что в связи участвует 0 или более экземпляров этой роли.

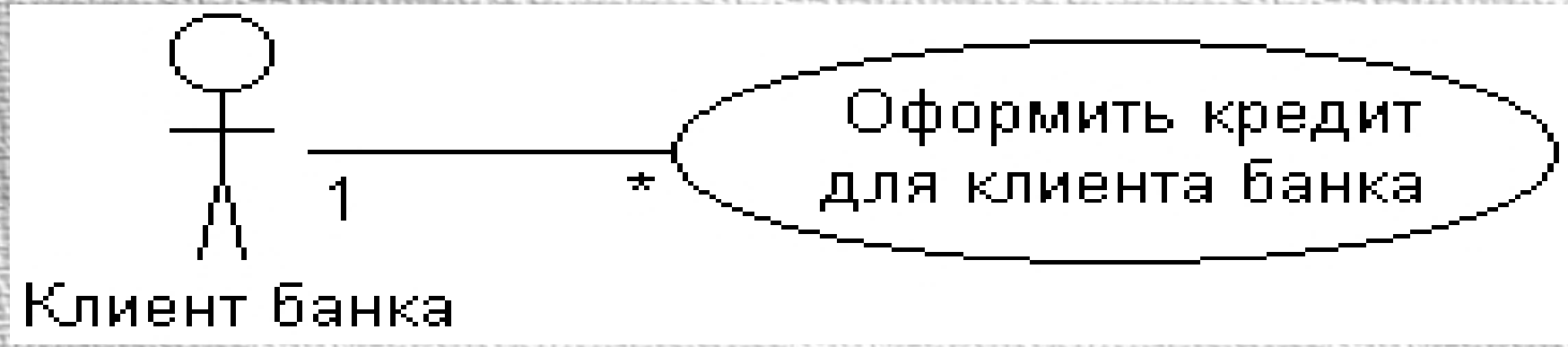
Роль — это элемент на одном конце ассоциации.

Основные формы записи кратности отношения ассоциации

Для диаграмм вариантов использования наиболее распространенными являются четыре основные формы записи кратности отношения ассоциации:

1. Целое неотрицательное число (включая цифру 0). Предназначено для указания кратности, которая является строго фиксированной для элемента соответствующей ассоциации. В этом случае количество экземпляров актеров или вариантов использования, которые могут выступать в качестве элементов отношения ассоциации, в точности равно указанному числу.

Примером этой формы записи кратности ассоциации является указание кратности "1" для актера «Клиент банка». Эта запись означает, что каждый экземпляр варианта использования «Оформить кредит для клиента банка» может иметь в качестве своего элемента единственный экземпляр актера «Клиент банка». Другими словами, при оформлении кредита в банке необходимо иметь в виду, что каждый конкретный кредит оформляется на единственного клиента этого банка.



2. Два целых неотрицательных числа, разделенные двумя точками и записанные в виде: «первое число .. второе число». Данная запись в языке UML соответствует нотации для множества или интервала целых чисел, которая применяется в некоторых языках программирования для обозначения границ массива элементов. Эту запись следует понимать как множество целых неотрицательных чисел, следующих в последовательно возрастающем порядке:

{первое_число, первое_число+1, первое_число+2, ..., второе_число].

Очевидно, что первое число должно быть строго меньше второго числа в арифметическом смысле, при этом первое число может быть равно 0.

Пример такой формы записи кратности ассоциации – «1..5». Эта запись означает, что количество отдельных экземпляров данного компонента, которые могут выступать в качестве элементов данной ассоциации, равно некоторому заранее неизвестному числу из множества целых чисел {1, 2, 3, 4, 5}. Эта ситуация может иметь место, например, в случае рассмотрения в качестве актера – клиента банка, а в качестве варианта использования – процедуру открытия счета в банке. При этом количество отдельных счетов каждого клиента в данном банке, исходя из некоторых дополнительных соображений, может быть не больше 5. Эти дополнительные соображения как раз и являются внешними требованиями по отношению к проектируемой системе и определяются ее заказчиком на начальных этапах ООАП.

3. Два символа, разделенные двумя точками. При этом первый из них является целым неотрицательным числом или 0, а второй – специальным символом "*". Здесь символ "*"обозначает произвольное конечное целое неотрицательное число, значение которого неизвестно на момент задания соответствующего отношения ассоциации.

Пример такой формы записи кратности ассоциации – «2..*». Запись означает, что количество отдельных экземпляров данного компонента, которые могут выступать в качестве элементов данной ассоциации, равно некоторому заранее неизвестному числу из подмножества натуральных чисел: {2, 3, 4}.

4. Единственный символ "*", который является сокращением записи интервала «0..*». В этом случае количество отдельных экземпляров данного компонента отношения ассоциации может быть любым целым неотрицательным числом. При этом 0 означает, что для некоторых экземпляров соответствующего компонента данное отношение ассоциации может вовсе не иметь места.

В качестве примера этой записи можно привести кратность отношения ассоциации для варианта использования «Оформить кредит для клиента банка». Здесь кратность "*" означает, что каждый отдельный клиент банка может оформить для себя несколько кредитов, при этом их общее число заранее неизвестно и ничем не ограничивается. При этом некоторые клиенты могут совсем не иметь оформленных на свое имя кредитов (вариант значения 0).

Если кратность обоих окончаний ассоциации равна 1, они не отображаются на схеме вариантов использования.

Структуризация прецедента

Для снижения сложности начальная модель вариантов использования может быть подвергнута структуризации, в ходе которой выделяются вспомогательные варианты использования (*включаемые или расширяющие*).

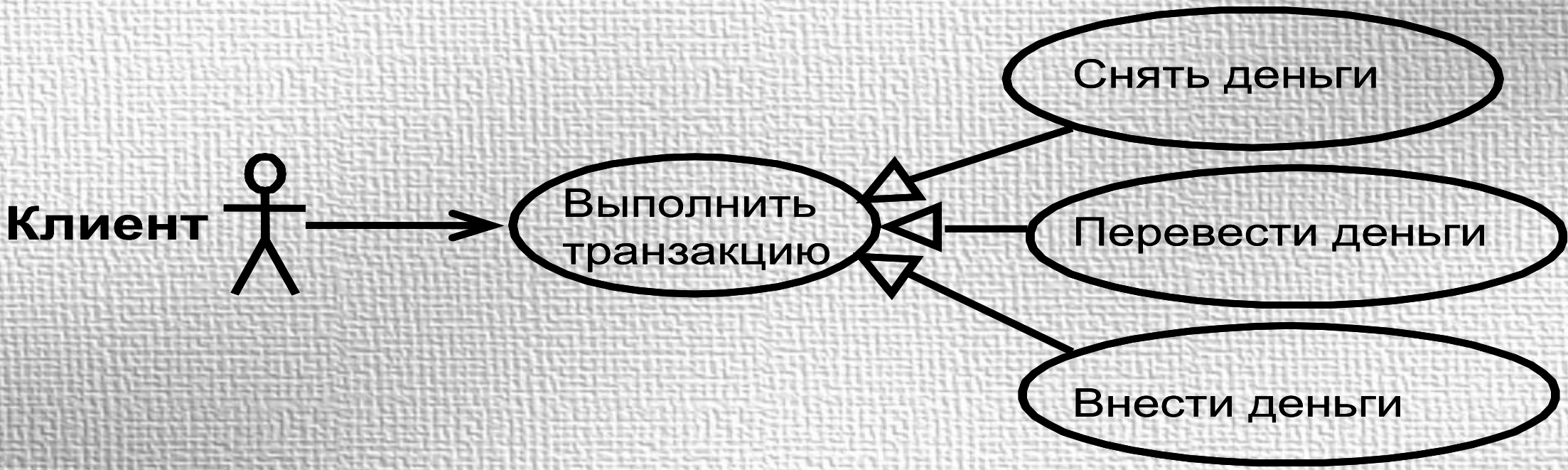
Сложность снижается за счет вынесения части описаний из основного варианта использования во **включаемый** (который может быть включен в несколько ВИ, что еще больше упрощает модель) или **расширяющий**.

Обобщения (generalization)

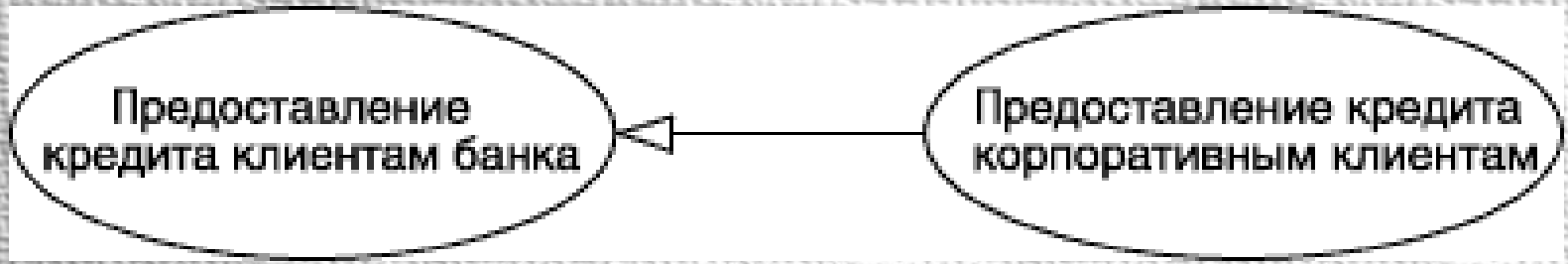
Одним из инструментов структуризации является обобщение вариантов использования.

В ходе моделирования вариантов использования может возникнуть ситуация, когда обнаруживается, что несколько ВИ демонстрируют очень схожее поведение. В этом случае можно использовать обобщение ВИ. Такой прием следует использовать крайне осторожно и только в том случае, если он упрощает модель.

При обобщении в базовый ВИ выносится описание общее для всех наследников, дочерние ВИ специализируют описание базового, они участвуют во всех связях расширения и/или включения базового.


Клиент
Выполнить транзакцию
Снять деньги
Перевести деньги
Внести деньги

Отношение ***обобщения*** между *вариантами использования* применяется в том случае, когда необходимо отметить, что дочерние *варианты использования* обладают всеми особенностями поведения родительских *вариантов*. При этом дочерние *варианты использования* участвуют во всех отношениях родительских *вариантов*. В свою очередь, дочерние *варианты* могут наделяться новыми свойствами поведения, которые отсутствуют у родительских *вариантов использования*, а также уточнять или модифицировать наследуемые от них свойства поведения.



Пример графического изображения отношения обобщения между вариантами использования

Отношение обобщения связывает более специализированный и более общий варианты использования. Прямой потомок наследует атрибуты, операции и последовательности поведения своего прямого предка, а также может добавлять к ним свои собственные атрибуты и операции. ВИ-потомок добавляет к поведению ВИ-предка инкрементное поведение. Для этого он вставляет в определенные точки последовательности поведения ВИ-предка дополнительные последовательности действий. Кроме этого, он также может изменять некоторые унаследованные операции и последовательности. Делать это нужно с большой осторожностью, так как при любом изменении должна сохраниться суть (intent) прямого предка. Изменить унаследованное поведение можно также при помощи отношений включения или расширения.

Пример

В банковской системе может существовать вариант использования ***Проверка пользователя***, который отвечает за идентификацию клиента. У него могут быть два *специализированных дочерних варианта использования* – ***Проверка пароля*** и ***Сканирование сетчатки***, каждый из которых ведет себя так же, как ***Проверка пользователя***, и может быть применен в любом месте, где появляется последний. При этом оба потомка добавляют свое собственное поведение:

- первый проверяет текстовый пароль,
- а второй – уникальный рисунок сетчатки глаза пользователя.

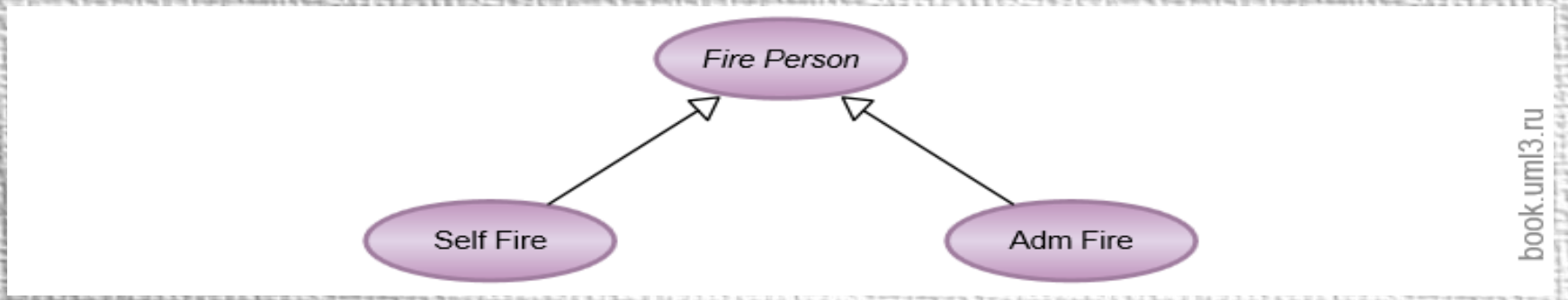
Абстрактный обобщающий вариант использования

Обобщающий вариант использования может быть абстрактным классификатором. Например, такой важный для сотрудника вариант использования, как увольнение, на самом деле является абстракцией: в каждом конкретном случае имеет место ровно один из возможных частных случаев увольнения, которые приводят к одному и тому же результату с точки зрения менеджера персонала, но весьма различны с точки зрения сотрудника.

Пример обобщения для ОК

ИЗМЕНЕНИЯ В ТЕХНИЧЕСКОМ ЗАДАНИИ. Система должна поддерживать два способа увольнения сотрудника:
1.по инициативе администрации
2.по собственному желанию.

Данное обстоятельство можно отразить в модели так, как показано на следующем фрагменте диаграммы.



Обобщенный абстрактный (имя написано курсивом) вариант использования *Fire Person* имеет две специализации, которые соответствуют увольнению работника по собственному желанию Self Fire и по инициативе администрации Adm Fire.

Включения (include)

Зависимость между вариантами использования

Зависимость между вариантами использования **показывает, что один вариант использования зависит от другого варианта использования.**

В UML имеются два стандартных стереотипа зависимости между вариантами использования, которые в некотором смысле двойственны друг другу:

- **«include»** — показывает, что в **каждый** сценарий зависимого ВИ в определенном месте вставляется сценарий независимого ВИ как подпоследовательность действий.
- **«extend»** — показывает, что в **некоторый** сценарий независимого варианта использования **может быть** в определенном месте вставлен в качестве подпоследовательности действий сценарий зависимого варианта использования.

Иногда в вариантах использования присутствует многократное описание одних и тех же действий.

Рассмотрим, к примеру, систему «Кадры»

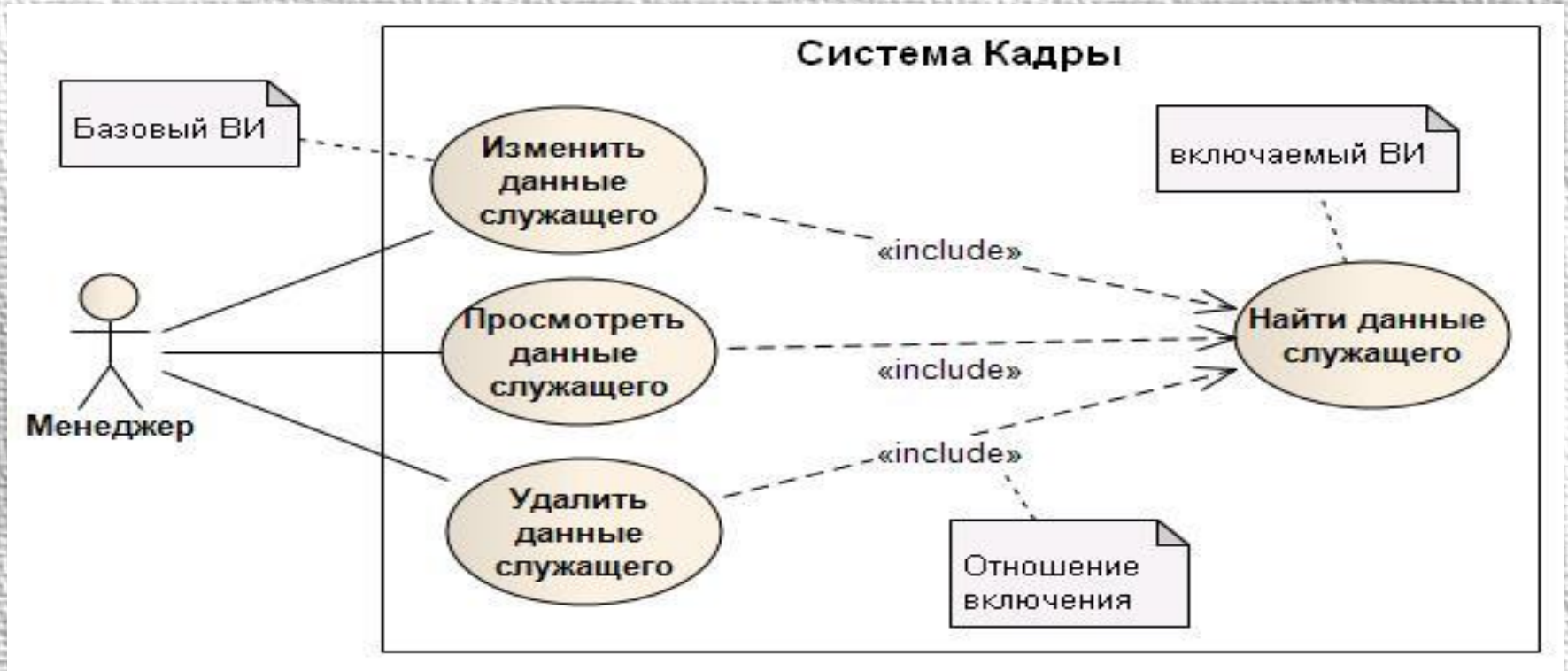
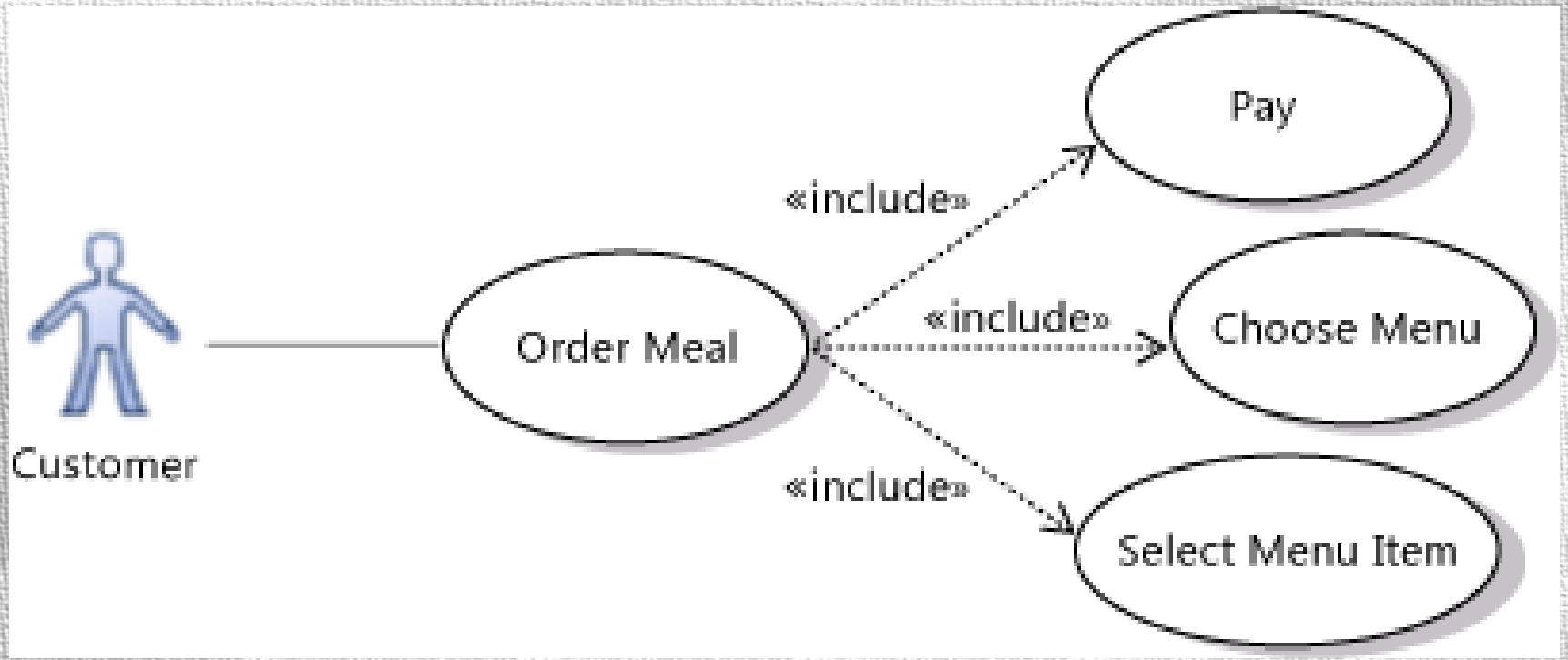


Схема вариантов использования эффективна для отображения вложенности небольших действий в более крупное действие, но не показывает поток работы. Например:



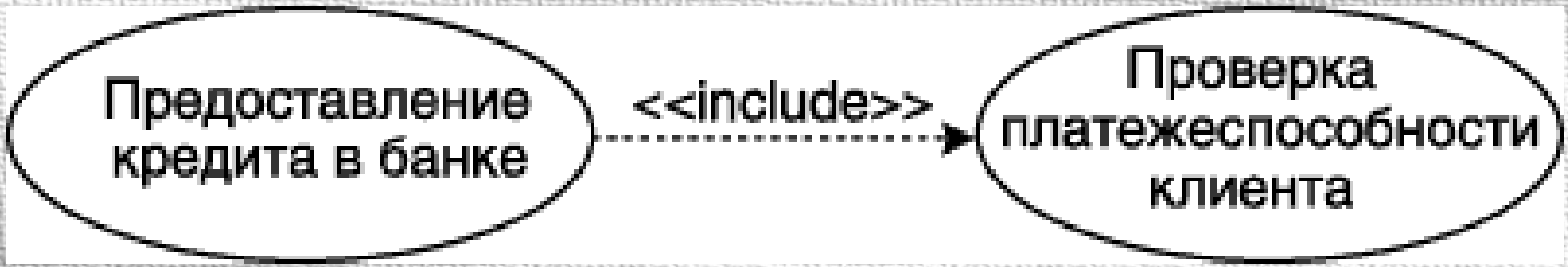
Практически любое действие системы начинается с получения данных о конкретном служащем. Если бы эту последовательность событий приходилось писать каждый раз, когда необходимы данные служащего, варианты использования имели бы повторяющиеся части. В этом нет особой беды до тех пор, пока вы не решаете по какой-то своей причине изменить эту повторяющуюся часть. Очевидно, придется внести изменения во все варианты использования, где встречается эта повторяющаяся часть.

Отношение включения («include»), устанавливаемое между вариантами использования, позволяет включить поведения одного варианта использования в поток другого варианта использования.

Включающий вариант использования назовем **базовым**, а тот, который включается, **включаемый**. Включаемый вариант использования предоставляет поведение своему базовому варианту использования.

В базовом ВИ необходимо точно указать место, где должно быть включено поведение включаемого ВИ. Синтаксис и семантика отношения включения немного напоминает вызов процедуры или функции.

Отношение включения имеет простую семантику. Базовый ВИ выполняется до момента включения. Затем выполнение переходит на включаемый ВИ. После завершения включаемого ВИ управление вновь возвращается в базовый ВИ.



Базовый ВИ (клиент)

Включаемый ВИ (поставщик)

Связь включения между вариантами использования означает, что базовый ВИ в определенном месте явно включает в себя поведение некоторого другого. Включенный ВИ не существует отдельно: он является экземпляром только внутри базового, который его содержит. Можно считать, что базовый ВИ заимствует поведение включаемого.

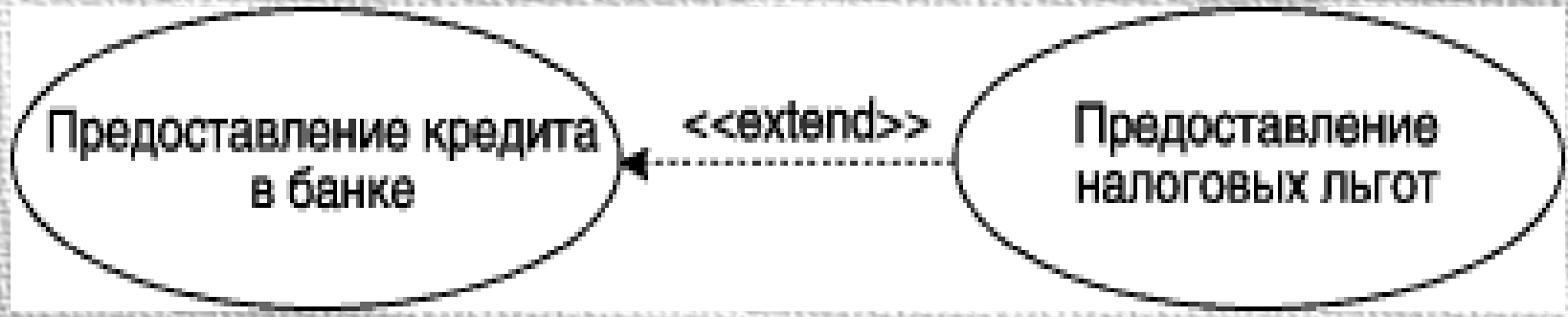
Благодаря наличию связей включения удается избежать многократного описания одного и того же потока событий, поскольку общее поведение можно представить в виде отдельного ВИ, включаемого в базовые. Связь включения являет собой пример делегирования, когда ряд обязанностей системы описывается в одном месте (во включаемом варианте использования), а остальные варианты использования при необходимости вводят эти обязанности в свой набор.

Связь включения изображается как зависимость со стереотипом **include**. Чтобы показать то место в потоке событий базового ВИ, куда включается поведение другого варианта использования, просто напишите *include* с последующим именем включаемого ВИ, как это сделано, например,в потоке Track order (Отследить заказ):

- ***Track order:***
- *Получить и проверить номер заказа;*
- ***include 'Validate user'*** *;*
- *для каждой части заказа запросить состояние;*
- *сообщить общее итоговое состояние пользователю.*

Расширения (extend)

Отношение ***расширения (extend)*** определяет взаимосвязь базового *варианта использования* с другим *вариантом использования*, функциональное поведение которого задействуется базовым не всегда, а только при выполнении дополнительных условий.



Базовый ВИ **Расширяющий ВИ**

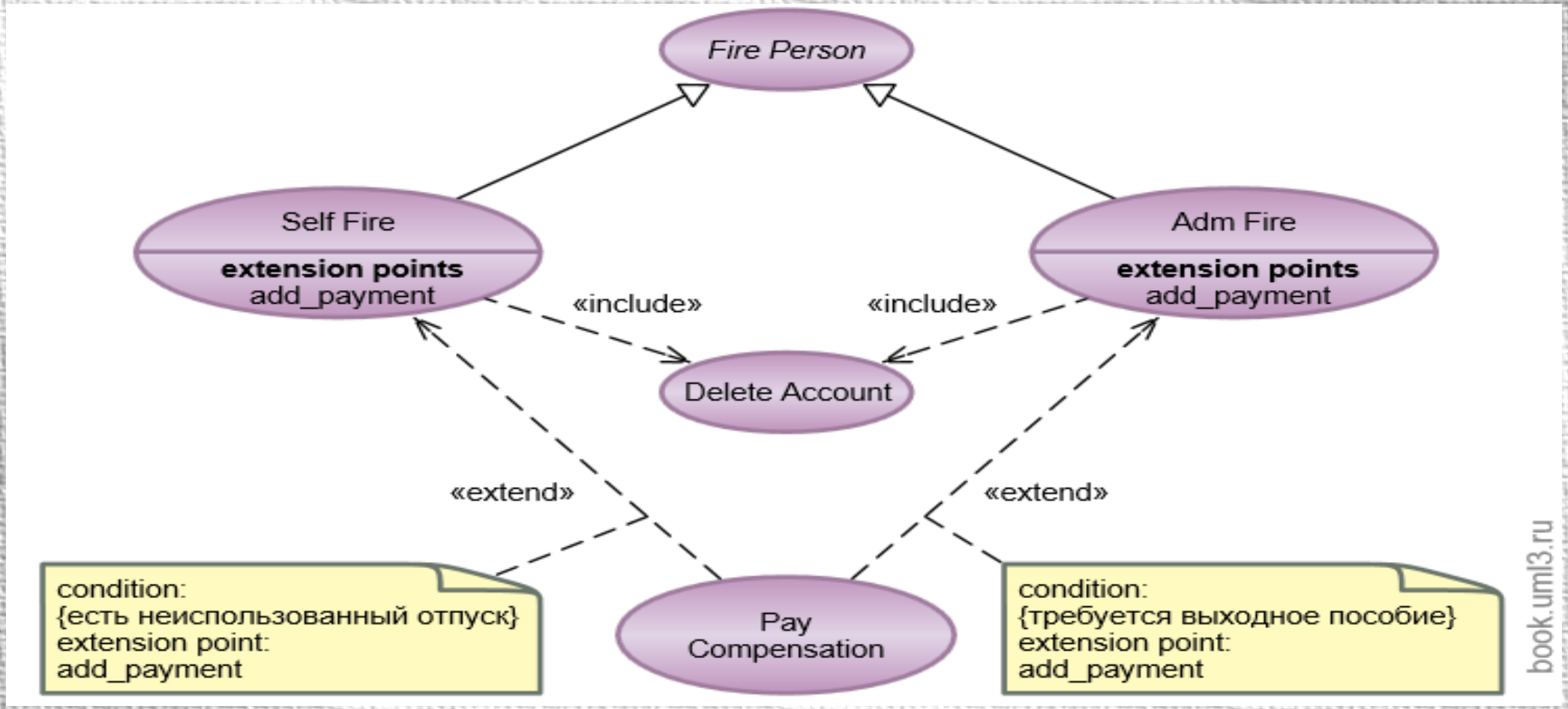
Один из вариантов использования может быть расширением для нескольких базовых вариантов, а также иметь в качестве собственных расширений несколько других вариантов.

В отношении «extend» любопытно то, что базовый ВИ ничего не знает о расширяющих ВИ, он просто предоставляет для них точки входа. **Базовый ВИ абсолютно полон и без расширений. Это существенно отличает «extend» от отношения «include», где базовые ВИ остаются неполными без включаемых ВИ**. Более того, точки расширения на самом деле не вводятся в поток событий базового ВИ; они накладываются поверх потока.

ИЗМЕНЕНИЯ В ТЕХНИЧЕСКОМ ЗАДАНИИ. При увольнении сотрудника должна быть осуществлена выплата денежной компенсации за неиспользованный отпуск. В случае вынужденного сокращения возможна выплата выходного пособия. Учетная запись сотрудника при увольнении должна быть заблокирована.

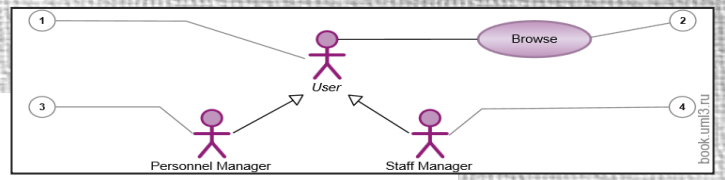
Увольнение сотрудника – один из самых сложных вариантов использования в реальных системах управления персоналом. Итак, нам известно, что при увольнении сотрудника следует в целях информационной безопасности заблокировать (или удалить) учетную запись пользователя в локальной сети организации. *Причем это действие должны быть выполнено в любом сценарии увольнения*. С другой стороны, как сказано в техническом задании, при выполнении определенных условий, при увольнении иногда выплачивается некоторая денежная компенсация (за неиспользованный отпуск, выходное пособие при сокращении и т.д.). Все это примеры последовательностей действий (т.е. вариантов использования), которые вполне могут быть востребованы как при увольнении, так и помимо него.

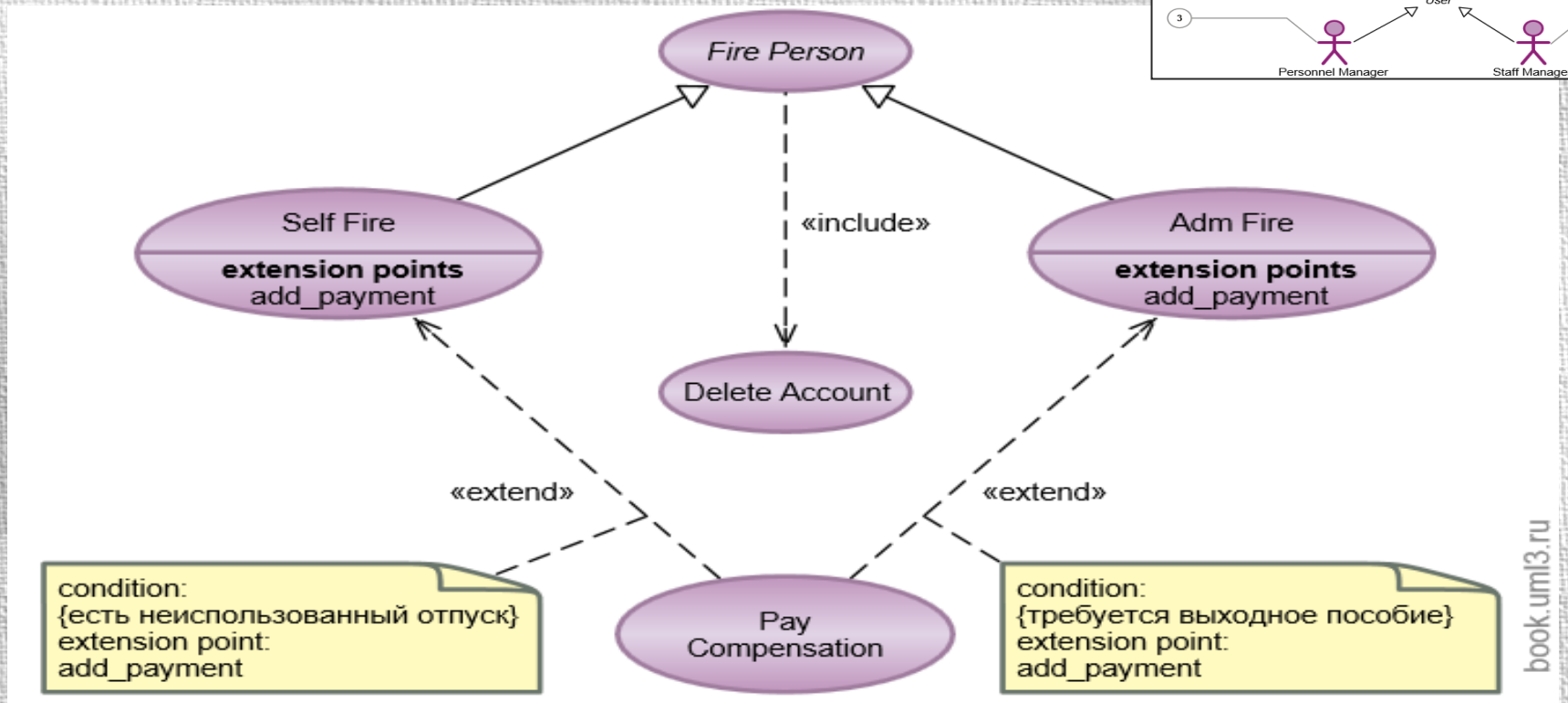
Зависимости между вариантами использования



Комбинация отношений обобщения и зависимости

Последний пример можно отобразить еще компактнее, комбинируя возможности отношений обобщения и зависимости, подобно тому, как скомбинированы возможности отношений обобщения и ассоциации



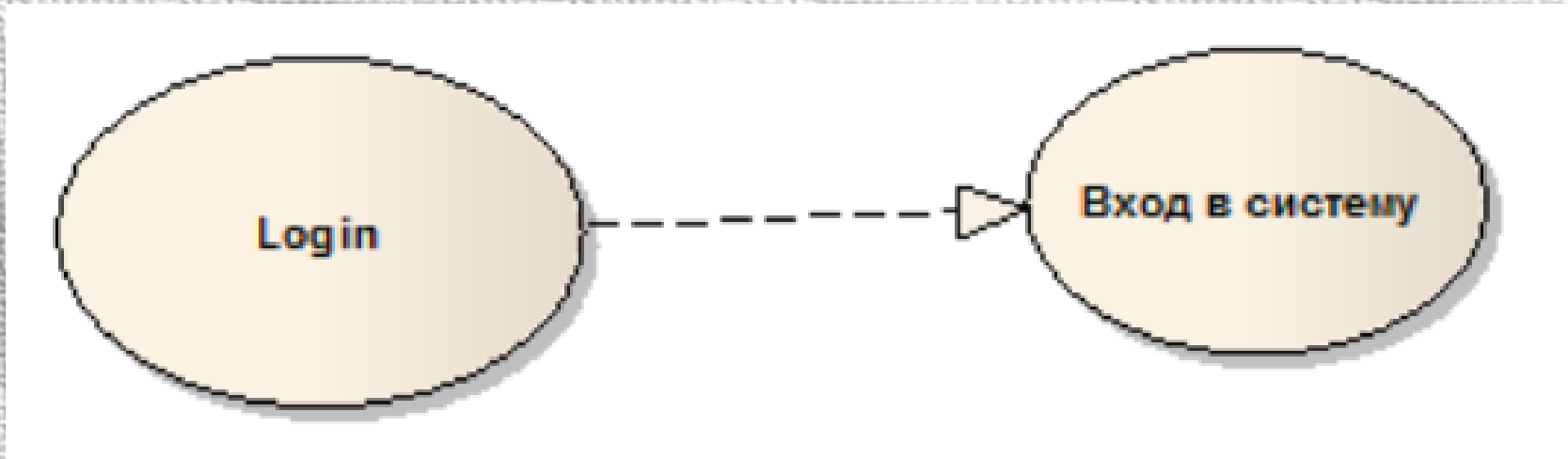


Реализации (realization)

«Реализация» означает, что один вариант использования является реализацией другого варианта использования.

Например, если один из них описан в терминах бизнес-процессов, а другой – в терминах проектируемой системы.

Например:



Кроме того, варианты использования могут быть связаны отношением «Реализация» с требованиями к системе и с классами. При наличии таких связей есть возможность проследить в каких классах реализованы требования и какие классы могут быть затронуты при изменении требований или вариантов использования. Например:

Правила разработки

- Каждый вариант использования должен быть инициирован одним основным действующим лицом. От этого лица к нему должна идти ассоциация.
- Не моделируйте ассоциации между действующими лицами. По определению действующие лица находятся вне сферы действия системы. Это означает, что связи между ними также не относятся к её компетенции.

- Не соединяйте ассоциацией два варианта использования непосредственно. Диаграммы описывают только, какие варианты использования доступны системе, а не порядок их выполнения. На диаграммах варианты использования могут быть связаны либо зависимостями (связями включения или расширения) или обобщением (наследованием). Для отображения порядка выполнения вариантов использования применяют диаграммы деятельности.
- Избегайте многочисленных и запутанных связей между действующими лицами и вариантами использования. Например, не следует делать двойные вложения вариантов использования, объединять их в иерархии наследования с большим количеством уровней.

Порядок построения диаграммы

1. Выделить группы действующих лиц (работающих с системой по-разному, часто из-за различных прав доступа);
2. Идентифицировать как можно больше вариантов использования (процессов, которые могут выполнять пользователи). При этом не следует делить процессы слишком мелко, нужно выбирать лишь те, которые дадут пользователю значимый результат. Например, кассир может “продать товар” (это будет являться прецедентом), однако “ввод штрих-кода товара для получения цены” самостоятельным прецедентом не является;

3. Дополнить прецеденты словесным описанием (сценарием):
   - для каждого прецедента создать разделы: “главная последовательность” и “альтернативные последовательности”;
   - при составлении сценария нужно упорно задавать заказчику вопросы “что происходит?”, “что дальше?”, “что еще может происходить?” и записывать ответы на них.

Связывание ВИ с документами и схемами

Можно связать вариант использования со следующими схемами и документами:

- **диаграммой последовательностей**, на которой показано, как взаимодействия между пользователями и системой или ее основными компонентами позволяют достигать целей варианта использования;
- **диаграммой деятельности**, подробно иллюстрирующей действия, которые совершаются пользователями, системой или ее основными компонентами при выполнении варианта использования;

- **страницей или абзацем OneNote** с подробным описанием варианта использования;
- **документом Word или презентацией PowerPoint** с подробным описанием варианта использования. Эти документы можно хранить в решении или другом расположении, доступ к которому имеют члены команды, например на сайте SharePoint.

Visual Studio. Чтобы связать вариант использования с документом, нужно создать артефакт на схеме вариантов использования и соединить вариант использования с артефактом. В свойствах артефакта нужно задать путь к файлу другой схемы или документа. Дважды щелкнув артефакт, можно открыть другую схему или документ. Каждый вариант использования можно соединить с неограниченным числом артефактов. Также можно связывать артефакты с другими видами элементов на схеме вариантов использования.

Модель завершена?

Модель вариантов использования можно считать завершенной, если есть утвердительные ответы на следующие вопросы:

- Можно ли на основании модели сформировать четкое представление о функциях системы и их взаимосвязях?
- Присутствует ли каждое функциональное требование хотя бы в одном варианте использования? Если требование не нашло отражение в варианте использования, оно не будет реализовано.
- Учли ли вы, как с системой будет работать каждое заинтересованное лицо?
- Какую информацию каждое заинтересованное лицо будет передавать системе?
- Учли ли вы все внешние системы, с которыми будет взаимодействовать данная?

Дополнительные
примеры

Типичные приемы моделирования

Например, система розничной торговли должна взаимодействовать с заказчиками, которые размещают заказы и отслеживают их исполнение. Сама система, в свою очередь, отгружает заказанные товары и выставляет счета клиентам к оплате. Как видно из рисунка, поведение такой системы можно смоделировать, объявив несколько вариантов использования:

- Place order (Разместить заказ),
- Track order (Отследить заказ),
- Ship order (Отгрузить заказ) и
- Bill customer (Выставить счет).

Можно выделить общее поведение **Validate customer** (Проверить клиента) и варианты типа Ship partial order (Отгрузить частично выполненный заказ). Для каждого из этих вариантов использования следует предусмотреть спецификацию поведения, выраженную текстом, автоматом или взаимодействием.

Обобщение, включение и расширение

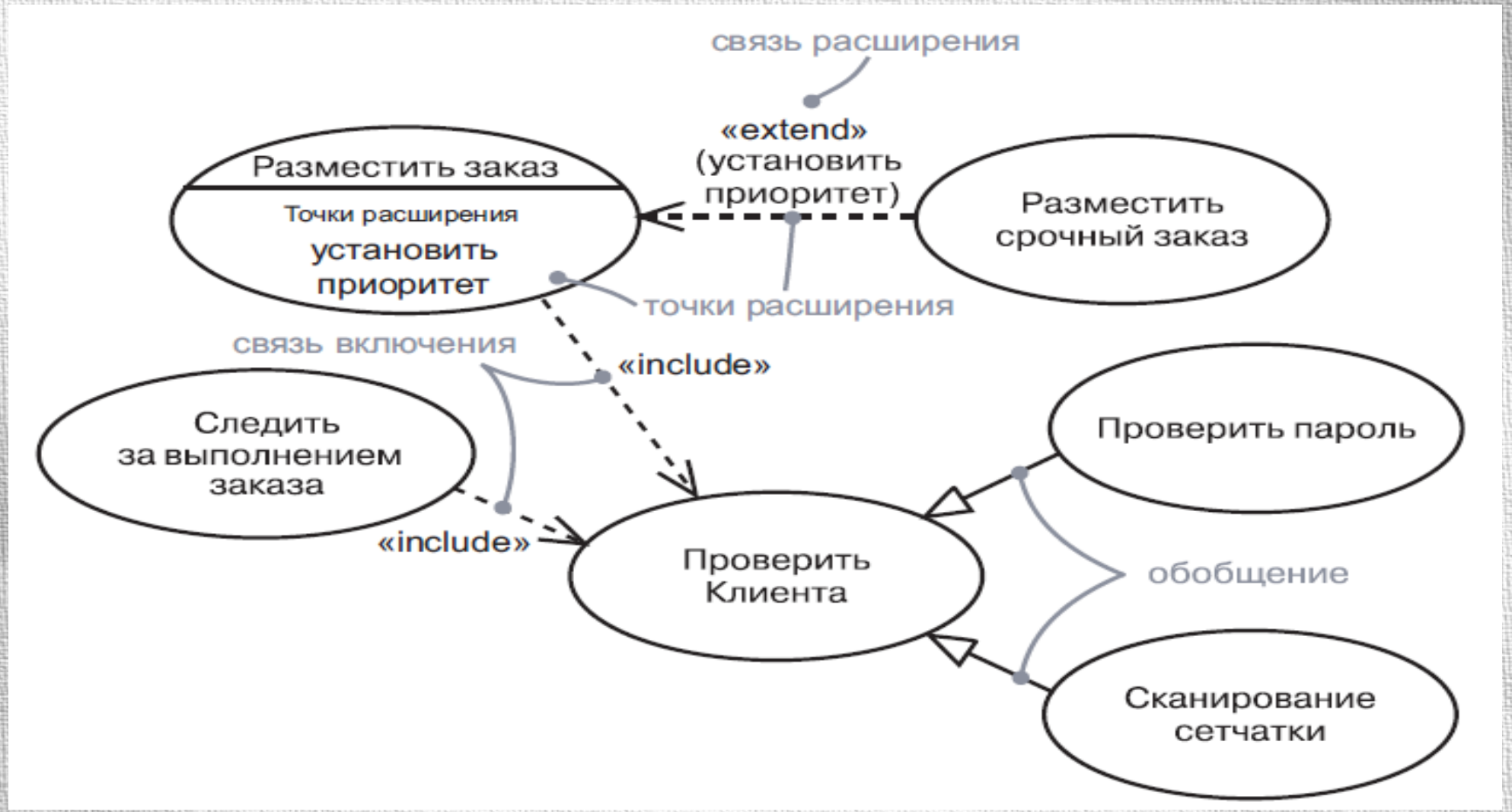
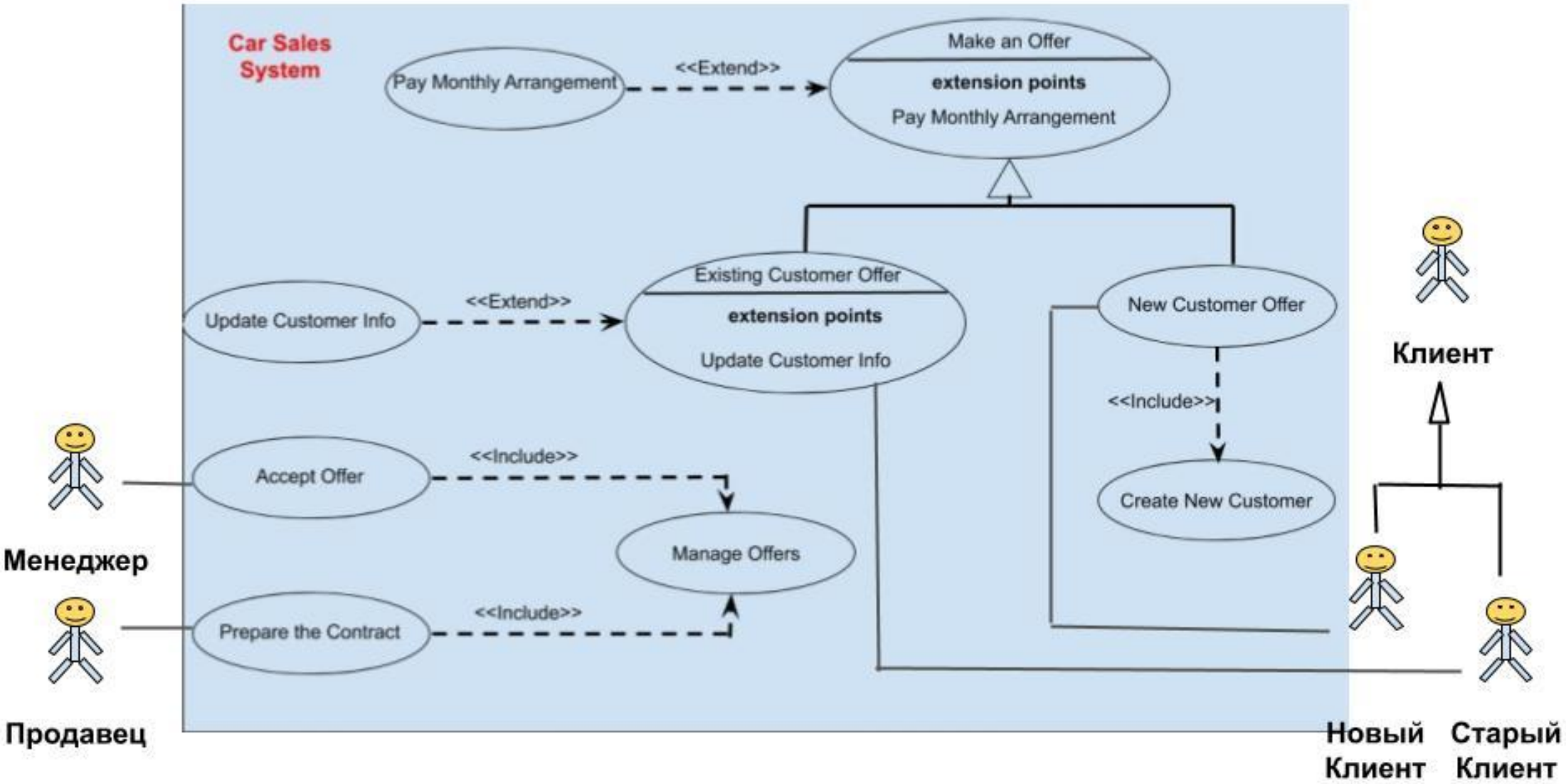
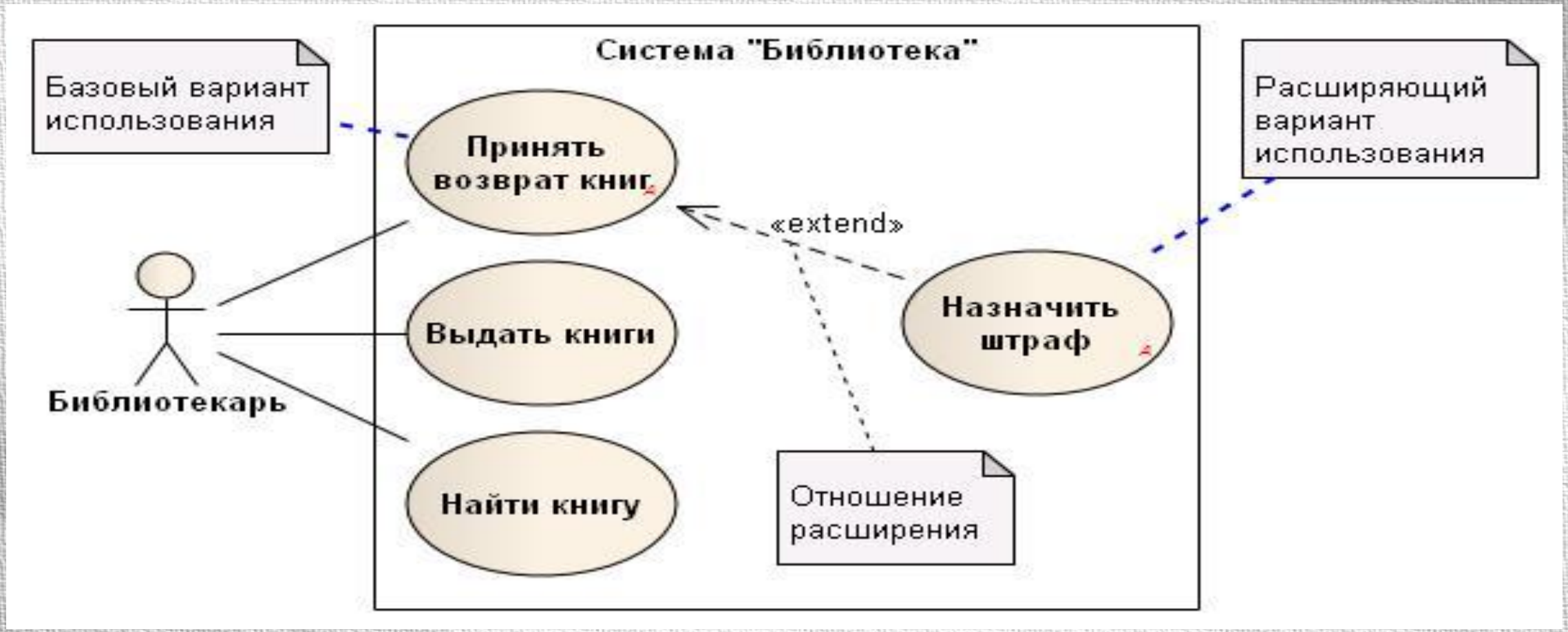
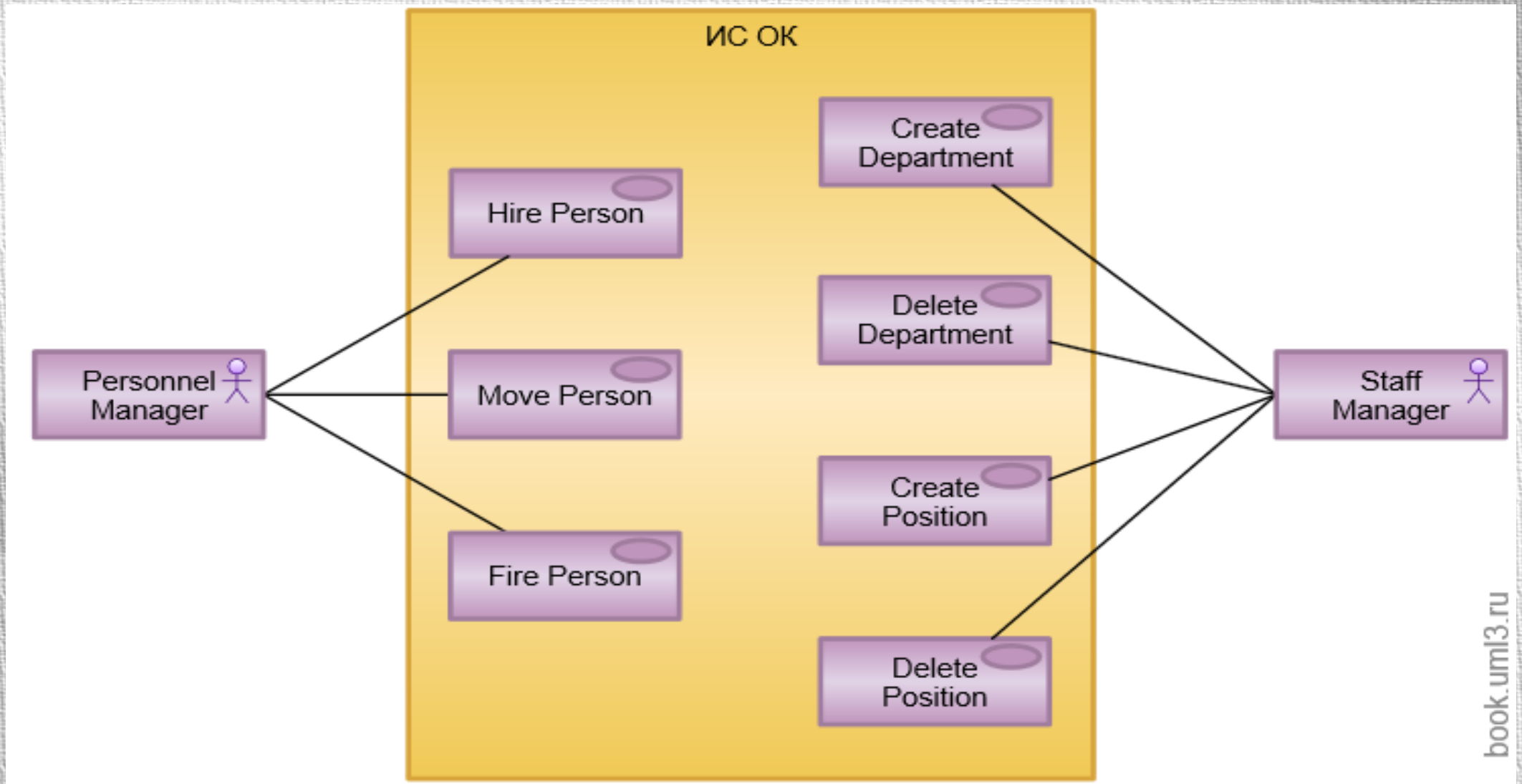


Диаграмма Прецедентов

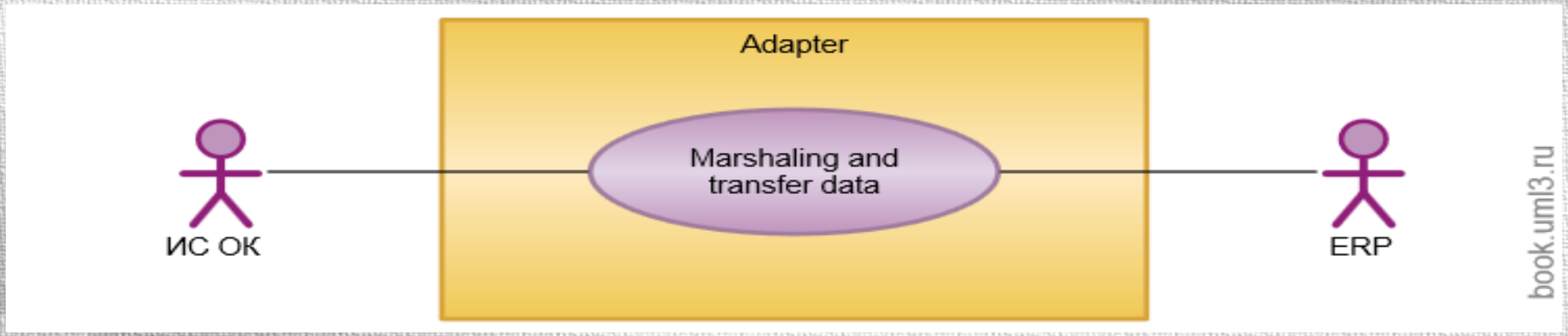


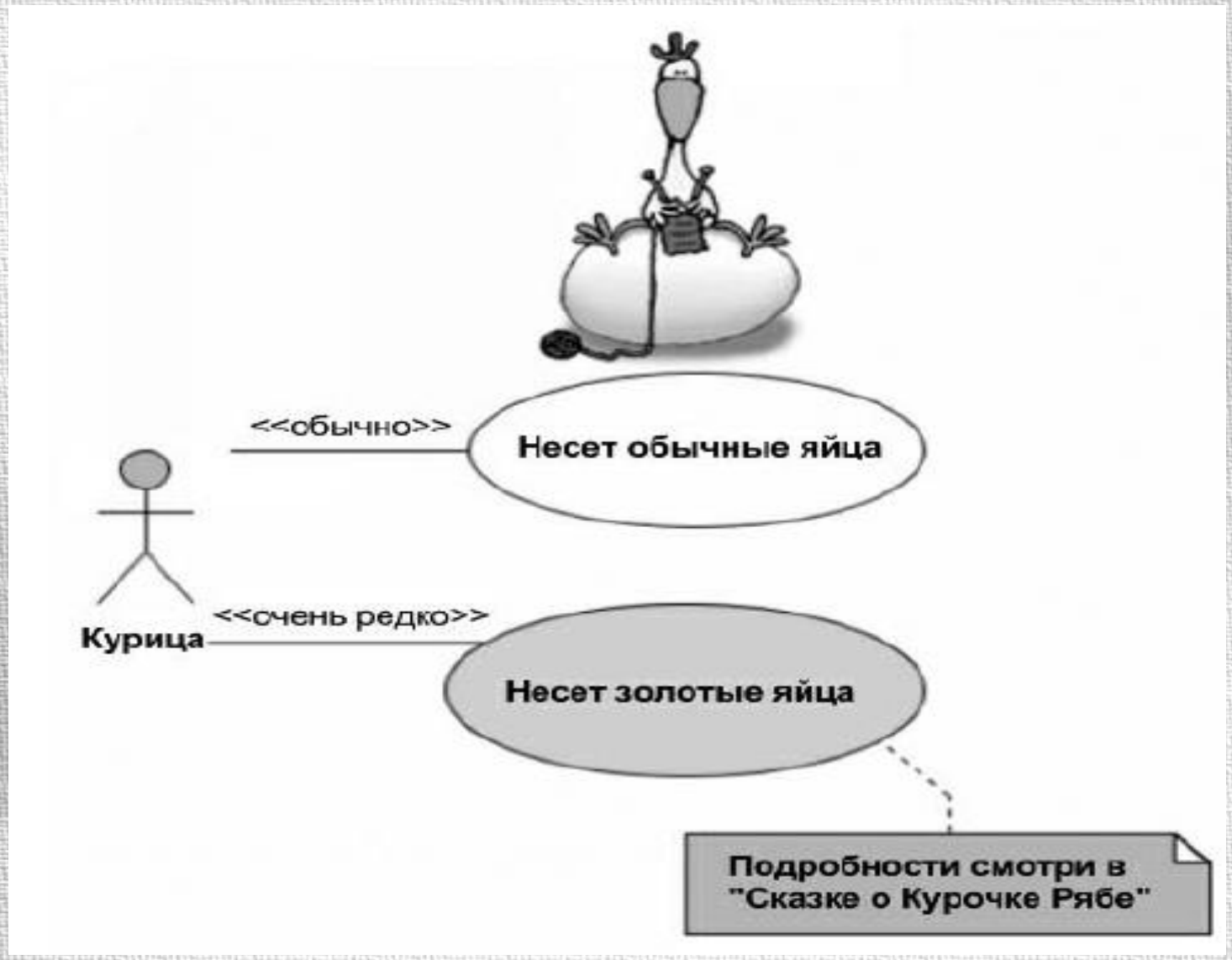

Система "Библиотека"
Базовый вариант использования
Принять возврат книг
Выдать книги
Найти книгу
«extend»
Назначить штраф
Расширяющий вариант использования
Библиотекарь
Отношение расширения

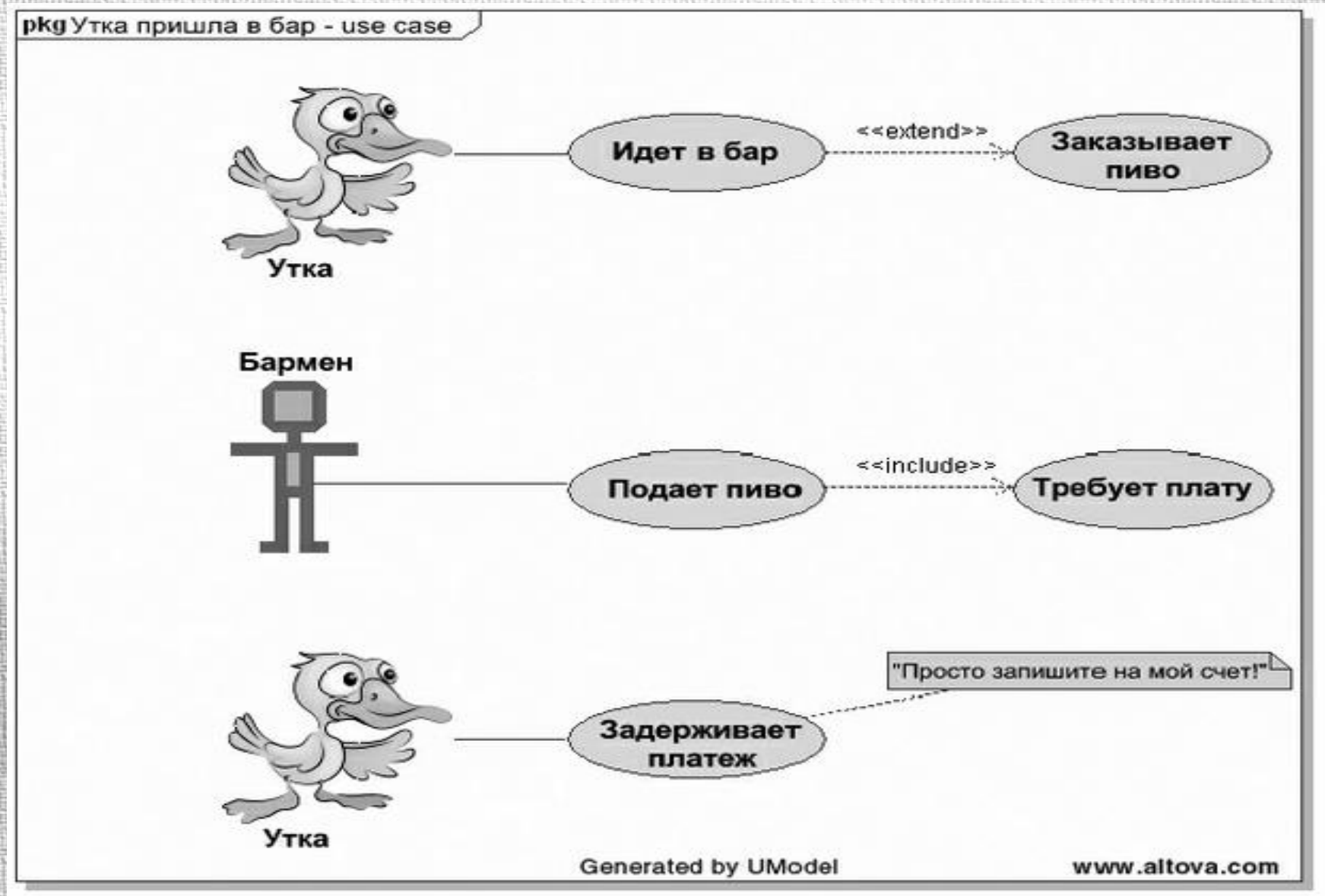
Границы системы



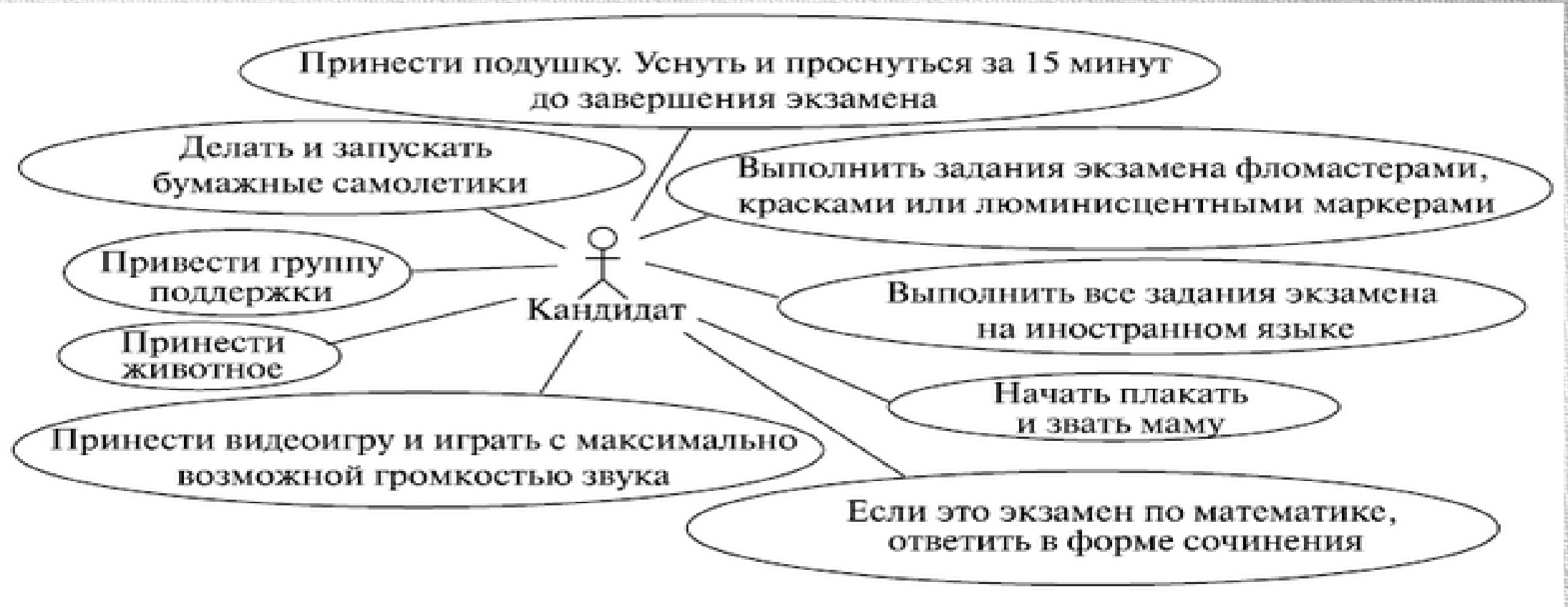
Программные системы в качестве действующих лиц




<<обычно>>
Несет обычные яйца
Курица
<<очень редко>>
Несет золотые яйца
Подробности смотри в
"Сказке о Курочке Рябе"


pkg Утка пришла в бар - use case
Идет в бар
<<extend>>
Заказывает пиво
Утка
Бармен
Подает пиво
<<include>>
Требует плату
"Просто запишите на мой счет!"
Задерживает платеж
Утка
Generated by UModel
www.altova.com

Как гарантированно провалить любой экзамен



Прямое и обратное проектирование

Прямое и обратное проектирование

Прецеденты полезны и для прямого и обратного проектирования. При прямом проектировании мы, *по* сути, осуществляем "перевод" с *UML* на некий *язык программирования*. И тестировать созданное *приложение* следует, основываясь именно на потоках событий, описываемых прецедентами. *Обратное проектирование* предполагает перевод с языка программирования на язык *UML*-диаграмм.

С какой целью?

1. С целью поиска ошибок и чтобы убедиться в адекватности проектирования: после первого перевода с UML на язык программирования сделать обратный перевод и сравнить исходные и восстановленные UML-модели (желательно, чтобы эти переводы выполнялись разными командами). Это позволит убедиться в том, что проектирование системы соответствует модели, никакая информация в ходе перевода не была утеряна, да и попросту выловить некоторые "баги". Такой подход называется обратной семантической трассировкой (или RST - Reverse Semantic *Traceability*).

2. Когда отсутствует документация: иногда стоит задача модификации существующей системы, код которой плохо документирован. В таком случае перевод с языка программирования на язык UML-диаграмм - отличный способ понять назначение системы и ее частей, функционал, предоставляемый ею, и т. д.

Прямое проектирование (forward engineering)

Прямое проектирование (forward engineering) – это процесс трансформации модели в код путем ее отображения на язык реализации. Диаграмма вариантов использования может быть подвергнута прямому проектированию с целью формирования тестов для того элемента, который она описывает. Каждый вариант использования на такой диаграмме специфицирует поток событий с его вариациями, и все эти потоки описывают ожидаемое поведение элемента – то есть нечто такое, что подлежит тестированию. Хорошо структурированный вариант использования даже специфицирует пред и постусловия, которые могут применяться в целях определения начального состояния теста и критериев его успешности. Для каждого варианта использования из диаграммы вы можете создать *сценарий тестирования (test case), который бу*дет выполняться при выпуске каждой новой версии данного элемента, таким образом подтверждая, что он работает надлежащим образом, прежде чем другие элементы смогут опираться на него в своей работе.

Обратное проектирование (reverse engineering)

*Обратное проектирование (reverse engineering) – процесс транс*формации кода в модель посредством его отображения из определенного языка реализации. Автоматически выполнить обратное проектирование диаграммы вариантов использования на данном этапе не представляется возможным, потому что на пути от спецификации поведения элемента к его реализации происходит потеря информации. Однако в ваших силах изучить существующую систему и распознать ее подразумеваемое поведение самостоятельно, с тем чтобы на этой основе оформить диаграмму вариантов использования. В любом случае неплохо это делать каждый раз, когда вы имеете дело с недокументированным программным обеспечением. Диаграммы вариантов использования UML просто предоставляют вам стандарт и выразительный язык, чтобы можно было зафиксировать то, что вы при этом обнаружите.

Варианты использования и кооперации

Варианты использования и кооперации

Вариант использования определяет поведение системы (или подсистемы, класса или интерфейса), которую вы разрабатываете, **не указывая того, как это поведение реализовано**.

Разделение спецификации поведения и его реализации важно, потому что анализ системы, специфицирующий поведение, должен по возможности быть независимым от решений реализации, которые специфицируют, *как именно оно обеспечивается*.

В конечном счете, однако, вы должны реализовать варианты использования путем создания сообществ классов и других элементов, работающих вместе для реализации поведения, описанного вариантом использования. Такое сообщество элементов, обладающее как статической, так и динамической структурой, моделируется в UML как кооперация.

На рисунке показано, что реализацию варианта использования можно специфицировать явно через кооперацию. **Поскольку большую часть времени данный вариант использования реализуется только одной кооперацией, нет необходимости моделировать эту связь явно.**

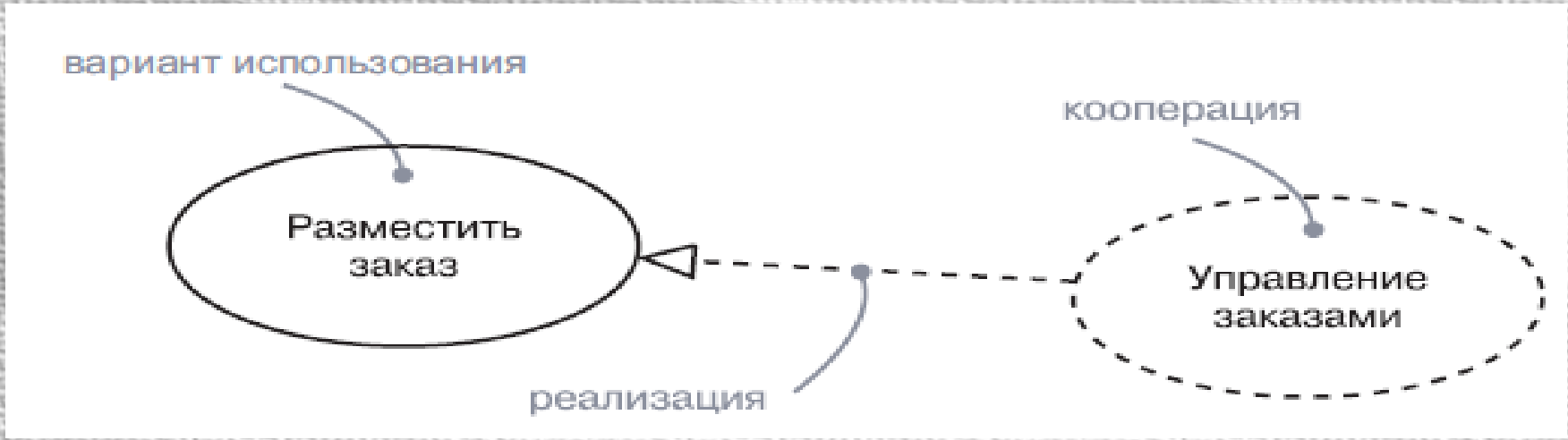


Диаграмма кооперации уровня спецификации

Диаграмма кооперации уровня спецификации показывает роли, которые играют участвующие во взаимодействии элементы. Элементами кооперации на этом уровне являются классы и ассоциации, которые обозначают отдельные роли классификаторов и ассоциации между участниками кооперации

Уровни Вариантов использования

Организация вариантов использования

Для организации вариантов использования их группируют в пакеты так же, как классы. Кроме того, вы можете организовать варианты использования, определив между ними связи обобщения, включения и расширения. Эти связи применяются для того, чтобы выделить некоторое общее поведение (извлекая его из других вариантов использования), а также разновидности (помещая такое поведение в другие варианты использования, которые расширяют данный).

Уровни абстракции

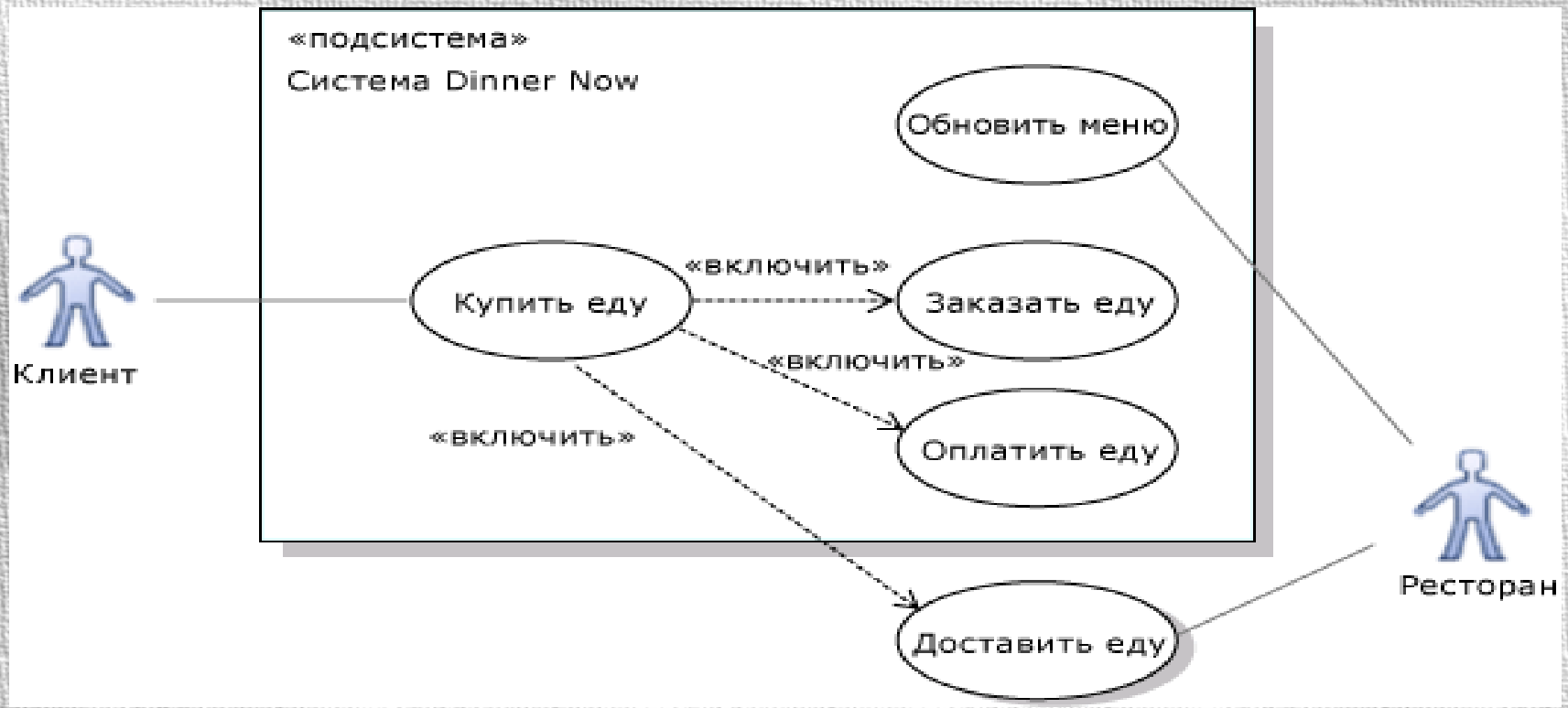
Варианты использования могут описывать систему на разных уровнях абстракции. Так, вариант использования, описывающий всю систему, можно разложить на несколько второстепенных вариантов использования, каждый из который специфицирует одну услугу подсистемы. Функциональность, которая определяется в варианте использования высшего уровня, полностью прослеживается в функциональности вариантов использования нижнего уровня. Таким образом, вариант использования высшего уровня и варианты использования нижнего уровня описывают одно и то же поведение, но на разных уровнях абстракции.

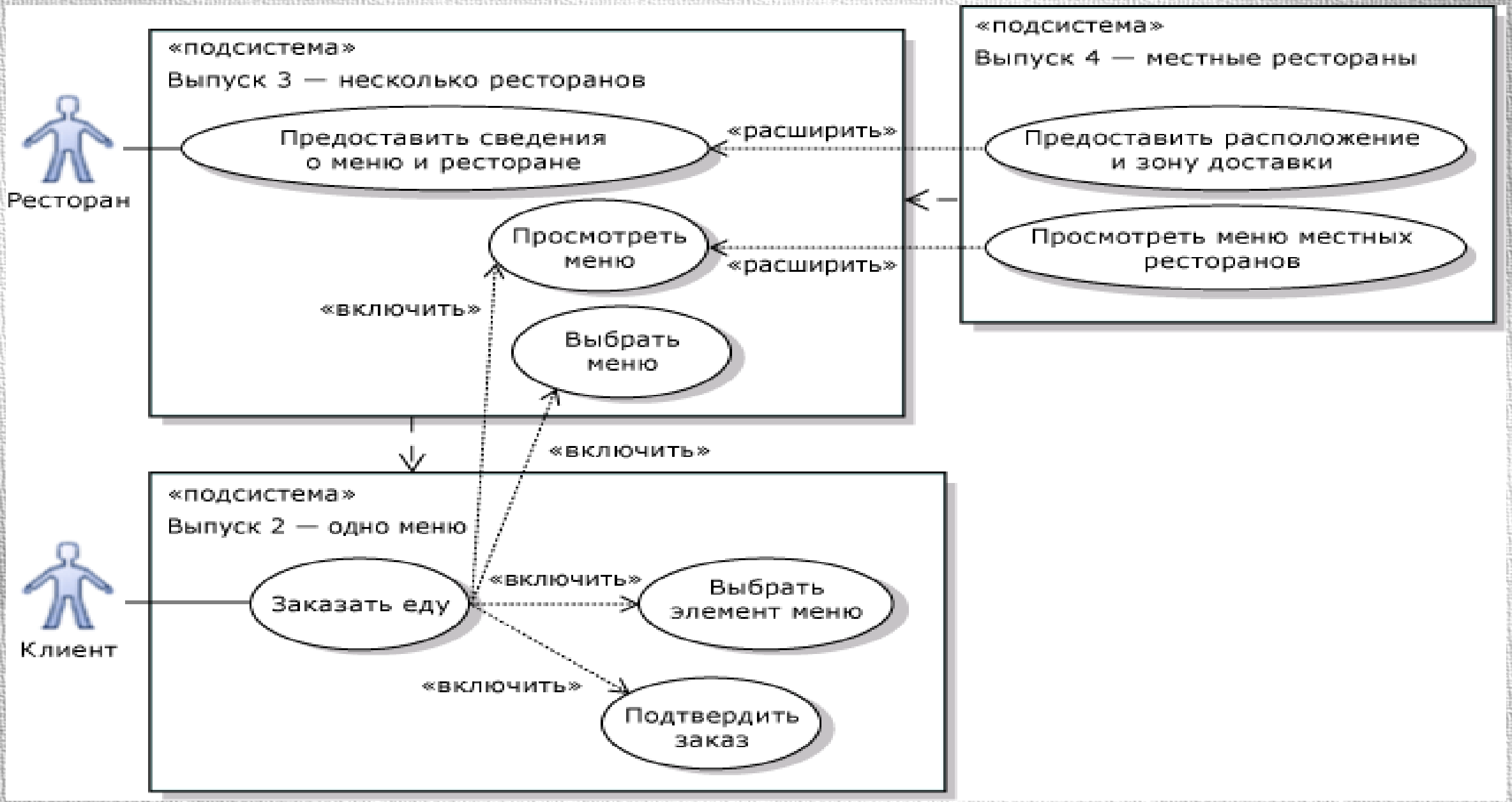
Для того чтобы обеспечить поведение высшего варианта использования, нижние варианты должны взаимодействовать. Взаимодействие вариантов использования нижнего уровня описывается с помощью кооперацій варианта использования верхнего уровня и может быть представлено на диаграммах кооперации. Субъекты, присутствующие в варианте использования верхнего Уровня, появляются и во второстепенных вариантах. Более того, второстепенные варианты использования являются друг для друга субъектами. Эта распределенная по уровням реализации приводит в итоге к появлению множества вложенных вариантов использования и кооперацій, которые и служат для программной реализации всей системы.

Если в проекте более 100 вариантов использования разработчики автоматически разделяют их по предметам областям. Обычно именовать естественные предметные области не составляет труда. Один проект использовал информацию клиента, продвижения, выписку счетов и рекламную деятельность. В другом были такие предметные области, как приём заказов, маршрутизация, отслеживание, доставка и выписка счётов. Зачастую внутри различных предметных областей существует подпроекты. Каждая предметная область может содержать от 20 до 100 вариантов использования. Несмотря на то, что отслеживать сотни вариантов использования довольно сложно, отслеживать 15— 20 групп вполне приемлемо. В большом проекте можно сгруппировать варианты использования по предметной области, чтобы получить 3-6 групп по 20-100 ВИ в каждой, а затем сгруппировать их по версии и команде разработчиков.

ПРИМЕР

Заказать еду
Клиент
Обновить меню
Ресторан


«подсистема»
Система Dinner Now
Обновить меню
Клиент
Купить еду
«включить»
Заказать еду
«включить»
«включить»
Оплатить еду
Доставить еду
Ресторан


«подсистема»
Выпуск 3 — несколько ресторанов
Предоставить сведения о меню и ресторане
Просмотреть меню
Выбрать меню
«включить»
«подсистема»
Выпуск 4 — местные рестораны
Предоставить расположение и зону доставки
Просмотреть меню местных ресторанов
«расширить»
«расширить»
Ресторан
«включить»
«подсистема»
Выпуск 2 — одно меню
Заказать еду
«включить»
Выбрать элемент меню
«включить»
Подтвердить заказ
Клиент

Типовая последовательность разработки функциональных требований в форме UC

1. Определить **Акторов** — тех, кто взаимодействует с системой
2. Выявить и составить **список** UC — задач,
   которые стейкхолдеры смогут решить с помощью системы
3. Для каждого ВИ определить **приоритет** — важность UC
   по отношению к другим вариантам использования
4. Для каждого важного ВИопределить **контекст** применения,
   конечную **цель и результат**
5. Описать **основной поток** каждого UC с высоким приоритетом
   в виде последовательности шагов, которая приведёт к достижению
   его цели — пользовательского сценария (use case scenario)

6. Дополнить основной поток **расширениями**, **исключениями и циклами**
7. Собрать **воедино** результаты шагов 4–6, детально описав каждый ВИ с высоким приоритетом как алгоритм взаимодействия пользователя с системой

Реестр ВИ

Потребность пользователя	UC
Связаться с другим человеком	Сделать звонок Отправить СМС
Скоротать время	Поиграть в игру Посмотреть Фотографии
Сохранить визуальную информацию	Сделать Фотографию

3. Для каждого UC определить его приоритет

Реализация юскейсов в коде — трудоёмкая работа, поэтому надо понимать, с каких лучше начать в первую очередь, а до каких очередь может не дойти никогда.

Реестр юскейсов может использоваться для приоритизации пользовательских функциональных требований с целью их ранжирования по степени важности, которая определяет порядок реализации.

Самым простым методом приоритизации, который часто применяется на практике, считается метод MoSCoW. Он получил своё название не в честь российской столицы, а как сокращение следующих 4 категорий:

1. **Must** — то, что необходимо сделать в любом случае. Считается, что без реализации этих требований решение не будет работать или при отсутствии этих фич продукт не будет нужен потребителям. Чаще всего этот приоритет называется 1-я очередь и обозначается цифрой 1
2. **Should** — требования 2-ой очереди, которые должны быть реализованы после тех, что отнесены к группе Must. Это приоритет номер 2
3. **Could** — желательные требования, которые можно сделать, если останется время и будут ресурсы. Это приоритет номер 3
4. **Would** — требования, которые хотелось бы реализовать, но пока их можно проигнорировать или перенести на следующие итерации без вреда для продукта. Это приоритет номер 4

Реестр ВИ

Потребность пользователя	UC	Приоритет
Связаться с другим человеком	Сделать звонок Отправить СМС	1 — Must 2 — Should
Скоротать время	Поиграть в игру Посмотреть Фотографии	4 — Would 3 — Could
Сохранить визуальную информацию	Сделать Фотографию	2 — Should

Справедливости ради стоит отметить, что метод MoSCoW в качестве базиса приоритизации выбирает менеджерскую точку зрения, то есть не учитывает технические зависимости между требованиями, которые определяются при трассировке требований, то есть. их связей друг с другом.

4. Для каждого высокоприоритетного (Must или Should) UC определить контекст применения, конечную цель и результат

Это можно оформить в виде списка или таблицы.

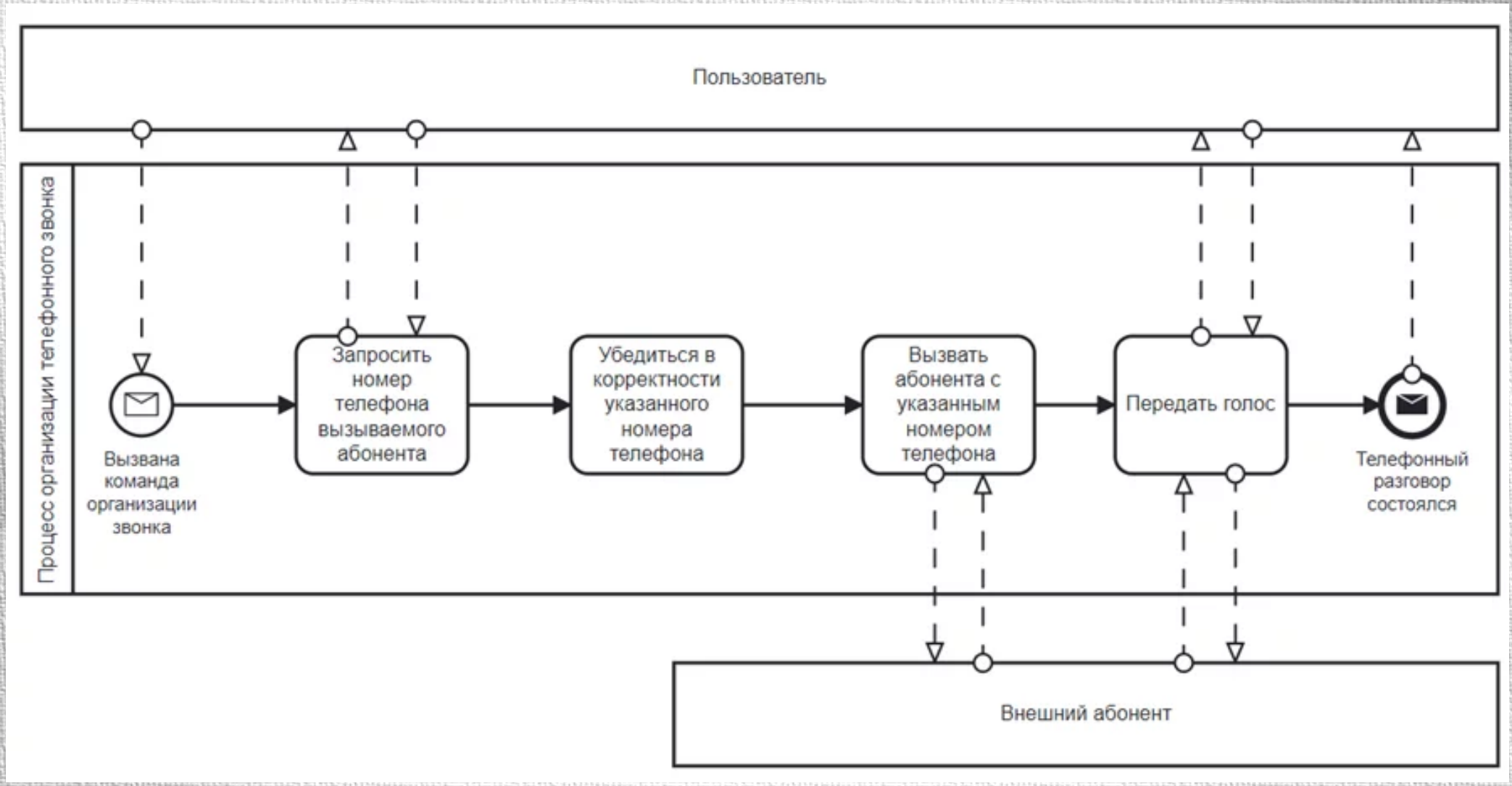
Имя	UC 1.Сделать Звонок
Область действия	Мобильный телефон
Контекст	Организовать удалённое голосовое взаимодействие с другим человеком
Актор	Пользователь мобильного телефона, Внешний абонент
Цель	Поговорить с другим человеком (внешним абонентом) по телефонной связи
Результат (постусловия)	Состоялся телефонный разговор пользователя с желаемым человеком длительностью не менее 3-х секунд

5. Раскрыть содержание основного потока каждого ВИ высокого приоритета в виде пользовательского сценария

Основную часть пользовательского сценария составляет так называемый **happy path** (основной поток), который представляет собой пошаговый алгоритм выполнения сценария без логических операторов XOR (исключающее или), используемых для ветвления потока управления в результате проверки каких-либо условий.

Для наглядности можно нарисовать эту линейную последовательность шагов в виде простого процесса в нотациях EPC, BPMN, UML activity или sequence, а также блок-схемы алгоритма по ГОСТ 19.701–90.

Последовательность шагов в виде BPMN-диаграммы



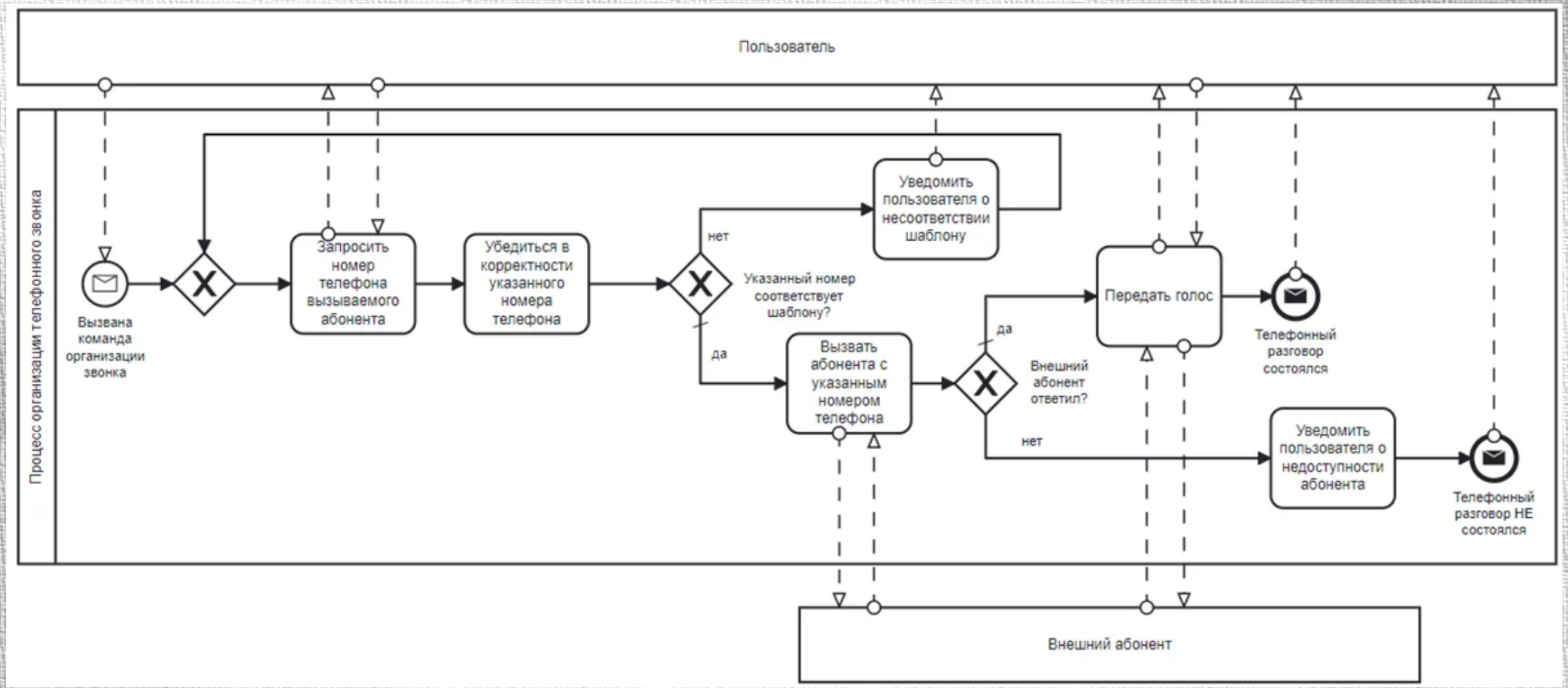
6. Дополнить линейную последовательность основного потока в описании UC расширениями, исключениями и циклами

На этом этапе аналитик усложняет ранее полученный happy path, включая в схему различные ветвления с помощью логического оператора XOR.

Вместо BPMN для отображения логики сценария нашего юскейса можно использовать любую другую нотацию (EPC, UML activity diagram, IDEF3 или даже простого текстового алгоритма).

Расширенная BPMN-диаграмма ВИ с исключениями и альтернативными потоками

На этом этапе аналитик усложняет ранее полученный happy path,



Вместо BPMN для отображения логики сценария нашего юскейса можно использовать любую другую нотацию (EPC, UML activity diagram, IDEF3 или даже простого текстового алгоритма).

7. Собрать воедино результаты выполнения трёх предыдущих шагов, детально описав каждый UC как алгоритм взаимодействия пользователя с системой

На этом этапе у каждого ВИ появляется описание, структура которого всегда примерно такая:

Имя
Приоритет
Область действия
Контекст
Актор
Цель
Предусловия
Результат (постусловие)
Основной поток
Расширения или альтернативные потоки
Бизнес-правила

CRUD и параметризованные варианты использования

ВАРИАНТЫ ИСПОЛЬЗОВАНИЯ CRUD

До сих пор нет общего мнения относительно того, как организовать все маленькие варианты использования типа **Создать NNN**, **Найти NNN**, **Обновить NNN** и **Удалить NNN**. Они известны как варианты использования **CRUD** (аббревиатура названий операций над базами данных Create, Retrieve, Update и Delete). Вопрос в том, входят ли все они в один вариант использования, чей размер больше варианта использования **Управлять NNN**, либо существуют самостоятельно?

Параметризованные варианты использования

Время от времени мы сталкиваемся с созданием почти одинаковых вариантов использования. Наиболее распространенные примеры — Найти клиента, Найти продукт, Найти данные о продвижении и т.д. Скорее всего, только одна группа разработчиков будет создавать общий механизм поиска, а другие команды будут его использовать.

Написать полдюжины похожих вариантов использования в свободной форме не составляет большого труда. Однако писать шесть подобных вариантов использования по полной форме довольно трудоемкое и нудное занятие. Здесь необходим метод, экономящий усилия.

Пример
(Коберн)

1. Пользователь определяет предмет, который необходимо найти.
2. Система производит поиск, выдает список возможных вариантов.
3. Пользователь выбирает, возможно, повторно сортирует список и изменяет критерий поиска.
4. Система находит предмет (или не находит).

При каждом использовании изменяются:

- **Название предмета**, который нужно найти.
- **Критерии поиска** (поля поиска) предмета, который нужно найти.
- **Набор выводимых атрибутов предмета**, который нужно найти (показываемые значения одно за другим).
- **Критерий сортировки** списка найденных вариантов.

Мы создали параметризованный вариант использования, т.е. такой, который работает с кратким именем для каждого из перечисленных элементов. Мы решили назвать этот варианта использования **Найти что угодно**. Понятно, что фраза “Служащий находит клиента” означает вызов “Найти что угодно” для поиска клиента.

Уровни точности

Для значений данных мы определили три уровня точности.

1. **Краткое имя данных.** Первым была фраза из текста варианта использования — краткое имя данных, такое как Профиль клиента.
2. **Списки полей**. Вторым были списки полей, связанных с кратким именем данных, именующим всю собранную под ним информацию, например имя, адрес, служебный телефон, домашний телефон и т.д.
3. **Характеристики поля**. Третий уровень точности был точным определением поля: длина, критерии оценки и т.д.

Вариант использования содержал только первый уровень точности. Описания данных, критерии поиска и сортировки были помещены на отдельной странице, гиперссылка на которую содержалась внутри шага варианта использования.

Результатом стал шаг действия, выглядевший примерно так:

- Служащий находит клиента с помощью параметров поиска клиента.
- На странице *Параметры поиска* клиента определялись поля поиска, последовательность вывода полей и критерии сортировки. Читатель ВИ щелкал по подчеркнутому выражению, чтобы увидеть содержимое страницы.

Читатели, авторы и разработчики легко и быстро освоили этот механизм.

Измененный ВИ «*Найти что угодно* »

Вариант использования *Найти что угодно* теперь выглядит так:

1. Пользователь определяет параметры поиска чего-либо.
2. Система находит все, что соответствует параметрам поиска, и выводит список их отображаемых значений.
3. Пользователь может пересортировать их согласно критерию сортировки.
4. Пользователь выбирает интересующий его вариант.

Этот изящный прием избавляет вызывающие варианты использования от загромождения многочисленными деталями (более низкого уровня) поиска, сортировки, пересортировки и т.д. Общее поведение при поиске локализуется и записывается лишь один раз. Постоянство для каждого, кто ведет поиск, обеспечено, и те, кому необходимо знать подробности для целей программирования, могут найти их.

Разработчикам очень понравилось, что им дали только одну спецификацию механизма поиска, поэтому они не интересовались, все ли виды поиска действительно одинаковы.

Дополнительные обозначения языка UML для бизнес-моделирования

Дополнительные обозначения языка UML для бизнес-моделирования

Бизнес-актер (business actor) – индивидуум, группа, организация, компания или система, которые взаимодействуют с моделируемой бизнес-системой, но не входят в нее, т.е. не являются частью моделируемой системы.

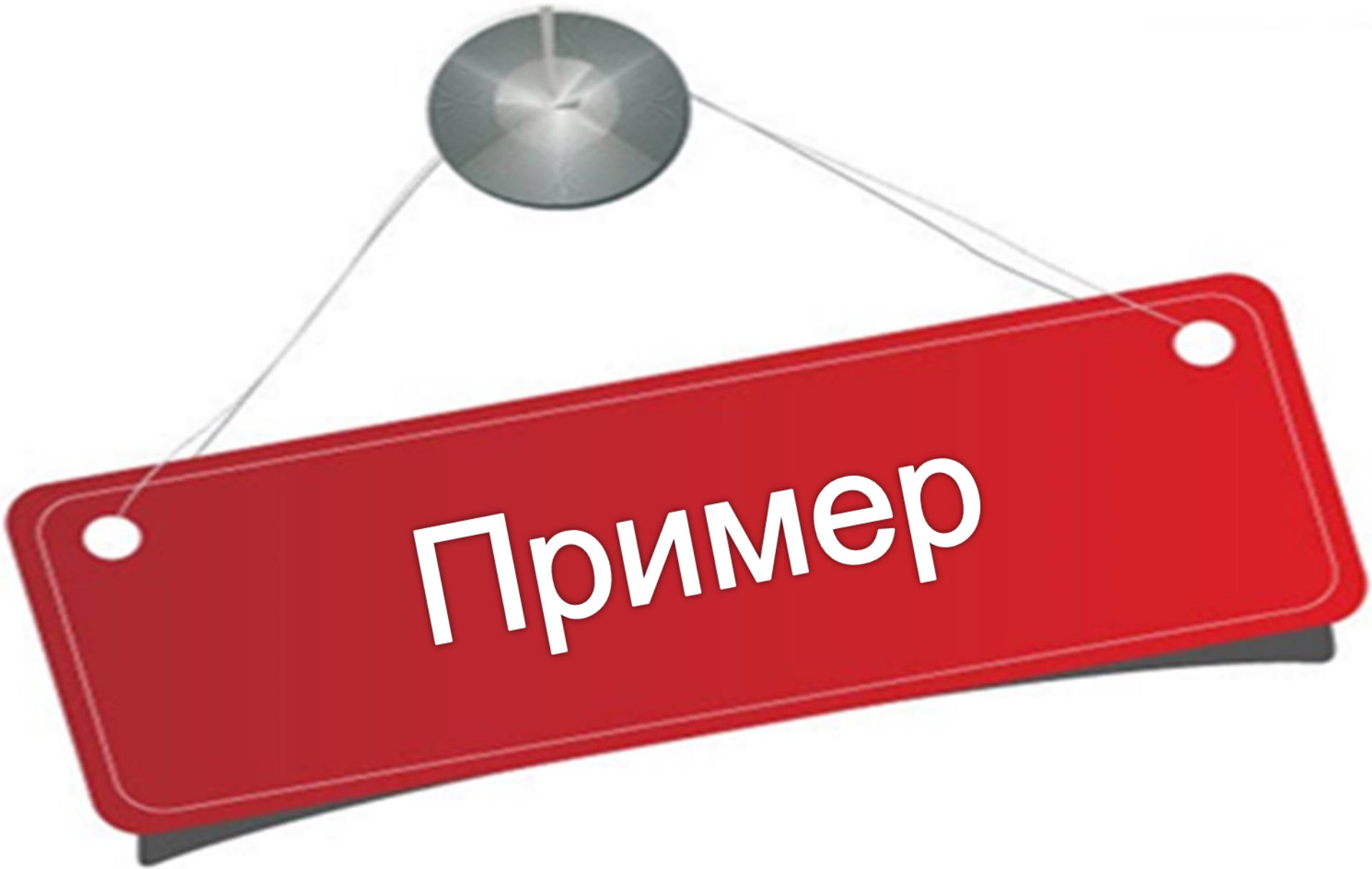
Сотрудник (business worker) – индивидуум, который действует внутри моделируемой бизнес-системы, взаимодействует с другими *сотрудниками* и является участником бизнес-процесса моделируемой системы.

Примерами *бизнес-актеров* являются клиенты, покупатели, поставщики, партнеры. Общее свойство *бизнес-актеров* состоит в том, что они являются инициаторами или клиентами бизнес-процессов моделируемой системы.

Примерами *сотрудников* являются менеджеры, администраторы, кассиры, инженеры. Общее свойство *сотрудников* заключается в том то, что они являются субъектами и входят в состав моделируемой системы.

Бизнес-вариант использования (business use case) — *вариант использования*, определяющий последовательность действий моделируемой системы, направленных на выполнение отдельного бизнес-процесса.

Общее свойство бизнес- *вариантов использования* состоит в том, что они являются концептуальной моделью отдельных бизнес-процессов моделируемой системы.


Пример

Применение этих элементов графической нотации иллюстрирует пример представления диаграмм *вариантов использования* для системы продажи товаров в супермаркете. Эта модель может быть использована при создании и автоматизации соответствующих информационных систем.

В качестве *бизнес-актера* данной системы можно рассматривать покупателя супермаркета, а в качестве *сотрудника* – продавца. При этом *покупатель* является клиентом сервиса "Оформление заказа на покупку товара", а *продавец* участвует в реализации соответствующего бизнес-процесса. Как следует из существа выдвигаемых к системе требований, этот сервис выступает в качестве базового *бизнес-варианта использования* данной системы.

С одной стороны, *продажа* товаров предполагает согласование условий оплаты с покупателем и заказ товара со склада. Поскольку эта функциональность выполняется всегда, она может быть выделена в самостоятельные *бизнес-варианты использования*, которые будут связаны с базовым отношением *включения*.

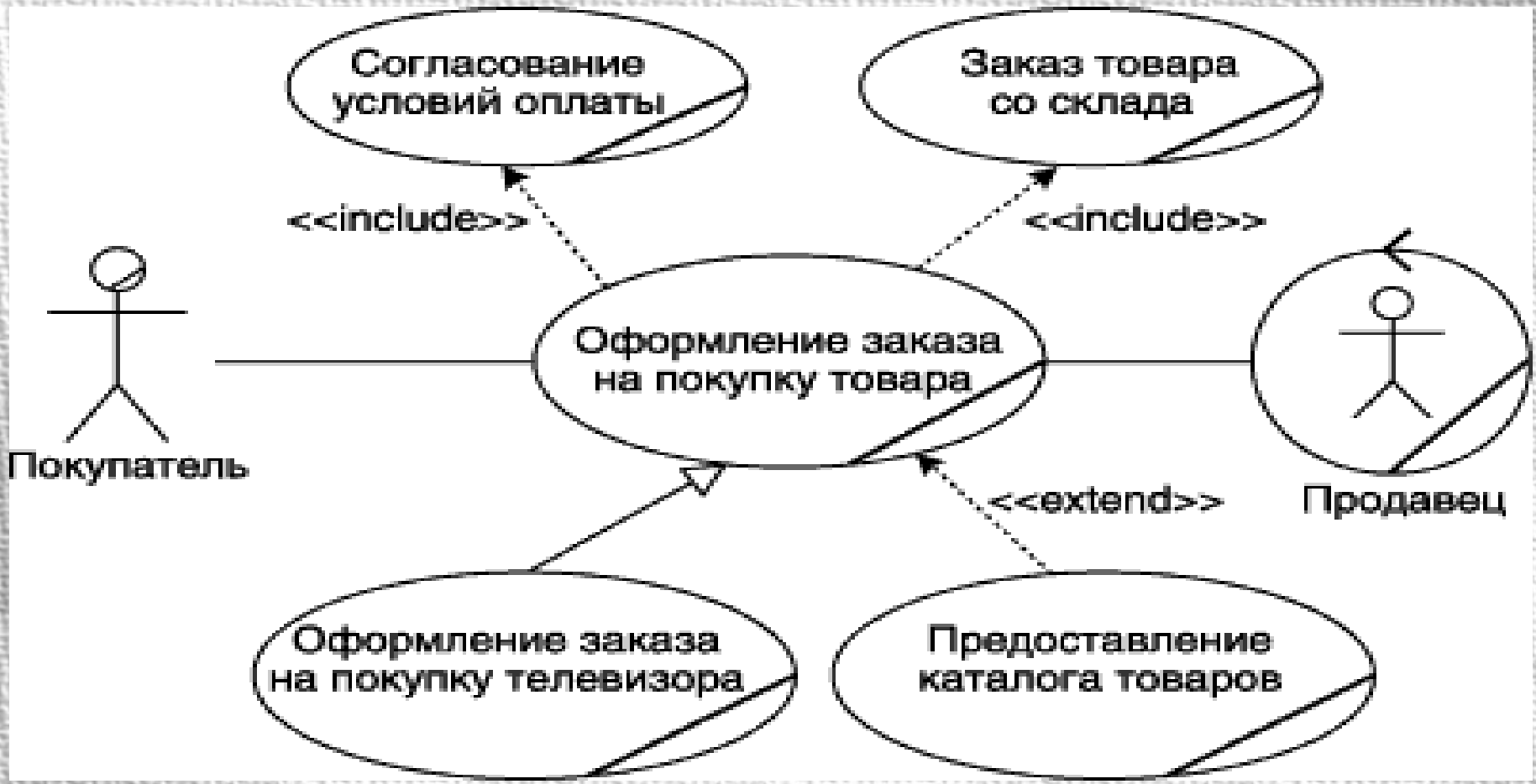
С другой стороны, *продажа* товаров может предполагать наличие отдельного информационного объекта — каталога товаров, который в некотором смысле не зависит от реализации сервиса по обслуживанию покупателей. В данном случае, каталог товаров может запрашиваться покупателем у продавца при необходимости выбора товара и уточнения его свойств. Вполне резонно представить сервис "Предоставление каталога товаров" в качестве самостоятельного *бизнес-варианта использования*.

Дальнейшая детализация данной модели может быть выполнена на основе установления дополнительных отношений, в частности отношения "обобщение-специализация" для уже имеющихся компонентов диаграммы вариантов использования. Так, в рамках рассматриваемой системы продажи товаров может иметь самостоятельное значение и специфические особенности отдельная категория товаров — телевизоры.

В этом случае *диаграмма* дополняется *вариантом использования* "Оформление заказа на покупку телевизора", который связан с сервисом "Оформление заказа на покупку товара" отношением *обобщения*.

Полученная в результате *диаграмма вариантов использования* будет содержать 5 *бизнес-вариантов использования*, одного *бизнес-актера* и одного *сотрудника*, между которыми установлены соответствующие отношения *включения*, *расширения* и *обобщения*.

ДВИ для системы продажи товаров в общих обозначениях языка UML



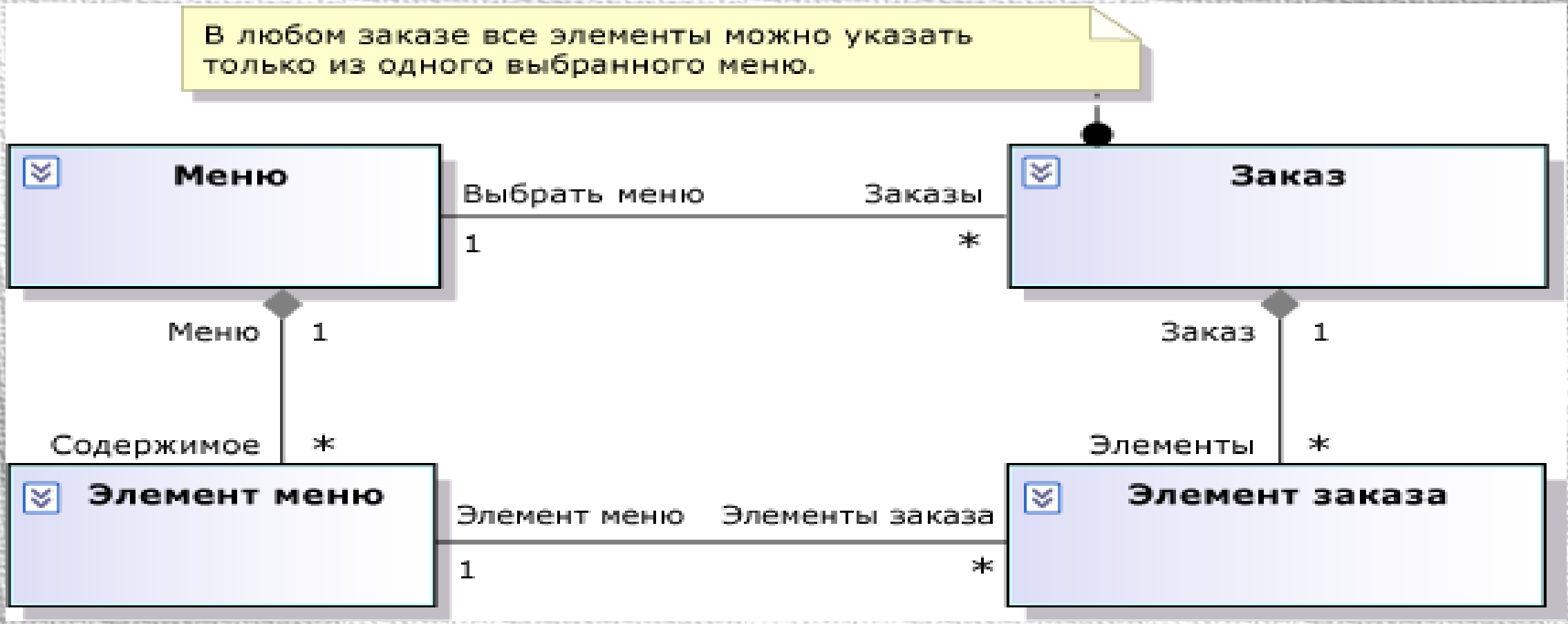
Анализируя рассматриваемую систему продажи товаров по каталогу, можно заметить, что она представляет собой концептуальную модель типичной бизнес-системы, особенности которой связаны с получением определенной прибыли от реализации соответствующих бизнес-процессов. При этом роли покупателя и продавца в рассматриваемой системе существенно отличаются. Действительно, *покупатель* является внешним по отношению к системе субъектом, в то время как *продавец* является частью бизнес-системы. Реализация рассмотренных *вариантов использования* не изображается на диаграммах *вариантов использования*.

Отображение нефункциональных требований

ОТОБРАЖЕНИЕ БИЗНЕС-ПРАВИЛ

Бизнес-правило — это требование, не связанное с определенным вариантом использования, которое необходимо соблюдать во всех частях системы.

Многие бизнес-правила представляют собой ограничения отношений между концептуальными классами. Эти *статические бизнес-правила* можно записать в виде комментариев, связанных с соответствующими классами на концептуальной схеме классов. Например:


В любом заказе все элементы можно указать только из одного выбранного меню.
Меню
Выбрать меню
Заказы
1
*
Заказ
Меню
1
Заказ
1
Содержимое
*
Элементы
*
Элемент меню
Элемент меню
Элементы заказа
1
*
Элемент заказа

Описание требований к качеству обслуживания

Существует несколько категорий требований к качеству обслуживания. Среди них можно выделить следующие.

- *Производительность*
- *Безопасность*
- *Полезность*
- *Надежность*
- *Устойчивость*

Некоторые из этих требований можно включить в описания конкретных вариантов использования. Другие требования не относятся к вариантам использования, поэтому рекомендуется записать их в отдельный документ. По возможности рекомендуется использовать словарь, определенный в модели требований.

ТРЕБОВАНИЯ И ВАРИАНТЫ ИСПОЛЬЗОВАНИЯ В RUP

Что такое RUP?

RUP – это процесс, направленный на поддержку коллективной разработки ПС. Все участники проекта используют единую базу знаний, единый процесс, единый взгляд на разработку, единый язык моделирования. RUP разрабатывался рука об руку с UML – промышленным стандартом ОО моделирования – тем же коллективом авторов. Все модели в RUP представляются в нотации UML. RUP – это технологический процесс по созданию ПС, позволяющий улучшить производительность коллективной разработки путем предоставления для всех этапов жизненного цикла методик выполнения основных видов деятельности, шаблонов документов, инструкций по работе с инструментальными средствами.

RUP предлагает итеративный подход к проектированию и разработке ПС, основанный на **спиральном** жизненном цикле. Весь жизненный цикл включает четыре фазы

1. вхождение в проект (исследование),
2. развитие (уточнение плана),
3. конструирование,
4. развертывание.

Каждая фаза складывается из последовательности итераций, число которых может быть любым. В каждой итерации перечисленные выше технологические процессы последовательно применяются к разработке небольшой части ПС. При этом допустимо предъявление результата заказчику. Он имеет возможность оценить выполненную реализацию, выдать свои замечания, которые могут привести к изменению и уточнению требований к ПС. Следующая итерация предполагает расширение уже разработанной части путем реализации и интеграции очередной порции требований и учета изменения требований в соответствии с замечаниями заказчика.

Такая организация процесса имеет целый ряд преимуществ.

- Изменения и уточнения требований выявляются уже в ранний период разработки, когда объем программного кода сравнительно небольшой, поэтому трудоемкость внесения изменений существенно ниже.
- Уже на ранней стадии процесса разработки имеется возможность привлечения специалистов заказчика для оценки промежуточных версий (прототипов) ПС. Как результат – значительно более высокая вероятность того, что конечный продукт будет делать именно то, чего ждет от него заказчик, то есть, гарантируется высокое качество ПС.

- Снижаются архитектурные и интеграционные риски. При итеративном подходе создается устойчивая архитектура, которая прорабатывается на стадиях исследования, а затем проверяется и уточняется в нескольких начальных итерациях. Каждая итерация предполагает интеграцию новых элементов в систему, то есть, количество интегрируемых элементов в каждой итерации невелико и легче прослеживается, чем при глобальной интеграции всех разработанных элементов ПС.
- Итеративный подход способствует более полному повторному использованию программных элементов. Анализ результатов каждой итерации позволяет архитекторам ПС выделить фрагменты, потенциально подлежащие повторному использованию, а в следующей итерации оформить их как повторно используемые коды. Это свойство очень хорошо сочетается с идеями ОО и КБ проектирования.

Принципы

В основе RUP лежат следующие принципы:

- Ранняя идентификация и непрерывное (до окончания проекта) устранение основных рисков.
- Концентрация на выполнении требований заказчиков к исполняемой программе (анализ и построение модели прецедентов).
- Ожидание изменений в требованиях, проектных решениях и реализации в процессе разработки.
- Компонентная архитектура, реализуемая и тестируемая на ранних стадиях проекта.
- Постоянное обеспечение качества на всех этапах разработки проекта (продукта).
- Работа над проектом в сплочённой команде, ключевая роль в которой принадлежит архитекторам.

Для каждого технологического процесса предусмотрены

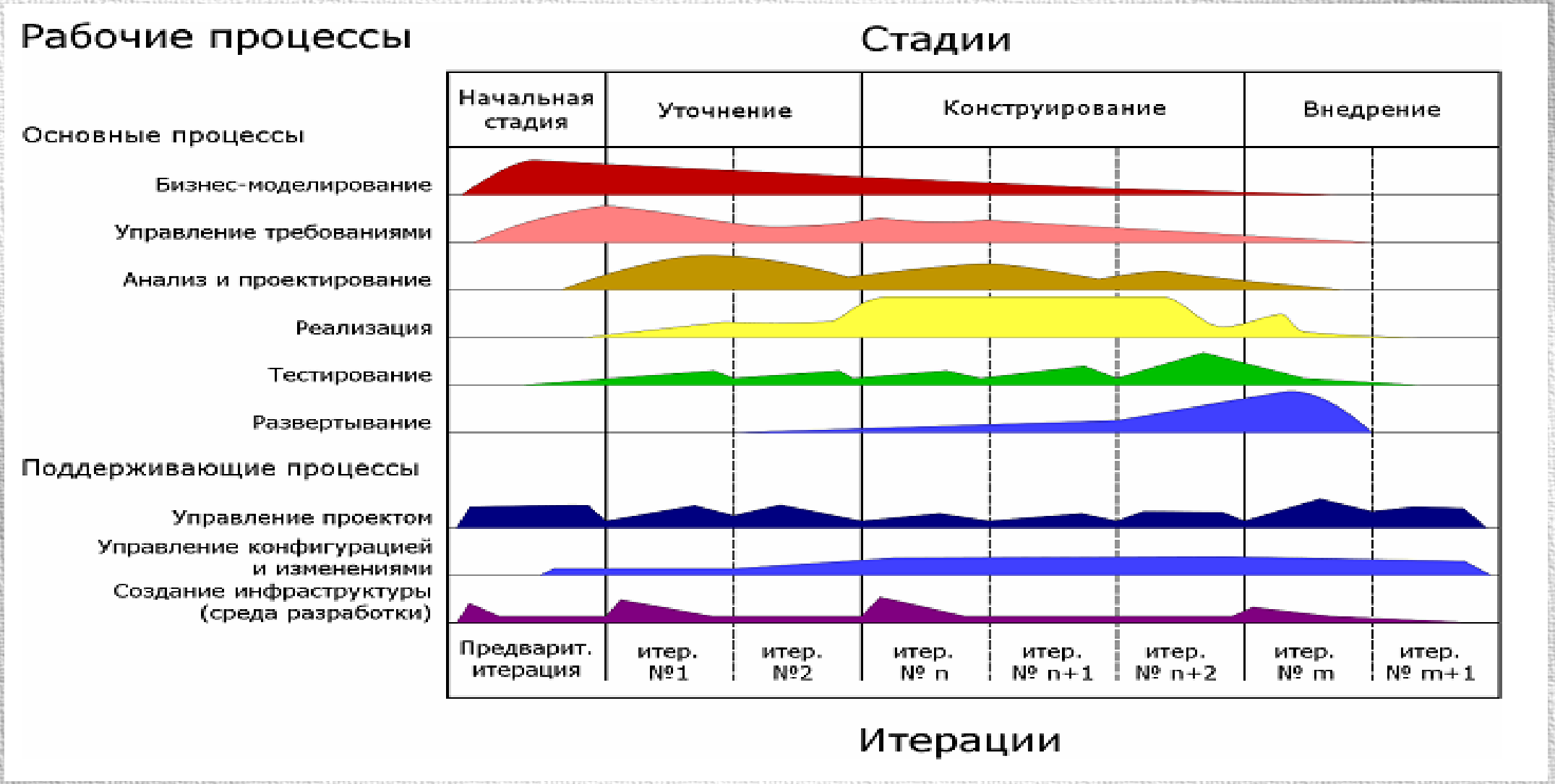
- **роли**, определяющие поведение и обязанности отдельных лиц и групп, работающих в одной команде (например, системный аналитик, тестировщик),
- **виды деятельности**, определяющие работы, выполняемые исполнителями (например, проектирование класса, проектирование ВИ),
- **артефакты** – документы, используемые, порождаемые или модифицируемые процессом. **Основные артефакты в RUP – модель, элемент модели, документ, исходный код, исполняемая программа.**

Жизненный цикл разработки

RUP использует итеративную модель разработки. В конце каждой итерации (в идеале продолжающейся от 2 до 6 недель) проектная команда должна достичь запланированных на данную итерацию целей, создать или доработать проектные артефакты и получить промежуточную, но функциональную версию конечного продукта. Итеративная разработка позволяет быстро реагировать на меняющиеся требования, обнаруживать и устранять риски на ранних стадиях проекта, а также эффективно контролировать качество создаваемого продукта.

Полный жизненный цикл разработки продукта состоит из четырех фаз, каждая из которых включает в себя одну или несколько итераций:

Графическое представление процесса разработки по RUP



1. Начало (Inception)

В фазе «Начало»:

- Формируются видение и границы проекта.
- Создается экономическое обоснование (business case).
- Определяются основные требования, ограничения и ключевая функциональность продукта.
- Создается базовая версия модели прецедентов.
- Оцениваются риски.
- При завершении начальной фазы оценивается достижение *вехи целей жизненного цикла*, которое предполагает соглашение заинтересованных сторон о продолжении проекта.

2. Уточнение
(Elaboration)

В фазе «Уточнение» производится анализ предметной области и построение исполняемой архитектуры. Это включает в себя:

- Документирование требований (включая детальное описание для большинства прецедентов).
- Спроектированную, реализованную и оттестированную исполняемую архитектуру.
- Обновленное экономическое обоснование и более точные оценки сроков и стоимости.
- Сниженные основные риски.

Успешное выполнение фазы разработки означает достижение *вехи архитектуры жизненного цикла*.

3. Построение или Конструирование
(Construction)

В фазе «Построение» происходит реализация большей части функциональности продукта. Фаза **Построение** завершается первым внешним релизом системы и вехой начальной функциональной готовности (Initial Operational Capability).

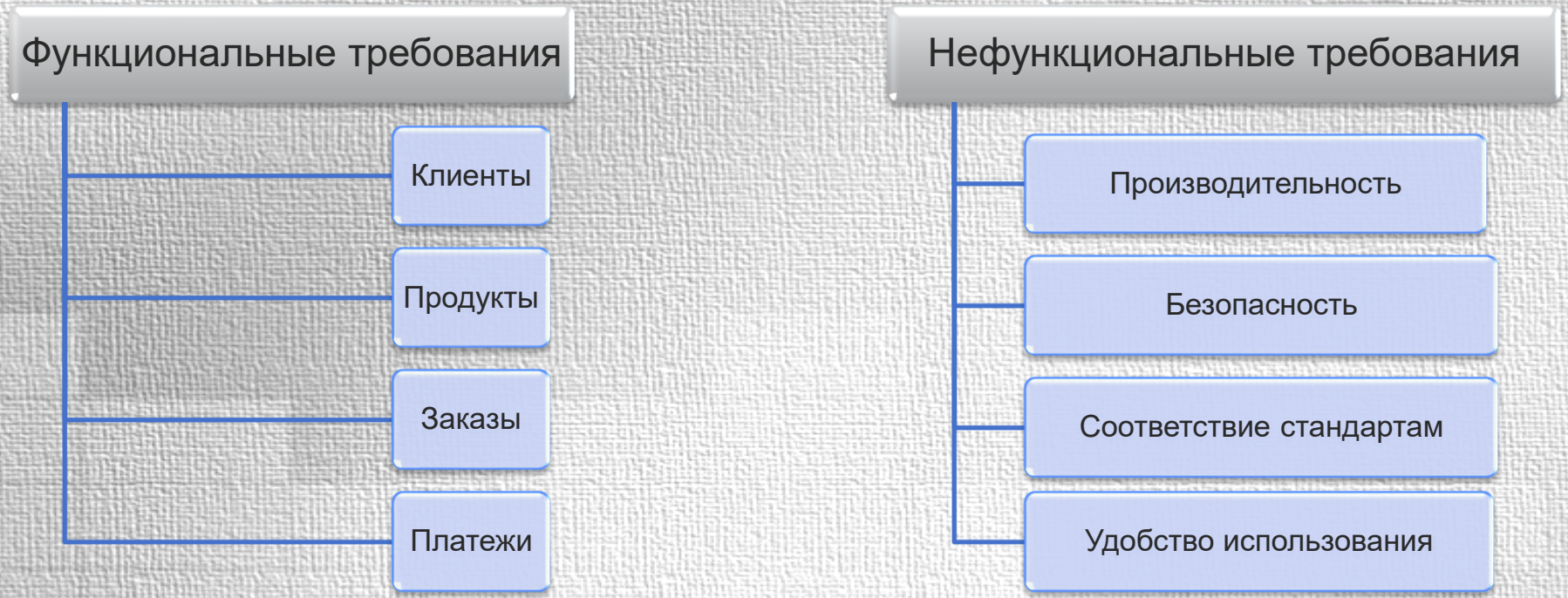
4. Внедрение
(Transition)

В фазе «Внедрение» создается финальная версия продукта и передается от разработчика к заказчику. Это включает в себя программу бета-тестирования, обучение пользователей, а также определение качества продукта. В случае, если качество не соответствует ожиданиям пользователей или критериям, установленным в фазе **Начало**, фаза **Внедрение** повторяется снова. Выполнение всех целей означает достижение вехи готового продукта (Product Release) и завершение полного цикла разработки.

Организация требований

Базовое разделение на функциональные и нефункциональные требования, описанное выше, является очень простой таксономией. Но можно пойти дальше и ввести категории требований, расширяя таксономию, как показано на рисунке.

Иерархия типов требований



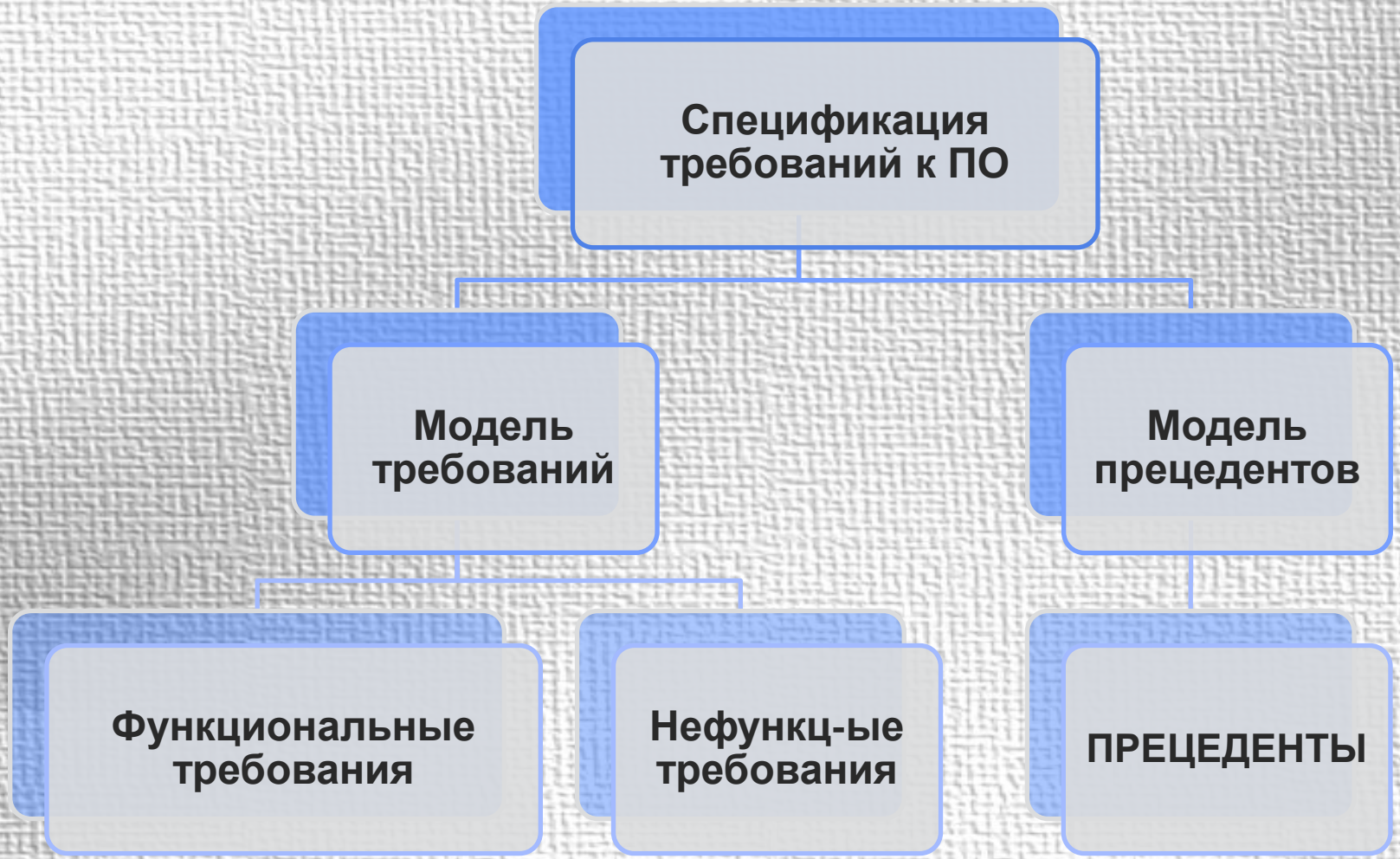
REQUIREMENT
VS
USE CASE

ПРАВИЛЬНО СФОРМИРОВАННЫЕ ТРЕБОВАНИЯ

UML не дает каких-либо рекомендаций по написанию традиционных требований. Фактически требования в UML представляются исключительно через механизм прецедентов.

Однако многие разработчики моделей полагают, что прецедентов недостаточно и все же нужны традиционные инструментальные средства выработки и управления требованиями.

МЕТАМОДЕЛЬ ТРЕБОВАНИЙ, ПРЕДЪЯВЛЯЕМЫХ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ



Метамодель показывает, что Спецификация требований к программному обеспечению (Software requirements specification, SRS) включает

- **Модель требований** (Requirements model)
- **Модель прецедентов** (Use case model).

Эти две модели являются разными, но тем не менее взаимодополняющими способами представления требований, предъявляемых к системе.

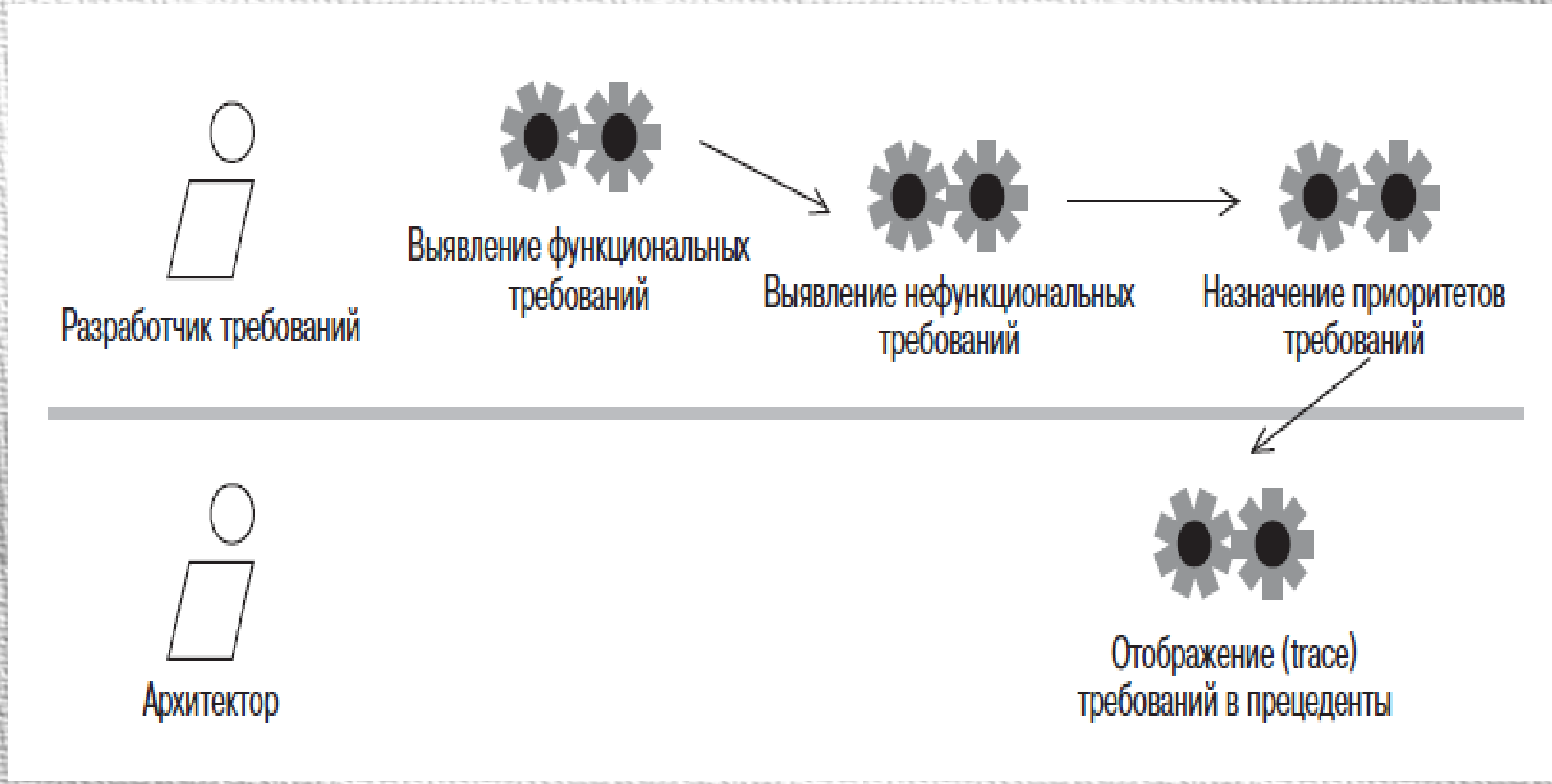
Модель прецедентов включает множество пакетов прецедентов, которые содержат прецеденты, актеров и отношения.

По сути, SRS – это начальная стадия процесса построения программного обеспечения. Это отправная точка ОО анализа и проектирования.

ПРОЦЕСС, НАПРАВЛЯЕМЫЙ ВАРИАНТАМИ ИСПОЛЬЗОВАНИЯ

Понятие UC, введенное в языке UML, является основой для выполнения всех этапов жизненного цикла, рассматриваемых в RUP. Понятие «business use case» (вид деятельности) является ключевым при бизнес-анализе. На этапах анализа требований, проектирования и реализации UC выступают в качестве вариантов использования системы (ВИ), являясь той «печкой», от которой «танцуют» аналитики и разработчики при выполнении проектирования и реализации ПС. При анализе требований, выделив ВИ, мы тем самым определяем требование или уровень иерархии требований к ПС. При детализации ВИ определяются объекты, и способы их взаимодействия, которые должны быть реализованы в программном коде.

Следует особо выделить роль ВИ в планировании итераций. Как определить, какая функциональность должна быть реализована в очередной итерации? Здесь на выручку приходят диаграммы ВИ. Каждому ВИ можно приписать приоритет, определяющий, в какой итерации его следует реализовать. Можно показать все ВИ, реализуемые в очередной итерации, на отдельной диаграмме (или нескольких диаграммах).


Разработчик требований
Выявление функциональных требований
Выявление нефункциональных требований
Назначение приоритетов требований
Архитектор
Отображение (trace) требований в прецеденты

RUP – процесс, основанный на архитектуре

В основе RUP лежит разработка ПС, предусматривающая разработку компонентов и сборку ПС из разработанных и готовых компонент. В основе любого проекта лежат кардинальные решения, определяющие принципы построения ПС, которые находят свое отражение в **архитектуре системы**. Разработка архитектуры ПС в RUP преследует следующие цели:

- *Описание организации ПС;*
- *Определение компонентов и их интерфейсов;*
- *Определение подсистем и объединение компонентов в подсистемы;*
- *Создание основы для повторного использования компонентов;*
- *Создание базиса для разработки;*
- *Контроль над проектом и поддержка целостности системы.*

ИТОГИ

- Диаграмма использования – первый шаг моделирования
- Основное назначение – показать, что делает система во внешнем мире
- Не обязательно соответствует структуре классов, модулей и компонентов
- Адекватная идентификация действующих лиц и вариантов использования – ключ к успеху
- Способ реализации – дело вкуса

Выводы

Из материала этой главы мы делаем следующие выводы: реализация вариантов использования диаграммами взаимодействия является наиболее трудоемким и сложным методом, но этот метод лучше всего согласован с объектно-ориентированным подходом и в наибольшей мере приближает нас к конечной цели, а

моделирование использования – это универсальный способ представления функциональных требований.

Контрольные вопросы

1. Что такое нефункциональные требования? Как они отображаются на диаграммах прецедентов?
2. Какие способы изображения актеров вы знаете?
3. В какие отношения могут вступать ДЛ между собой?
4. В чем состоит смысл отношений включения и расширения?
5. Что такое точка расширения?
6. Перечислите известные вам причины использования прецедентов.
7. Как прецеденты применяют в прямом и обратном проектировании?

МОДЕЛИРОВАНИЕ ПОВЕДЕНИЯ

- Диаграммы деятельности
- Диаграммы взаимодействия
- Диаграммы состояний

Модели поведения

При создании программной системы недостаточно ответить на вопросы что делает система? и из чего она состоит? — требуется ответить на вопрос **как работает система?**.

Ответ на этот вопрос дает модель поведения. Поведение реальной программы целиком и полностью определяется ее кодом — как программа составлена, так она и выполняется (с точностью до сбоев) — "от себя" компьютер ничего не придумывает. Но программа — это просто запись алгоритма. Таким образом, мы приходим к следующему определению.

Модель поведения (behavior model) – это описание алгоритма работы системы.

СРЕДСТВА МОДЕЛИРОВАНИЯ ПОВЕДЕНИЯ

В UML предусмотрено несколько различных средств для описания поведения. Выбор того или иного средства диктуется типом поведения, которое нужно описать.

Разделим все средства моделирования поведения в UML на четыре группы

1. **описание поведения с явным выделением состояний;**
2. **описание поведения с явным выделением потоков данных и управления;**
3. **описание поведения как последовательности сообщений во времени;**
4. **описание параллельного поведения.**

❶ Явное выделение состояний

При использовании объектно-ориентированного подхода, программная система представляет собой множество объектов, взаимодействующих друг с другом.

При этом возможна ситуация, когда поведение некоторых объектов разумно рассматривать в терминах их жизненного цикла, т.е. текущее поведение объекта определяется его историей.

Жизненный цикл (lifecycle) — последовательность изменений состояния объекта.

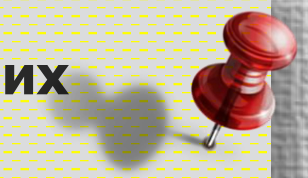
❷ Поток управления и поток данных

При использовании

Поток управления – это последовательность выполнения операторов (команд) в программе.

Помимо потока управления в UML также используется поток данных.

Поток данных – это описание связи выходных результатов одних действий с входными аргументами других действий.



❸ Поведение как последовательности сообщений во времени

При использовании объектно-ориентированного подхода, программная

4 Параллельное поведение

При использовании объектно-ориентированного подхода, программная

Для построения поведенческих моделей в языке UML 2.0 существуют следующие средства:

1. **взаимодействия** (*interactions*), семантика которых задает отношение частичной упорядоченности событий в различных компонентах системы и актеров
2. **активности** (*activities*), семантически эквивалентные иерархическим раскрашенным сетям Петри
3. **машины состояний** (*state machines*), использующие семантику расширенных конечных автоматов в алфавите событий системы.

- Элементы ДД
- Использование ДД

ДИАГРАММА ДЕЯТЕЛЬНОСТИ
(ACTIVITY DIAGRAM)

ОТОБРАЖЕНИЕ РАБОЧЕГО ПРОЦЕССА МЕЖДУ ПОЛЬЗОВАТЕЛЯМИ И СИСТЕМОЙ

Для отображения рабочего процесса между разными вариантами использования можно использовать схему деятельности. Во многих случаях рекомендуется начинать создание модели требований с составления схемы деятельности, на которой должны быть отображены основные задачи, выполняемые пользователем — как в системе, так и за ее пределами.

НАЗНАЧЕНИЕ

Выполнение прецедента описывается с помощью диаграмм видов деятельности, которые отображают исполнителей и последовательность выполнения соответствующих бизнес-процессов

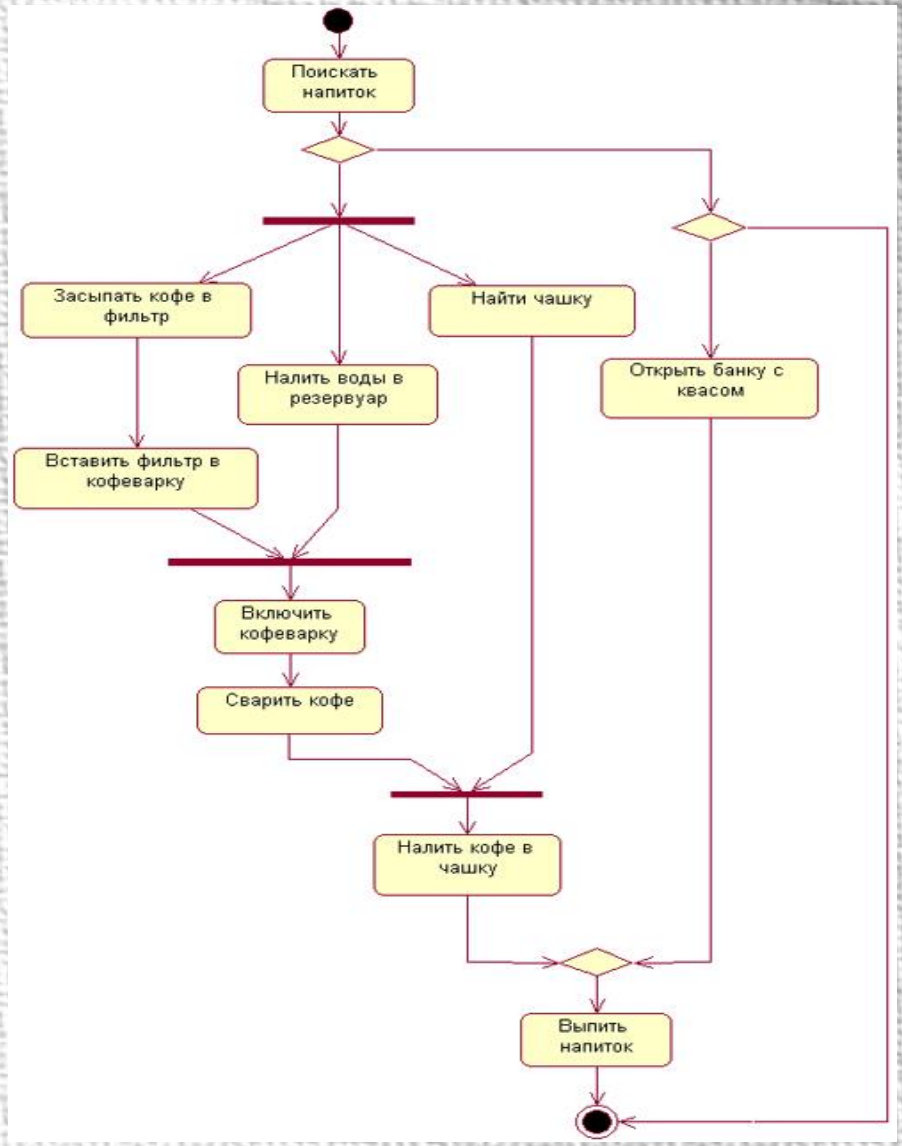
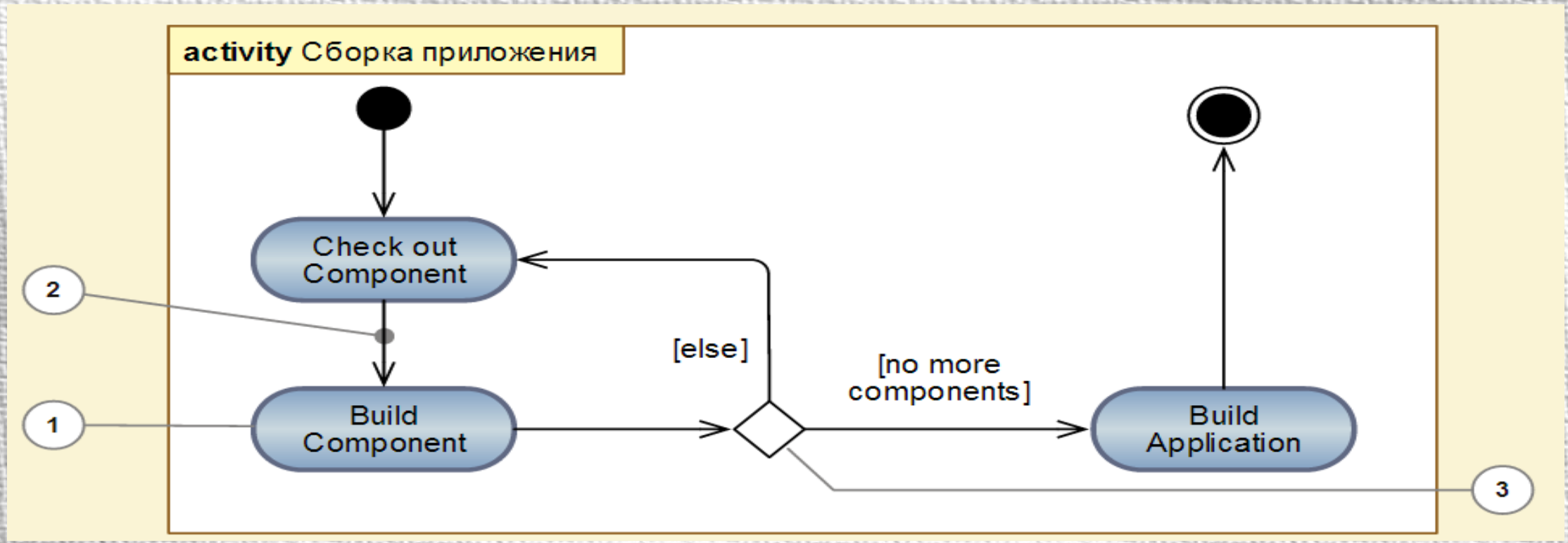


Диаграмма деятельности – еще один способ описания поведения, который визуально напоминает старую добрую блок-схему алгоритма. Однако за счет модернизированных обозначений, согласованных с объектно-ориентированным подходом, а главное, за счет новой семантической составляющей (свободная интерпретация сетей Петри), диаграмма деятельности UML является мощным средством для описания поведения системы.



На диаграмме деятельности применяют один основной тип сущностей – действие **1**, и один тип отношений – переходы **2** (передачи управления и данных). Также используются такие конструкции как развилки, слияния, соединения, ветвления **3**, которые похожи на сущности, но таковыми на самом деле не являются, а представляют собой графический способ изображения некоторых частных случаев многоместных отношений.

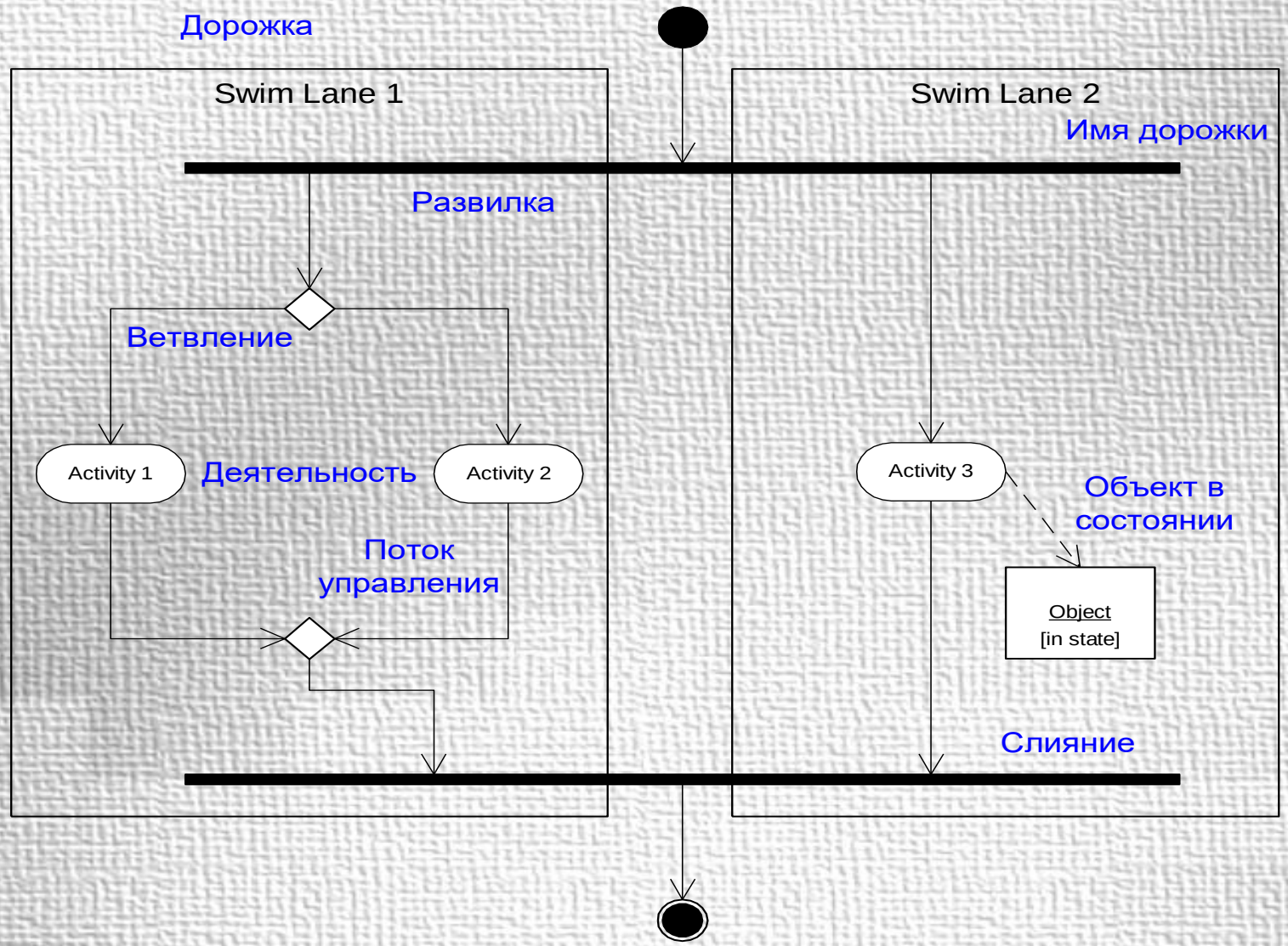
Элементы диаграммы деятельности

- Состояние деятельности:
  - простое состояние и
  - внутренняя активность (entry) является деятельностью
- Выражение деятельности
  - синтаксис не определен
- Переход
  - По завершении
  - Сегментированный
  - Со сторожевыми условиями
  - С действиями

Элементы диаграммы деятельности

- Главной сущностью этого типа диаграмм является **активность** (activity) - активное состояние системы, в котором она выполняет некоторую работу. После ее завершения происходит переход в другую активность. Возможны и более сложные случаи переходов между активностями. Например, переход по событию.
- На диаграмме должны присутствовать символы начала (start) и конца (finish).
- Далее, на диаграмме может использоваться **параллельный разветвитель** (fork), который запускает несколько одновременно работающих веток. Такие ветки могут объединяться (все или только часть) конструкцией под названием **параллельный соединитель** (join).
- Наконец, на диаграмме могут использоваться **символы логического ветвления и логического соединения** (decision). На ветках, идущих из логического ветвления, обозначаются условия перехода.

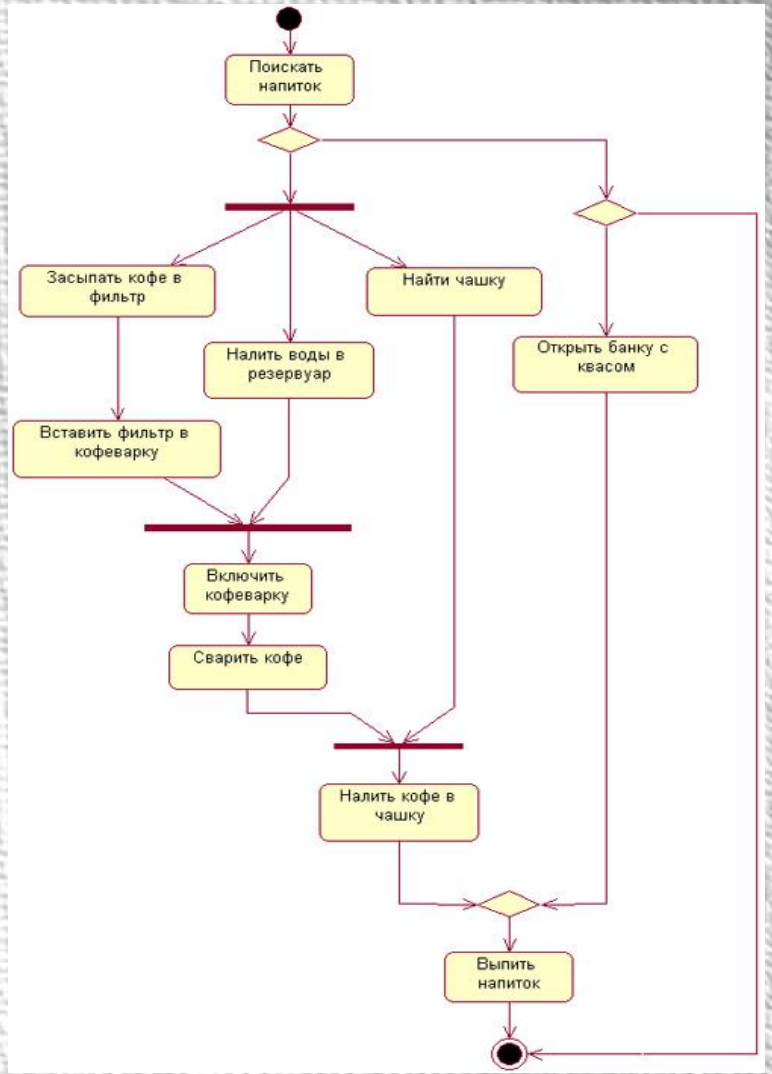
Нотация диаграммы деятельности

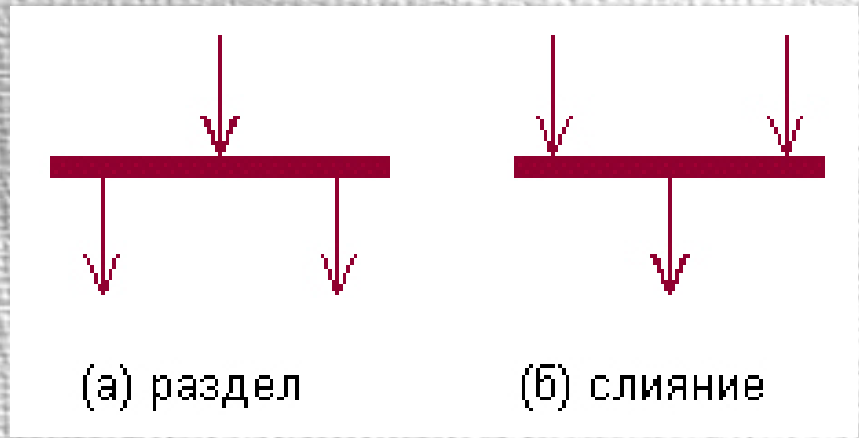


РАЗЛИЧНЫЕ ВАРИАНТЫ ВЕТВЛЕНИЙ НА ДИАГРАММЕ ДЕЯТЕЛЬНОСТИ

ДИАГРАММА ДЕЯТЕЛЬНОСТИ ДЛЯ ПРИМЕРА С ПРИГОТОВЛЕНИЕМ НАПИТКА

Для иллюстрации особенностей параллельных процессов выполнения действий рассмотрим ставший уже классическим пример с приготовлением напитка. Достоинство этого примера состоит в том, что он практически не требует никаких дополнительных пояснений в силу своей очевидности

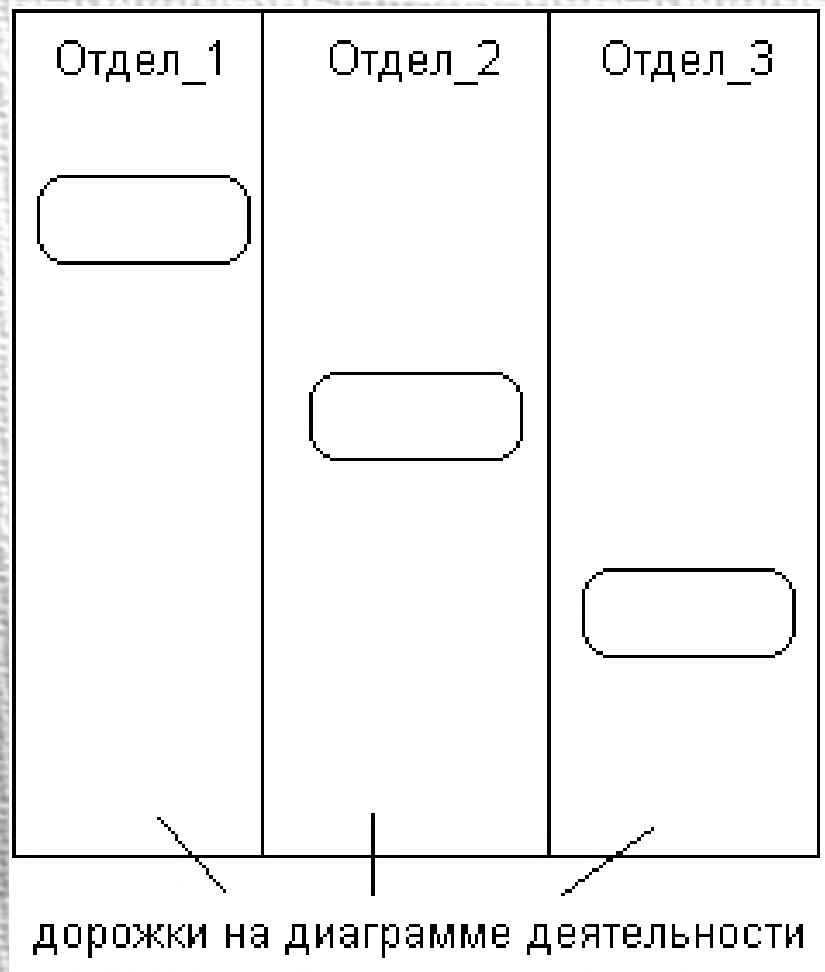




ДОРОЖКИ

Диаграммы деятельности могут быть использованы не только для спецификации алгоритмов вычислений или потоков управления в программных системах. Не менее важная область их применения связана с моделированием бизнес-процессов. Действительно, деятельность любой компании (фирмы) также представляет собой не что иное, как совокупность отдельных действий, направленных на достижение требуемого результата. Однако применительно к бизнес-процессам желательно выполнение каждого действия ассоциировать с конкретным подразделением компании. В этом случае подразделение несет ответственность за реализацию отдельных действий, а сам бизнес-процесс представляется в виде переходов действий из одного подразделения к другому.

Для моделирования этих особенностей в языке UML используется специальная конструкция, получившее название **дорожки** (swimlanes). Имеется в виду визуальная аналогия с плавательными дорожками в бассейне, если смотреть на соответствующую диаграмму. При этом все состояния действия на диаграмме деятельности делятся на отдельные группы, которые отделяются друг от друга вертикальными линиями. Две соседние линии и образуют дорожку, а группа состояний между этими линиями выполняется отдельным подразделением (отделом, группой, отделением, филиалом) компании



дорожки на диаграмме деятельности

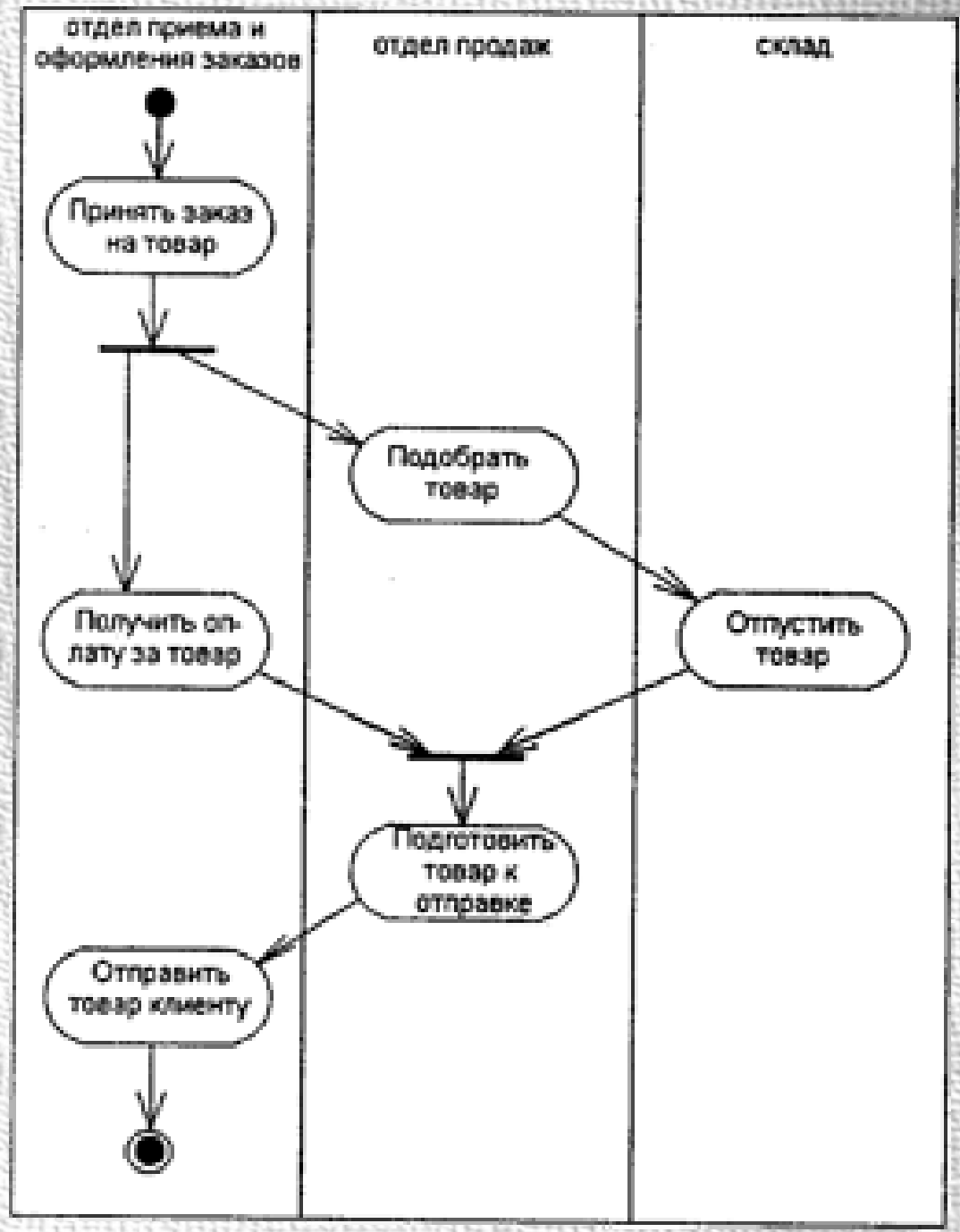
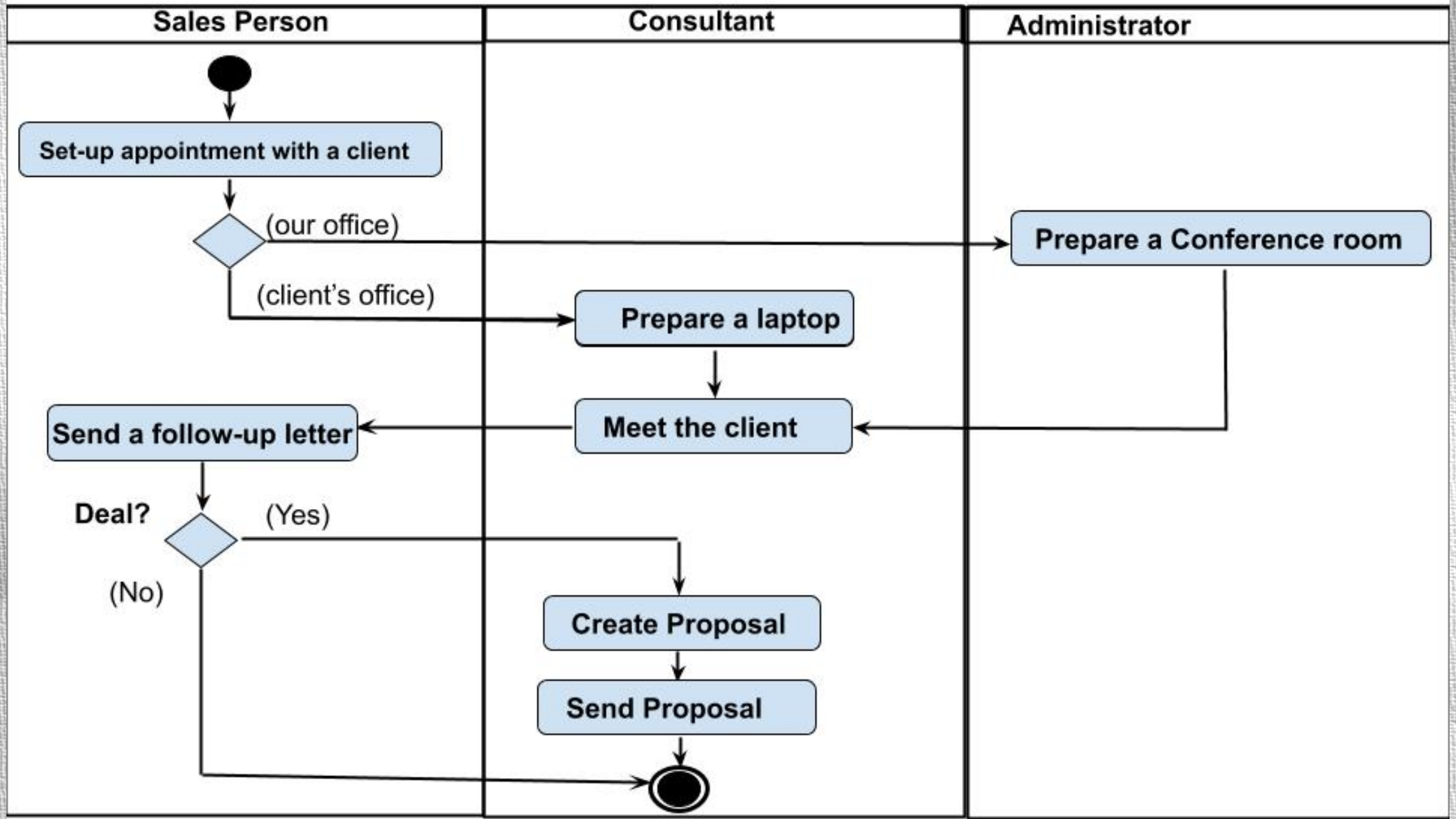


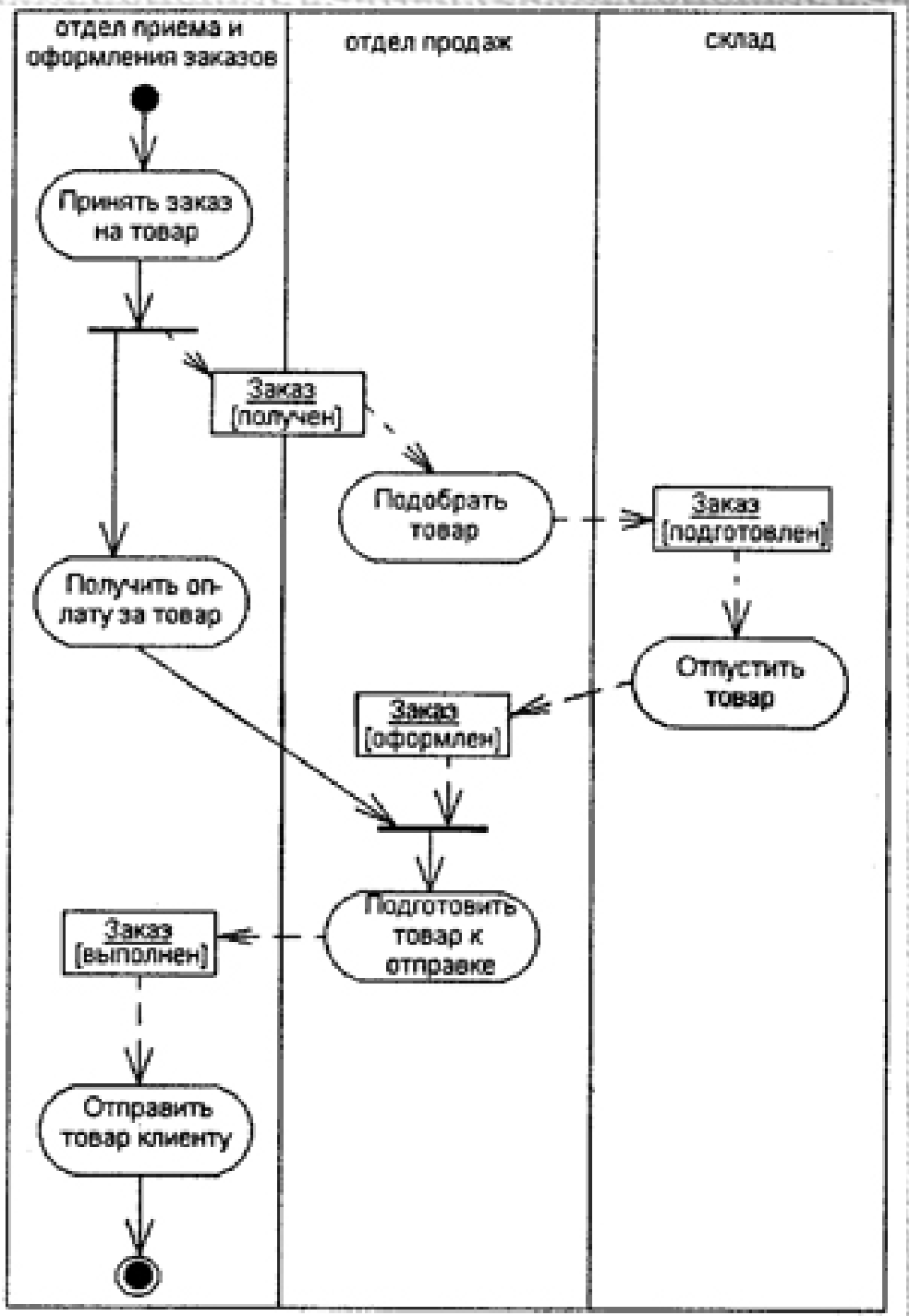
Диаграмма Деятельности



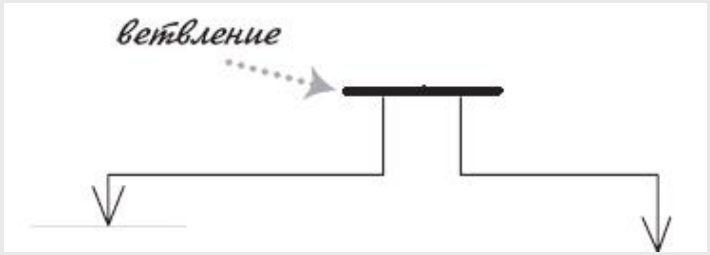
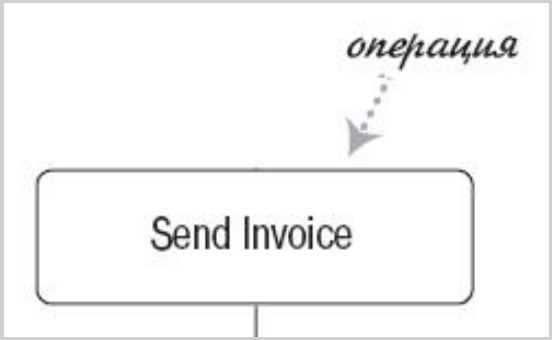
ОБЪЕКТЫ

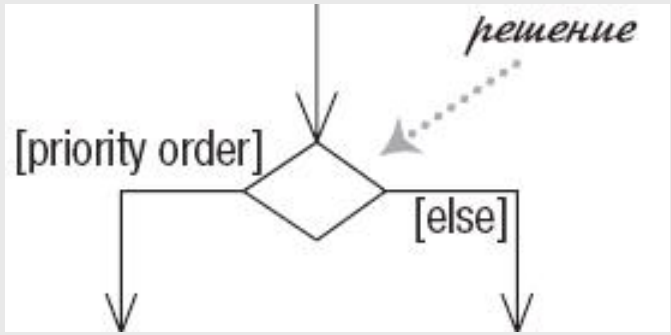
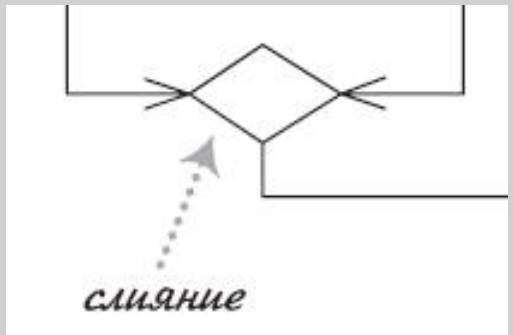
В общем случае действия на диаграмме деятельности выполняются над теми или иными объектами. Эти объекты либо инициируют выполнение действий, либо определяют некоторый результат этих действий. При этом действия специфицируют вызовы, которые передаются от одного объекта графа деятельности к другому. Поскольку в таком ракурсе объекты играют определенную роль в понимании процесса деятельности, иногда возникает необходимость явно указать их на диаграмме деятельности.

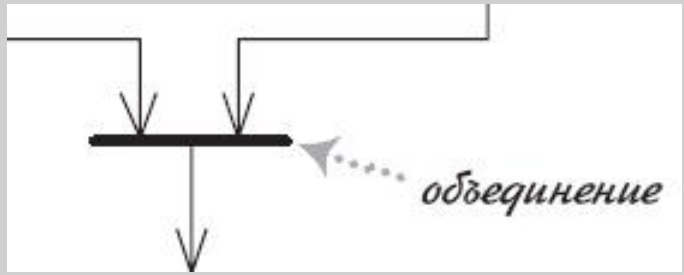
Фрагмент диаграммы деятельности торговой компании с объектом-заказом

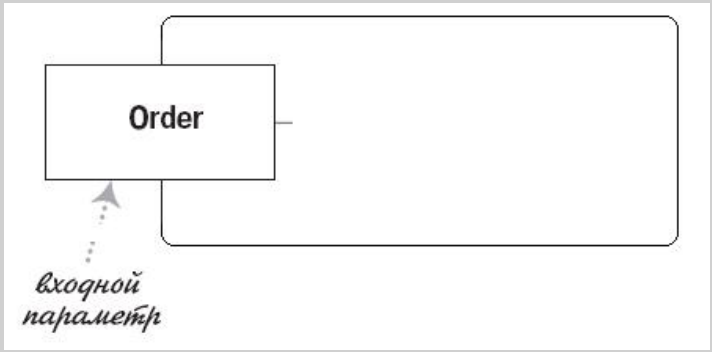
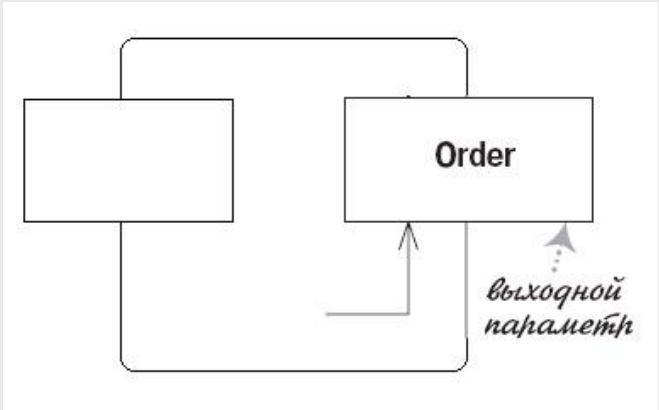
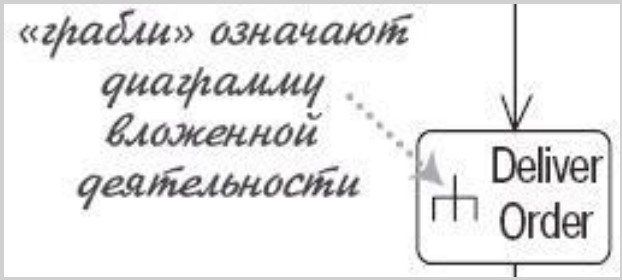


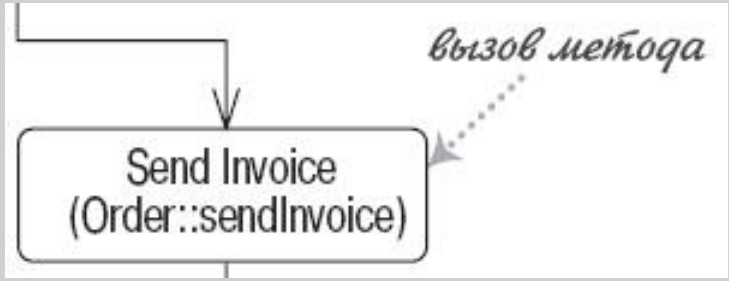
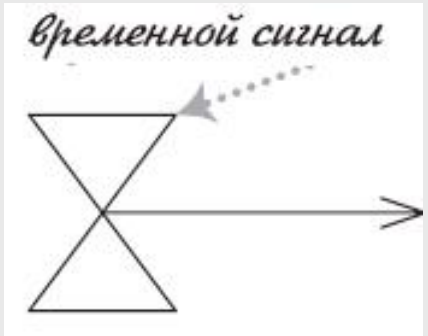
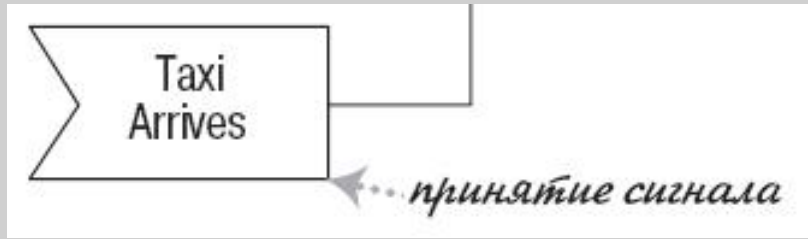
Основной набор символов диаграммы деятельности

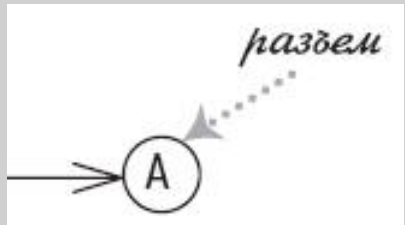
Термин	Изображение	Описание
Начальный узел		Старт диаграммы
Ветвление		Ветвление (fork), имеет один входной поток и несколько выходных параллельных потоков.
Операция		

Поток / ребро		В UML 2 параллельно употребляются термины поток (flow) и ребро (edge) для обозначения связи между двумя операциями. Самый простой вид ребра – это обычная стрелка между двумя операциями.
Решение (Ветвление)		Каждый раз при достижении решения выбирается только один из выходных потоков, поэтому защиты должны быть взаимно исключающими.
Слияние		

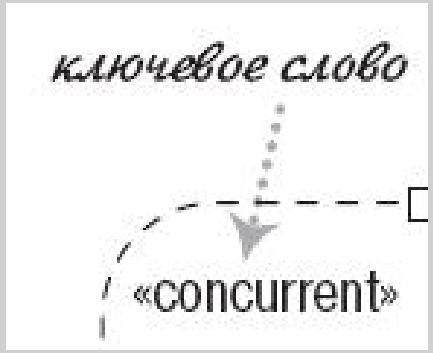
Объединение		При наличии параллелизма необходима синхронизация. Мы не закрываем заказ, пока он не оплачен и не доставлен. Это показывается с помощью объединения
Конец деятельности		Окончание диаграммы
Имя деятельности		

Входной параметр		
Выходной параметр		
"Грабли"		Операции могут быть реализованы или как вложенные деятельности или как методы классов. Вложенную деятельность можно обозначить с помощью символа «граблей».

Вызов метода		
Временной сигнал		Временной сигнал (time signal) приходит по прошествии времени. Такие сигналы могут означать конец месяца в отчетном периоде или приходить каждую секунду в контроллере реального времени.
Принятие сигнала		О

Посылка сигнала		
Прием сигнала		
Разъем		O

Узел объекта		
Контакт		Процедуры, как и методы, могут иметь параметры. Показывать на диаграмме деятельности информацию о параметрах не обязательно, но при желании можно отобразить параметры с помощью контактов (pins)
Выражение преобразования		Если требуется нарисовать точную диаграмму деятельности, то необходимо обеспечить соответствие выходных параметров одной процедуры входным параметрам другой. Если они не совпадают, то можно указать преобразование (transformation) (рис. 11.8) для перехода от одной процедуры к другой.

Ключевое слово		
Выходной список контактов		
Область расширения		Область расширения (expansion region) отмечает область диаграммы деятельности, где операции выполняются один раз для каждого элемента коллекции.

Окончание потока		Окончание потока (flow final) означает завершение конкретного потока без завершения всей активности.
Описание объединения		Описание объединения (join specification) – это логическое выражение, присоединенное к объединению.

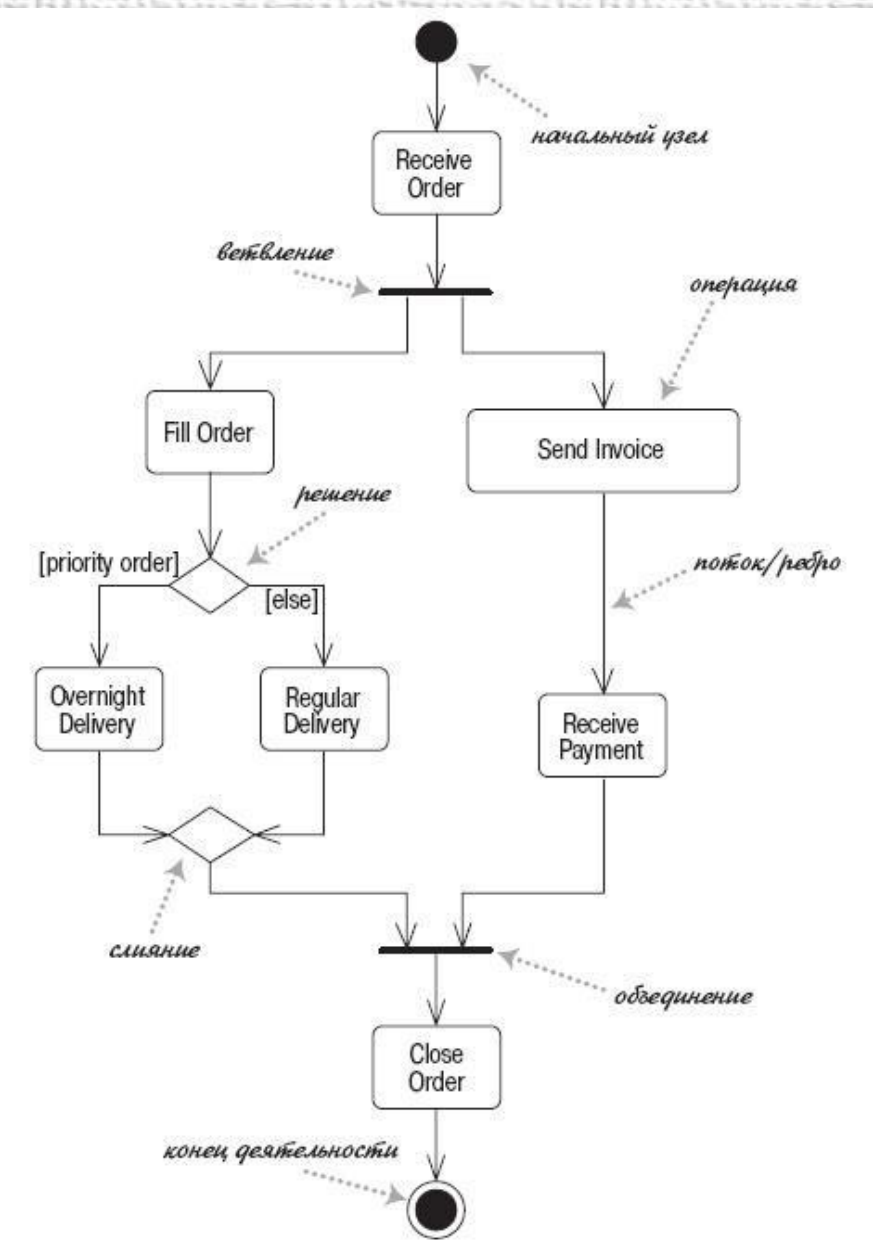


Рис. 11.1. Простая диаграмма деятельности

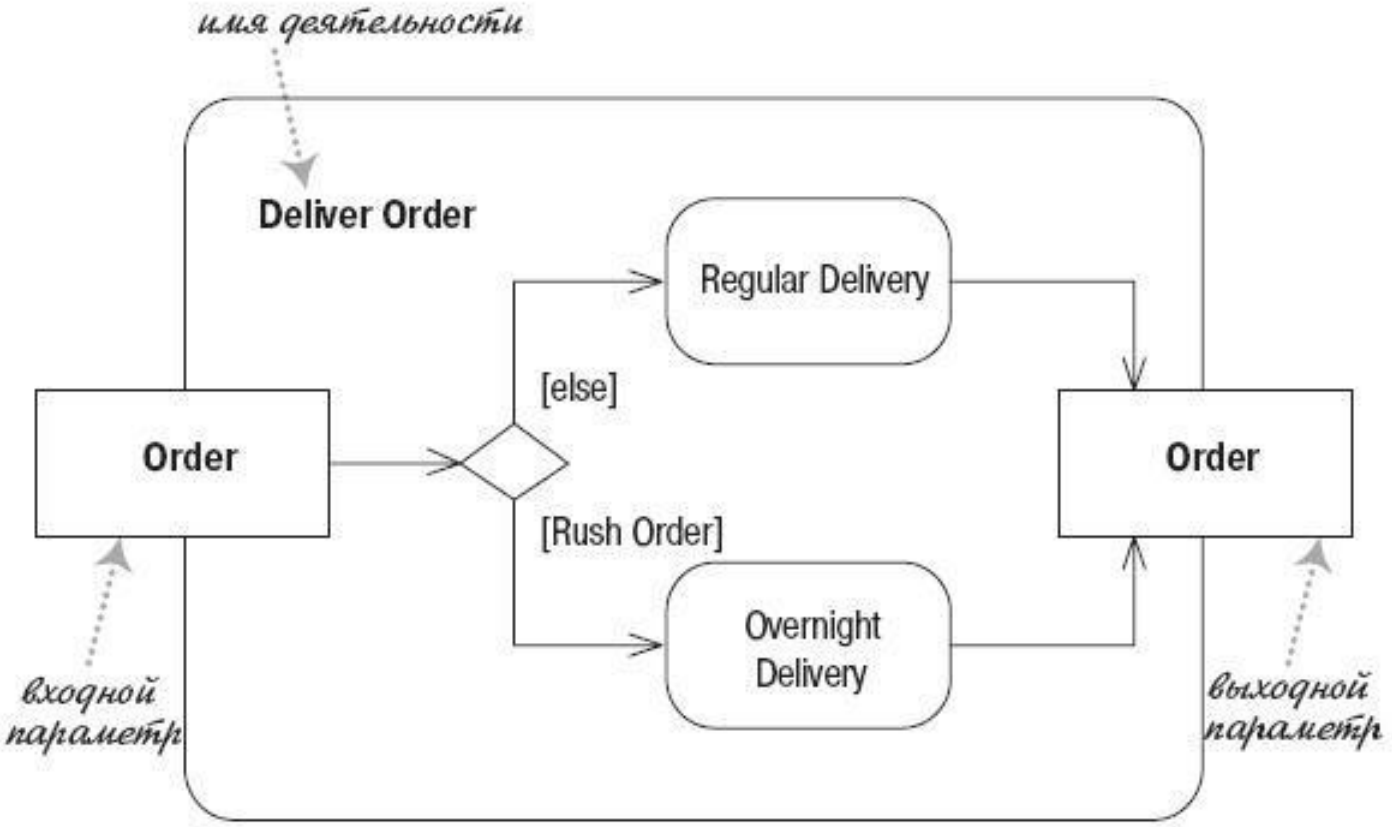


Рис. 11.2. Дополнительная диаграмма деятельности

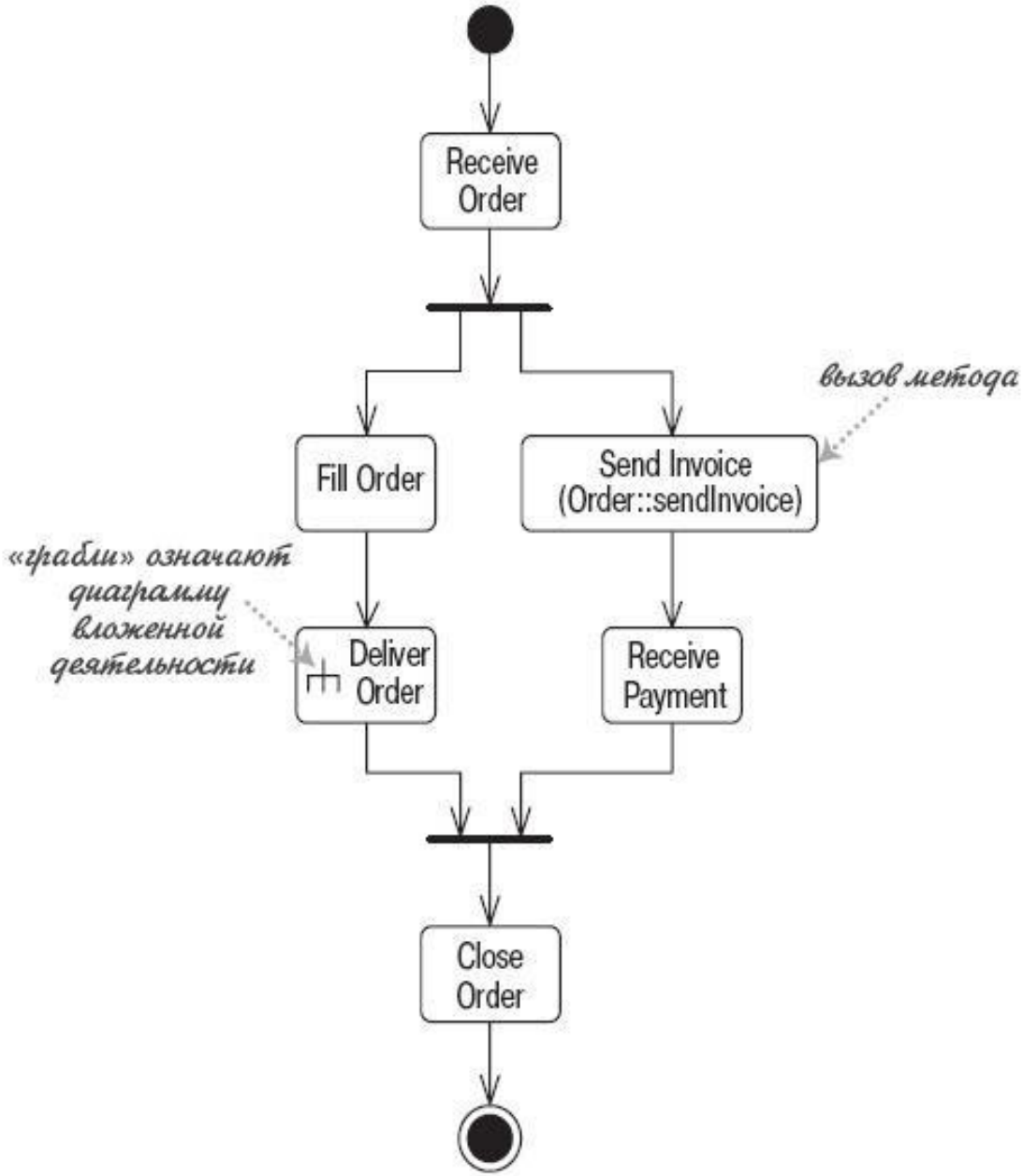


Рис. 11.3. *Деятельность из рис. 11.1 модифицирована для вызова деятельности из рис. 11.2*

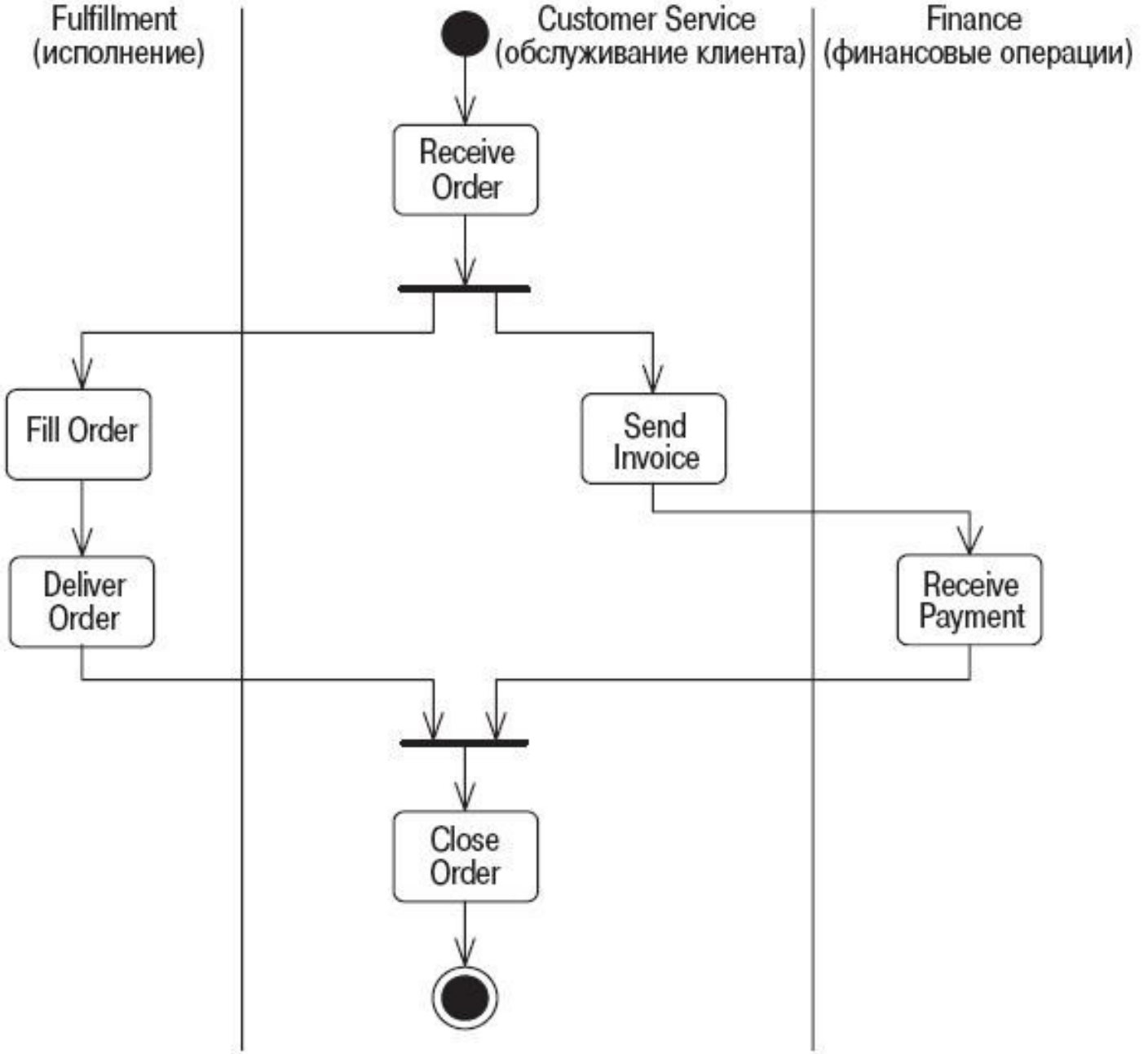


Рис. 11.4. *Разбиение диаграммы деятельности на разделы*

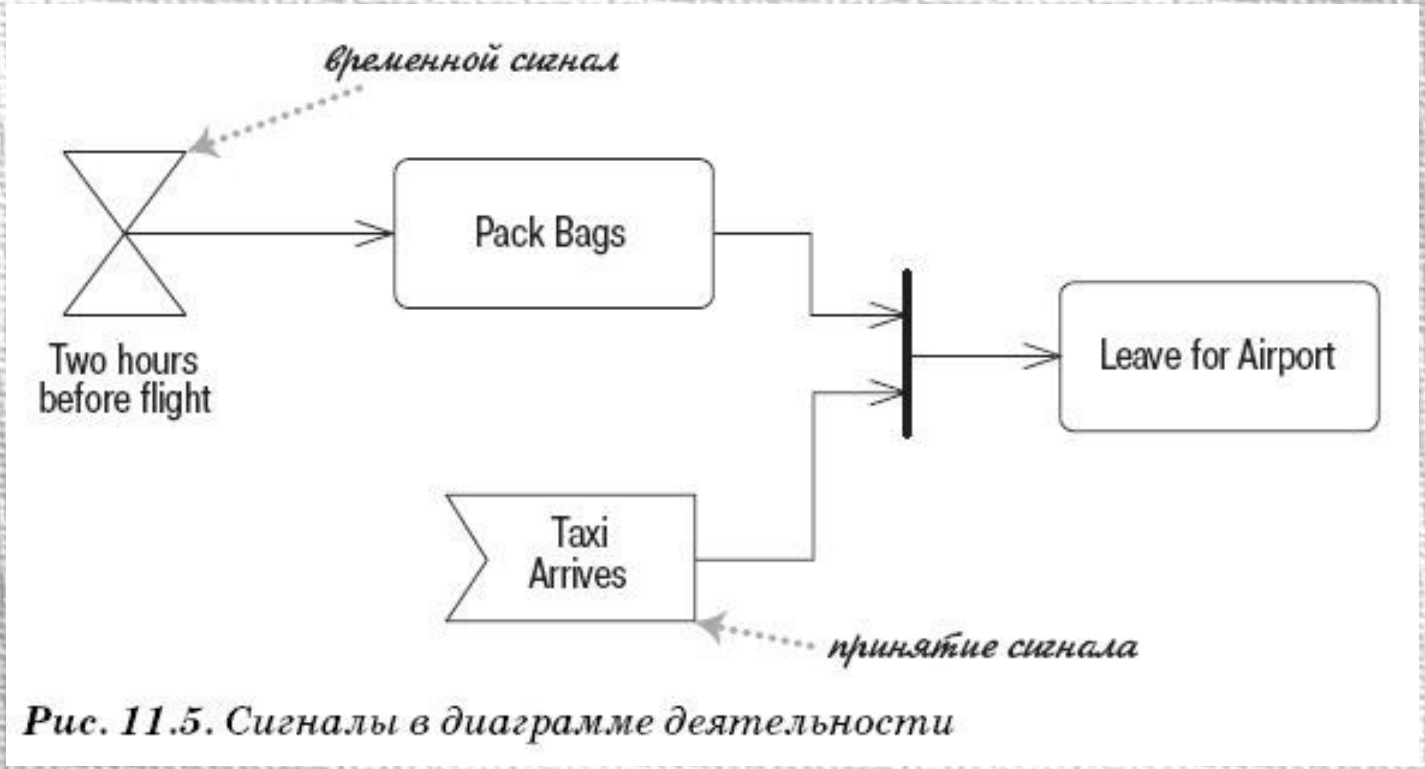


Рис. 11.5. Сигналы в диаграмме деятельности

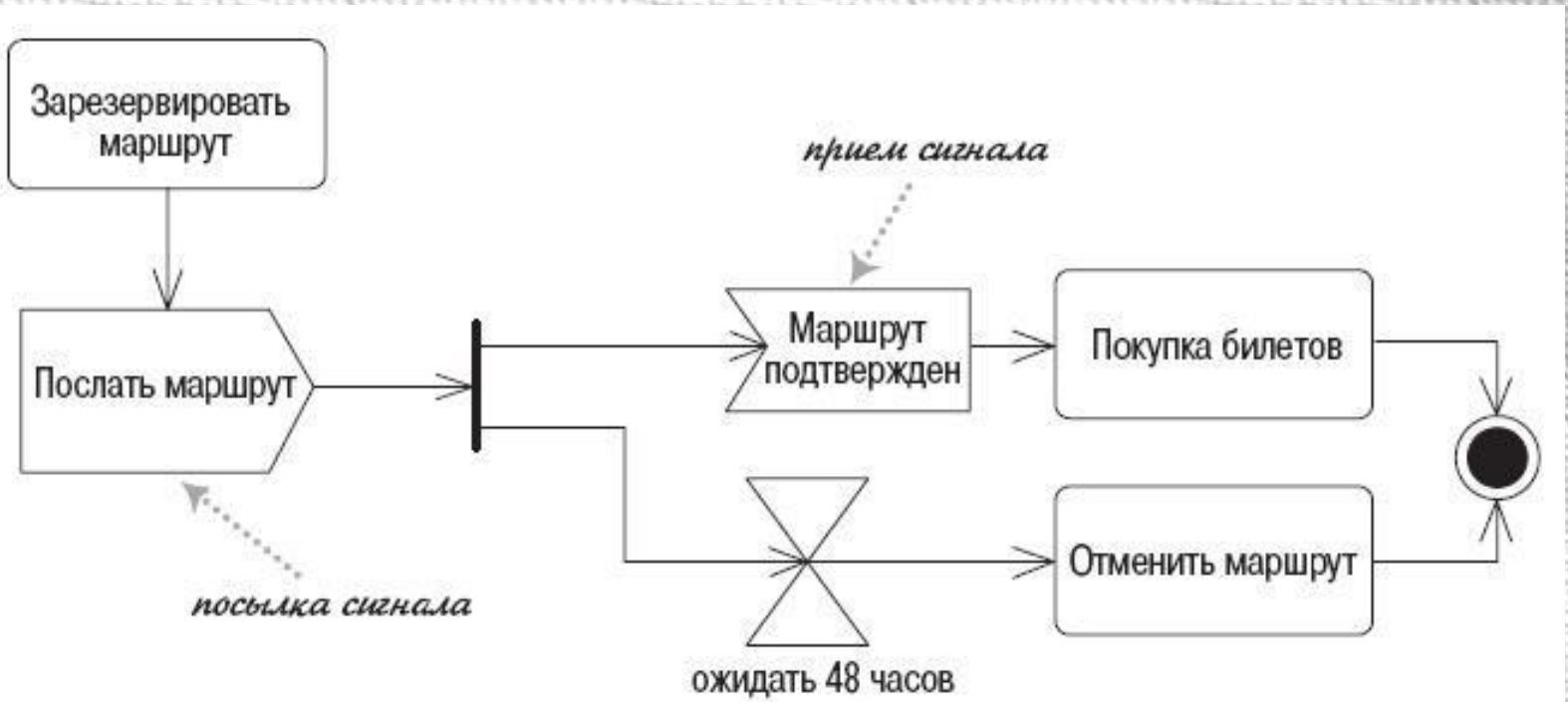


***Рис. 11.6.** Отправка и прием сигналов*

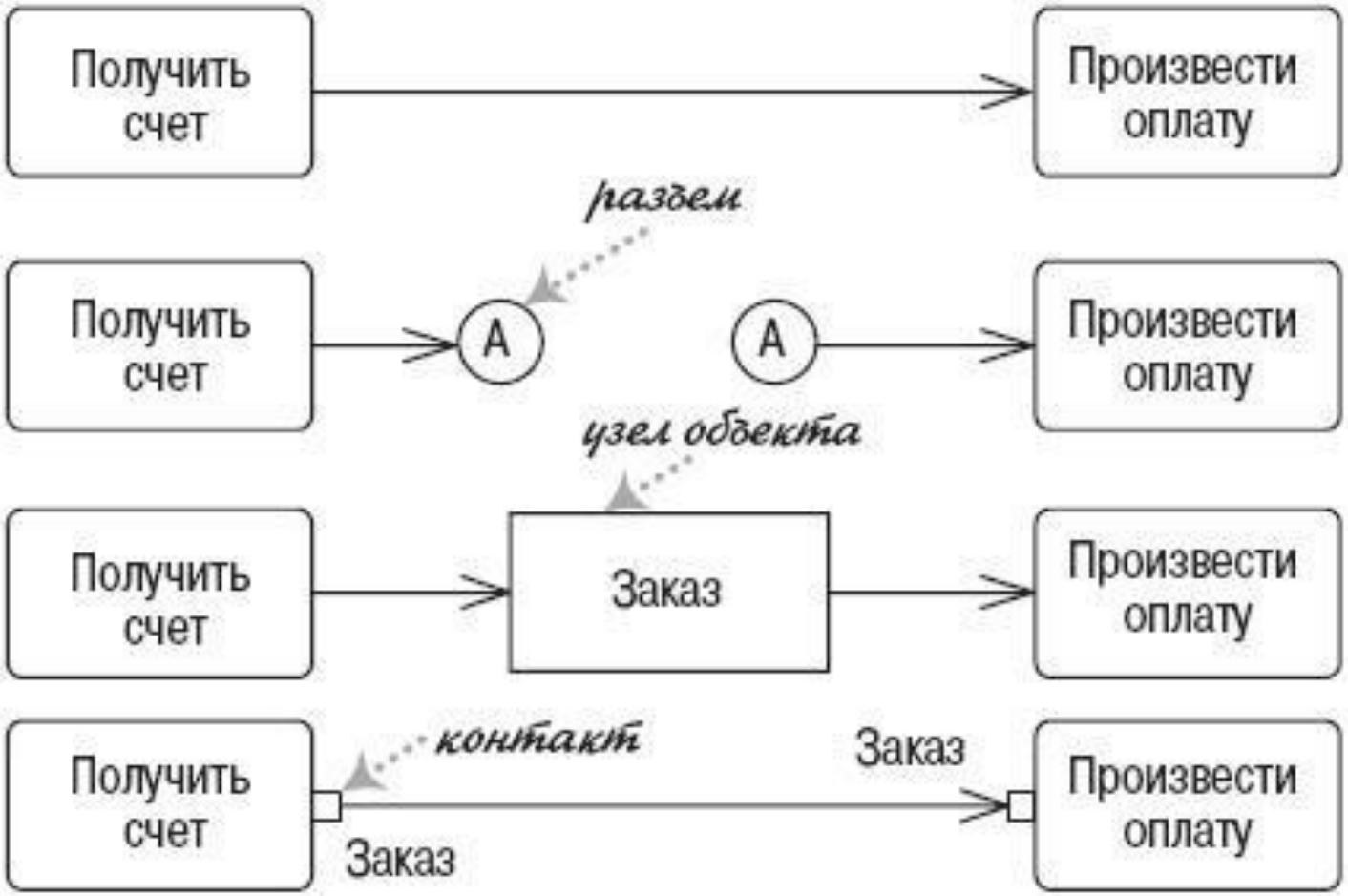


Рис. 11.7. *Четыре способа представления ребер*

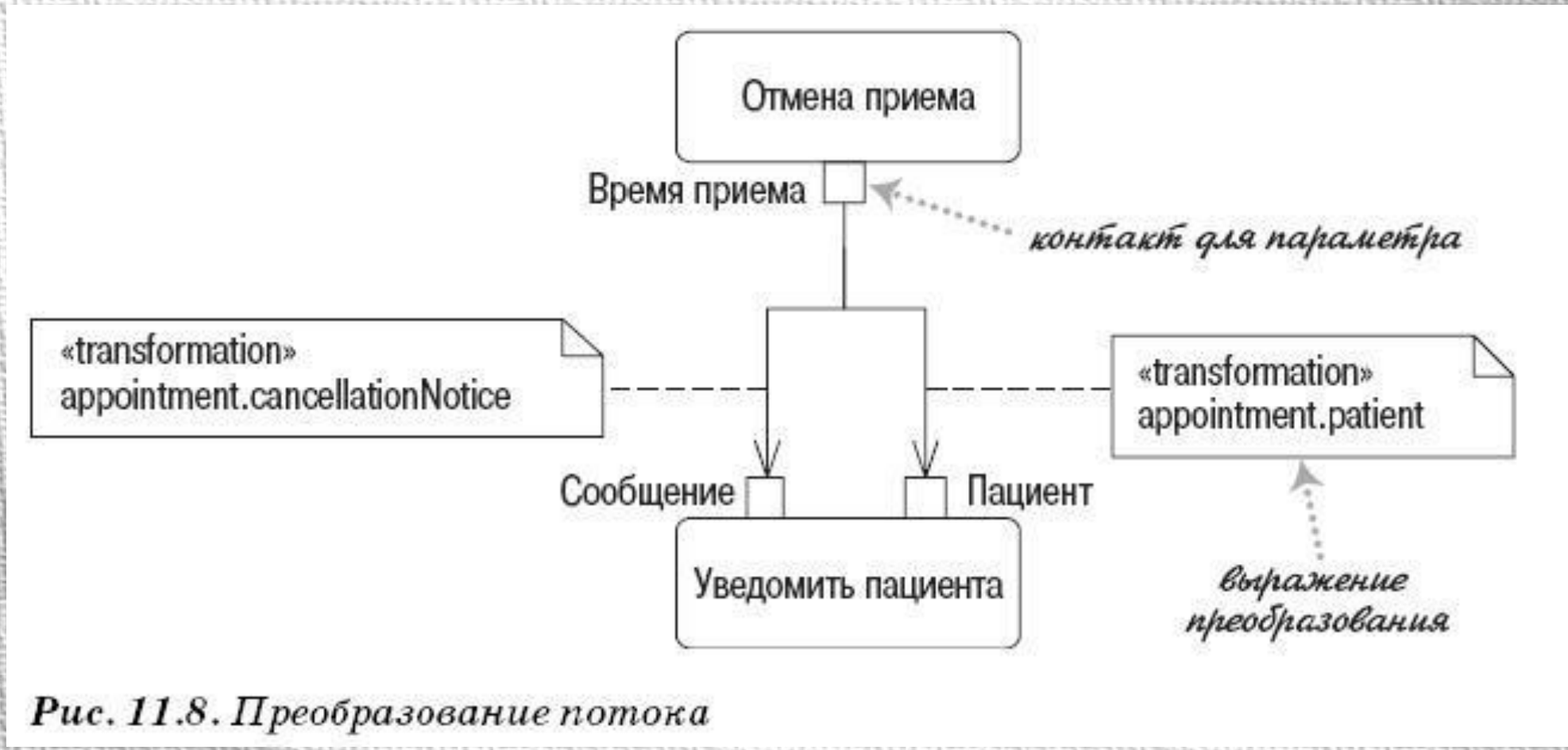


Рис. 11.8. Преобразование потока

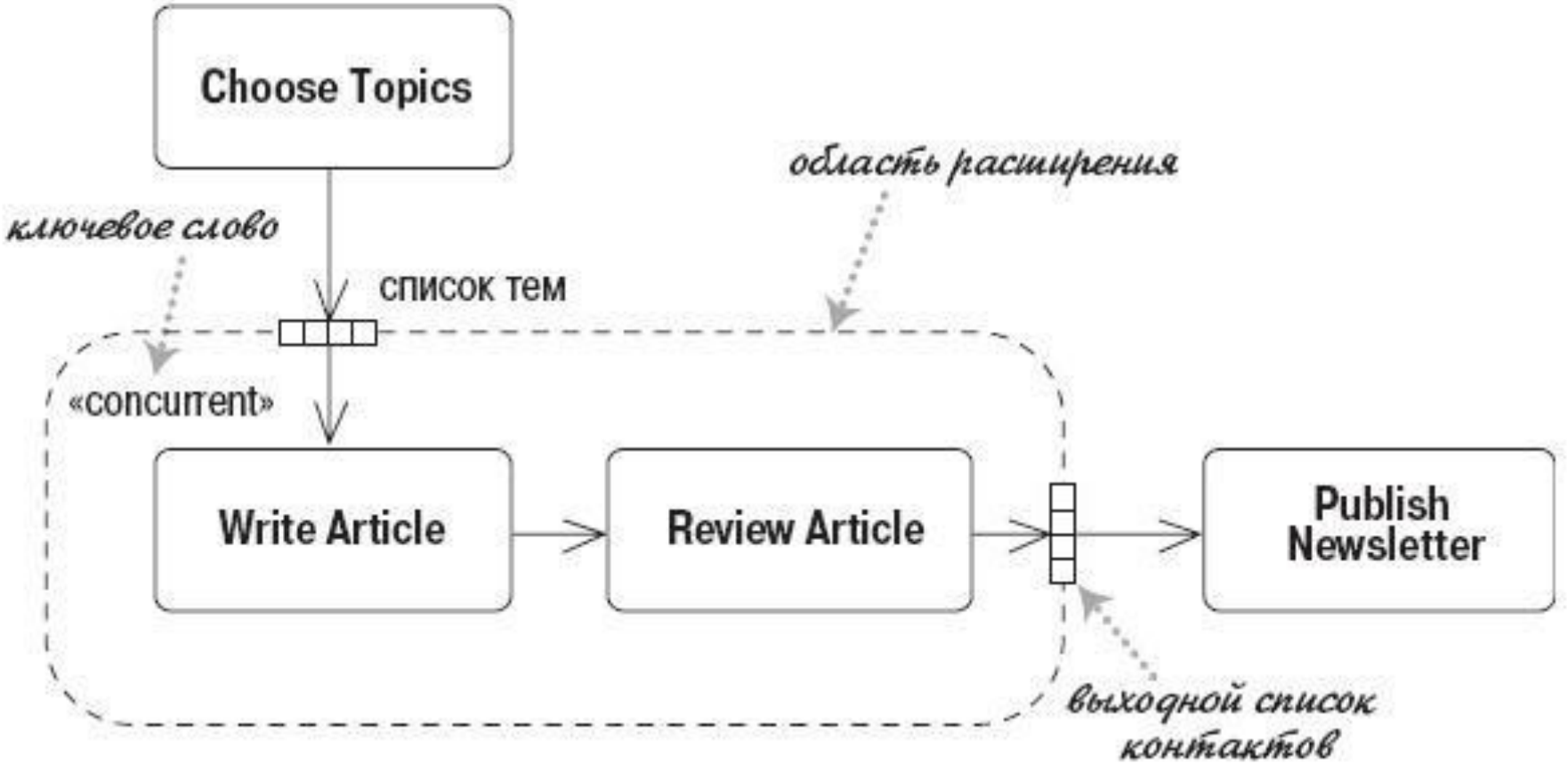


Рис. 11.9. Область расширения

Рис. 11.10. Нотация для единственной процедуры в области расширения

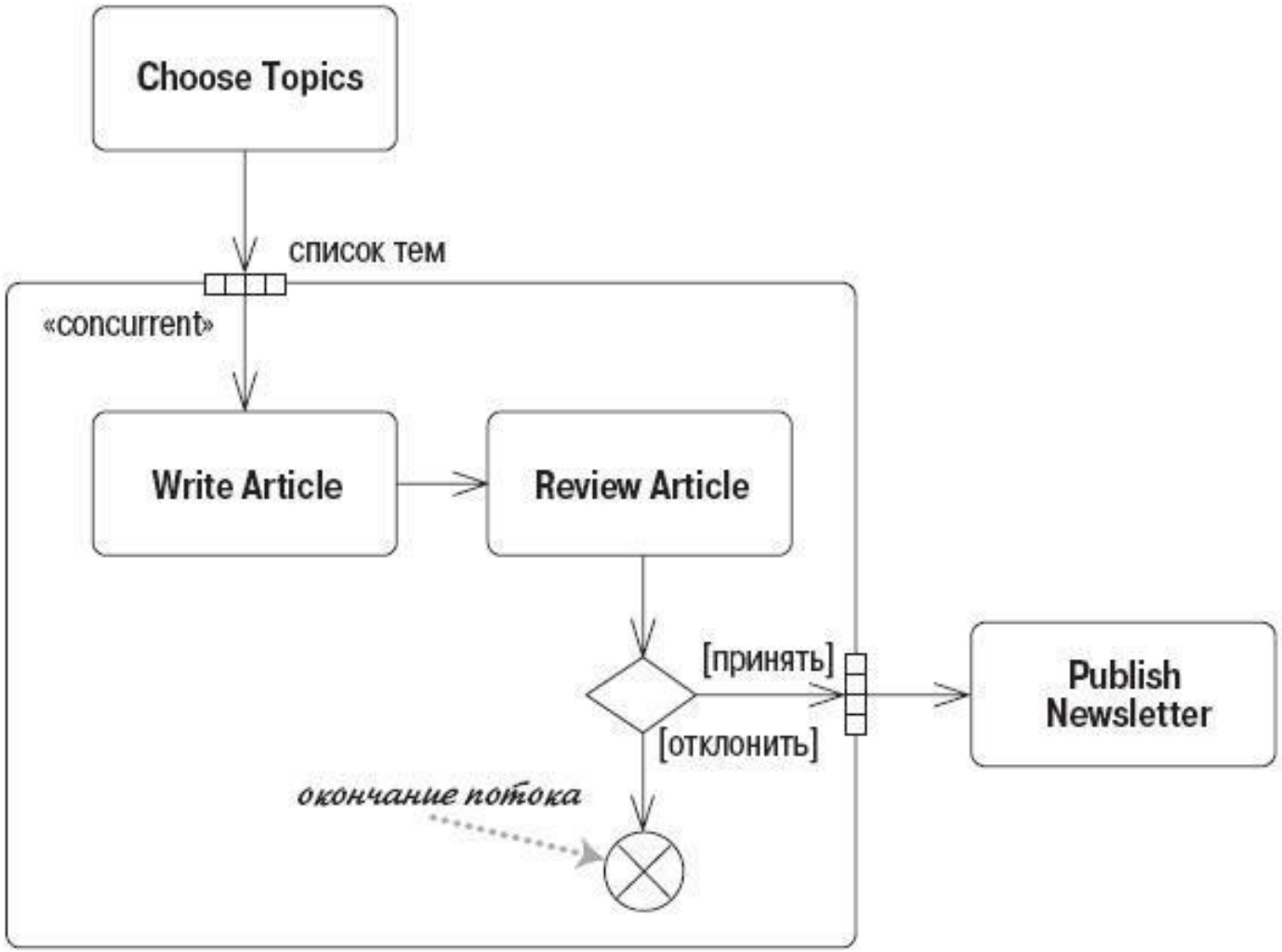


Рис. 11.11. Окончание потока в активности

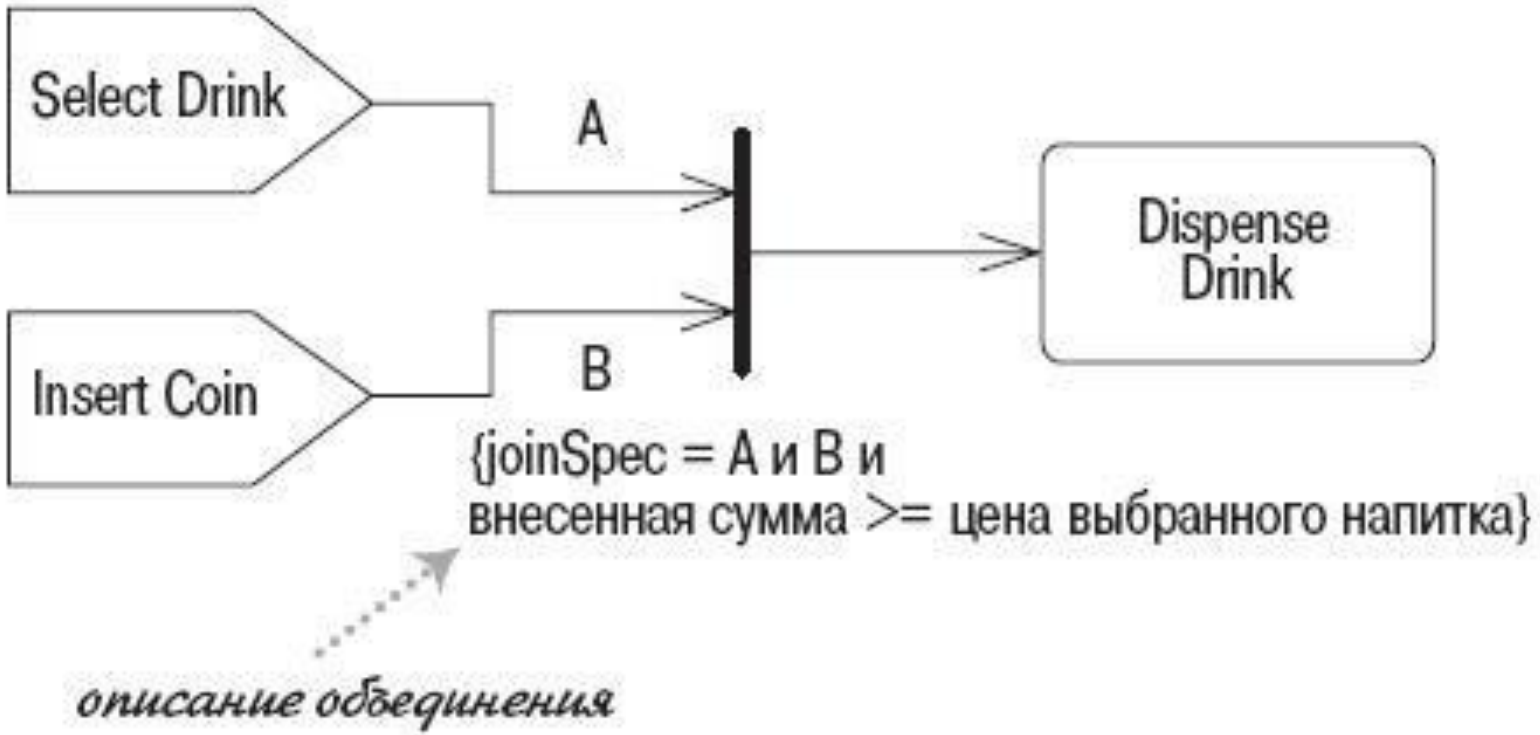


Рис. 11.12. Описание объединения

Где и как использовать ДД?

- Самое большое достоинство диаграмм деятельности заключается в том, что они поддерживают и стимулируют применение параллельных процессов. Именно благодаря этому они представляют собой мощное средство моделирования потоков работ.
- Можно также применять диаграмму деятельности в качестве совместимой с языком UML блок-схемы.
- Диаграммы деятельности иногда применяются для описания прецедентов.

Рекомендации по построению диаграмм деятельности

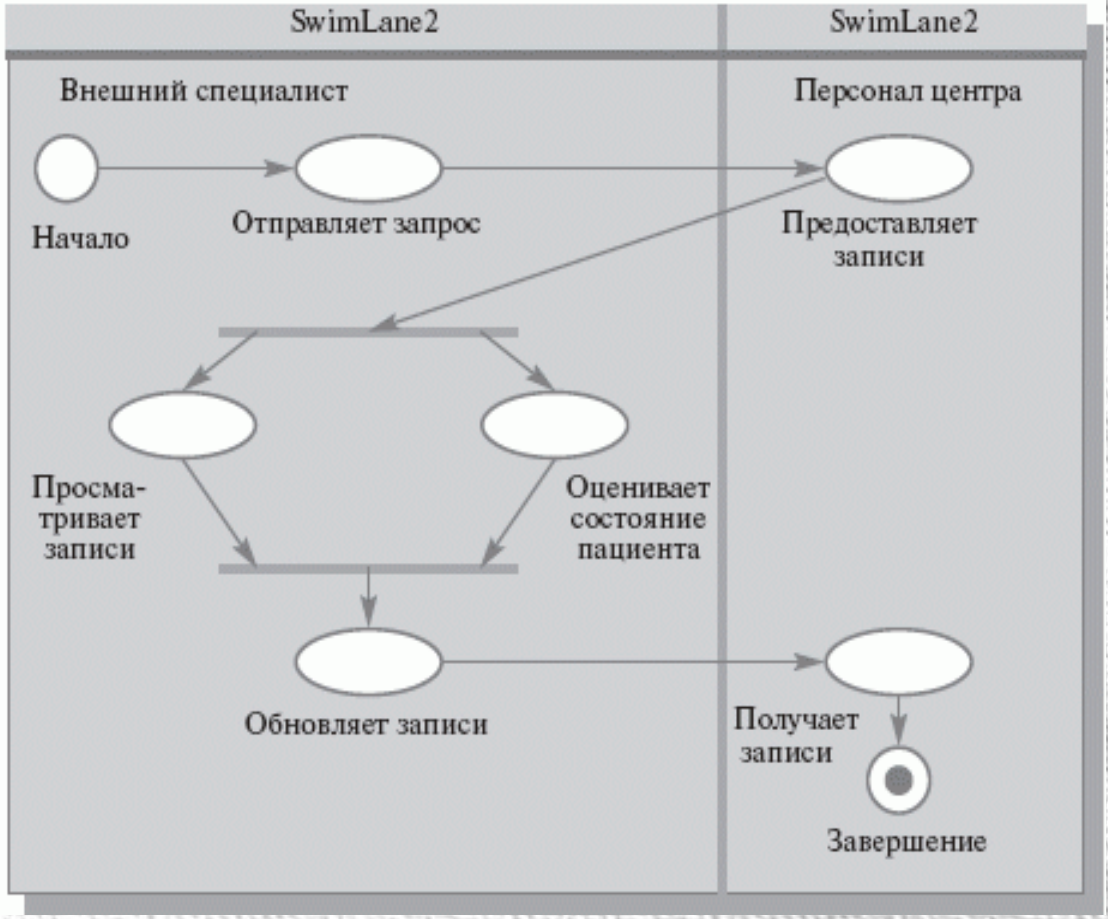
Диаграмма строится для отдельного класса, варианта использования, отдельной операции класса или целой подсистемы.

С одной стороны, на начальных этапах проектирования, когда детали реализации деятельностей в проектируемой системе неизвестны, построение диаграммы деятельности начинают с выделения под-деятельностей, которые в совокупности образуют деятельность подсистем. В последующем, по мере разработки диаграмм классов и состояний, эти под-деятельности уточняются в виде отдельных вложенных диаграмм деятельности компонентов подсистем, какими выступают классы и объекты.

Отображение прецедента на диаграмму деятельности

Модель бизнес-прецедентов, составляющих обслуживание пациента

Диаграмма видов деятельности для прецедента «Оказание медицинской помощи»



Телефонная обработка заявок

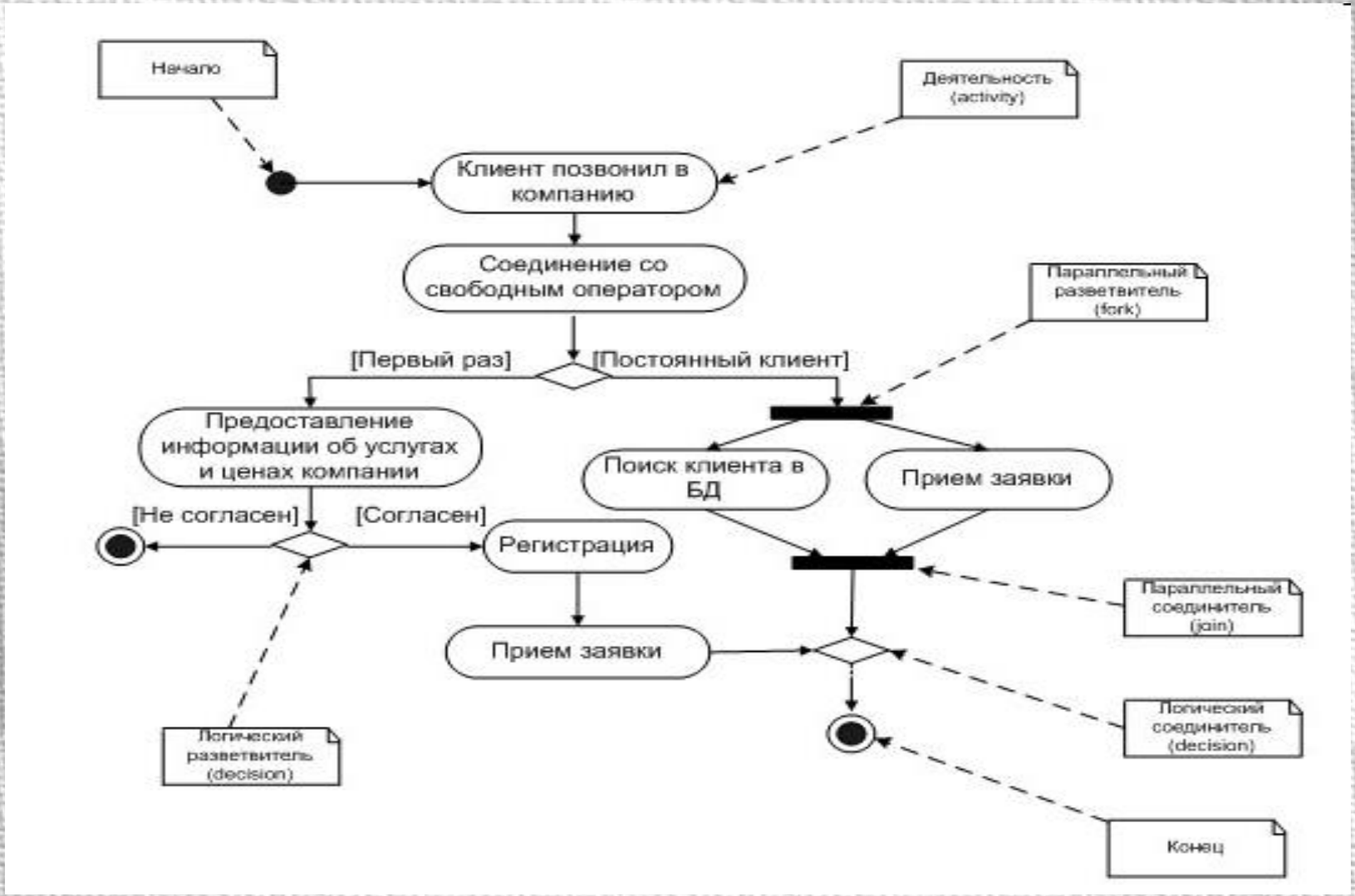
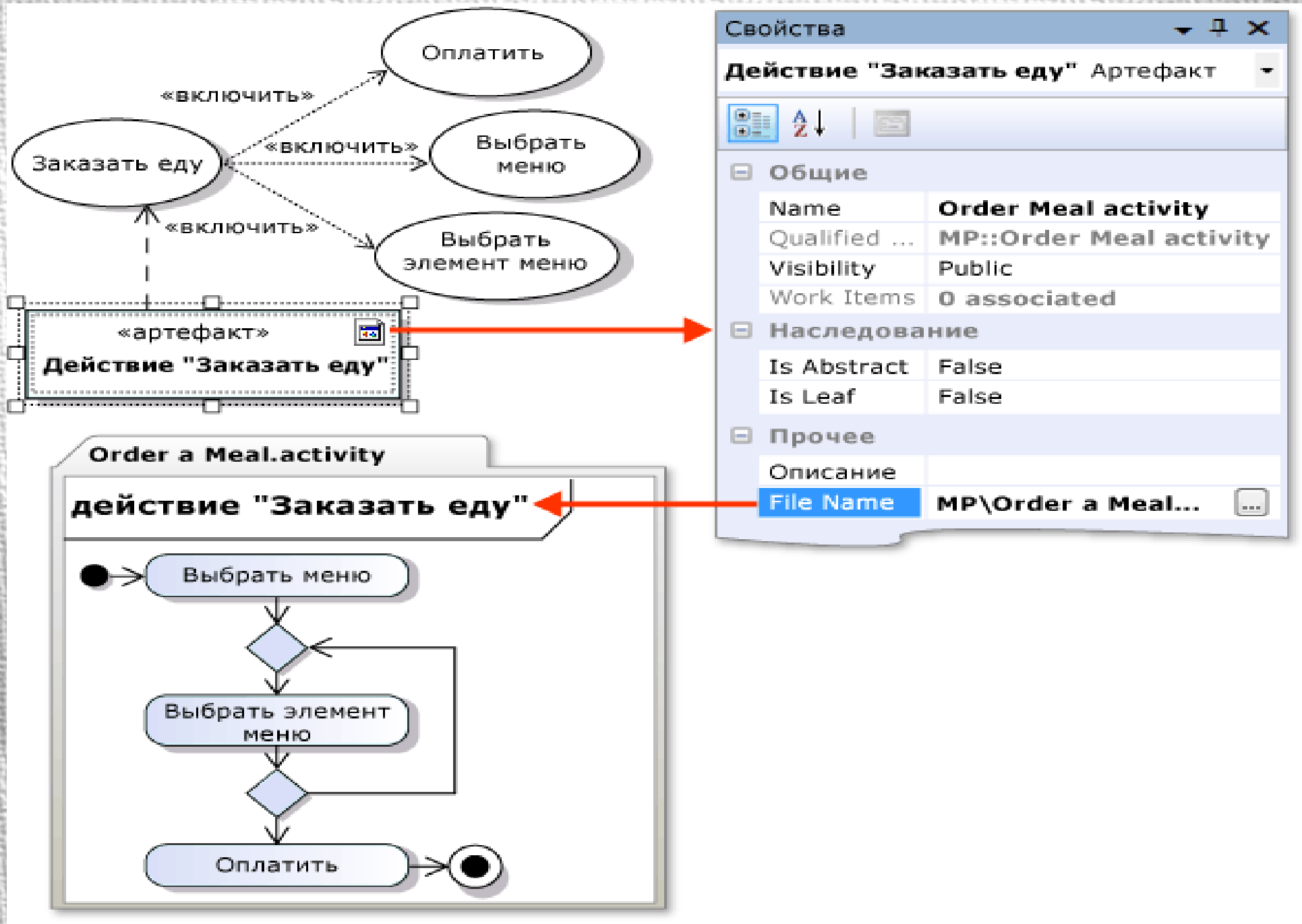


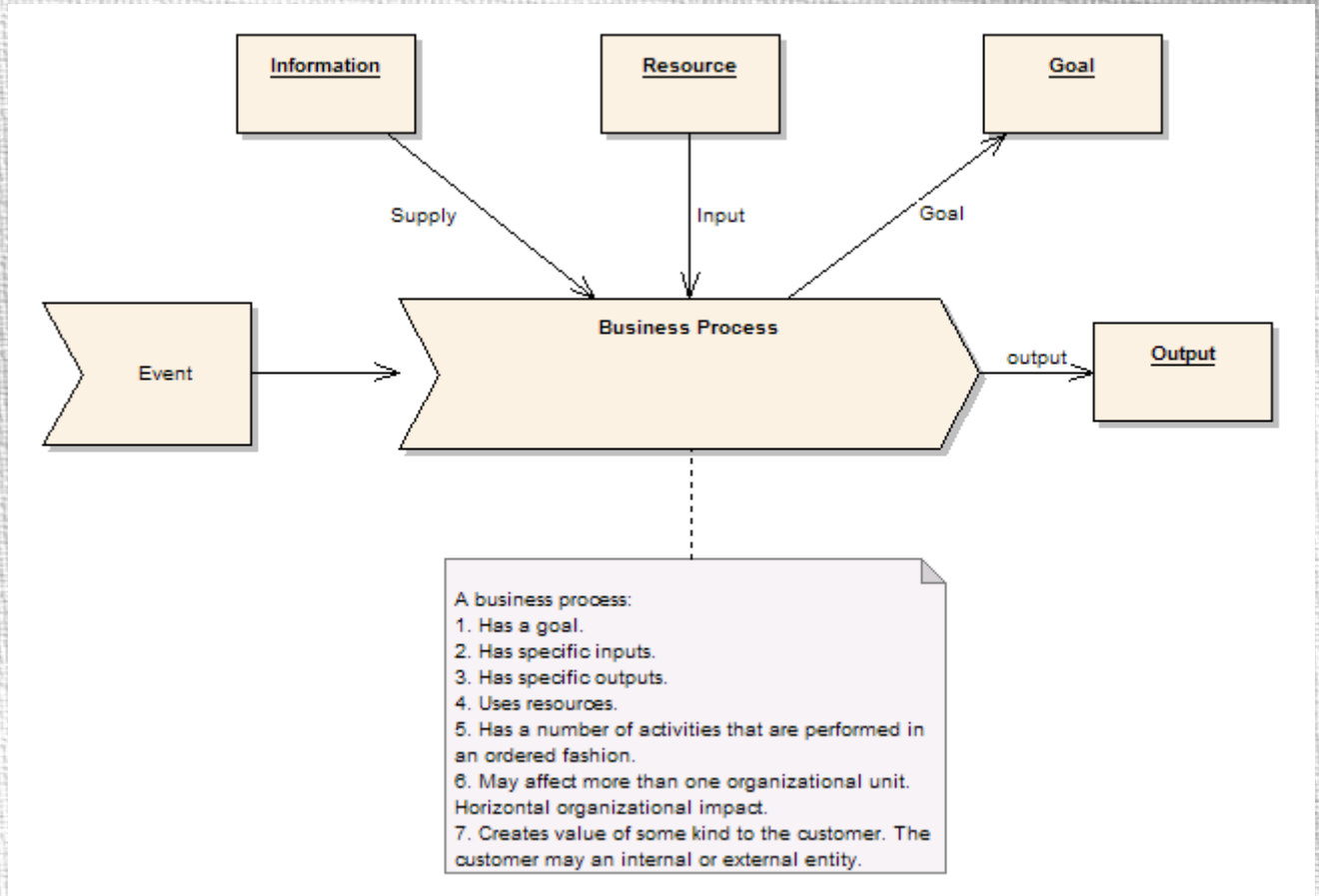
ДИАГРАММА ДЕЯТЕЛЬНОСТИ В VISUAL STUDIO 2013

- Схема вариантов использования никак не описывает последовательность совершения шагов и не сообщает, обязательно ли выполнять то или иное действие во всех вариантах использования.
- Чтобы прояснить порядок совершения шагов, можно использовать артефакт, чтобы прикрепить отдельный документ к включающему варианту использования. В следующем примере схема активности прикреплена к варианту использования "Заказ еды". Кроме того, можно использовать текстовый документ, включающий список шагов или последовательность снимков экрана.


Оплатить
«включить»
Заказать еду
«включить»
Выбрать меню
«включить»
Выбрать элемент меню
«артефакт»
Действие "Заказать еду"
Свойства
Действие "Заказать еду" Артефакт
Общие
Name
Order Meal activity
Qualified ...
MP::Order Meal activity
Visibility
Public
Work Items
0 associated
Наследование
Is Abstract
False
Is Leaf
False
Прочее
Описание
File Name
MP\Order a Meal...
Order a Meal.activity
действие "Заказать еду"
Выбрать меню
Выбрать элемент меню
Оплатить

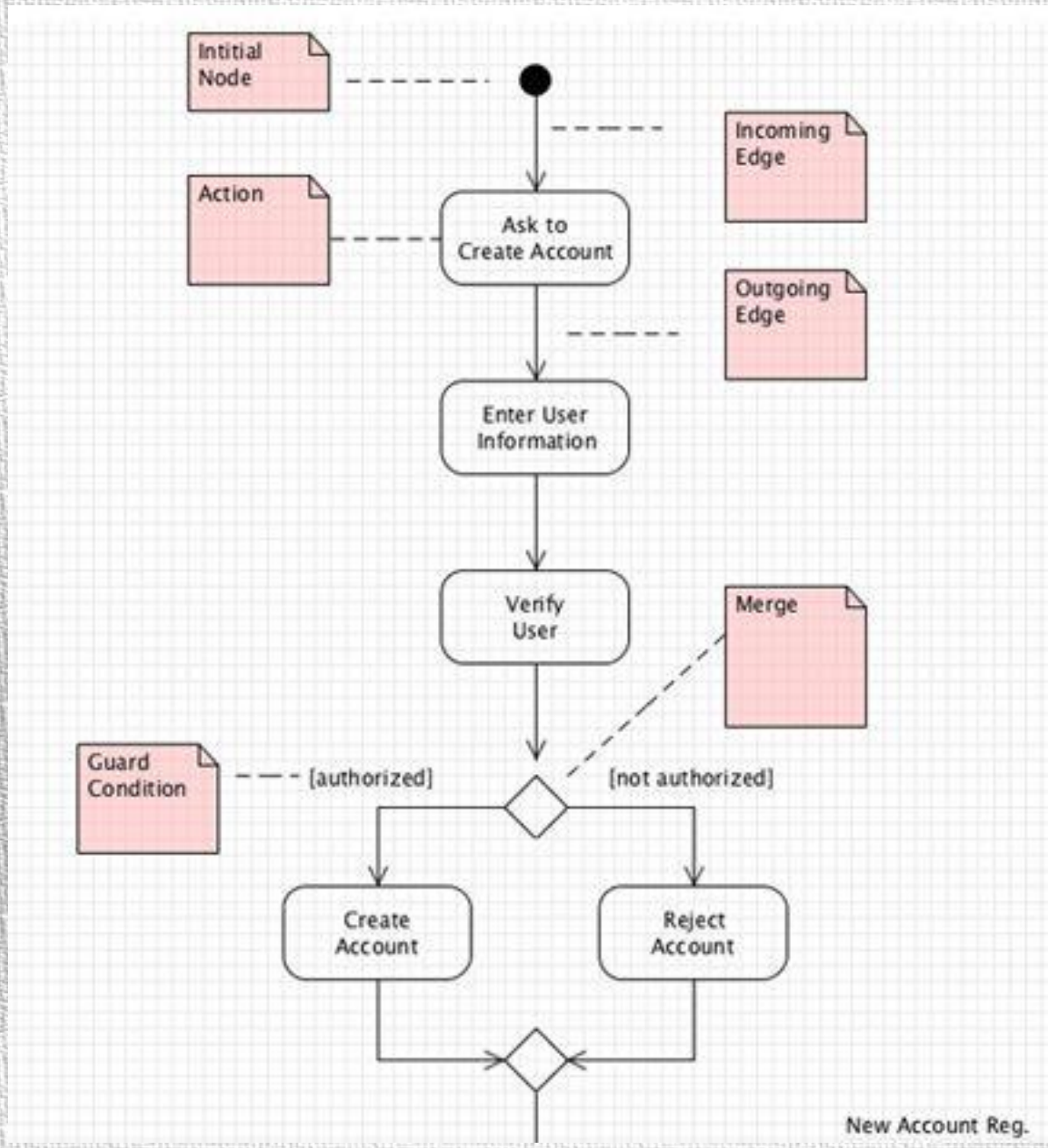
Модель процесса

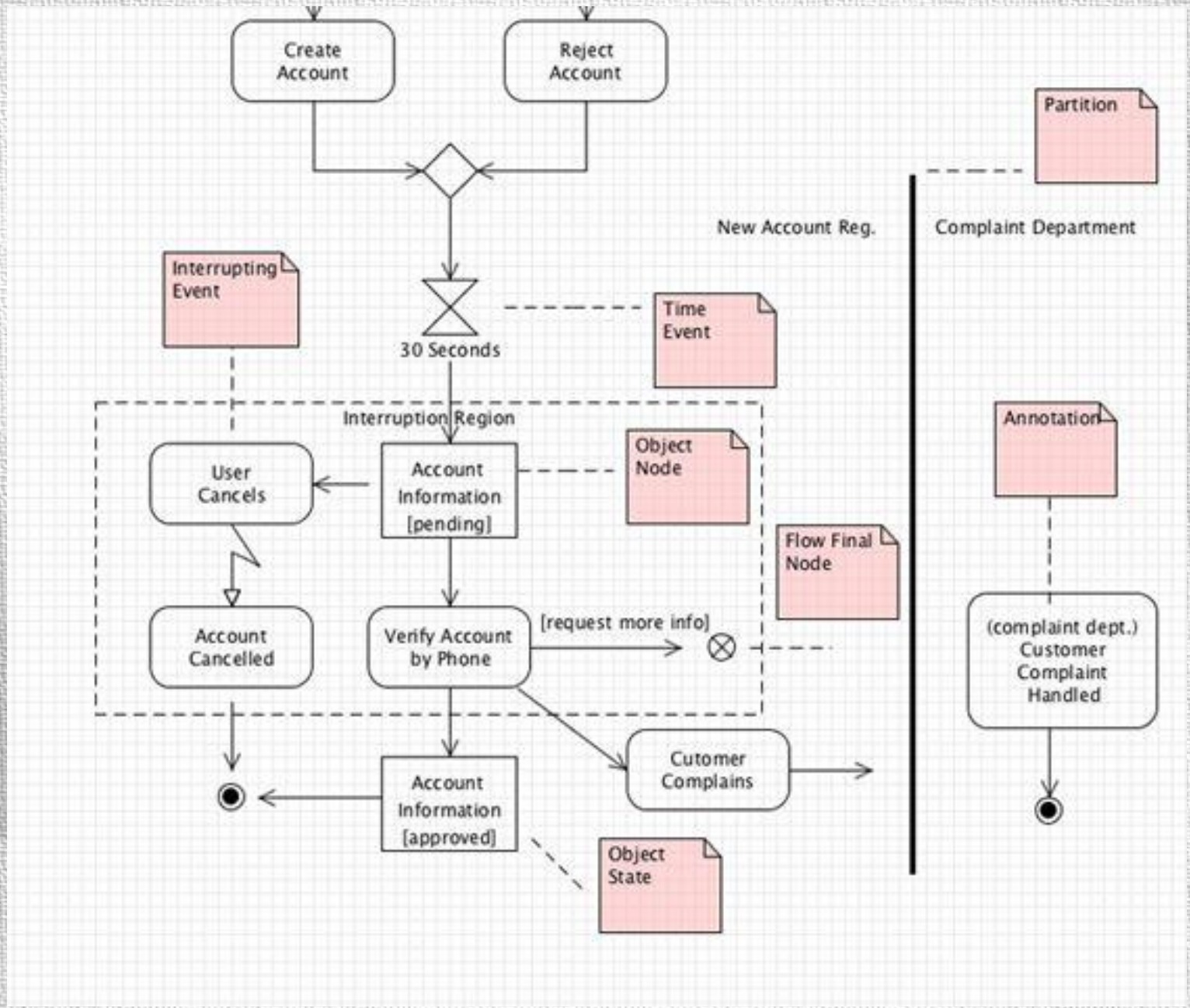
Модель процесса является расширением UML диаграммы деятельности, используемой для моделирования бизнес-процесса - эта диаграмма показывает, какую цель имеет процесс, входные данные, выходные данные, события и информация, которые участвуют в процессе.

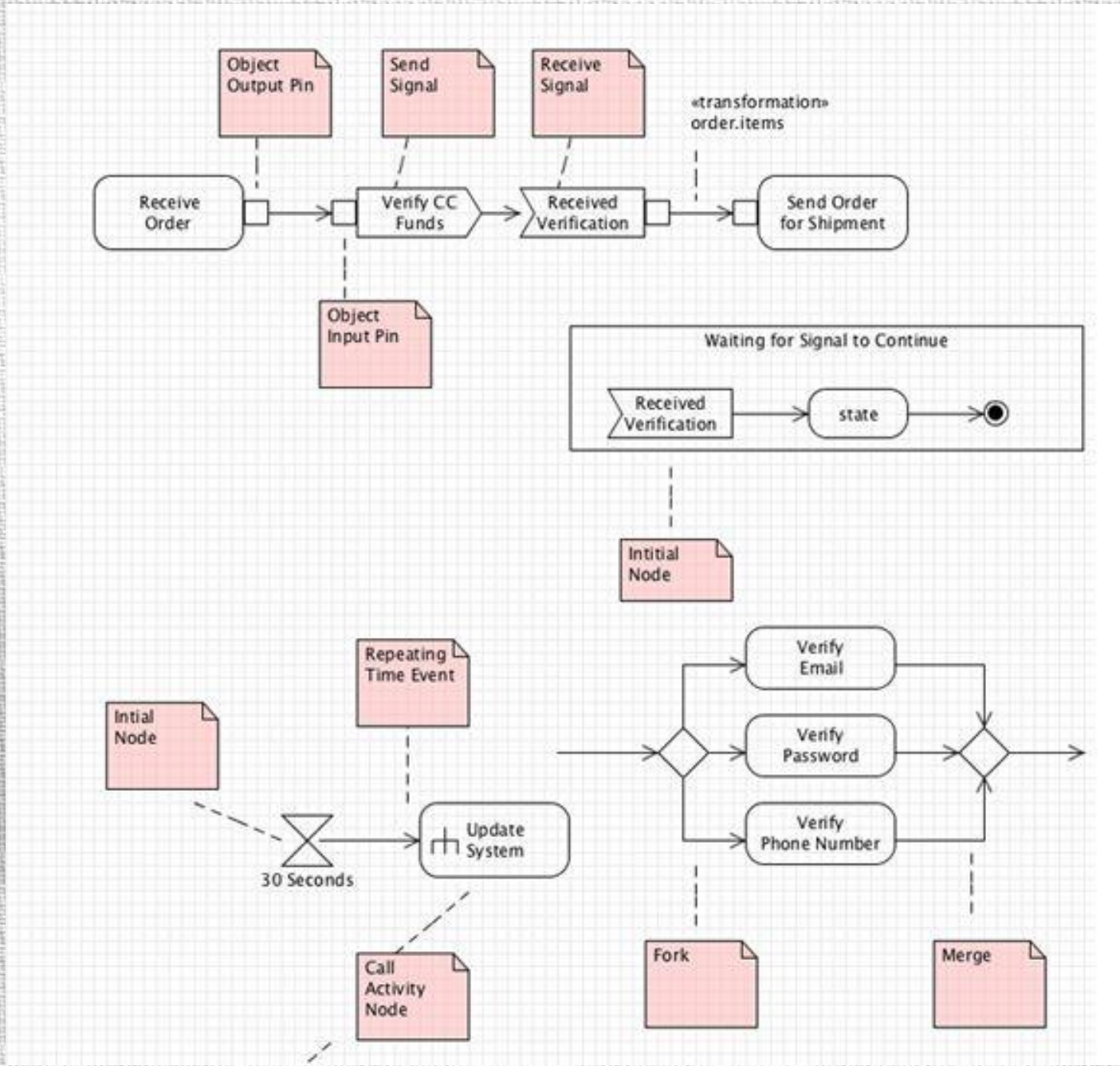


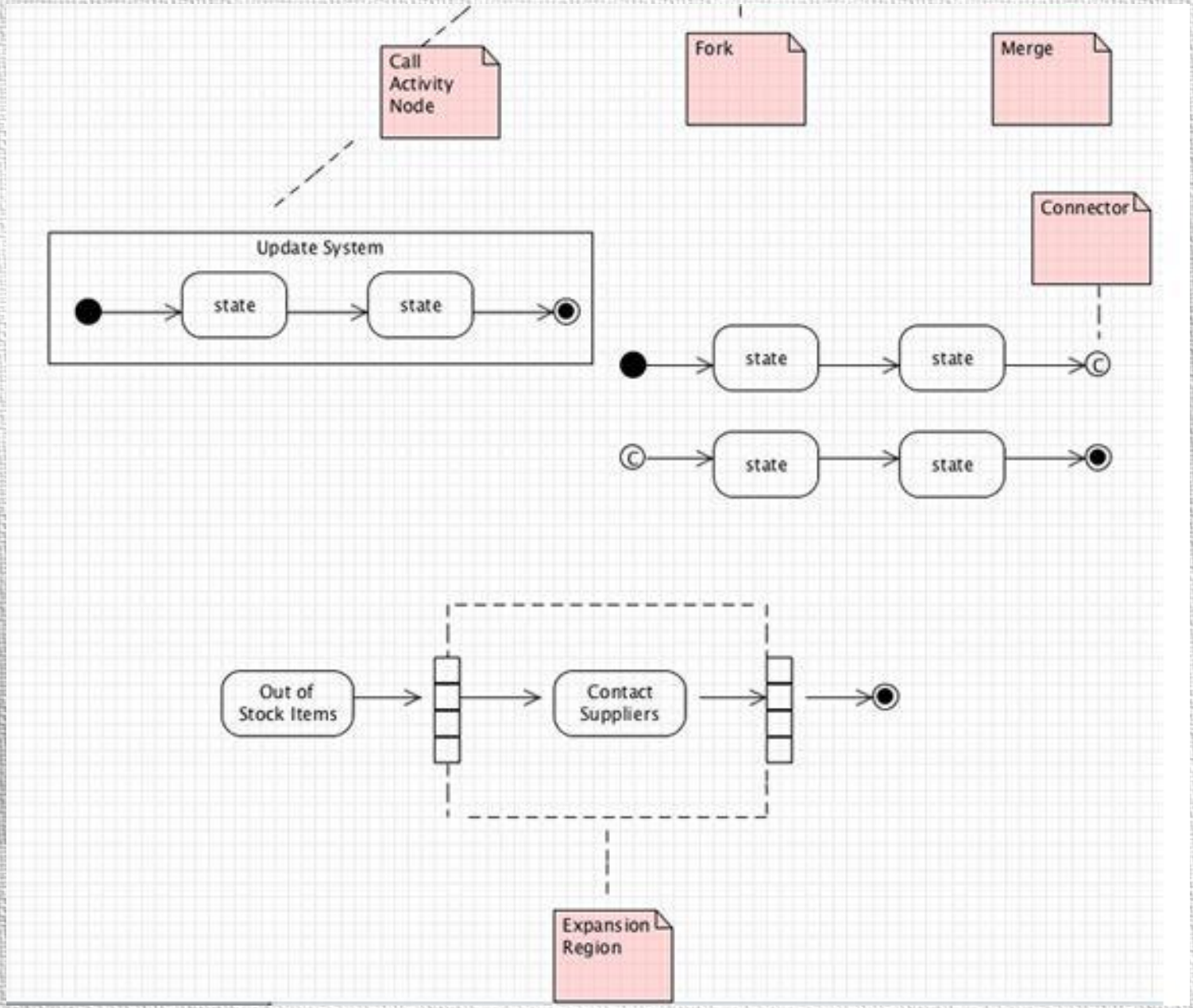
- Activity Diagrams

ШПАРГАЛКА


Intitial Node
Incoming Edge
Action
Ask to Create Account
Outgoing Edge
Enter User Information
Verify User
Merge
Guard Condition
[authorized]
[not authorized]
Create Account
Reject Account
New Account Reg.


Create Account
Reject Account
Partition
New Account Reg.
Complaint Department
Interrupting Event
30 Seconds
Time Event
Interruption Region
Annotation
User Cancels
Account Information [pending]
Object Node
Flow Final Node
Account Cancelled
Verify Account by Phone
[request more info]
(complaint dept.) Customer Complaint Handled
Cutomer Complains
Account Information [approved]
Object State


Object Output Pin
Send Signal
Receive Signal
«transformation» order.items
Receive Order
Verify CC Funds
Received Verification
Send Order for Shipment
Object Input Pin
Waiting for Signal to Continue
Received Verification
state
Intitial Node
Repeating Time Event
Intial Node
Verify Email
Verify Password
Verify Phone Number
Update System
30 Seconds
Call Activity Node
Fork
Merge


Call
Activity
Node
Fork
Merge
Connector
Update System
state
state
state
state
state
state
Out of
Stock Items
Contact
Suppliers
Expansion
Region

- Диаграмма последовательности (sequence diagram)
- Диаграмма кооперации (collaboration diagram)

ДИАГРАММЫ ВЗАИМОДЕЙСТВИЯ (INTERACTION DIAGRAMS)

Как объекты взаимодействуют друг с другом?

Для систем различной природы и назначения характерно взаимодействие между собой отдельных образующих их элементов. Для представления динамических особенностей взаимодействия элементов модели в контексте реализации ВИ предназначены диаграммы кооперации и последовательности.

Это этап детального проектирования, на котором происходит распределение функций программы между выявленными объектами.

Какие объекты участвуют в каждом прецеденте?

Как объекты взаимодействуют друг с другом в каждом прецеденте?

Как объекты взаимодействуют друг с другом?

Сообщения

предшественники / повторность номер :
атрибуты = ИМЯ (аргументы) : переменные

Сообщение (message) — передача управления и данных от отправителя к получателю

- Отправка сообщения — действие
- Получение сообщения — событие

Сообщения

Для записи сообщений в языке UML принят следующий синтаксис:

имяАтрибута = имяСообщения (аргументы):
ВозвращаемоеЗначение,

где **имяАтрибута** задает атрибут, куда помещается возвращаемое значение.

Сообщение (*message*) – передача управления и данных от отправителя к получателю.

- Отправка сообщения – *действие*
- Получение сообщения – *событие*

Примеры записи сообщений

коорд = ТекущееПоложение(самолет Т1)	Вызов операции, возврат значения
оповещение()	Посылка сигнала
установитьМаршрут(х)	Вызов операции с действительным параметром
<<create>>	Стандартное сообщение для создания объекта

Сообщения

Когда объект посылает сообщение в другой объект (делеги - руя некоторое действие получателю), объект-получатель, в свою очередь, может послать сообщение в третий объект и т. д. Так формируется поток событий – последовательность управления. Очевидно, что сообщения в последователь - ности должны быть пронумерованы. Номера записываются перед именами сообщений, направления сообщений указываются стрелками (размещаются над линями связей).

Типы действий

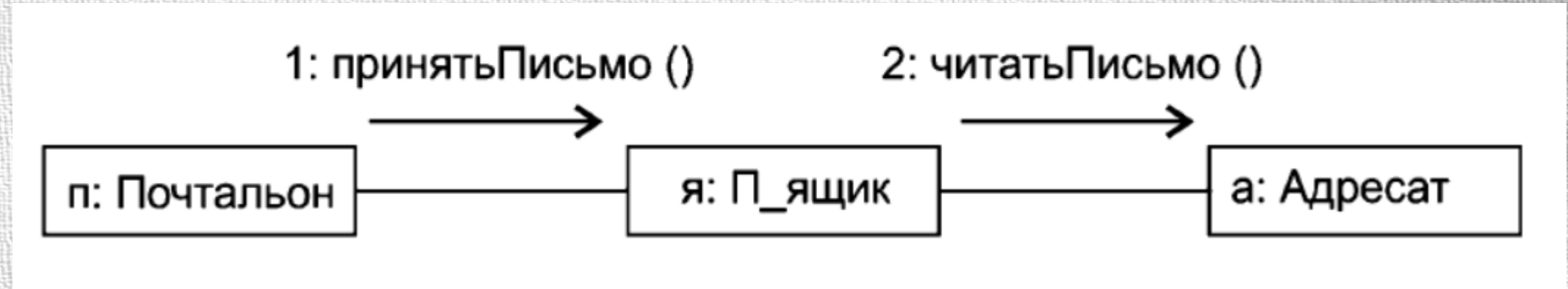
1. Вызовы (Call)
2. Возврат (Return)
3. Создание (Create)
4. Уничтожение (Destroy)
5. Посылка сигнала (Send)

Поток синхронных сообщений (с вложениями)

Наиболее общую форму управления задает процедурный поток, иначе называемый **потоком с вложениями**, - поток синхронных сообщений
Как показано на рисунке, процедурный поток рисуется стрелками с закрашенными наконечниками.

Здесь сообщение **2.1:напиток = *изготовить(СмесьN3)*** определено как первое сообщение, вложенное во второе сообщение ***2:заказать(СмесьN 3)*** последовательности, а сообщение ***2.2:принести(Напиток)*** как второе вложенное сообщение. Все сообщения процедурной последовательности считаются синхронными. Работа с синхронным сообщением подчиняется следующему правилу: передатчик ждет до тех пор, пока получатель не примет и не обработает сообщение. В нашем примере это означает, что третье сообщение будет послано только после обработки сообщений **2.1** и **2.2**. Отметим, что степень вложенности сообщений может быть любой. **Главное, чтобы соблюдалось правило - последовательность сообщений внешнего уровня возобновляется только после завершения вложенной последовательности**.

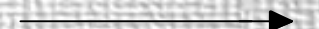
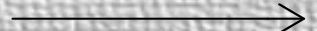
Поток асинхронных сообщений



Менее общую форму управления задаёт асинхронный поток управления. Как показано на рисунке, асинхронный поток рисуется обычными стрелками. Здесь все сообщения считаются асинхронными, при которых передатчик не ждёт реакции от получателя сообщения. Такой вид коммуникации иметь семантику почтового ящика. Получатель принимает сообщение по мере готовности. Иными словами, передатчик и получатель не синхронизируют свою работу. Скорее один объект «избавляется» от сообщения для другого объекта. Нашим примере сообщения **читатьПисьмо()** определено как 2-е сообщение в потоке управления.

Типы передачи сообщений

Сообщения на диаграмме *кооперации* изображаются дополнительными стрелками рядом с соответствующей *связью* или ролью ассоциации. Направление стрелки указывает на получателя *сообщения*. Внешний вид стрелки *сообщения* имеет определенный смысл.

Стрелка	Тип передачи
	Вложенный поток управления
	Простой поток управления. Управление передается получателю . **Только UML1**.
	Асинхронный поток управления **UML1** **UML2**
	Возврат управления после выполнения всех действий

Сообщения и их графическое изображение

- ***сообщение*** (message) — спецификация передачи информации от одного элемента модели к другому с ожиданием выполнения определенных действий со стороны принимающего элемента.
- При этом первый *объект* предполагает, что после получения *сообщения* вторым *объектом* последует выполнение некоторого действия. На диаграмме *кооперации сообщение* является причиной или стимулом начала выполнения операций, отправки сигналов, создания и уничтожения отдельных *объектов*. *Связь* обеспечивает канал для направленной передачи *сообщений* между *объектами* от объекта-источника к объекту-получателю.

- **Сплошная линия с треугольной стрелкой** обозначает вызов процедуры или другого вложенного потока управления. Может быть также использована совместно с параллельно активными объектами, когда один из них передает сигнал и ожидает, пока не закончится некоторая вложенная последовательность действий. Обычно все такие сообщения являются синхронными, т. е. инициируемыми по завершении некоторой деятельности или при выполнении некоторого условия.
- **Сплошная линия с V-образной стрелкой** обозначает простой поток управления. Каждая такая стрелка изображает один этап в последовательности потока управления. Обычно все такие сообщения являются асинхронными.

- **Сплошная линия с полустрелкой** используется для обозначения асинхронного потока управления в UML1. Соответствующие сообщения формируются в произвольные, заранее не известные моменты времени, как правило, активными объектами. Обычно сообщения этого типа являются начальными в последовательности потока управления и чаще всего инициируются актерами.
- **Пунктирная линия с V-образной стрелкой** обозначает возврат из вызова процедуры. Стрелки этого типа зачастую отсутствуют на диаграммах кооперации, поскольку неявно предполагается их существование после окончания процесса активизации некоторой деятельности.

Стереотипы сообщений

В языке UML предусмотрены стандартные действия, выполняемые в ответ на получение соответствующего *сообщения*. Они могут быть явно указаны на диаграмме *кооперации* в форме стереотипа перед именем *сообщения*, к которому они относятся, или выше его. В этом случае они записываются в угловых кавычках.

- **<<call>>** (вызвать) – *сообщение*, требующее вызова операции или процедуры объекта-получателя. Если *сообщение* с этим стереотипом рефлексивное, то оно инициирует локальный вызов операции у пославшего это *сообщение объекта*.
- **<<return>>**(возвратить) – *сообщение*, возвращающее значение выполненной операции или процедуры вызвавшему ее *объекту*. Значение результата может инициировать ветвление потока управления.
- **<<create>>** (создать) – *сообщение*, требующее создания другого *объекта* для выполнения определенных действий. Созданный *объект* может стать активным (ему передается поток управления), а может остаться пассивным.

- **<<destroy>>** (уничтожить) – *сообщение* с явным требованием уничтожить соответствующий *объект*. Посылается в том случае, когда необходимо прекратить нежелательные действия со стороны существующего в системе *объекта*, либо когда *объект* больше не нужен и должен освободить задействованные им системные ресурсы.
- **<<send>>** (послать) – обозначает посылку другому *объекту* сигнала, который асинхронно инициируется одним *объектом* и принимается (перехватывается) другим. Отличие сигнала от *сообщения* заключается в том, что сигнал должен быть явно описан в том классе, *объект* которого инициирует его передачу.

ФОРМАТ СООБЩЕНИЯ

Каждое *сообщение* может быть помечено строкой текста, которая имеет следующий формат:

<Предшествующие сообщения> / <Выражение последовательности> **:** *<Возвращаемое значение :=*

имя сообщения> <(Список аргументов)>

Предшествующие сообщения

Предшествующие сообщения — есть разделенные запятыми номера сообщений, записанные перед наклонной черточкой:

<Номер сообщения ','>< Номер сообщения,'> '/'

Если список номеров сообщений пуст, то вся запись, включая наклонную черточку (слэш), опускается.

Каждый номер сообщения может быть выражением последовательности без рекурсивных символов. Выражение должно определять номер другого сообщения в этой же последовательности.

Пример записи предшествующих сообщений:

A3, B4/ C5: ошибка записи (сектор)

Смысл указания предшествующих сообщений заключается в том, что данное сообщение не может быть передано, пока не будут переданы своим адресатам все сообщения, номера которых записаны в данном списке.

Выражение последовательности

Выражение последовательности — есть разделенный точками список отдельных термов последовательностей, после которого записывается двоеточие:

<Терм последовательности'.'><Терм последовательности'.'>':'

Каждый из термов представляет отдельный уровень процедурной вложенности в форме законченной итерации. Наиболее верхний уровень соответствует самому левому терму последовательности. Если все потоки управления параллельные, то вложенность отсутствует. Каждый из термов последовательности имеет следующий синтаксис:

[Целое число| Имя] [Символ рекуррентности]

Возвращаемое значение

Возвращаемое значение представляется в форме списка имен значений, возвращаемых по окончании коммуникации или взаимодействия в полной итерации данной процедурной последовательности. Эти идентификаторы могут выступать в качестве аргументов в последующих сообщениях. Если сообщение не возвращает никакого значения, то ни значение, ни оператор присваивания на диаграмме кооперации не указываются.

Например, сообщение

1.2.3: p:= найти_документ (спецификация_документа)

означает передачу вложенного сообщения с запросом поиска в базе данных нужного документа по его спецификации, причем источнику сообщения должен быть возвращен найденный документ.

Имя сообщения

Имя сообщения, записанное в сигнатуре после возвращаемого значения, означает имя события, которое инициируется объектом-получателем сообщения после его приема. Наиболее часто таким событием является вызов операции объекта. Это может быть реализовано различными способами, один из которых — вызов операции. Тогда соответствующая операция должна быть определена в том классе, которому принадлежит объект-получатель.

Список аргументов

Список аргументов представляет собой разделенные запятыми и заключенные в круглые скобки действительные параметры той операции, вызов которой инициируется данным сообщением. Список аргументов может быть пустым, однако скобки все равно записываются. Для записи аргументов также может быть использован некоторый псевдокод или язык программирования.

- Атрибуты

ДИАГРАММА КОММУНИКАЦИИ (COMMUNICATION DIAGRAM)

Диаграмма кооперации

Диаграмма *кооперации* предназначена для описания поведения системы на уровне отдельных *объектов*, которые обмениваются между собой *сообщениями*, чтобы достичь нужной цели или реализовать некоторый вариант использования. С точки зрения аналитика или архитектора системы в проекте важно представить структурные *связи* отдельных *объектов* между собой. Такое представление структуры модели как совокупности взаимодействующих *объектов* и обеспечивает диаграмма *кооперации*.

Диаграмма коммуникации **(communication diagram) – способ описания поведения, семантически эквивалентный диаграмме последовательности.**

Фактически, это такое же описание последовательности обмена сообщениями взаимодействующих экземпляров классификаторов, только выраженное другими графическими средствами. Более того, большинство инструментов умеет автоматически преобразовывать диаграммы последовательности в диаграммы коммуникации и обратно.

Таким образом, на диаграмме коммуникации также как и на диаграмме последовательности применяют один основной тип сущностей – экземпляры взаимодействующих классификаторов **1** и один тип отношений – связи **2**. Однако здесь акцент делается не на времени, а на структуре связей между конкретными экземплярами.

Нотация диаграммы коммуникации

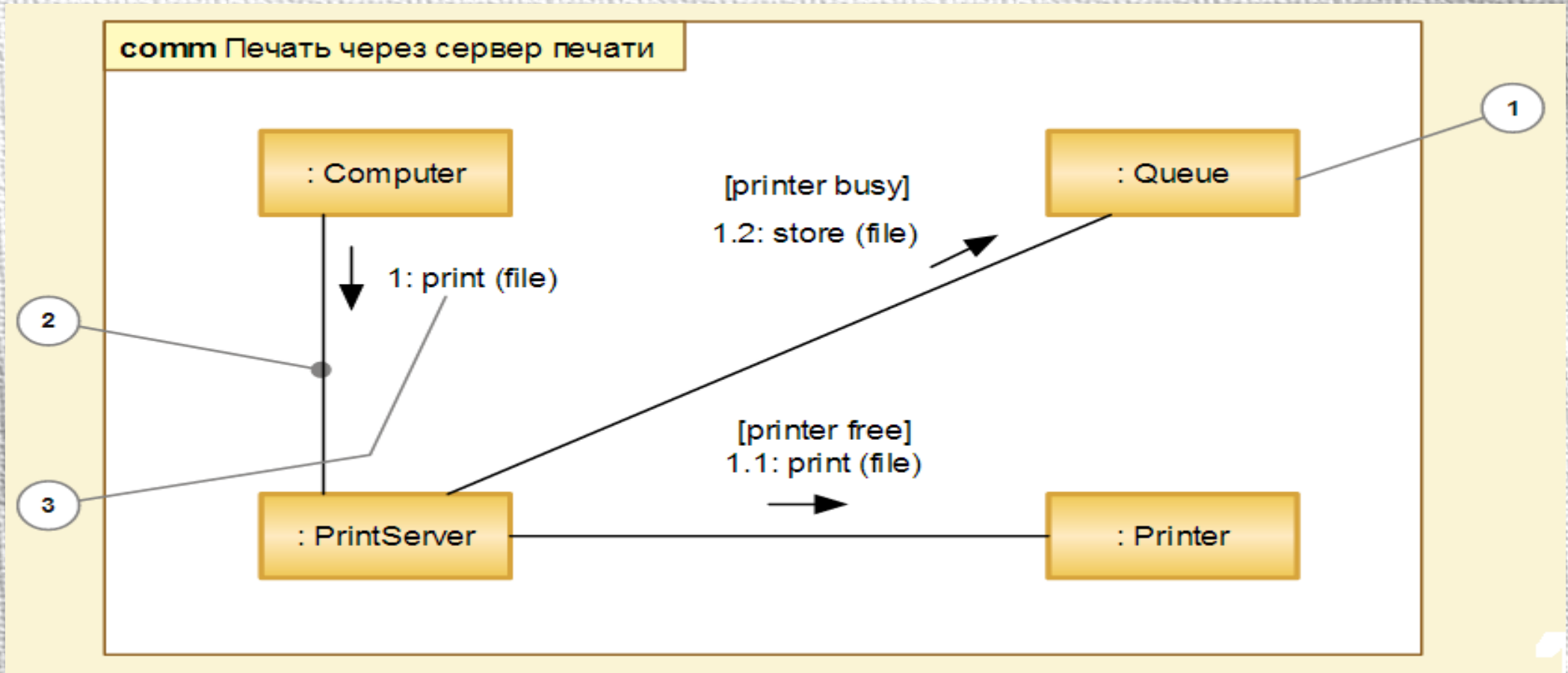
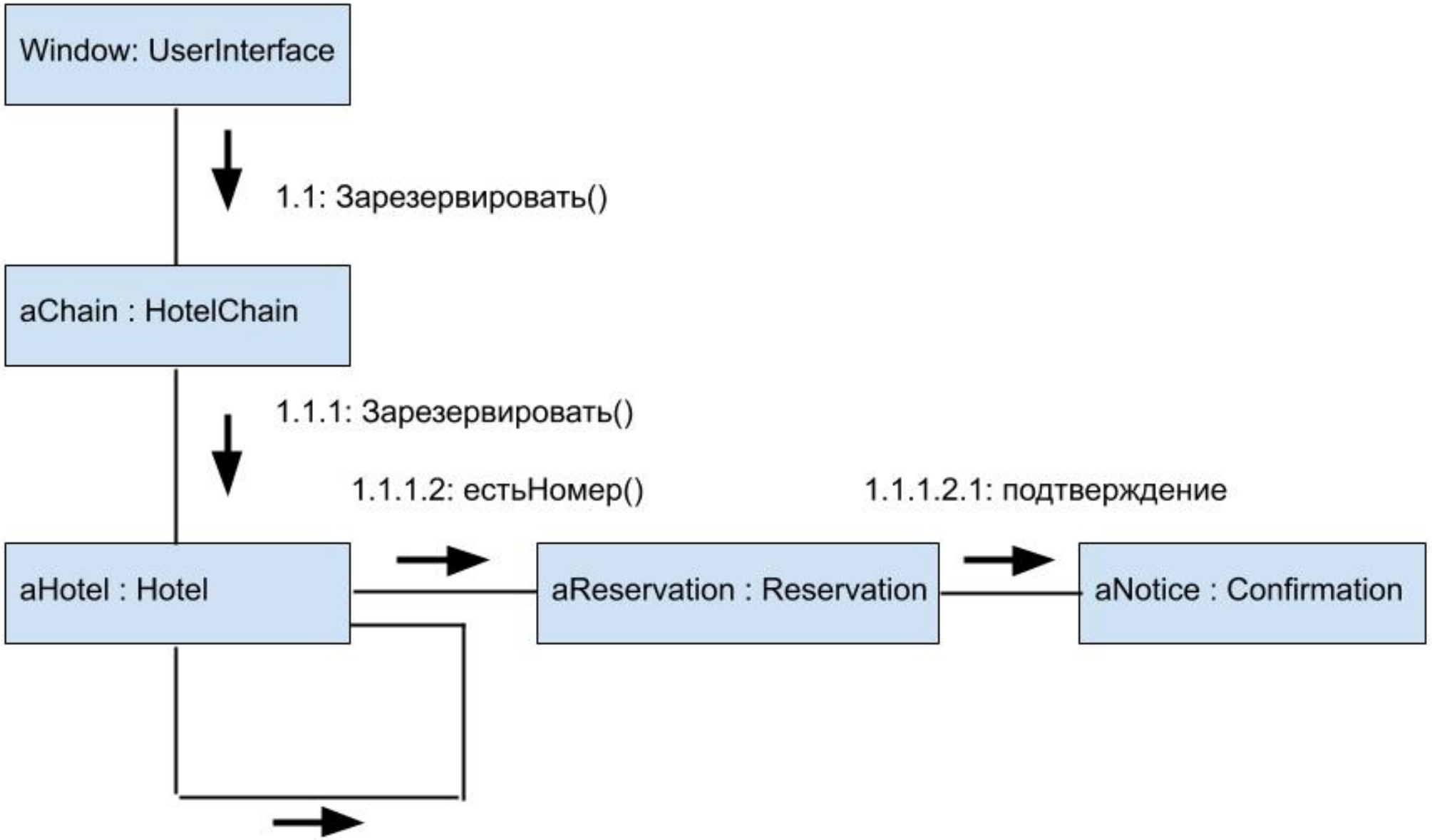


Диаграмма Коммуникации



На рисунке показаны основные элементы нотации, применяемые на диаграмме коммуникации. Для обозначения самих взаимодействующих объектов применяется стандартная нотация – прямоугольник с именем экземпляра классификатора. Взаимное положение элементов на диаграмме кооперации не имеет значения – важны только связи (чаще всего экземпляры ассоциаций), вдоль которых передаются сообщения **3**. Для отображения упорядоченности сообщений во времени применяется иерархическая десятичная нумерация.

Сравните этот рисунок с рис. **Нотация диаграммы последовательности** (на них изображено одно и то же поведение), и вам все станет понятно.

Элементы и нотация диаграммы кооперации

Диаграмма кооперации

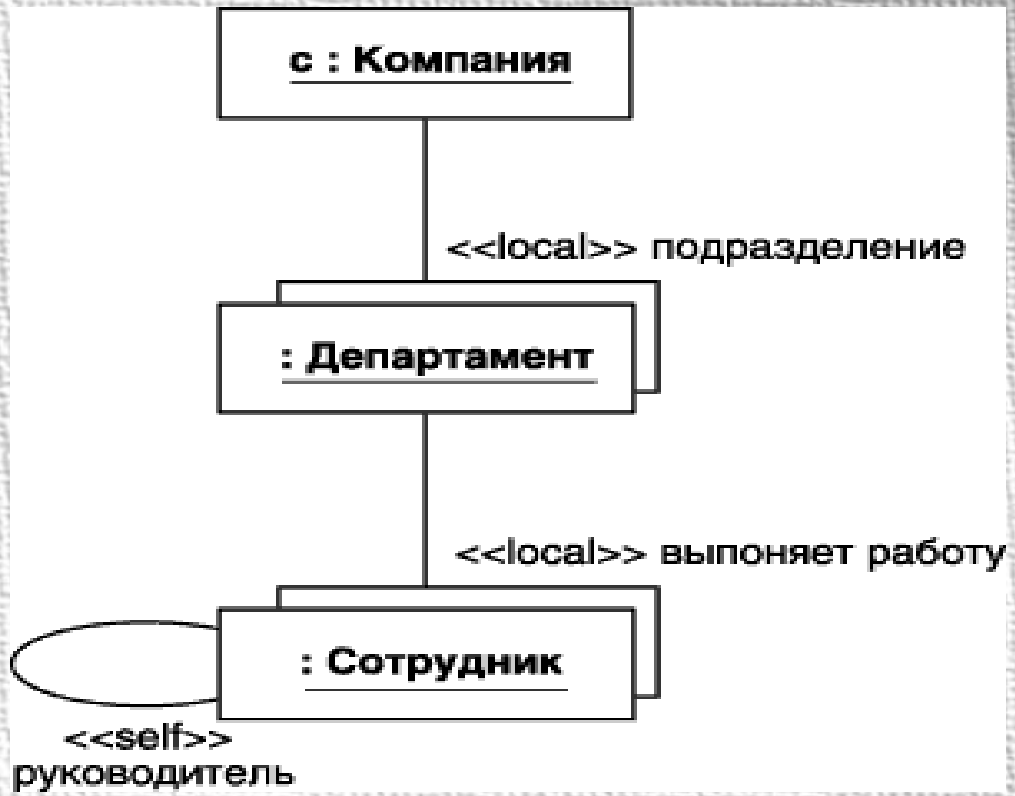
- На диаграмме кооперации размещаются **объекты**, представляющие собой экземпляры классов, связи между ними, которые в свою очередь являются экземплярами ассоциаций и сообщения. Показываются только те объекты, которые участвуют в реализации моделируемой кооперации.
- Далее, как и на диаграмме классов, показываются **структурные отношения** между объектами в виде различных соединительных линий. Связи могут дополняться именами ролей, которые играют объекты в данной взаимосвязи.
- И, наконец, изображаются **динамические взаимосвязи** — потоки сообщений в форме стрелок с указанием направления рядом с соединительными линиями между объектами, при этом задаются имена сообщений и их порядковые номера в общей

Отличие от диаграммы классов

Одна и та же совокупность объектов может участвовать в реализации различных кооперaций. В зависимости от рассматриваемой кооперации, могут изменяться как связи между отдельными объектами, так и поток сообщений между ними. Именно это отличает диаграмму кооперации от диаграммы классов, на которой должны быть указаны все без исключения классы, их атрибуты и операции, а также все ассоциации и другие структурные отношения между элементами модели.

Связи на диаграмме кооперации

Бинарная *связь* на диаграмме *кооперации* изображается отрезком сплошной линии, соединяющей два прямоугольника *объектов* На концах этой линии дополнительно могут быть явно указаны имена ролей соответствующей ассоциации.



Варианты возможных записей полного имени объекта

Формат полного имени объекта

<собственное имя объекта >/<Имя роли класса> :<Имя класса >

- **o : C** – объект с собственным именем **o**, экземпляр класса C.
- **: C** – анонимный объект, экземпляр класса C.
- **o :(или просто o)** — объект-сирота с собственным именем **o**.
- **o / R : C** - объект с собственным именем **o**, экземпляр класса C, играющий роль R.
- **/ R : C** - анонимный объект, экземпляр класса C, играющий роль R.
- **o / R** - объект-сирота с собственным именем **o**, играющий роль R.
- **/ R** - анонимный объект и одновременно объект-сирота, играющий роль R.

Примеры графических изображений объектов

Если собственное имя *объекта* отсутствует, то такой *объект* принято называть **анонимным**. Однако в этом случае обязательно ставится двоеточие перед именем соответствующего класса (в). Отсутствовать может и имя класса – такой *объект* называется **сиротой**. Для него записывается только собственное имя *объекта*, двоеточие не ставится, имя класса не указываются (г). Если для *объектов* указываются атрибуты, то в большинстве случаев они принимают конкретные значения (б). Для отдельных *объектов* (д, е) могут быть дополнительно указаны роли, которые они играют в *кооперации*.

Активный объект

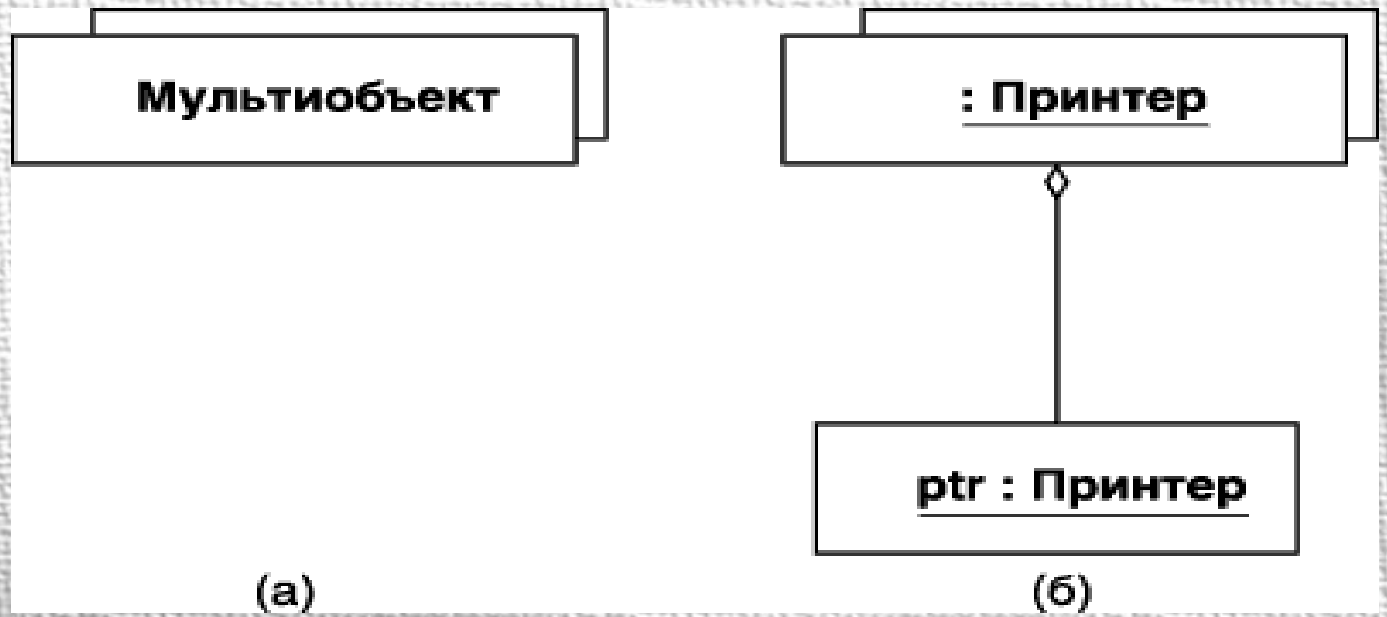
Графическое изображение активного объекта (слева) на диаграмме кооперации

В контексте языка UML все *объекты* делятся на две категории: **пассивные и активные.**

Пассивный объект оперирует только данными и не может инициировать деятельность по управлению другими *объектами*. Однако пассивные *объекты* могут посылать сигналы в процессе выполнения запросов, которые они обрабатывают.

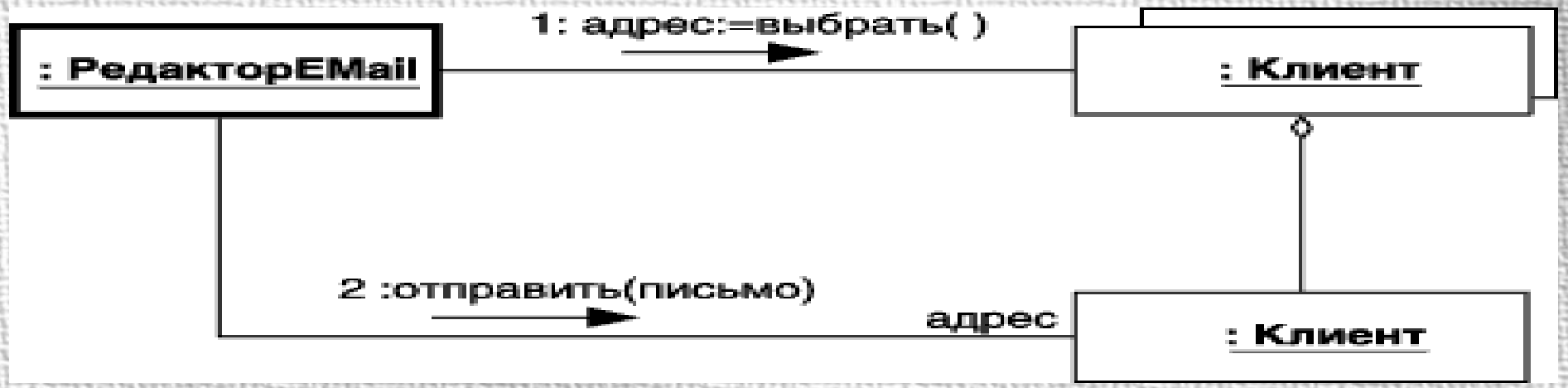
Активный объект (active object) имеет собственный процесс управления и может инициировать деятельность по управлению другими *объектами*.

Мультиобъект (multiobject)



На диаграмме *кооперации мультиобъект* используется для того, чтобы показать операции и сигналы, которые адресованы всему множеству анонимных *объектов*.
При этом стрелка взаимосвязи относится ко всему множеству *объектов*, которые обозначает данный *мультиобъект*. На диаграмме *кооперации* может быть явно указано отношение агрегации (композиции) между *мультиобъектом* и отдельным *объектом* из его множества

Фрагмент диаграммы кооперации для выбора адреса клиента для отправки электронного письма

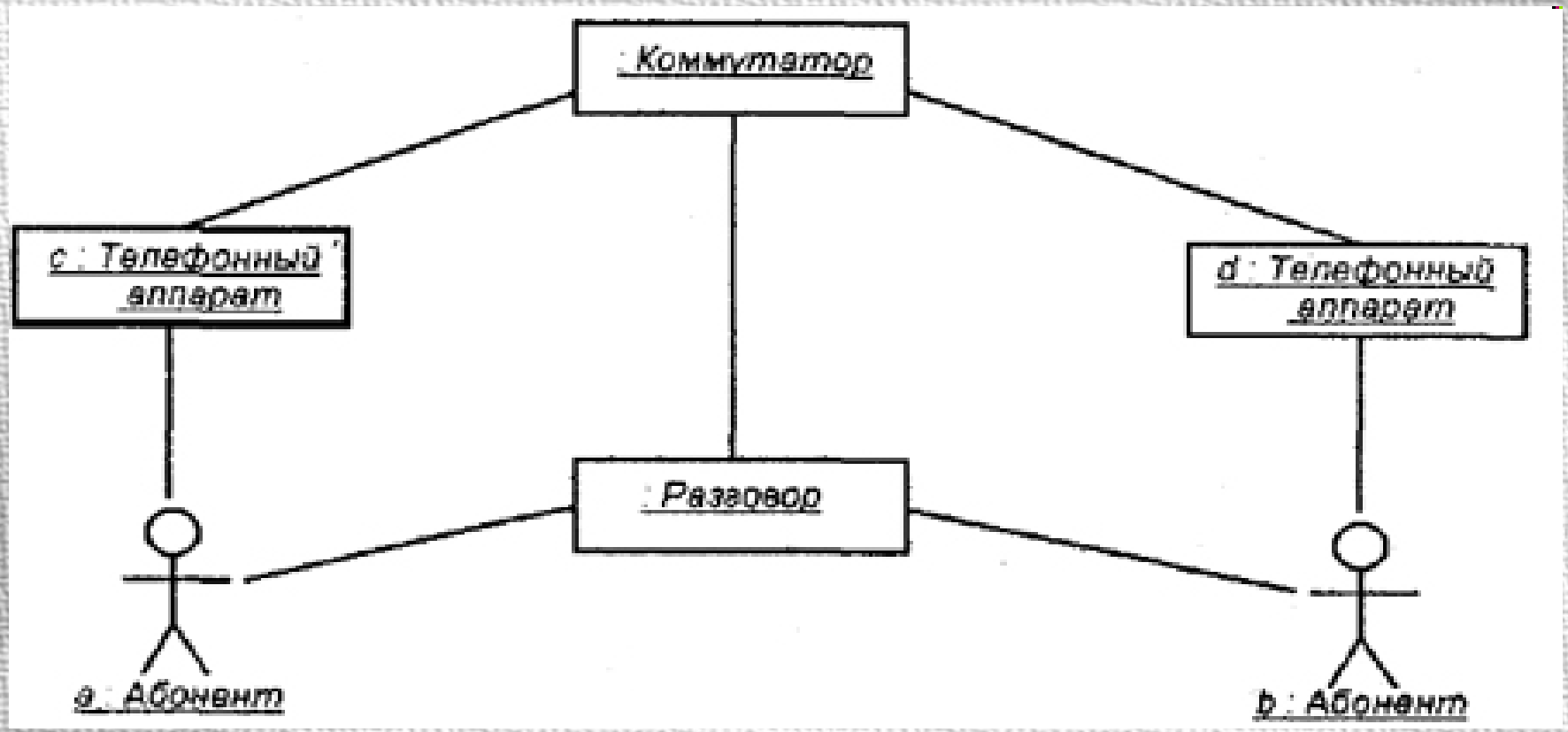


В следующем примере рассматривается ситуация с отправкой почтового *сообщений* клиенту из редактора электронной почты. Анонимный *активный объект* класса Редактор Email вначале посылает *сообщение* анонимному *мультиобъекту* класса Клиент. Это *сообщение* инициирует выбор единственного *объекта* класса Клиент, удовлетворяющего дополнительным условиям. После этого выбранному *объекту* посылается *сообщение* о необходимости отправить электронное письмо.

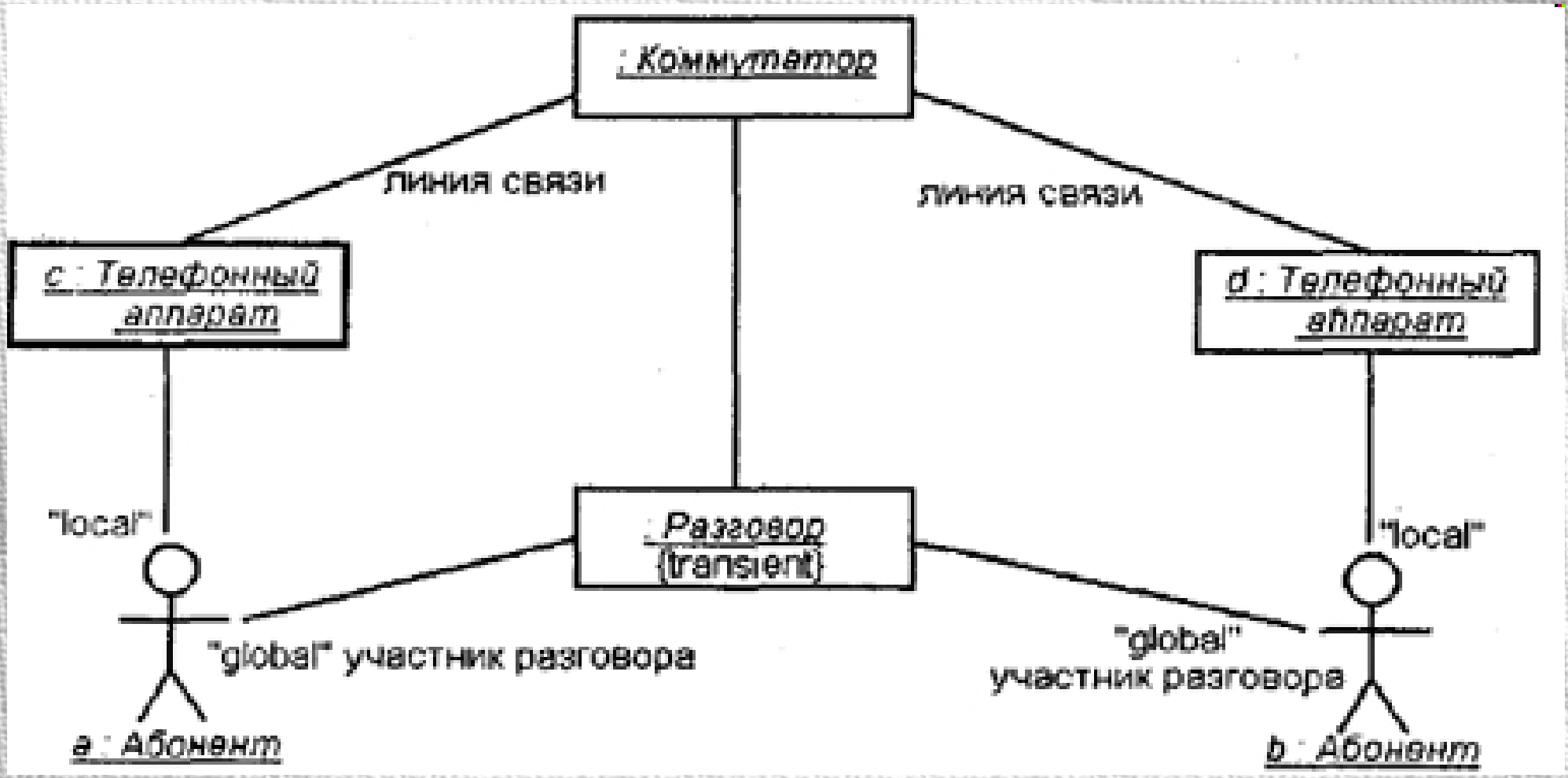
Составной объект
(composite object)

Составной объект(composite object) или объект-композит предназначен для представления *объекта*, имеющего собственную структуру и внутренние потоки (нити) управления.
Составной объект является экземпляром класса-композита, который связан отношением композиции со своими частями. Аналогичные отношения связывают между собой и соответствующие *объекты*.

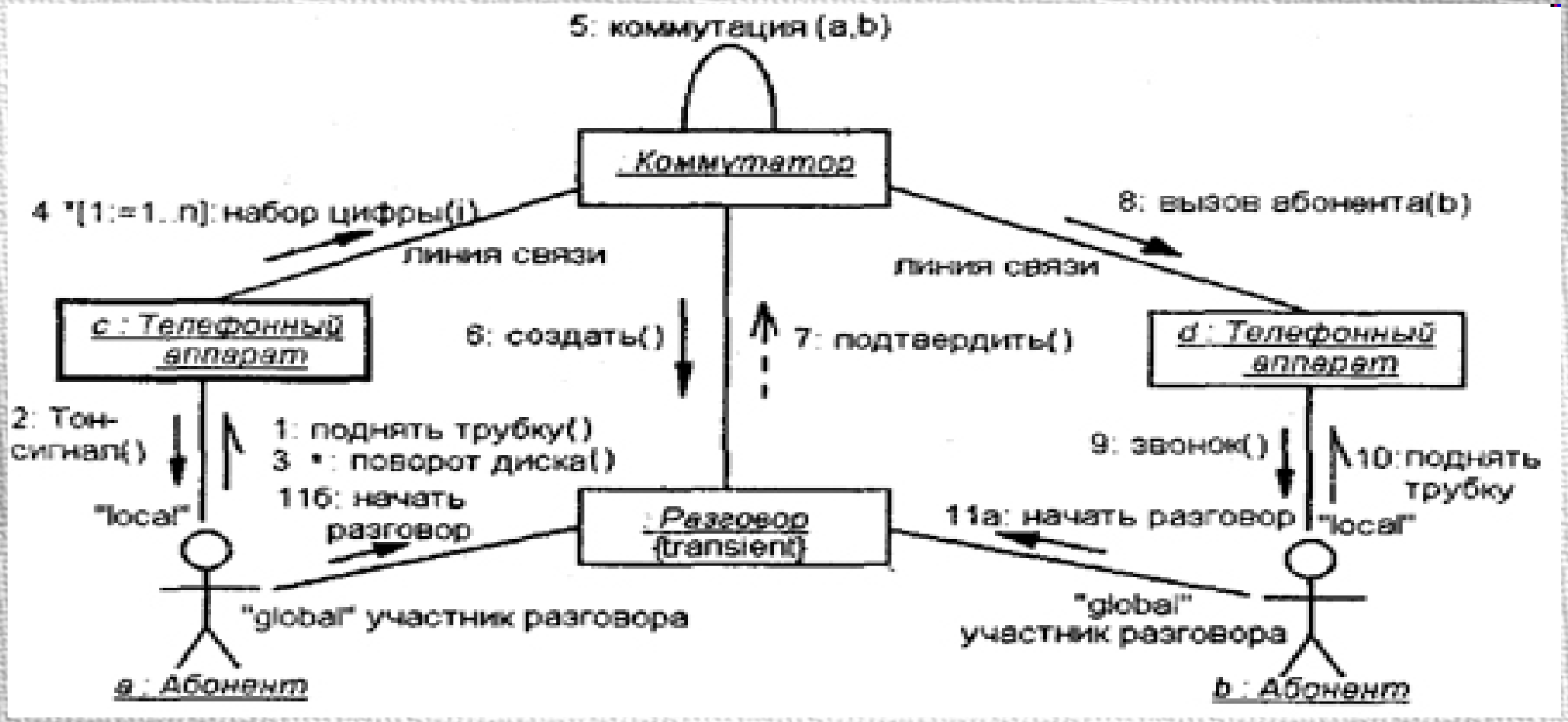
Пример построения диаграммы кооперации (начальный фрагмент)



Фрагмент диаграммы кооперации, дополненный стереотипами ролей связей, именами ассоциаций и помеченным значением объекта



Окончательный вариант диаграммы кооперации для моделирования телефонного разговора

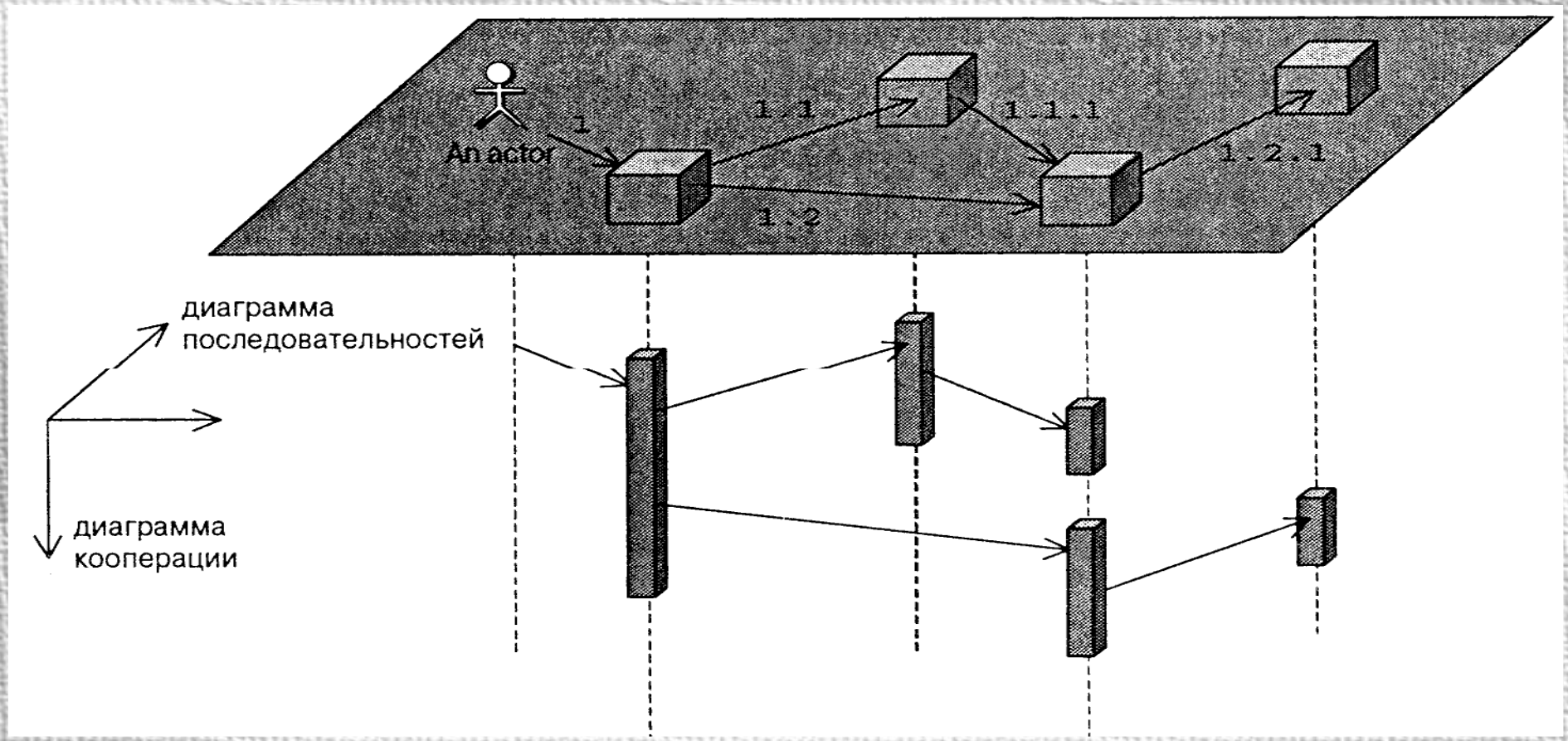


Рекомендации по построению диаграмм кооперации

Построение диаграммы кооперации можно начинать сразу после построения диаграммы вариантов использования. В этом случае каждый из вариантов использования может быть специфицирован в виде отдельной диаграммы кооперации уровня спецификации. Эта диаграмма способствует более . полному пониманию особенностей реализации функций системой, хотя и не может содержать всю информацию, необходимую для их реализации.

В дальнейшем, после построения диаграммы классов, каждая из диаграмм кооперации может уточняться в виде соответствующей диаграммы уровня примеров. Важно понимать, что диаграмма кооперации этого уровня может содержать те и только те объекты и связи, которые уже определены на построенной ранее диаграмме классов

Распределение поведения, реализуемого вариантом использования, между классами реализуется с помощью диаграмм последовательности и диаграмм коммуникации. Они являются как бы двумя ортогональными проекциями, описывающими одно и то же взаимодействие (проекция на вертикальную плоскость – диаграмма последовательности, на горизонтальную – диаграмма кооперации):



- Линия жизни
- Объекты на ДП
- Фокус управления

ДИАГРАММА ПОСЛЕДОВАТЕЛЬНОСТИ (SEQUENCE DIAGRAM)

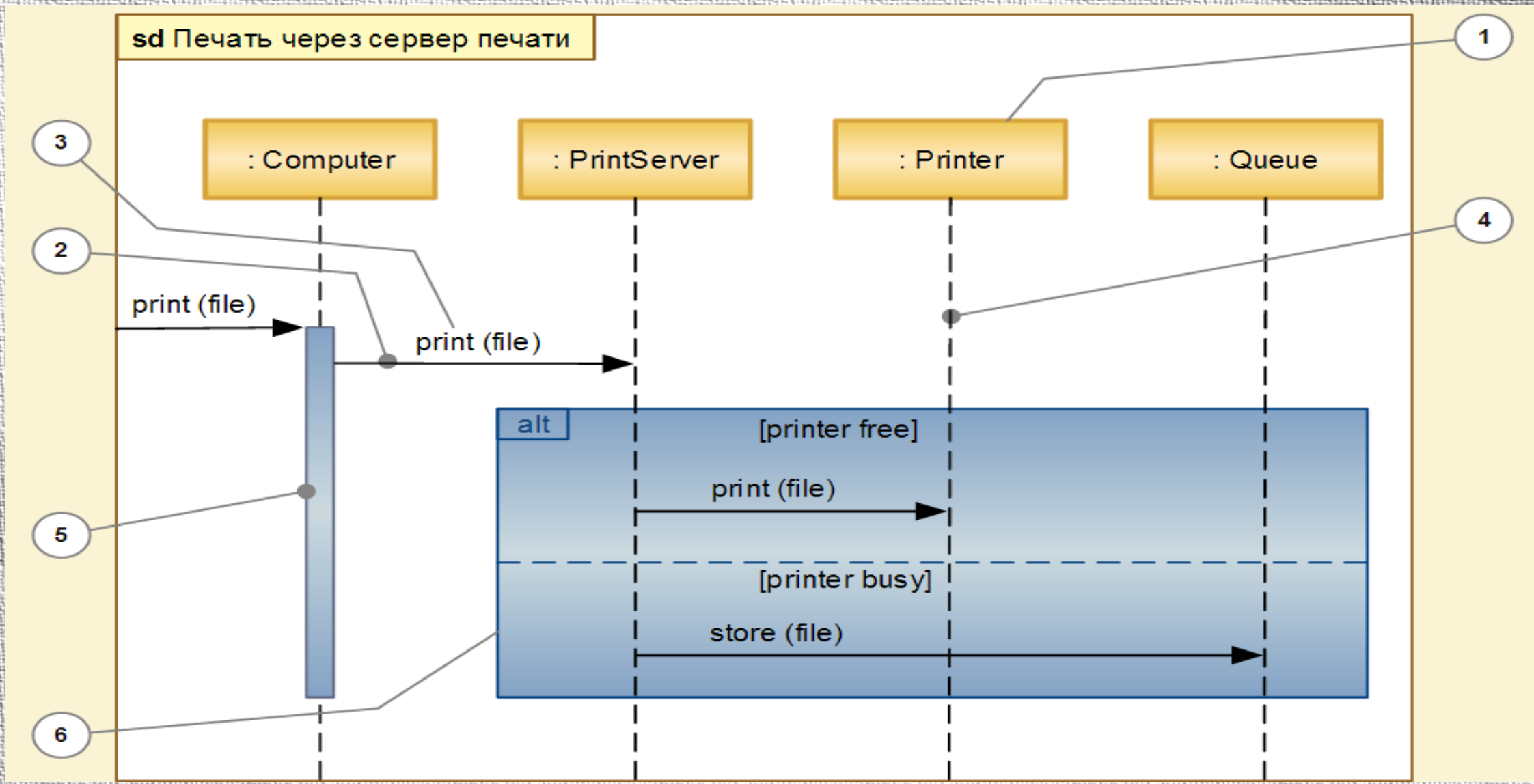
Видимость (visibility) определяет, может ли составляющая одного классификатора (в том числе имя) использоваться в другом классификаторе.

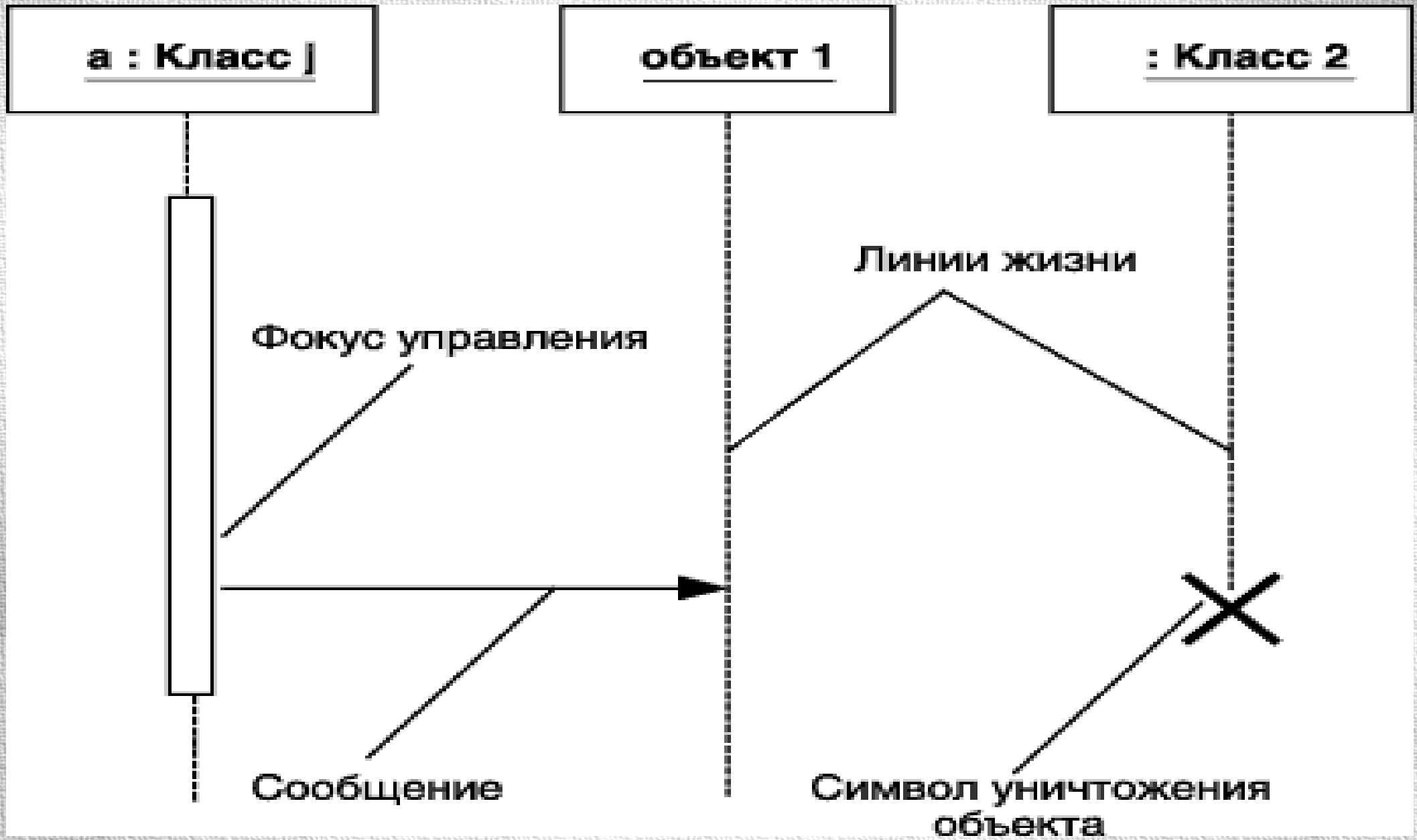
Фактически, диаграмма последовательности – это запись протокола конкретного сеанса работы системы (или фрагмента такого протокола). В объектно-ориентированном программировании самым существенным во время выполнения является пересылка сообщений между взаимодействующими объектами. Именно последовательность посылок сообщений отображается на данной диаграмме, отсюда и название.

На диаграмме последовательности применяют один основной тип сущностей – экземпляры взаимодействующих классификаторов **1** (в основном классов, компонентов и действующих лиц), и один тип отношений – связи **2**, по которым происходит обмен сообщениями **3**. Предусмотрено несколько способов посылки сообщений, которые в графической нотации различаются видом стрелки, соответствующей отношению.

Для обозначения самих взаимодействующих объектов применяется стандартная нотация – прямоугольник с именем экземпляра классификатора. Пунктирная линия, выходящая из него, называется **линией жизни 4**. Это не обозначение отношения в модели, а графический комментарий, призванный направить взгляд читателя диаграммы в правильном направлении. Фигуры в виде узких полосок, наложенных на линию жизни, также не являются изображениями моделируемых сущностей. Это графический комментарий **фокус управления**, показывающий отрезки времени, в течении которых объект владеет потоком управления **5** или другими словами имеет место активация объекта. Составные шаги взаимодействия **6** позволяют на диаграмме последовательности, отражать и алгоритмические аспекты протокола взаимодействия.

Нотация диаграммы последовательности




a : Класс]
объект 1
: Класс 2
Линии жизни
Фокус управления
Сообщение
Символ уничтожения объекта

Объекты на ДП

Каждый *объект* графически изображается в форме прямоугольника и располагается в верхней части своей *линии жизни* . Внутри прямоугольника записываются собственное имя *объекта* со строчной буквы и имя класса, разделенные двоеточием. При этом вся запись подчеркивается, что является признаком *объекта*, который, как указывалось ранее, представляет собой экземпляр класса.

Если на *диаграмме последовательности* отсутствует собственное имя *объекта*, то при этом должно быть указано имя класса. Такой *объект* считается **анонимным**. Может отсутствовать и имя класса, но при этом должно быть указано собственное имя *объекта*. Такой *объект* считается **сиротой**. Роль классов в именах *объектов* на *диаграммах последовательности*, как правило, не указывается.

Крайним слева на диаграмме изображается объект - инициатор моделируемого процесса взаимодействия (объект **a** на рисунке). Правее - другой объект, который непосредственно взаимодействует с первым. Таким образом, порядок расположения объектов на диаграмме последовательности определяется исключительно соображениями удобства визуализации их взаимодействия друг с другом.

Диаграмма последовательностей служит основным способом расшифровки последовательности действий в процессе выполнения того или иного варианта использования.

Иными словами, если вариант использования отвечает на вопрос «Что делает актер?», то последовательность отвечает на вопрос «Как работает система при выполнении данного варианта использования?».

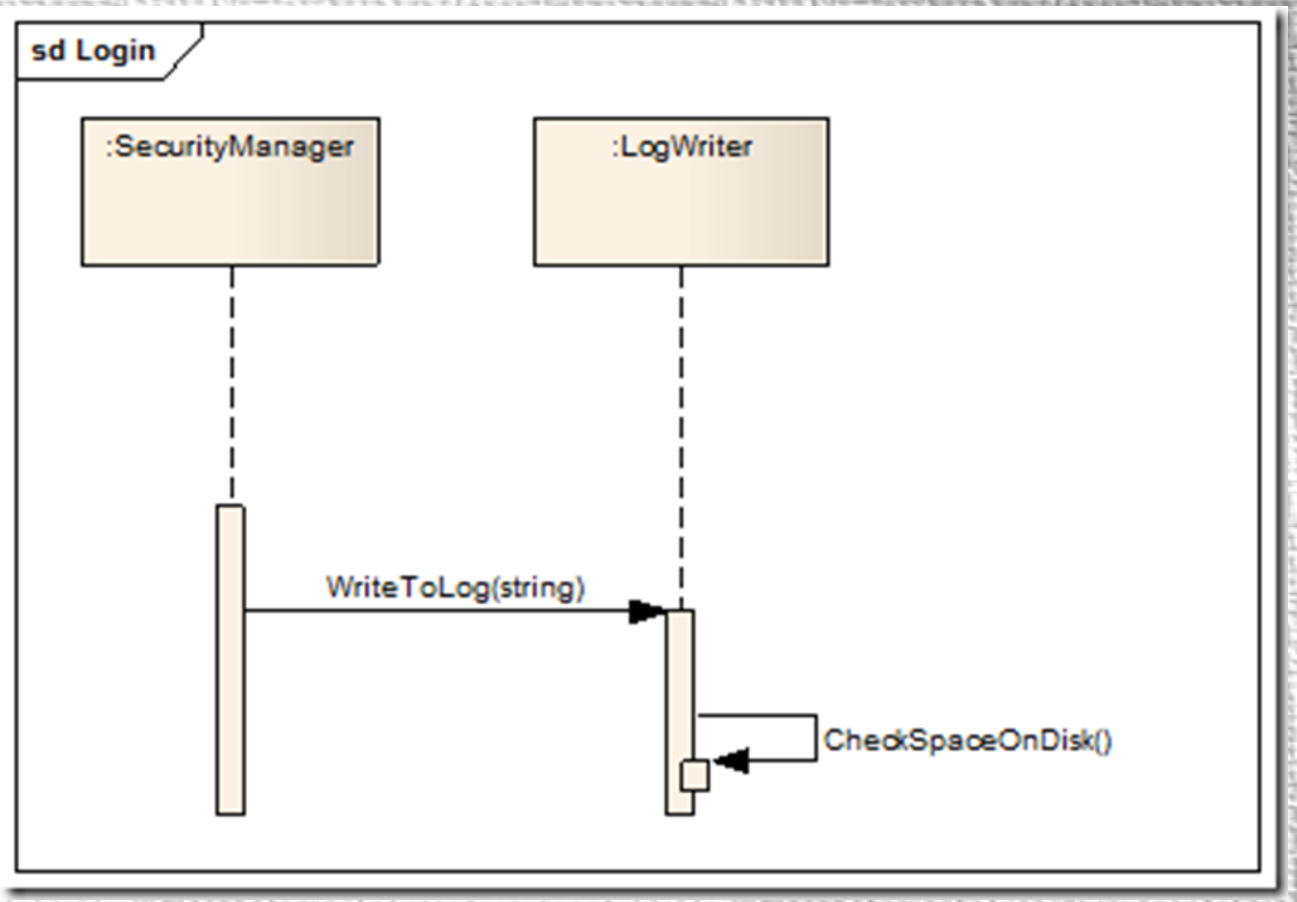
Таким образом, диаграмма последовательностей всегда создается в привязке к варианту использования. Каждый вариант использования может содержать несколько диаграмм последовательностей, на тот случай, если они описывают несколько альтернативных вариантов развития событий.

Сообщения на ДП

Процесс взаимодействия объектов реализуется посредством сообщений, которые посылаются одними объектами другим. Сообщения изображаются в виде горизонтальных стрелок с именем сообщения и образуют определенный порядок относительно времени своей инициализации. Другими словами, сообщения, расположенные на диаграмме последовательности выше, передаются раньше тех, которые расположены ниже. При этом масштаб на оси времени не указывается, поскольку диаграмма последовательности моделирует лишь временную упорядоченность взаимодействий типа "раньше-позже".

Self- message

Кроме сообщений, которые вызываются другими объектами, существуют собственные сообщения, которые объект вызывает **сам у себя** (Self-message).

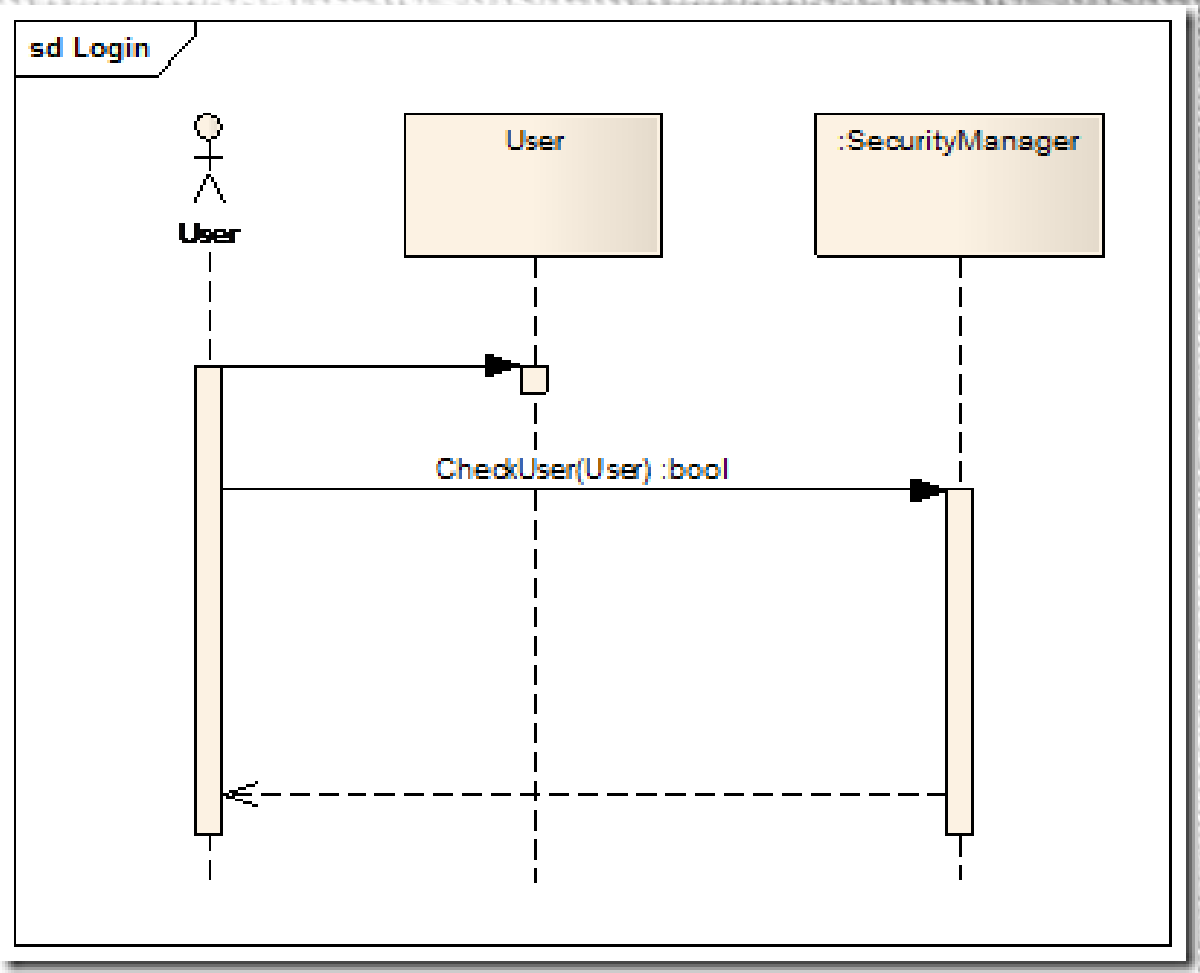


Return message

Кроме того, существуют также **обратные сообщения** (Return message), которые обозначают передачу некоторой информации вызывающему сообщению. В терминах классов это означает возврат некоторого значения.

На этой диаграмме метод возвращает результаты своей работы.

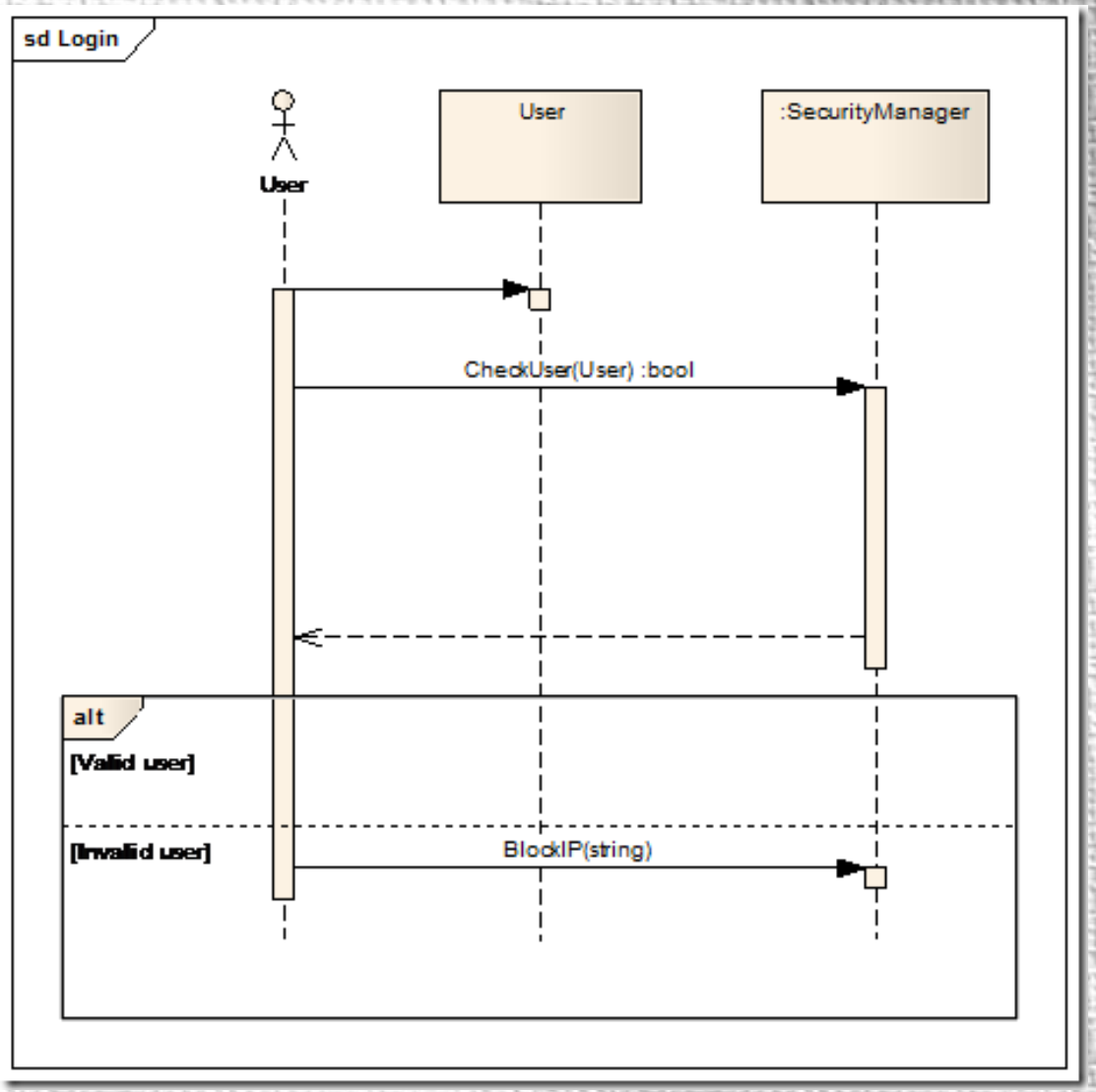
Собственные сообщения также могут быть обратными.



Combined fragment

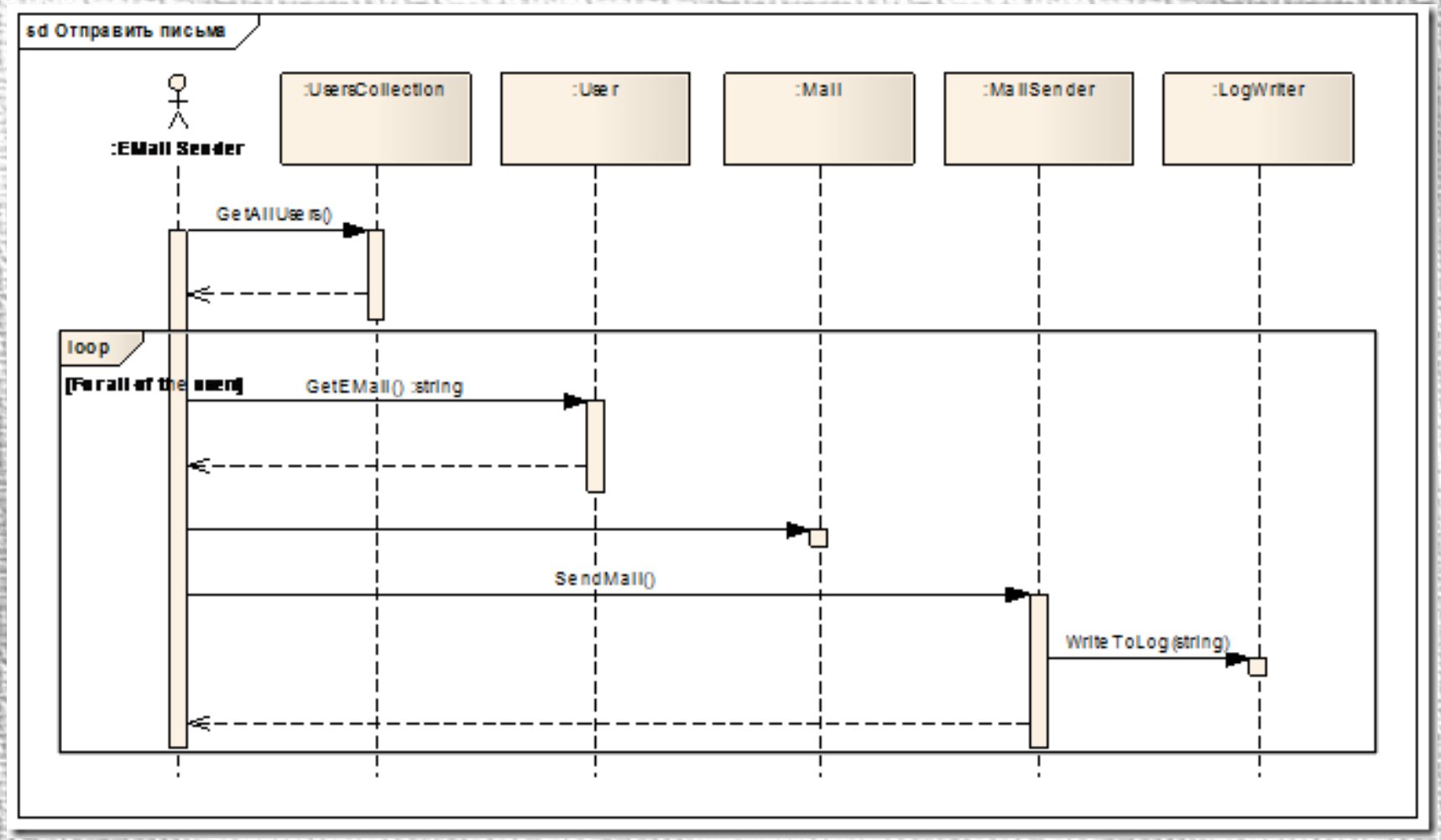
Сообщения могут объединяться в **комбинированные фрагменты** (**Combined fragment**), которые предназначены для отображения циклов, ветвлений, критических секций и прочее.

Пример ветвления:

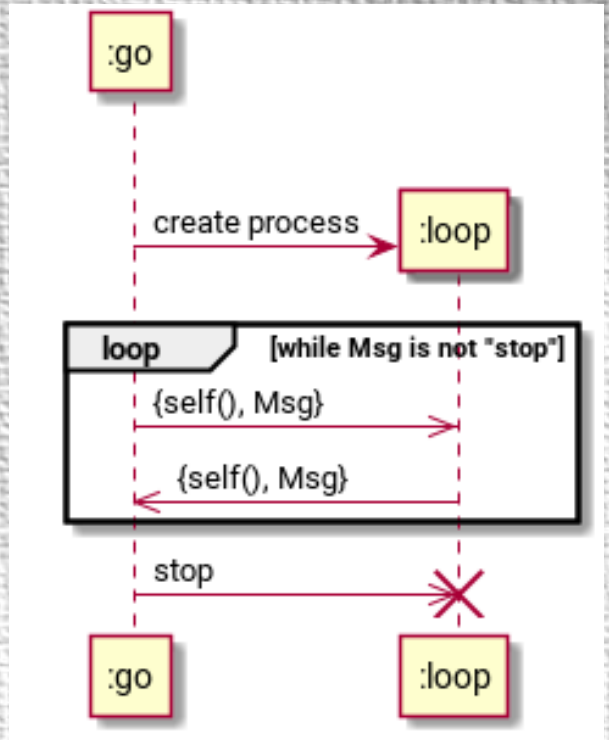


Combined fragment

Пример цикла:

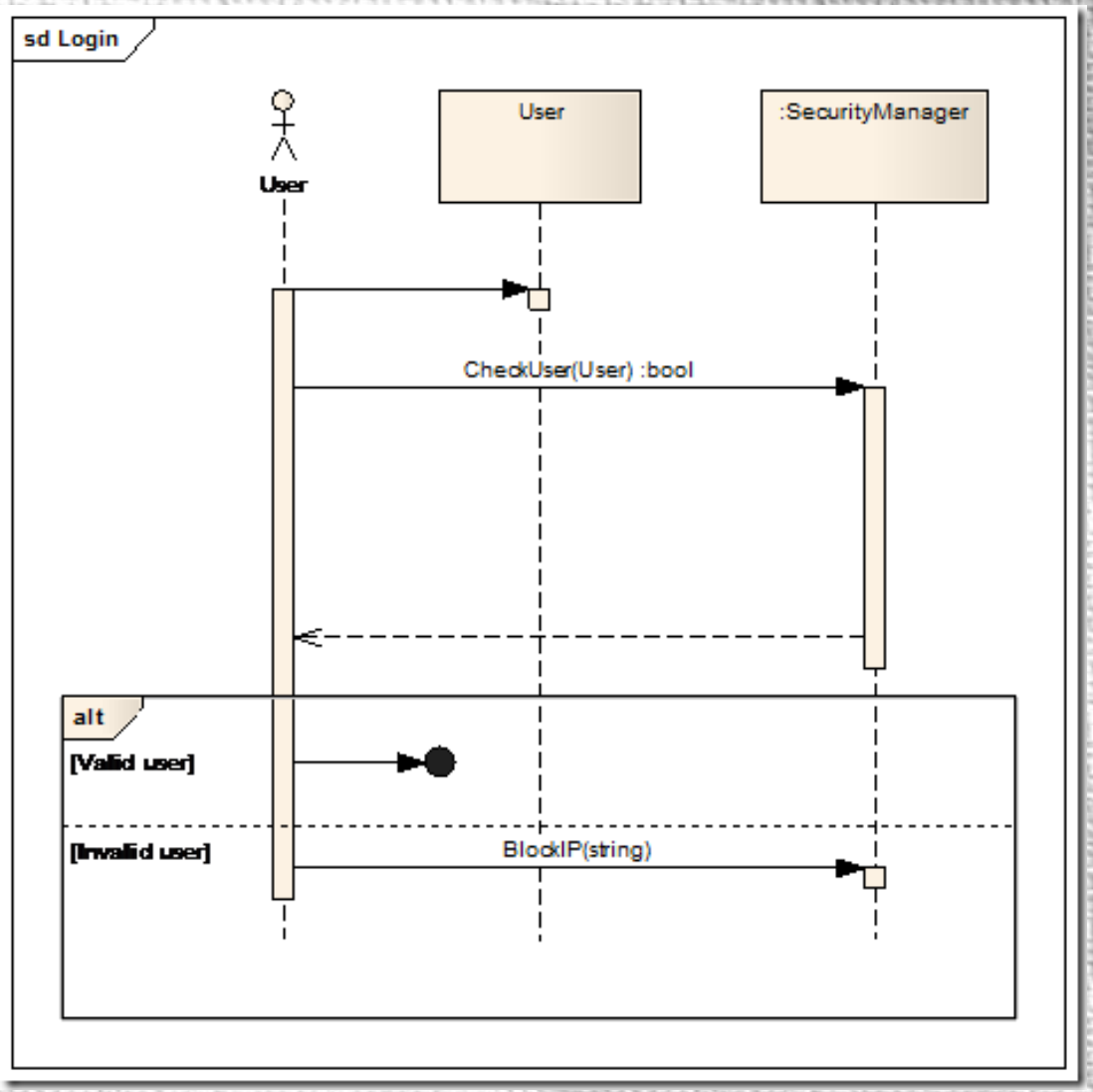


На приведенной диаграмме показано взаимодействие процессов — главный процесс (*go*) после запуска создает дочерний (*loop*), затем в цикле посылает ему свой идентификатор (*self()*) и некоторое сообщение (*Msg*). Дочерний процесс отправляет назад сообщение, но уже со своим идентификатором. Внутри части *фрейма loop* подписывается условие выхода из цикла - в данном случае пересылка сообщений продолжается до тех пор, пока *Msg* не станет равно *«stop».* После цикла главный процесс отправляет дочернему сообщение *stop,* которое является сигналом завершения процесса.



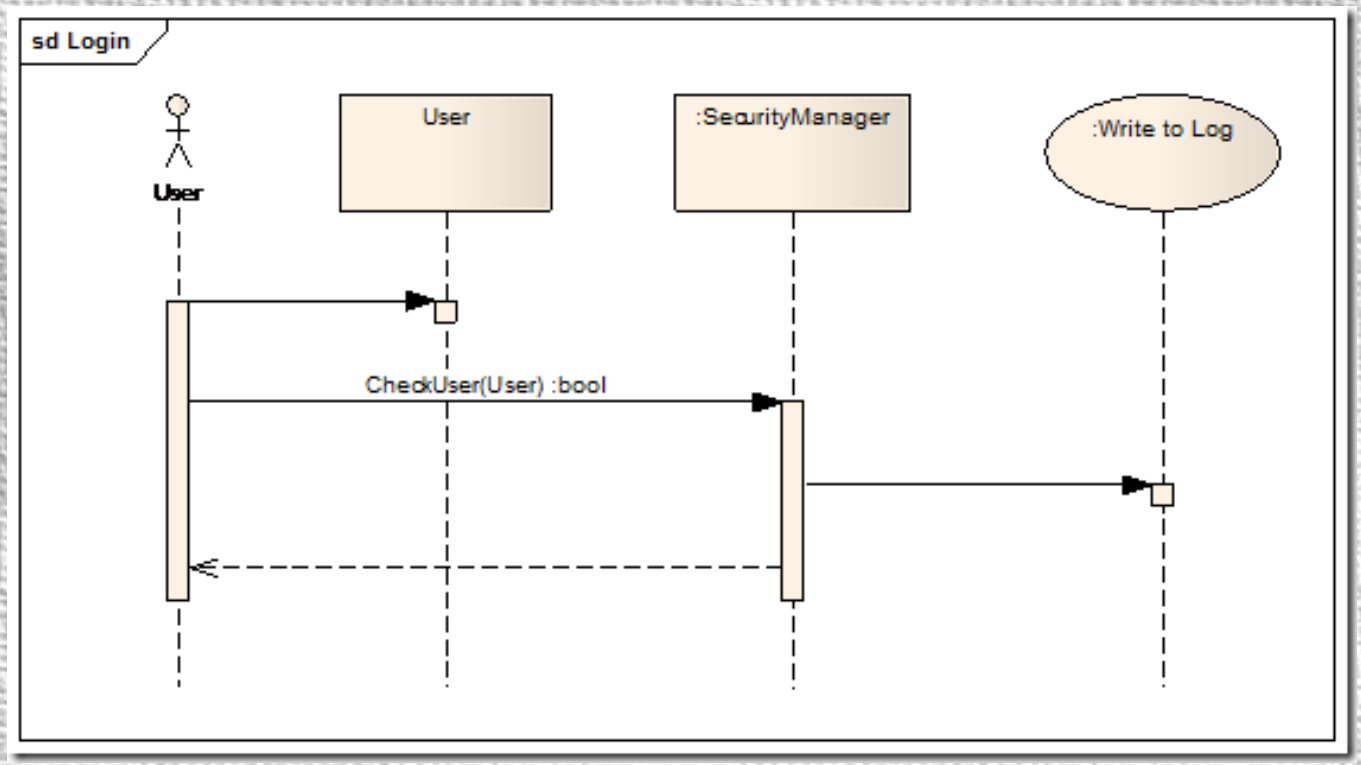
End point

Некоторые сообщения могут заканчиваться **Конечными точками** (End point), которые означают выход из алгоритма.



Другие объекты

В качестве участника диаграммы могут выступать не только классы, но и любые другие объекты – например, варианты использования в тех случаях, когда есть необходимость продемонстрировать включение или расширение.



Линия жизни объекта (object lifeline)

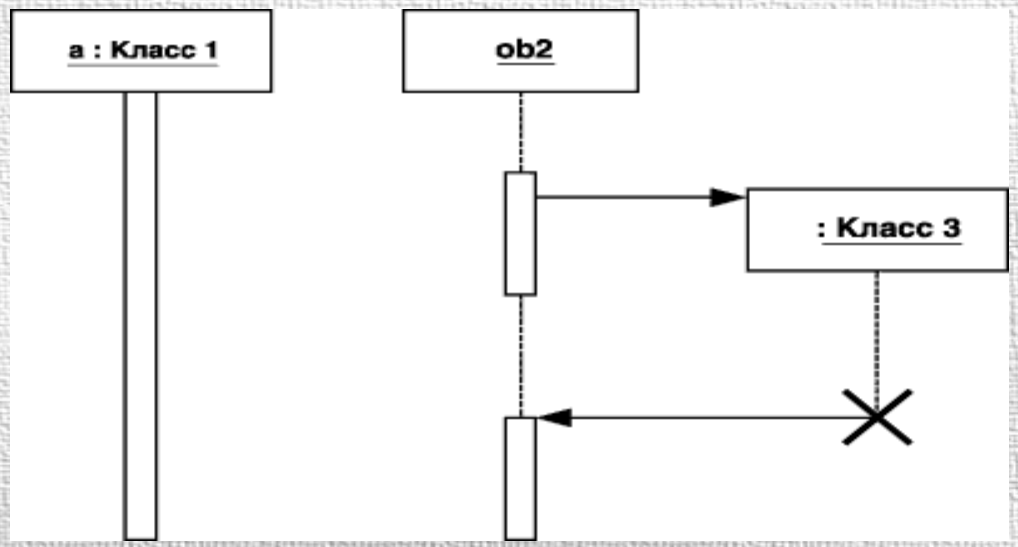
Линия жизни объекта (object lifeline) - вертикальная линия на диаграмме последовательности, которая представляет существование объекта в течение определенного периода времени.

Линия жизни объекта изображается пунктирной вертикальной линией, ассоциированной с единственным *объектом* на *диаграмме последовательности*. *Линия жизни* служит для обозначения периода времени, в течение которого *объект* существует в системе и, следовательно, может потенциально участвовать во всех ее взаимодействиях.

Если *объект* существует в системе постоянно, то и его *линия жизни* должна продолжаться по всей рабочей области *диаграммы последовательности* от самой верхней ее части до самой нижней (*объект* 1 и анонимный *объект* Класса 2 на рисунке).

Отдельные *объекты*, закончив выполнение своих операций, могут быть уничтожены , чтобы освободить занимаемые ими ресурсы. Для таких *объектов линия жизни* обрывается в момент его уничтожения.

Создание объекта



При этом прямоугольник *объекта* вертикально располагается в том месте диаграммы, которое по оси времени совпадает с моментом его возникновения в системе. *Объект* создается со своей *линией жизни* а, возможно, и с *фокусом управления*.

Вовсе не обязательно создавать все *объекты* в начальный момент времени. Отдельные *объекты* в системе могут создаваться по мере необходимости, существенно экономя ресурсы системы и повышая ее производительность. В этом случае прямоугольник такого *объекта* изображается не в верхней части *диаграммы последовательности*, а в той, которая соответствует моменту создания *объекта* (анонимный *объект*, образованный от Класса 3 на рис.).

Фокус управления
(focus of control)

В процессе функционирования объектно-ориентированных систем одни *объекты* могут находиться в активном состоянии, непосредственно выполняя определенные действия, или в состоянии пассивного ожидания *сообщений* от других *объектов*. ***Фокус управления*** - символ, применяемый для того, чтобы явно выделить подобную активность *объектов* на *диаграммах последовательности*.

Фокус управления (focus of control) - специальный символ на диаграмме последовательности, указывающий период времени, в течение которого объект выполняет некоторое действие, находясь в активном состоянии.

Фокус управления изображается в форме вытянутого узкого прямоугольника . Этот прямоугольник располагается ниже обозначения соответствующего *объекта* и может заменять его *линию жизни* (*объект* **a** на рис. 2), если на всем ее протяжении он активен.

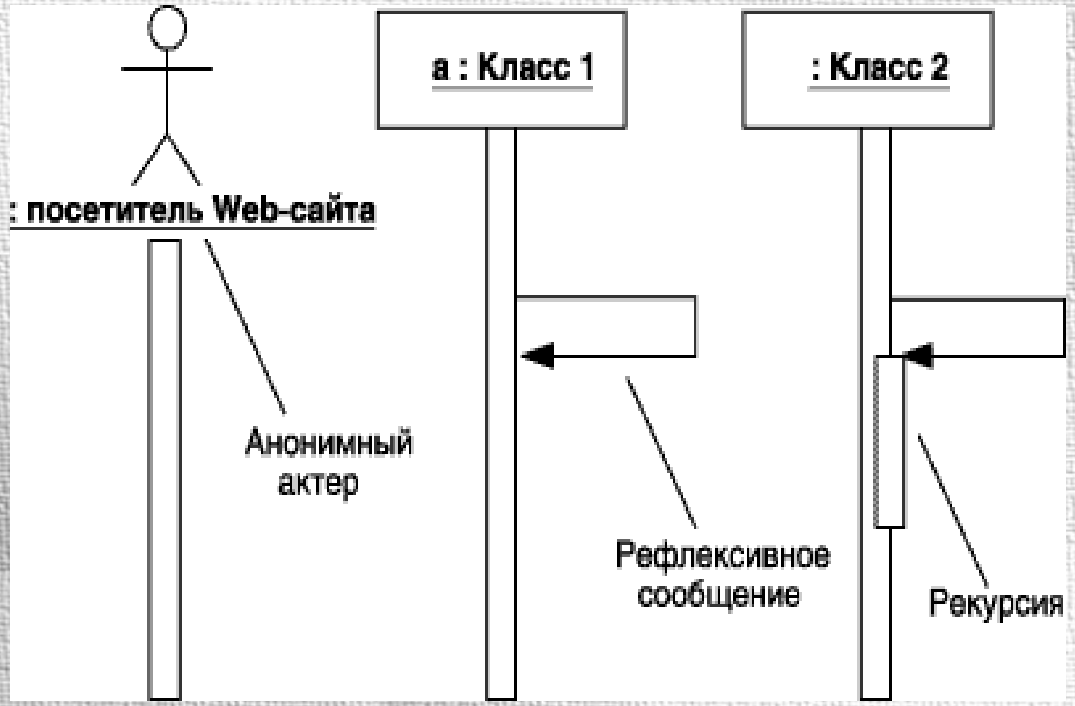
Периоды активности *объекта* могут чередоваться с периодами его пассивности или ожидания. В этом случае у такого *объекта фокусы управления* изменяют свое изображение на *линию жизни* и наоборот (*объект* сирота ob2). Важно понимать, что получить *фокус управления* может только *объект*, у которого в этот момент имеется *линия жизни*. Если же *объект* был уничтожен, то вновь возникнуть в системе он уже не может. Вместо него может быть создан лишь экземпляр этого же класса, который, строго говоря, будет другим *объектом*.

Графическое изображение различных видов сообщений между объектами на ДП

1. Первая разновидность *сообщения* (рис. а) наиболее распространена и используется для вызова процедур, выполнения операций или обозначения отдельных вложенных потоков управления. Начало этой стрелки, как правило, соприкасается с *фокусом управления* того объекта-клиента, который инициирует это *сообщение*. Конец стрелки соприкасается с *линией жизни* того *объекта*, который принимает это *сообщение* и выполняет в ответ определенные действия. При этом принимающий *объект* может получить *фокус управления*, становясь в этом случае активным. Передающий *объект* может потерять *фокус управления* или остаться активным.
2. Вторая разновидность *сообщения* (рис. б) используется для обозначения простого асинхронного *сообщения*, которое передается в произвольный момент времени. Передача такого *сообщения* обычно не сопровождается получением *фокуса управления* объектом-получателем.

3. Третья разновидность *сообщения* (рис. **в**) используется для возврата из вызова процедуры. Примером может служить простое *сообщение* о завершении вычислений без предоставления результата расчетов объекту-клиенту. В процедурных потоках управления эта стрелка может быть опущена, поскольку ее наличие неявно предполагается в конце активизации *объекта*. В тоже время считается, что каждый вызов процедуры имеет свою пару - возврат вызова. Для непроцедурных потоков управления, включая параллельные и асинхронные *сообщения*, стрелка возврата должна указываться явным образом.

Графическое изображение актера, рефлексивного сообщения и рекурсии на диаграмме последовательности



В отдельных случаях *объект* может посылать *сообщения* самому себе, инициируя так называемые **рефлексивные** *сообщения*. Для этой цели служит специальное изображение (*сообщение* у *объекта* **a**). Такие *сообщения* изображаются в форме *сообщения*, начало и конец которого соприкасаются с *линией жизни* или *фокусом управления* одного и того же *объекта*. Подобные ситуации возникают, например, при обработке нажатий на клавиши клавиатуры при вводе текста в редактируемый документ, при наборе цифр номера телефона абонента.

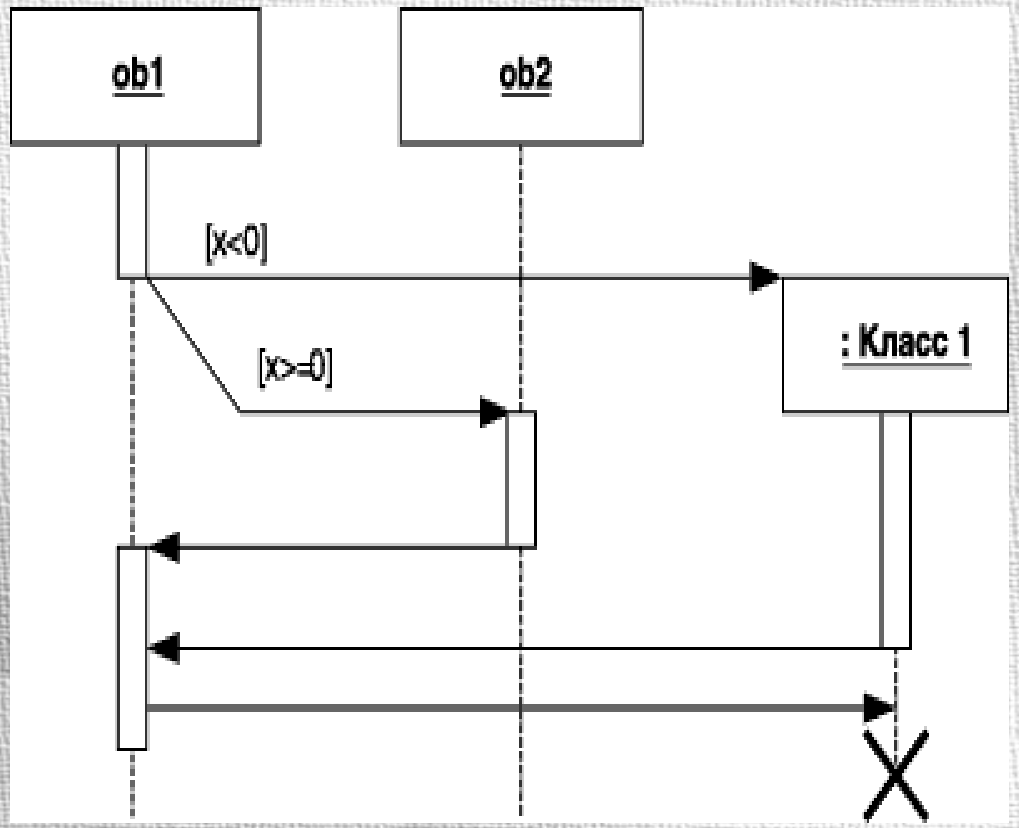
Вложенный *фокус управления*

Если в результате рефлексивного *сообщения* создается новый подпроцесс или нить управления, то говорят о рекурсивном или вложенном *фокусе управления*. На *диаграмме последовательности* рекурсия обозначается небольшим прямоугольником, присоединенным к правой стороне *фокуса управления* того *объекта*, для которого изображается данное рекурсивное взаимодействие (анонимный *объект* Класса 2).

Обычно *сообщения* изображаются горизонтальными стрелками, соединяющими *линии жизни* или *фокусы управления* двух *объектов* на *диаграмме последовательности*. При этом неявно предполагается, что время передачи *сообщения* достаточно мало по сравнению с процессами выполнения действий *объектами*. Считается также, что за время передачи *сообщения* с соответствующими *объектами* не может произойти никаких событий. Другими словами, состояния *объектов* не изменяются. Если же это предположение не может быть признано справедливым, то стрелка *сообщения* изображается под наклоном, так чтобы конец стрелки располагался ниже ее начала.

- Каждое сообщение на диаграмме последовательности ассоциируется с определенной операцией, которая должна быть выполнена принявшим его объектом. При этом операция может иметь аргументы или параметры, значения которых влияют на получение различных результатов. Соответствующие параметры операция будет иметь и вызывающее это действие сообщение. Более того, значения параметров отдельных сообщений могут содержать условные выражения, образуя ветвление или альтернативные пути основного потока управления.

Ветвление потока управления



Одна из особенностей *диаграммы последовательности* - возможность визуализировать простое *ветвление* процесса. Для изображения *ветвления* используются две или более стрелки, выходящие из одной точки *фокуса управления объекта* (*объект* **ob1**). При этом рядом с каждой из них должно быть явно указано соответствующее условие ветви в форме булевского выражения.

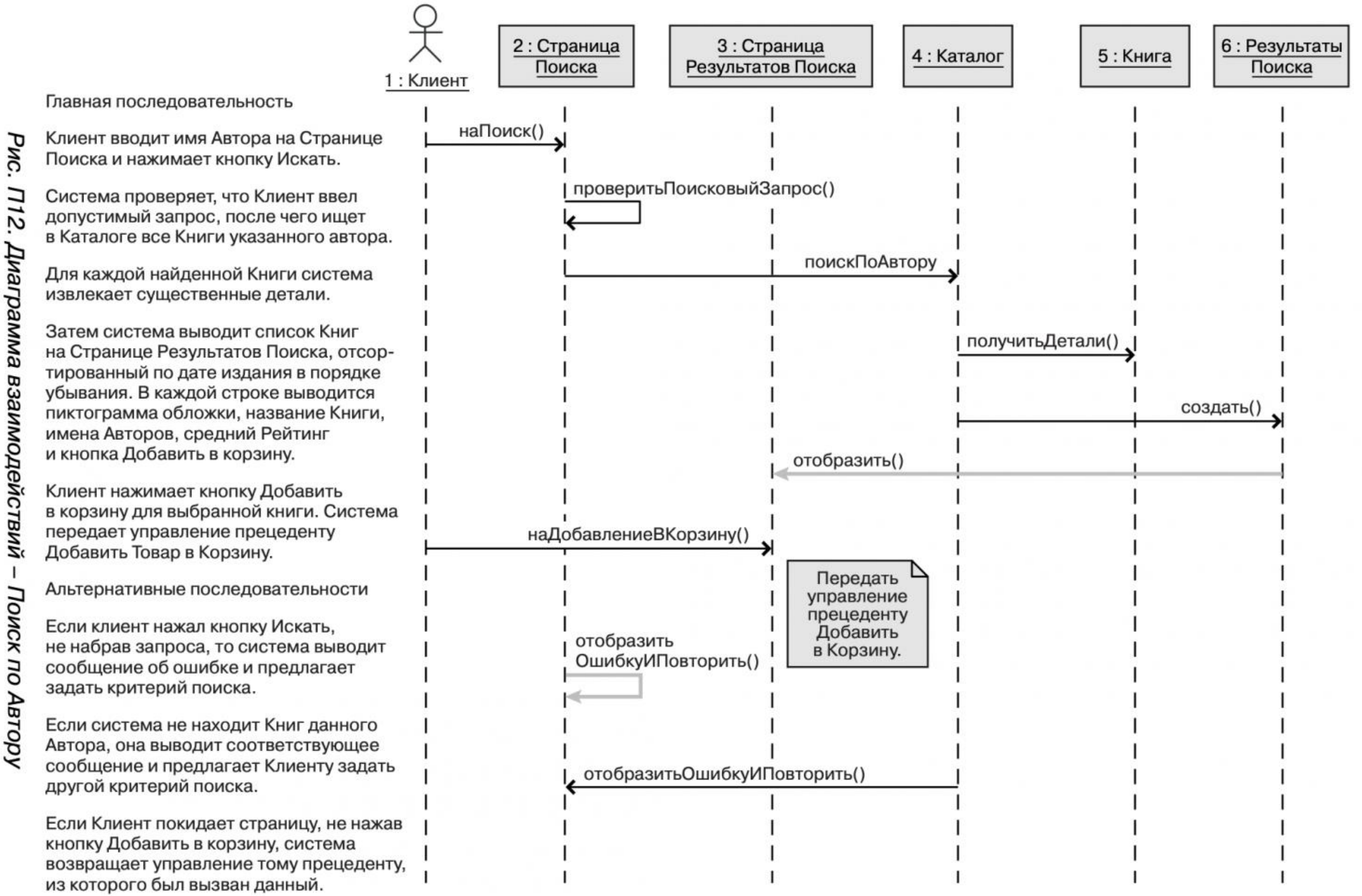
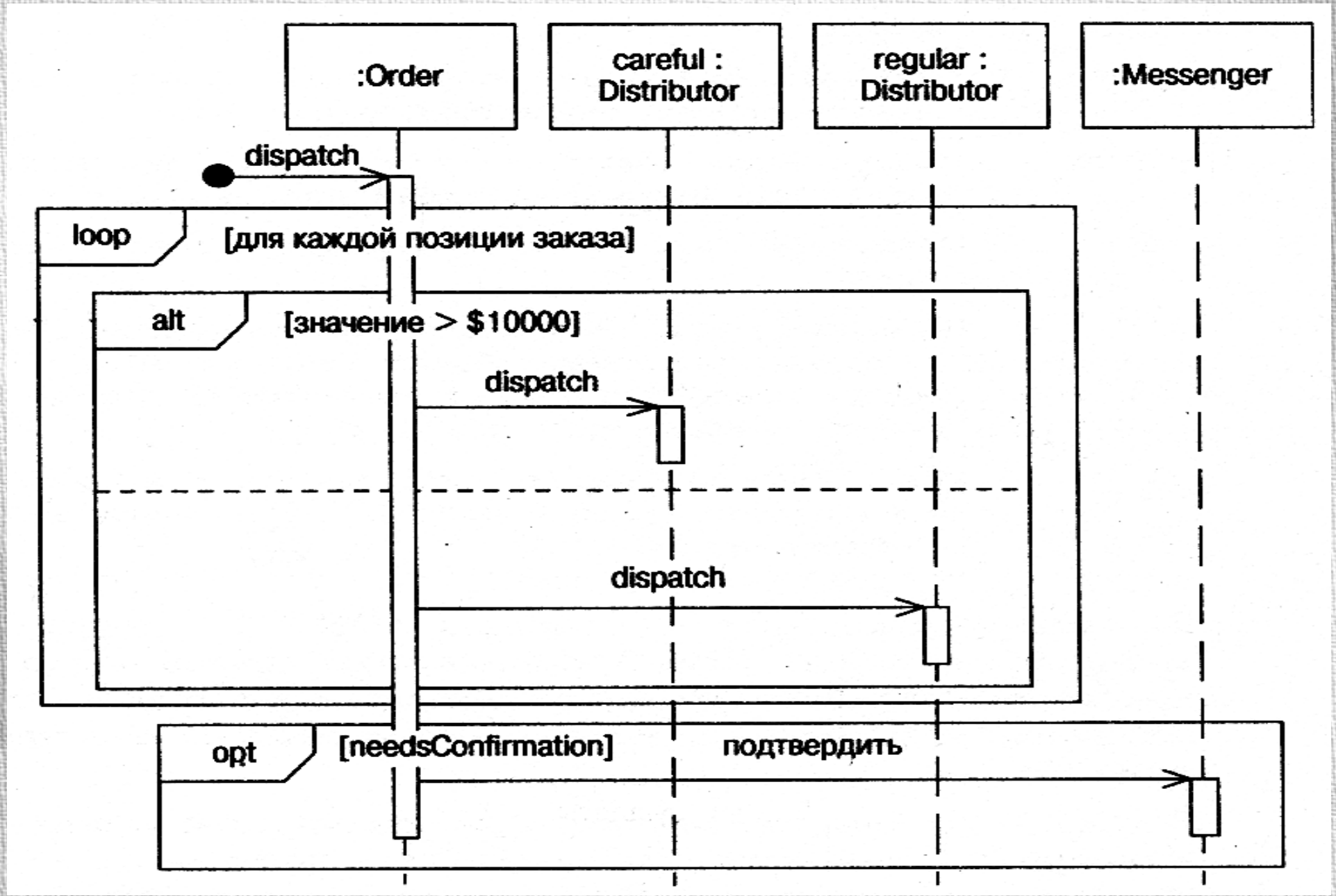


Рис. П12. Диаграмма взаимодействий – Поиск по Автору (диаграмма последовательности)

Типы блоков

В UML 2.0 диаграммы последовательности могут содержать блоки разных типов:

- **alt** – несколько альтернатив (каждая альтернатива – часть блока, помеченная сторожевым условием);
- **opt** – необязательный блок (взаимодействие выполняемое при истинности сторожевого условия);
- **par** – блок из параллельно выполняемых разделов;
- **loop** – цикл (пока истинно условие);
- **region** – критический участок;
- **neg –** неверное взаимодействие;
- **ref** – ссылка на другую диаграмму;
- **sd** – блок, включающий диаграмму последовательности целиком.


:Order
careful : Distributor
regular : Distributor
:Messenger
dispatch
loop
[для каждой позиции заказа]
alt
[значение > $10000]
dispatch
dispatch
opt
[needsConfirmation]
подтвердить

Когда применять?

Существуют различные взгляды на применение этого вида диаграмм:

- Фаулер предлагает строить диаграммы последовательности для визуализации наиболее сложных отношений на диаграмме классов;
- Буч рассматривает их в качестве альтернативы диаграмм объектов и использует для анализа семантики сценариев на ранних стадиях проектирования (до создания протоколов отдельных классов);
- Розенберг строит диаграммы последовательности в рамках процесса ICONIX, поэтому они строятся для каждого прецедента (а не только для наиболее интересных отношений, как у Фаулера). В процессе ICONIX разработке этого вида диаграмм предшествует построение диаграмм робастности (пригодности), поэтому уже выделены объекты, участвующие в прецеденте.

Правила построения

- текст последовательности действий прецедента наносится в виде комментария;
- объекты подписываются в формате “**объект:класс**” и размещаются по горизонтали. Могут изображаться как в виде обычных прямоугольников, так и с использованием обозначений, взятых с диаграммы пригодности (граничный объект, актер,, процесс, сущностный объект);
- время жизни объектов изображается вертикальными пунктирными линиями, выходящими из объектов снизу;
- сообщения (отражают взаимодействие, в том числе вызов функций), изображаются горизонтальными стрелками, концы которых располагаются на линиях жизни. Стрелки направлены от отправителя к адресату и упорядочены по времени с помощью линии жизни;

- интервалы активности объектов отражают периоды, который методы объектов находится в стеке (обычно между получением фокуса управления и завершением обработки). В ряде случаев интервалы активности позволяют различать на диаграмме с чем связана активность (какой метод инициирует передачу сообщений), в остальных случаях их можно опустить (как у виджета добавления студента), т.к. их рисование довольно трудоемко.
- вложенные друг в друга интервалы активности отражают самовызов, т.е. объект главного окна в обработчике сигнала от базы данных вызывает метод show. Оба метода в какой-то момент оказываются в стеке, поэтому один интервал активности изображается поверх другого. Часто такие детали опускаются;
- создание объекта изображается на диаграмме стрелкой, входящей в символ объекта, а уничтожение – крестом на линии жизни.

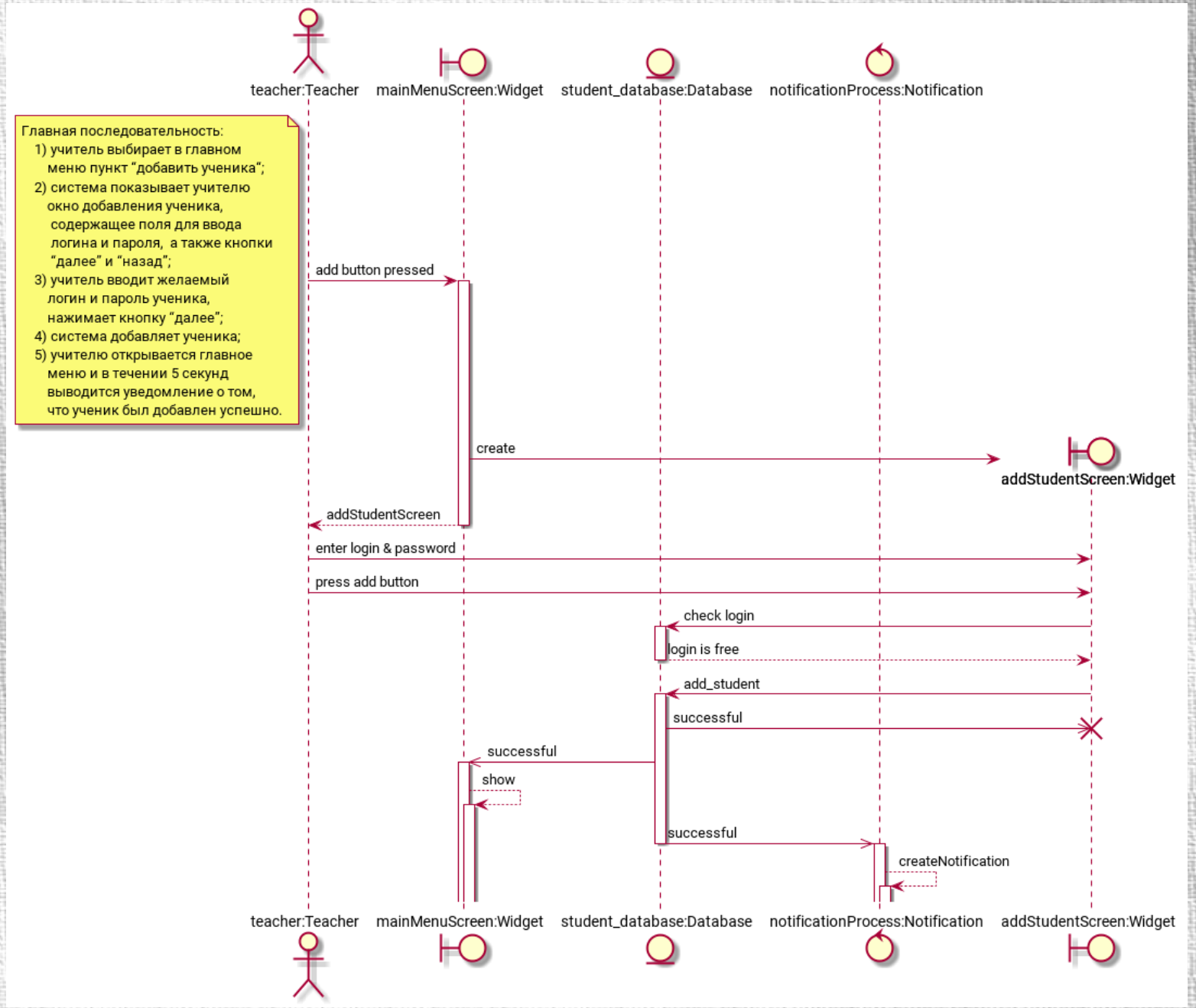
- Sequence Diagrams
- Communication Diagrams

ПРИМЕРЫ

- Sequence Diagrams
- Communication Diagrams

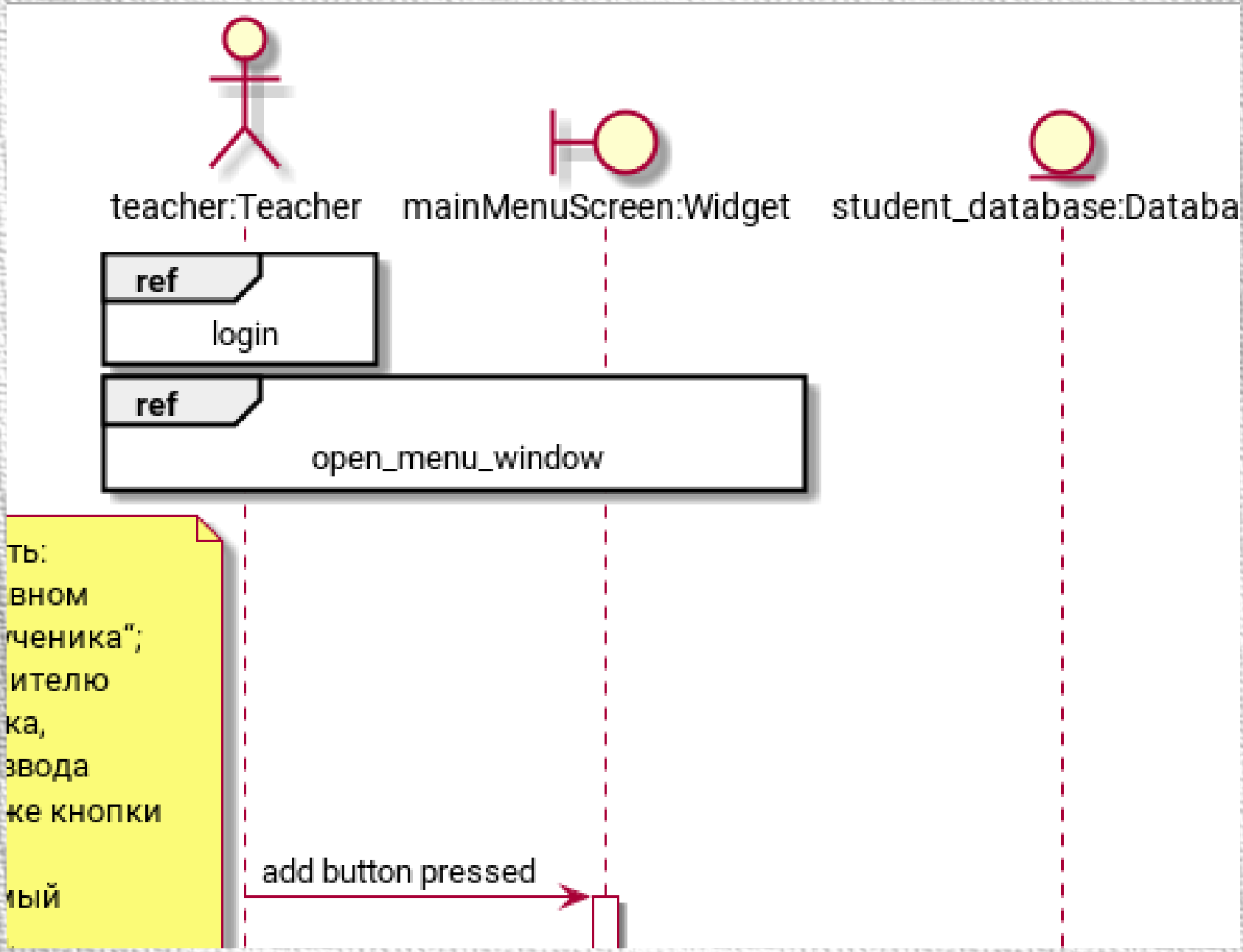
ПРИМЕРЫ

Пример: диаграмма последователь ности сценария добавления ученика в систему тестирования

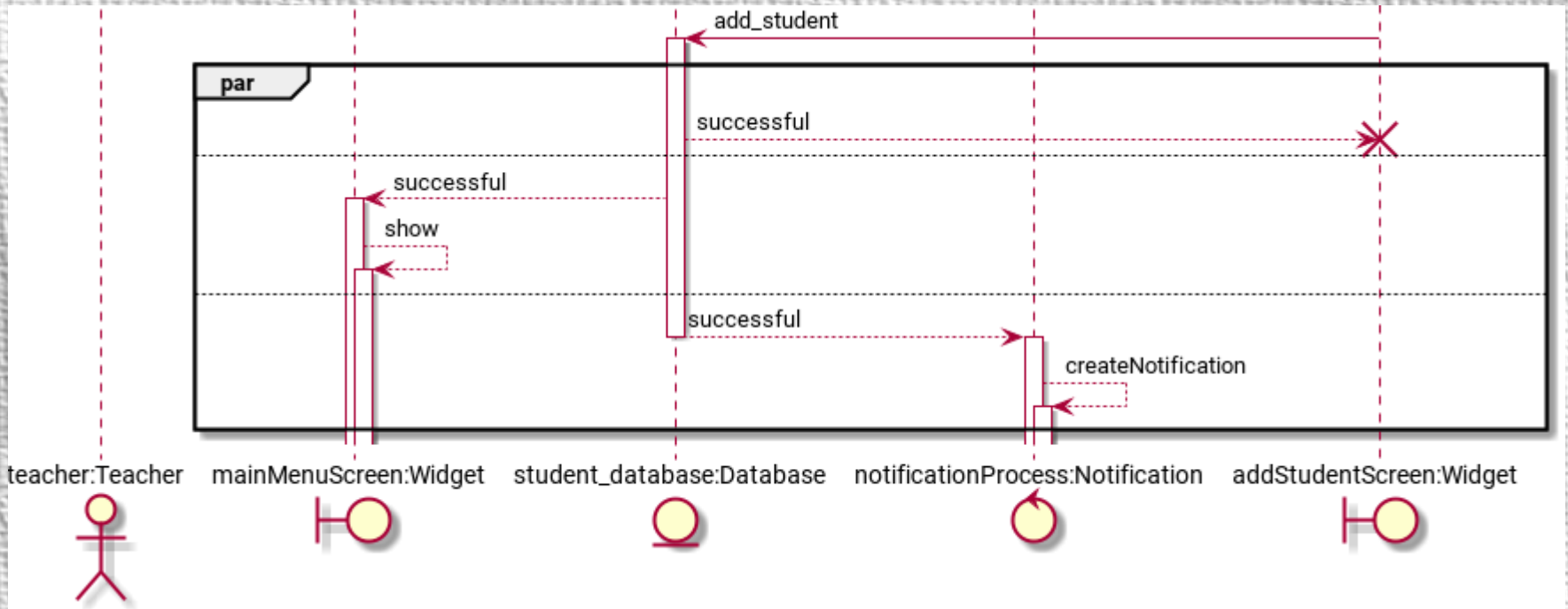


Как отобразить предусловие?

Видно, что приведенная диаграмма очень хорошо отражает взаимодействие объектов, однако прецеденты могут содержать предусловия (требования, которые должны быть выполнены для начала выполнения действий прецедента) — они могут быть показаны при помощи механизма использования взаимодействий. Этот механизм позволяет указать что повторно используется какое-либо взаимодействие, определенное в другом месте, изображается с помощью ***метки ref***. Так, можно показать, что до выполнения прецедента учитель должен войти в систему (выполнить *прецедент login*) и открыть окно главного меню:

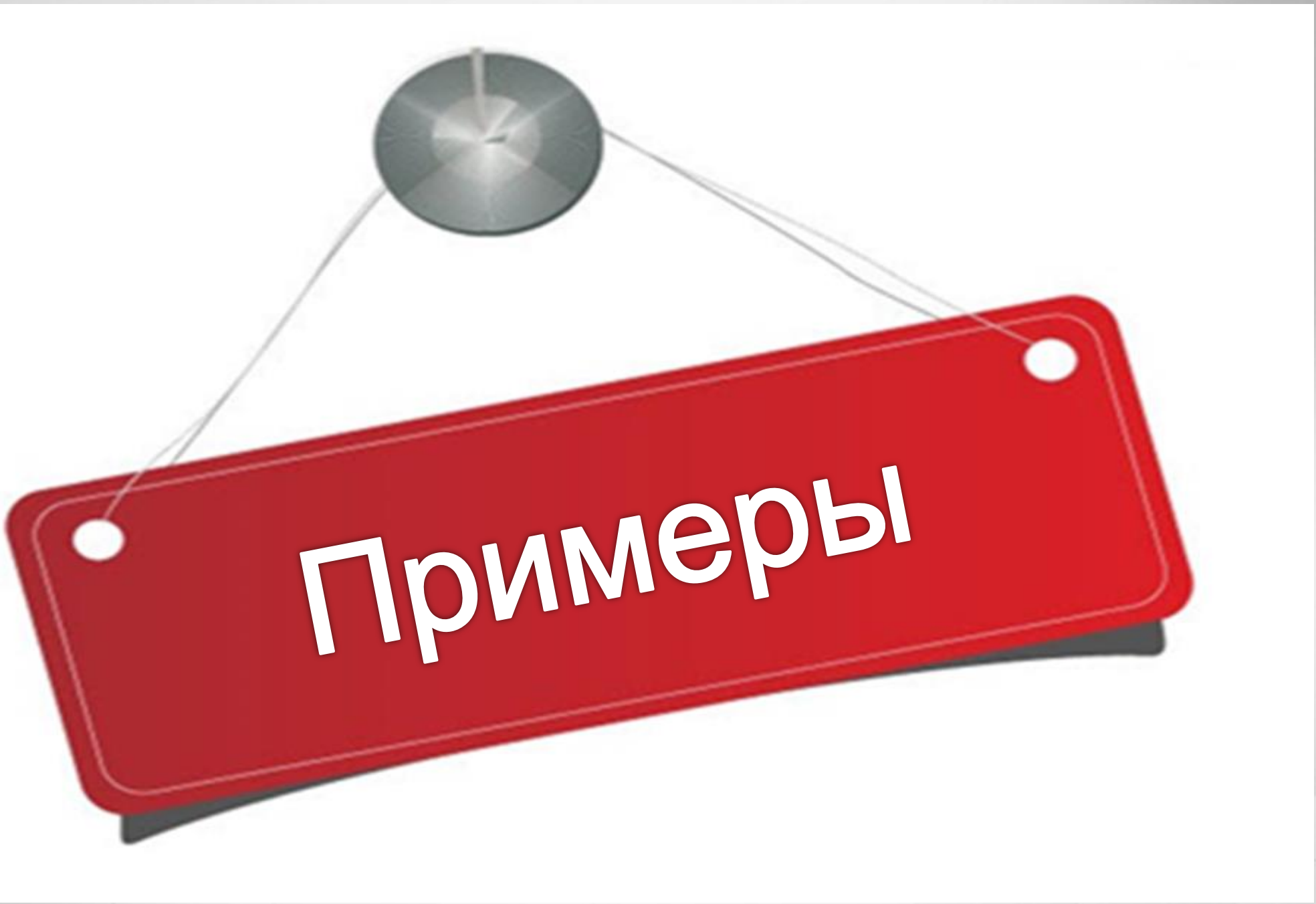

teacher:Teacher
mainMenuScreen:Widget
student_database:Databa
ref
login
ref
open_menu_window
ть:
вном
ченика";
ителю
ка,
звода
же кнопки
мый
add button pressed

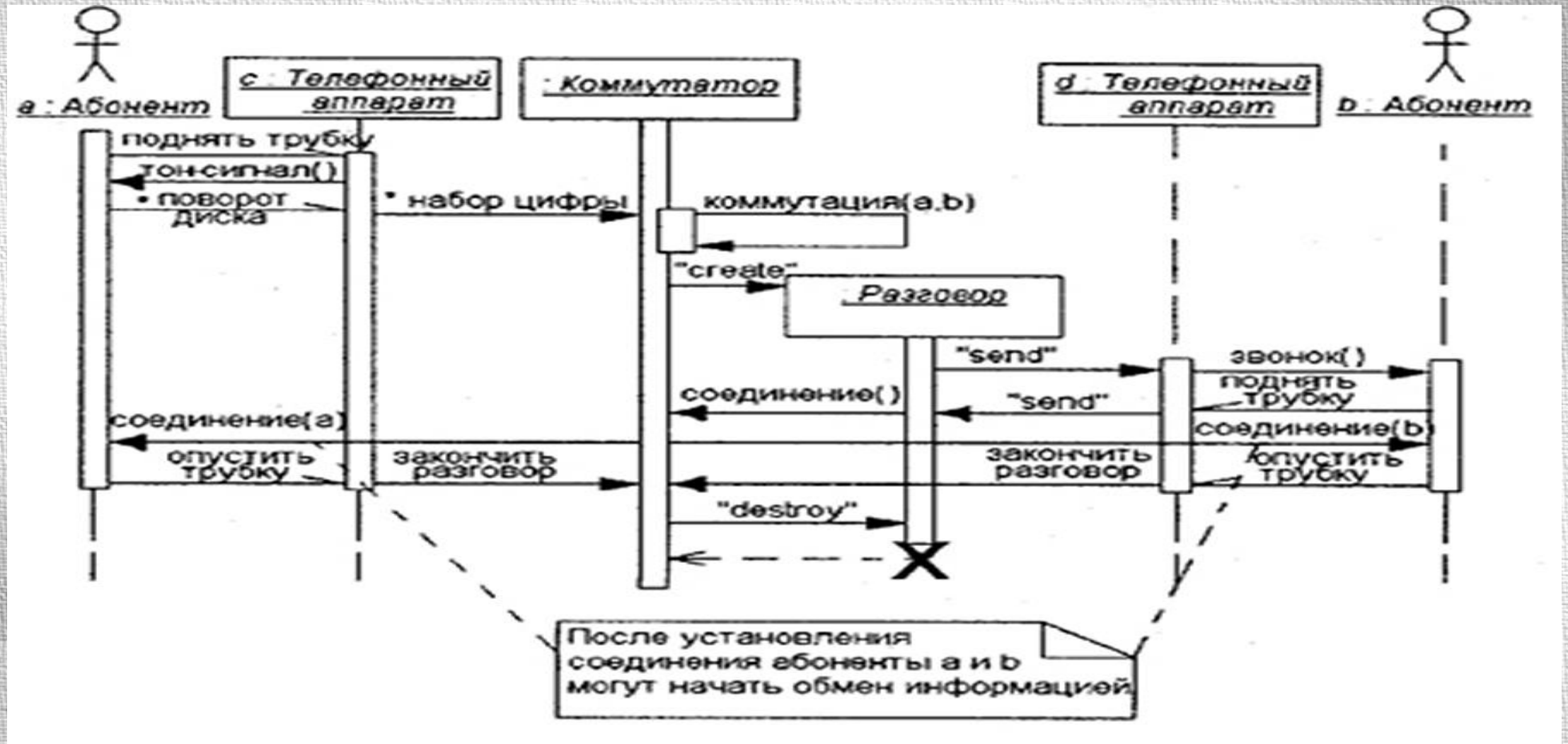
Фреймы взаимодействия могут использоваться для изображения параллельных секций, при этом часть сообщений рисуются внутри *фрейма par*, так например, мы могли бы показать, что база данных уведомляет параллельно всех трех подписчиков и все они параллельно и независимо друг от друга начинают работать:



Три основных вида стрелок

1. **синхронные** (например вызов функции, вызывающий код получает управление после того, как сообщение будет обработано) изображаются сплошной линией с закрытой стрелкой. *Ниже*: *меню добавления студента вызывает функцию проверки логина у объекта базы данных и дожидается ответа*;
2. **асинхронные** (управление возвращается немедленное после передачи сообщения, т.е. сообщение инициирует обработку, но отправитель не дожидается ее завершения) – сплошная линия с открытой стрелкой. В случае успешного добавления студента база данных асинхронно уведомляет подписчиков (см. шаблон проектирования Модель-Вид-Контроллер);
3. **ответ на сообщение** (результат вычислений) – штриховая линия с открытой стрелкой. В случае если логин не занят, метод базы данных возвращает “login is free“.


Примеры


a : Абонент
c : Телефонный аппарат
: Коммутатор
d : Телефонный аппарат
b : Абонент
поднять трубку
тон сигнал()
• поворот диска
* набор цифры
коммутация(a,b)
"create"
Разговор
"send"
звонок()
поднять трубку
"send"
соединение()
соединение(a)
соединение(b)
опустить трубку
закончить разговор
закончить разговор
опустить трубку
"destroy"
После установления соединения абоненты a и b могут начать обмен информацией

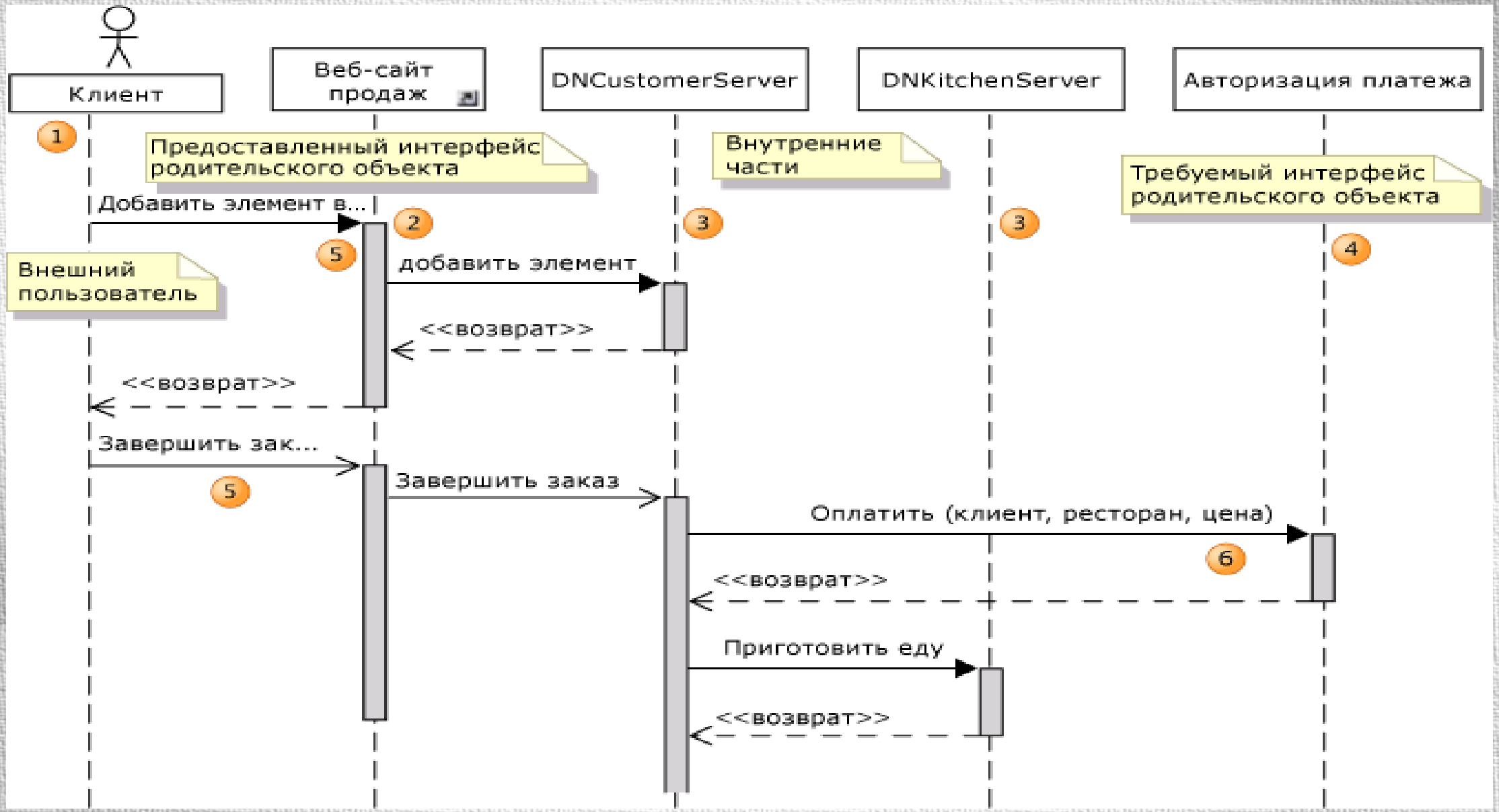
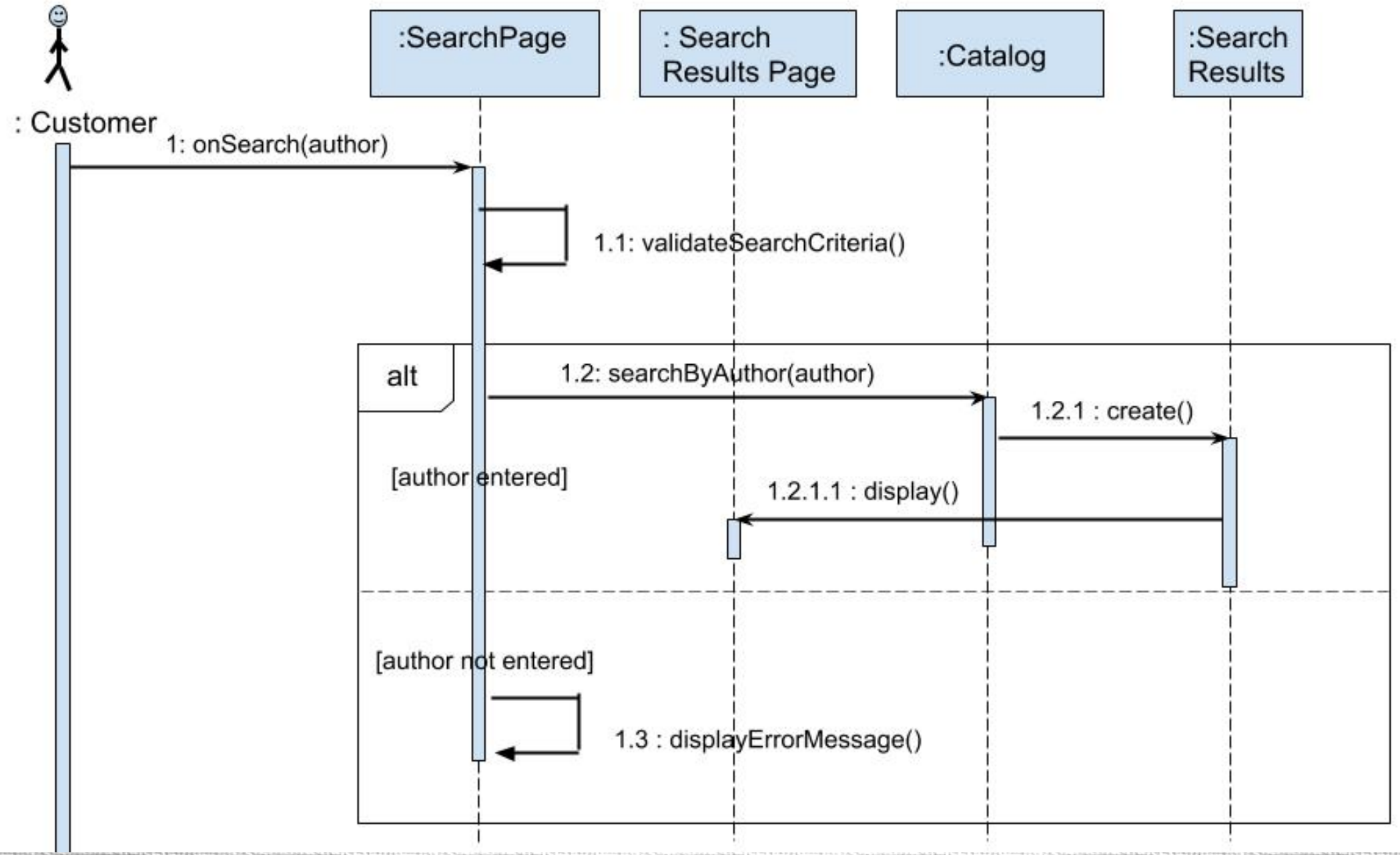
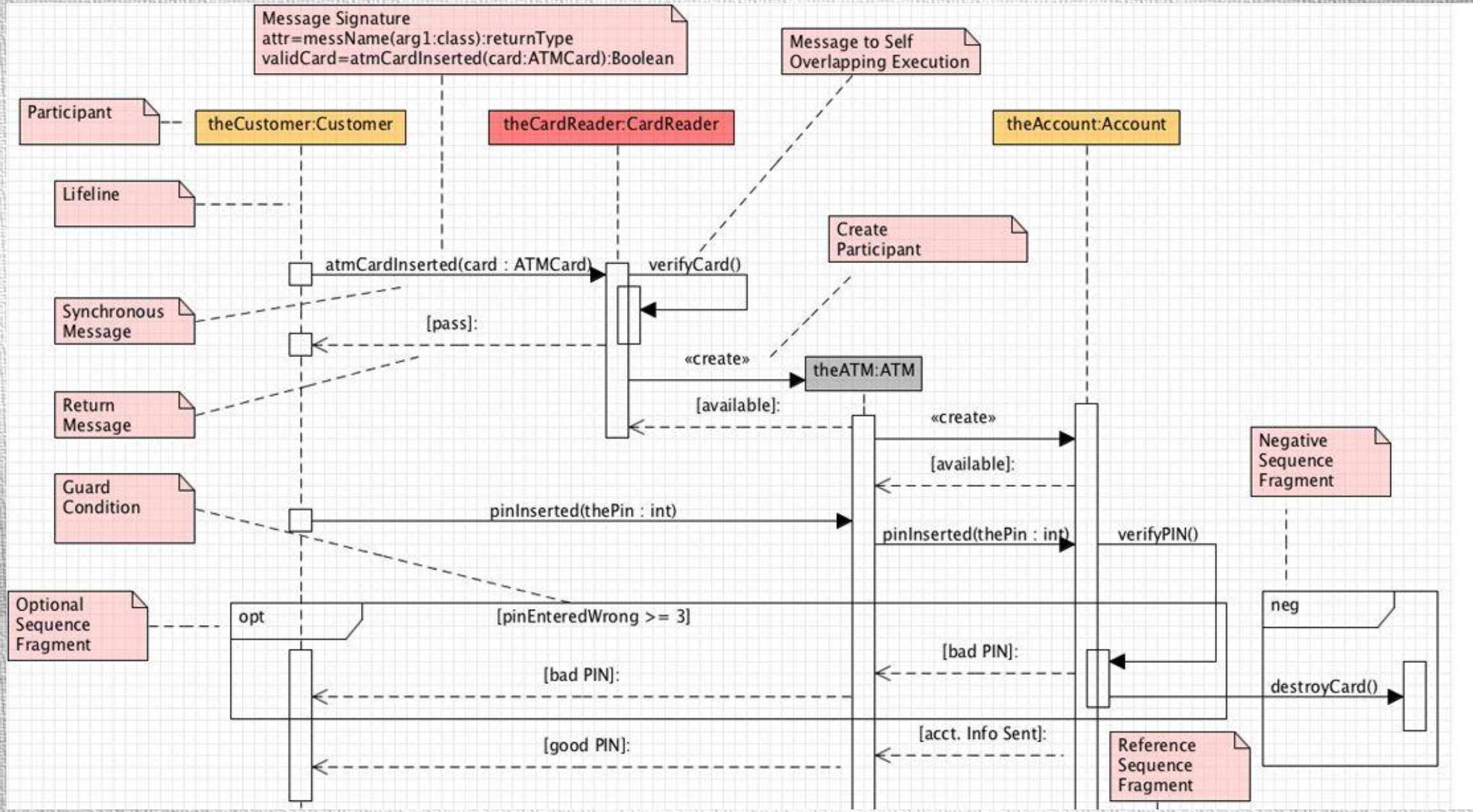

Клиент
Веб-сайт продаж
DNCustomerServer
DNKitchenServer
Авторизация платежа
1
Предоставленный интерфейс родительского объекта
Внутренние части
Требуемый интерфейс родительского объекта
Добавить элемент в...
2
3
3
4
5
Внешний пользователь
добавить элемент
<<возврат>>
<<возврат>>
Завершить зак...
5
Завершить заказ
Оплатить (клиент, ресторан, цена)
6
<<возврат>>
Приготовить еду
<<возврат>>

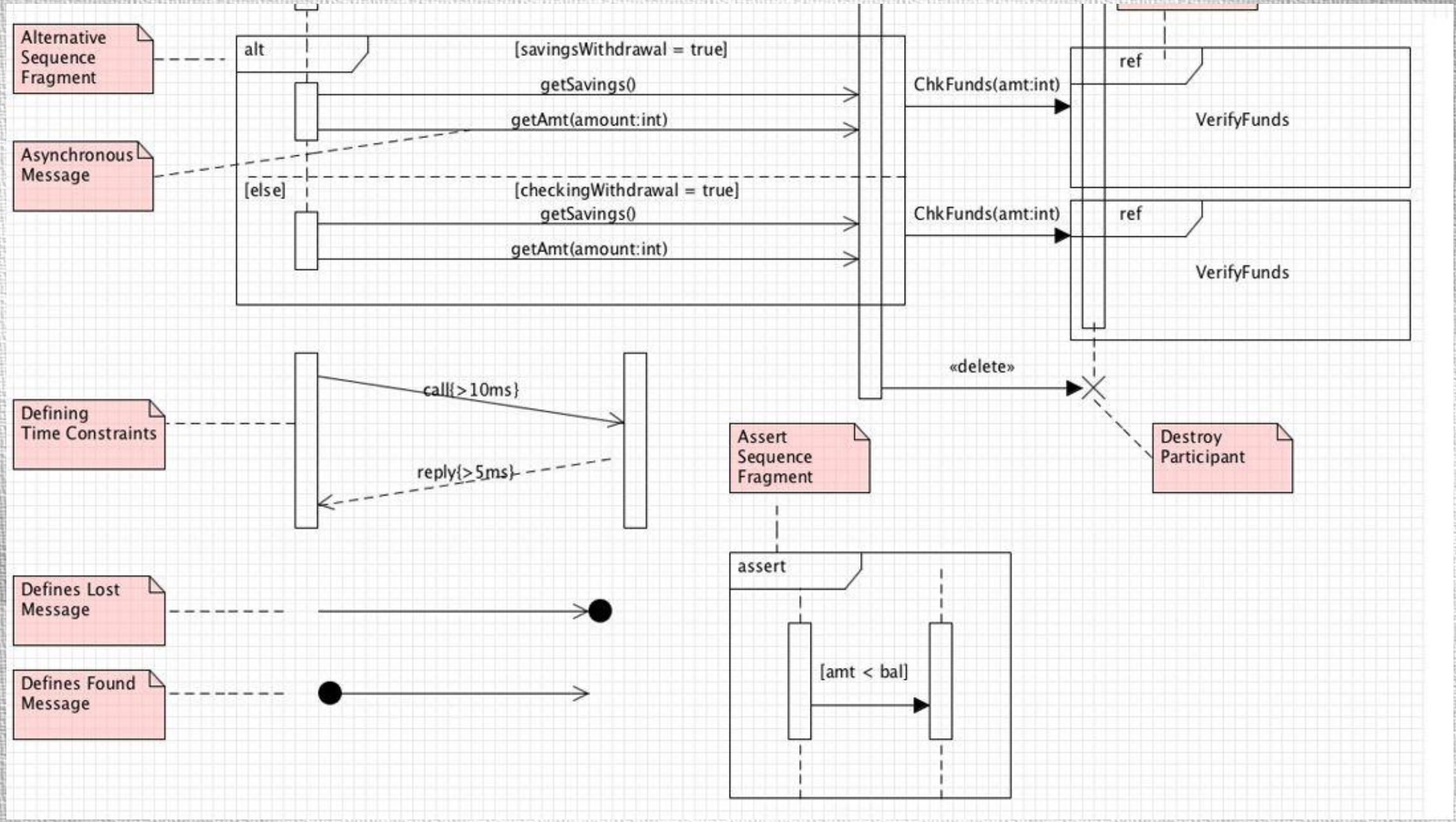
Диаграмма Последовательности

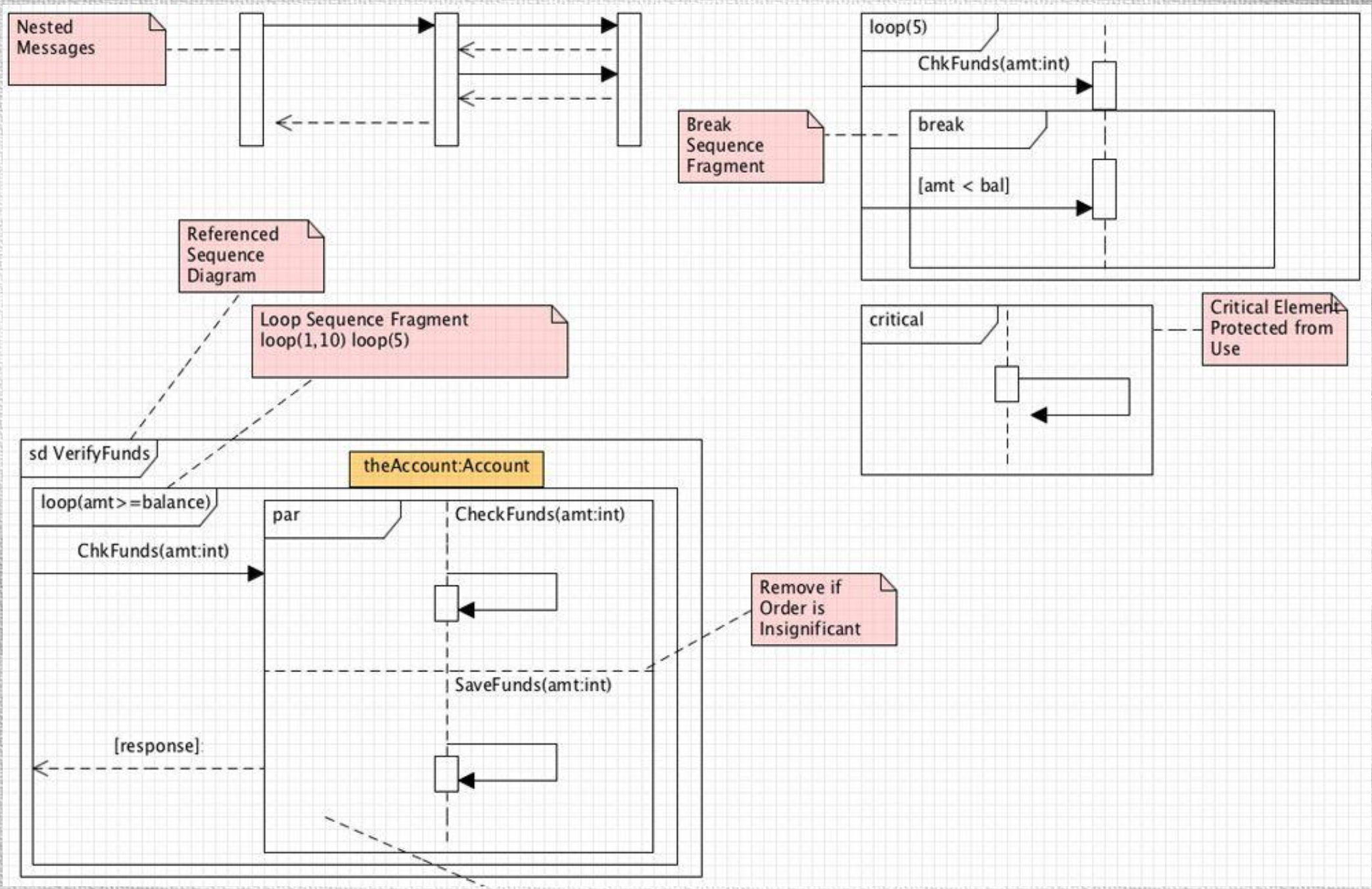


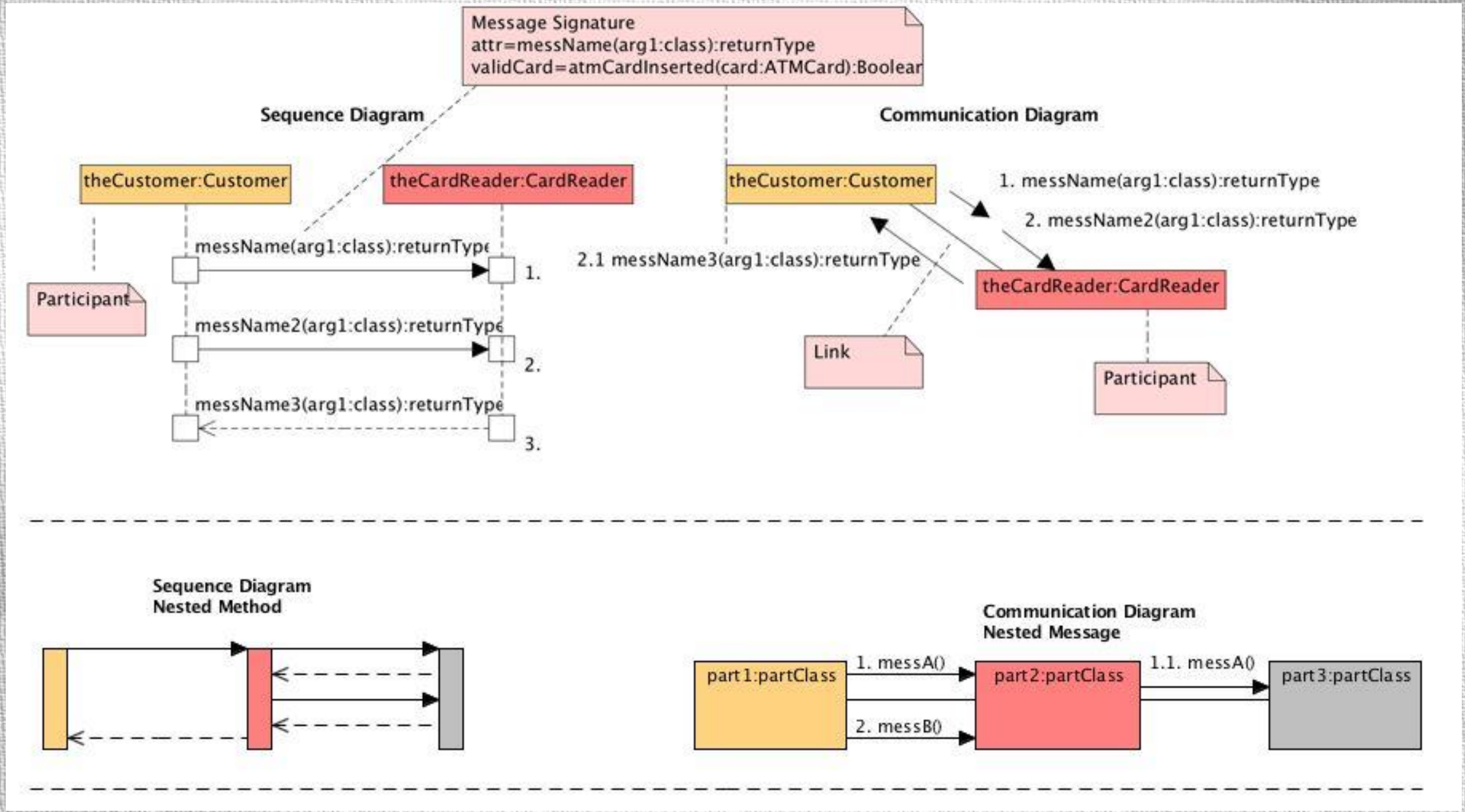
- Sequence Diagrams
- Communication Diagrams

ШПАРГАЛКА

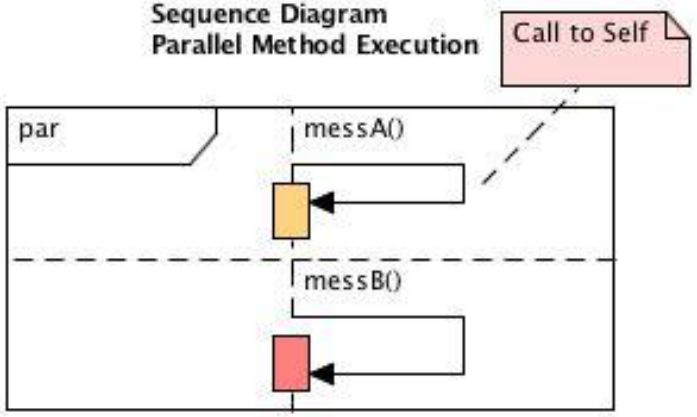

Message Signature
attr=messName(arg1:class):returnType
validCard=atmCardInserted(card:ATMCard):Boolean
Message to Self
Overlapping Execution
Participant
theCustomer:Customer
theCardReader:CardReader
theAccount:Account
Lifeline
Create
Participant
atmCardInserted(card : ATMCard)
verifyCard()
Synchronous
Message
[pass]:
«create»
theATM:ATM
Return
Message
[available]:
«create»
Negative
Sequence
Fragment
[available]:
Guard
Condition
pinInserted(thePin : int)
pinInserted(thePin : int)
verifyPIN()
Optional
Sequence
Fragment
opt
[pinEnteredWrong >= 3]
neg
[bad PIN]:
[bad PIN]:
destroyCard()
[acct. Info Sent]:
[good PIN]:
Reference
Sequence
Fragment


Alternative Sequence Fragment
alt
[savingsWithdrawal = true]
getSavings()
getAmt(amount:int)
Asynchronous Message
[else]
[checkingWithdrawal = true]
getSavings()
getAmt(amount:int)
ChkFunds(amt:int)
ref
VerifyFunds
ChkFunds(amt:int)
ref
VerifyFunds
«delete»
Destroy Participant
Defining Time Constraints
call{>10ms}
reply{>5ms}
Assert Sequence Fragment
assert
[amt < bal]
Defines Lost Message
Defines Found Message

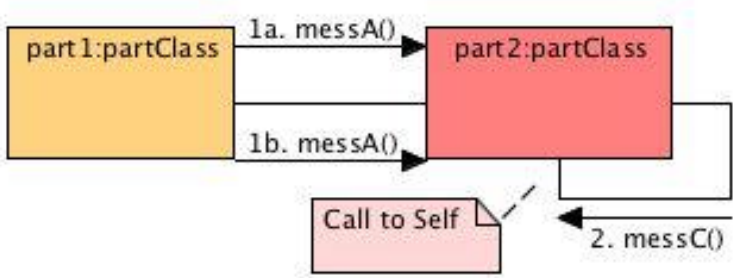

Nested Messages
loop(5)
ChkFunds(amt:int)
Break Sequence Fragment
break
[amt < bal]
Referenced Sequence Diagram
Loop Sequence Fragment loop(1,10) loop(5)
critical
Critical Element Protected from Use
sd VerifyFunds
theAccount:Account
loop(amt>=balance)
par
CheckFunds(amt:int)
ChkFunds(amt:int)
Remove if Order is Insignificant
SaveFunds(amt:int)
[response]


Message Signature
attr=messName(arg1:class):returnType
validCard=atmCardInserted(card:ATMCard):Boolear
Sequence Diagram
Communication Diagram
theCustomer:Customer
theCardReader:CardReader
messName(arg1:class):returnType
1.
Participant
messName2(arg1:class):returnType
2.
messName3(arg1:class):returnType
3.
2.1 messName3(arg1:class):returnType
1. messName(arg1:class):returnType
2. messName2(arg1:class):returnType
Link
Participant
Sequence Diagram
Nested Method
Communication Diagram
Nested Message
part1:partClass
1. messA()
2. messB()
part2:partClass
1.1. messA()
part3:partClass

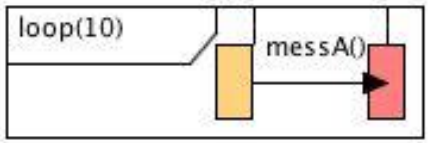
Sequence Diagram
Parallel Method Execution



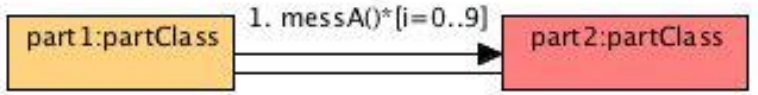
Communication Diagram
Parallel Method Execution



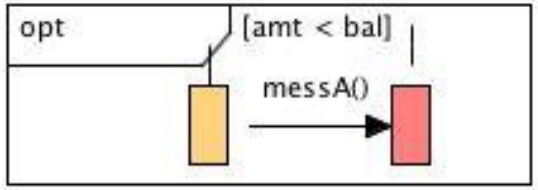
Sequence Diagram
Execute Multiple Times



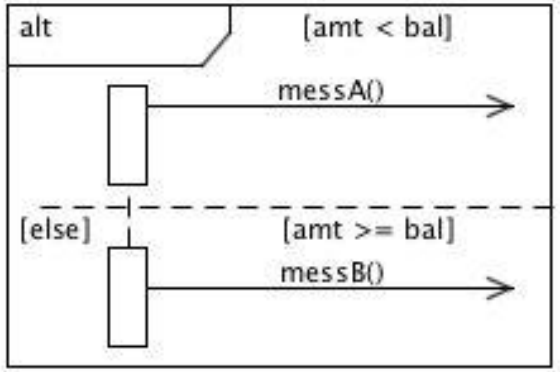
Communication Diagram
Execute Multiple Times



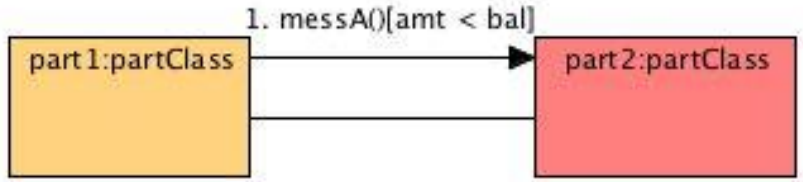
Sequence Diagram
Conditional Execution

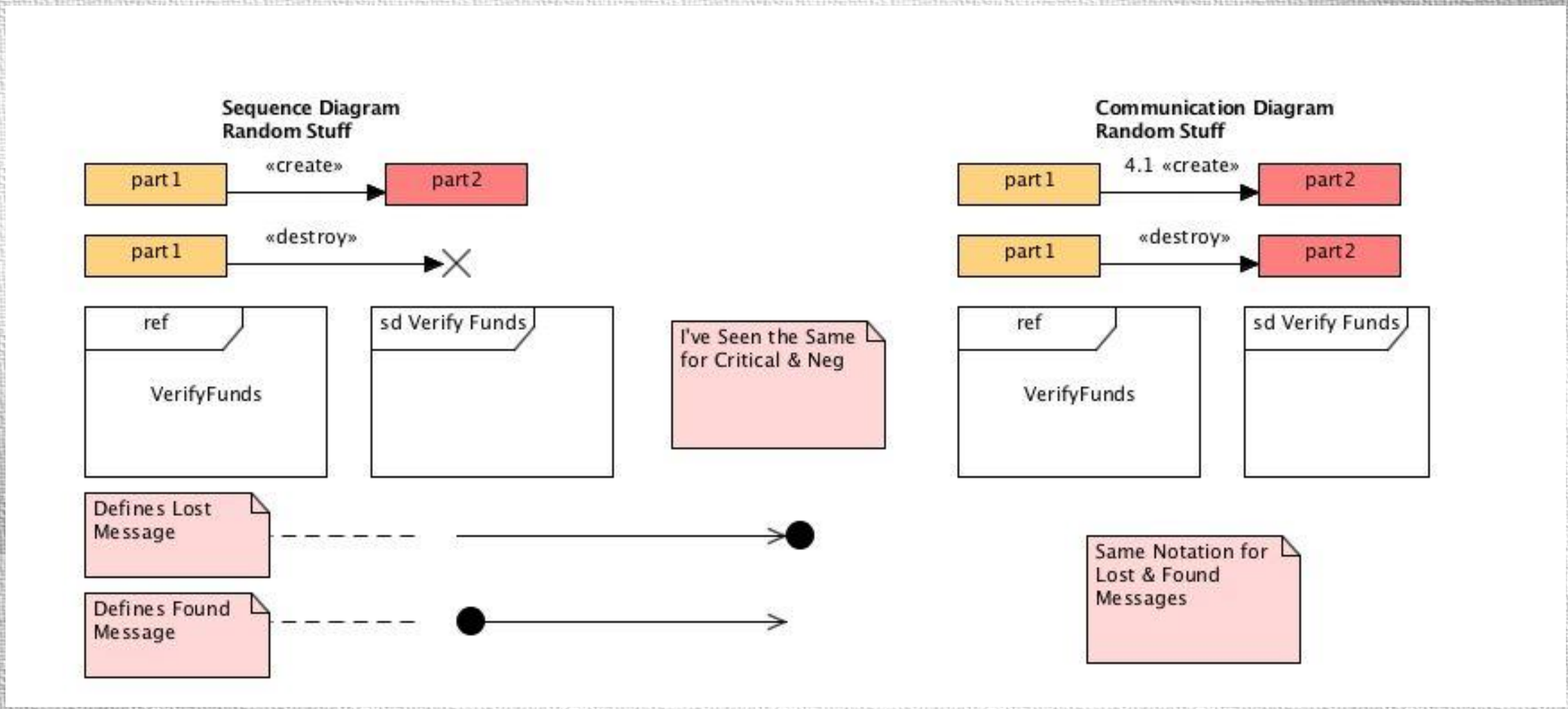


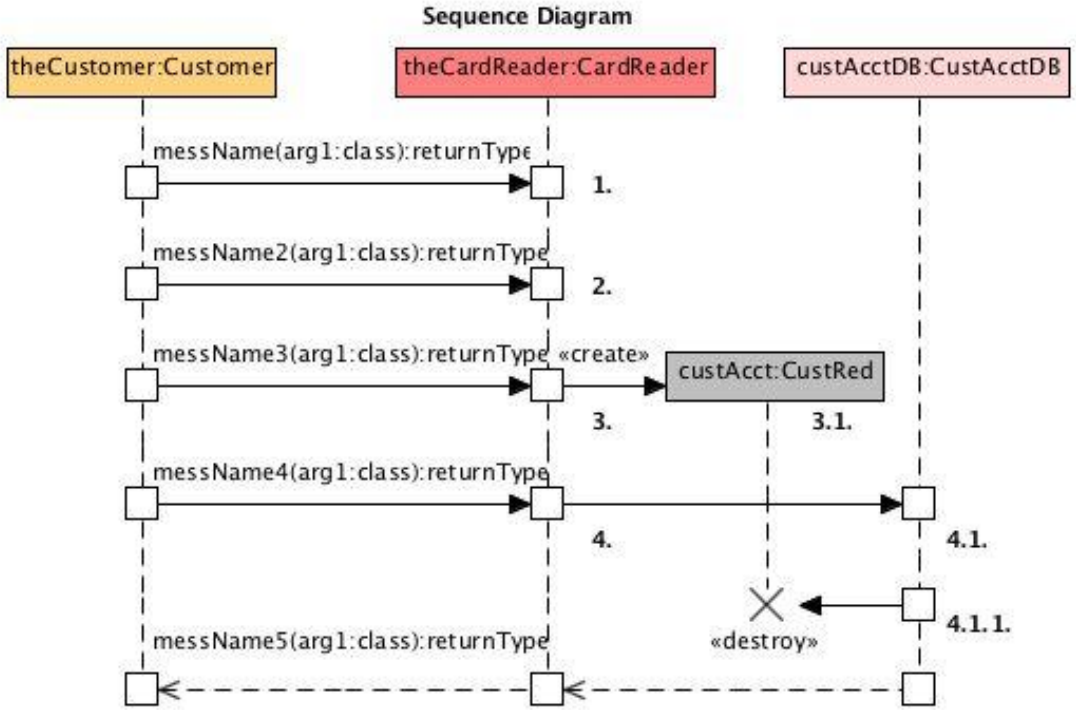
Assert Works the Same

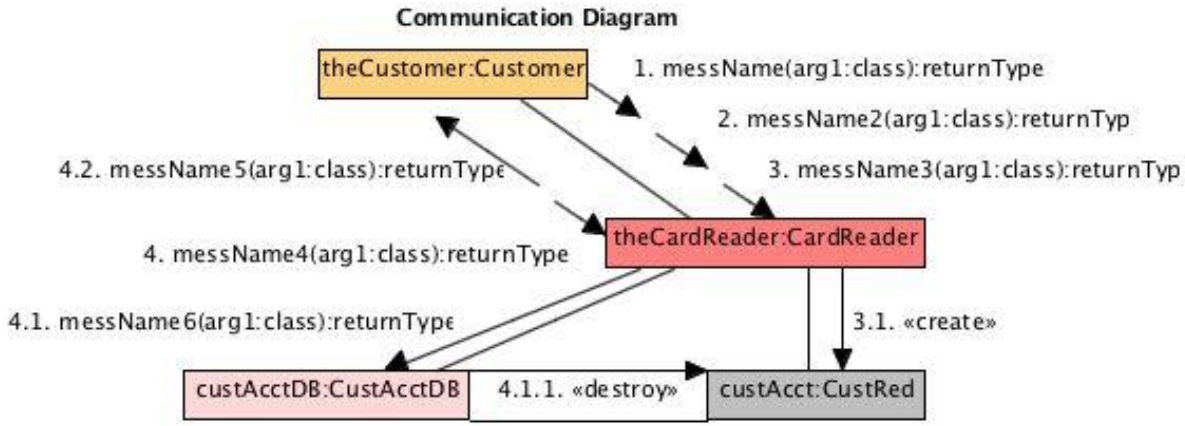


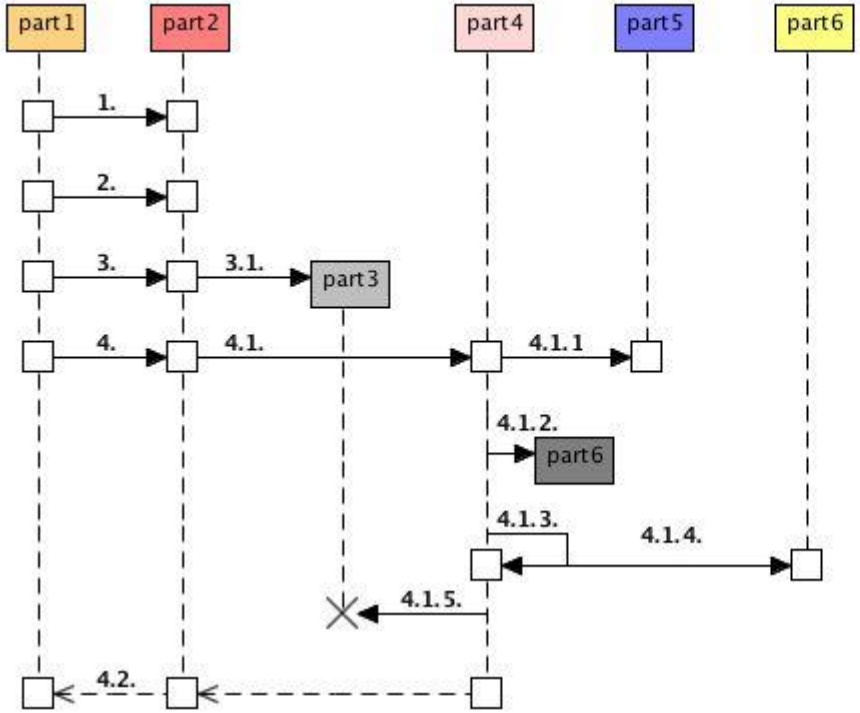
Communication Diagram
Conditional Execution

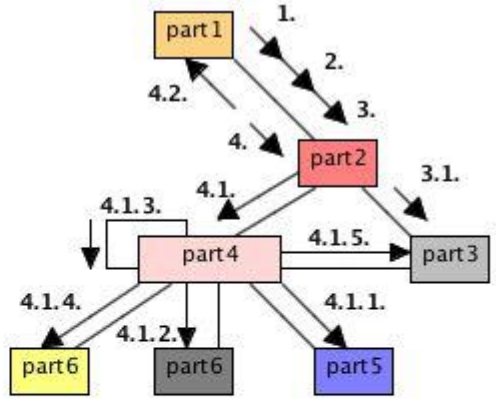



Sequence Diagram
Random Stuff
part1
«create»
part2
part1
«destroy»
ref
VerifyFunds
sd Verify Funds
I've Seen the Same for Critical & Neg
Defines Lost Message
Defines Found Message
Communication Diagram
Random Stuff
part1
4.1 «create»
part2
part1
«destroy»
part2
ref
VerifyFunds
sd Verify Funds
Same Notation for Lost & Found Messages

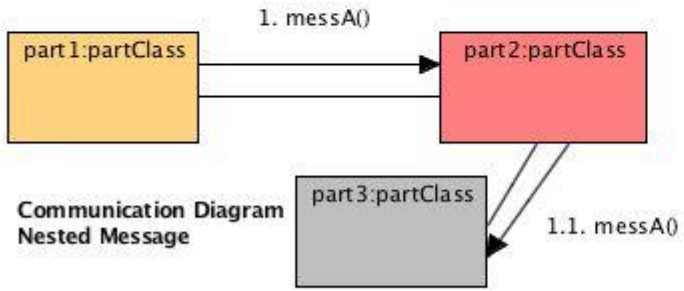

Sequence Diagram
theCustomer:Customer
theCardReader:CardReader
custAcctDB:CustAcctDB
messName(arg1:class):returnType
1.
messName2(arg1:class):returnType
2.
messName3(arg1:class):returnType
«create»
custAcct:CustRed
3.
3.1.
messName4(arg1:class):returnType
4.
4.1.
4.1.1.
«destroy»
messName5(arg1:class):returnType

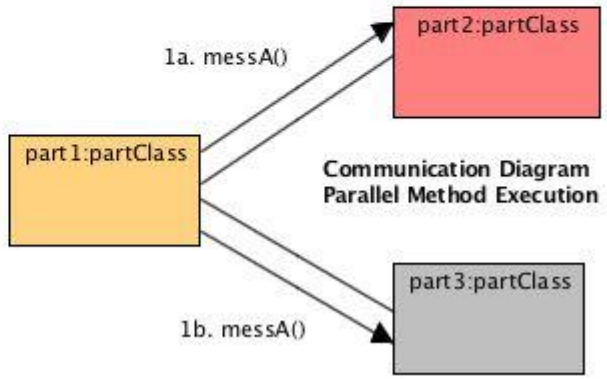

Communication Diagram
theCustomer:Customer
1. messName(arg1:class):returnType
2. messName2(arg1:class):returnTyp
3. messName3(arg1:class):returnTyp
4.2. messName5(arg1:class):returnType
theCardReader:CardReader
4. messName4(arg1:class):returnType
4.1. messName6(arg1:class):returnType
3.1. «create»
custAcctDB:CustAcctDB
4.1.1. «destroy»
custAcct:CustRed





I. 1st Message after a new Participant add a Number
II. Every message after increments the number





- Состояния
- Переходы

ДИАГРАММА АВТОМАТОВ (STATECHART DIAGRAM)

Зачем?

Диаграмма деятельности полезна для описания алгоритма действий, но она не дает представления о поведении определенного объекта в рамках отдельного варианта использования или системы в целом, что необходимо при объектно-ориентированном программировании.

На сегодняшний день при проектировании сложной Системы принято делить ее на части, каждую из которых затем рассматривать отдельно. Таким образом, при объектной декомпозиции Система разбивается на объекты или компоненты, которые взаимодействуют друг с другом, обмениваясь сообщениями.

Сообщения описывают или представляют собой некоторые события. Получение объектом сообщения активизирует его и побуждает выполнять предписанные его программным кодом действия.

При данном подходе Система становится событийно управляемой, поэтому разработчикам зачастую важно знать, как должен реагировать тот или иной объект на определенные события. Инициаторами событий могут быть как объекты самой Системы, так и её внешнее окружение.

Описать поведение отдельно взятого объекта помогает диаграмма состояний.

Назначение

Диаграмма состояний (statechart diagram) - диаграмма, которая представляет конечный автомат.

Главное назначение *диаграммы состояний* - описать возможные последовательности *состояний* и *переходов*, которые в совокупности характеризуют поведение моделируемой системы в течение всего ее жизненного цикла.

Диаграмма состояний представляет динамическое поведение сущностей, на основе спецификации их реакции на восприятие некоторых конкретных *событий*.

Применение

Диаграммы состояний чаще всего используются для описания поведения отдельных систем и подсистем. Они также могут быть применены для спецификации функциональности экземпляров отдельных классов, т. е. для моделирования всех возможных изменений *состояний* конкретных объектов. *Диаграмма состояний* по существу является графом специального вида, который служит для представления *конечного автомата*.

В контексте языка UML понятие *конечного автомата* обладает дополнительной семантикой. Вершинами графа *конечного автомата* являются *состояния* и другие типы элементов модели, которые изображаются соответствующими графическими символами. Дуги графа служат для обозначения *переходов из состояния* в *состояние*.

Конечный автомат описывает поведение отдельного объекта в форме последовательности *состояний*, охватывающих все этапы его жизненного цикла, начиная от создания объекта и заканчивая его уничтожением. Каждая *диаграмма состояний* представляет собой *конечный автомат.*

ДИАГРАММЫ СОСТОЯНИЙ

- **Состояния (state)**
  - Простые (simple)
  - Составные (composite)
  - Специальные (pseudo)
- **Переходы (transition)**
  - Начало (source)
  - Событие (event)
  - Условие (guard condition)
  - Действие (action)
  - Конец (target)

Графическое изображение состояний на диаграмме состояний

Список внутренних действий

Эта секция содержит перечень внутренних действий или деятельностей, которые выполняются в процессе нахождения моделируемого элемента в данном состоянии.

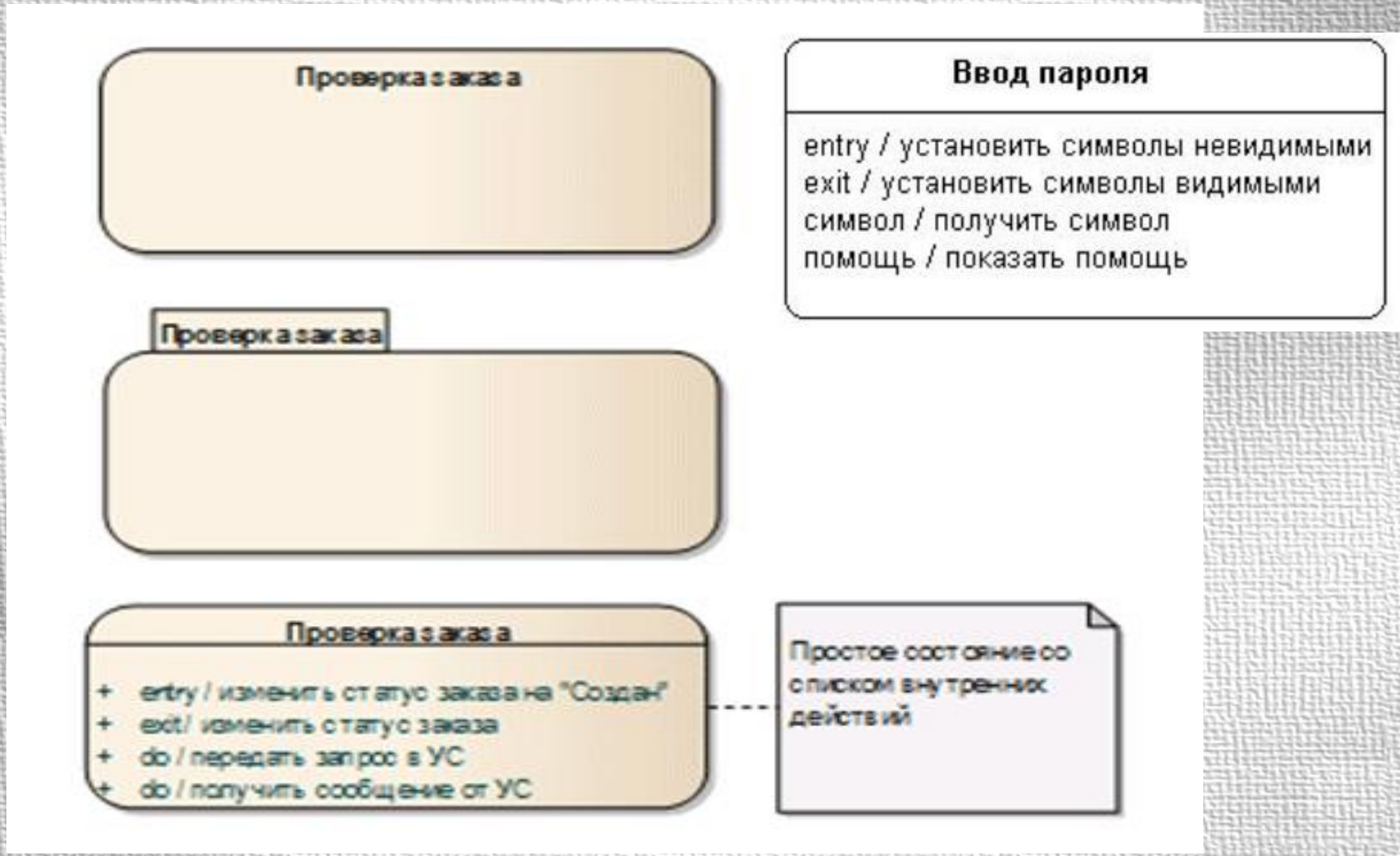
Каждое из действий записывается в виде отдельной строки и имеет следующий **формат:**

<метка-действия '/' выражение-действия>

Метка действия указывает на обстоятельства или условия, при которых будет выполняться деятельность, определенная выражением действия. При этом выражение действия может использовать любые атрибуты и связи, которые принадлежат области имен или контексту моделируемого объекта. Если список выражений действия пустой, то разделитель в виде наклонной черты '/' может не указываться.

Перечень меток действия имеет фиксированные значения в языке UML, которые не могут быть использованы в качестве имен событий. Эти значения следующие:

- **entry** – эта метка указывает на действие, специфицированное следующим за ней выражением действия, которое выполняется в момент входа в данное состояние (входное действие);
- **exit** – эта метка указывает на действие, специфицированное следующим за ней выражением действия, которое выполняется в момент выхода из данного состояния (выходное действие);
- **do** – эта метка специфицирует выполняющуюся деятельность, которая выполняется в течение всего времени, пока объект находится в данном состоянии, или до тех пор, пока не закончится вычисление, специфицированное следующим за ней выражением действия.

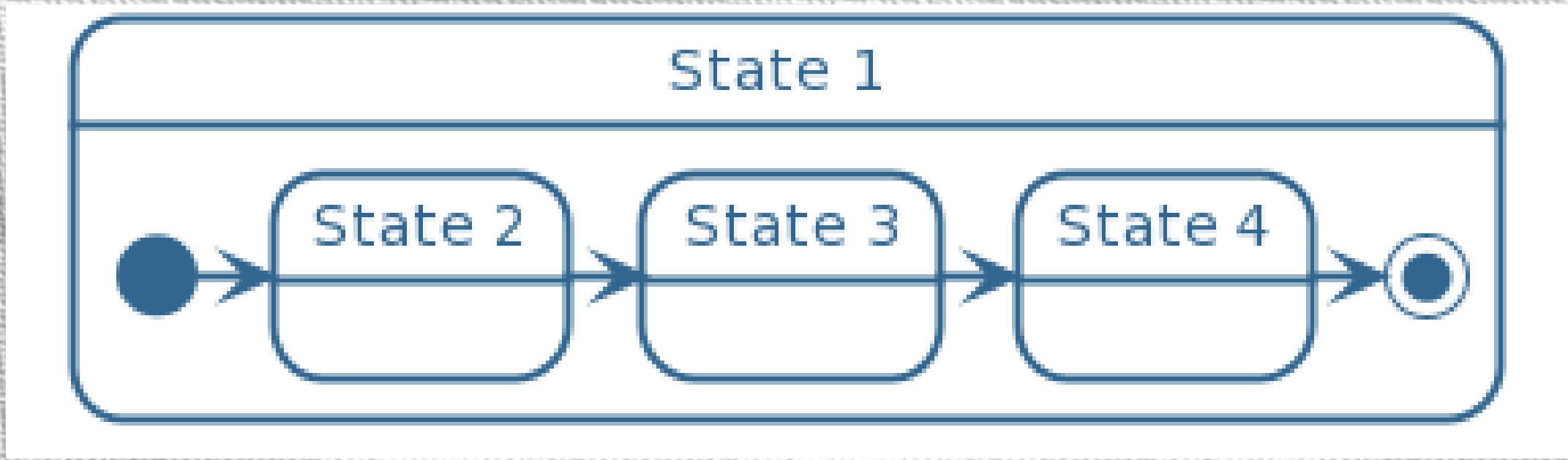

Проверка заказа
Ввод пароля
entry / установить символы невидимыми
exit / установить символы видимыми
символ / получить символ
помощь / показать помощь
Проверка заказа
Проверка заказа
+ entry / изменить статус заказа на "Создан"
+ exit / изменить статус заказа
+ do / передать запрос в УС
+ do / получить сообщение от УС
Простое состояние со списком внутренних действий

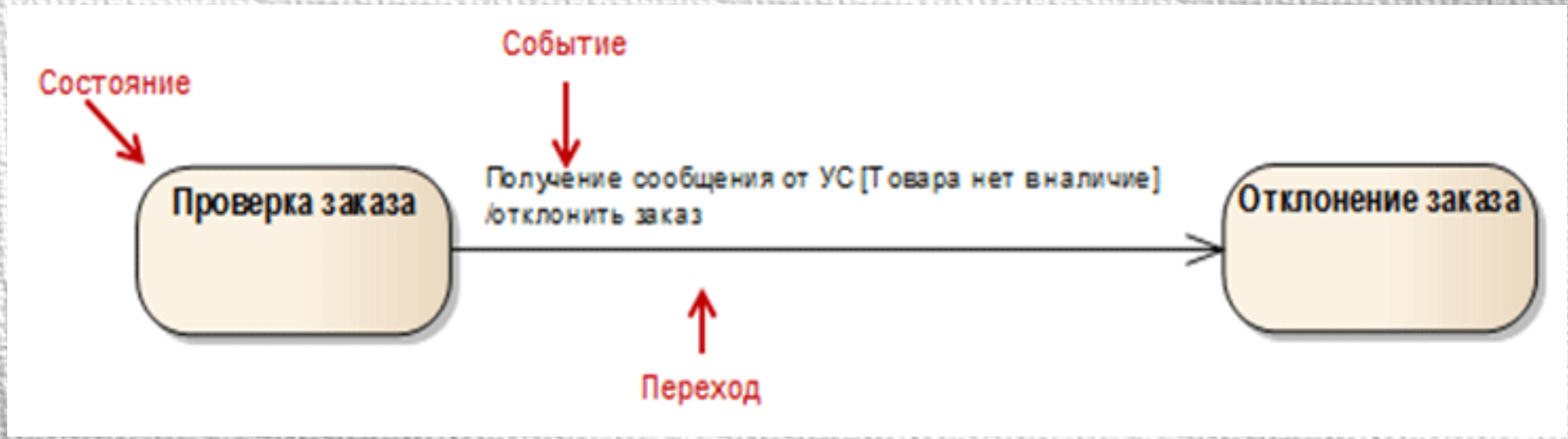
Составное состояние (Composite State)

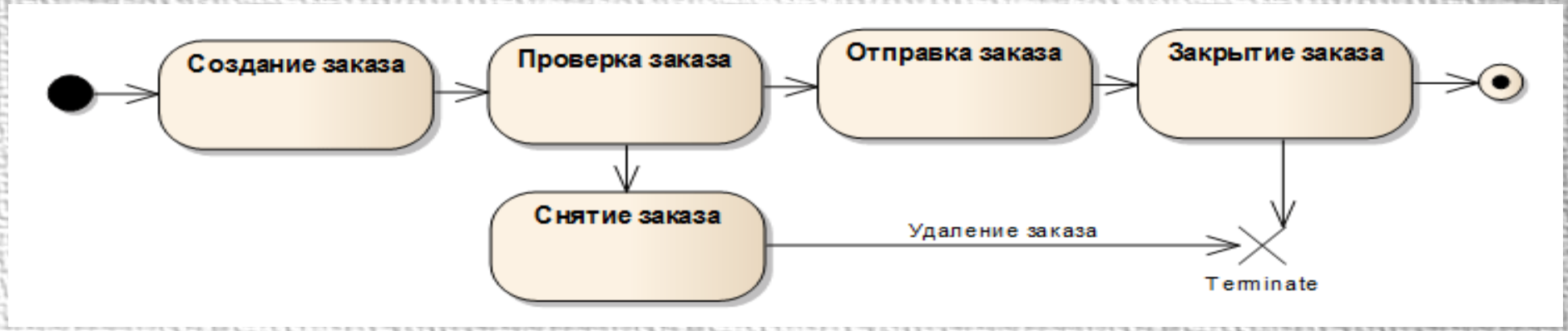
Составное состояние в диаграмме состояний UML представляет собой состояние, которое содержит внутренние состояния и переходы между ними. Оно позволяет структурировать более сложные состояния и их поведение внутри объекта или системы.

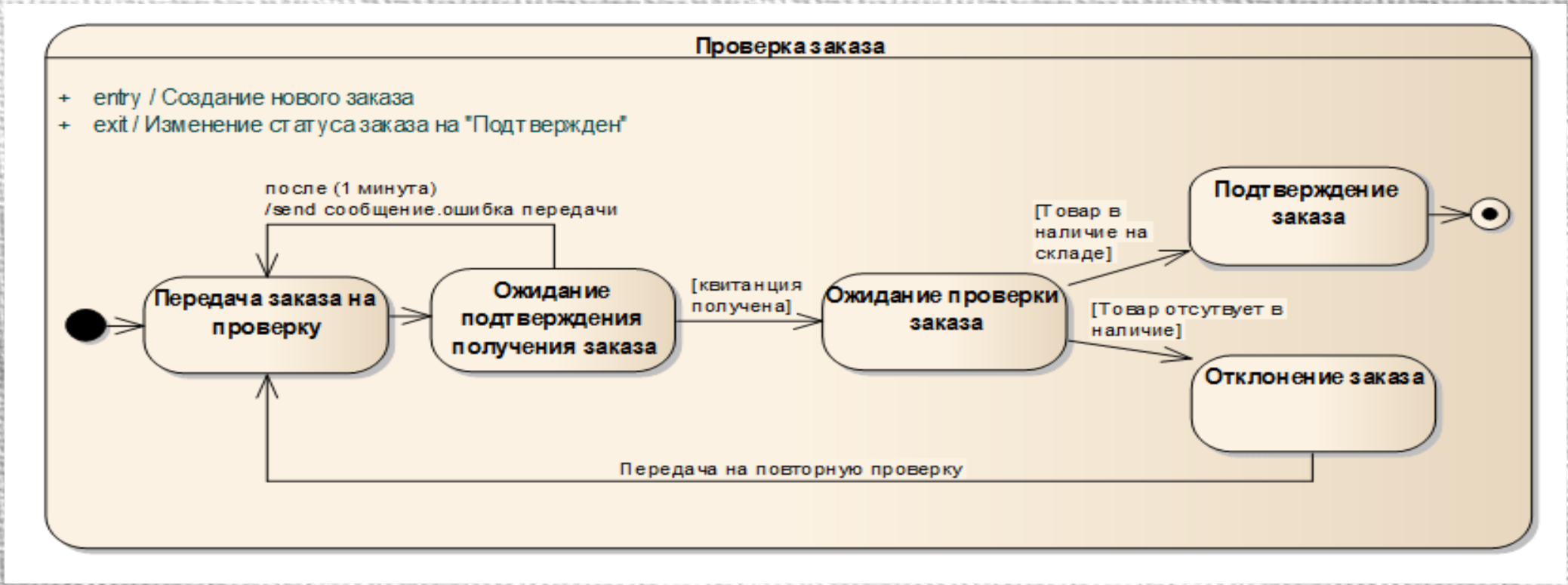
Составное состояние обычно представляется в виде прямоугольника, внутри которого содержатся другие состояния и переходы. Внутренние состояния и переходы могут быть представлены как отдельные элементы внутри составного состояния или в виде вложенных диаграмм состояний.

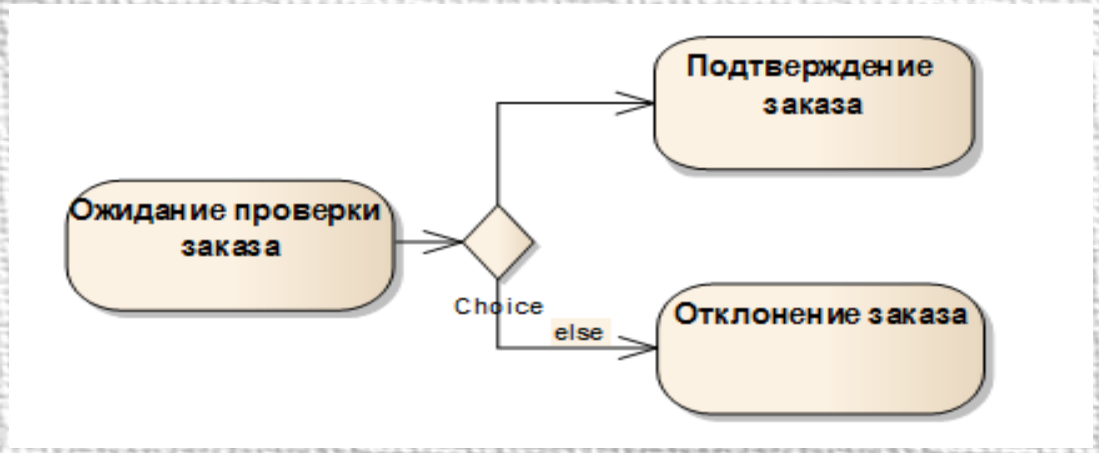
В составном состоянии можно определить иерархию вложенных состояний, которая отображает различные уровни детализации поведения объекта или системы. Когда объект находится в составном состоянии, его поведение может зависеть от текущего внутреннего состояния и переходить между ними в ответ на определенные события или условия.



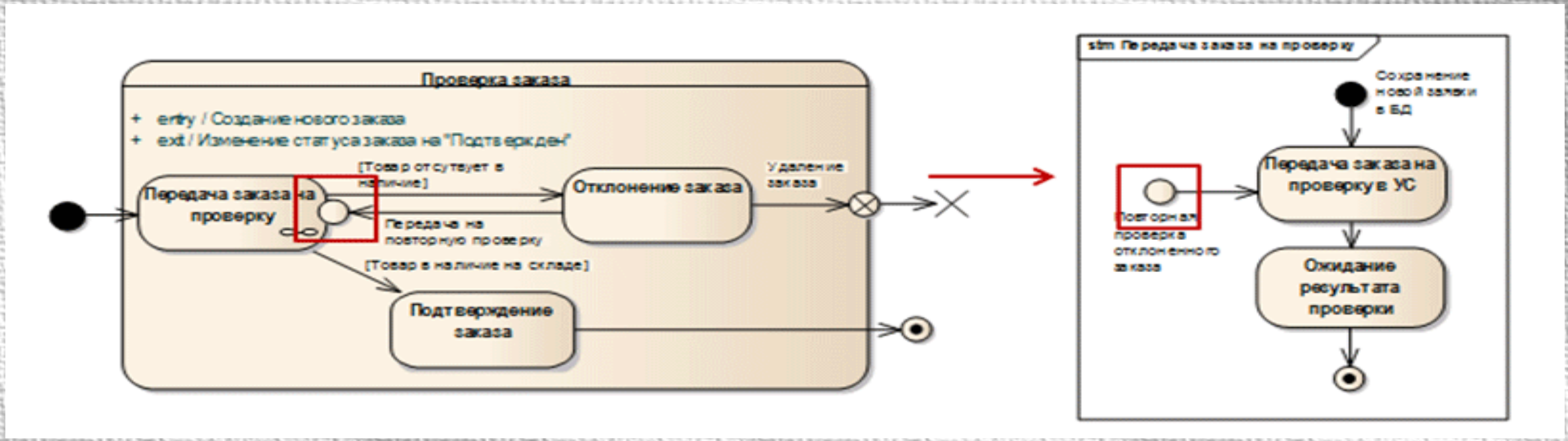

Событие
Состояние
Проверка заказа
Получение сообщения от УС [Товара нет в наличие]
/отклонить заказ
Отклонение заказа
Переход


Создание заказа
Проверка заказа
Отправка заказа
Закрытие заказа
Снятие заказа
Удаление заказа
Terminate


Проверка заказа
+ entry / Создание нового заказа
+ exit / Изменение статуса заказа на "Подтвержден"
после (1 минута)
/send сообщение.ошибка передачи
Передача заказа на проверку
Ожидание подтверждения получения заказа
[квитанция получена]
Ожидание проверки заказа
[Товар в наличие на складе]
Подтверждение заказа
[Товар отсутвует в наличие]
Отклонение заказа
Передача на повторную проверку


Подтверждение заказа
Ожидание проверки заказа
Choice
else
Отклонение заказа

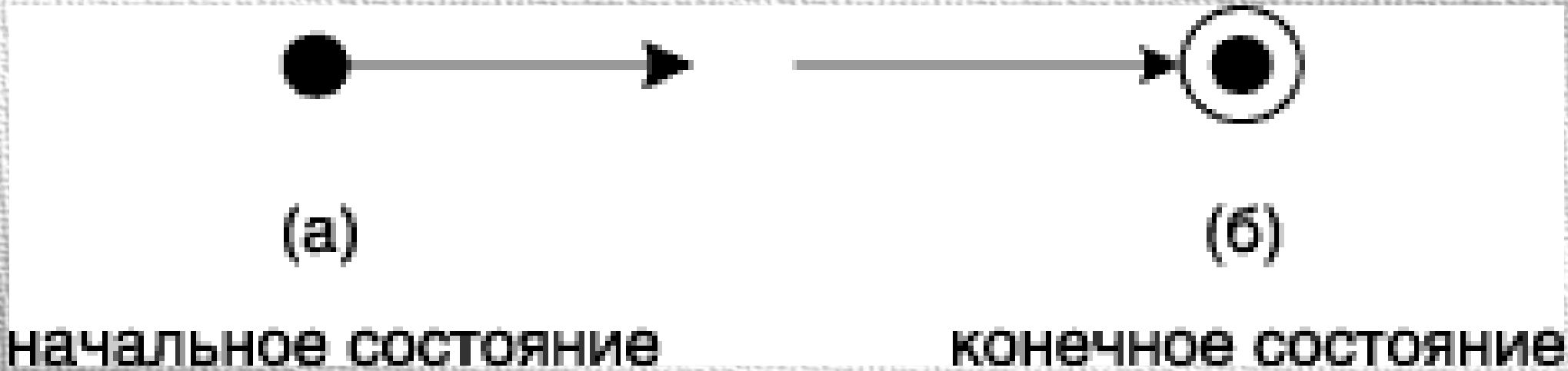
Подготовить заказ
Оплатить заказ
Подтвердить заказ
Отгрузить заказ
Доставить заказ

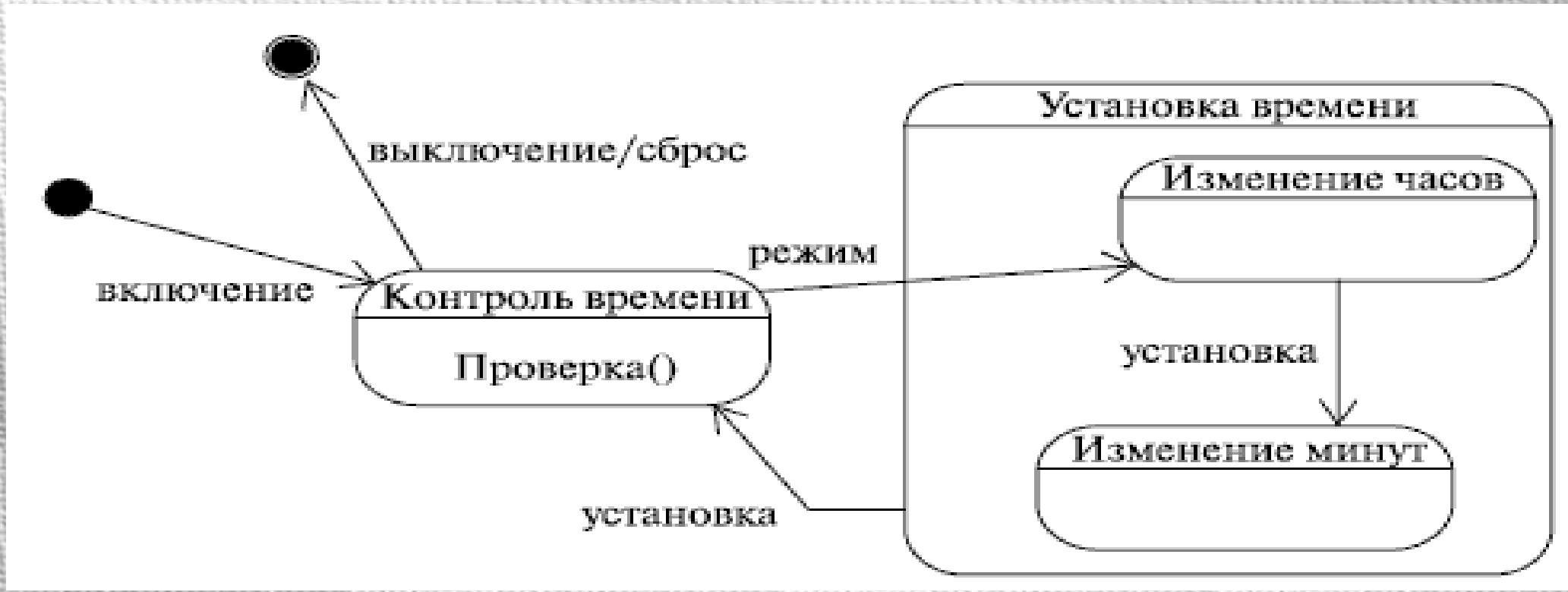

Проверка заказа
+ entry / Создание нового заказа
+ exit / Изменение статуса заказа на "Подтвержден"
Передача заказа на проверку
[Товар отсутствует в наличие]
Передача на повторную проверку
Отклонение заказа
Удаление заказа
[Товар в наличие на складе]
Подтверждение заказа
stm Передача заказа на проверку
Сохранение новой заявки в БД
Передача заказа на проверку в УС
Повторная проверка отклоненного заказа
Ожидание результата проверки

ИСТОРИЧЕСКОЕ СОСТОЯНИЕ

Историческое состояние используется для запоминания того из последовательных подсостояний, которое было текущим в момент выхода из составного состояния. Существует две разновидности исторического состояния: *недавнее* и *давнее*.

Начальное и конечное состояния




выключение/сброс
включение
Контроль времени
Проверка()
режим
Установка времени
Изменение часов
установка
Изменение минут
установка

Ожидание
сработал датчик
Тревога
do / издавать звук сирены
(а)
Обращение к БД
do / выполнение SQL запроса
Анализ отчета
(б)

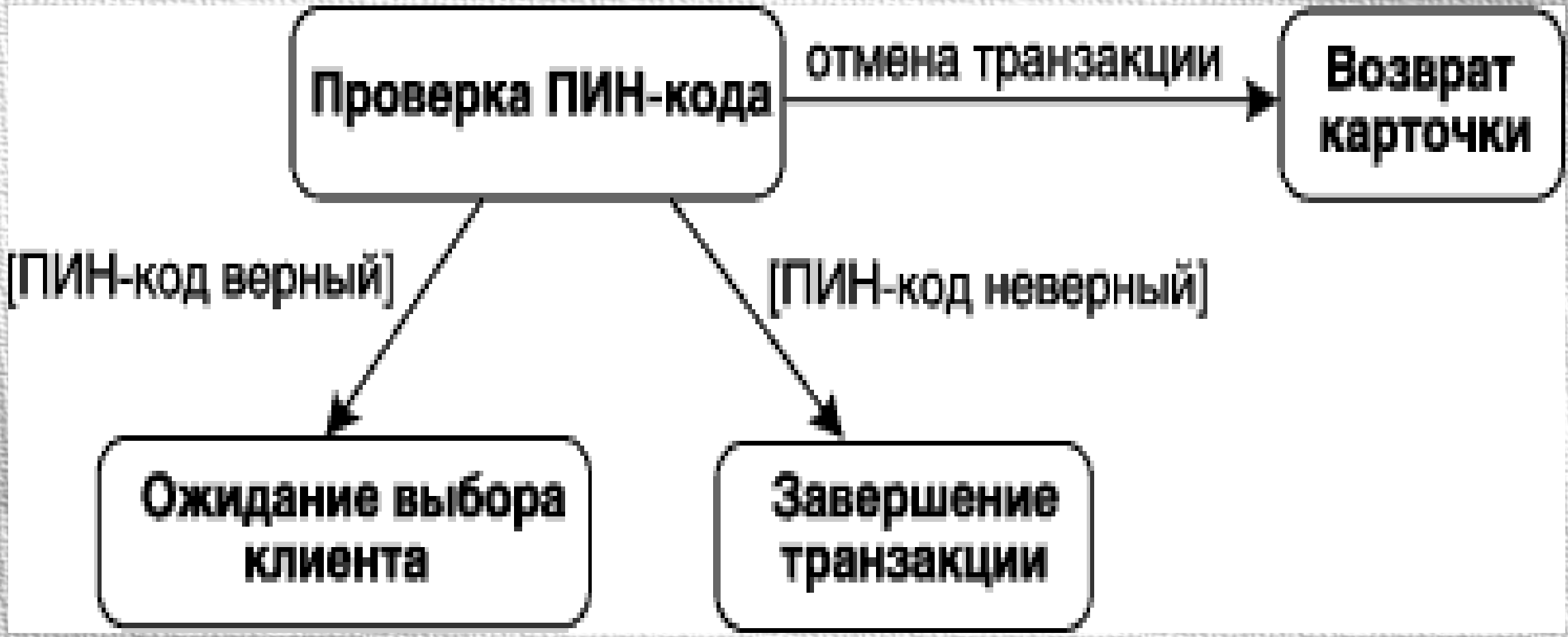
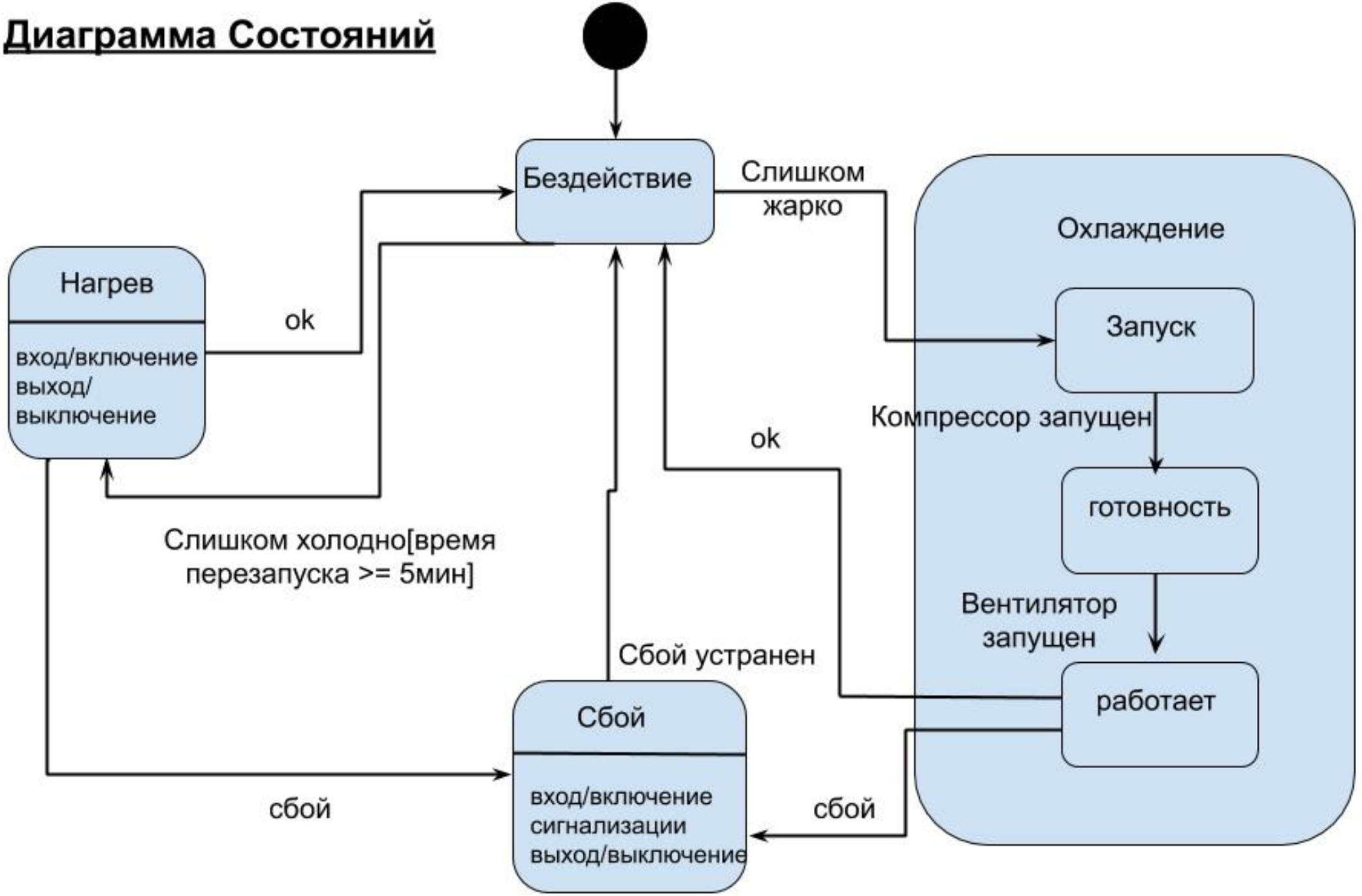

Проверка ПИН-кода
отмена транзакции
Возврат карточки
[ПИН-код верный]
[ПИН-код неверный]
Ожидание выбора клиента
Завершение транзакции

Диаграмма Состояний



ОСНОВЫ СТРУКТУРНОГО МОДЕЛИРОВАНИЯ

- Диаграмма классов
- Интерфейсы
- Пакеты

МОДЕЛИРОВАНИЕ СТРУКТУРЫ

Ниже мы рассматриваем способы ответа на вопрос

Из чего состоит система?

В простых случаях одного короткого ответа на этот вопрос бывает достаточно для достижения целей моделирования. Например: молоток состоит из ударной части и рукоятки. В более сложных случаях нужны иерархические уточнения, например: ударная часть столярного молотка имеет с одной стороны боёк, а с другой гвоздодёр и изготовляется из стали.

В любом случае в центре внимания находятся отношения "часть–целое" и статические свойства частей и целого. При моделировании структуры мы не рассматриваем такие отношения, как "причина–следствие" или "раньше–позже". Поэтому план состоит в том, чтобы сначала обсудить общие принципы моделирования структуры, а затем разобрать разнообразные конкретные средства моделирования структуры, предусмотренные UML.

Моделируя структуру, мы описываем составные части системы и отношения между ними. UML в большинстве случаев применяется в качестве объектно-ориентированного языка моделирования, поэтому не удивительно, что основным видом составных частей, из которых состоит система при таком подходе, являются классы и отношения между ними. В каждый конкретный момент функционирования системы можно указать конечный набор конкретных объектов (экземпляров классов) и существующих между ними связей (экземпляров отношений). Однако в процессе работы этот набор не остается неизменным: объекты создаются и уничтожаются, связи устанавливаются и теряются. Число возможных вариантов наборов объектов и связей, которые могут иметь место в процессе функционирования системы, если и не бесконечно, то может быть необозримо велико.

Представить их все в модели практически невозможно, а главное бессмысленно, поскольку такая модель из-за своего объема будет недоступна для понимания человеком, а значит и бесполезна при разработке системы. Каким же образом можно строить компактные (полезные) модели необозримых (потенциально бесконечных) систем?

Назначение структурного моделирования

Рассмотрим более детально, какие именно структуры нужно моделировать и зачем. Мы выделяем следующие структуры:

1. **структура связей между объектами во время выполнения программы;**
2. **структура хранения данных;**
3. **структура программного кода;**
4. **структура компонентов в приложении;**
5. **структура сложных объектов, состоящих из взаимодействующих частей;**
6. **структура артефактов в проекте;**
7. **структура используемых вычислительных ресурсов.**

СТРУКТУРА СВЯЗЕЙ МЕЖДУ ОБЪЕКТАМИ ВО ВРЕМЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ

В парадигме объектно-ориентированного программирования процесс выполнения программы состоит в том, что программные объекты взаимодействуют друг с другом, обмениваясь сообщениями. Наиболее распространенным типом сообщения является вызов метода объекта одного класса из метода объекта другого класса. Для того чтобы вызвать метод объекта, нужно иметь доступ к этому объекту. На уровне программной реализации этот доступ может быть обеспечен самыми разнообразными механизмами.

- Например, объект, вызывающий метод, может хранить указатель (ссылку) на объект, содержащий вызываемый метод.
- Еще вариант: ссылка на объект с вызываемым методом может быть передана в качестве аргумента объекту, который этот метод вызовет.
- Возможно, используется какой-либо механизм удаленного вызова процедур, обеспечивающий доступ к объектам (например, такой, как CORBA) по их идентификаторам.

- Если атрибуты объектов представлены записями в таблице базы данных, а методы (нередкий вариант реализации) – хранимыми процедурами системы управления базами данных (СУБД), то идентификация объектов осуществляется по первичному ключу таблицы.

Как бы то ни было, во всех случаях имеет место следующая ситуация: один объект "знает" другие объекты и, значит, может вызвать открытые методы, использовать и изменять значения открытых атрибутов и т.д. В этом случае, мы говорим, что объекты связаны, т.е. между ними есть связь. **Для моделирования структуры связей в UML используются отношения ассоциации на диаграмме классов.**

СТРУКТУРА ХРАНЕНИЯ ДАННЫХ

Программы обрабатывают данные, которые хранятся в памяти компьютера. В парадигме объектно-ориентированного программирования для хранения данных во время выполнения программы предназначены атрибуты классов. Однако большая часть приложений для автоматизации делопроизводства устроена так, что определенные данные (не все) должны храниться в памяти компьютера не только во время сеанса работы приложения, но постоянно, т.е. между сеансами. Объекты, которые сохраняют (по меньшей мере) значения своих атрибутов даже после того, как завершился породивший их поток управления, мы будем называть хранимыми.

В UML для моделирования данного атрибута объектов и их составляющих применяется стандартное свойство {*persistence*}, которое может быть назначено классификатору, ассоциации или атрибуту и может принимать одно из двух значений:

- **persistent** – экземпляры классификатора, ассоциации или значения атрибута, соответственно, должны быть хранимыми;
- **transient** – противоположно предыдущему — сохранять экземпляры не требуется (значение по умолчанию).

В настоящее время самым распространенным способом хранения объектов является использование СУБД. При этом хранимому классу соответствует таблица базы данных, а хранимый объект (точнее говоря, набор значений хранимых атрибутов) представляется записью в таблице. Вопрос структуры хранения данных является первостепенным для приложений баз данных. К счастью, известны надежные методы решения этого вопроса – схемы баз данных, диаграммы "сущность–связь". **Эти же методы (с точностью до обозначений) применяются и в UML в форме ассоциаций с указанием кратности полюсов.**

СТРУКТУРА ПРОГРАММНОГО КОДА

Не секрет, что программы существенно отличаются по величине – бывают программы большие и маленькие. Удивительным является то, насколько велики эти различия: от сотен строк кода (и менее) до сотен миллионов строк (и более). Столь большие количественные различия не могут не проявляться и на качественном уровне. Действительно, для маленьких программ структура кода практически не имеет значения, для больших – наоборот, имеет едва ли не решающее значение. Поскольку UML не является языком программирования, модель не определяет структуру кода непосредственно, однако косвенным образом структура модели существенно влияет на структуру кода. Большинство инструментов поддерживает полуавтоматическую генерацию кода для одного или нескольких, чаще объектно-ориентированных, языков программирования. В большинстве случаев классы модели транслируются в классы (или эквивалентные им конструкции) целевого языка. Кроме того, многие инструменты учитывают структуру пакетов в модели и транслируют ее в соответствующие "надклассовые" структуры целевой системы программирования. Таким образом, **если задействовано средство автоматической генерации кода, то структура классов и пакетов в модели фактически полностью моделирует структуру кода приложения**.

СТРУКТУРА КОМПОНЕНТОВ В ПРИЛОЖЕНИИ

Приложение, состоящее из одной компоненты, имеет тривиальную структуру компонентов, моделировать которую нет нужды. Но большинство современных приложений на этапе проектирования представляют собой взаимосвязь многих компонентов, даже если и не являются распределенными. Компонентная структура предполагает описание двух аспектов:

- во-первых, как классы распределены по компонентам,
- во-вторых, как (через какие интерфейсы) компоненты взаимодействуют друг с другом.

Оба эти аспекта моделируются диаграммами компонентов UML.

СТРУКТУРА СЛОЖНЫХ ОБЪЕКТОВ, СОСТОЯЩИХ ИЗ ВЗАИМОДЕЙСТВУЮЩИХ ЧАСТЕЙ

Для моделирования этой структуры применяется новое средство UML 2 — диаграмма внутренней структуры классификатора (*см. ниже Специальные диаграммы*). Данная диаграмма используется для описания внутренней структуры классов и компонентов. Существует еще одна сущность, которая также позволяет описать взаимодействие множества частей. Это сущность называется кооперацией и служит для описания взаимодействия в некотором контексте. С точки зрения внутренней структуры основное отличие кооперации от класса и компонента состоит в том, что кооперация не является владельцем своих частей, и соединители частей кооперации могут не иметь явного выражения в виде ассоциации. Однако, как у классов и компонентов, у кооперации могут быть экземпляры, которые функционируют во время исполнения.

СТРУКТУРА АРТЕФАКТОВ В ПРОЕКТЕ

Только самые простые приложения состоят из одного артефакта – исполнимого кода программы. Большинство реальных приложений насчитывает в своем составе десятки, сотни и тысячи различных компонентов: исполнимых двоичных файлов, файлов ресурсов, файлов исходного кода, различных сопровождающих документов, справочных файлов, файлов с данными и т.д. Для большого приложения важно не только иметь точный и полный список всех артефактов, но и указать, какие именно из них входят в конкретный экземпляр системы. Дело в том, что для больших приложений в проекте сосуществуют разные версии одного и того же артефакта. Это исчерпывающим образом моделируется диаграммами компонентов и размещения UML, где предусмотрены стандартные стереотипы для описания артефактов разных типов.

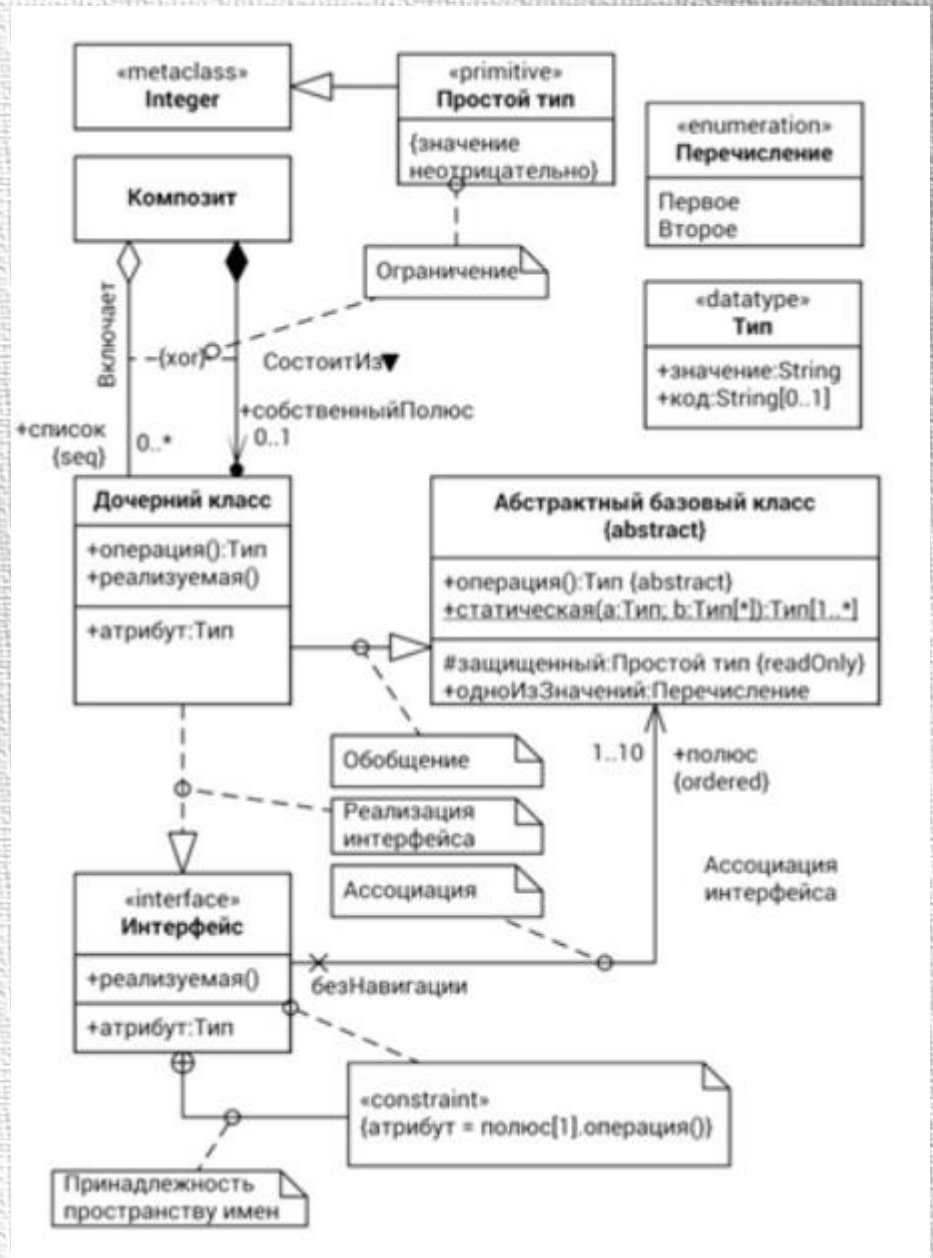
СТРУКТУРА ИСПОЛЬЗУЕМЫХ ВЫЧИСЛИТЕЛЬНЫХ РЕСУРСОВ

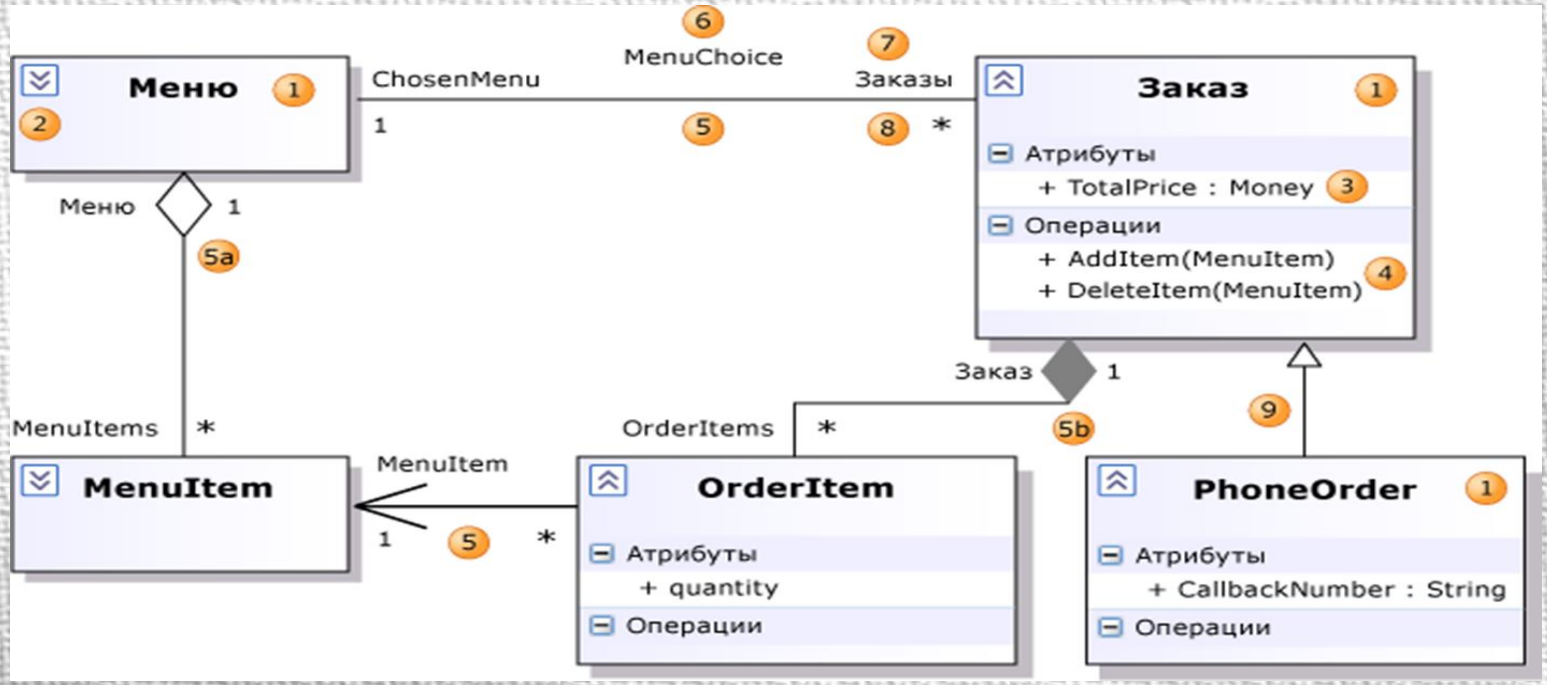
Приложение, состоящие из многих артефактов, как правило, бывает распределенным, т.е. различные артефакты размещаются на разных компьютерах. **Диаграммы размещения позволяют включить в модель описание и этой структуры**.

- Атрибуты
- Операции
- Виды отношений
- Виды классов

ДИАГРАММА КЛАССОВ (CLASS DIAGRAM)

Основная нотация диаграмм классов





СУЩНОСТИ НА ДИАГРАММЕ КЛАССОВ

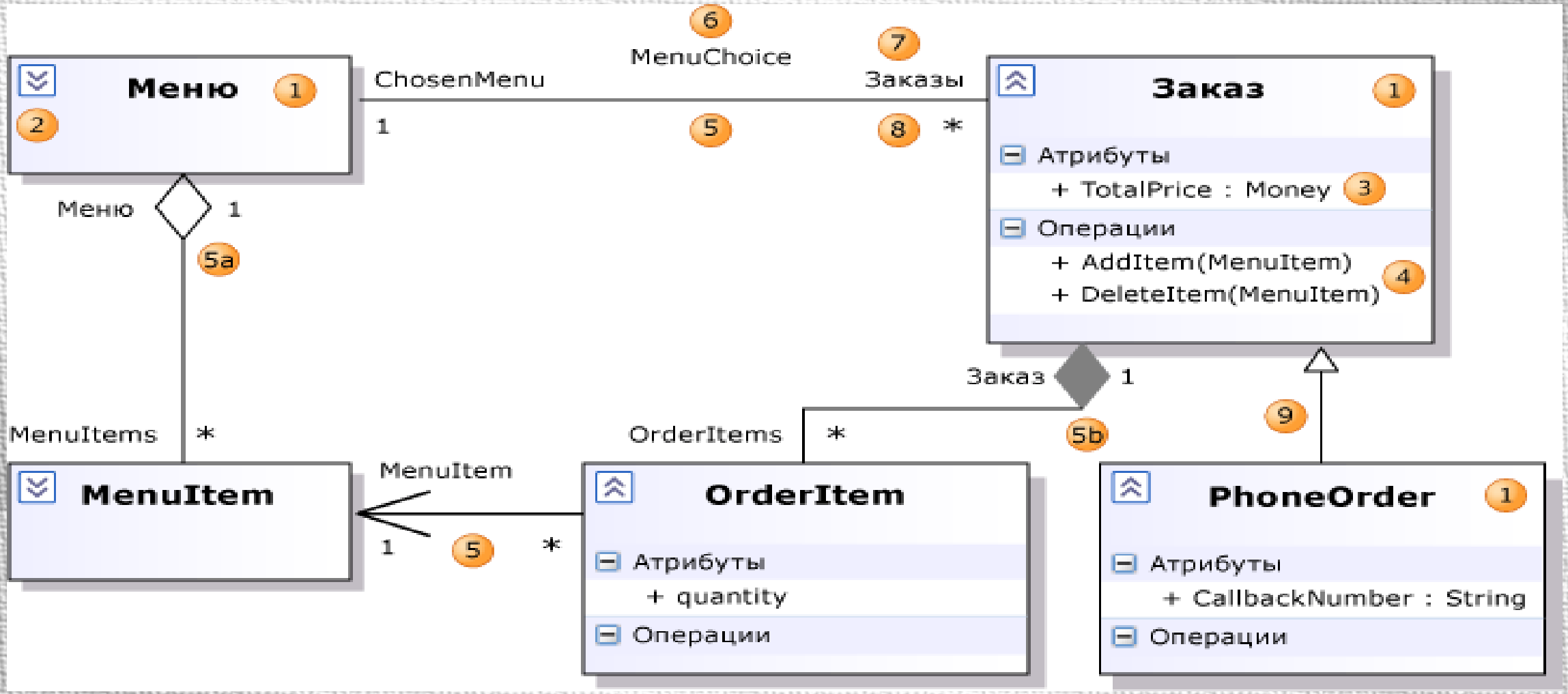
Назначение диаграммы классов

Диаграмма *классов* предназначена для представления статической структуры модели системы в терминологии *классов* объектно-ориентированного программирования. При этом диаграмма *классов* может содержать интерфейсы, пакеты, отношения и даже отдельные экземпляры классификаторов, такие как объекты и связи.

Диаграмма классов является основным средством моделирования структуры в UML, а класс, соответственно, основной структурной единицей. Это совсем не удивительно и вполне естественно, поскольку UML является в большой степени объектно-ориентированным языком. Диаграммы классов наиболее информационно насыщены по сравнению с другими типами канонических диаграмм UML, инструменты генерируют код в основном по описанию классов, структура классов точнее всего соответствует окончательной структуре кода приложения.

На диаграммах классов в качестве сущностей применяются, прежде всего, классы, как в своей наиболее общей форме, так и в форме многочисленных стереотипов и частных случаев: интерфейсы, типы данных, активные классы и др. Кроме того, на диаграмме классов могут использоваться (как и везде) пакеты и комментарии.

Диаграмма классов



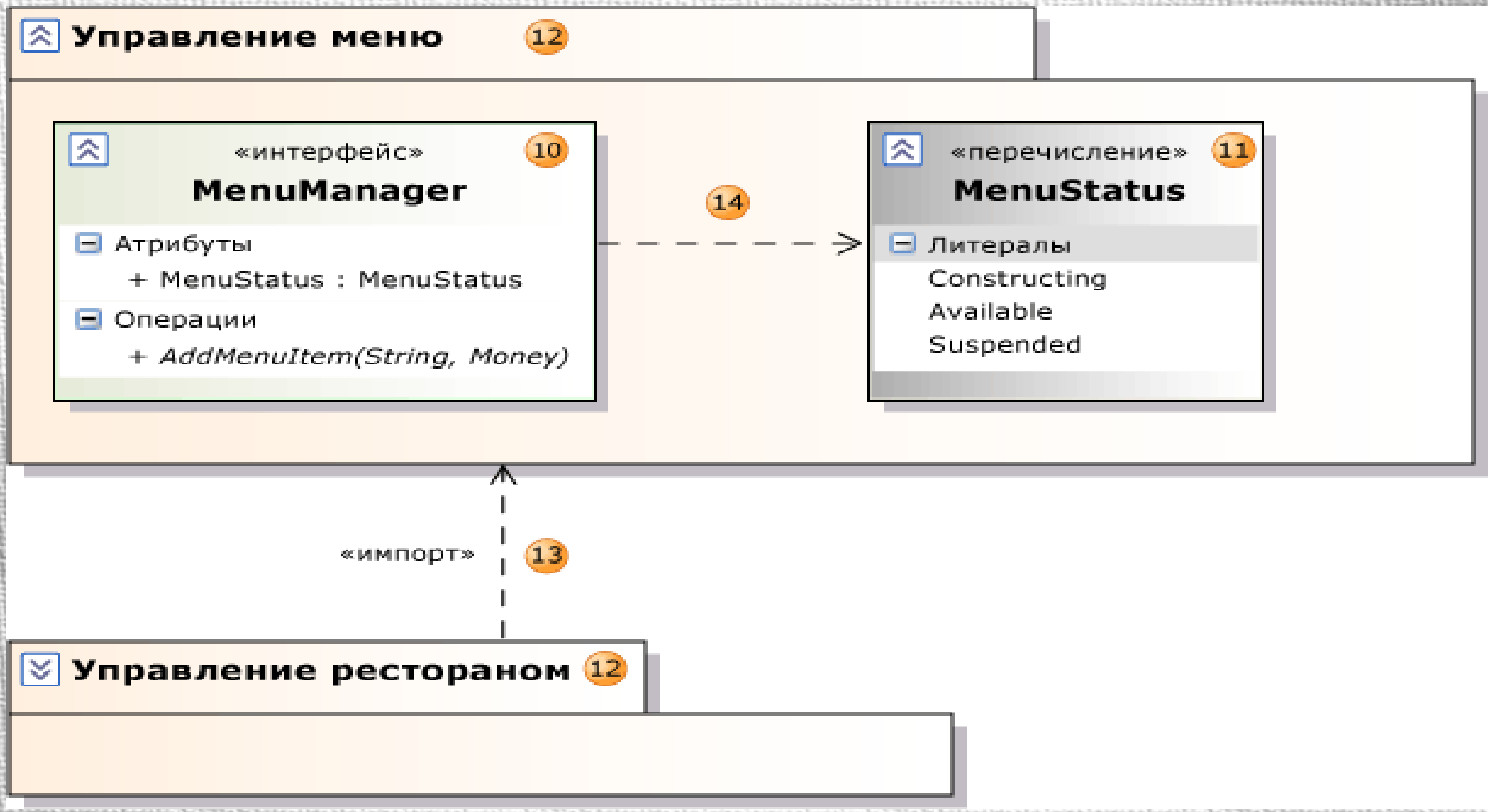
Элементы ДК

Фигура	Элемент	Описание
1	**Класс**	Определение объектов, совместно обладающих данными характеристиками структуры и поведения.
1	**Классификатор**	Общее имя для класса, интерфейса или перечисления. Компоненты, варианты использования и субъекты также являются классификаторами.
2	**Элемент управления "свернуть/развернуть"**	Если подробностей классификатора не видно, щелкните расширитель в верхней левой части классификатора. Иногда также нужно щелкнуть [+] для каждого сегмента.
3	**Атрибут**	Типизированное значение, прикрепленное к каждому экземпляру классификатора. Чтобы добавить атрибут, щелкните раздел **Атрибуты** и нажмите **ВВОД**. Введите сигнатуру атрибута.

4	**Операция**	Метод или функция, которую можно выполнить с помощью экземпляров классификатора. Чтобы добавить операцию, щелкните раздел **Операции** и нажмите **ВВОД**. Введите сигнатуру операции.
5	**Ассоциация**	Отношение между членами двух классификаторов.
5а	**Агрегат**	Ассоциация, представляющая отношение совместного владения. Свойству **Агрегат** роли-владельца присвоено значение **Сделано общим**.
5б	**Композиция**	Ассоциация, представляющая отношение целого и части. Свойству **Агрегат** роли-владельца присвоено значение **Составной**.
6	**Имя ассоциации**	Имя ассоциации. Имя может оставаться пустым.

7	**Имя роли**	Имя роли, т. е. одного из окончаний ассоциации. Может использоваться для ссылки на связанный объект. На предыдущей иллюстрации для любого заказа `O` связанным меню является `O.ChosenMenu`. Каждая роль имеет собственные свойства, которые перечислены в свойствах ассоциации.
8	**Кратность**	Указывает, сколько объектов на этом окончании можно связать с объектами на другом окончании. Каждый заказ в этом примере должен быть связан только с одним меню. ***** означает, что ограничения числа ссылок, которые можно создать, не существует.
9	**Обобщение**	*Конкретный* классификатор наследует часть своего определения от *общего* классификатора. Общий классификатор находится на окончании соединителя с указателем стрелки. Атрибуты, ассоциации и операции наследуются конкретным классификатором. Воспользуйтесь инструментом **Наследование**, чтобы создать обобщение между двумя классификаторами.

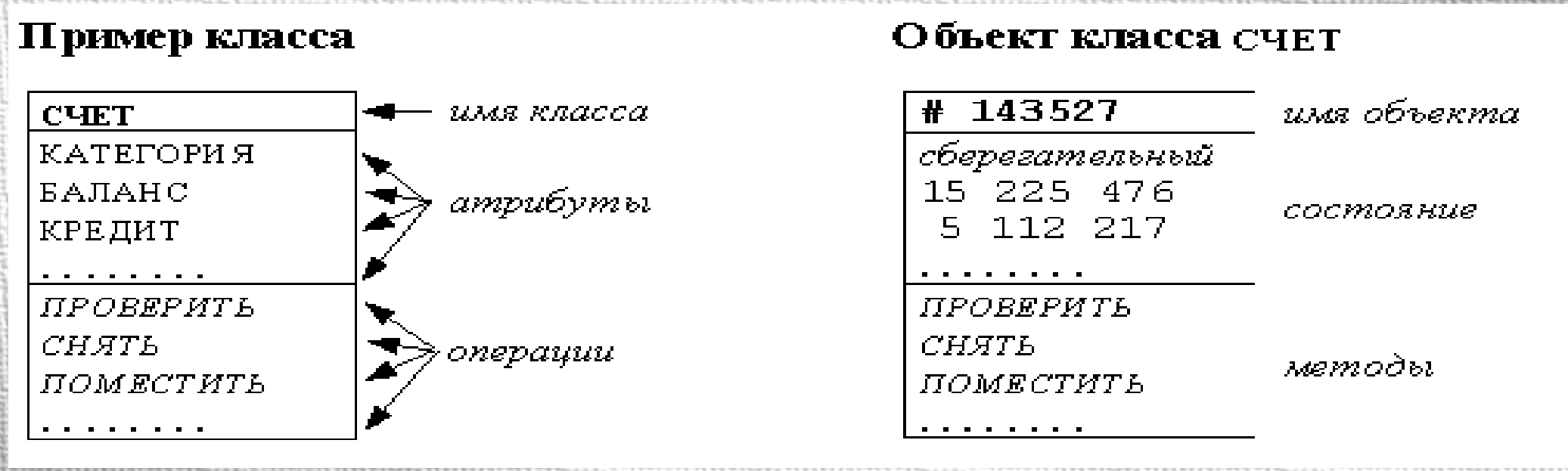
Использование пакетов



Фигура	Элемент	Описание
10	**Интерфейс**	Определение части внешне видимого поведения объекта.
11	**Перечисление**	Классификатор, состоящий из набора строковых литералов.
12	**Пакет**	Группа классификаторов, ассоциаций, действий, линий жизни, компонентов и пакетов. Логическая схема классов показывает, что членами данного пакета являются классификаторы и пакеты. Область видимости имен ограничивается пакетами, так что Класс1 в Пакет1 отличается от Класс1 вне этого пакета. Имя пакета отображается как часть свойств **Полное имя** его содержимого. Свойство **Связанный пакет** любой UML-схемы можно настроить так, чтобы оно ссылалось на пакет. В этом случае все элементы, создаваемые на этой схеме, станут частью пакета. Они отображаются в пакете в **Проводнике по моделям UML**.
13	**Импорт**	Отношение между пакетами, указывающее, что один пакет включает все определения другого.
14	**Зависимость**	Определение или реализация зависимого классификатора может измениться, если изменяется классификатор на окончании с наконечником стрелки.

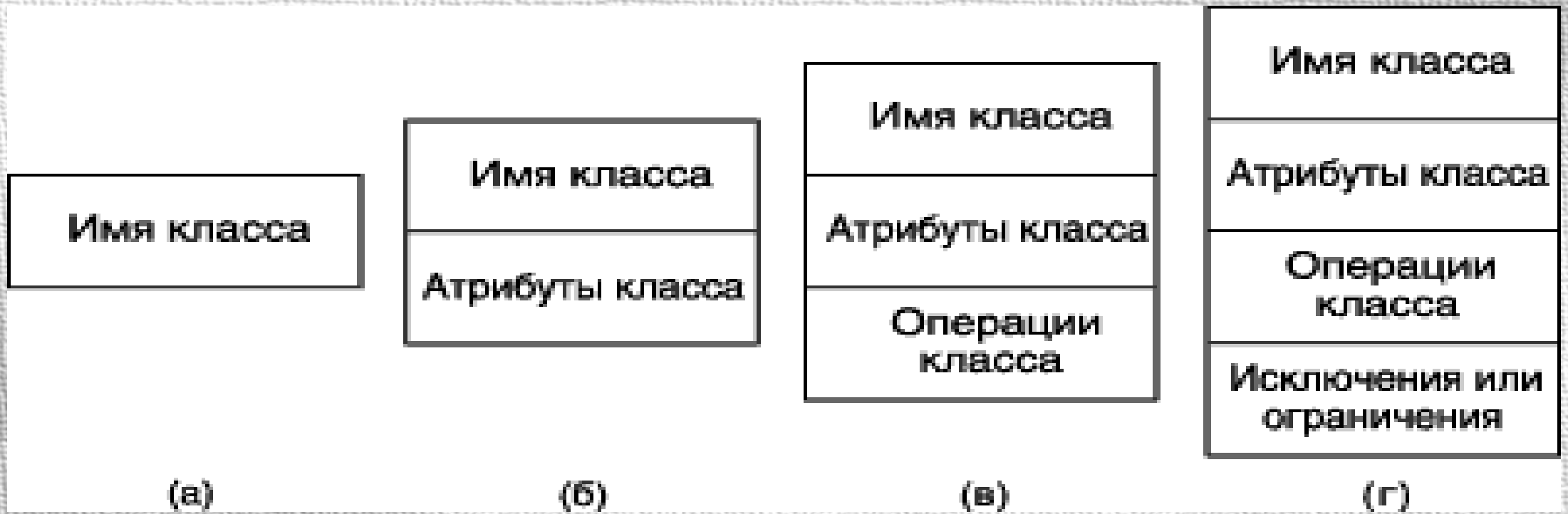
Фигура	Элемент	Описание
15	**Реализация**	Класс реализует операции и атрибуты, определенные интерфейсом. Воспользуйтесь инструментом **Наследование**, чтобы создать реализацию между классом и интерфейсом.
16	**Реализация**	Альтернативное представление того же отношения. Метка на символе обозначения указывает на интерфейс. Чтобы создать эту презентацию, выделите существующее отношение реализации. Рядом с ассоциацией появится тег действия. Щелкните тег действия и выберите **Показывать без описания операций**.

Класс и объекты



Объект является **Экземпляром класса**. Однако класс также может нуждаться в конкретизации – он может быть объектом. Поэтому необходимо различать объект-экземпляр (*instance object*) и объект-класс (*class object*).

Варианты графического изображения класса



Синтаксис

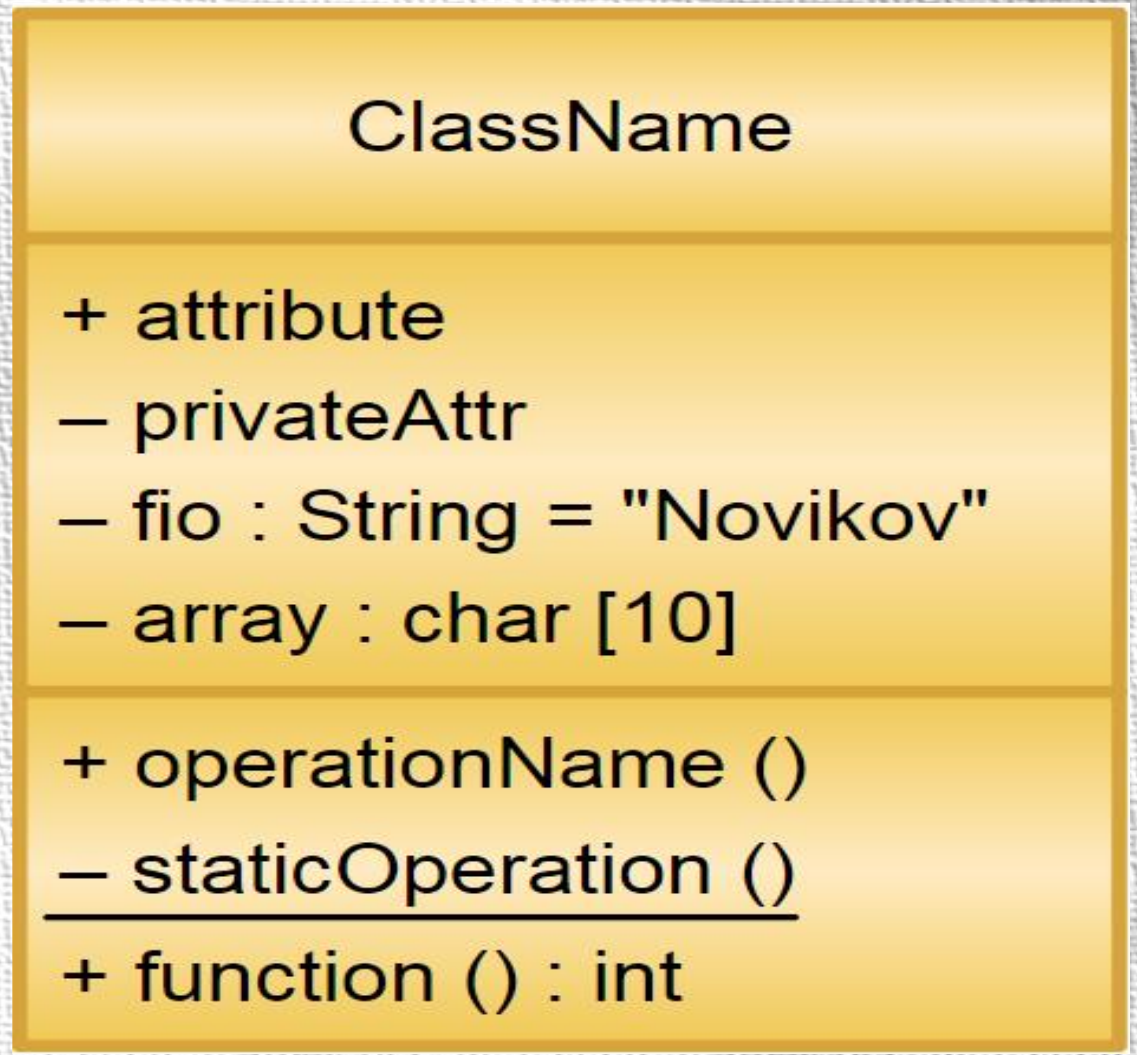
ИМЯ_КЛАССА
имя_атрибута 1: тип_1 = значение_по_умолчанию_1 *имя_атрибута 2: тип_2 = значение_по_умолчанию_2* . . . . .
имя_операции_1 (список_аргументов_1): тип_результата *сигнатура_операции_2* *сигнатура_операции_3* . . . . .

Типичная нотация класса

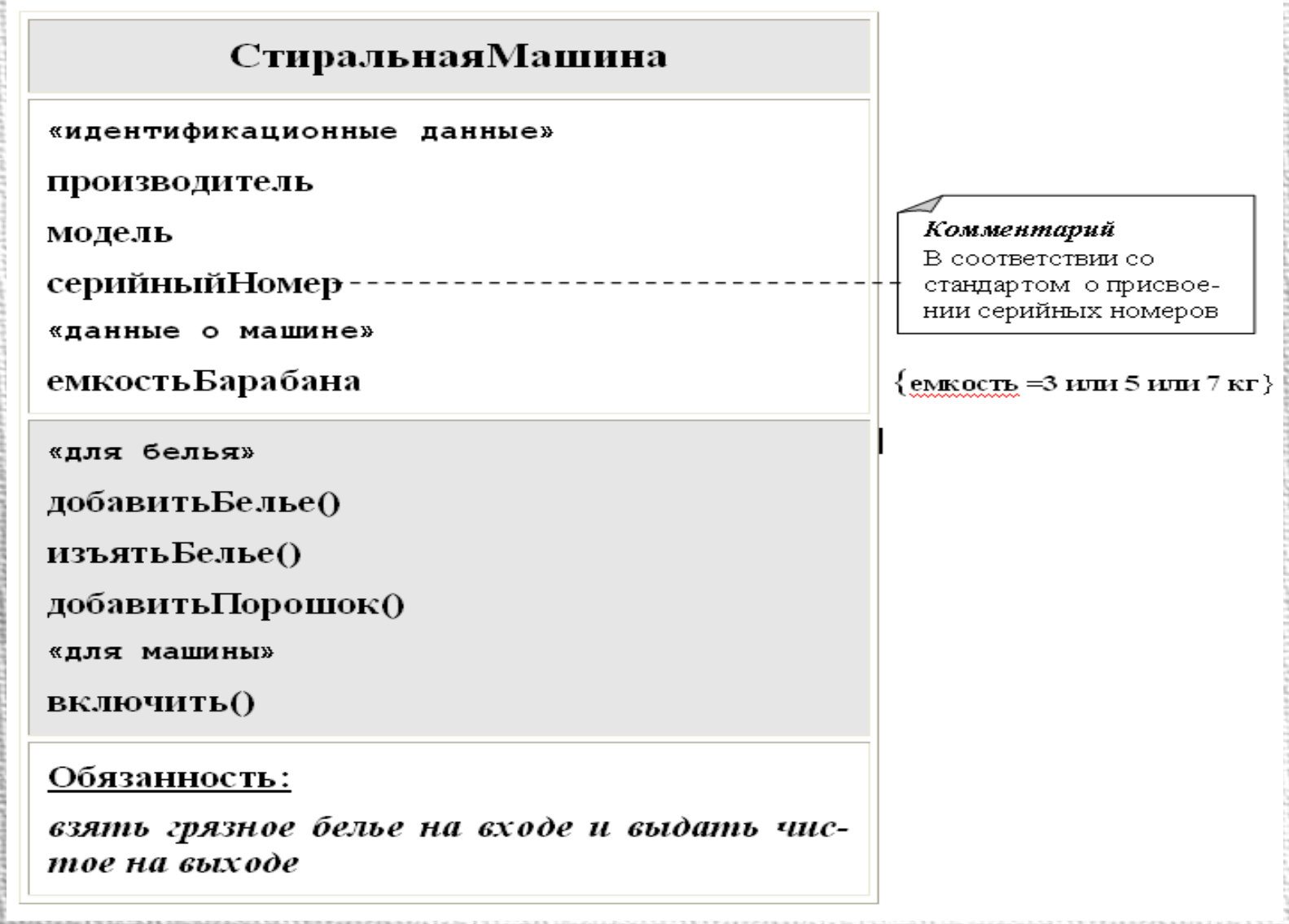
Стандартных секций три:

1. **секция имени** – наряду с обязательным именем может содержать также стереотип, кратность и список именованных значений;
2. **секция атрибутов** – содержит список описаний атрибутов класса;
3. **секция операций** – содержит список описаний операций класса.

Наряду со стандартными секциями, описание класса может содержать и произвольное количество дополнительных секций. Семантически дополнительные секции эквиваленты примечаниям.



Пример



Для более полного описания класса можно добавить ограничения в виде произвольного текста, заключенного в фигурные скобки.

Имя класса

Имя класса должно быть уникальным в пределах пакета, который может содержать одну или несколько диаграмм *классов*.
Имя класса записывается по центру секции *имени* полужирным шрифтом.
Обычно каждое слово в имени класса пишется с заглавной буквы Например, Customer(Клиент), Wall(Стена), TemperatureSensor(ДатчикТемпературы).
Секция имени класса в общем случае имеет следующий синтаксис.

«стереотип» ИМЯ {свойства} кратность

В некоторых случаях необходимо явно указать, к какому пакету относится тот или иной *класс*. Для этой цели используется специальный символ разделитель – двойное двоеточие (::). Синтаксис строки *имени класса* в этом случае будет следующий:

<Имя пакета>::<Имя класса>

Стандартные стереотипы классов

Стереотип	Описание
«actor»	Действующее лицо
«auxiliary»	Вспомогательный класс
«enumeration»	Перечислимый тип данных
«exception»	Исключение (только в UML 1)
«focus»	Основной класс
«implementationClass»	Реализация класса
«interface»	Все составляющие абстрактные
«metaclass»	Экземпляры являются классами
«powertype»	Метакласс, экземплярами которого являются все наследники данного класса (только в UML 1)

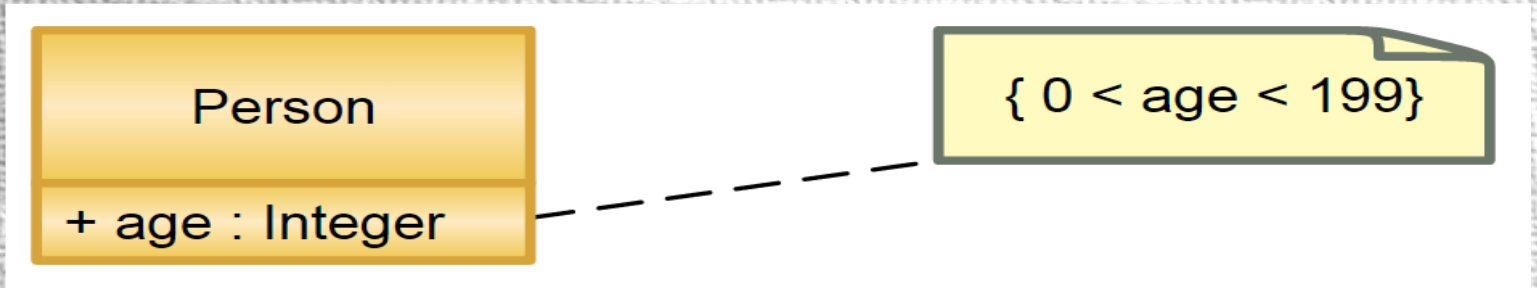
Стереотип	Описание
«process»	Активный класс
«thread»	Активный класс (только в UML 1)
«signal»	Класс, экземплярами которого являются сигналы
«stereotype»	Новый элемент на основе существующего
«type»	Тип данных
«dataType»	Тип данных
«utility»	Нет экземпляров, служба

Обязательное имя класса может быть выделено курсивом и в этом случае данный класс является абстрактным, т.е. не имеющим непосредственных экземпляров.

Если имя подчеркнуто, то это уже не имя класса, а имя объекта.

Класс, а также отдельные элементы его описания могут иметь произвольные заданные пользователем ограничения и именованные значения.

Кратность класса задается по общим правилам.



Пример использования ограничения

Пример секции имени класса

Рассмотрим пример секции имени класса для информационной системы отдела кадров.

Если мы предполагаем, что проектируемая информационная система отдела кадров будет использоваться на одном предприятии, то хорошим решением будет определение служебного класса **Company** со стереотипом «**utility**» для хранения глобальных атрибутов и операций информационной системы отдела кадров. Секция имени такого класса показана ниже.

Атрибуты

Атрибут **— это именованное место (или, как говорят, *слот*), в котором может храниться значение.**

ИМЯ АТРИБУТА

Имя атрибута - единственный обязательный элемент синтаксического обозначения *атрибута*, должно начинаться со строчной (малой) буквы и не должно содержать пробелов. Каждое слово в имени атрибута, кроме самого первого, обычно пишется с заглавной буквы. Например *name* или *loadBearing*

Формат записи атрибута

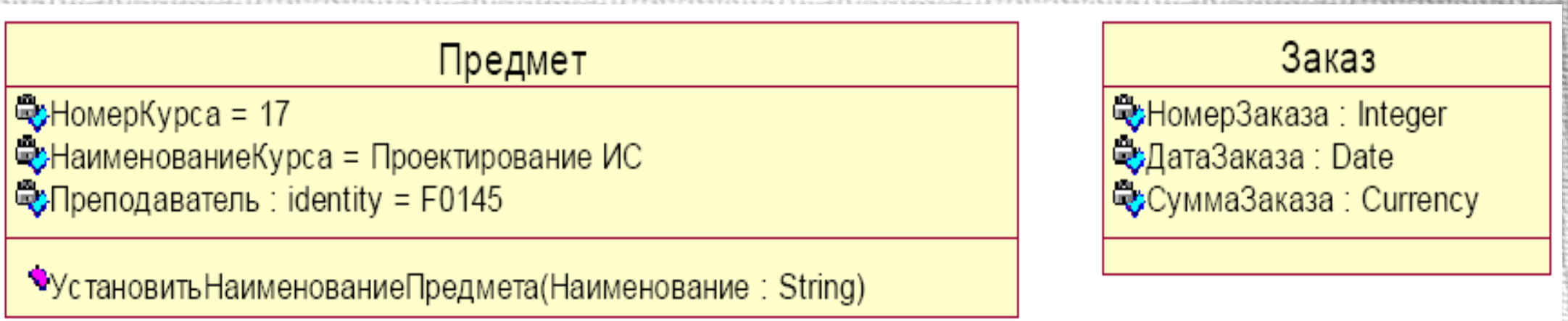
Общий формат записи отдельного *атрибута класса* следующий:

<квантор видимости> <имя атрибута>

[кратность] :<тип атрибута>

= <исходное значение> {строка-свойство}

Атрибуты классов



Тип атрибута может быть встроенным элементарным типом (*primitive type*) или другим классом. **Элементарный тип** – это непосредственно распознаваемый и поддерживаемый базовой объектно-ориентированной средой тип данных.

ТИП АТРИБУТА, ОБОЗНАЧАЮЩИЙ КЛАСС

Тип атрибута также может обозначать класс. Применительно к конкретному объекту некоторого класса подобный атрибут содержит значение идентификатора объекта (OID), указывающее на объект другого класса. **В UML-моделях анализа атрибуты, типы которых обозначают классы, не приводятся в среднем «отделении» представления класса (в отличие от примитивных типов). Вместо этого они представляются с помощью ассоциации (association) между классами.**

ВИДИМОСТЬ

Видимость (visibility) — качественная характеристика описания элементов *класса*, характеризующая потенциальную возможность других объектов модели оказывать влияние на отдельные аспекты поведения данного *класса*.

Видимость свойств класса определяют для того, чтобы скрыть детали его реализации и показать только те особенности, которые необходимы для осуществления обязанностей, продекларированных абстракцией.

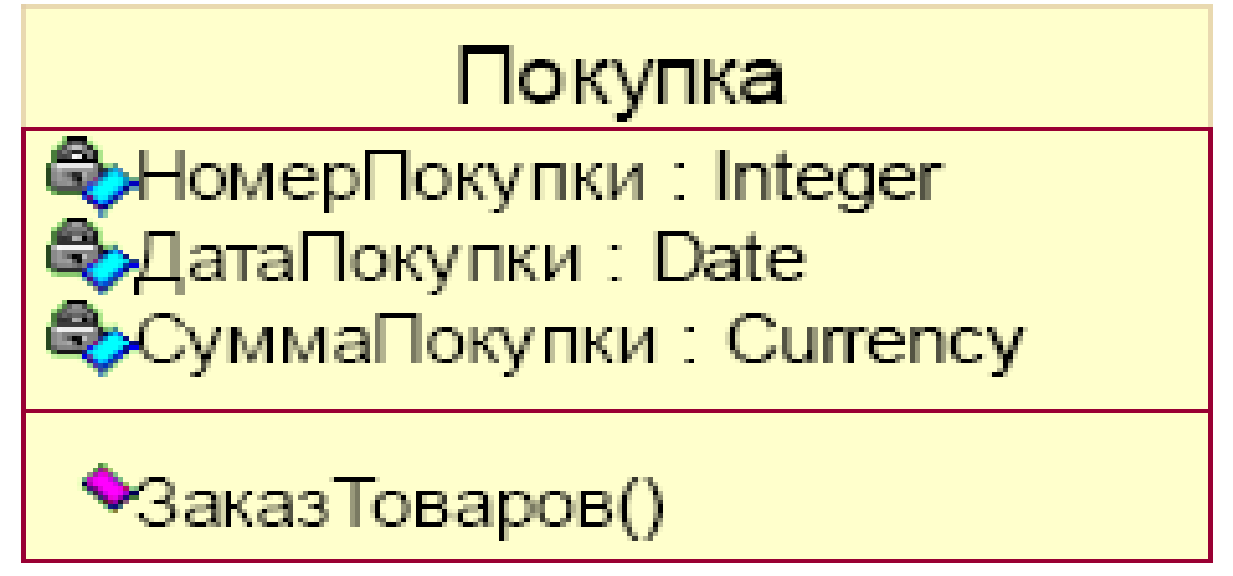
Квантор видимости

Видимость в языке UML специфицируется с помощью *квантора видимости (visibility)*, который может принимать одно из 4-х возможных значений и отображаться при помощи специальных символов.

1. Символ **"+"** – обозначает *атрибут* с областью видимости типа общедоступный (*public*). *Атрибут* с этой областью видимости доступен или виден из любого другого *класса* пакета, в котором определена диаграмма.
2. Символ **"#"** – обозначает *атрибут* с областью видимости типа защищенный (*protected*). *Атрибут* с этой областью видимости недоступен или невиден для всех *классов*, за исключением подклассов данного *класса*.

3. Символ **"-"** – обозначает *атрибут* с областью видимости типа закрытый (*private*). *Атрибут* с этой областью видимости недоступен или невиден для всех *классов* без исключения.
4. И, наконец, символ **«~»** - обозначает *атрибут* с областью видимости типа пакетный (*package*). *Атрибут* с этой областью видимости недоступен или невиден для всех *классов* за пределами пакета, в котором определен *класс*-владелец данного *атрибута*.

ВИДИМОСТЬ АТРИБУТОВ



В некоторых CASE-средствах эти символы заменены на графические пиктограммы

КРАТНОСТЬ АТРИБУТА

Кратность атрибута характеризует общее количество конкретных *атрибутов* данного типа, входящих в состав отдельного *класса*.

В общем случае *кратность* записывается в форме строки текста из цифр в квадратных скобках после имени соответствующего *атрибута*, при этом цифры разделяются двумя точками: [*нижняя граница .. верхняя граница*]. Если в качестве *кратности* указывается единственное число, то *кратность атрибута* принимается равной данному числу.

Дополнительные синтаксические конструкции

Подчеркивание строки *атрибута*

означает, что соответствующий *атрибут* общий для всех объектов данного *класса*, т.е. его значение у всех создаваемых объектов одинаковое.

Пояснительный текст в фигурных скобках

Косая черта перед именем *атрибута*

Знак «/» перед именем *атрибута* указывает на то, что данный *атрибут* является производным от некоторого другого *атрибута* этого же *класса*.

Значения свойства изменяемости атрибута

Значение	Описание
{changeable} **{unrestricted}**	Никаких ограничений на изменение значения атрибута не накладывается. Данное значение имеет место по умолчанию, поэтому указывать в модели его излишне. Первый вариант используется в UML 1, второй – в UML 2
{addOnly}	При изменении значения атрибута новое значение добавляется в массив значений, но старые значения не меняются и не исчезают. Такой атрибут "помнит" историю своего изменения. В UML 2 не используется, т.к. семантика определена нечетко.
{frozen} **{readOnly}**	Значение атрибута задается при инициализации объекта и не может меняться. Первый вариант используется в UML 1, второй – в UML 2

Например, в информационной системе отдела кадров класс **Person**, скорее всего, должен иметь атрибут, хранящий имя сотрудника. В следующей таблице приведен список примеров описаний такого атрибута. Все описания синтаксически допустимы и могут быть использованы в соответствии с текущим уровнем детализации модели.

Примеры описаний атрибутов

Пример	Пояснение
name	Минимальное возможное описание – указано только имя атрибута
+name	Указаны имя и открытая видимость – предполагается, что манипуляции с именем будут производиться непосредственно
-name : String	Указаны имя, тип и закрытая видимость – манипуляции с именем будут производиться с помощью специальных операций
-name[1..3] : String	В дополнение к предыдущему указана кратность (для хранения трех составляющих; фамилии, имени и отчества)
-name : String="Novikov"	Дополнительно указано начальное значение
+name : String{readOnly}	Атрибут объявлен не меняющим своего значения после начального присваивания и открытым

Операции

ОПЕРАЦИИ

Операция объявляется в классе. Процедура, реализующая операцию, называется **методом** (method). Операция (или метод, если быть точным) вызывается с помощью отправленного ей сообщения. Имя сообщения и имя операции совпадают. Операция может содержать список параметров, которым в вызове сообщения могут быть присвоены определенные значения, и может возвращать значение вызывающему объекту.

Имя операции вместе со списком типов формальных аргументов называется **сигнатурой** (signature) операции. Сигнатура в пределах класса должна быть уникальной. Это значит, что класс может обладать множеством операций с одним и тем же именем, при условии, что списки типов параметров этих операций отличаются.

Если сигнатуры различны, то и операции различны (даже если совпадают имена). В одном классе не может быть двух операций с одной сигнатурой — модель считается противоречивой. Если в подклассе определена операция с той же самой сигнатурой, то возможны два случая.

1. Если описание операции в подклассе в точности то же самое или если оно является непротиворечивым расширением (например, в классе не был указан тип результата, а в подклассе он указан), то это повторное описание той же самой операции.
2. Если же описание операции с совпадающей сигнатурой в подклассе противоречит описанию в классе (например, явно указаны различные направления передачи параметров), то модель считается противоречивой.

СИНТАКСИС

Общий формат записи отдельной операции класса следующий:

<квантор видимости> <имя операции> (список параметров): <выражение типа возвращаемого значения> {строка-свойство}

ИМЯ ОПЕРАЦИИ

Имя *операции* - единственный обязательный элемент синтаксического обозначения *операции*, должно начинаться со строчной (малой) буквы, и, как правило, записываться без пробелов.

СПИСОК ПАРАМЕТРОВ

Список параметров является перечнем разделенных запятой формальных *параметров*, каждый из которых, в свою очередь, может быть представлен в следующем виде:

<направление параметра> <имя параметра>: <выражение типа> = <значение параметра по умолчанию>

Параметр может включать имя, тип, направление и значение по умолчанию.

Направление параметра — есть одно из ключевых слов **in**, **out** или **inout** со значением **in** по умолчанию, в случае если вид *параметра* не указывается.

Ключевое слово	Назначение параметра
`in`	Входной параметр — аргумент д. б. значением, которое используется в операции, но не изменяется
`out`	Выходной параметр — аргумент д. б. хранилищем, в которое операция помещает значение
`inout`	Входной и выходной параметр — аргумент д. б. хранилищем, содержащим значение
`return`	Значение, возвращаемое операцией

Терминология: **параметры** – формальные
аргументы – фактические

Примеры операций

Пример	Пояснение
move	Минимальное возможное описание — указано только имя операции
+move(in from, in to)	Указаны видимость операции, направления передачи и имена параметров
+move(in from : Dpt, in to : Dpt)	Подробное описание сигнатуры: указаны видимость операции, направления передачи, имена и типы параметров
+getName() : String {isQuery}	Функция, возвращающая значение атрибута и не имеющая побочных эффектов
+setPwd(in pwd : String = "password")	Процедура, для которой указано значение аргумента по умолчанию

Типы операций

- **Операции реализации** – реализуют некоторую бизнес-функциональность. Можно найти, исследуя диаграммы Взаимодействия. Диаграммы этого типа фокусируются на бизнес-функциональности, и каждое сообщение диаграммы, скорее всего, можно соотнести с операцией реализации.
- **Операции управления** (manager operations) управляют созданием и разрушением объектов. В эту категорию попадают конструкторы и деструкторы классов.

- **Операции доступа.** Атрибуты обычно бывают закрытыми или защищенными. Тем не менее, другие классы иногда должны просматривать или изменять их значения. Для этого предназначены операции доступа (access operations).
- **Вспомогательные (helper operations) операции,** которые необходимы для выполнения ответственностей, но о которых другие классы не должны ничего знать. Это закрытые и защищенные операции класса.

Интерфейсы и типы данных

В UML имеется несколько частных случаев классификаторов, которые, подобно классам, предназначены для моделирования структуры, но обладают рядом специфических особенностей. Наиболее важными из них являются **интерфейсы** и **типы данных**.

Интерфейс **– это именованный набор составляющих, описывающий контракт между поставщиками и потребителями услуг.**

Интерфейсы позволяют наладить множественное наследование объектов и в то же время решить проблему ромбовидного наследования.

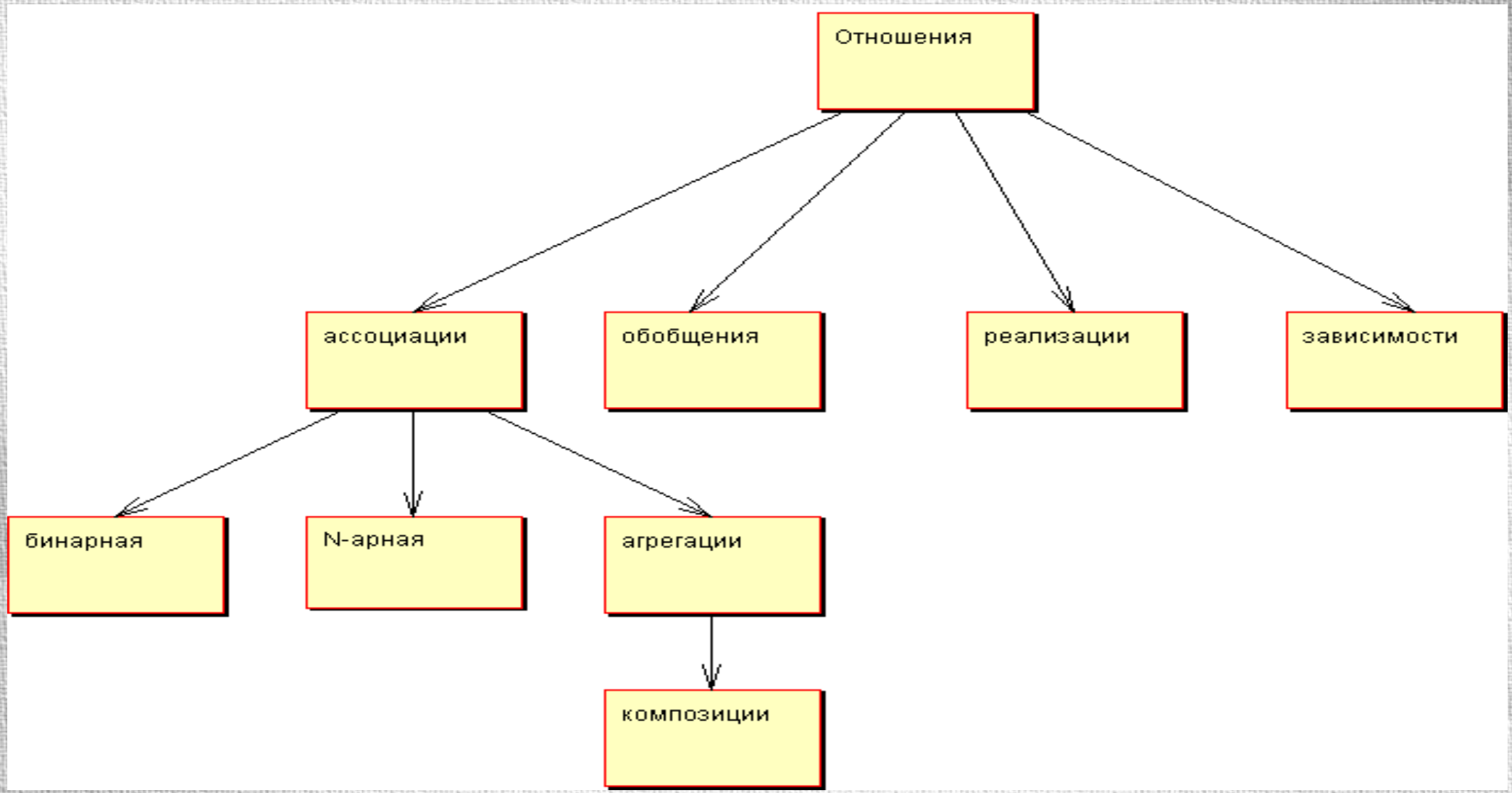
ОТНОШЕНИЯ КЛАССОВ

- Агрегации
- Композиции
- Обобщения
- Зависимости
- Реализации

Виды отношений между классами

- Отношение ассоциации (association relationship)
- Отношение агрегации (aggregation relationship)
- Отношение композиции (composition relationship)
- Отношение обобщения (generalization relationship)
- Отношение зависимости (dependency relationship)
- Отношение реализации

ОТНОШЕНИЯ МЕЖДУ КЛАССАМИ



Сущности на диаграммах классов связываются главным образом отношениями **ассоциации** (в том числе агрегирования и композиции) и **обобщения**. Отношения зависимости и реализации на диаграммах классов применяются реже

НОТАЦИЯ UML ДЛЯ ОТОБРАЖЕНИЯ ВЗАИМОСВЯЗИ МЕЖДУ КЛАССАМИ

Отношение ассоциации
(association relationship)

СВЯЗЬ ОБЪЕКТОВ ПРИ АССОЦИАЦИИ

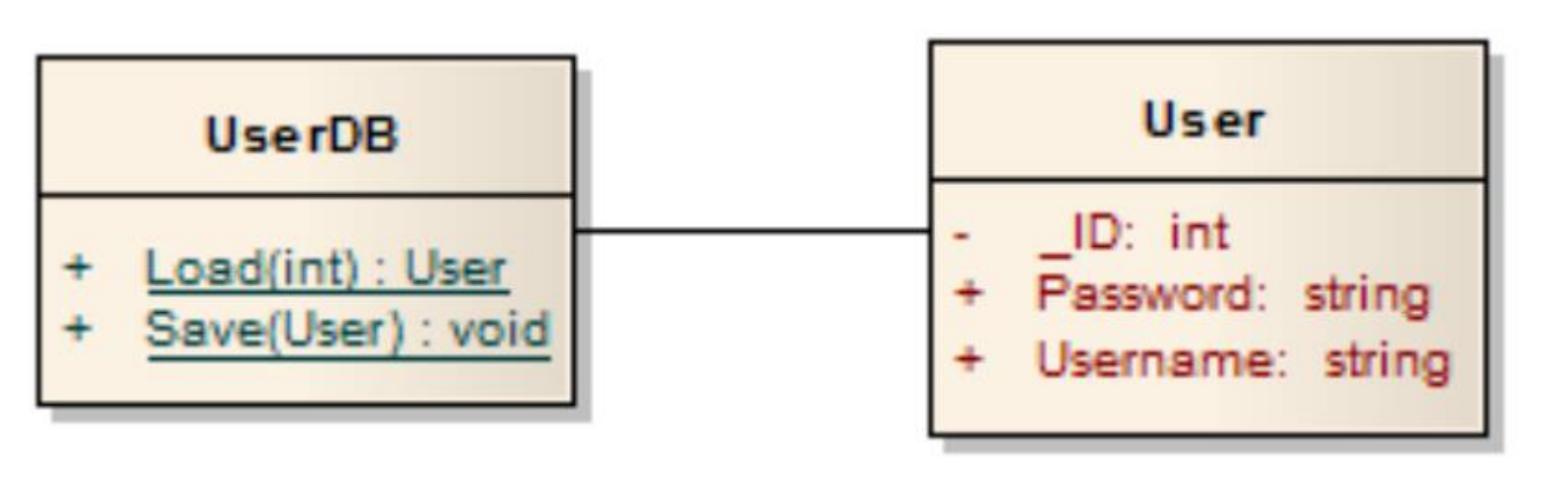
Если два класса связаны друг с другом ассоциацией, то это означает, что их экземпляры (объекты) определенным образом связаны друг с другом, например:

- **вызывают методы друг друга,**
- **работают с общей памятью,**
- **объекты одного класса являются параметрами методов другого,**
- **один класс имеет атрибут с типом другого класса (или указателя на него).**

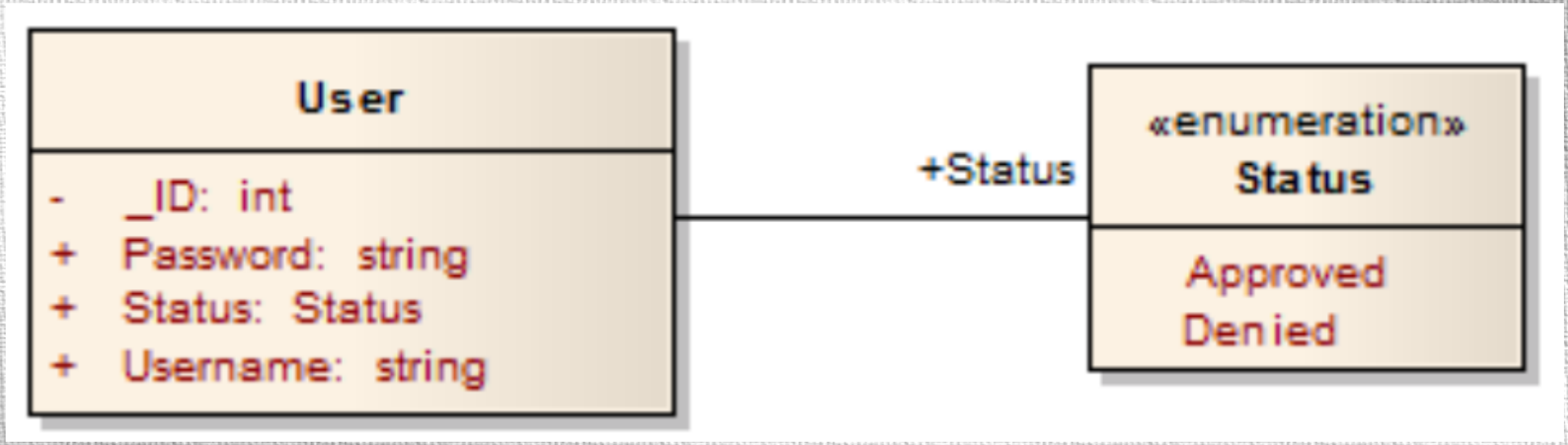
Ассоциация как связь между классами обязательно переходит в связь между экземплярами этих классов. Этим она принципиально отличается от наследования. Ниже мы подробно остановимся на отличиях агрегирования и наследования.

Ассоциация - наиболее широко используемая связь между классами. Она имеет достаточно широкое значение и может означать, например, следующее:

1. Один класс осуществляет взаимодействие с другим каким-либо образом. Например:

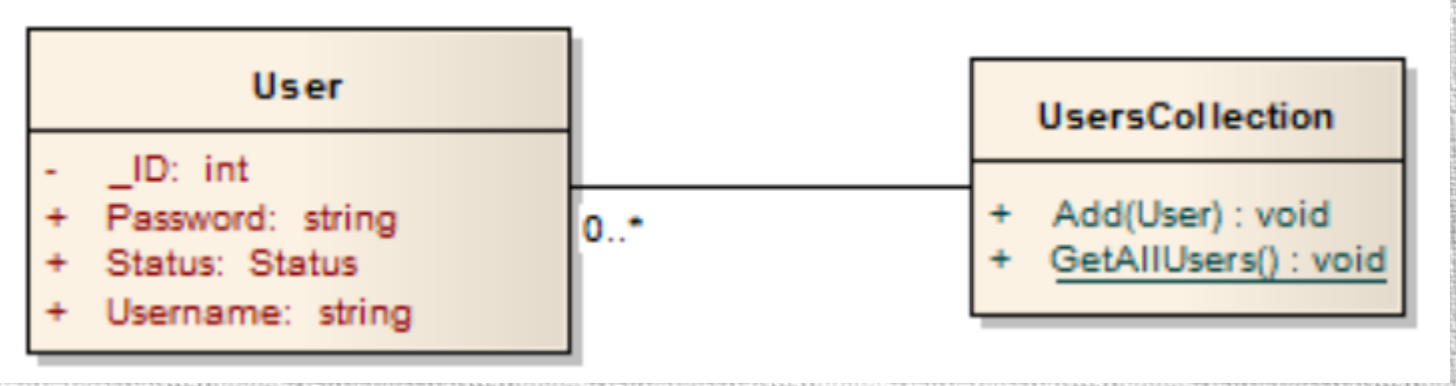


2. Один класс включает в себя экземпляр другого класса. Например:



Видим, что перечисление Status включено в класс User, при этом имя поля Status, с областью видимости public.

3. Один класс включает в себя несколько экземпляров другого класса. Например:



Видим, что коллекция UsersCollection может включать от нуля до бесконечности объектов User.

Возможные значения количества объектов:

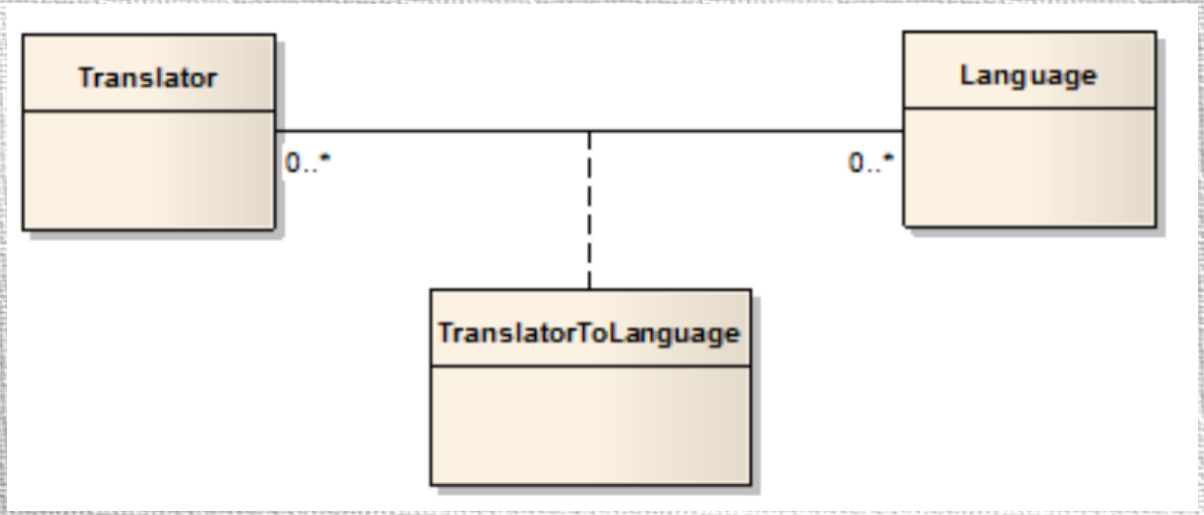
«*» или «0..*» - любое количество объектов

«0..n» - любое количество объектов, но не больше n, например «0..5»

«n» - точное количество объектов, например «1», «5»

«n..*» - любое количество объектов, но не меньше n, например«1..*» - хотя бы один.

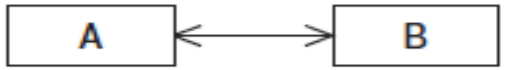
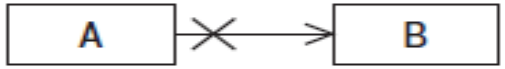
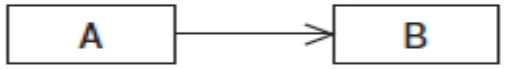
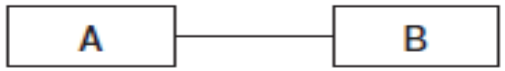
Если Ассоциация не может быть реализована без дополнительных классов (например, отношение «многие-ко-многим»), то она реализуется при помощи дополнительных классов ассоциации, которые предназначены решить эту задачу.



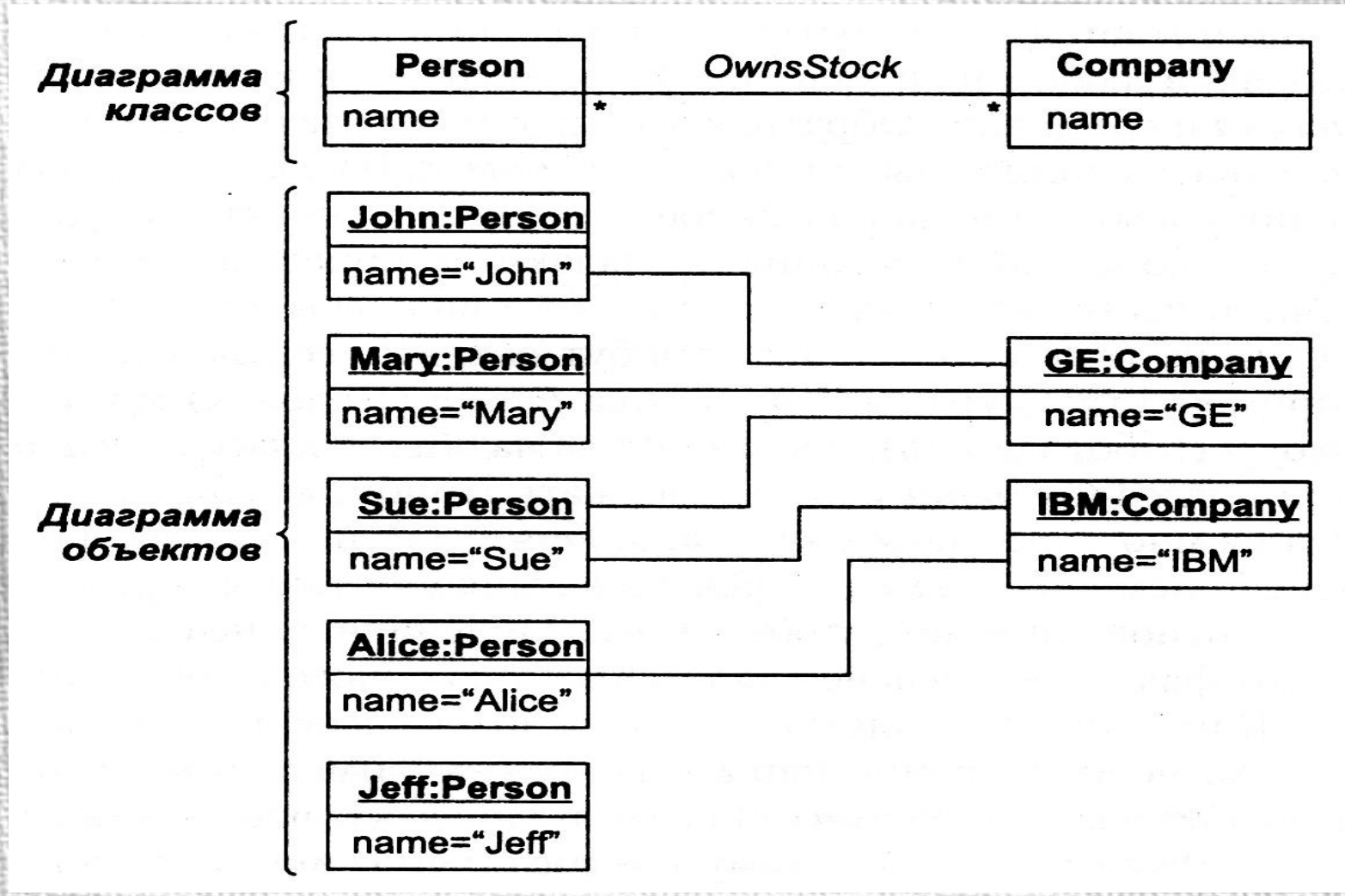
В данном примере вы видите, что для того, чтобы установить связь «многие-ко-многим» между переводчиками (класс Translator) и языками (класс Language) используется вспомогательный класс TranslatorToLanguage.

ОТНОШЕНИЕ АССОЦИАЦИИ

Данное отношение обозначается сплошной линией со стрелкой или без нее и с дополнительными символами, которые характеризуют специальные свойства ассоциации.

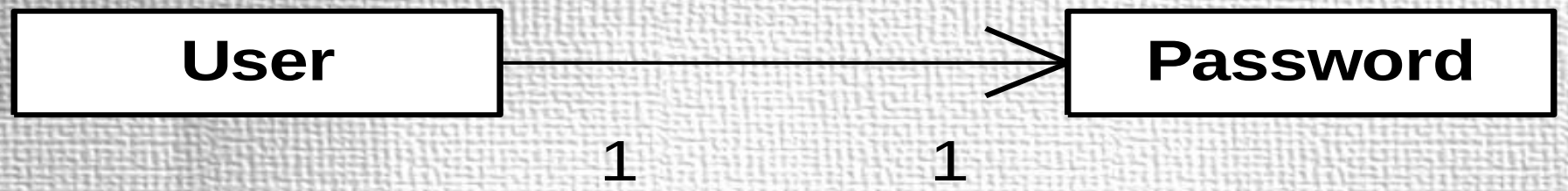
Стили представления возможности навигации в UML 2			
Синтаксис UML 2	Стиль 1: Строгое обозначение возможности навигации в UML 2	Стиль 2: Возможность навигации не обозначается	Стиль 3: Общепринятая практика
	Есть возможность навигации от A к B Есть возможность навигации от B к A		
	Есть возможность навигации от A к B Нет возможности навигации от B к A		
	Есть возможность навигации от A к B Возможность навигации от B к A не определена		Есть возможность навигации от A к B Нет возможности навигации от B к A
	Возможность навигации от A к B не определена Возможность навигации от B к A не определена	Возможность навигации от A к B не определена Возможность навигации от B к A не определена	Есть возможность навигации от A к B Есть возможность навигации от B к A
	Нет возможности навигации от A к B Нет возможности навигации от B к A		

Ассоциация классов и соединение объектов



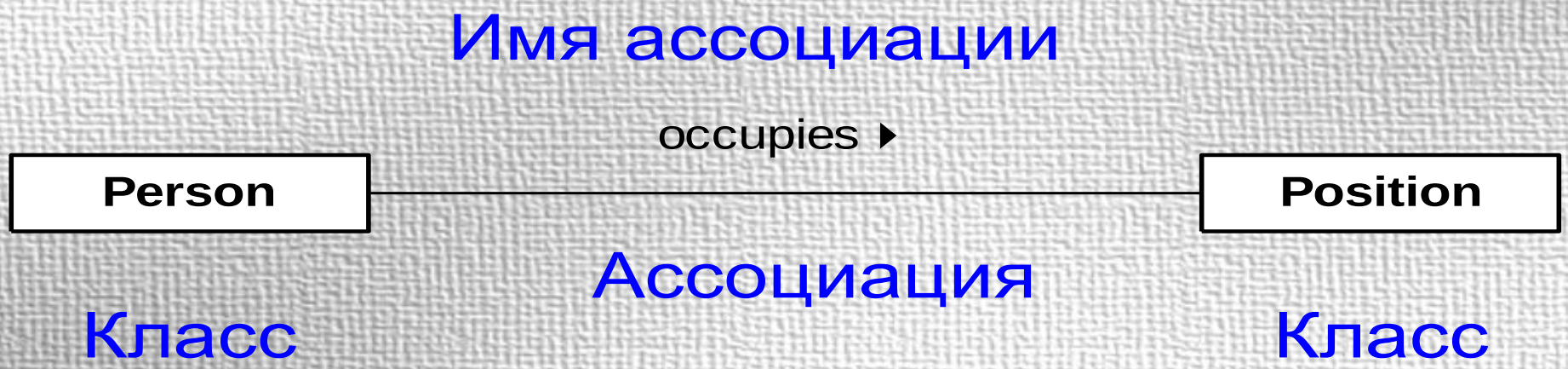
Направление навигации

Ассоциация
с указанным
направлением
навигации



Имя ассоциации

Можно указать направление чтения



Роли

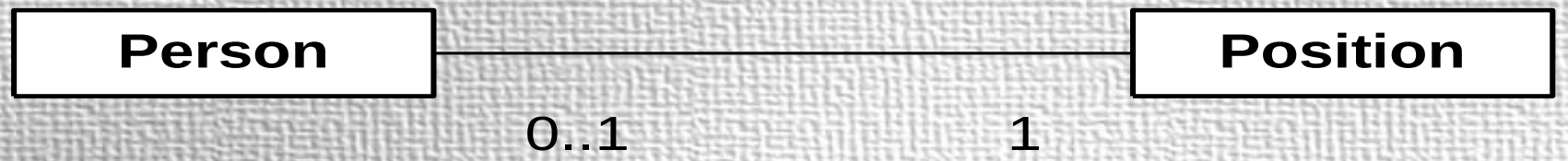
Каждый конец ассоциации называется ролью (role). Роль дополнительно может иметь следующие характеристики:

- **Имя**
- **Кратность**
- **Направление связи**

Кратность полюса ассоциации

Значение кратности определяет, сколько экземпляров одного класса может быть корректно связано с экземпляром другого класса в некоторый конкретный момент времени.

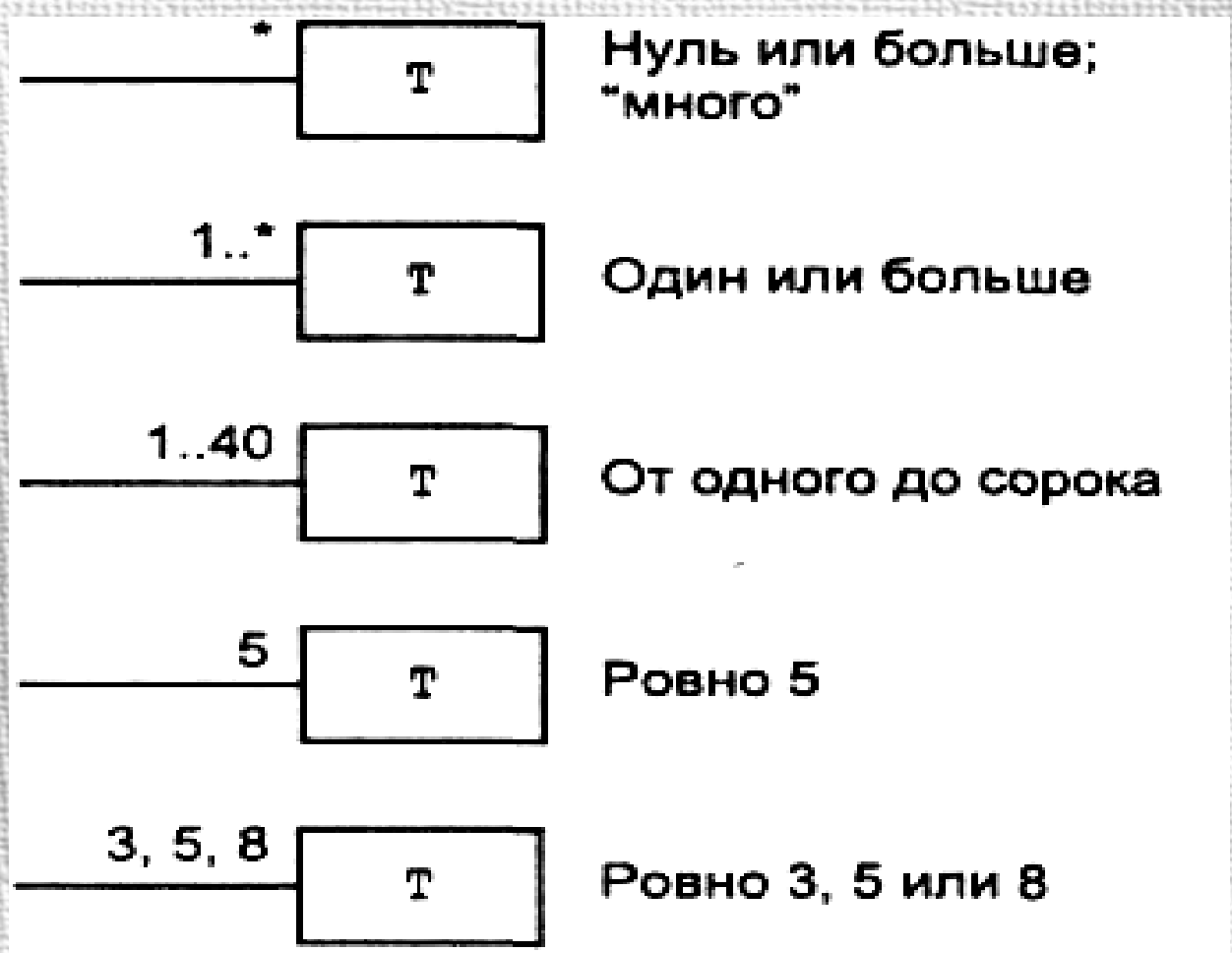
* - неопределенное число



Кратность задана диапазоном

Кратность задана числом

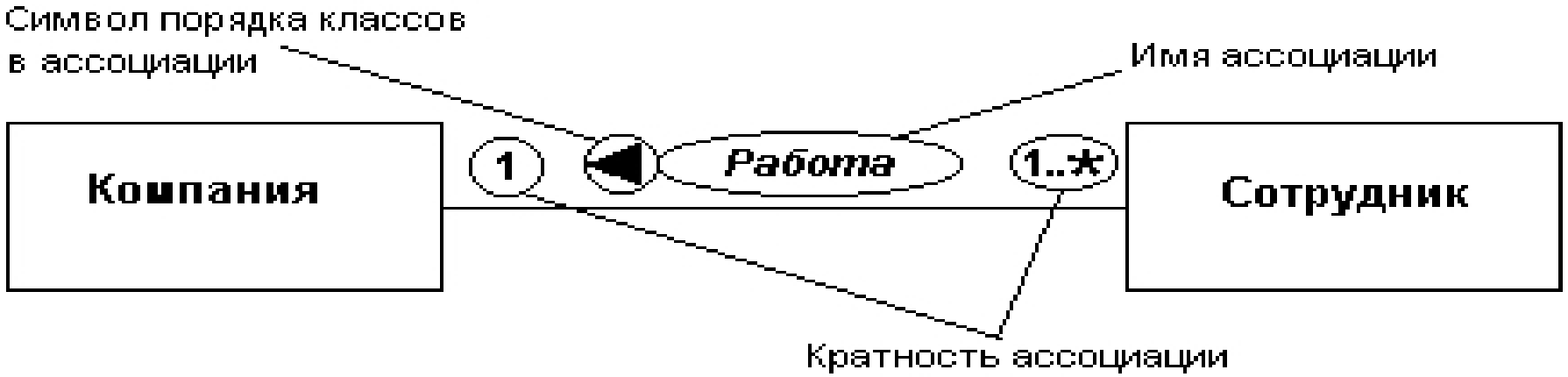
Примеры кратности



Бинарная ассоциация

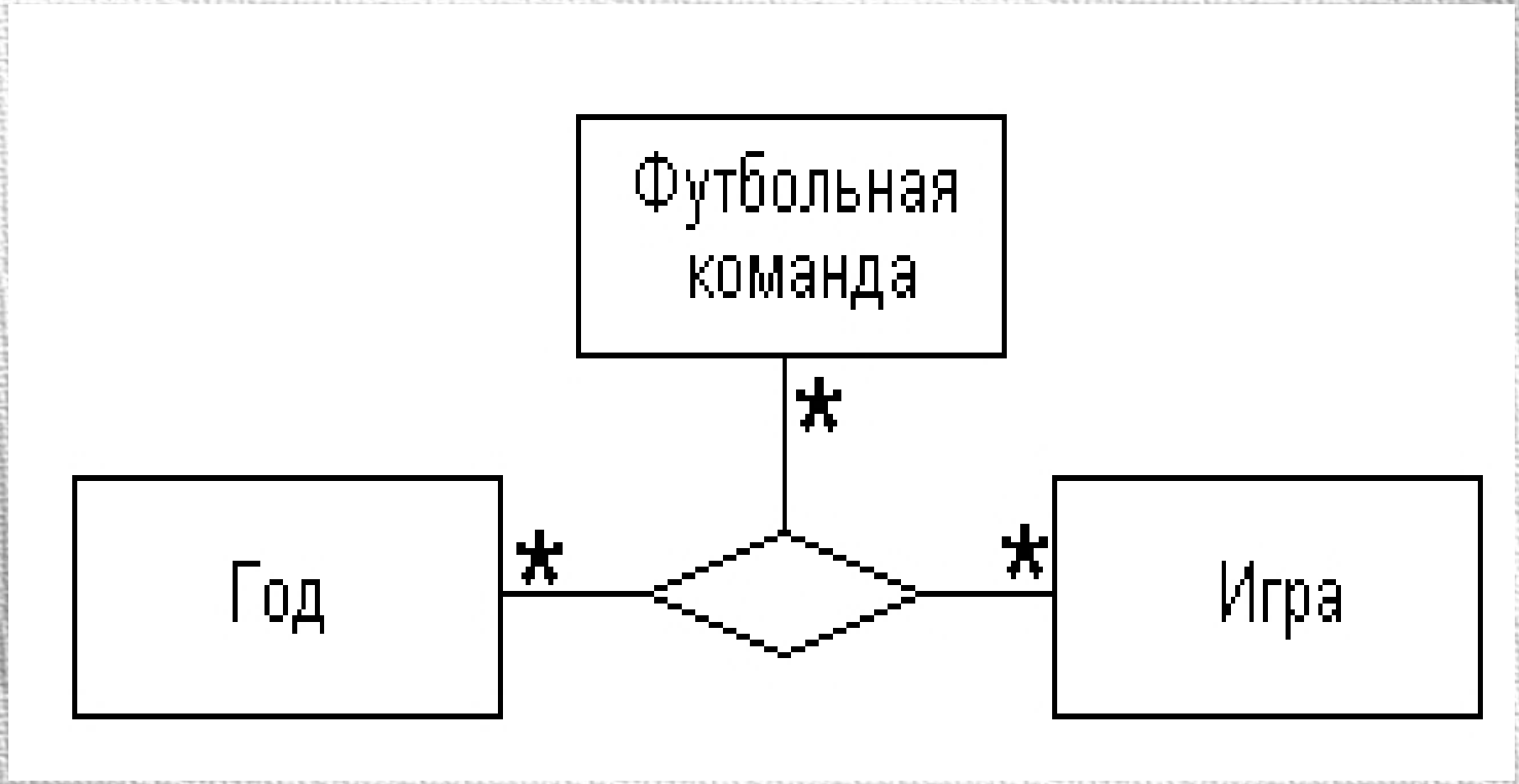
Наиболее простой случай данного отношения — бинарная ассоциация. Она связывает в точности два класса и, как исключение, может связывать класс с самим собой. Для бинарной ассоциации на диаграмме может быть указан порядок следования классов с использованием треугольника в форме стрелки рядом с именем данной ассоциации. Направление этой стрелки указывает на порядок классов, один из которых является первым (со стороны треугольника), а другой — вторым (со стороны вершины треугольника). Отсутствие данной стрелки рядом с именем ассоциации означает, что порядок следования классов в рассматриваемом отношении не определен.

Бинарная ассоциация

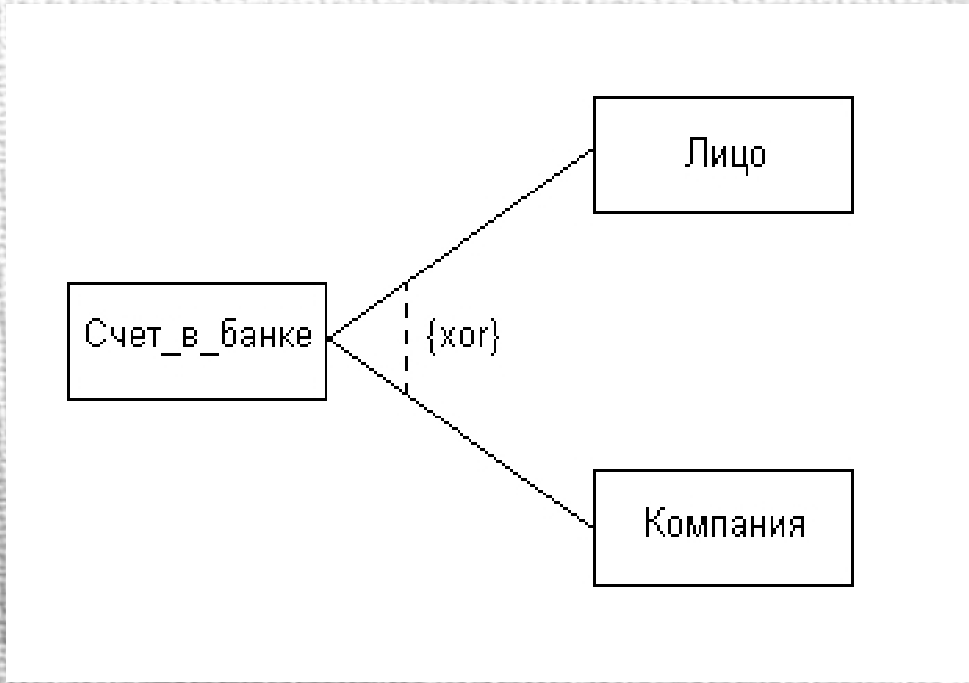


Классы связаны между собой бинарной ассоциацией ***Работа***, имя которой указано на рисунке рядом с линией ассоциации. Для данного отношения определен порядок следования классов, первым из которых является класс "Сотрудник", а вторым — класс "Компания". Отдельным примером или экземпляром данной ассоциации может являться пара значений (Петров И. И., "Рога&Копыта"). Это означает, что сотрудник Петров И. И. работает в компании "Рога&Копыта".

Тернарная ассоциация



Исключающая ассоциация (Xor-association)

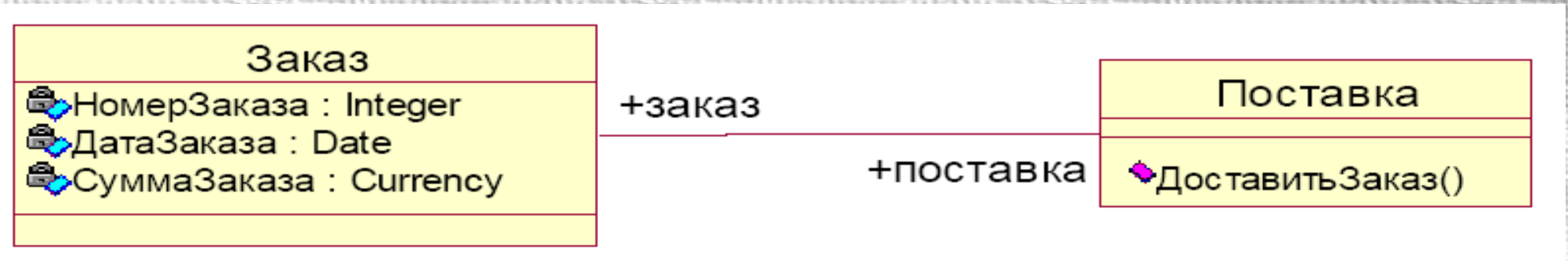


Семантика данной ассоциации указывает на тот факт, что из нескольких потенциально возможных вариантов данной ассоциации в каждый момент времени может использоваться только один ее экземпляр. На диаграмме классов исключающая ассоциация изображается пунктирной линией, соединяющей две и более ассоциации, рядом с которой записывается строка-ограничение "{xor}".

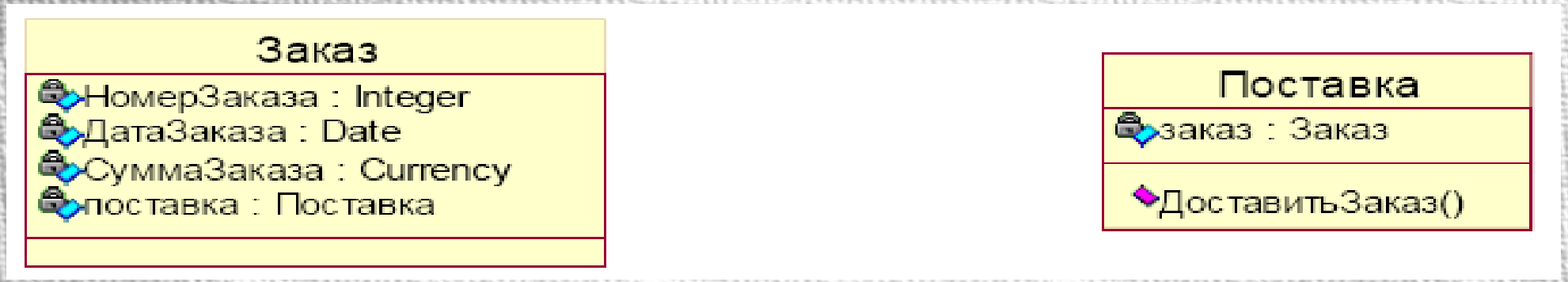
Например, счет в банке может быть открыт для клиента, в качестве которого может выступать физическое лицо (индивидум) или компания, что изображается с помощью исключающей ассоциации.

Ролевые имена, обозначающие ассоциацию между классами

Модель анализа



Модель реализации

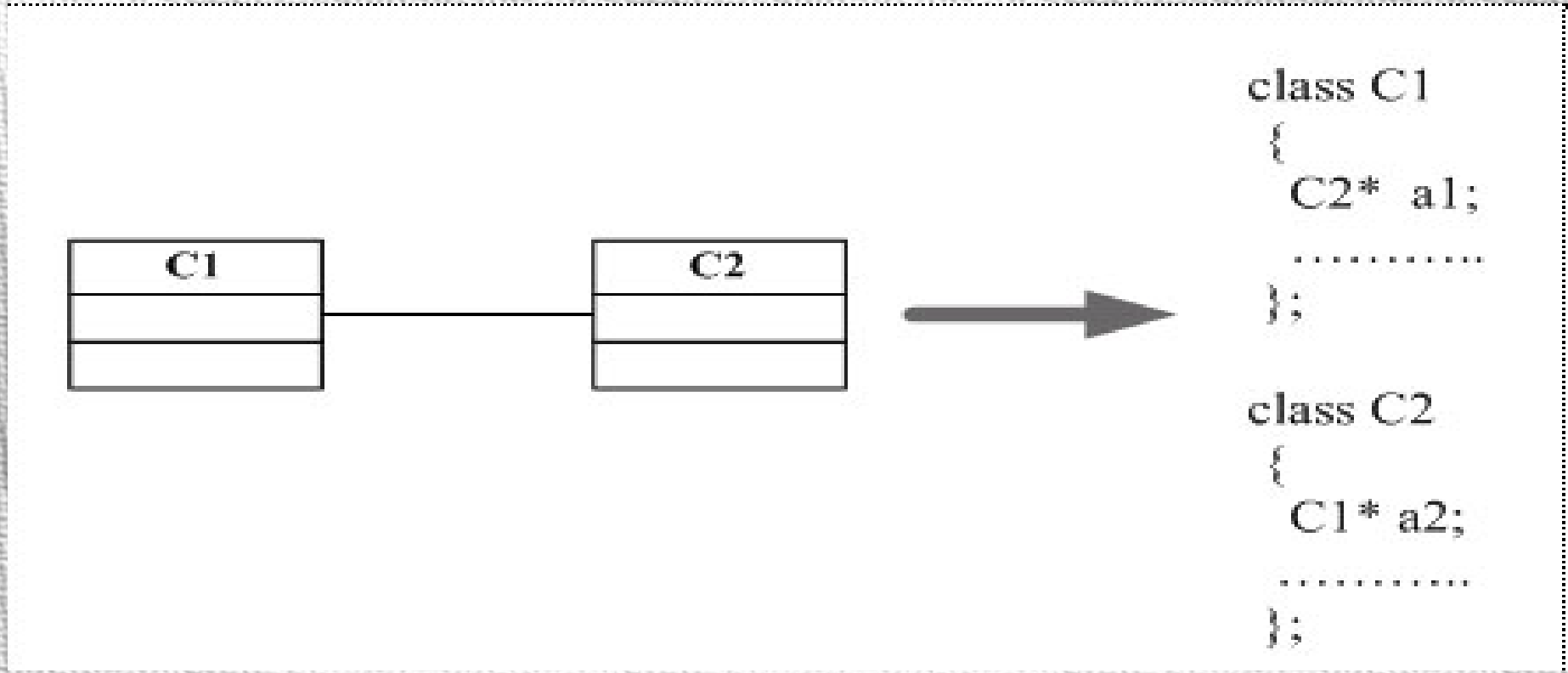


Как отображается отношение «ассоциация» на ДК в код?

- На C#?
- На Java?
- На Python?

Пример представления ассоциаций в программном коде языка C++

Ассоциации в программном коде могут быть реализованы, например, через атрибуты-указатели языка C++, как показано на рисунке.



Отношение агрегации
(aggregation relationship)

Отношение композиции
(composition relationship)

Механизм наследования используется для моделирования отношений типа «**является**», то есть отношения, когда объекты одного класса по существу являются теми же самыми, что и объекты какого-то другого класса, но с некоторыми отличиями, такими как дополнительные атрибуты данных или дополнительные методы.

Другой подход основан на использовании **механизма агрегирования (или композиции)**– когда **класс включает одну или более переменных экземпляра, являющихся экземплярами других классов**. Механизм агрегирования используется для моделирования отношений типа «**имеет**».

Два типа ассоциации в UML

Агрегация

Некоторые объекты слабо взаимосвязаны
друг с другом, как компьютер
и его периферийное оборудование

Композиция

Некоторые объекты тесно взаимосвязаны,
как дерево и его листья

АГРЕГАЦИЯ И КОМПОЗИЦИЯ

В UML используются два частных, но очень важных случая отношения ассоциации, которые называются агрегацией и композицией. В обоих случаях речь идет о моделировании отношения типа "часть – целое". Ясно, что отношения такого типа следует отнести к отношениям ассоциации, поскольку части и целое обычно взаимодействуют.

Агрегация **(aggregation) – это ассоциация между классом A (часть) и классом B (целое), которая означает, что экземпляры (один или несколько) класса A входят в состав экземпляра класса B.**

- **Агрегация**
  - Отношение **Часть → Целое**
  - Не изменяет направления навигации и не накладывает ограничений на время жизни
- **Композиция**
  - Буквально «**состоит из**»
  - Часть принадлежит только одному целому
  - Время жизни частей совпадает с временем жизни целого (*f.e. лист и дерево*)

Как проявляется агрегация и композиция?

Агрегация и композиция моделируют отношение «является частью» (HAS-A Relationship) и **обычно выражаются в том, что класс целого содержит поля (или свойства) своих составных частей**.

(пример)

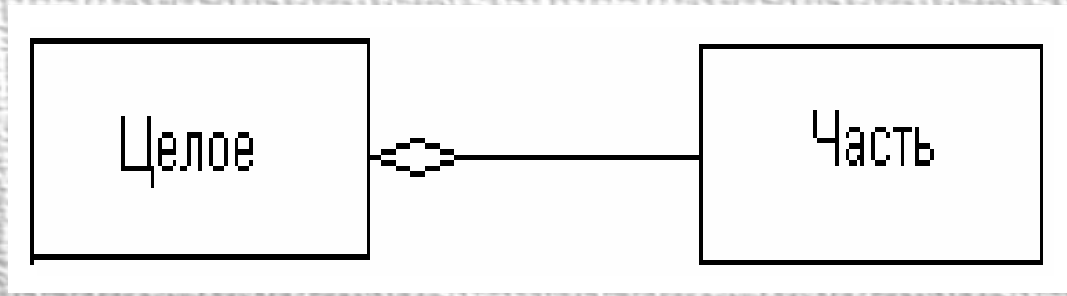
ОТНОШЕНИЕ АГРЕГАЦИИ

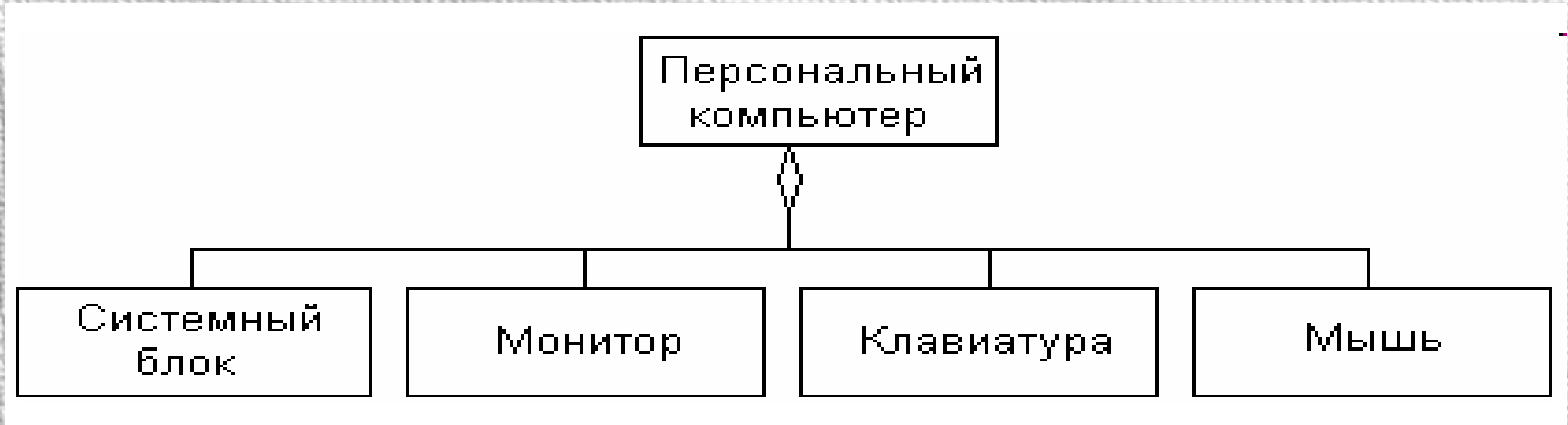
Отношение агрегации показывает, из каких компонентов состоит система и как они связаны между собой. С точки зрения модели отдельные части системы могут выступать как в виде элементов, так и в виде подсистем, которые, в свою очередь, тоже могут образовывать подсистемы.

Это отношение по своей сути описывает декомпозицию или разбиение сложной системы на более простые составные части, которые также могут быть подвергнуты декомпозиции, если в этом возникнет необходимость в последующем.

Части системы никак не обязаны наследовать ее свойства и поведение, поскольку являются вполне самостоятельными сущностями. Более того, части целого обладают своими собственными атрибутами и операциями, которые существенно отличаются от атрибутов и операций целого.

Графическое изображение отношения агрегации в языке UML

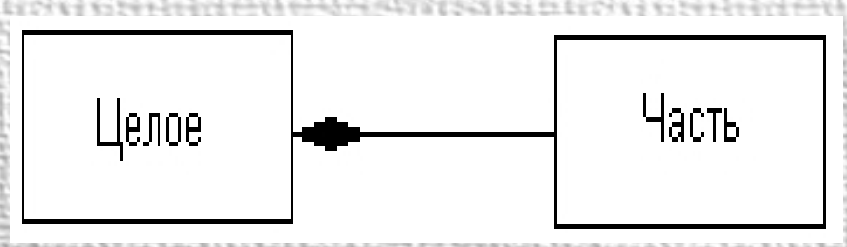


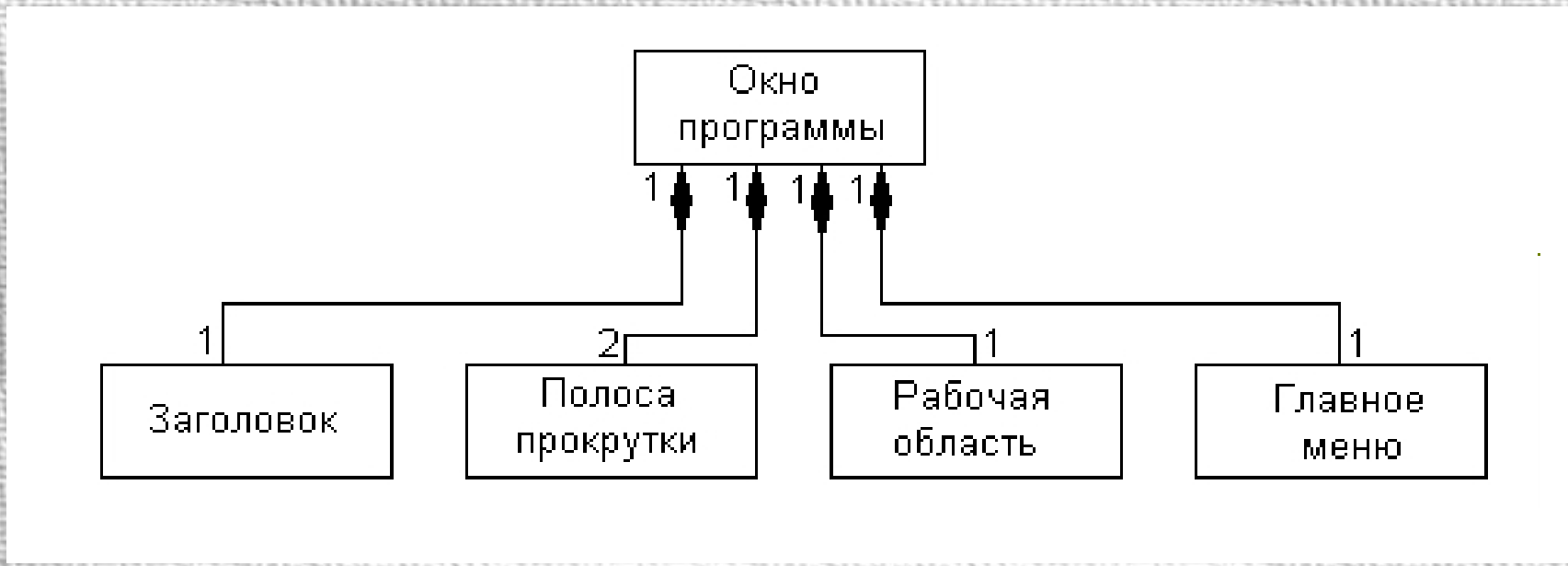


Отношение композиции

Отношение композиции является частным случаем отношения агрегации. Это отношение служит для выделения специальной формы отношения "часть-целое", при которой составляющие части в некотором смысле находятся внутри целого. Специфика взаимосвязи между ними заключается в том, что части не могут выступать в отрыве от целого, т. е. с уничтожением целого уничтожаются и все его составные части.

Графическое изображение отношения композиции в языке UML





Различия между композицией и агрегацией

Приведём наглядный пример. Комната является частью квартиры, следовательно здесь подходит композиция, потому что комната без квартиры существовать не может. А, например, мебель не является неотъемлемой частью квартиры, но в то же время, квартира содержит мебель, поэтому следует использовать агрегацию.

Как отображается отношение «агрегация» на ДК в код?

- На C#
- На Java
- На Python

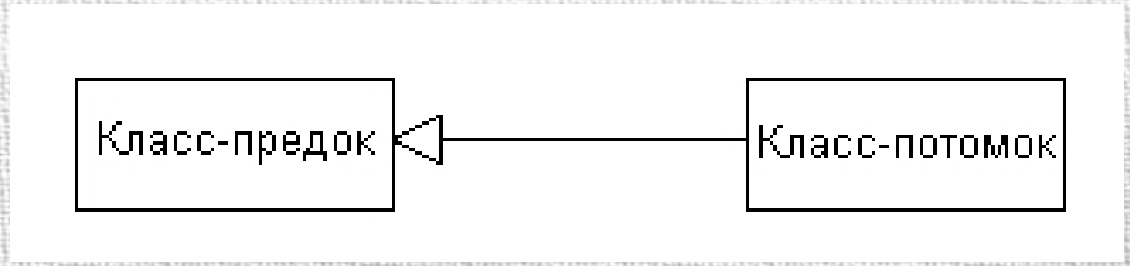
Отношение обобщения (generalization relationship)

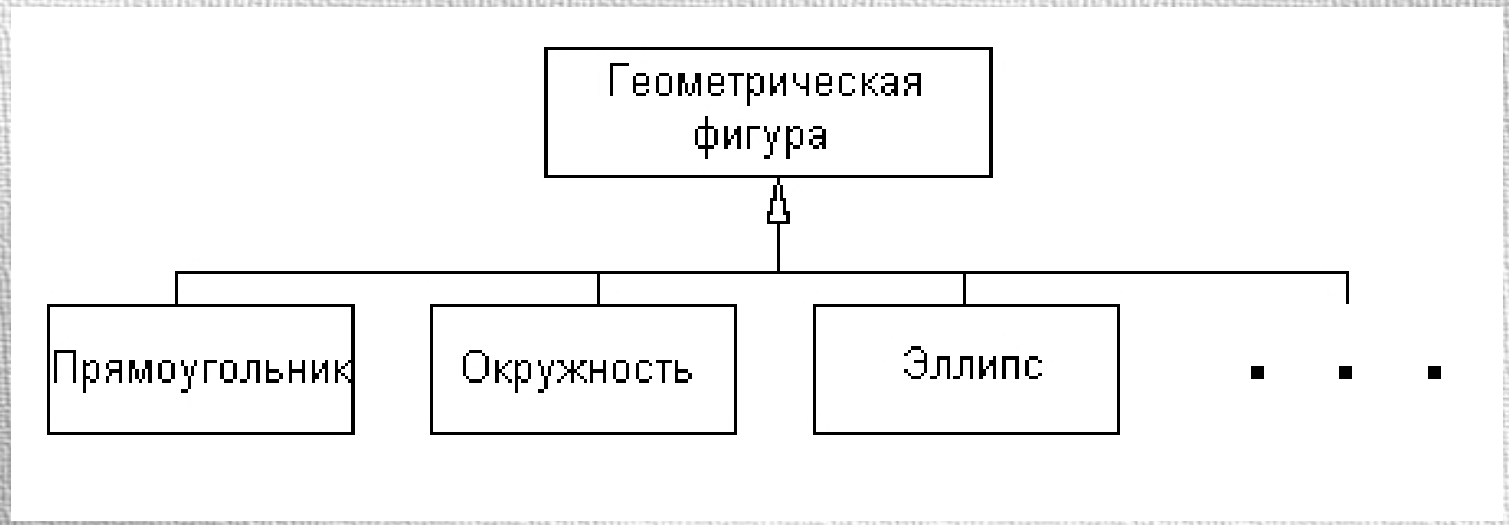
Отношение обобщения

Отношение обобщения является обычным таксономическим отношением между более общим элементом (родителем или предком) и более частным или специальным элементом (дочерним или потомком). Данное отношение может использоваться для представления взаимосвязей между пакетами, классами, вариантами использования и другими элементами языка UML.

Применительно к диаграмме классов данное отношение описывает иерархическое строение классов и наследование их свойств и поведения. При этом предполагается, что класс-потомок обладает всеми свойствами и поведением класса-предка, а также имеет свои собственные свойства и поведение, которые отсутствуют у класса-предка.

Графическое изображение отношения обобщения в языке UML

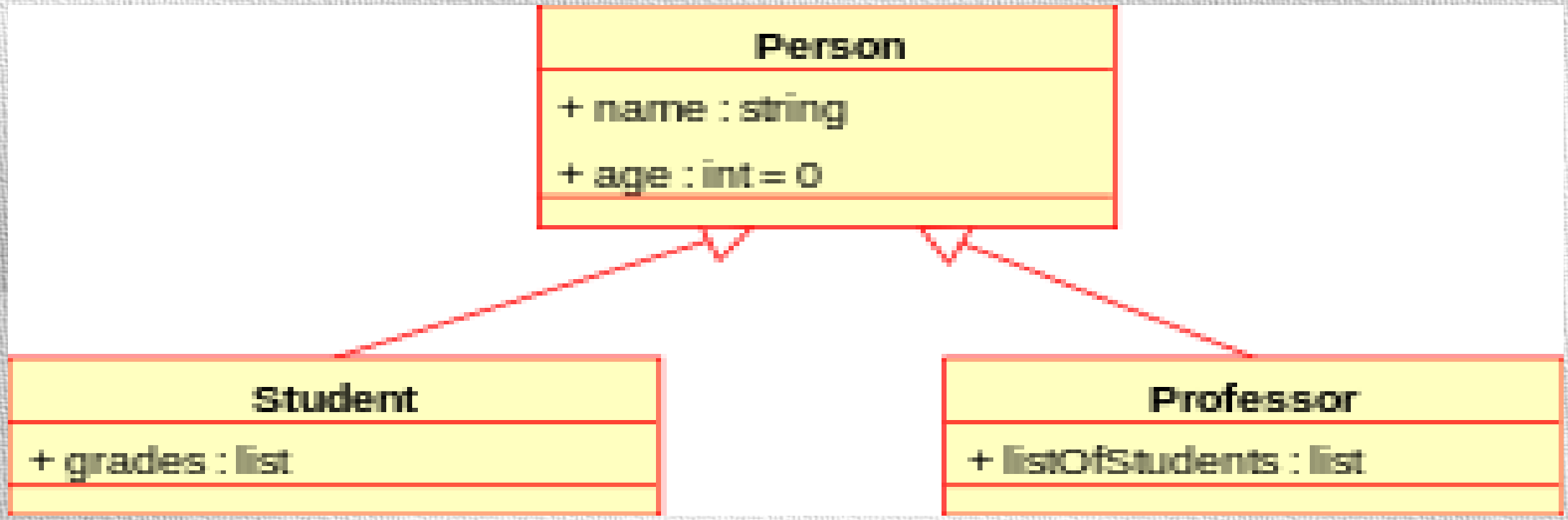




Рядом со стрелкой обобщения может размещаться строка текста, указывающая на некоторые дополнительные свойства этого отношения. При этом текст следует рассматривать как ограничение, и тогда он записывается в фигурных скобках.

Диаграмма классов, показывающая наследование двух подклассов от одного суперкласса

Обобщение также известно как наследование или «**is a**» взаимосвязь (или отношение «является»).



Ограничения

В качестве ограничений могут быть использованы следующие ключевые слова языка UML:

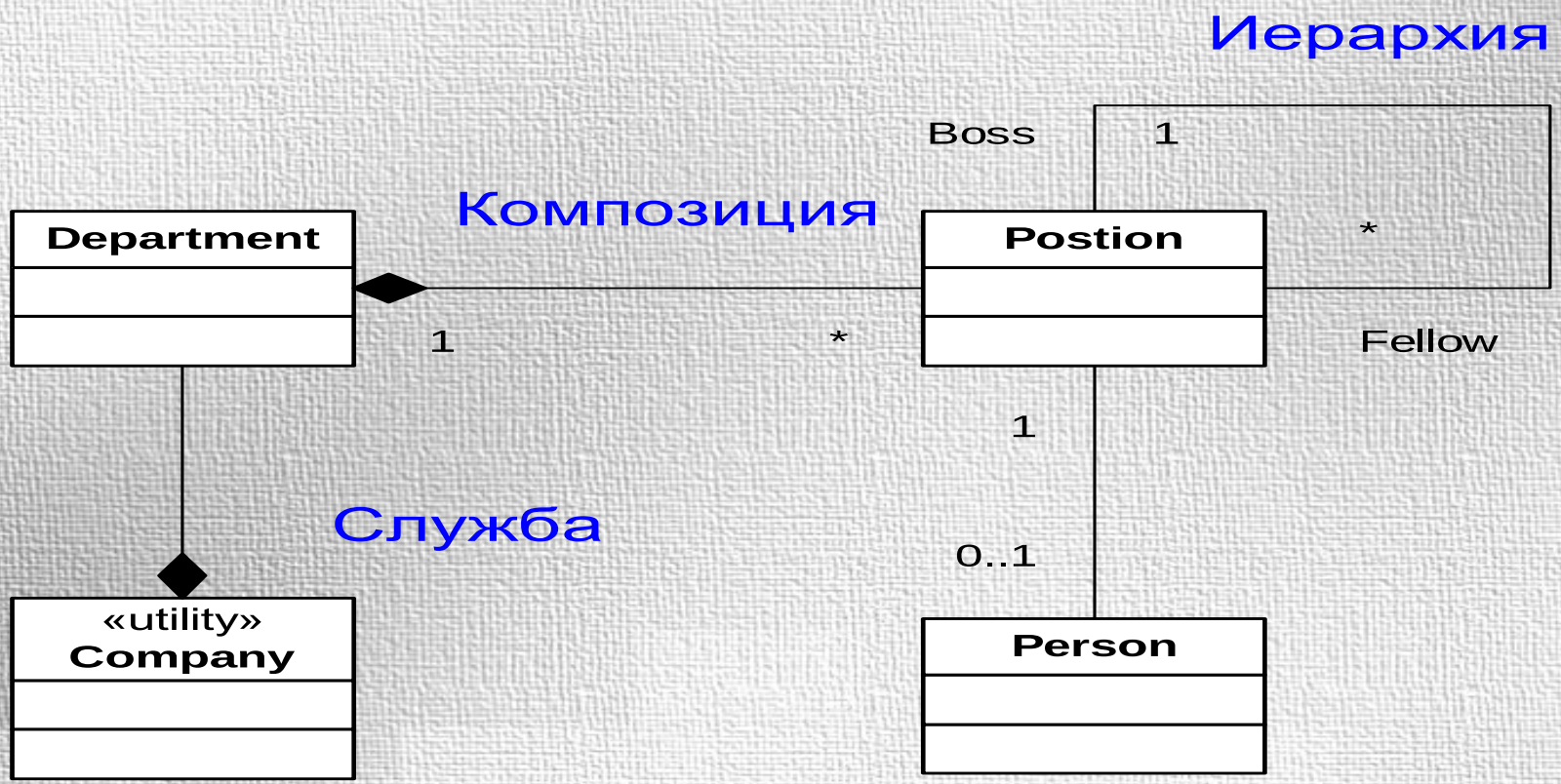
{complete} — означает, что в данном отношении обобщения специфицированы все классы-потомки, и других классов-потомков у данного класса-предка быть не может. Пример — класс **Клиент_банка** является предком для двух классов: **Физическое_лицо** и **Компания**, и других классов-потомков он не имеет. На соответствующей диаграмме классов это можно указать явно, записав рядом с линией обобщения данную строку-ограничение;

{disjoint} — означает, что классы-потомки не могут содержать объектов, одновременно являющихся экземплярами двух или более классов. В приведенном выше примере это условие также выполняется, поскольку предполагается, что никакое конкретное физическое лицо не может являться одновременно и конкретной компанией. В этом случае рядом с линией обобщения можно записать данную строку-ограничение;

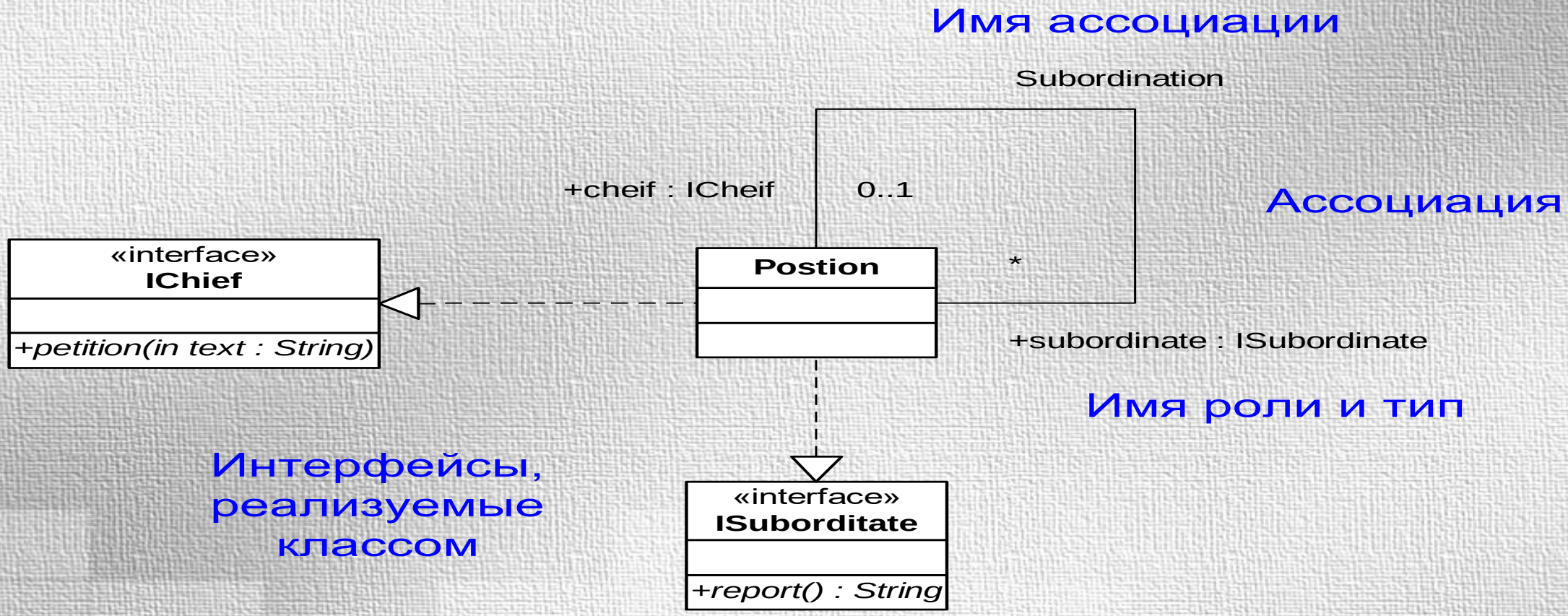
{**incomplete**} — означает случай, противоположный первому. А именно, предполагается, что на диаграмме указаны не все классы-потомки. В последующем возможно восполнить их перечень не изменяя уже построенную диаграмму. Пример — диаграмма класса "Автомобиль", для которой указание всех без исключения моделей автомобилей соизмеримо с созданием соответствующего каталога. С другой стороны, для отдельной задачи, такой как разработка системы продажи автомобилей конкретных моделей, в этом нет необходимости. Но указать неполноту структуры классов-потомков все же следует;

{**overlapping**} — означает, что отдельные экземпляры классов-потомков могут принадлежать одновременно нескольким классам. Пример — класс "Многоугольник" является классом-предком для класса "Прямоугольник" и класса "Ромб". Однако существует отдельный класс "Квадрат", экземпляры которого одновременно являются объектами первых двух классов. Вполне естественно такую ситуацию указать явно с помощью данной строки-ограничения

Примеры ассоциаций



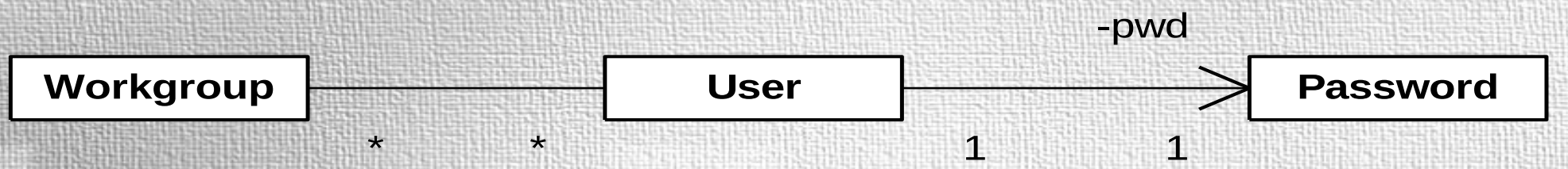
Роль полюса ассоциации



Видимость полюса ассоциации

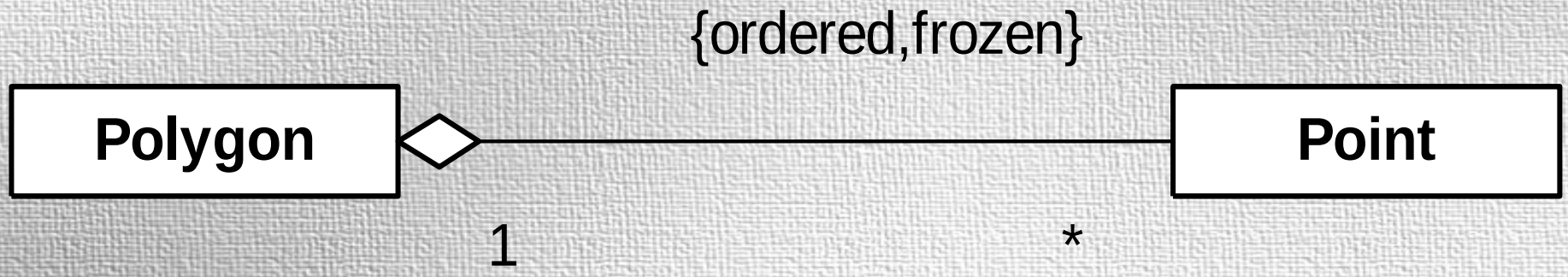
Для чужих – связанные ассоциацией всегда видят друг друга

Ассоциация с направлением навигации и огрничением видимости



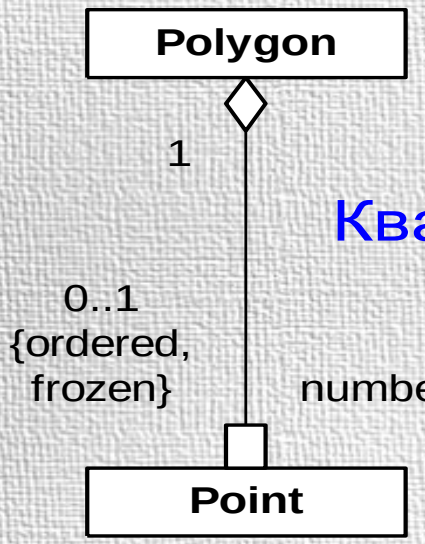
Ассоциация "многие ко многим"

Упорядоченность и изменяемость множества объектов на полюсе связи



Квалификатор полюса ассоциации

Кратность подмножества, выделяемого квалификтором



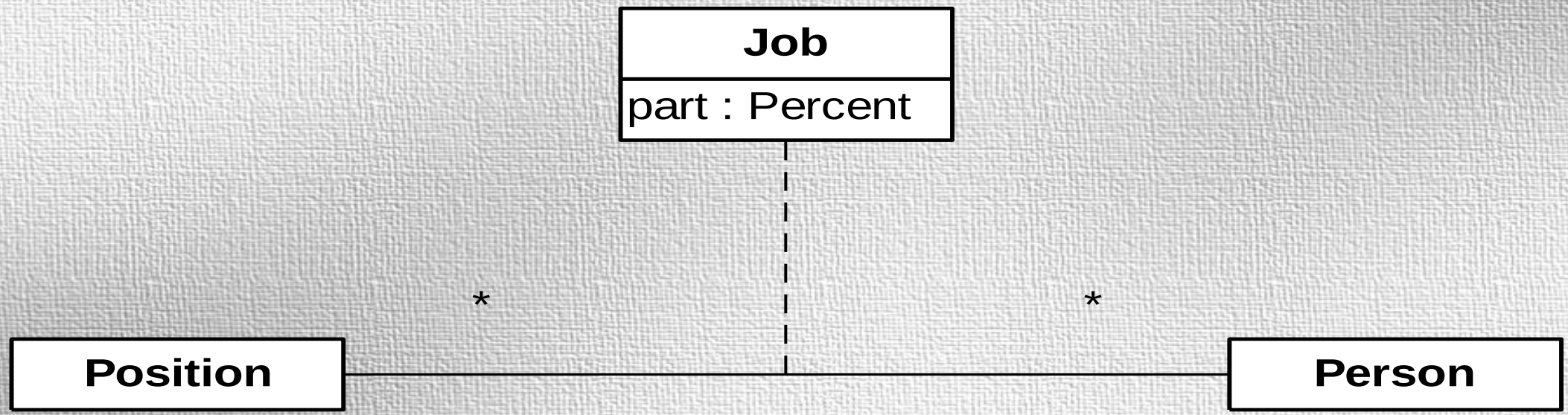
Квалифицированная ассоциация

Имя квалификатора

Квалификатор – это атрибут ассоциации

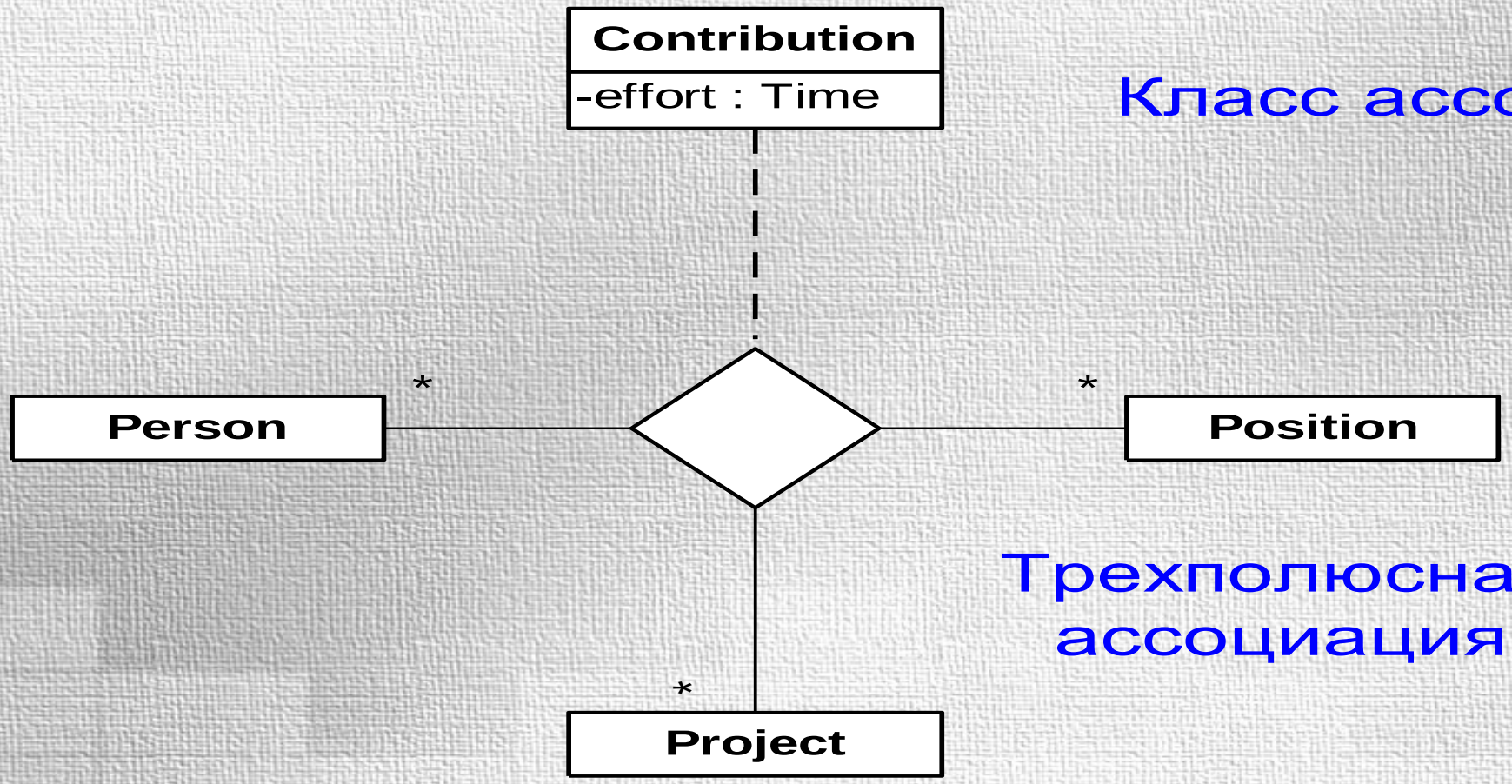
Класс ассоциации

Класс ассоциации



Линия ассоциации

Многополюсные ассоциации



Класс ассоциации

Трехполюсная ассоциация

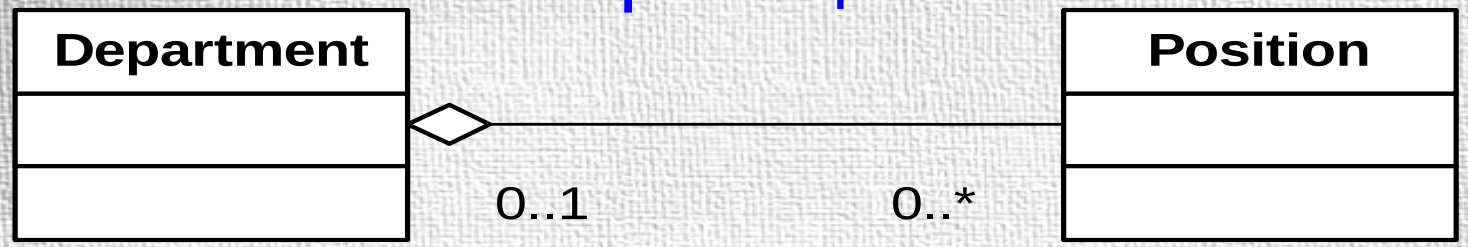
Примеры композиции и агрегации

Жесткая структура

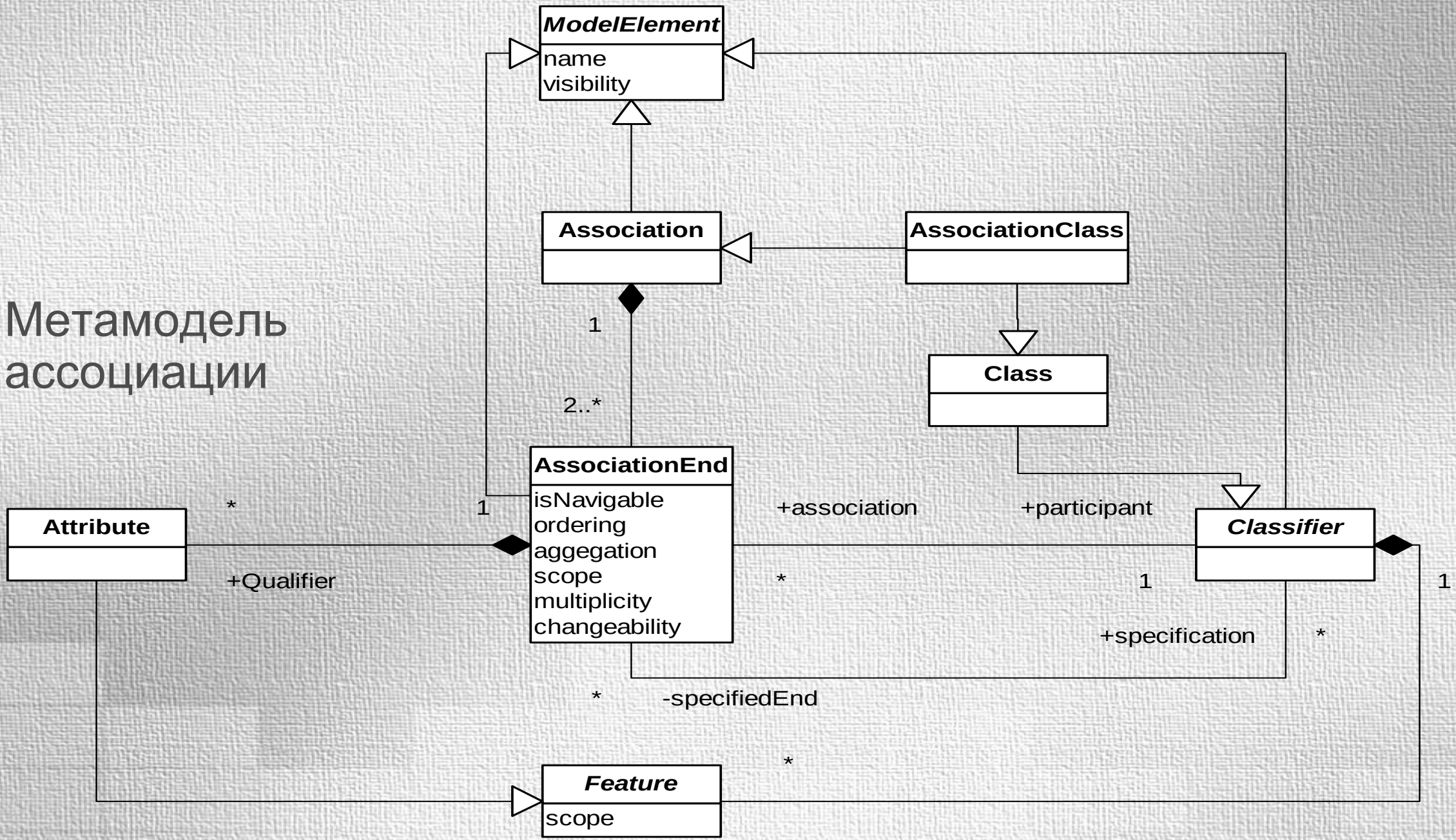
Композиция

Мягкая структура

Агрегация



Метамодель ассоциации



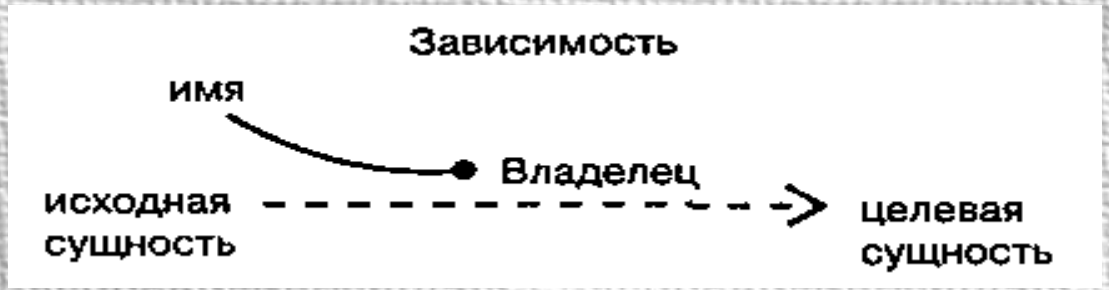
Как отображается отношение «обобщение» на ДК в код?

- На C#
- На Java
- На Python

Отношения зависимости и реализации

ЗАВИСИМОСТЬ (DEPENDENCY)

Зависимость (Dependency) - это семантическое отношение между двумя сущностями, при котором изменение одной из них (независимой сущности) может отразиться на семантике другой (зависимой).
Графически зависимости изображают в виде пунктирной линии со стрелкой, направленной в сторону независимой сущности.



Отношение зависимости используется в такой ситуации, когда некоторое изменение одного элемента модели может потребовать изменения другого зависимого от него элемента модели. Применяйте зависимости, если хотите показать, что одна сущность использует другую. **Наиболее общим случаем зависимости является использование одного класса в сигнатуре операции другого класса.**

СТЕРЕОТИПЫ ОТНОШЕНИЯ ЗАВИСИМОСТИ

Всего в UML определено довольно большое количество стандартных стереотипов отношения зависимости, которые можно разделить на несколько групп:

- *между классами и объектами на диаграмме классов;*
- *между пакетами;*
- *между вариантами использования;*
- *другие.*

Зависимости на диаграммах классов используются сравнительно редко, потому что имеют более расплывчатую семантику по сравнению с ассоциациями и обобщением.

Ниже рассматриваются зависимости первой группы, которые перечислены в следующей таблице.

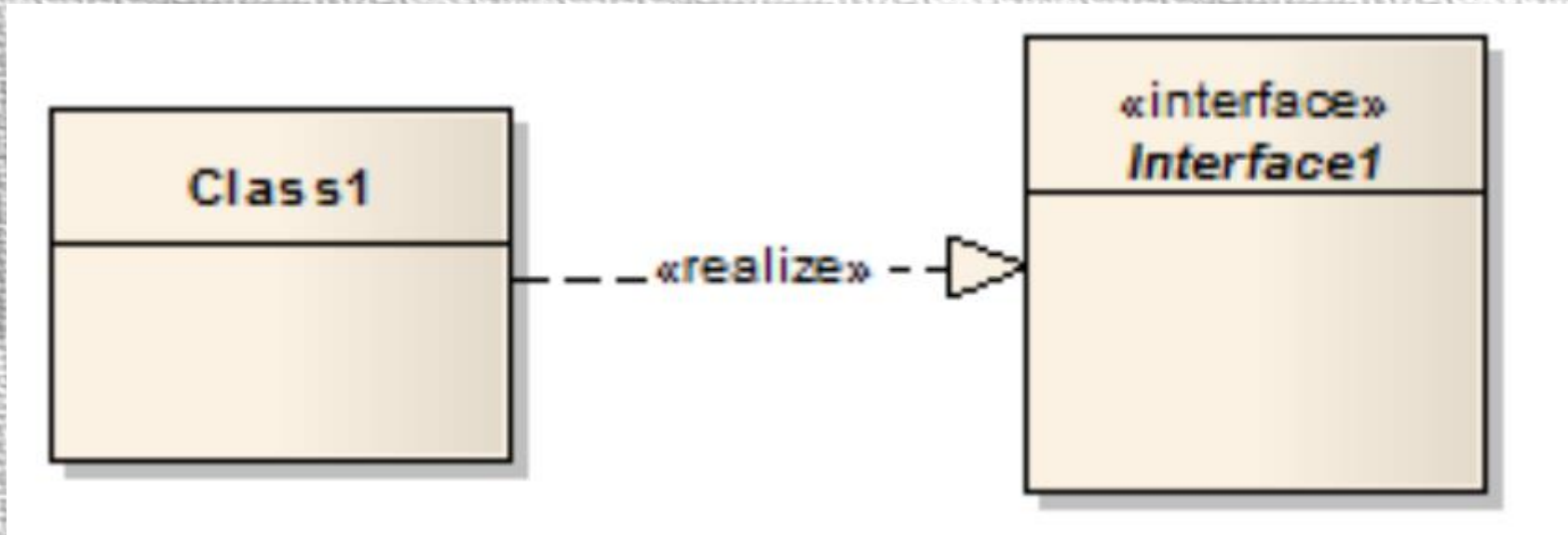
Примеры стереотипов для отношения зависимости

Стереотип	Описание
«bind»	Подстановка параметров в шаблон. Независимой сущностью является шаблон (класс с параметрами), а зависимой – класс, который получается из шаблона заданием аргументов.
«call»	Указывает зависимость между двумя операциями: операция зависимого класса вызывает операцию независимого класса.
«derive»	Буквально означает "может быть вычислен по". Зависимость с данным стереотипом применяется не только к классам, но и к другим элементам модели: атрибутам, ассоциациям и т.д. Суть состоит в том, зависимый элемент может быть восстановлен по информации, содержащейся в независимом элементе. Таким образом, данная зависимость показывает, что зависимый элемент, вообще говоря, излишен и введен в модель из соображений удобства, наглядности и т.д.
«friend»	Назначает специальные права видимости. Зависимый класс имеет доступ к составляющим независимого класса, даже если по общим правилам видимости такие права у него отсутствуют

Стереотип	Описание
«instanceOf»	Указывает, что зависимый объект (или класс) является экземпляром независимого класса (метакласса).
«instantiate»	Указывает, что операции независимого класса создают экземпляры зависимого класса.
«powertype»	Показывает, что экземплярами зависимого класса являются подклассы независимого класса. Таким образом, в данном случае зависимый класс является метаклассом.
«refine»	Указывает, что зависимый класс уточняет (конкретизирует) независимый. Данная зависимость показывает, что связанные классы концептуально совпадают, но находятся на разных уровнях абстракции.
«use»	Зависимость самого общего вида, показывающая, что зависимый класс каким-либо образом использует независимый класс.

РЕАЛИЗАЦИЯ

Реализация — отношение между двумя элементами модели, в котором один элемент (клиент) реализует поведение, заданное другим (поставщиком). **Реализация — отношение целое-часть**. Графически реализация представляется так же, как и наследование, но с пунктирной линией.



ОТНОШЕНИЕ РЕАЛИЗАЦИИ

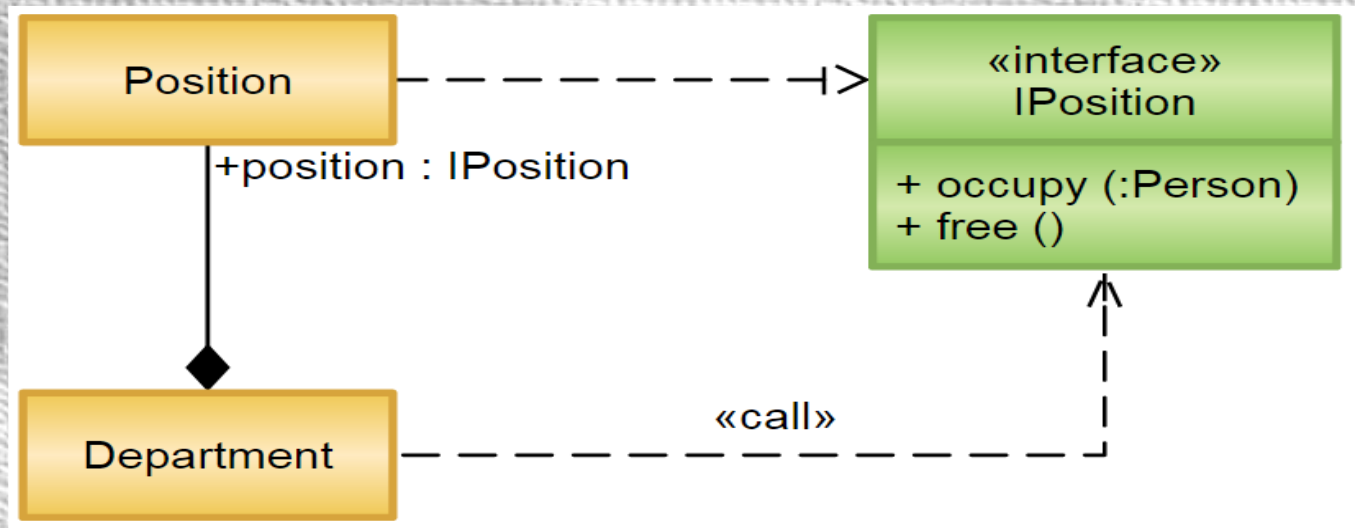
Между интерфейсами и другими классификаторами, в частности, классами, на диаграмме классов применяются два отношения:

1. классификатор (в частности, класс) использует интерфейс – это показывается с помощью зависимости со стереотипом «call»;
2. классификатор (в частности, класс) реализует интерфейс – это показывается с помощью отношения реализации.

Никаких ограничений на использование отношения реализации не накладывается: класс может реализовывать много интерфейсов, и наоборот, интерфейс может быть реализован многими классами. Нет ограничений и на использование зависимостей со стереотипом «call» – класс может вызывать любые операции любых видимых интерфейсов. Семантика зависимости со стереотипом «call» очень проста – эта зависимость указывает, что в операциях класса, находящегося на независимом полюсе, вызываются операции класса (в частности, интерфейса) находящегося на зависимом полюсе.

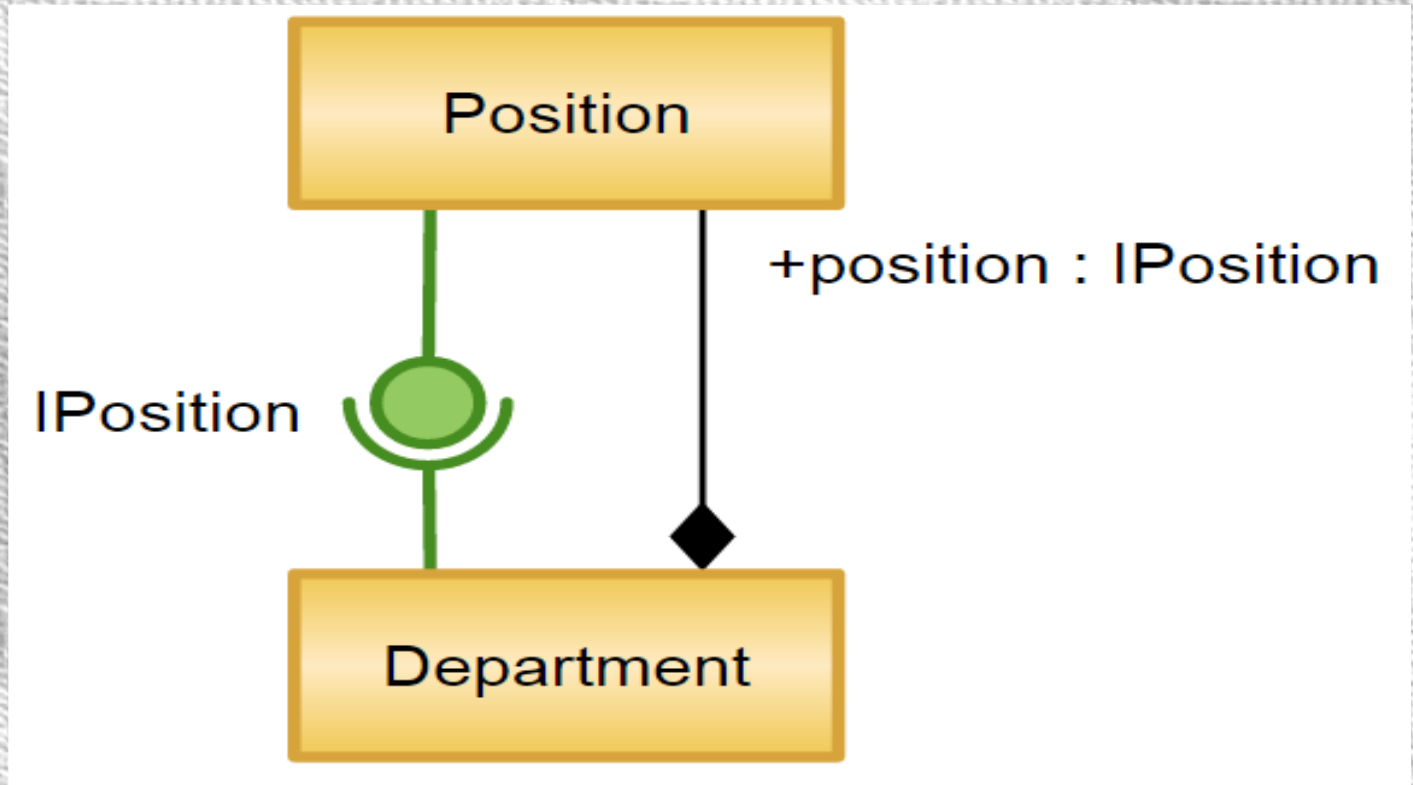
Отношения реализации и использования интерфейсов

Рассмотрим пример из информационной системы отдела кадров. Допустим, что класс **Department** для реализации операций связанных с движением кадров, использует операции класса **Position**, позволяющие занимать и освобождать должность – другие операции класса **Position** классу **Department** не нужны. Для этого, как показано на рисунке можно определить соответствующий интерфейс **IPosition** и связать его отношениями с данными классами.



Использование нотации "чупа-чупс"

Используя нотацию "чупа-чупс", появившуюся в UML 2, эту же модель можно изобразить лаконично, симметрично и просто, как показано ниже



Как отображается отношение «реализация» на ДК в код?

- На C#
- На Java
- На Python

Как отображается отношение «зависимость» на ДК в код?

- На C#
- На Java
- На Python

Композиция vs наследование

Повторное использование кода может быть достигнуто с помощью как наследования, так и композиции. Но при этом **композиция обеспечивает более высокий уровень инкапсуляции, чем наследование**, так как изменения в back-end классе не обязательно затронут код, который относится к front-end классу.

Композиция — это техника проектирования, применяющая в классах отношения типа “has-a” (имеет, включает в себя).

Для повторного использования кода могут применяться как наследование, так и композиция объекта.

Суть композиции заключается в выражении отношения «has a» между объектами. Подумайте о стуле. У стула есть (has a) сидение. У стула есть (has a) спинка. У стула есть (has a) определенное количество ножек. Фраза ”has a” (“есть”) предполагает отношения, в которых стул имеет или, как минимум, использует другой объект. Это как раз и есть отношения “has-a”, являющиеся основой композиции.

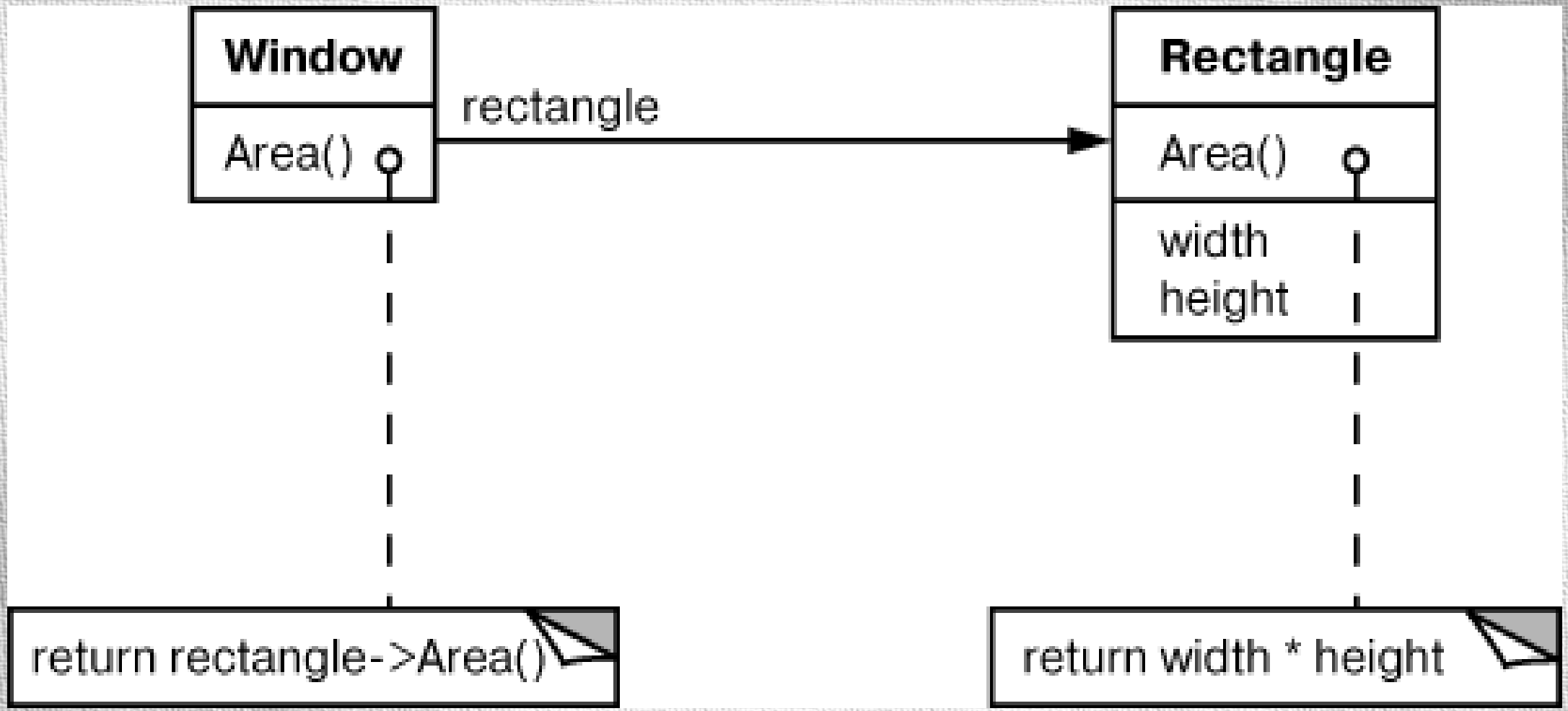
Преимущества:

- Контроль видимости
- Реализация может быть заменена во время выполнения (run-time)
- Слабая связанность, так как класс-интерфейс не зависит от реализации.

Примечание: Не используйте наследование только для того, чтобы обеспечить повторное использование кода. Если нет отношений “is a“ (является), для этих целей используется композиция.

Делегирование

С помощью делегирования композицию можно сделать столь же мощным инструментом повторного использования, сколь и наследование. При делегировании в процесс обработки запроса вовлечено два объекта: получатель поручает выполнение операций другому объекту – уполномоченному. Примерно так же подкласс делегирует ответственность своему родительскому классу. Но унаследованная операция всегда может обратиться к объекту-получателю через переменную-член (в C++) или переменную self (в Smalltalk). Чтобы достичь того же эффекта для делегирования, получатель передает указатель на самого себя соответствующему объекту, чтобы при выполнении делегированной операции последний мог обратиться к непосредственному адресату запроса.


Window
Area()
rectangle
Rectangle
Area()
width
height
return rectangle->Area()
return width * height

РАЗЛИЧИЯ МЕЖДУ КОМПОЗИЦИЕЙ И НАСЛЕДОВАНИЕМ

№	Композиция (has a / имеет)	Наследование (is a / является)
1	Поддерживает полиморфизм и повторное использование кода.	Поддерживает полиморфизм и повторное использование кода.
2	Объект во время выполнения (run-time) уже создан.	Объект создается динамически во время компиляции.
3	Реализация может быть заменена во время выполнения (run-time).	Реализация может быть заменена во время компиляции.
4	Подкласс не зависит от класса-родителя, что благоприятствует слабому связыванию (особенно под управлением интерфейса).	Подкласс зависит от реализации класса-родителя, поэтому связывание считается сильным.
5	Использование: в Доме есть Ванная комната. Неправильно говорить, что Дом — это Ванная комната.	Наследование является однонаправленным: Дом — это Здание. Но здание не является домом

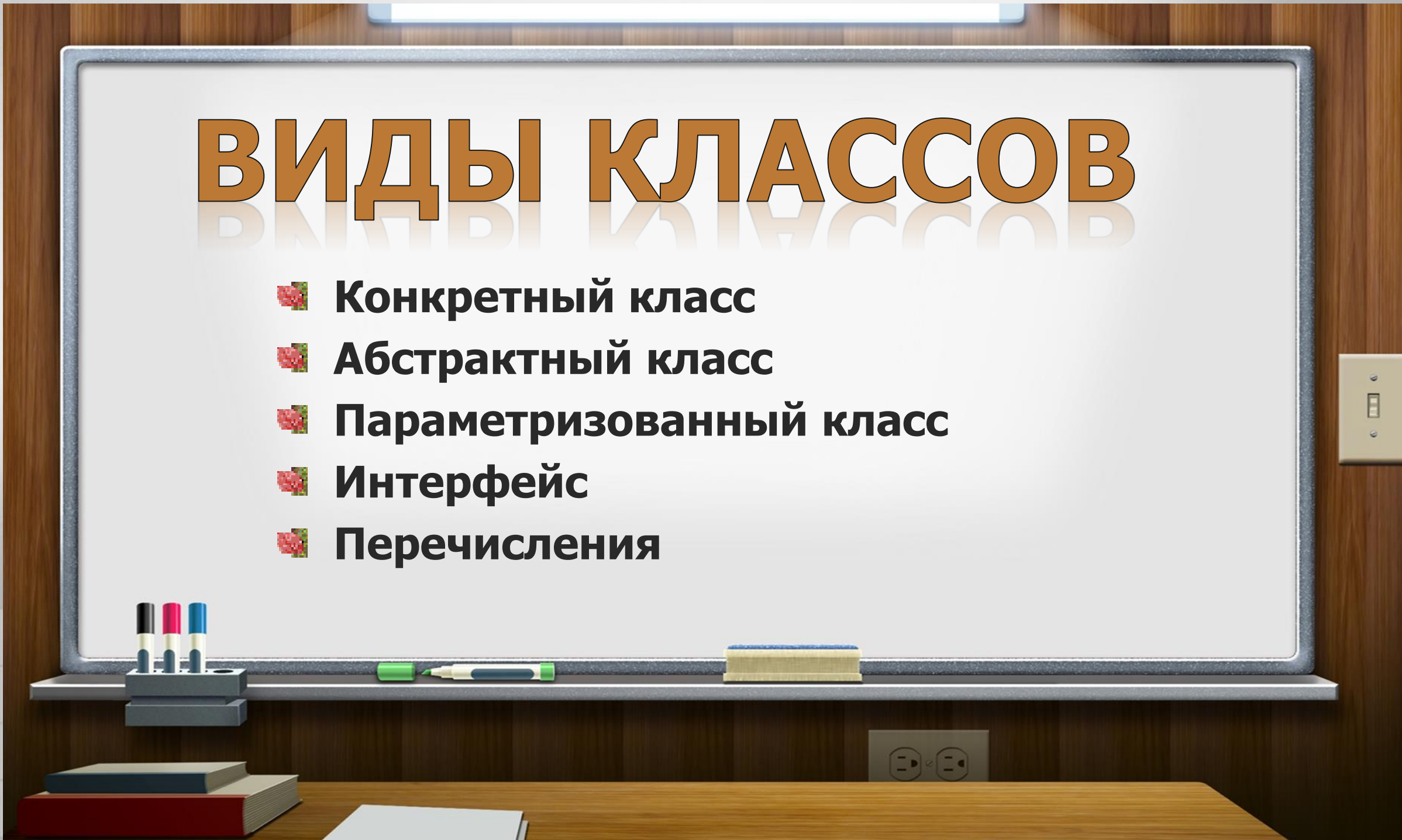
ПРЕИМУЩЕСТВА У КОМПОЗИЦИИ ПЕРЕД НАСЛЕДОВАНИЕМ

1. *Нет конфликта имён, возможного при наследовании*
2. *Возможность смены агрегируемого объекта в runtime.*
3. *Полная замена агрегируемого объекта в классах, производных от класса, включающего агрегируемый объект.*

В последних двух случаях очень желательно, чтобы сменяемые агрегируемые объекты имели общий интерфейс. А в третьем – чтобы метод, возвращающий такой объект, был виртуальным.

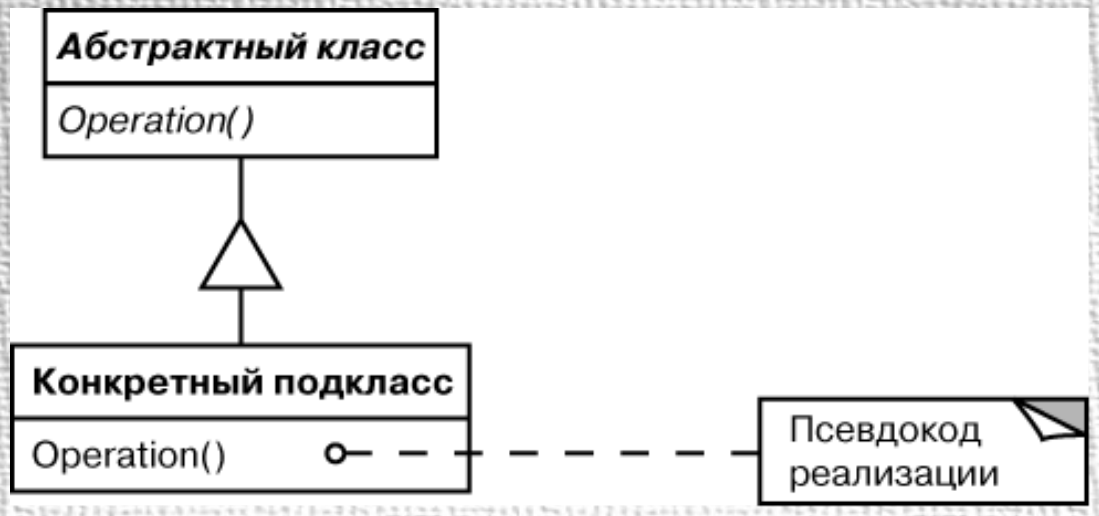
ВИДЫ КЛАССОВ

- Конкретный класс
- Абстрактный класс
- Параметризованный класс
- Интерфейс
- Перечисления



КОНКРЕТНЫЕ И АБСТРАКТНЫЕ КЛАССЫ

Класс может иметь или не иметь экземпляров или объектов. В зависимости от этого в языке UML различают ***конкретные*** и ***абстрактные*** *классы*.



КОНКРЕТНЫЙ КЛАСС

Конкретный класс (concrete class) — *класс*, на основе которого могут быть непосредственно созданы экземпляры или объекты.

Рассмотренные выше обозначения относятся к *конкретным классам*. От них следует отличать *абстрактные классы*.

Абстрактный метод

Методы, которые всего лишь определяет свои сигнатуры и не обеспечивает реализацию, называются «чистыми виртуальными» (англ. pure virtual) или абстрактными.

Фактическая его реализация оставлена его подклассам, которые расширяют абстрактный класс. Абстрактный метод не может быть использован объектом, только другой класс может его расширить.

АБСТРАКТНЫЙ КЛАСС

Класс, содержащий хотя бы один абстрактный метод, тоже будет абстрактным. Объект такого класса создать нельзя (в некоторых языках допускается, но вызов абстрактного метода приведёт к ошибке). Наследники абстрактного класса должны предоставить реализацию для всех его абстрактных методов, иначе они, в свою очередь, будут абстрактными классами.

Абстрактные классы реализуют на практике один из принципов ООП — полиморфизм.

Абстрактный класс можно рассматривать в качестве интерфейса к семейству классов, порождённому им, но, в отличие от классического интерфейса, абстрактный класс может иметь определённые методы, а также свойства.

Абстрактные методы всегда являются виртуальными, однако «абстрактный» и «виртуальный» - разные понятия.

Когда необходимо использовать абстрактный класс?

Абстрактные классы позволяют вам определить некоторое поведение по умолчанию и заставить подклассы обеспечить любое конкретное поведение.

Например: **List** (список) является интерфейсом, в свою очередь **AbstractList** определяет основное поведение **Списка**, которое может быть использовано как есть или уточнено в подклассе, например, в **ArrayList** (списочный массив).

ИНТЕРФЕЙС

Абстрактный класс, который содержит только абстрактные методы, называется **интерфейсом**.

Что такое интерфейс?

В концепции интерфейса лежит абстрактный класс, но интерфейс (определяется ключевым словом *interface*) шагнул дальше. Он предотвращает вообще любую реализацию метода или функции. Вы можете только объявлять метод или функцию, но не обеспечивать их реализацию. **Класс, который реализует данный интерфейс, должен как раз и позаботиться о фактической реализации**. Интерфейсы очень полезны и повсеместно используются в ООП. Так как они разделяют сам интерфейс и реализацию, они предоставляют много преимуществ своего использования.

Преимущества интерфейса

1. Множественное наследование.
2. Слабая связанность. Происходит абстракция операции, такая как разделение на уровни, а конкретной реализацией может быть что угодно.
3. Программа-интерфейс не реализуется.
4. Полиморфизм с динамическим связыванием: раскрывается программный интерфейс объекта без раскрытия его фактической реализации.
5. Абстрактные уровни, разделение функциональности.

Различие между классом объекта и его типом

Важно понимать различие между классом объекта и его типом. Класс объекта определяет, как объект реализован, то есть внутреннее состояние и реализацию операций объекта. Напротив, тип относится только к интерфейсу объекта - множеству запросов, на которые объект отвечает. У объекта может быть много типов, и объекты разных классов могут иметь один и тот же тип.

Различие между наследованием класса и наследованием интерфейса

Важно также понимать различие между наследованием класса и наследованием интерфейса (или порождением подтипов).

В случае наследования класса реализация объекта определяется в терминах реализации другого объекта. Проще говоря, это механизм разделения кода и представления. Напротив, наследование интерфейса (порождение подтипов) описывает, когда один объект можно использовать вместо другого.

Различие между интерфейсом и абстрактным классом

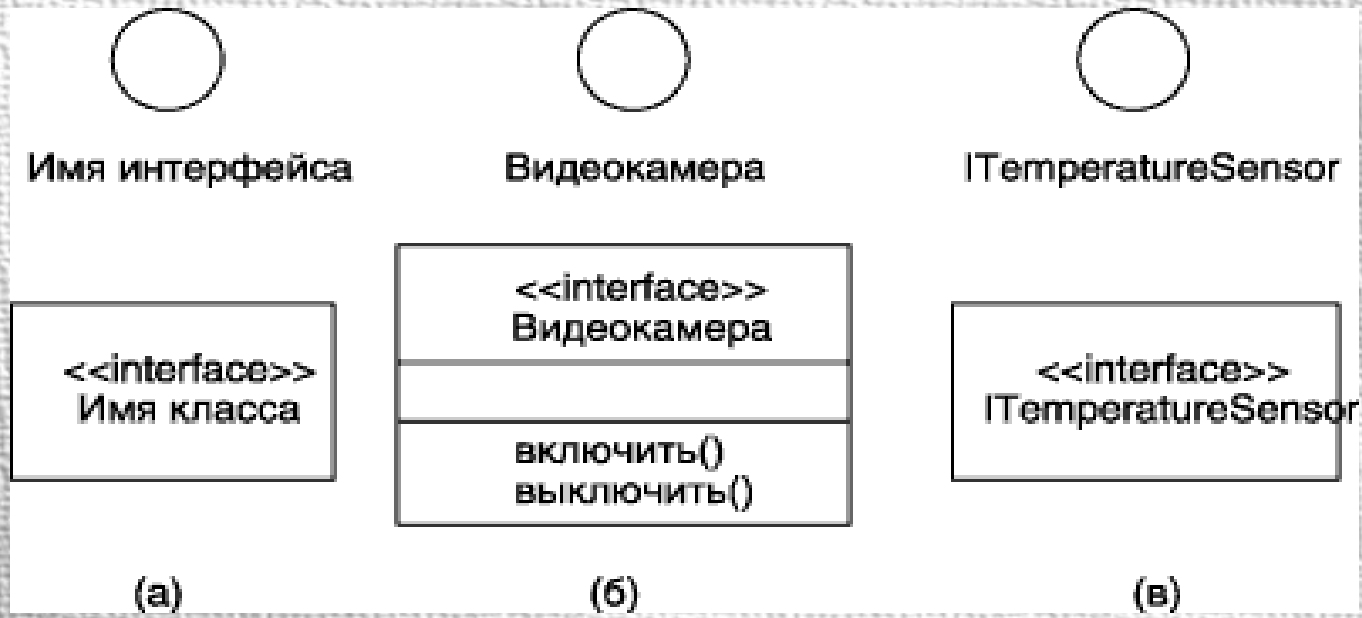
1. Интерфейс — это договорные отношения с классами, которые этот интерфейс реализуют, о том, что реализация происходит путём, обозначенным интерфейсом. Это пустая оболочка с объявленными методами.
2. Абстрактный класс определяет некоторое общее поведение и просит свои подклассы определить нетипичное или конкретное поведение для своего класса.
3. Методы и члены абстрактного класса могут быть обозначены любым модификатором доступа, в свою очередь все методы интерфейса обязаны быть открытыми (public).

4. Когда происходит наследование абстрактного класса, класс-наследник должен определить абстрактные методы, в то время как интерфейс может наследовать другой интерфейс и при этом не обязательно определять его методы.
5. Класс-наследник может расширять только один абстрактный класс, а интерфейс может расширять или класс может реализовывать множество других интерфейсов.
6. Класс-наследник может определять абстрактные методы с тем же или менее ограниченным модификатором доступа, при этом класс, реализующий интерфейс, должен определять методы с тем же уровнем видимости.
7. Интерфейс не содержит конструкторы, в том время, как они есть в абстрактном классе.

Главное отличие класса от интерфейса — в том, что класс состоит из интерфейса и реализации.

Для обозначения *интерфейса* используется специальный графический символ окружность или стандартный способ – прямоугольник *класса* со стереотипом <<**interface**>>

Примеры графического изображения интерфейсов на диаграммах классов



С учетом языка реализации модели имя *интерфейса*, как и *имена* других *классов*, рекомендуется записывать на английском и начинать с заглавной буквы I, например, ITemperatureSensor, IsecureInformation

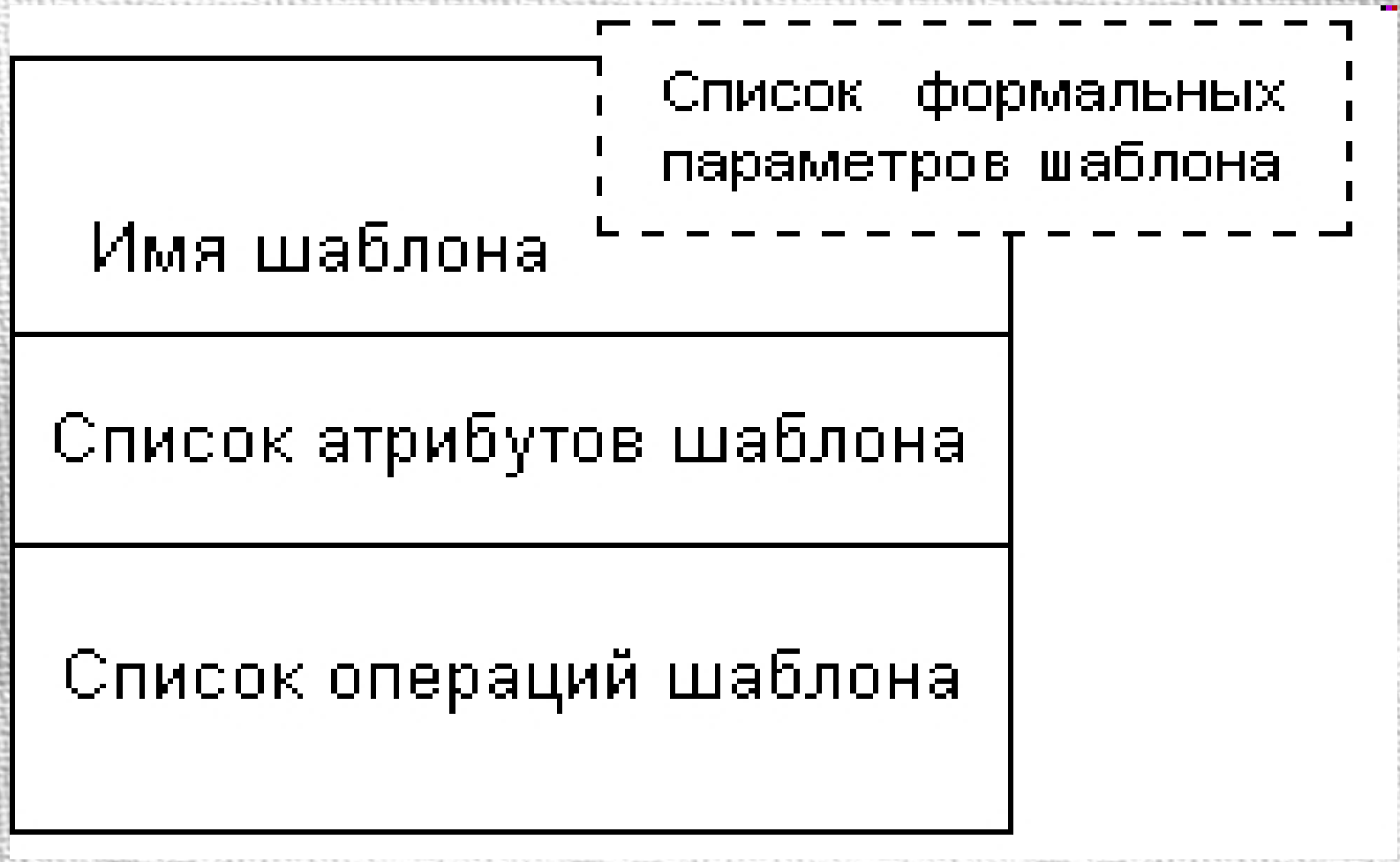
Подведем итог…

- Интерфейс = именованный набор абстрактных операций
- Класс реализует Интерфейс
- Класс использует Интерфейс
- Класс может реализовывать много интерфейсов (и наоборот)
- Роль = интерфейс, который предоставляет класс в данной ассоциации

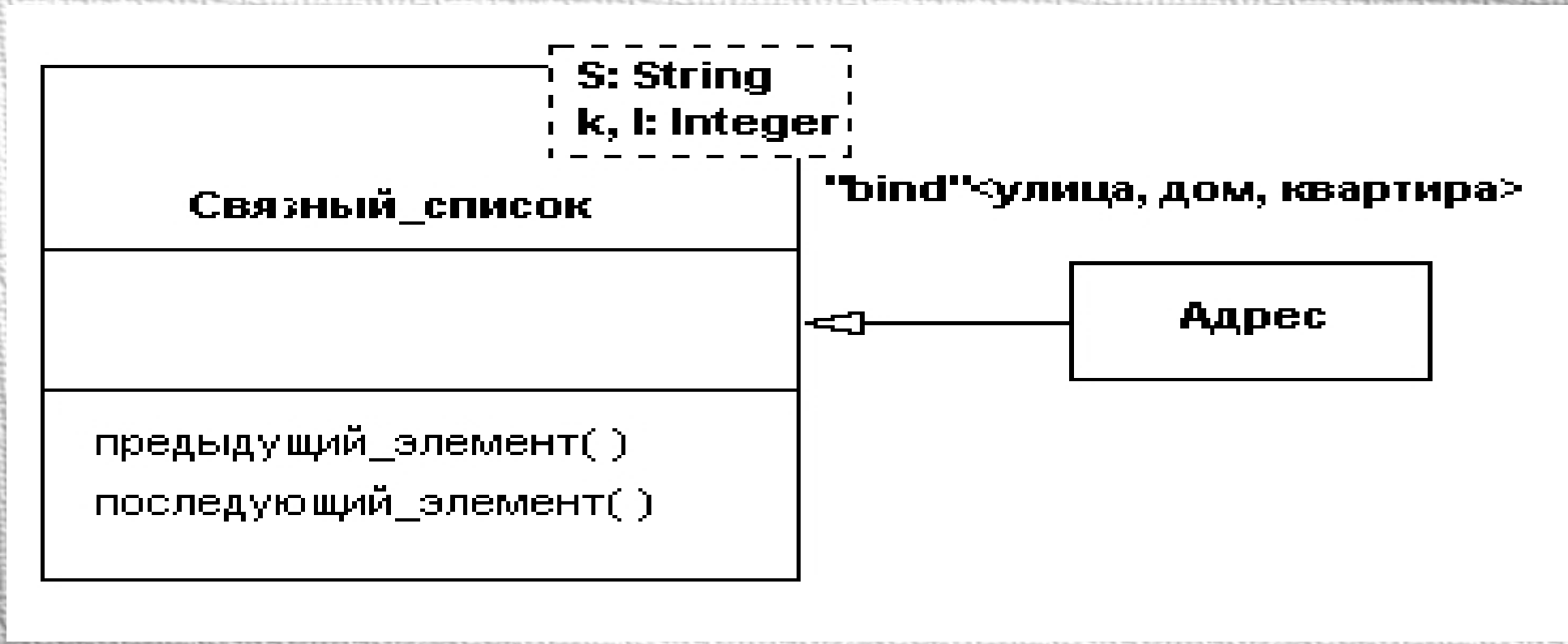
Шаблоны или параметризованные классы

Шаблон (template) или **параметризованный класс** (parametrized class) предназначен для обозначения такого класса, который имеет один (или более) нефиксированный формальный параметр. Он определяет целое семейство или множество классов, каждый из которых может быть получен связыванием этих параметров с действительными значениями. Обычно параметрами шаблонов служат типы атрибутов классов, такие как целые числа, перечисление, массив строк и др. В более сложном случае формальные параметры могут представлять и операции класса.

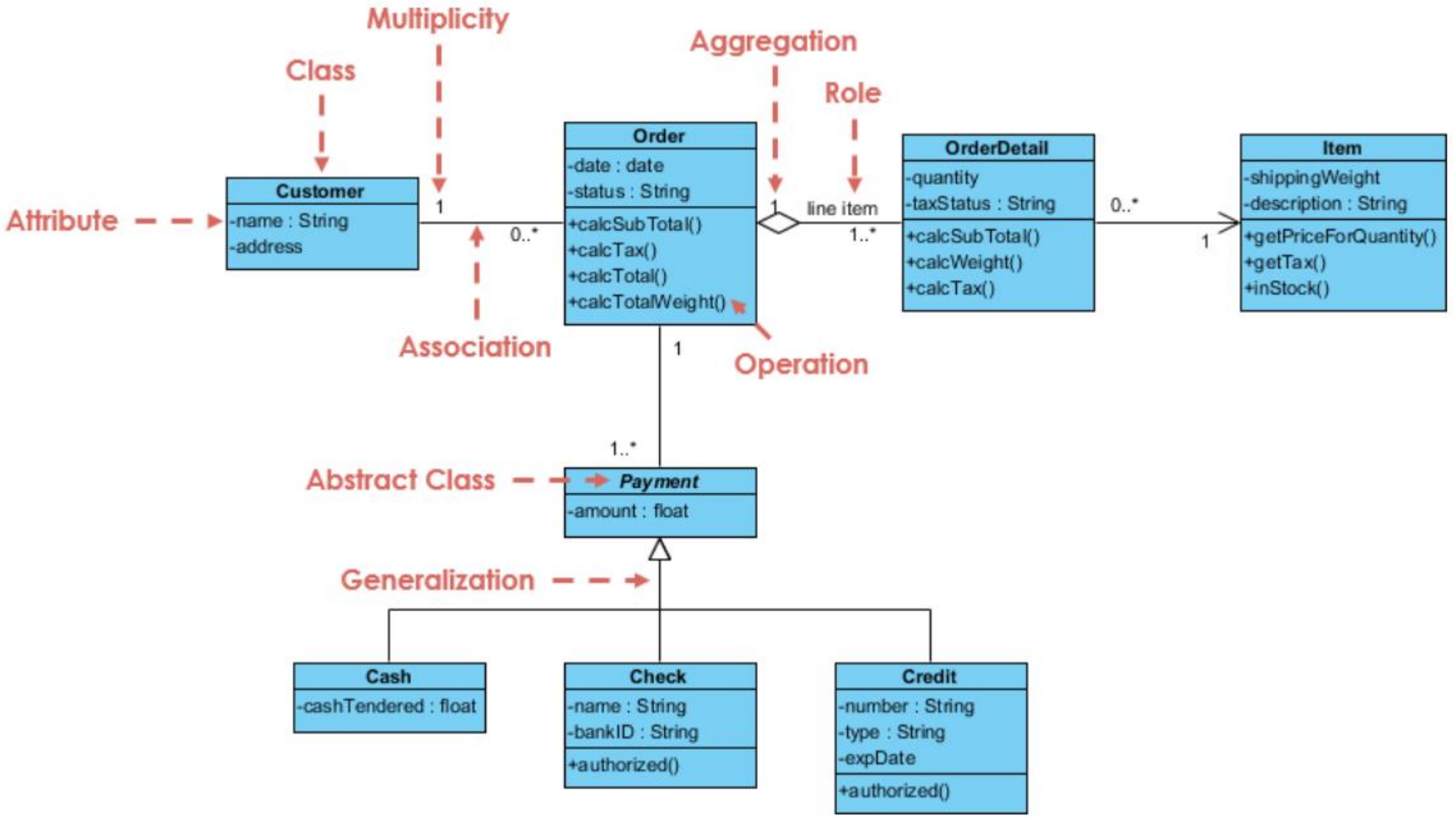
Графическое изображение шаблона на диаграмме классов



Пример использования шаблона на диаграмме классов



В данном примере отмечен тот факт, что класс "Адрес" может быть получен из шаблона Связный_список на основе актуализации формальных параметров "S, k, l" фактическими атрибутами "улица, дом, квартира".

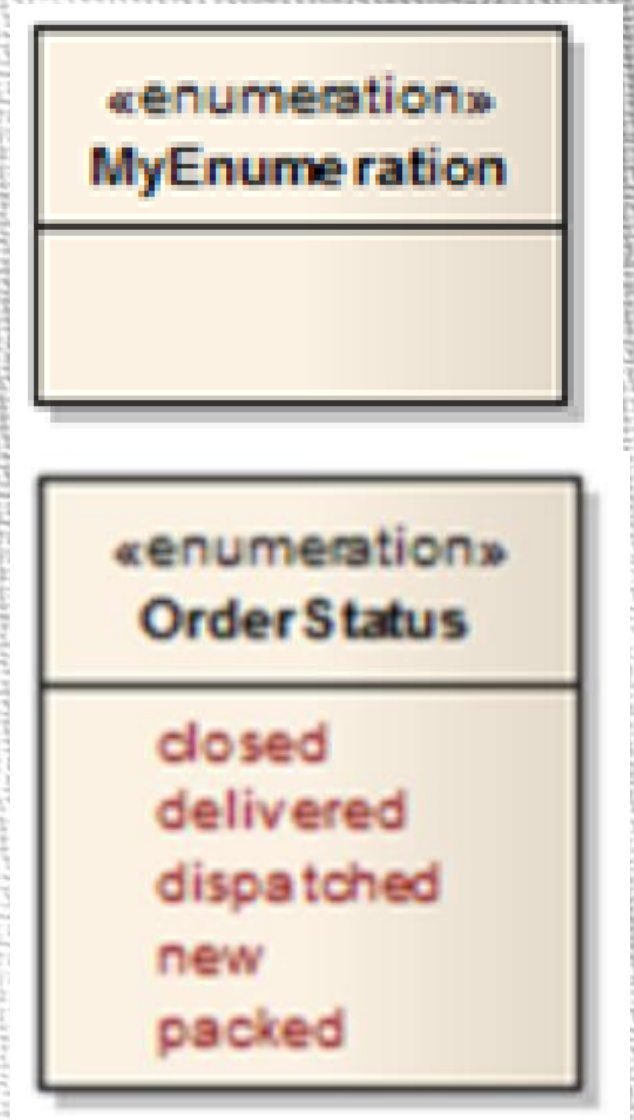

Multiplicity
Class
Aggregation
Role
Attribute
Customer
-name : String
-address
1
0..*
Association
Order
-date : date
-status : String
+calcSubTotal()
+calcTax()
+calcTotal()
+calcTotalWeight()
1
line item
1..*
Operation
OrderDetail
-quantity
-taxStatus : String
+calcSubTotal()
+calcWeight()
+calcTax()
0..*
1
Item
-shippingWeight
-description : String
+getPriceForQuantity()
+getTax()
+inStock()
1
1..*
Abstract Class
Payment
-amount : float
Generalization
Cash
-cashTendered : float
Check
-name : String
-bankID : String
+authorized()
Credit
-number : String
-type : String
-expDate
+authorized()

Перечисления

Кроме классов и интерфейсов на диаграмме классов также могут помещаться перечисления.

Перечисление обозначается так:

Обычно перечисление указывается с перечнем возможных значений, например:



РАСШИРЕНИЕ ЯЗЫКА UML

Механизмы расширения UML позволяют ввести в рассмотрение дополнительные графические обозначения, ориентированные для решения задач из определенной предметной области.
Язык UML содержит два специальных расширения:

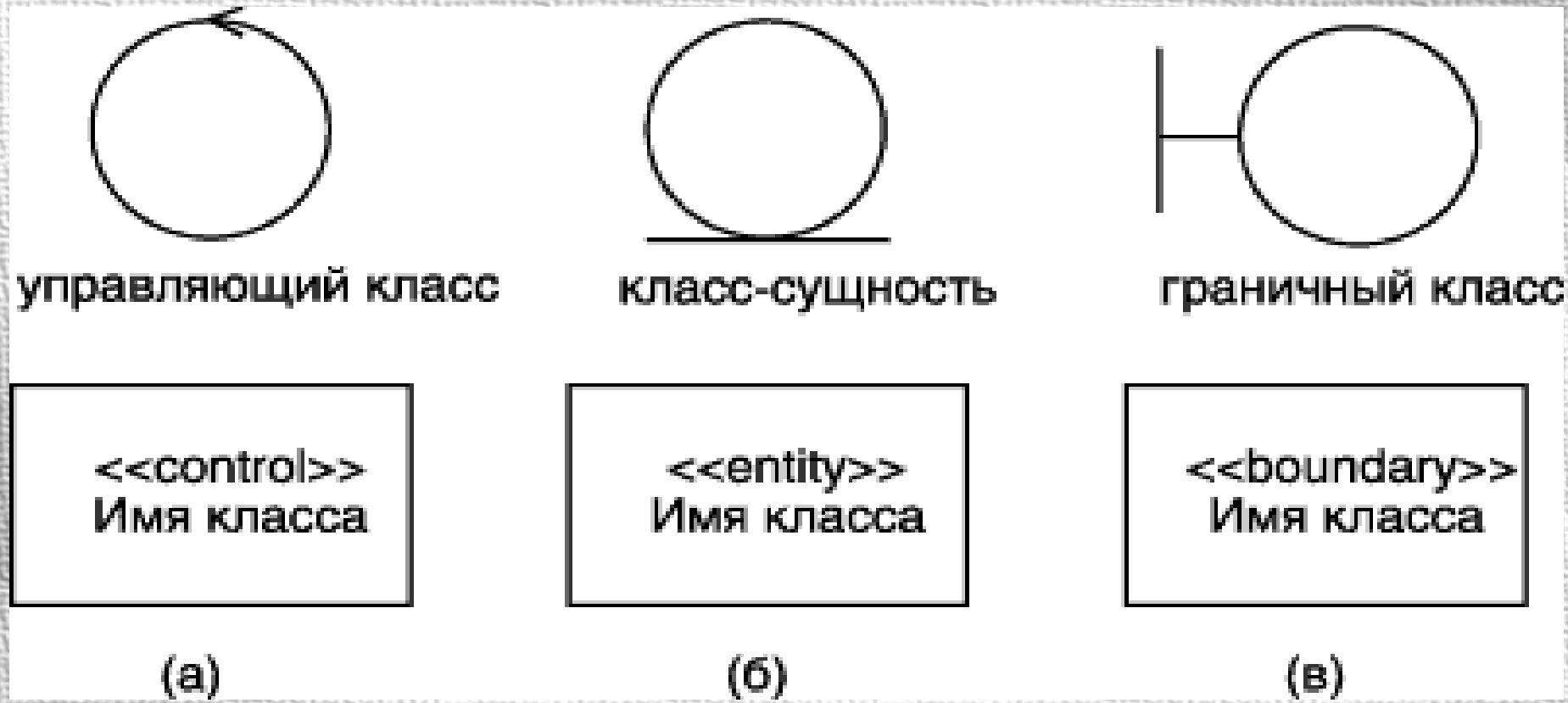
1. профиль для процесса разработки программного обеспечения (The UML Profile for Software Development Processes);
2. профиль для бизнес-моделирования (The UML Profile for Business Modeling).

Графические примитивы классов для моделирования программного обеспечения

В рамках первого из них предложено три специальных графических примитива, которые могут быть использованы для уточнения семантики отдельных *классов* при построении различных диаграмм:

- **Управляющий класс** (control class)
- **Класс-сущность** (entity class)
- **Граничный класс** (boundary class)

Графическое изображение классов



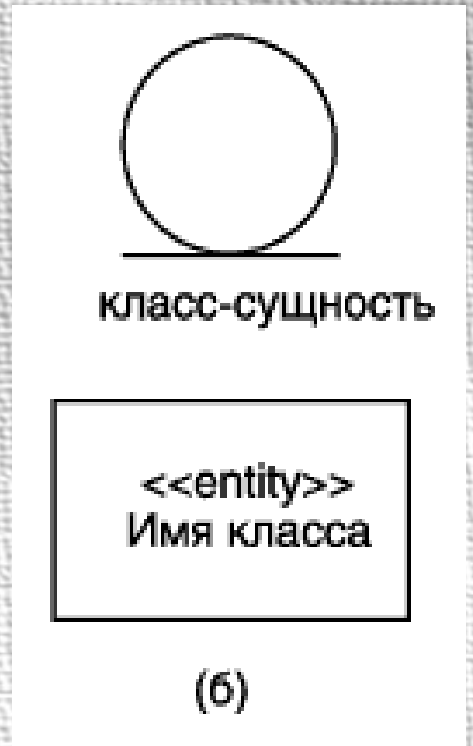
Управляющий класс

***Управляющий* класс** (control class) — *класс*, отвечающий за координацию действий других *классов*. На каждой диаграмме *классов* должен быть хотя бы один управляющий *класс*, причем количество посылаемых объектам управляющего *класса* сообщений мало, по сравнению с числом рассылаемых ими. Управляющий *класс* отвечает за координацию действий других *классов*. У каждой диаграммы *классов* должен быть хотя бы один управлящий *класс*, контролирующий последовательность выполнения действий этого варианта использования. Как правило, данный *класс* является активным и инициирует рассылку множества сообщений другим *классам* модели. Кроме специального обозначения управляющий *класс* может быть изображен в форме прямоугольника *класса* со стереотипом <<control>>

Класс-сущность

Класс-*сущность* (entity class) — пассивный *класс*, информация о котором должна храниться постоянно и не уничтожаться с выключением системы. *Класс*-сущность содержит информацию, которая должна храниться постоянно и не уничтожается с уничтожением объектов данного *класса* или прекращением работы моделируемой системы, связанные с выключением системы или завершением программы. Как правило, этот *класс* соответствует отдельной таблице базы данных. В этом случае его *атрибуты* являются полями таблицы, а *операции* – хранимыми процедурами.

Этот *класс* пассивный и лишь принимает сообщения от других *классов* модели. *Класс*-сущность может быть изображен также стандартным образом в форме прямоугольника *класса* со стереотипом <<**entity**>>

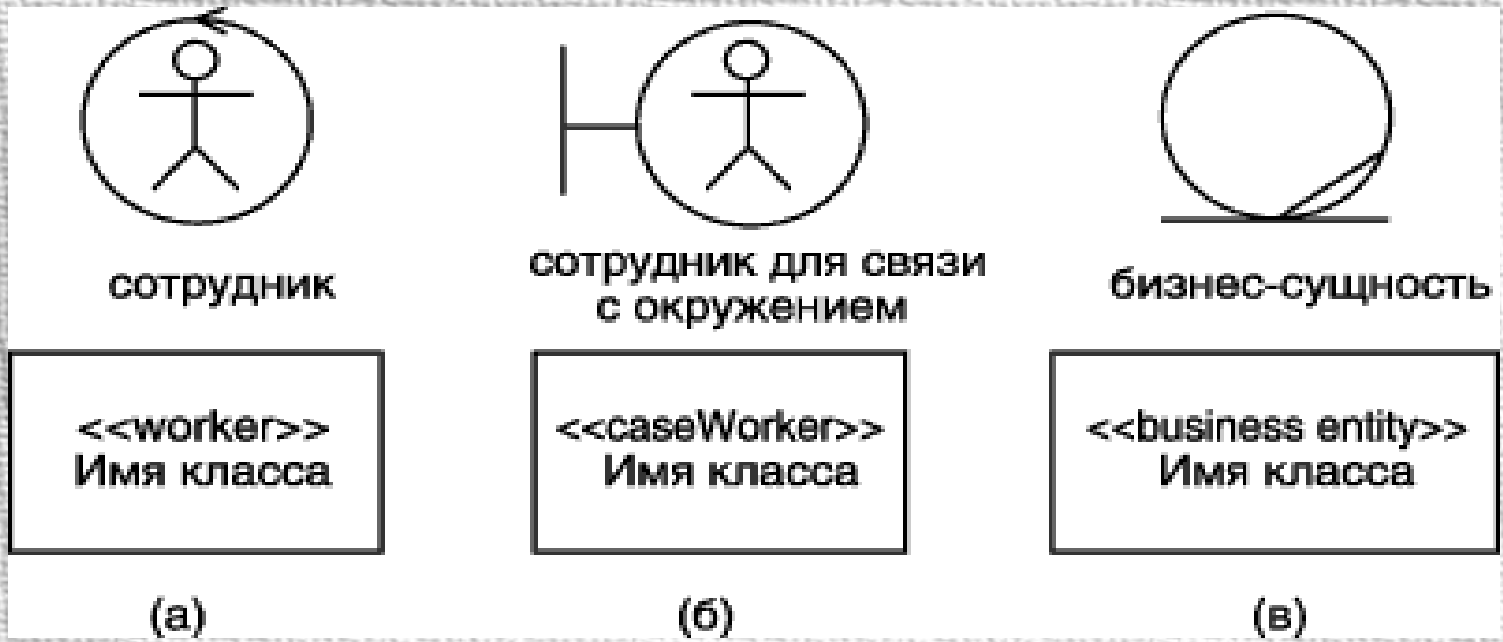


Граничный класс

***Граничный* класс** (boundary class) — *класс*, который располагается на границе системы с внешней средой и непосредственно взаимодействует с актерами, но является составной частью системы. Граничный *класс* может быть изображен также стандартным образом в форме прямоугольника *класса* со стереотипом <<**boundary**>>

Специальные графические примитивы ДК при построении моделей бизнес-систем

В рамках второго профиля также предложено три специальных графических примитива, которые могут быть использованы для уточнения семантики отдельных *классов* при построении моделей бизнес-систем:



Сотрудник

Сотрудник (business worker) — класс, служащий на диаграмме классов для представления любого сотрудника, который является элементом бизнес-системы и взаимодействует с другими сотрудниками при реализации бизнес-процесса. Этот класс также может быть изображен в форме прямоугольника класса со стереотипом <<**worker**>> или <<**internalWorker**>>

Сотрудник для связи с окружением (caseworker) – *класс*, служащий для представления в бизнес-системе такого сотрудника, который, являясь элементом бизнес-системы, непосредственно взаимодействует с актерами (бизнес-актерами) при реализации бизнес-процесса. Этот *класс* также может быть изображен в форме прямоугольника *класса* со стереотипом <<**caseWorker**>>

Бизнес-сущность

Бизнес-сущность (business entity) — специальный случай *класса*-сущности, который также не инициирует никаких сообщений. Этот *класс* служит для сохранения информации о результатах выполнения бизнес-процесса в моделируемой бизнес-системе или организации. Этот *класс* также может быть изображен в форме прямоугольника *класса* со стереотипом <<**business entity**>>

Куда ж нам плыть?

(Осень, А.С. Пушкин)

После разработки диаграммы классов процесс ООАП может быть продолжен в двух направлениях.

1. С одной стороны, если поведение системы тривиально, то можно приступить к разработке диаграмм кооперации и последовательности.
2. Однако для сложных динамических систем, в частности для параллельных систем и систем реального времени, важнейшим аспектом их функционирования является поведение. Специфика поведения таких систем может быть представлена на диаграммах состояний и деятельности, разработка которых в общем случае не обязательна, а определяется конкретным проектом.

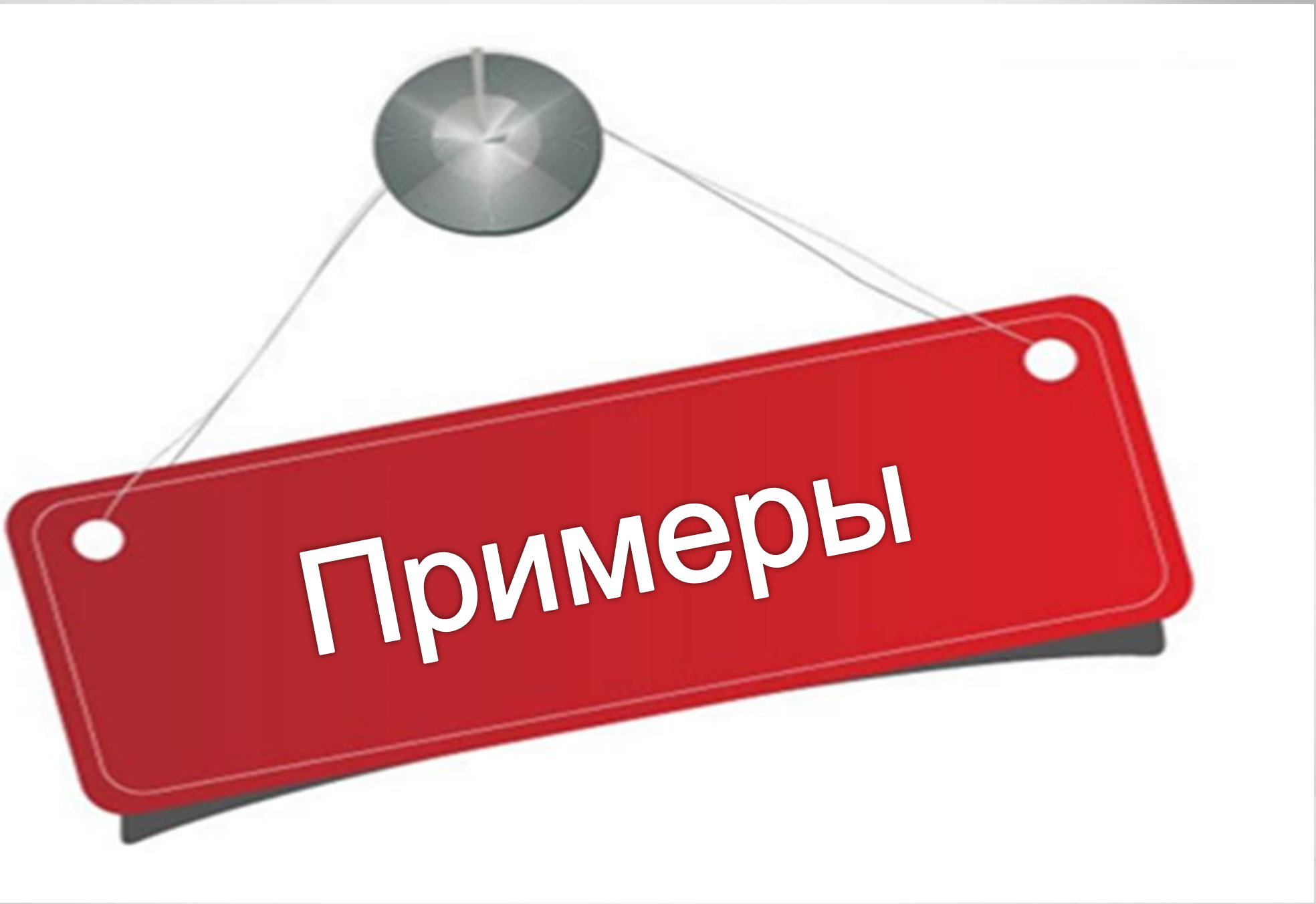
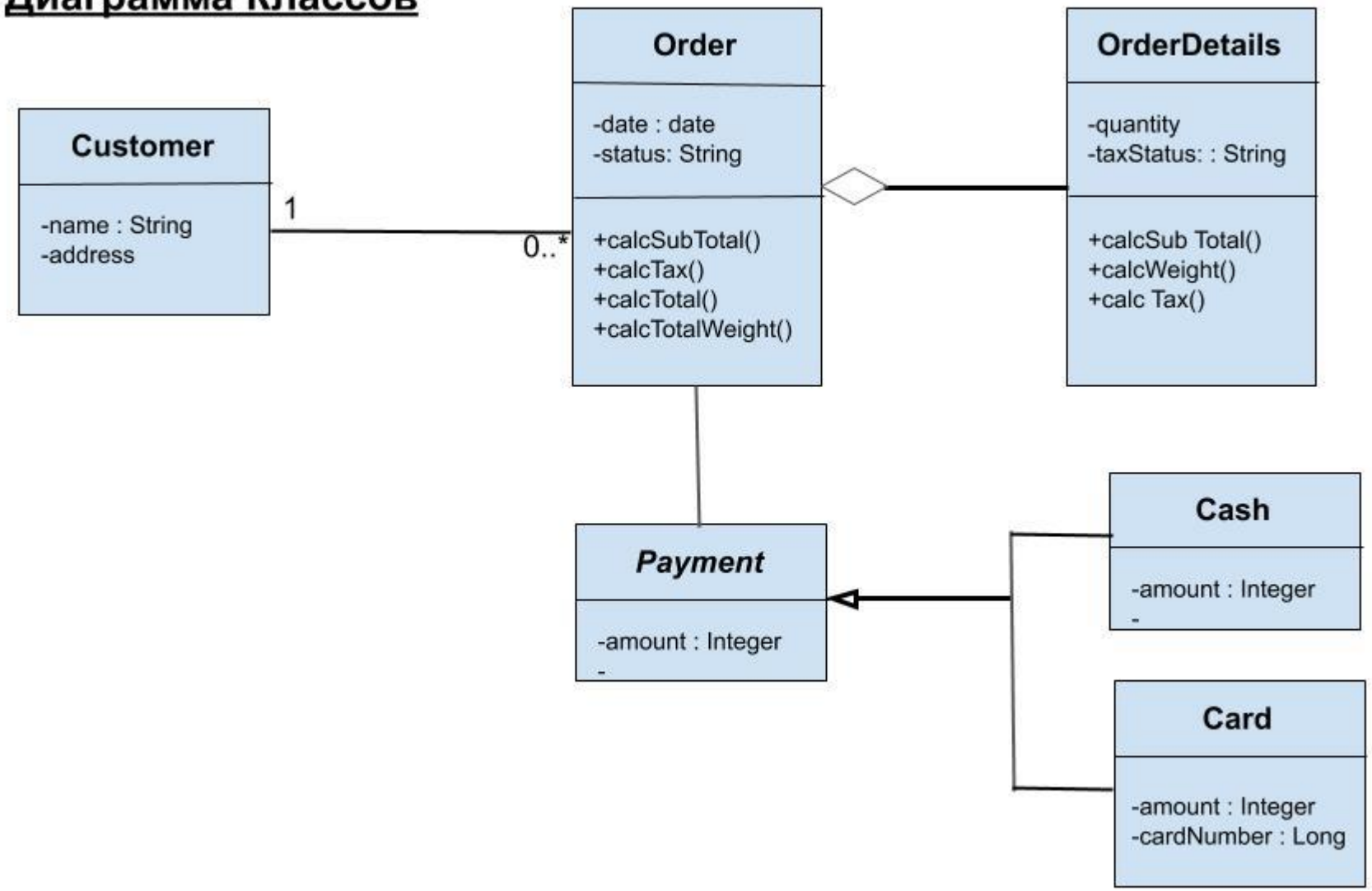

Примеры

Диаграмма Классов



- Графическое изображение
- Отношения

ДИАГРАММА ПАКЕТОВ (PACKAGE DIAGRAM)

Пакеты – это **способ группировки любых элементов языка UML** – диаграмм, вариантов использования, классов и пр.

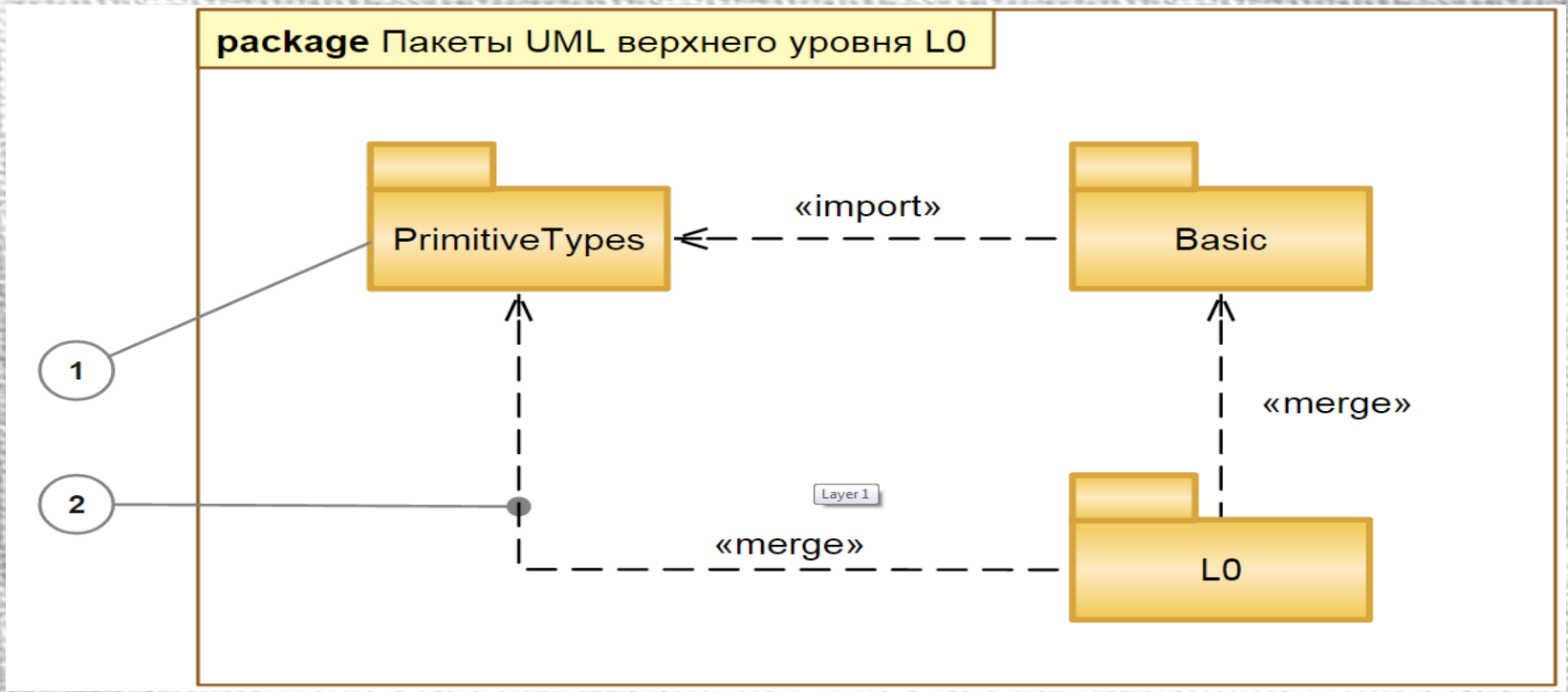
Обычно пакеты представляют собой:

- **подсистемы, если речь идет о системе**
- **компоненты или слои, если речь идет о классах**
- **наборы бизнес-логики, если речь идет о вариантах использования и т.п.**

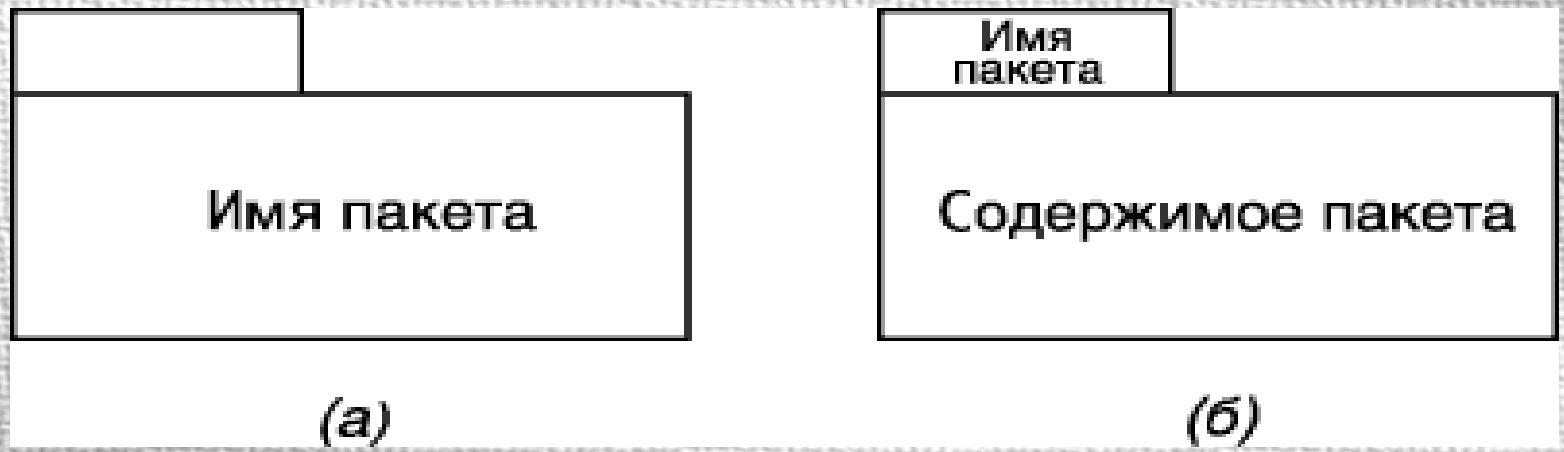
Конкретный способ группировки элементов в пакеты остается на усмотрение проектировщика.

Пакеты могут включать в себя не только объекты, но и диаграммы различных типов. Т.е. в каждом пакете могут быть свои диаграммы вариантов использования, последовательностей, классов и пр.

Диаграмма пакетов – единственное средство, позволяющее управлять сложностью самой модели. Основные элементы нотации – пакеты ❶ и зависимости с различными стереотипами ❷, применяемые на диаграмме, показаны на следующем рисунке.



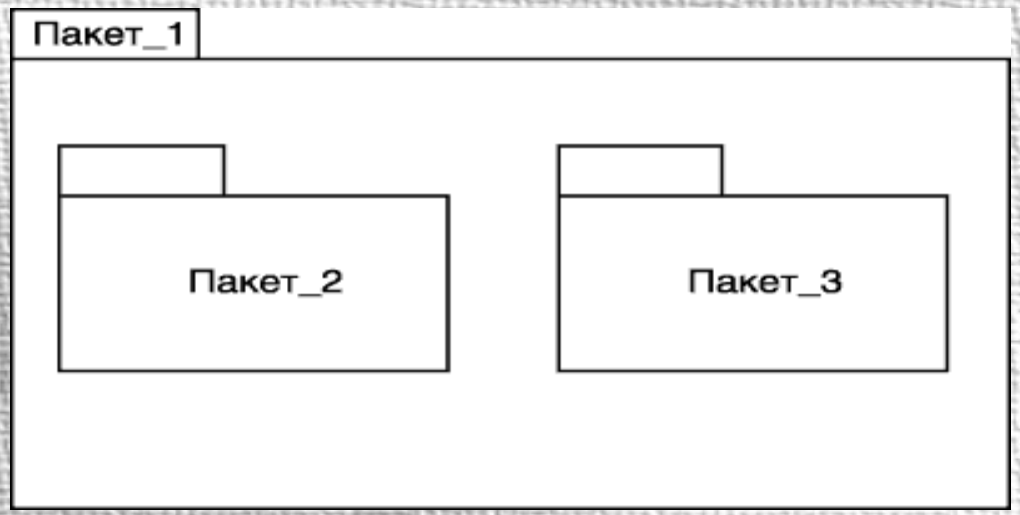
Графическое изображение пакетов в языке UML

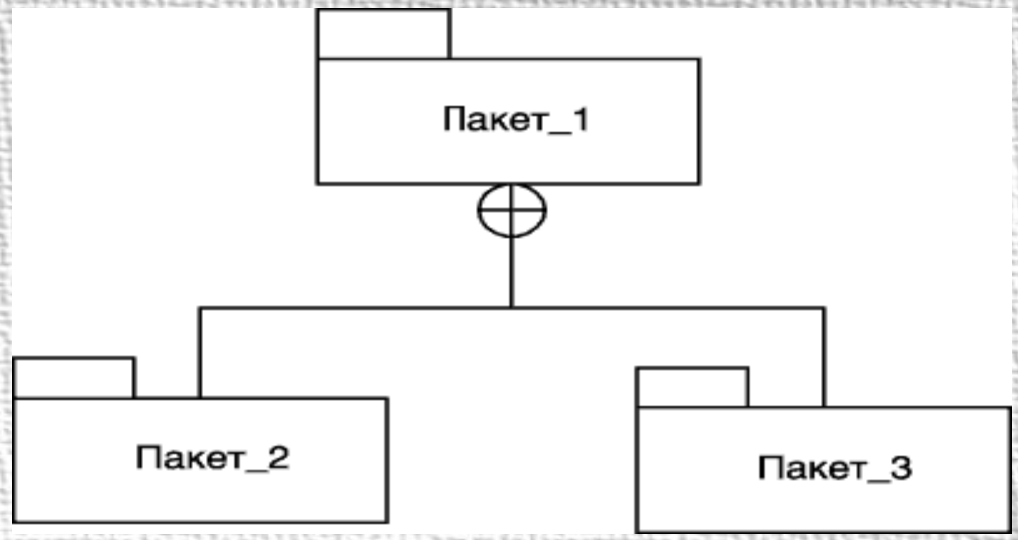


Перед именем *пакета* может помещаться строка текста, содержащая ключевое слово, заранее определенное в языке UML, и называемое *стереотипом*, например facade, framework, stub и topLevel. В качестве содержимого *пакета* могут выступать имена его отдельных элементов и их свойства, такие как видимость элементов за пределами *пакета*.

Отношения между пакетами

Отношение вложенности или включения *пакетов* друг в друга.





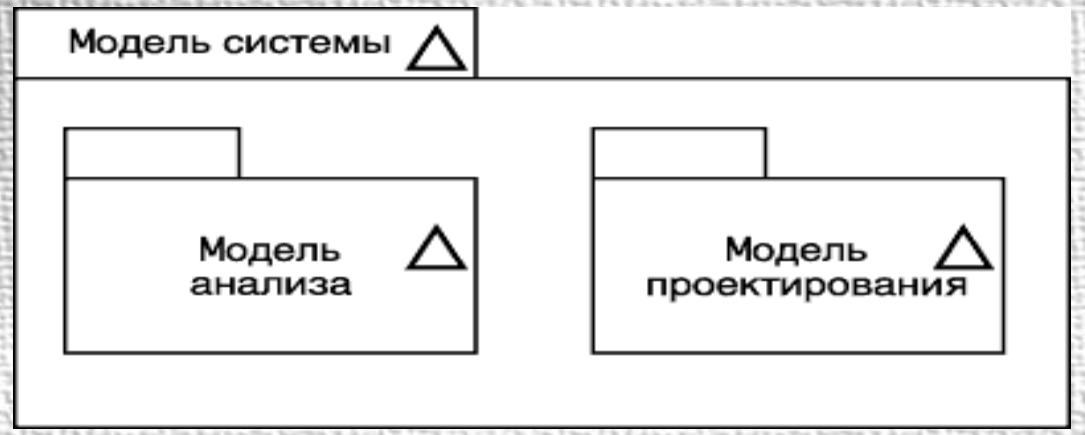
В основном пакеты служат для объединения классов в группы, и именно этот способ их применения здесь описывается, но помните, что пакеты могут применяться для любой другой конструкции языка UML.

В дополнение к стандартным отношениям зависимостей в UML есть два специальных вида зависимостей, определенных между пакетами:

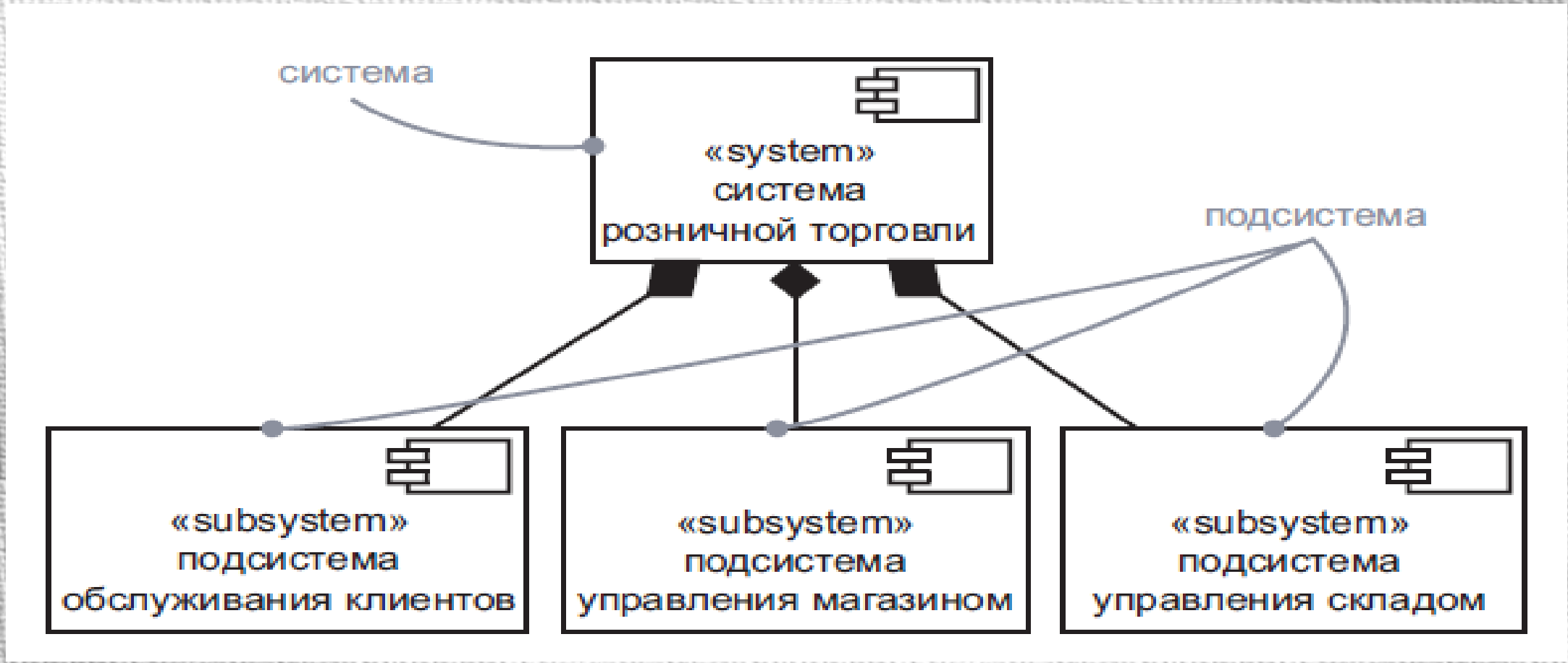
- Импортирование пакета
- Слияние пакета

Модели и пакеты

В языке UML для одной и той же физической системы могут быть определены различные модели, каждая из которых описывает систему с различных точек зрения. Модели, как и пакеты, могут быть вложены друг в друга. Пакет может включать в себя несколько различных моделей одной и той же системы. Общая модель системы в контексте языка UML содержит в себе модель анализа и модель проектирования, что явно отражает связь с ООАП.



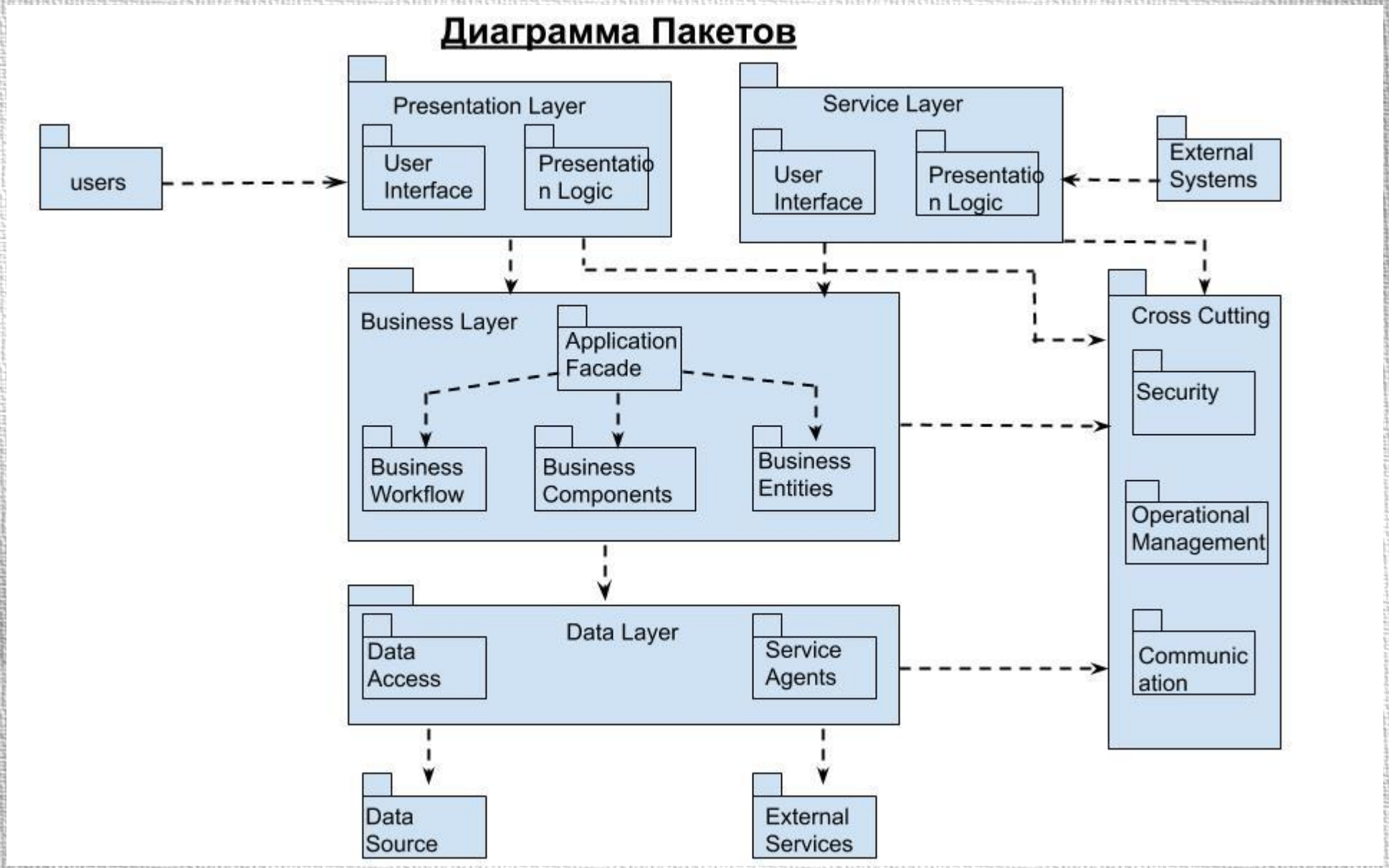
Системы и подсистемы



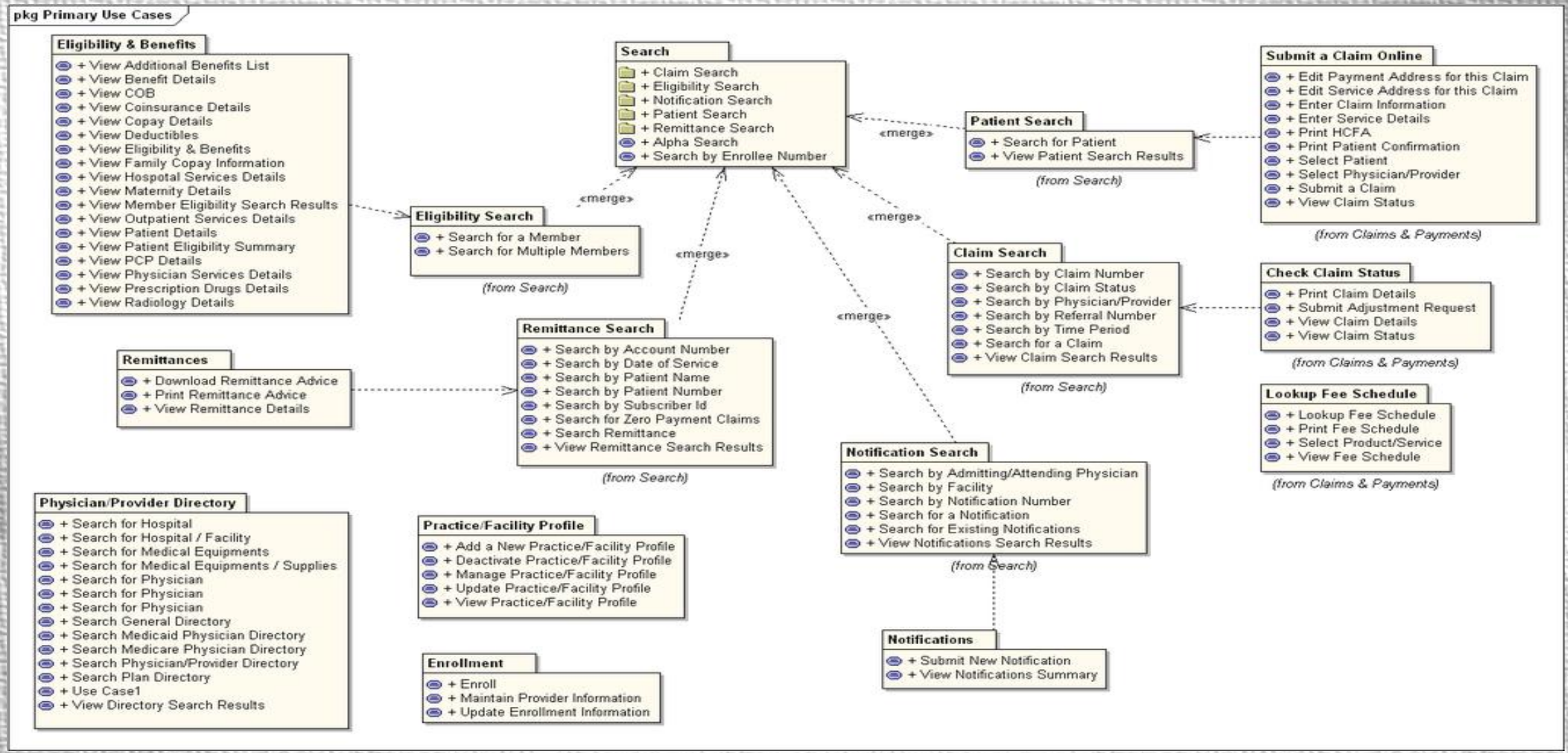
Подсистема

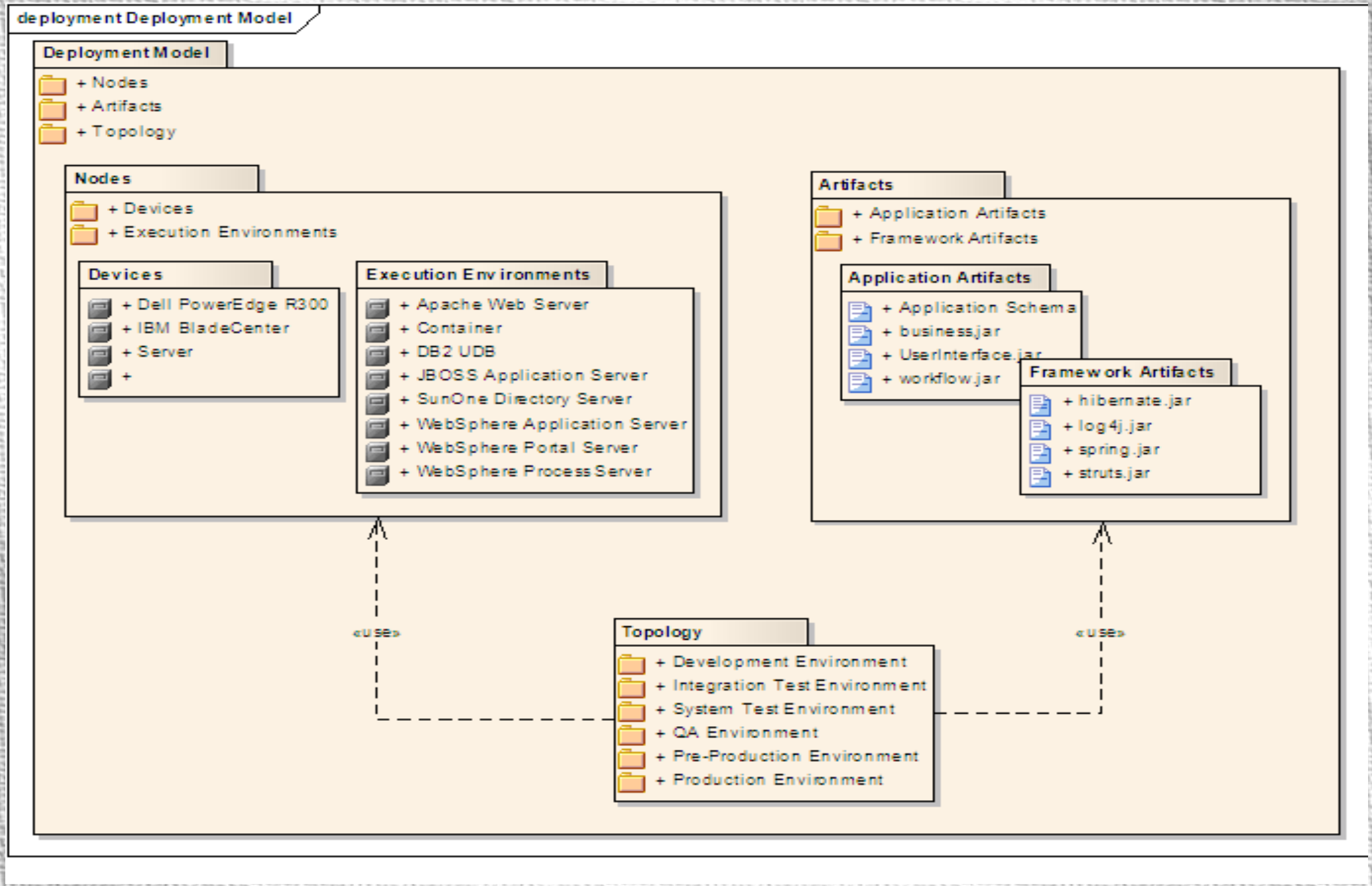
Для графического представления подсистемы применяется специальное обозначение – прямоугольник, как в случае *пакета*, но дополнительно разделенный на три секции.

Диаграмма Пакетов



Пакет содержит use case




deployment Deployment Model
Deployment Model
+ Nodes
+ Artifacts
+ Topology
Nodes
+ Devices
+ Execution Environments
Devices
+ Dell PowerEdge R300
+ IBM BladeCenter
+ Server
+
Execution Environments
+ Apache Web Server
+ Container
+ DB2 UDB
+ JBOSS Application Server
+ SunOne Directory Server
+ WebSphere Application Server
+ WebSphere Portal Server
+ WebSphere Process Server
Artifacts
+ Application Artifacts
+ Framework Artifacts
Application Artifacts
+ Application Schema
+ businessjar
+ UserInterface.jar
+ workflow.jar
Framework Artifacts
+ hibernate.jar
+ log4j.jar
+ spring.jar
+ struts.jar
«use»
«use»
Topology
+ Development Environment
+ Integration Test Environment
+ System Test Environment
+ QA Environment
+ Pre-Production Environment
+ Production Environment

МОДЕЛИРОВАНИЕ АРХИТЕКТУРЫ

- Диаграммы реализации
  - Диаграмма Компонентов
  - Диаграмма Развертывания
- Кооперация
- Паттерны и каркасы

- Диаграмма компонентов (component diagram)
- Диаграмма развертывания (deployment diagram)

ДИАГРАММЫ РЕАЛИЗАЦИИ (IMPLEMENTATION DIAGRAMS)

Диаграммы реализации

Данный раздел посвящен сразу двум диаграммам:

1. **компонентов и**
2. **размещения,**

для которых можно использовать обобщающее название – **диаграммы реализации**. Связано это с тем, что данные диаграммы приобретают особую важность на позднейших фазах разработки – на фазах реализации и поставки. В то время как на ранних фазах разработки – анализа и проектирования – эти диаграммы либо вообще не используются, либо имеют самый общий, не детализированный вид.

С точки зрения реализации проектируемая система состоит из **компонентов** (представленных на диаграммах компонентов), распределенных по **вычислительным узлам** (представленным на диаграммах размещения).

КОМПОНЕНТЫ И АРТЕФАКТЫ

В UML 2 по сравнению с UML 1 произошло значительное изменение, а именно, понятие "компонент" было разделено на две составляющие:

1. **логическую**
2. **физическую**

Логическая составляющая, продолжающая носить имя **компонент** (component), является элементом логической модели системы, в то время как физическая составляющая, называемая **артефактом** (artifact), олицетворяет физический элемент проектируемой системы, размещающийся на **вычислительном узле** (node).

Диаграммы компонентов и размещения имеют много общего, объединяя воедино следующие, теснейшим образом связанные, вещи:

- структуру логических элементов (компонентов);
- отображения логических элементов (компонентов) на физические элементы (артефакты);
- структуру используемых ресурсов (узлов) с распределенными по ним физическими элементами (артефактами).

- Компонент
- Нотация
- Интерфейсы
- Порты

ДИАГРАММА КОМПОНЕНТОВ (COMPONENT DIAGRAM)

Компонент

ЧТО ТАКОЕ КОМПОНЕНТ

Спецификация UML 2.0 [UML2S] гласит: «Компонент представляет модульную часть системы, которая инкапсулирует ее содержимое, и реализация компонента **замещаема** в рамках его окружения».
Компонент как черный ящик, внешнее поведение которого полностью определяется его **предоставляемыми и требуемыми интерфейсами**.
Поэтому один компонент может быть заменен другим, поддерживающим тот же протокол.
Компоненты могут иметь атрибуты и операции и участвовать в отношениях ассоциации и обобщения. Компоненты – это *структурированные классификаторы*. У них может быть внутренняя структура, включающая части и соединители.

Взаимодействие структурированного классификатора с его окружением моделируется его **интерфейсами** и **портами**.

Компоненты могут представлять что-то, экземпляр чего может быть создан во время выполнения. Или ими может быть представлена абсолютно логическая конструкция, такая как подсистема, экземпляры которой создаются только косвенно через создание экземпляров ее частей. Компонент может быть представлен одним или более артефактами.

Артефакт представляет некоторую физическую сущность, например исходный файл.

Итак …

Следует различать логические и физические элементы системы.

- **Логические элементы – это компоненты (**design time**).**
- **Физические элементы – это артефакты** (run time)**.**

В качестве логических элементов могут выступать функциональные, информационные части, подсистемы, …. не имеющие может быть пока физического оформления. В процессе проектирования компоненты отображаются в артефакты.

В качестве физических элементов могут выступать файлы, библиотеки, модули, исполняемые файлы, пакеты и т. п.

В терминологии UML компонент — это классификатор, т. е. дескриптор, описывающий множество однотипных объектов, и как всякий классификатор, компонент может иметь экземпляры.

С понятием "компонент" часто ассоциируют **компонентное или сборочного программирование**, идея которого также стара, как и само программирование.

- Действительно, затраты на тиражирование программ любой величины и сложности исчезающе малы по сравнению с затратами на их разработку. . Отсюда возникает естественное желание не создавать каждую новую программу "с нуля", а использовать ранее созданные программы в качестве готовых строительных элементов, благо затраты на копирование столь не велики.

Понятие компонента в UML не соответствует тому понятию, которое имеет в виду сборочное программирование. Компонент UML является частью модели, и описывает логическую сущность, которая существует только во время проектирования (design time), хотя в дальнейшем ее можно связать с физической реализацией (артефактом) времени исполнения (run time).

Компонент (component) – это модульный фрагмент логического представления системы, взаимодействие с которым описывается набором обеспеченных и требуемых интерфейсов.

Компонент скрывает свою реализацию за набором внешних интерфейсов. Компоненты системы, разделяющие общие интерфейсы, могут замещать друг друга, сохраняя при этом одинаковое логическое поведение.

При выделении компонентов применяются следующие неформальные критерии.

Компонент нетривиален. Это нечто более сложное и объемное, чем фрагмент кода или одиночный класс.

Компонент независим, но не самодостаточен. Он содержит все, что нужно для функционирования, но предназначен для работы во взаимодействии с другими компонентами.

Компонент однороден. Он выполняет несколько взаимосвязанных функций, которые могут быть естественным образом охарактеризованы как единое целое в контексте более сложной системы.

Компонент заменяем. Он поддерживает строго определенный набор интерфейсов и может быть без ущерба для функционирования системы заменен другим компонентом, поддерживающим те же интерфейсы.

Компоненты понимаются в UML в наиболее общем смысле: это не только исполнимые файлы с кодами программы, но и исходные тексты программ, вебстраницы, справочные файлы, сопроводительные документы, файлы с данными и вообще любые артефакты, которые тем или иным способом используются при работе приложения и входят в его состав.

Для того чтобы как-то отражать такое разнообразие типов артефактов, являющихся компонентами, в UML предусмотрены стандартные стереотипы компонентов перечисленные в таблице ниже. Помимо стандартных стереотипов, многие инструменты поддерживают дополнительные стереотипы компонентов, часто со специальными значками и фигурами, обеспечивающими высокую наглядность диаграмм компонентов.

Стандартные стереотипы компонентов

Стереотип	Описание
«buildComponent»	Компонент, служащий для разработки приложения
«entity»	Постоянно хранимый информационный компонент, представляющий некоторое понятие предметной области
«service»	Функциональный компонент без состояния, возвращающий запрашиваемые значения без побочных эффектов
«subsystem»	Единица иерархической декомпозиции большой системы

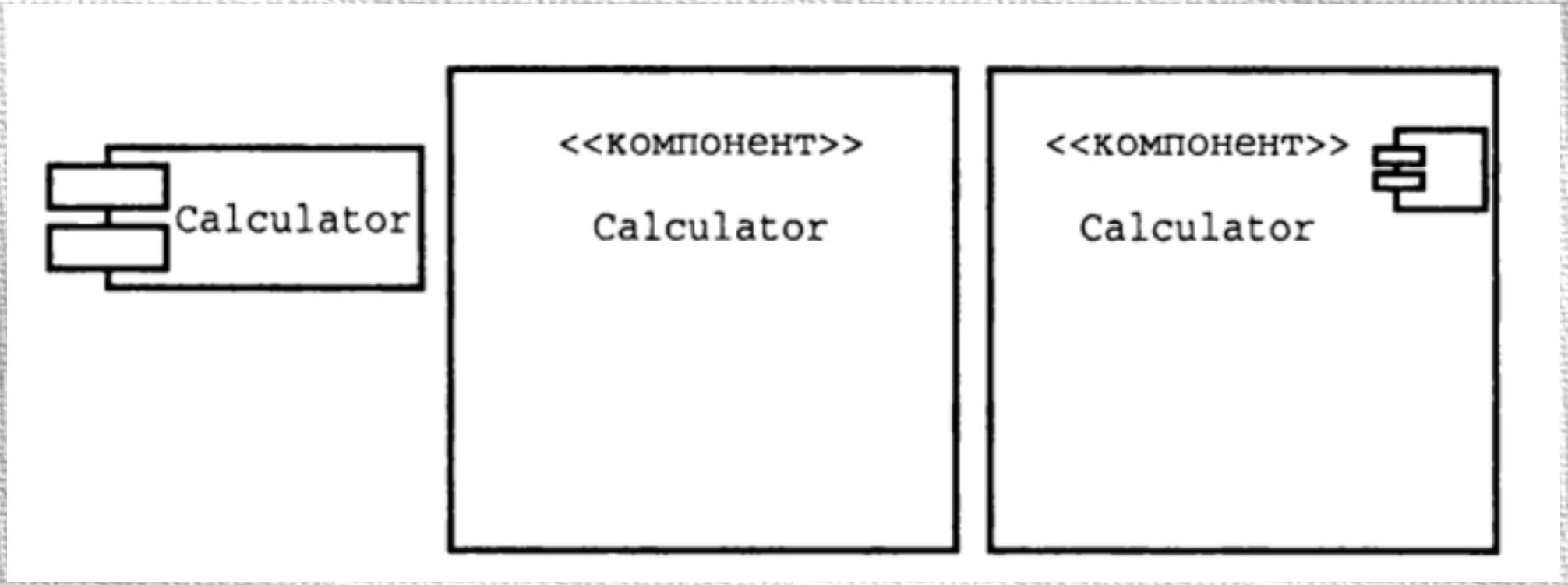
Стереотип	Описание
«process»	Компонент, ориентированный на транзакции
«implementation»	Определение компонента, не имеющего собственной спецификации. Он является реализацией отдельного компонента, обозначенного стереотипом «specification», с которым имеет отношение зависимости.
«specification»	Классификатор, определяющий область объектов без описания физической реализации этих объектов. Например, компонент, обозначенный стереотипом «specification», имеет только предоставляемые и требуемые интерфейсы и не имеет реализующих классификаторов.

Стереотип	Описание
«executable»	Выполнимая программа любого вида. Подразумевается по умолчанию, если никакой стереотип не указан.
«document»	Документ любого типа, например, файл с документацией к программе. .
«file»	Файл с исходным кодом программы или с данными, которые программа использует.
«library»	Статическая или динамическая библиотека.
«table»	Таблица базы данных.

Примеры компонентов

- Класс
- Подсистема
- СУБД (DBMS)
- Браузер
- Сервер приложений

Обозначение компонента в UML 1.x и две версии пиктограммы компонента в UML 2.0



Применение диаграмм компонентов

Начнем с элементарного соображения: в случае разработки "монолитного" настольного приложения диаграмма компонентов не нужна — она оказывается тривиальной и никакой полезной информации не содержит. Таким образом, диаграммы компонентов применяются только при моделировании многокомпонентных приложений.

Если приложение поставляется в виде "конструктора" (набора "кубиков" — компонентов) из которого при установке собирается конкретный уникальный экземпляр приложения, то диаграммы компонентов оказываются просто незаменимым средством.

При разработке приложений, которые должны взаимодействовать с так называемыми унаследованными приложениями и данными, без диаграмм компонентов также трудно обойтись. Дело в том, что фактически единственным средством UML, позволяющим как-то описать и включить в модель унаследованные приложения и данные являются компоненты (и их интерфейсы).
Сюда же относится случай моделирования физической структуры данных в базе, работать с которой требуется помимо "родной" СУБД.
Последним примером применения диаграмм компонентов является моделирование систем динамической архитектуры, то есть таких систем, которые меняют состав и количество своих компонентов во время выполнения. Например, многие приложения Интернета меняют свою конфигурацию во время выполнения в зависимости от текущей нагрузки.

Диаграмма

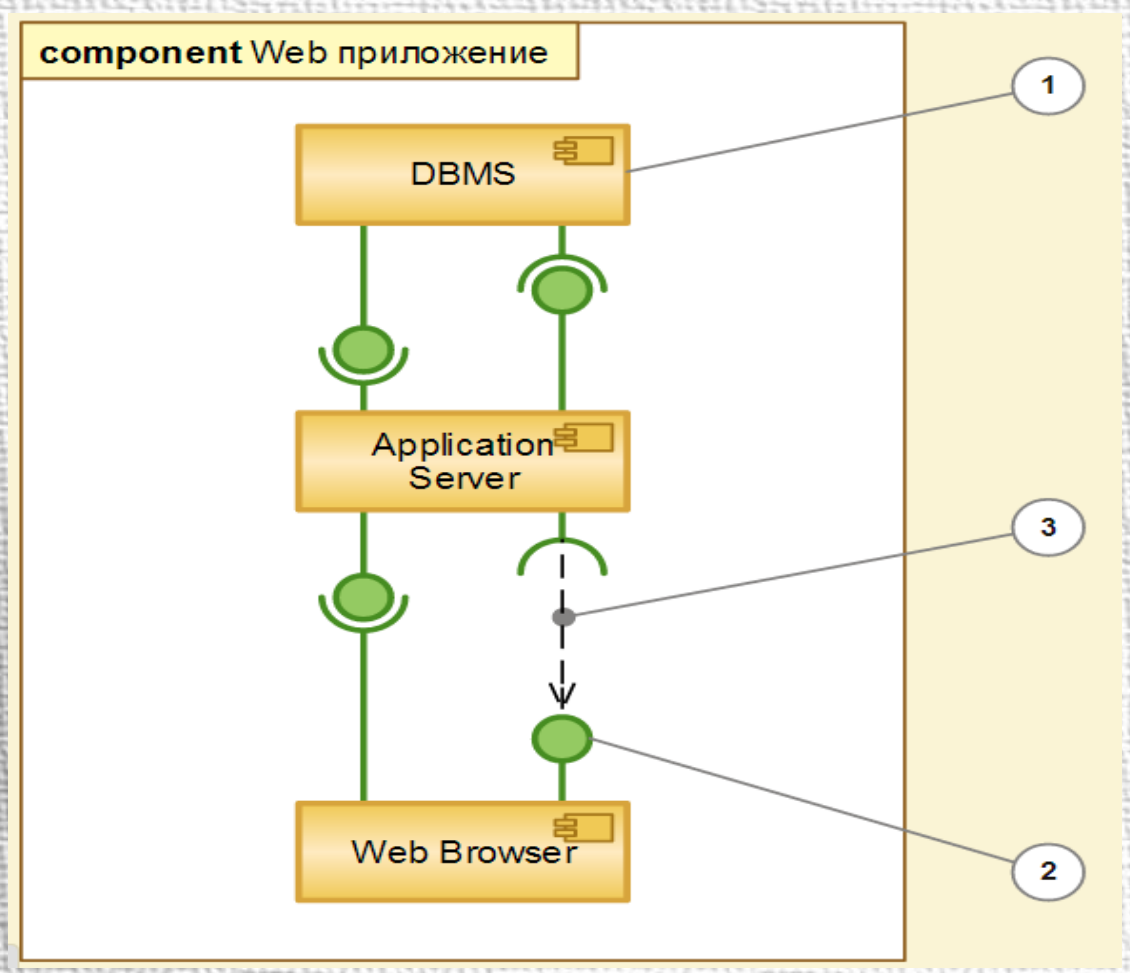
Диаграмма компонентов (component diagram) – показывает взаимосвязи между модулями (логическими или физическими), из которых состоит моделируемая система.

С помощью диаграммы компонентов представляются инкапсулированные классы вместе с их интерфейсными оболочками, портами и внутренними структурами (которые тоже могут состоять из компонентов и коннекторов)

Основной тип сущностей на диаграмме компонентов – это сами компоненты ❶, а также интерфейсы ❷, посредством которых указывается взаимосвязь между компонентами. На диаграмме компонентов применяются следующие отношения:

- реализации между компонентами и интерфейсами (компонент реализует интерфейс);
- зависимости между компонентами и интерфейсами (компонент использует интерфейс) ❸.

Нотация диаграммы компонентов



1. компоненты

2. интерфейсы

3. зависимости между компонентами и интерфейсами

Типичная схема компонентов для большой системы может включать следующие компоненты:

- **Представление**. Компонент, предоставляющий пользователю доступ, обычно выполняемый в веб-браузере.
- **Компоненты веб-службы**. Обеспечивает связь между клиентами и серверами.
- **Контроллеры вариантов использования**. Проведите пользователя по шагам каждого сценария.
- **Ядро бизнеса**. Содержит классы, основанные на классах в модели требований, реализует ключевые операции и налагает бизнес-ограничения.
- **База данных**. Хранит бизнес-объекты.
- **Компоненты ведения журнала** и обработки ошибок.

Интерфейс компонента

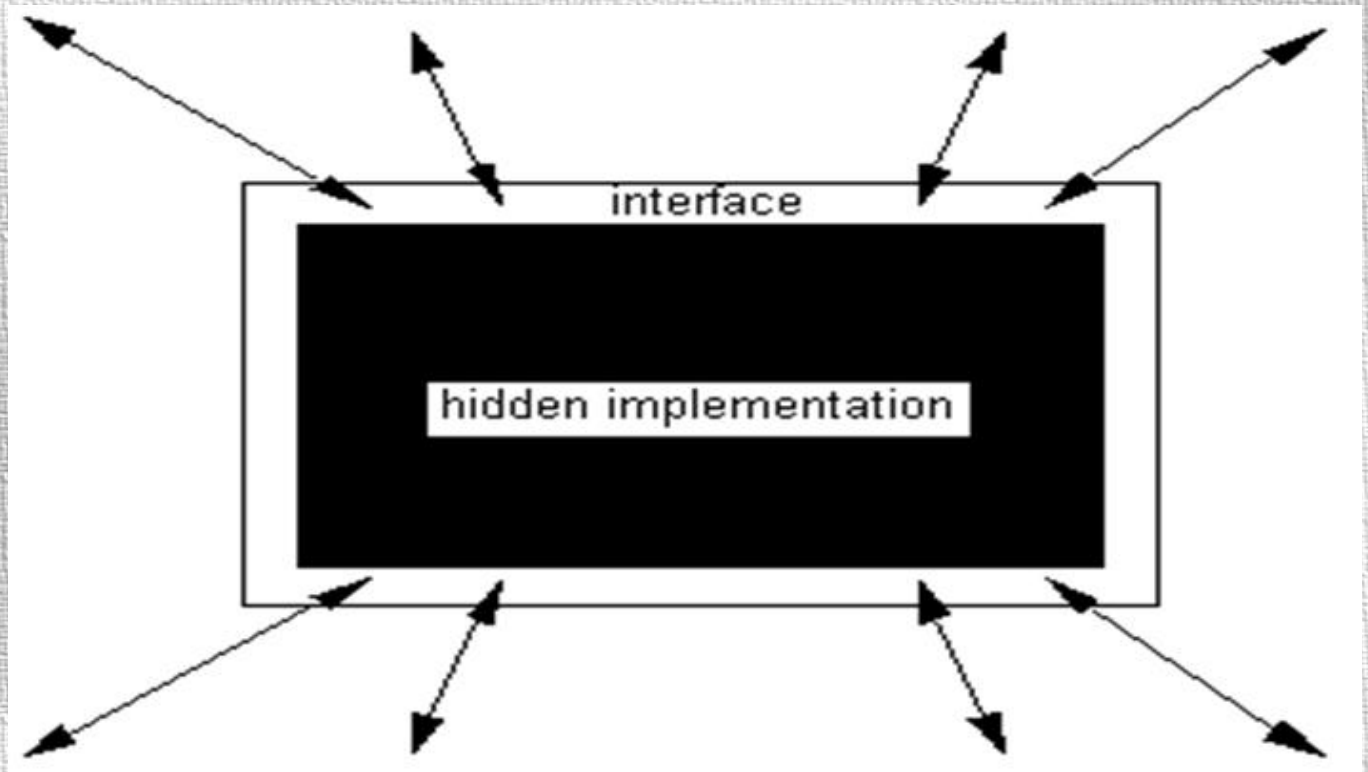
Интерфейс – это именованный набор составляющих, описывающий контракт между поставщиками и потребителями услуг.

Другими словами, интерфейс – это абстрактный класс, в котором все составляющие – атрибуты и операции – абстрактны.

Что значит абстрактные атрибуты? Абстрактные атрибуты интерфейса – это атрибуты, которые обязательно должны появиться в классе, реализующем интерфейс.

Поскольку интерфейс – это абстрактный класс, он не может иметь непосредственных экземпляров.

Программный компонент — это черный ящик, все взаимодействия с которым происходят через опубликованный интерфейс



Главная идея, лежащая в основе интерфейсов, – разделение описания функциональности (интерфейс) от ее реализации классификатором, таким как класс или подсистема. Создать экземпляр интерфейса невозможно. Он просто объявляет контракт, который может быть реализован классификаторами. Все, что реализует интерфейс, - принимает и соглашается следовать определяемому интерфейсом контракту.

Возможности, которые могут быть описаны в интерфейсах, перечислены в таблице. В ней также показаны обязанности реализующих их классификаторов по отношению к интерфейсу.

В интерфейсах также должны присутствовать описания семантики их возможностей (обычно в виде текста или псевдокода) как руководства для тех, кто будет их реализовывать.

Интерфейс описывает	Реализующий классификатор
Операция	Должен иметь операцию с аналогичной сигнатурой и семантикой.
Атрибут	Должен иметь открытые операции для задания и получения значения атрибута – не требуется, чтобы у реализующего классификатора на самом деле был атрибут, определенный интерфейсом, но его поведение должно быть таким, как будто он существует.
Ассоциация	Должен иметь ассоциацию с целевым классификатором – если интерфейс описывает ассоциацию с другим интерфейсом, классификатор, реализующий эти интерфейсы, должен реализовывать ассоциацию между ними.
Ограничение	Должен поддерживать ограничение.
Стереотип	Имеет стереотип.

.

Интерфейс описывает	Реализующий классификатор
Помеченное значение	Имеет помеченное значение.
Протокол (например, определенный конечным автоматом протокола)	Должен реализовывать протокол

.

Атрибуты и операции в интерфейсе должны быть полностью описаны и включать следующее:

- полную сигнатуру операции (имя, типы всех параметров и возвращаемый тип);
- семантику операции – она может быть записана в виде текста или псевдокода;
- имена и типы атрибутов;
- все стереотипы, ограничения или помеченные значения операции или атрибута.

Важно помнить, что интерфейс определяет только описание своих возможностей и что он никогда не заключает в себе какой-либо конкретной реализации.

Таким образом, интерфейс описывает видимое извне поведение элемента. Может представлять полное поведение класса или компонента либо только часть такого поведения. Определяет набор спецификаций операций (то есть их сигнатуру), но никогда не определяет детали их реализации.

Если язык реализации не поддерживает интерфейсы непосредственно, используйте абстрактные классы.

Интерфейсы определяют контракт

Интерфейсы могут иметь огромное влияние на способ проектирования. До сих пор проектирование заключалось в соединении определенных классов. Этот процесс можно было бы назвать «**проектирование реализации**». Однако более гибким подходом является «**проектирование контракта**», когда классы соединяются с интерфейсом, после чего этот интерфейс может быть реализован любым количеством классов и других классификаторов.

Часто говорят, что интерфейс определяет ***сервис***, предлагаемый классом, подсистемой или компонентом. Современные программные архитектуры часто основываются на сервисах.

Главное отличие класса от интерфейса — в том, что класс состоит из интерфейса и реализации.

Любой класс всегда **неявно** объявляет свой **интерфейс** — то, что доступно при использовании класса извне. Если у нас есть класс **Ключ** и у него публичный метод ***Открыть***, который вызывает приватные методы ***Вставить***, ***Повернуть*** и ***Вынуть***, то интерфейс класса **Ключ** состоит из метода ***Открыть***. Когда мы унаследуем какой-то класс от класса **Ключ**, он унаследует этот интерфейс.

Кроме этого интерфейса у класса есть также **реализация** — методы ***Вставить***, ***Повернуть***, ***Вынуть*** и их вызов в методе ***Открыть***. Наследники **Ключа** наследуют вместе с интерфейсом и реализацию.

И вот здесь таятся проблемы. Предположим, у нас есть некая модель, которая предполагает использование ключа для открытия двери. Она знает интерфейс **Ключа** и поэтому вызывает метод ***Открыть***.

Но, предположим, некоторые двери открываются не таким вот поворотным ключом, а магнитной карточкой — которая ведь тоже по своей сути ключ! Интерфейс этой карточки никак принципиально не отличается от интерфейса обычного ключа — можно ***Открыть*** ключом, а можно ***Открыть*** карточкой.

И мы хотим сделать класс **Магнитную Карточку**, который тоже будет содержать интерфейс **Ключа**. Для этого мы унаследуем **Магнитную Карточку** от **Ключа**. Но вместе с интерфейсом унаследовалась и реализация, которая в методе ***Открыть*** вызывает методы ***Вставить***, ***Повернуть*** и ***Вынуть*** — а это совершенно не подходит для **Магнитной Карточки**.

Нам придётся самое меньшее перегружать в **Магнитной Карточке** реализацию метода **Открыть**, используя уже последовательность **Вставить**, **Провести** и **Вынуть**. Это уже плохо, потому что мы не знаем детали реализации класса Ключ — вдруг мы упустили какое-то очень важное изменение данных, которое должно было быть сделано — и было сделано в методе **Ключ**:: **Открыть.** Нам придётся лезть во внутреннюю реализацию **Ключа** и смотреть, что и как — даже если у нас есть такая техническая возможность, это грубое нарушение инкапсуляции, которое ни к чему хорошему не приведёт.

Именно так и пишут Гамма и др.: **наследование является нарушением инкапсуляции**.

Правильный подход к данному вопросу — **отделить интерфейс от реализации.** Во многих современных языках для этого предусмотрен специальный синтаксис.

Мы должны опираться на интерфейсы, а не на классы.

- Объявим явно интерфейс **Ключ**, содержащий метод ***Открыть***.
- Объявим класс **Поворотный Ключ**, реализующий интерфейс **Ключ** при помощи своих методов ***Вставить***, ***Повернуть*** и ***Вынуть***.
- Объявим класс **Магнитная Карточка**, тоже реализующий интерфейс **Ключ**, но уже по-своему — и без каких-либо неприятных пересечений с реализацией **Поворотного Ключа**.

Этого помогло нам достичь отделение интерфейса от реализации.

ИСПОЛЬЗОВАНИЕ ИНТЕРФЕЙСА

Использование интерфейсов возможно двумя способами:

1. **Класс может реализовывать интерфейс.** Реализация интерфейса заключается в том, что в описании класса данный интерфейс указывается как реализуемый, а в коде класса обязательно определяются все методы, которые описаны в интерфейсе, в полном соответствии с сигнатурами из описания этого интерфейса. То есть, если класс реализует интерфейс, для любого экземпляра этого класса существуют и могут быть вызваны все описанные в интерфейсе методы. Один класс может реализовать несколько интерфейсов одновременно.

2. **Возможно объявление переменных и параметров методов как имеющих тип «интерфейс».** В такую переменную или параметр может быть записан экземпляр любого класса, реализующего интерфейс. Если интерфейс объявлен как тип возвращаемого значения функции, это означает, что функция возвращает объект класса, реализующего данный интерфейс.

Использование интерфейсов — один из вариантов обеспечения полиморфизма в объектных языках и средах. Все классы, реализующие один и тот же интерфейс, с точки зрения определяемого ими поведения, ведут себя внешне одинаково. Это позволяет писать обобщённые алгоритмы обработки данных, использующие в качестве типов параметры интерфейсов, и применять их к объектам различных типов, всякий раз получая требуемый результат.

Например, интерфейс «Cloneable» может описать абстракцию клонирования (создания точных копий) объектов, специфицировав метод «Clone», который должен выполнять копирование содержимого объекта в другой объект того же типа. Тогда любой класс, объекты которого может понадобиться копировать, должен реализовать интерфейс Cloneable и предоставить метод Clone, а в любом месте программы, где требуется клонирование объектов, для этой цели у объекта вызывается метод Clone. Причём, использующему этот метод коду достаточно иметь только описание интерфейса, он может ничего не знать о фактическом классе, объекты которого копируются. Таким образом, интерфейсы позволяют разбить программную систему на модули без взаимной зависимости кода.

Интерфейсы и абстрактные классы

Можно заметить, что интерфейс, с формальной точки зрения, — это просто чистый абстрактный класс, то есть класс, в котором не определено ничего, кроме абстрактных методов. Если язык программирования поддерживает множественное наследование и абстрактные методы (как, например, C++), то необходимости во введении в синтаксис языка отдельного понятия «интерфейс» не возникает. Данные сущности описываются с помощью абстрактных классов и наследуются классами для реализации абстрактных методов.

Однако поддержка множественного наследования в полном объёме достаточно сложна и вызывает множество проблем, как на уровне реализации языка, так и на уровне архитектуры приложений. Введение понятия интерфейсов является компромиссом, позволяющим получить многие преимущества множественного наследования (в частности, возможность удобно определять логически связанные наборы методов в виде сущностей, подобных классам и допускающих наследование и реализацию), не реализуя его в полном объёме и не сталкиваясь, таким образом, с большинством связанных с ним трудностей.

Между интерфейсами и другими классификаторами, в частности, классами, на диаграмме классов применяются два отношения:

1. классификатор (в частности, класс) используется интерфейс – это показывается с помощью **зависимости** со стереотипом «**call**»;
2. классификатор (в частности, класс) реализует интерфейс – это показывается с помощью отношения **реализации**.

Никаких ограничений на использование отношения реализации не накладывается: класс может реализовывать много интерфейсов, и наоборот, интерфейс может быть реализован многими классами.

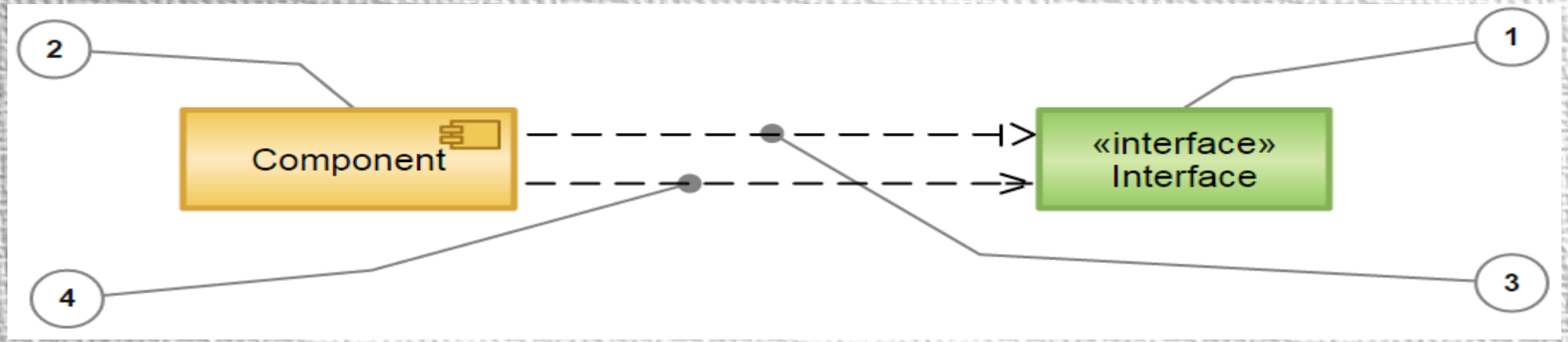
Нет ограничений и на использование зависимостей со стереотипом «**call**» – класс может вызывать любые операции любых видимых интерфейсов.

Семантика зависимости со стереотипом «**call**» очень проста – эта зависимость указывает, что в операциях класса, находящегося на независимом полюсе, вызываются операции класса (в частности, интерфейса) находящегося на зависимом полюсе.

Предоставляемые и требуемые интерфейсы

Можно выделить две роли, которые играют интерфейсы **1** по отношению к классификаторам, например, компонентам **2**. Интерфейс может быть

- предоставляемым (реализованный, обеспеченный)
- требуемым (запрашиваемый).



Реализуемый классификатором набор интерфейсов называют **предоставляемыми** (provided) интерфейсами

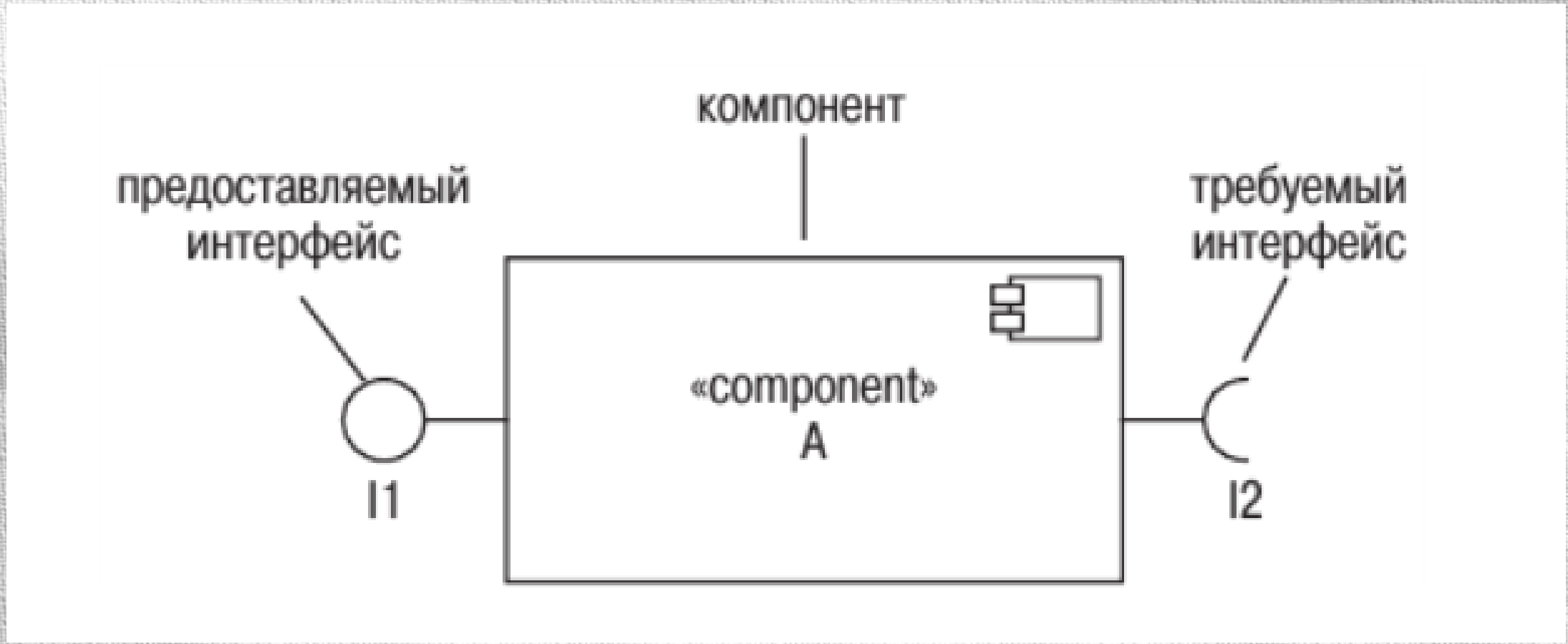
Интерфейсы, необходимые для работы классификатора, называют **требуемыми** (required) интерфейсами.

Нельзя забывать, что сам по себе интерфейс – это просто описание контракта, а обеспеченным или требуемым он становится в зависимости от того, как этот интерфейс используется:

- если класс реализует интерфейс – то для данного классификатора это **предоставляемый интерфейс** и данный факт показывается с помощью отношения реализации **3**;
- если класс вызывает операции интерфейса — то для данного классификатора это **требуемый интерфейс** и данный факт показывается с помощью отношения зависимости **4**.

Если требуется показать отношения между компонентами без указания обеспеченных и требуемых интерфейсов, то можно воспользоваться отношением зависимости.

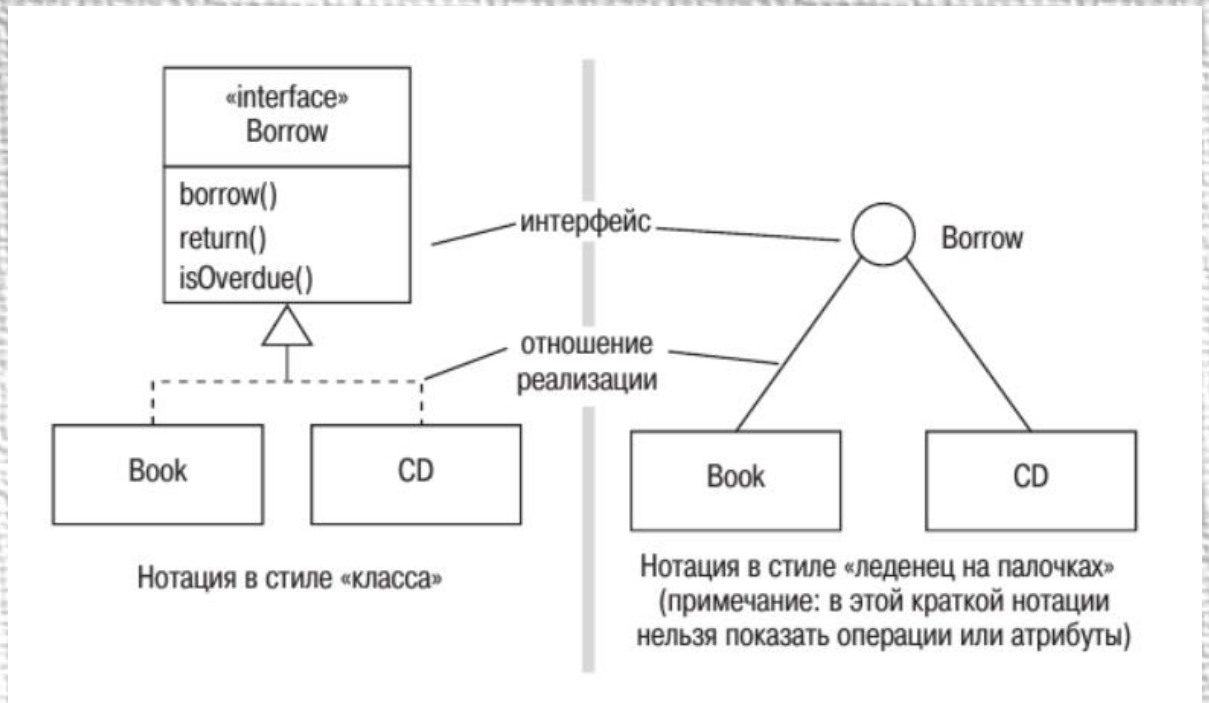
Компонент отображается в виде прямоугольника со стереотипом «компонент» и/или пиктограммой компонента верхнем правом углу. У компонентов могут быть предоставляемые и требуемые интерфейсы и порты.



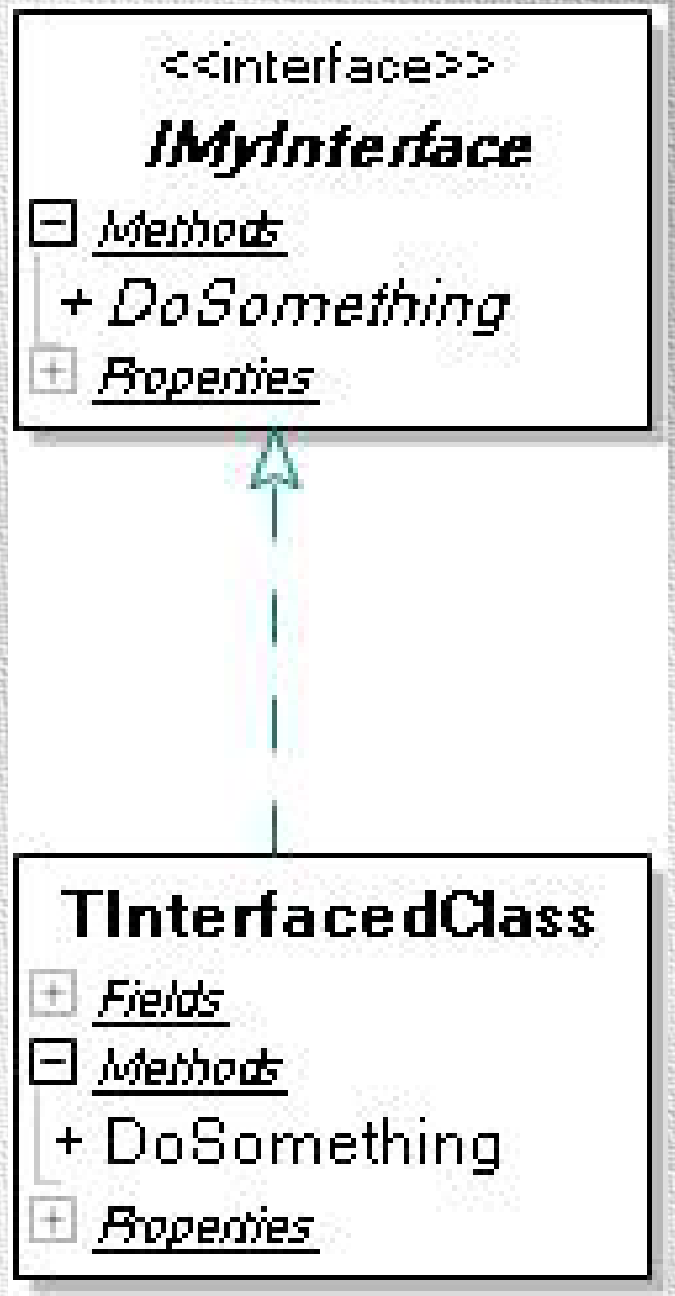
Два варианта синтаксиса для предоставляемых интерфейсов

Два варианта UML - синтаксиса для предоставляемых интерфейсов: нотация в стиле «класса» с ключевым словом «interface» (в которой можно показывать атрибуты и операции) и краткая нотация в стиле «леденец на палочках» (в которой можно не показывать атрибуты и операции).

Нотацию класса используют, когда хотят показать возможности интерфейса, а нотацию «леденец на палочках» – когда этого делать не надо. Но они обе представляют одно и то же.



Обычно имена интерфейсов и классов совпадают; они записываются стиле *UpperCamelCase*. Однако в C# действует общепринятый стандарт – имя каждого интерфейса начинается с большой **I**, например IBorrow. Другой стиль присваивания имен – добавление в конце имени суффикса «able», например Borrowable или IBorrowable.

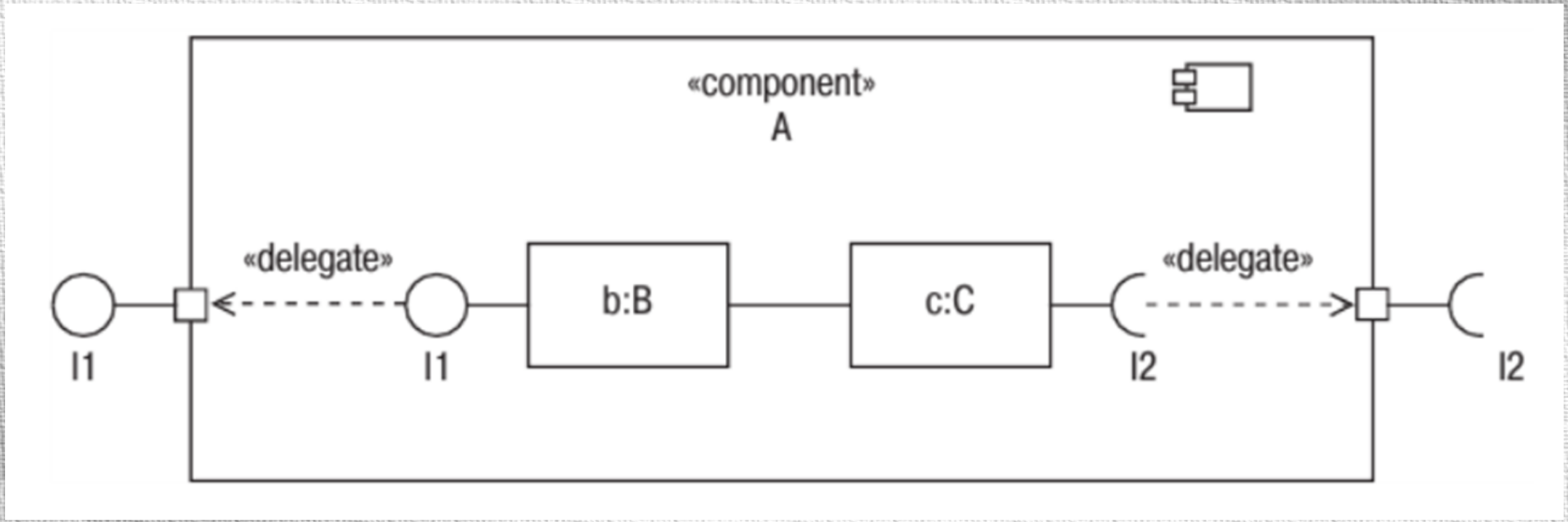


Отношение реализации – это отношение между описанием (в данном случае интерфейсом) и тем, что реализует это описание (в данном случае классами Book и CD). Позже мы увидим, что **не только классы могут реализовывать интерфейс. Это также могут делать и другие классификаторы, такие как пакеты и компоненты.**

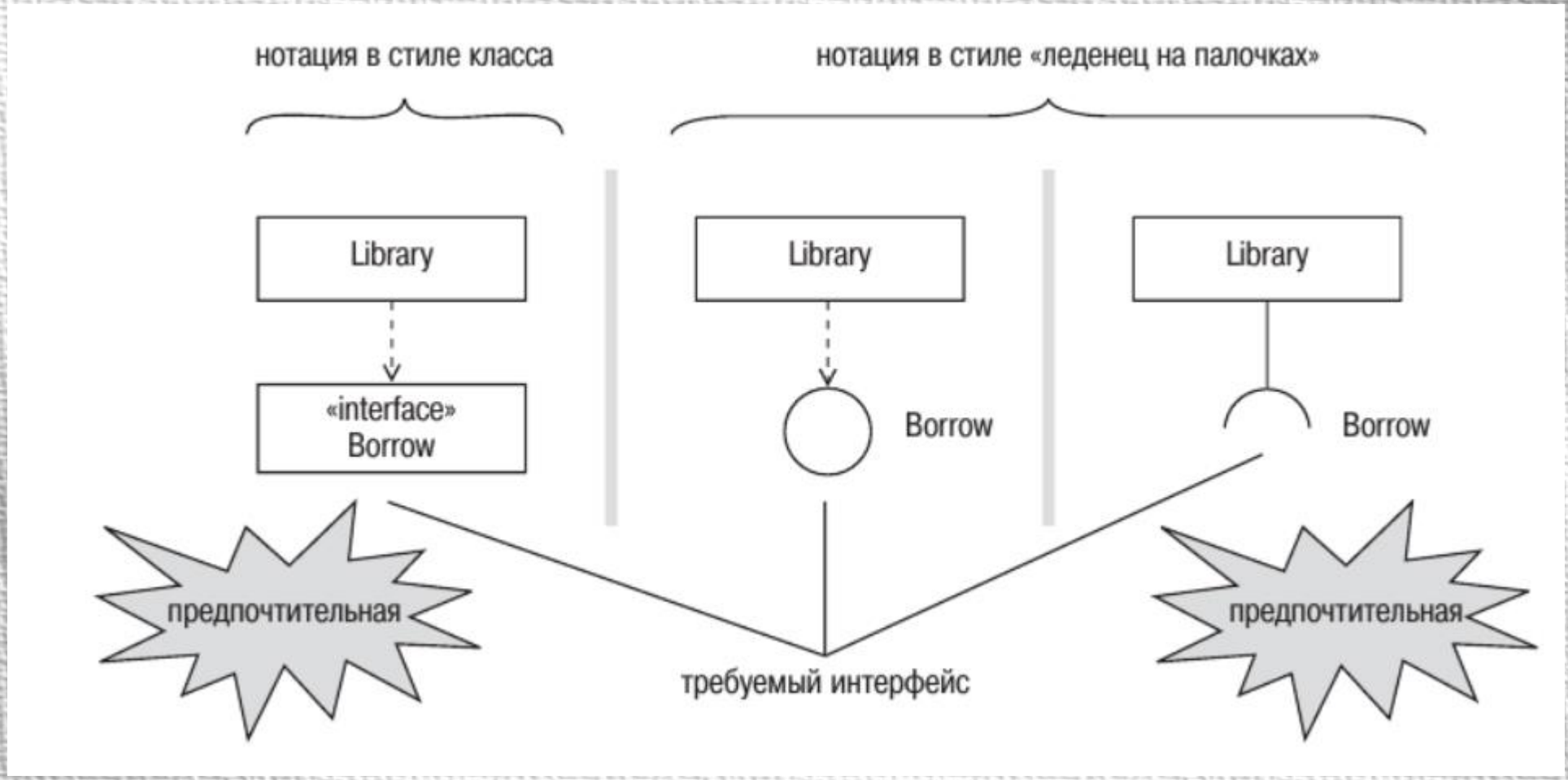
Внутренняя структура компонента

Компонент может иметь внутреннюю структуру. Части можно показать вложенными внутрь компонента (рисунок ниже) или находящимися снаружи и соединенными с ним отношением зависимости. Обе формы синтаксически эквивалентны, хотя первая нотация кажется более наглядной.

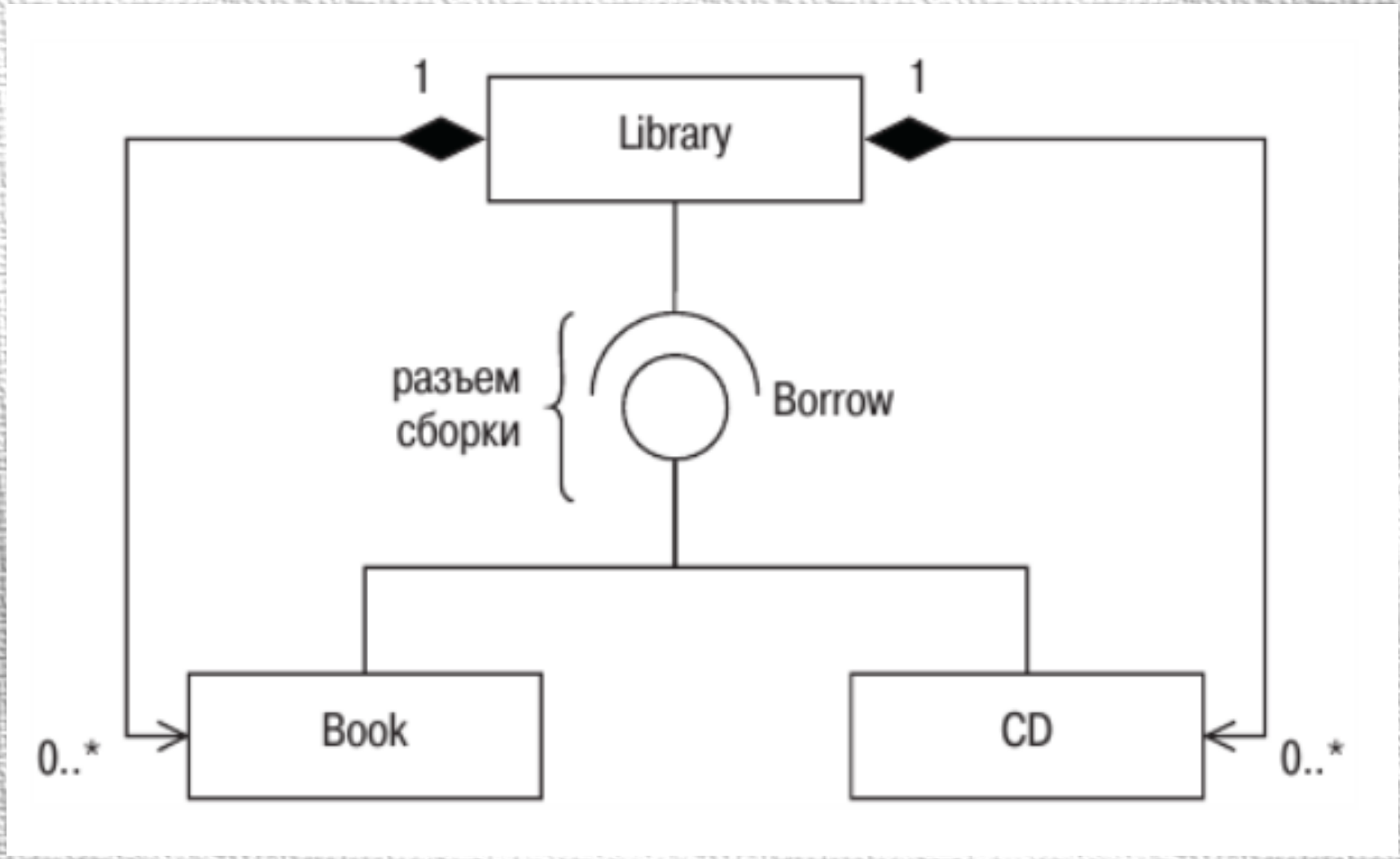
Если у компонента есть внутренняя структура, как правило, он будет делегировать обязанности, определенные его интерфейсами, своим внутренним частям. На рисунке компонент ***A*** предоставляет интерфейс ***I1*** и требует интерфейс ***I2***. Он инкапсулирует две части типа ***b*** и ***c***. Он делегирует поведение, описанное его предоставляемым и требуемым интерфейсами ***b*** и ***c*** соответственно.


«component»
A
«delegate»
«delegate»
b:B
c:C
I1
I1
I2
I2

Варианты синтаксиса требуемого интерфейса

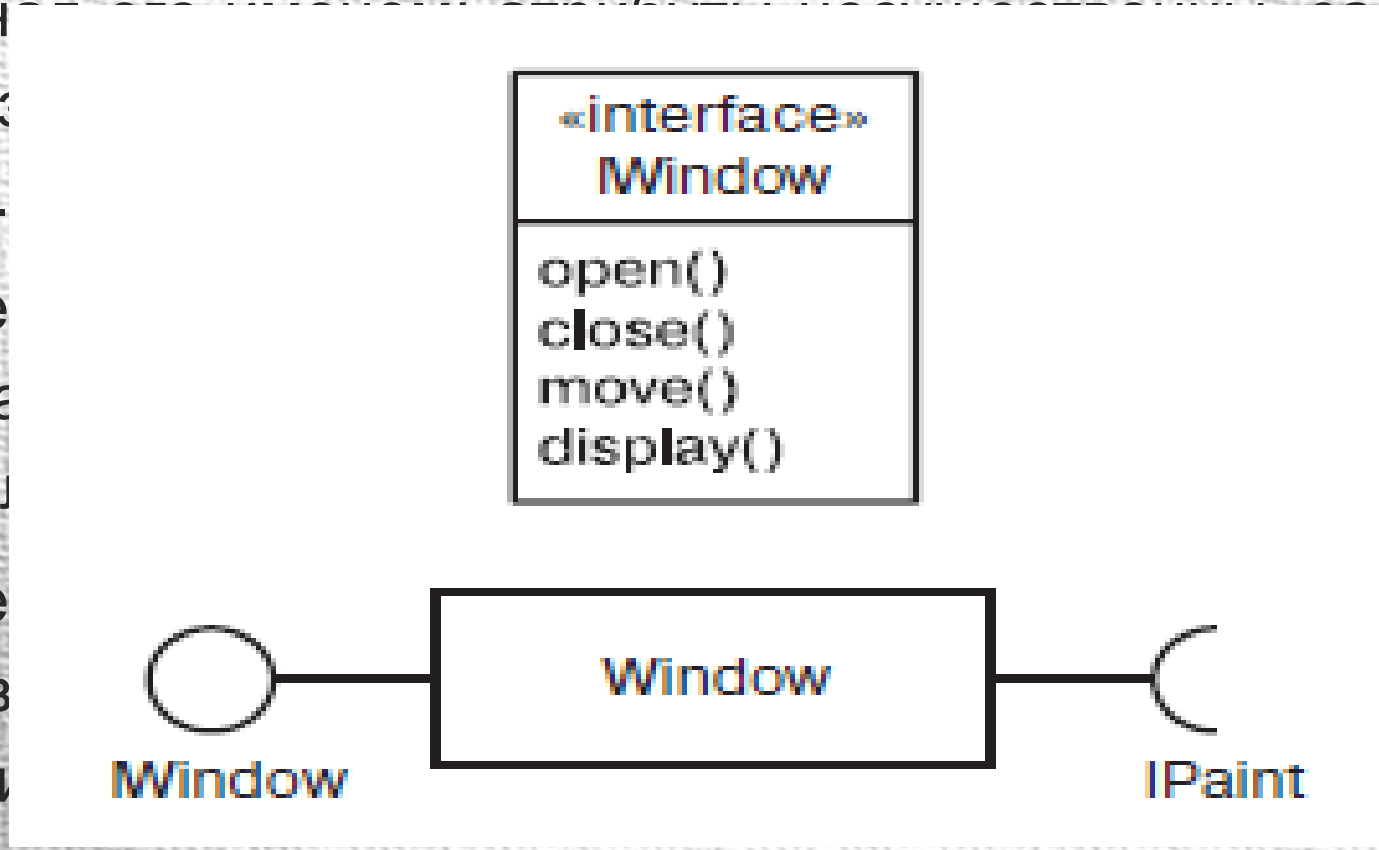


Система Library в сборке



Объявление интерфейса изображается как класс с ключевым словом «interface» н исключением иногда показ ко существует сам по себе.

- Интерфе шнего мира, изобража ого линией с рамкой к
- Интерфе ругого класса, представ м с рамкой класса ли



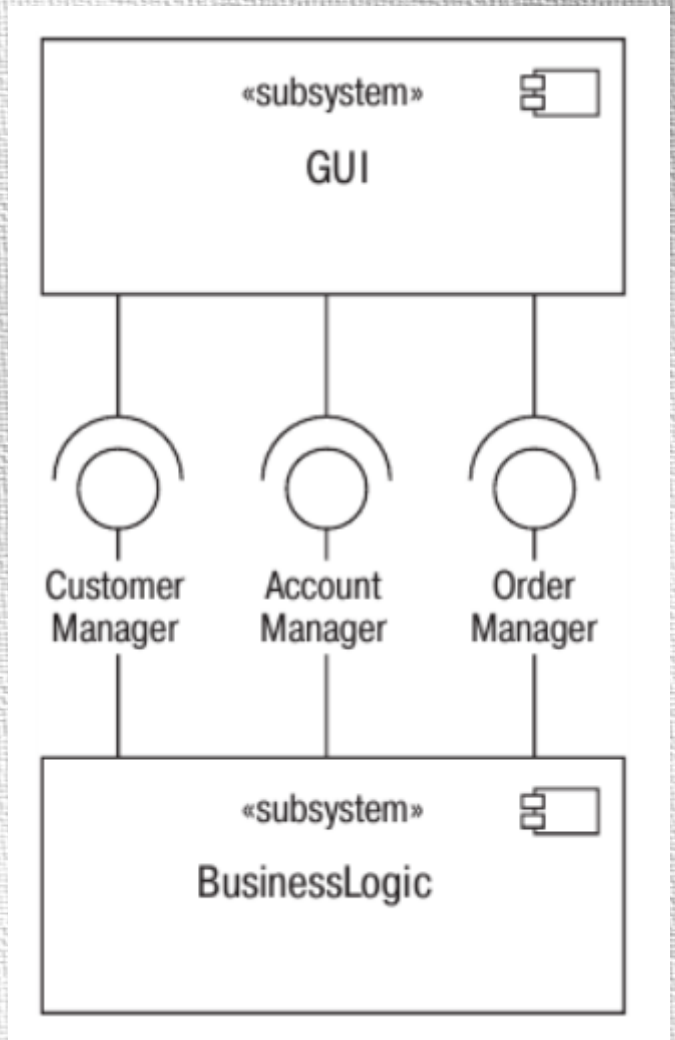
Часто компоненты отображаются просто как «черные ящики», к которым прикреплены их предоставляемые и требуемые интерфейсы. Однако компонент может быть представлен и как «белый ящик». Такое полное представление раскрывает внутренние детали компонента. В нем могут быть показаны любые предоставляемые интерфейсы, требуемые интерфейсы, реализации или ассоциированные артефакты.

Полное представление компонента

Подсистемы

Подсистема – это компонент, действующий как единица декомпозиции большой системы. Подсистемы изображаются как компонент со стереотипом «subsystem».

Интерфейсы идут рука об руку с подсистемами, как показано на рисунке. В данном примере подсистеме GUI известны только интерфейсы CustomerManager, AccountManager и OrderManager. Она ничего не знает о внутренней реализации подсистемы BusinessLogic (бизнес-слогика).



Это значит, что в принципе подсистему **BusinessLogic** можно было бы полностью заменить даже несколькими подсистемами, если вместе они будут предоставлять такой же набор интерфейсов. Аналогично можно было бы заменить подсистему **GUI** другой подсистемой **GUI**, требующей такой же набор интерфейсов. Такое использование интерфейсов разъединяет подсистемы и обеспечивает гибкость архитектуры.

ПРОЕКТИРОВАНИЕ С ИСПОЛЬЗОВАНИЕМ ИНТЕРФЕЙСОВ

Один из путей сделать систему гибкой и способной изменяться – спроектировать ее так, чтобы обеспечить простое подключение расширений. Интерфейс – ключ к этому.

Если есть возможность проектировать системы с использованием интерфейсов, значит, ассоциации и сообщения будут привязаны не к объектам конкретного класса, а к определенному интерфейсу. Это упрощает добавление в систему новых классов, поскольку интерфейсы определяют протоколы, которые должны поддерживаться новыми классами, чтобы из можно было подключить. Поскольку интерфейсы определяют только контракт, они обеспечивают возможность существования любого количества конкретных реализаций, подчиняющихся этому контракту.

Шаблон Фасад

Сокрытие сложных подсистем за хорошо структурированным простым интерфейсом известно как шаблон Фасад (Fasade). Общая цель проектирования – свести до минимума общение и зависимости между подсистемами. Один из способов достижения этой цели – введение фасадного объекта, предоставляющего единственный упрощенный интерфейс для наиболее общих возможностей подсистемы. Шаблон Фасад обеспечивает возможность сокрытия информации и разделения задач – сложные детали внутренней работы подсистемы можно спрятать за простым интерфейсом.

Интерфейсы, используемые в качестве фасада, могут применяться для создания «швов» в системе. Это делается следующим образом:

- выявляются связные части системы;
- они упаковываются в «subsystem»;
- определяется интерфейс для этой подсистемы.

Шаблон Разбиение на уровни

Шаблон *Разбиение на уровни* компонует проектные подсистемы и интерфейсы в уровни. При этом подсистемы каждого уровня семантически связаны. Суть создания устойчивой многоуровневой архитектуры – управление связанностью подсистем благодаря:

- введению новых интерфейсов там, где необходимо;
- такой переорганизации классов в новые подсистемы, которая сокращает количество взаимосвязей между подсистемами.

С зависимостями между уровнями необходимо обращаться очень аккуратно, поскольку они представляют взаимосвязь уровней.

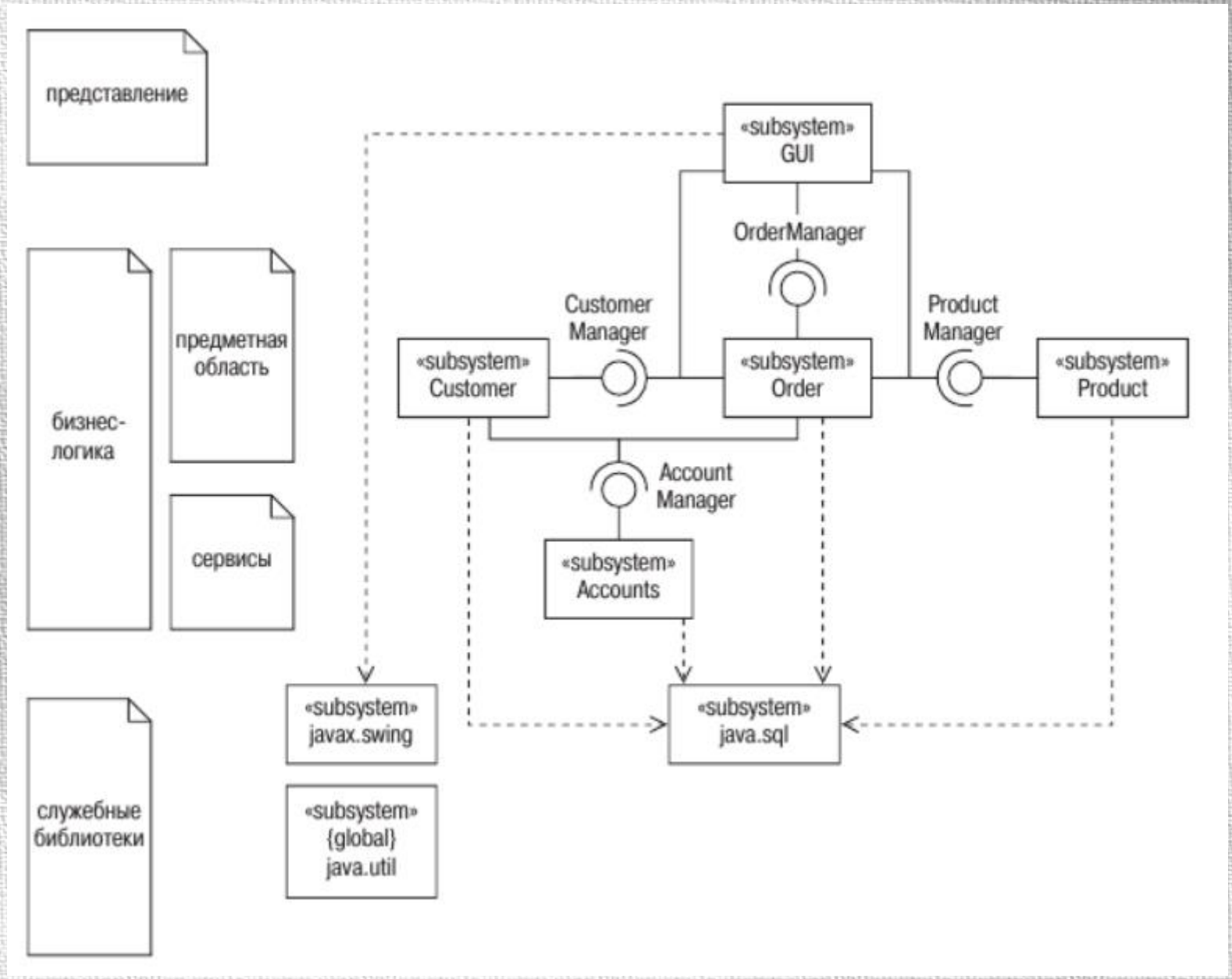
В идеале уровни должны быть максимально разъединенными, поэтому попытайтесь обеспечить, чтобы:

- зависимости были направлены в одну сторону;
- во всех зависимостях присутствовали интерфейсы-посредники.

Иначе говоря, подсистема определенного уровня по возможности должна требовать интерфейсы от нижележащего уровня и предоставлять интерфейсы более высокому уровню.

Существует много способов создания многоуровневых архитектур. Уровней может быть столько, сколько нужно. Однако в основном системы разделяют на представление, бизнес-логику и сервисные уровни.

Также довольно распространено более глубокое разбиение уровня бизнес-логики. В данном случае имеется два уровня – предметная область и сервисы.



Уровень предметной области включает подсистемы, характерные для данного конкретного приложения. В уровень сервиса вошли подсистемы, которые могут повторно использоваться в других приложениях.

Везде, где это возможно, лучше проектировать интерфейс.

На рис. показано, что все разработанные нами подсистемы соединяются посредством интерфейсов, а пакеты Java просто объединены зависимостями, хотя каждый предоставляет несколько интерфейсов. Причина в том, что если отображение собственных интерфейсов представляет некоторый интерес и пользу, то показывать интерфейсы стандартных библиотек Java не имеет никакого практического смысла..

Также обратите внимание, что пакет java.util, содержащий такие универсальные компоненты, как Date, используется повсеместно, поэтому обозначен меткой {global} (глобальный). Эта метка указывает на то, что все открытое содержимое данного пакета является видимым везде. Таким способом показывается, что было решено не отображать зависимости к этому пакету, поскольку они не дают никакой полезной информации.

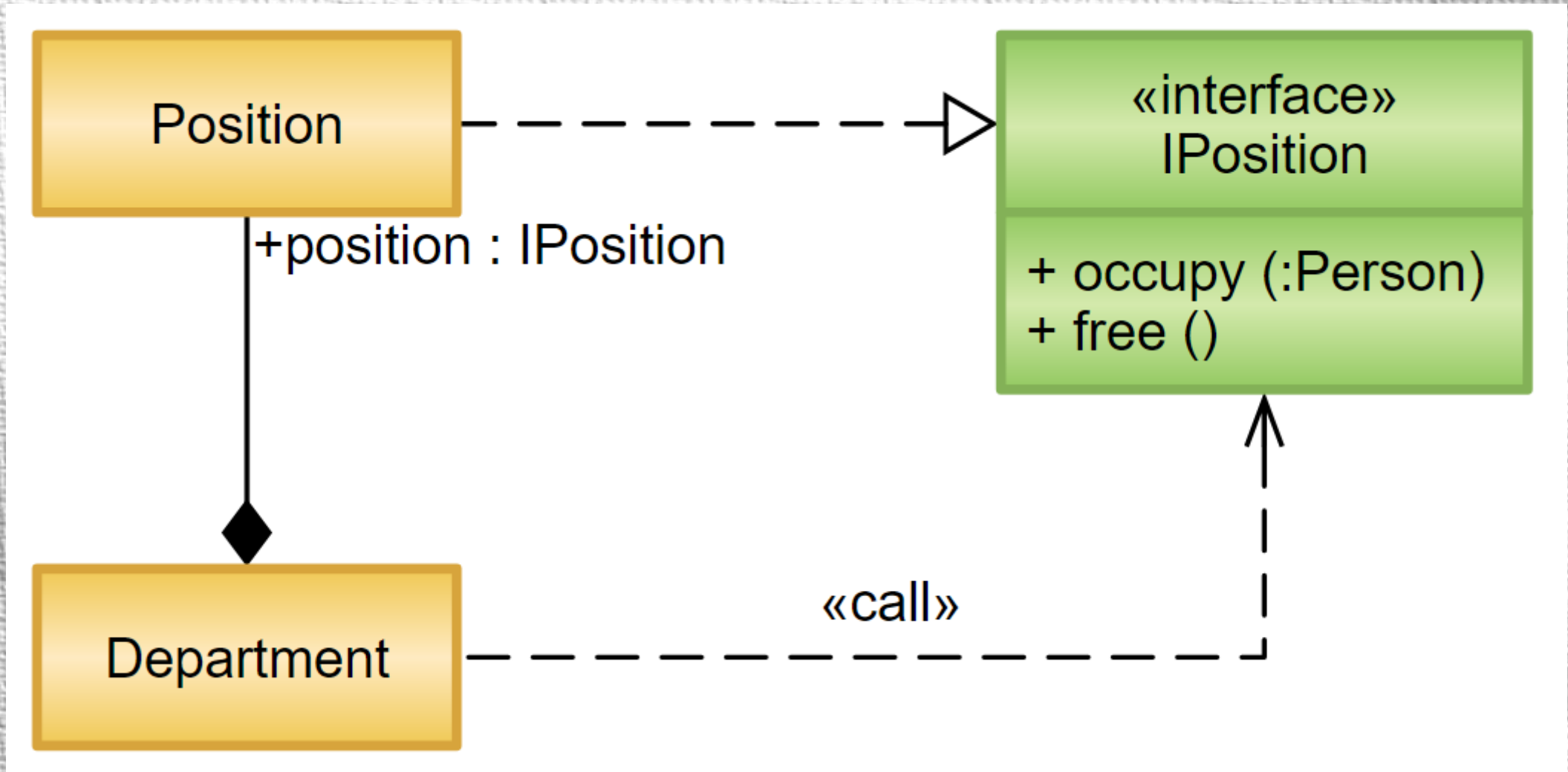
Однако в использовании интерфейсов есть и недостатки. Как правило, если что-то становится более гибким, оно становится и более сложным.

Поэтому при проектировании с использованием интерфейсов необходимо искать компромисс между гибкостью и сложностью. В принципе, сделать интерфейсом можно каждую операцию каждого класса, но понять такую систему было бы просто невозможно! Кроме того, часто жертвой гибкости становится производительность, но обычно ее потери малы в сравнении с увеличением сложности. Откровенно говоря, правильное моделирование системы важнее, чем ее гибкость. Запомните правило KISS – **keep interfaces sweet and simple** (интерфейсы должны быть удобными и простыми)!

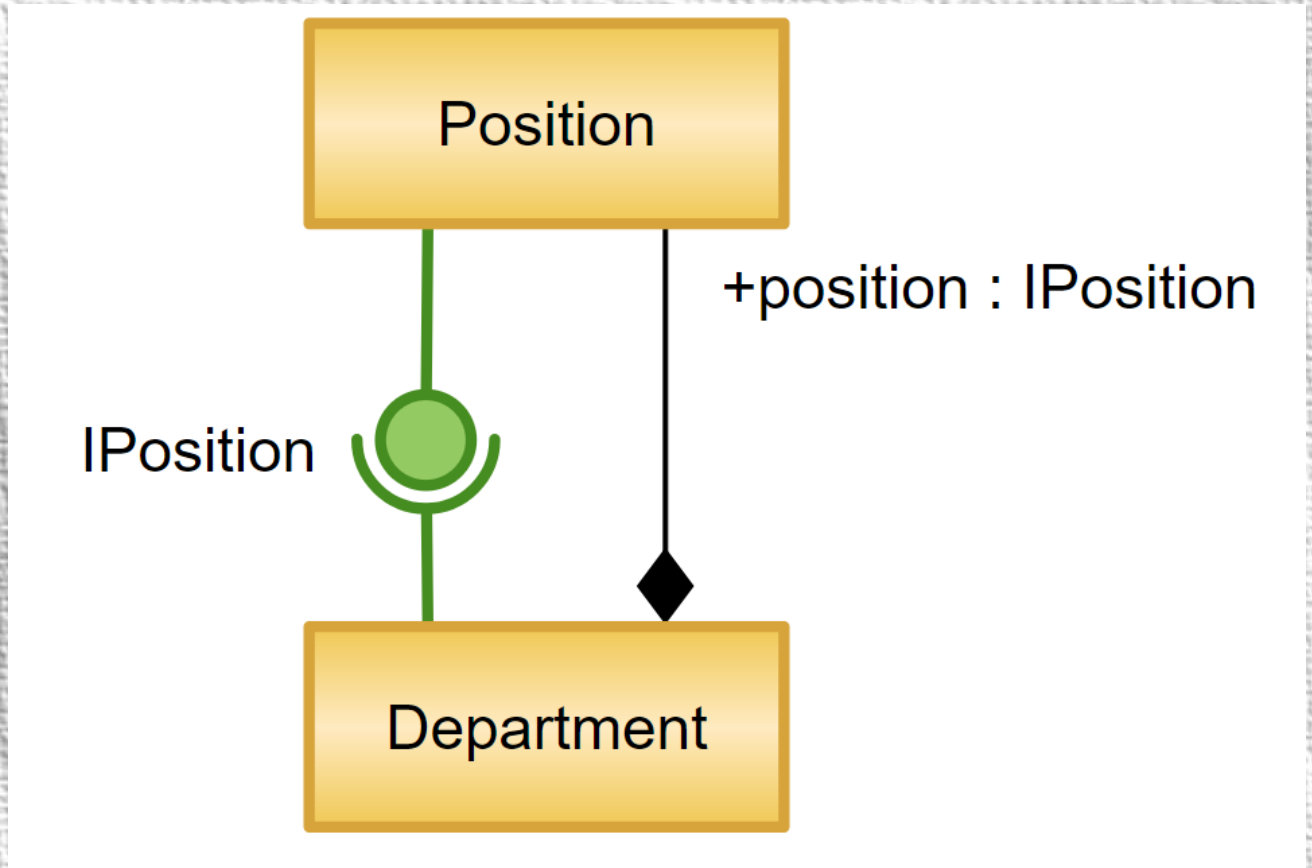
ПРИМЕР 1

Рассмотрим пример из информационной системы отдела кадров. Допустим, что класс Department для реализации операций связанных с движением кадров, использует операции класса Position, позволяющие занимать и освобождать должность – другие операции класса Position классу Department не нужны. Для этого, как показано на рисунке можно определить соответствующий интерфейс IPosition и связать его отношениями с данными классами.

Отношения реализации и использования интерфейсов

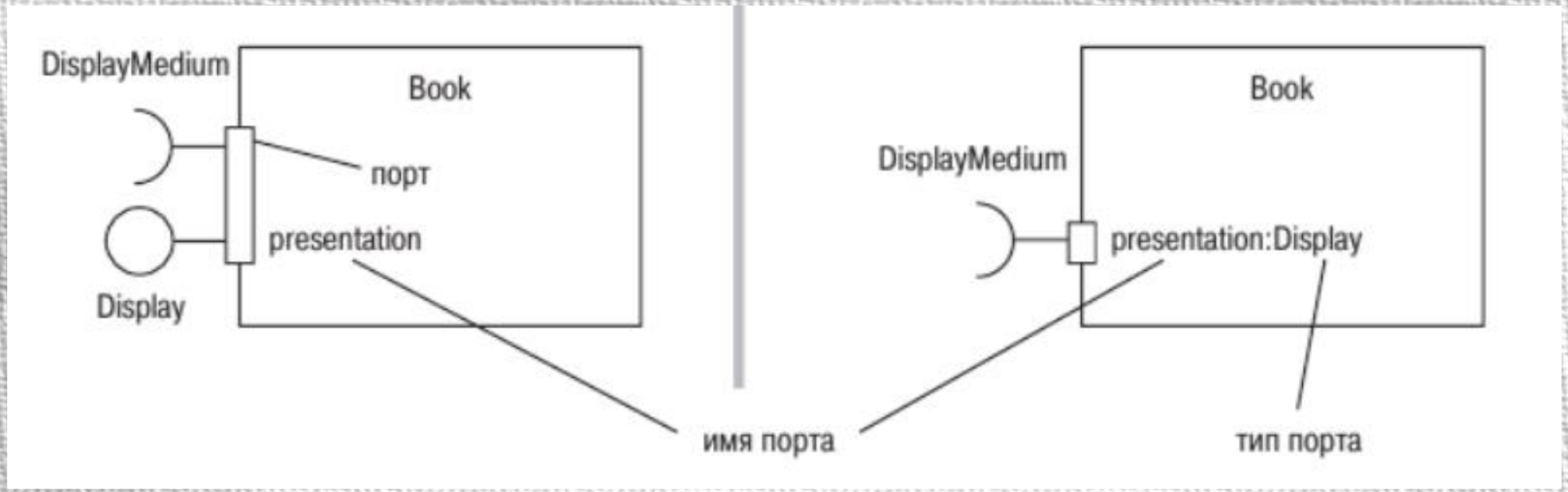


Используя *нотацию "чупа-чупс"*, появившуюся в UML 2, эту же модель можно изобразить лаконично, симметрично и просто.



Порты компонента

Порт группирует **семантически** связанный набор предоставляемых и требуемых интерфейсов. Он указывает на конкретную точку взаимодействия классификатора и его окружения.



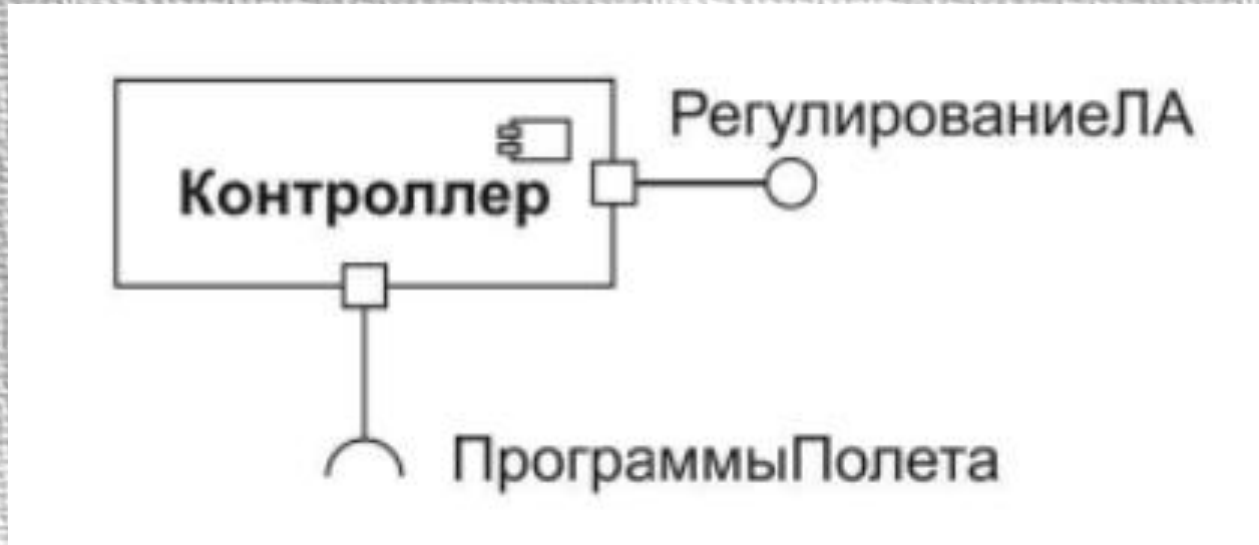
Пример на рисунке иллюстрирует нотацию порта. Здесь показан класс **Book**, имеющий порт ***presentation*** (представление). Этот порт состоит из требуемого интерфейса **DisplayMedium** (устройство отображения) и предоставляемого интерфейса **Display** (отображение). Имя порта является необязательным.

На рисунке показано два варианта нотации порта:

1. слева – обычная,
2. справа – более краткий альтернативный вариант.

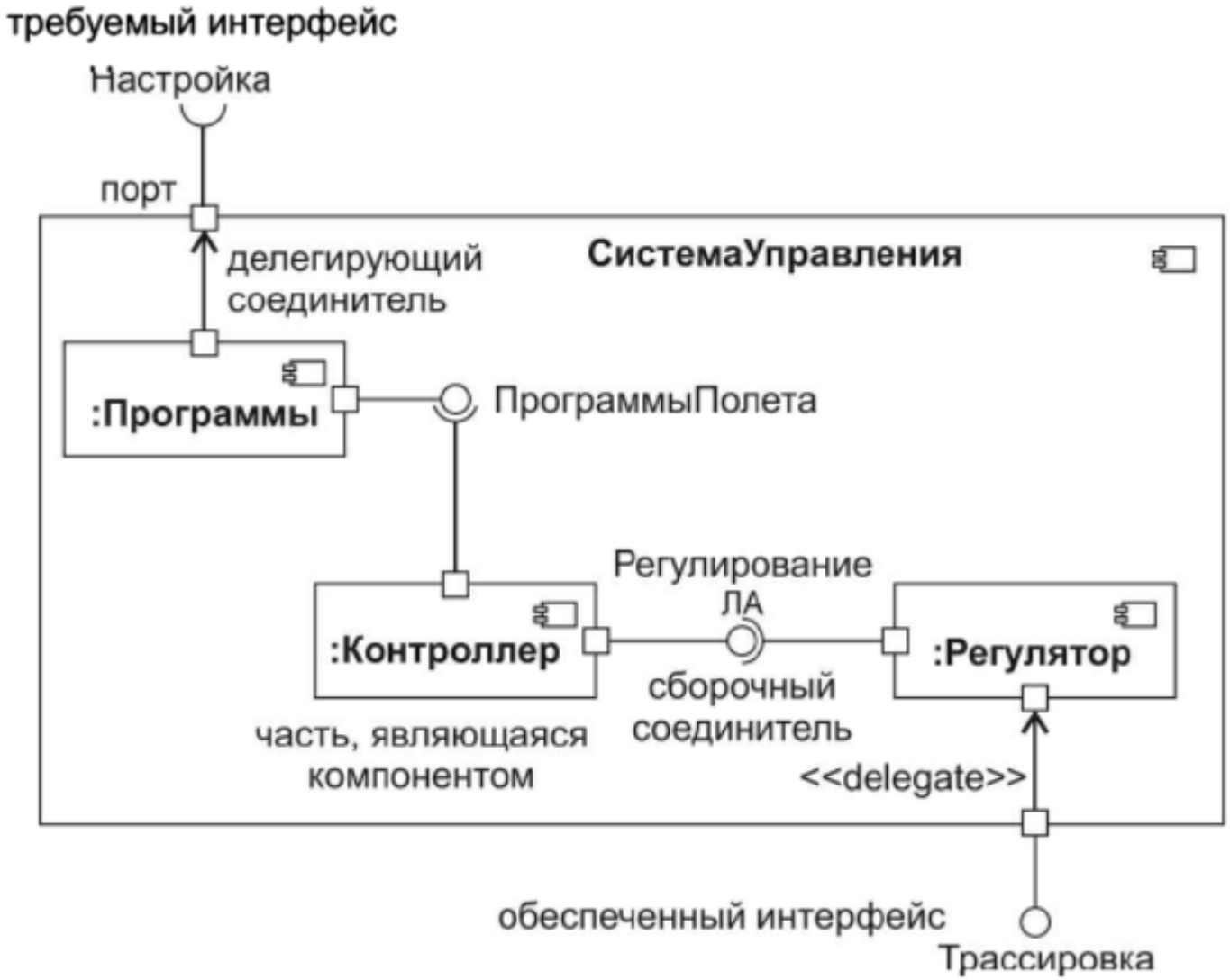
Порт является окном в капсулу компонента. Через окно происходит взаимодействие внешней среды с закрытым содержимым компонента.

Порт изображается как маленький квадрат на границе символа компонента. К символу порта может быть присоединён как обеспеченный, так и требуемый интерфейс. Соединение показывается сплошной линией.



Обеспеченный интерфейс изображает услуги, которые внешний клиент может запросить через данный порт, а требуемый интерфейс — услуги, получаемые от какого-либо другого компонента через порт. С одним портом могут быть связаны несколько интерфейсов. Каждый порт может иметь имя, задающее его индивидуальность. Имя компонента вместе с именем порта поддерживает адресные взаимодействие компонентов.

Внутренняя структура компонента

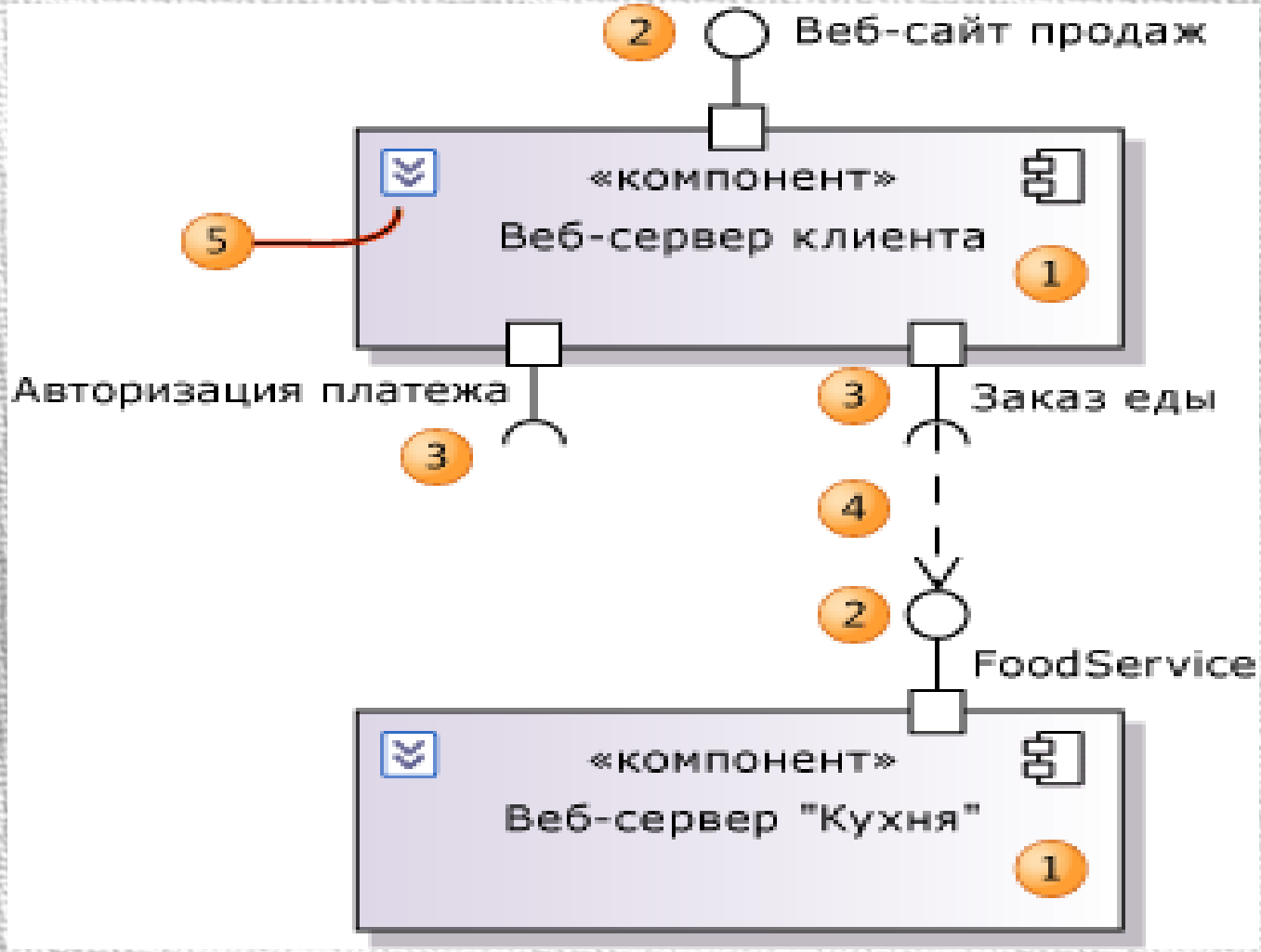


Отношение Делегирование

Делегирующий соединитель связывает внешний порт компонента с портом одного из его внутренних субкомпонентов. Делегирующие соединители поддерживают реализацию сложных операций с помощью внутренних субкомпонентов. Делегирующие соединители рисуются в виде сплошных стрелок с обычными наконечниками, связывающих порт на границе внешнего компонента с символом интерфейса или портом субкомпонента. Стрелку делегирования можно помещать стереотипом <<delegate>>.

Есть ли порт поддерживает сложный интерфейс, включающий как требуемый, так и обеспеченный интерфейсы, направление соединителя не указывается.

ПРИМЕР 1


2
Веб-сайт продаж
«компонент»
Веб-сервер клиента
1
5
Авторизация платежа
3
3
Заказ еды
4
2
FoodService
«компонент»
Веб-сервер "Кухня"
1

ОТОБРАЖЕНИЕ ПОРТОВ КОМПОНЕНТА

Порт (❷, ❸) представляет группу сообщений или вызовов операций, входящих в компонент или выходящих из него. Группа описывается интерфейсом, который определяет тип порта. Порт может либо предоставлять, либо требовать интерфейс.

Порт с *предоставленным* интерфейсом (❷) предоставляет операции, реализуемые компонентом, которые также могут использоваться другими компонентами.

В качестве примеров можно назвать пользовательский интерфейс, веб-службу или коллекцию функций на любом языке программирования.

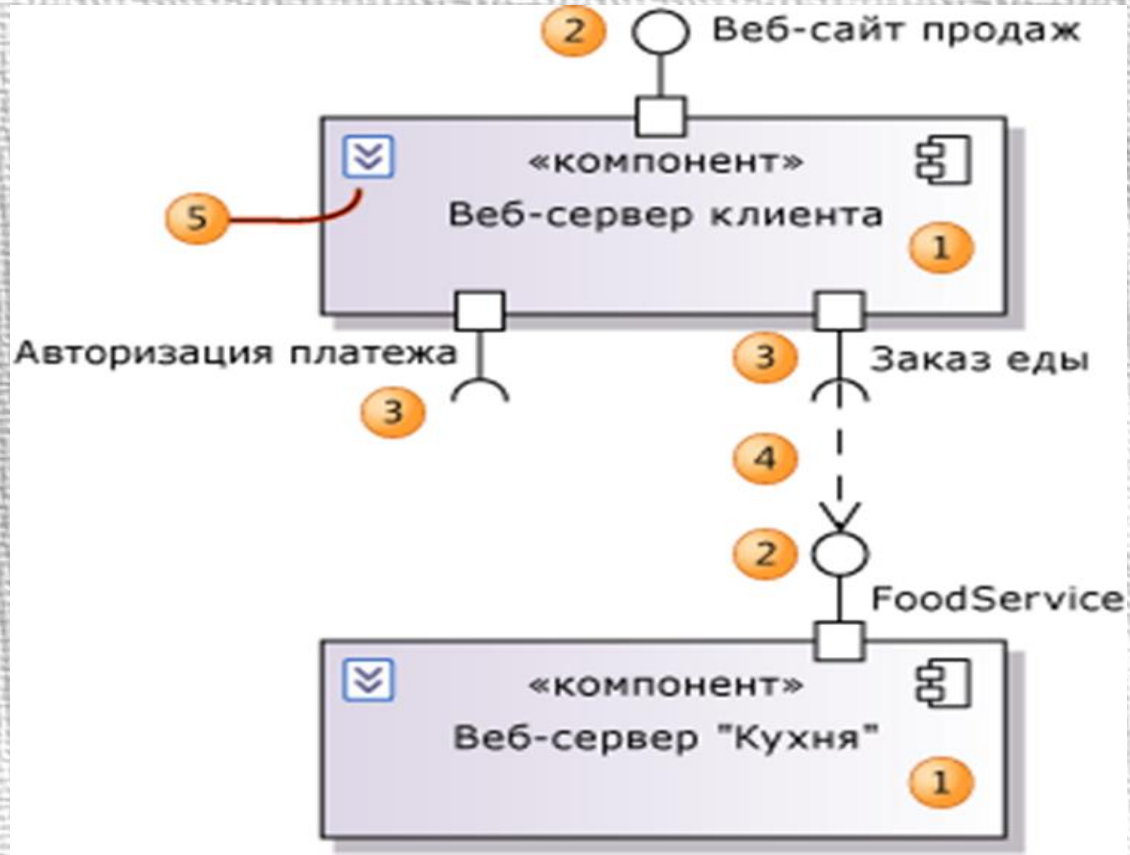
Порт с *требуемым* интерфейсом (❸) представляет требование компонента к группе операций или служб, которые должны быть предоставлены другими компонентами или внешними системами.

Например, веб-браузер требует веб-серверы, а надстройка приложения требует службы из приложения.

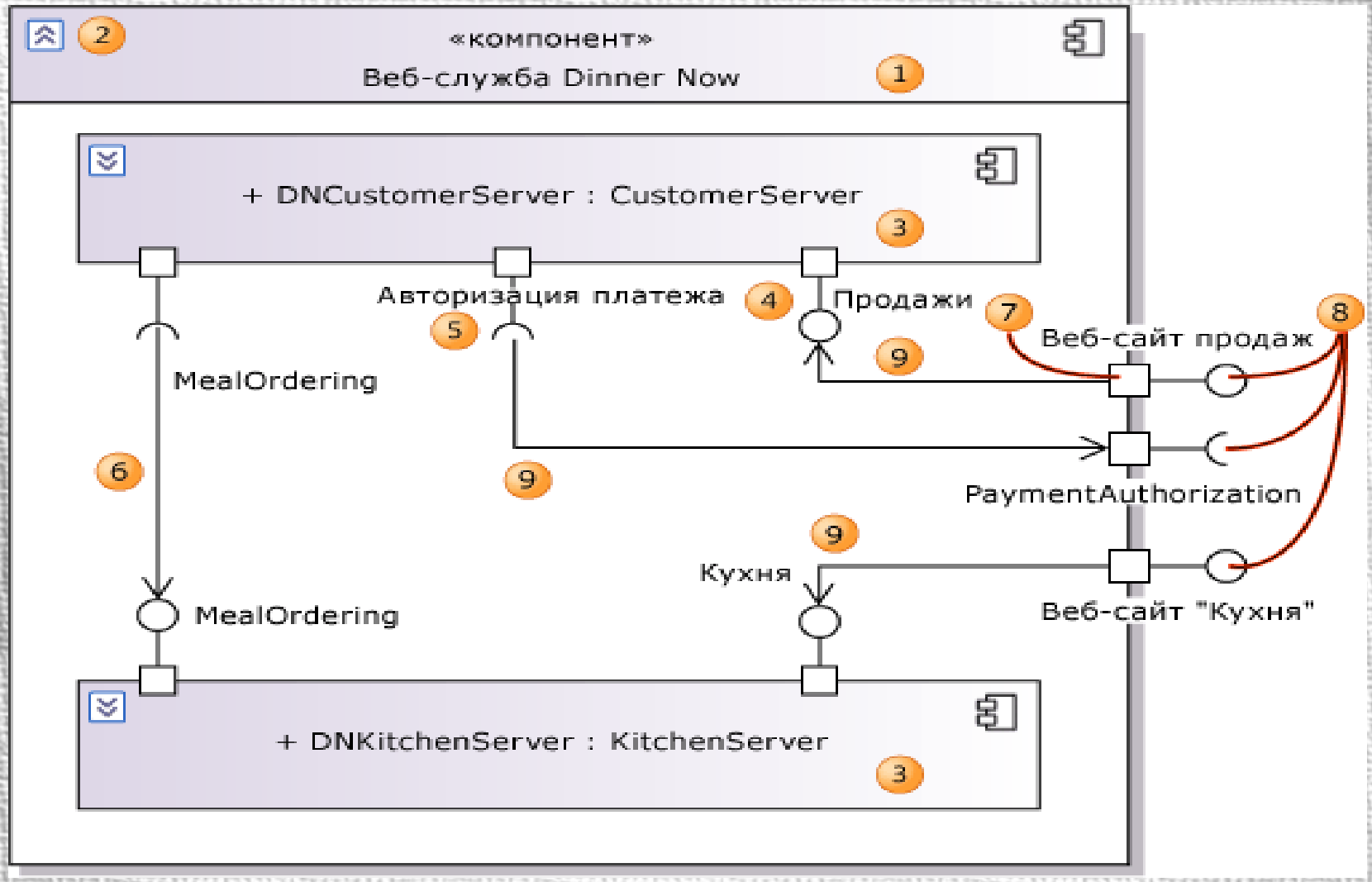
Компонент может иметь неограниченное число портов.

Ссылки между компонентами

Используйте зависимость ❹, чтобы показать, что требования одного компонента можно удовлетворить операциями или службами, предоставленными другим компонентом.



Отображение внутренних частей компонента



Можно разместить части (❸) в компоненте (❶), чтобы показать, что он состоит из более мелких компонентов, взаимодействующих друг с другом.

Схема на иллюстрации показывает, что каждый экземпляр веб-службы Dinner Now содержит один экземпляр сервера "Клиенты" и один экземпляр сервера "Кухня".

Часть — это свойство родительского компонента. Отношения между ними можно сравнить с тем, как атрибут принадлежит к обычному классу. Часть имеет собственный тип, который, как правило, также является компонентом. Метка части имеет ту же форму, что и обычный атрибут.

+ partName : TypeName

Внутри родительского компонента каждая часть показывает предоставляемые и требуемые интерфейсы, определенные для ее типа ❹,❺. Операции или службы, требуемые одной частью, могут быть предоставлены другой. Можно использовать соединители *Сборки части*, чтобы показать, как части соединены друг с другом ❻.

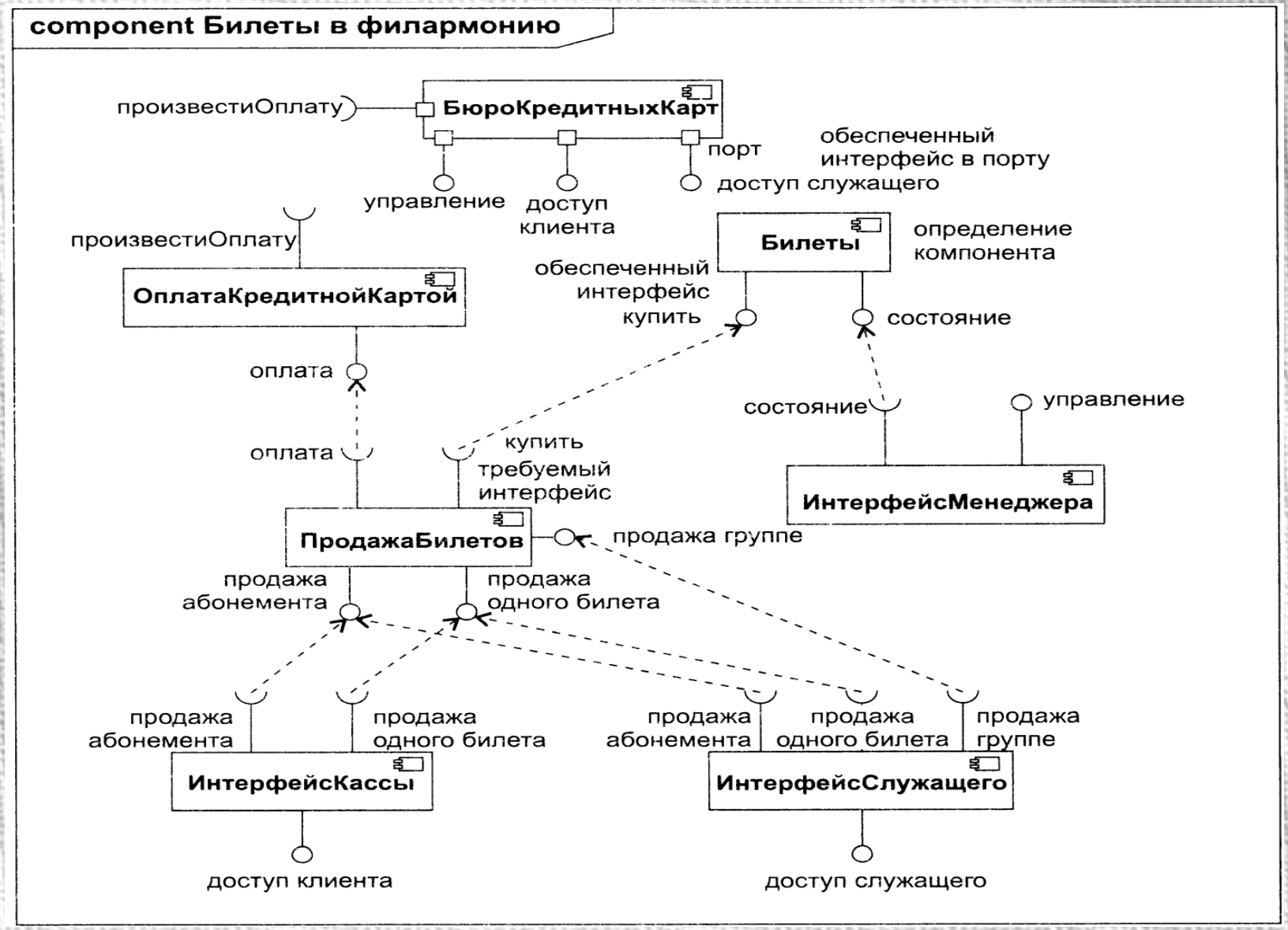
Также можно показать, что одна из частей родительского компонента фактически предоставляет или требует его интерфейс. Можно подключить порт родительского компонента к порту внутренней части, воспользовавшись отношением ***Делегирование*** ❾. Оба порта должны относиться к одному виду (предоставленные или требуемые) и иметь совместимые типы интерфейса.

Новую часть можно создать либо с помощью нового типа, либо из существующего типа.

ПРИМЕР 2

На рисунке ниже приведена диаграмма компонентов для системы продажи билетов в филармонию. Как видим, можно купить один билет на концерт, произвести групповую покупку, приобрести абонемент на весь сезон. Покупку можно выполнить в интернет-кассе филармонии или в ее офисе. Правда, групповая покупка осуществляется только в офисе. Оплата производится с помощью кредитной карты. Пунктирными линиями здесь показаны стрелки зависимости требуемых интерфейсов от обеспеченных интерфейсов. Впрочем, от стрелок можно отказаться при использовании нотации шарик и гнездо (чупа-чупс.

Диаграмма компонентов показывает компоненты в системе, то есть программные подсистемы, из которых создается программная архитектура, а также зависимости между компонентами.



- Узел
- Спецификация развертывания
- Отношение зависимости

ДИАГРАММА РАЗВЕРТЫВАНИЯ
(DEPLOYMENT DIAGRAM)

АРТЕФАКТЫ

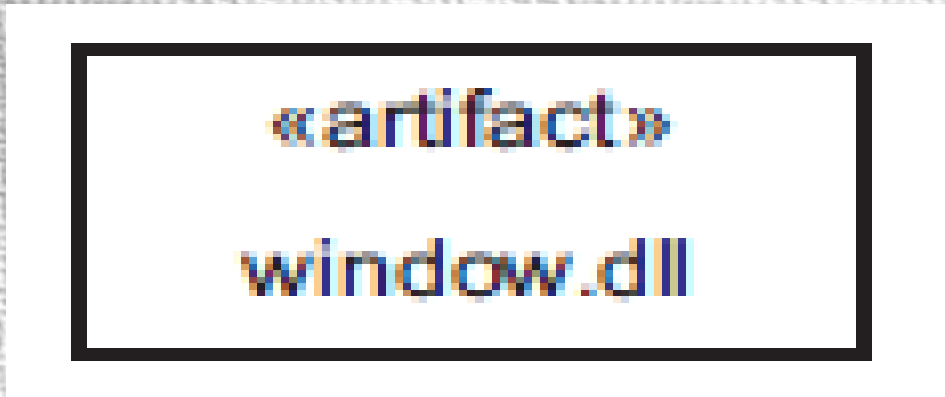
Аналогом компонента в смысле сборочного программирования является понятие артефакта в UML. Причем не любого артефакта, а только некоторых из его стереотипов.

К элементам программной системы, а, следовательно, и к артефактам, могут относиться исполняемые файлы, исходные тексты программ, веб-страницы, справочные файлы, сопроводительные документы, файлы с данными, модели и многое другое, являющееся физическими элементами информации.

Другими словами, артефактами являются те информационные элементы, которые тем или иным способом используются при работе программной системы и входят в ее состав.

Артефакт *(artifact)* – это **физическая** и замещаемая часть системы, несущая физическую информацию («биты»).

Обычно артефакт представляет собой физический пакет с исходным или исполняемым кодом. Изображается как прямоугольник, снабженный ключевым словом «artifact», расположенным над его именем.



Стандартные стереотипы артефактов

Стереотип	Описание
«file»	Файл любого типа, хранимый в файловой системе
«document»	Артефакт, представляющий файл (документ), который не является ни файлом исходных текстов, ни исполняемым файлом
«executable»	Выполнимая программа любого вида. Подразумевается по умолчанию, если никакой стереотип не указан
«library»	Статическая или динамическая библиотека
«script»	Файл, содержащий текст, допускающий интерпретацию соответствующими программным средствами
«source»	Файл с исходным кодом программы

МАНИФЕСТАЦИЯ

Самым важным аспектом использования понятия артефакта в UML является то, что артефакт может участвовать в отношении манифестации.

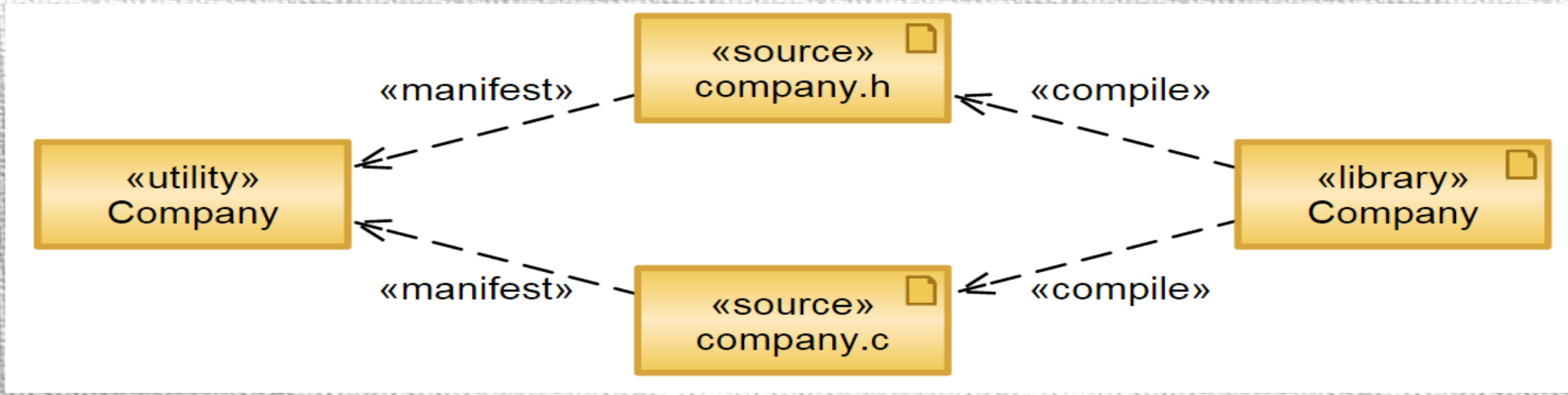
Манифестация **– это отношение зависимости со стереотипом «manifest», связывающее элемент модели (например, класс или компонент) и его физическую реализацию в виде артефакта.**

Вообще говоря, отношение манифестации – это отношение типа "*многие ко* многим", один элемент модели может быть реализован многими артефактами, и один артефакт может участвовать в реализации многих элементов модели.

Манифестацию графически изображают отношением зависимости со стереотипом «**manifest**» от артефакта к реализуемой сущности.

Поскольку манифестация – это отношение типа "многие ко многим", для полного описания отношения манифестации могут потребоваться несколько отношений зависимости в модели.

Ниже представлен класс **Company**, который связан отношением манифестации (зависимость со стереотипом «*manifest*») с двумя артефактами со стереотипом «*source*», которые в свою очередь определяют артефакт времени выполнения динамическую библиотеку (со стереотипом «*library*») **Company**.



УЗЕЛ

Узел **(node) – это физический вычислительный ресурс, участвующий в работе системы.**

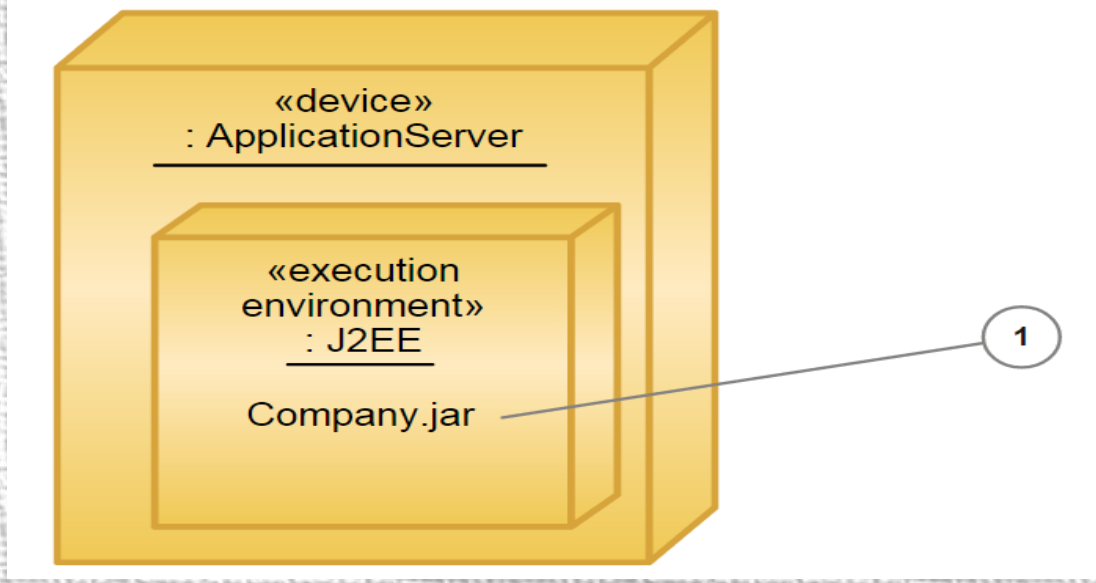
В UML предусмотрено два стереотипа для узлов

1. «***executionEnvironment***»
2. «***device***».

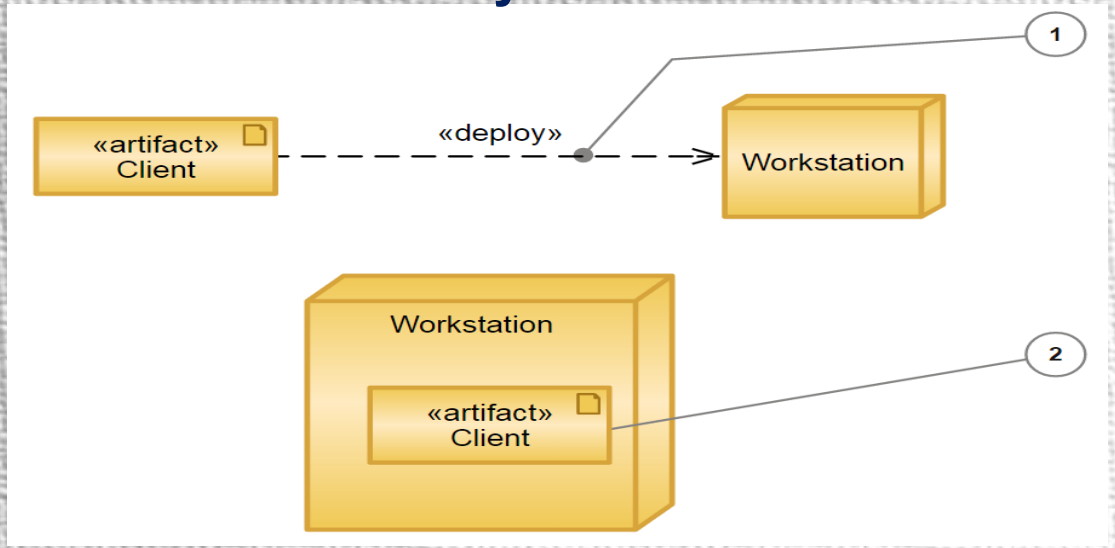
Узел со стереотипом «***executionEnvironment***» позволяет моделировать аппаратно-программную платформу, на которой происходит выполнение приложения. Примерами среды выполнения являются: операционная система, система управления базами данных и т.д.

Узел со стереотипом «***device***» также моделирует аппаратно-программную платформу, но допускает возможность вложение одного узла в другой, как это показано на следующем рисунке.

Нотация узла



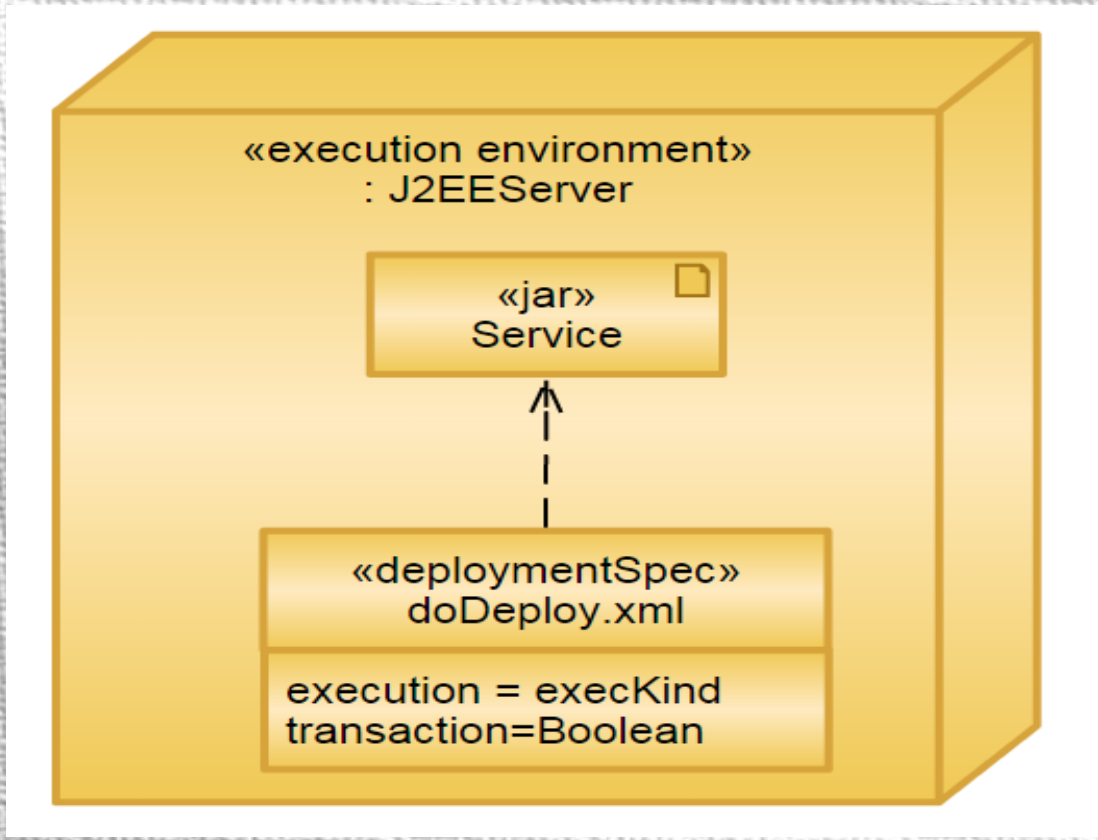
Нотации размещения артефакта на узле



Артефакты системы во время ее работы размещаются на узлах, что графически выражается либо их перечислением внутри узла 1 (см. рисунок выше), либо отношением зависимости со стереотипом «deploy» между артефактом и узлом 1 (см. рисунок ниже), либо изображением артефакта внутри изображения узла 2 (см. рисунок ниже). Все варианты нотации равноправны.

Использование спецификации развертывания

Если при размещении артефакта на узле важную роль играют специфичные для программной среды параметры, то они могут быть заданы посредством спецификации развертывания (***deployment specification***).



Спецификация развертывания изображается, как и классификатор (в виде прямоугольника), но со стереотипом «**deploymentSpec**» и связывается отношением зависимости с артефактом.

НОТАЦИЯ ДИАГРАММЫ РАЗВЕРТЫВАНИЯ

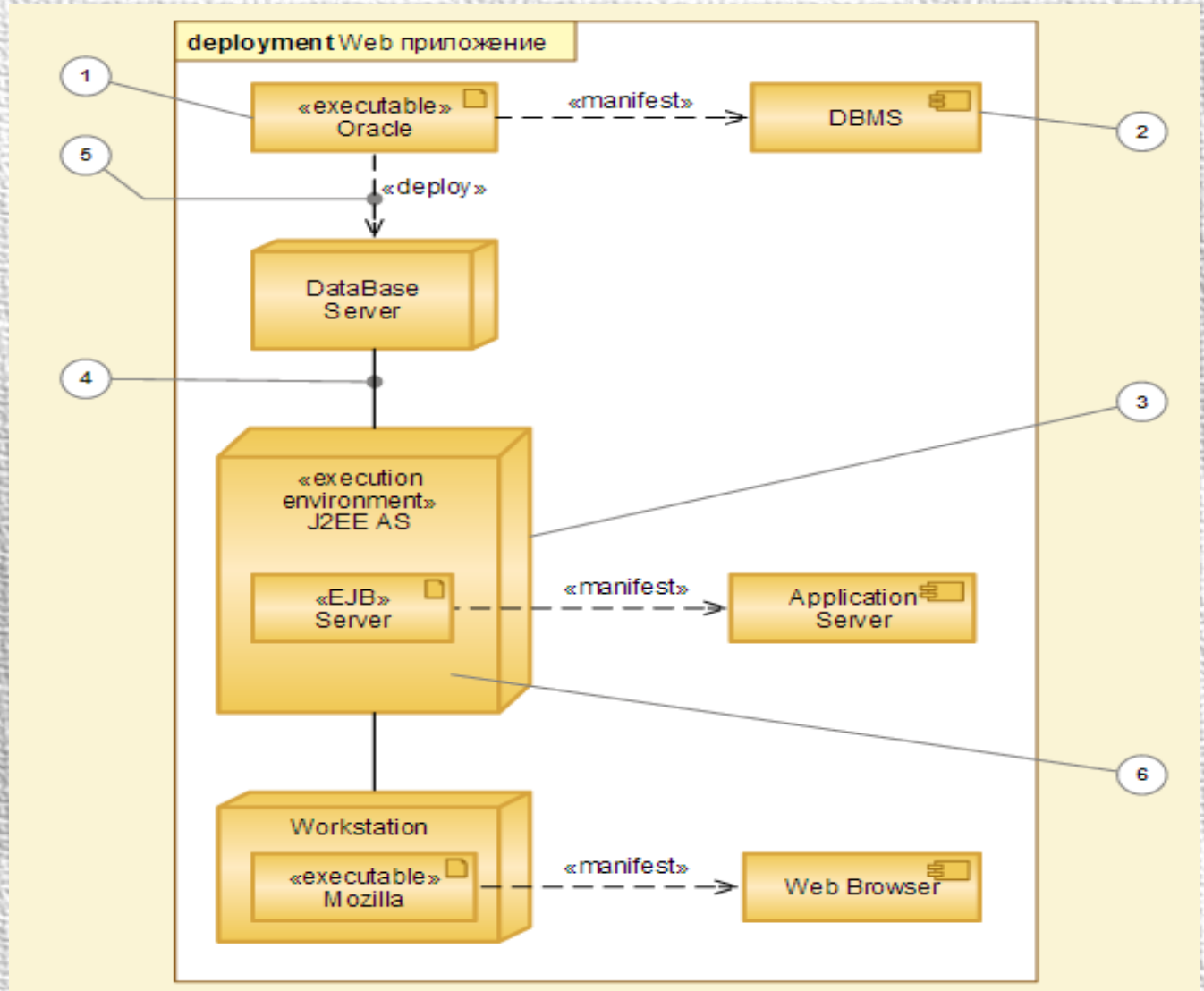


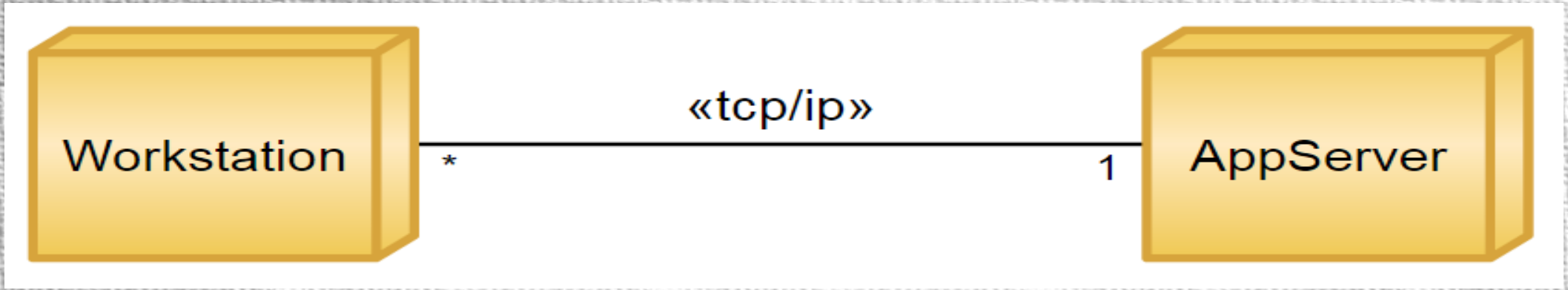
Диаграмма развертывания (deployment diagram) наряду с отображением состава и связей элементов системы показывает, как они физически размещены на вычислительных ресурсах во время выполнения.

Таким образом, на диаграмме развертывания, по сравнению с диаграммой компонентов, добавляется два типа сущностей: артефакт **1**, который является реализацией компонента **2** и узел **3** (может быть как классификатор, описывающий тип узла, так и конкретный экземпляр), а также отношение ассоциации между узлами **4**, показывающее, что узлы физически связаны во время выполнения.

На рисунке показаны основные элементы нотации, применяемые на диаграмме размещения. Для того чтобы показать, что одна сущность является частью другой, применяется либо отношение зависимости «**deploy**» **5**, либо фигура одной сущности помещается внутрь фигуры другой сущности **6**.

ОТНОШЕНИЕ АССОЦИАЦИИ МЕЖДУ УЗЛАМИ

Если узлы связаны между собой отношением ассоциации, то это означает то же, что и в других контекстах: *возможность обмена сообщениями*. Применительно к вычислительным сетям, состоящим из узлов, ассоциация означает наличие канала связи. Если нужно указать дополнительную информацию о свойствах канала, то это можно сделать, используя общие механизмы: стереотипы (например, «tcp/ip» см. на рисунке ниже), ограничения и именованные значения.



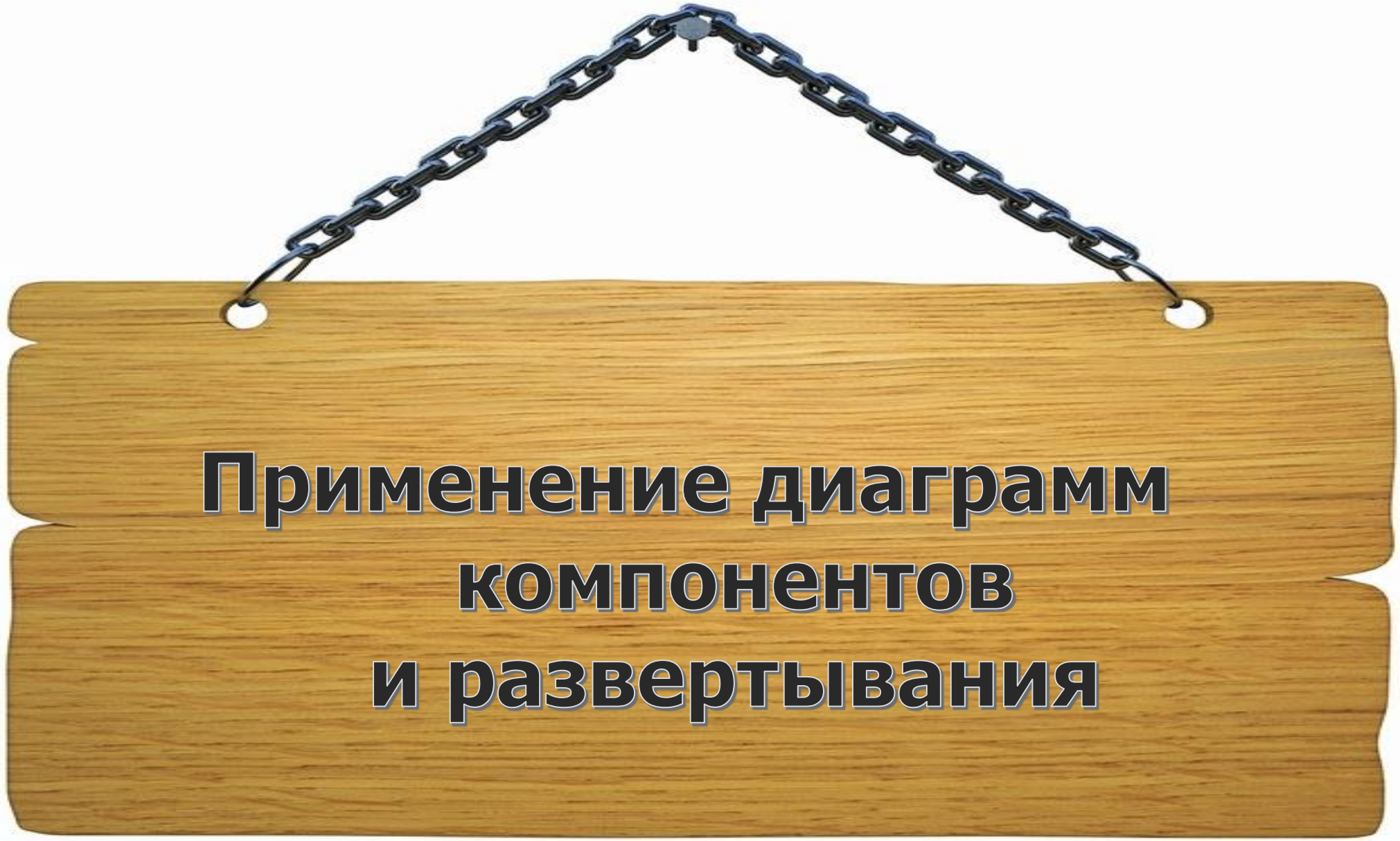
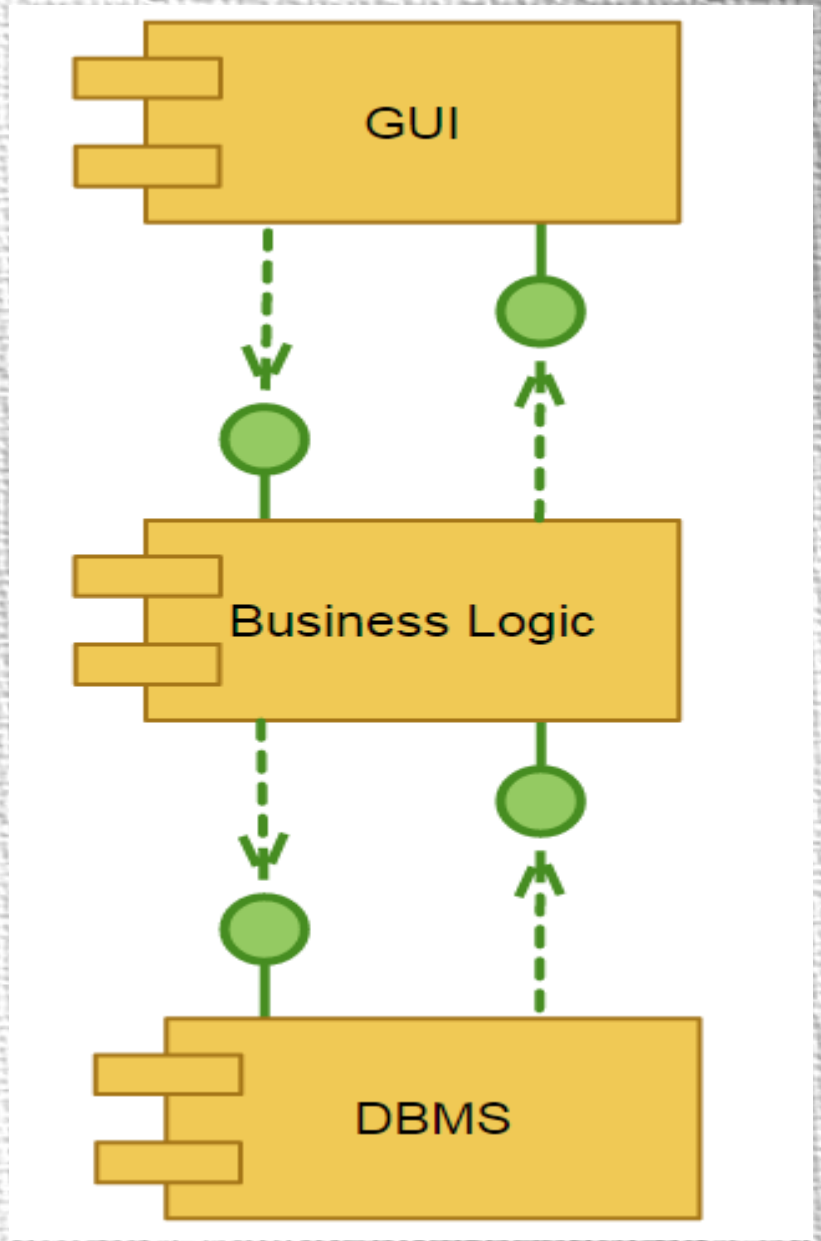

Применение диаграмм
компонентов
и развертывания

Диаграмма компонентов ИС ОК, выполненная в нотации UML 1

Какие интерфейсы, компоненты и артефакты можно выделить в ИС ОК, и как целесообразно разместить разработанное программное обеспечение на вычислительных узлах?

Основное назначение проектируемой информационной системы – хранить данные о кадрах и выполнять по указанию пользователя некоторые операции с этими данными. Анализируя состав операций, мы видим, что они сводятся к созданию, модификации и удалению хранимых элементов данных. Стандартным решением в таких ситуациях является применение готовой СУБД (DBMS – Data Base Management System).



СУБД возьмет на себя все функции по непосредственному манипулированию данными: создание, удаление и поиск записей в таблицах и т.д. Реализация операций нашей информационной системы отдела кадров сводится к некоторой последовательности элементарных операций с данными. Например, операция перевода сотрудника с одной должности на другую, видимо, потребует изменения в трех элементах данных: в данных о сотруднике и в данных о старой и новой должностях. Однако целесообразно ли считать, что определение и выполнение самой последовательности элементарных операций с данными также является прерогативой выделенного нами компонента – СУБД? Общепринятая практика отвечает на этот вопрос отрицательно.

По многим причинам лучше выделить это в отдельный компонент, обычно называемый бизнес-логикой. Кроме этого, мы должны предположить, что в нашей системе должен быть компонент, ответственный за пользовательский интерфейс. В первом приближении мы приходим к структуре компонентов, приведенной ниже, которая называется «трехуровневая архитектура».

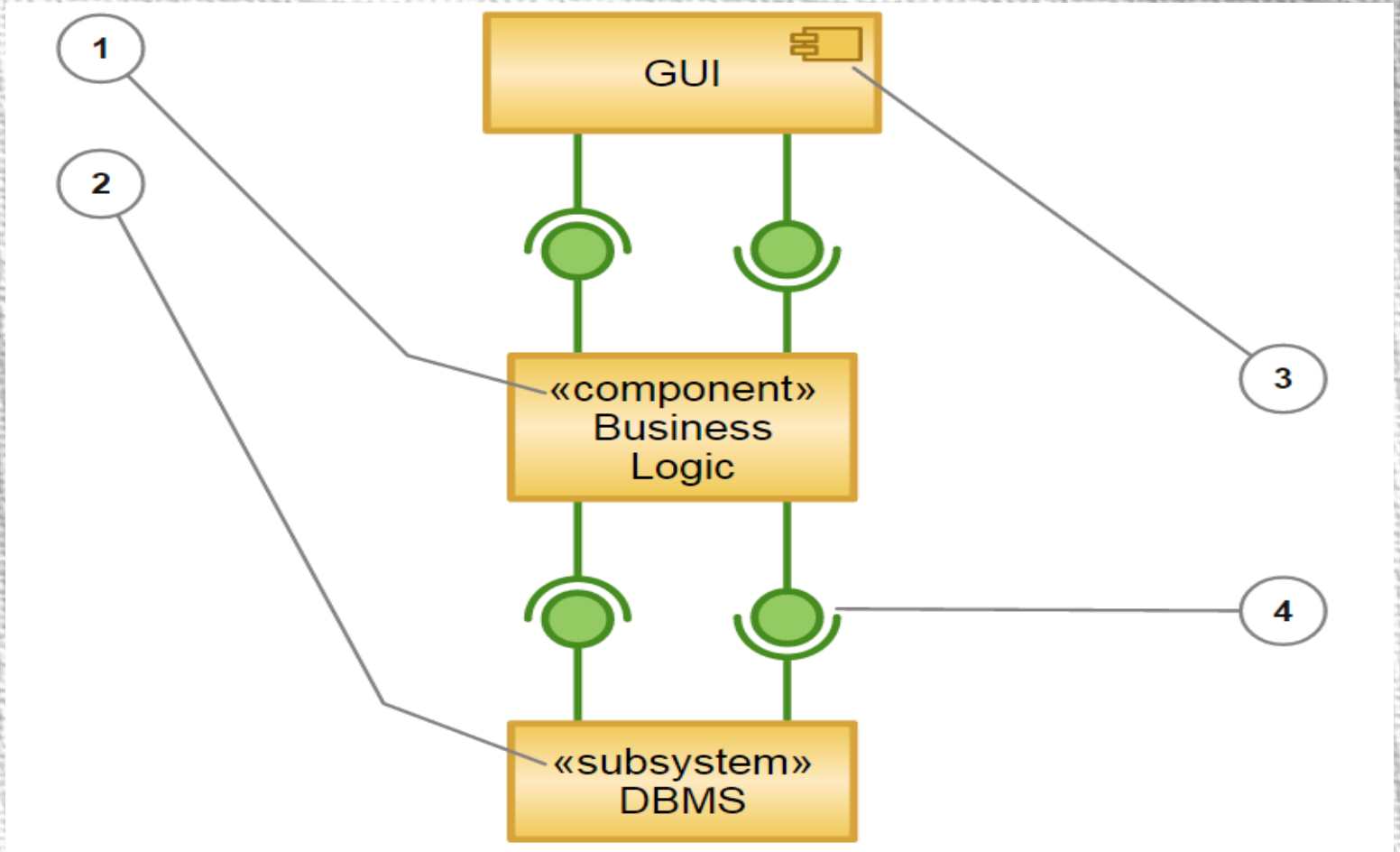
Изменения в нотации диаграмм компонентов

В версии UML 2 произошли следующие изменения в нотации диаграмм компонентов.

1. Во-первых, компоненты, как и любой классификатор, можно изображать единообразно, в виде прямоугольников, в которых, указывается либо стереотип «component» ❶, либо один из уточняющих стереотипов, приведенных в табл. *Стандартные стереотипы компонент* ❷, либо соответствующий значок в правом верхнем углу прямоугольника ❸.
2. Во-вторых, требуемые и обеспеченные интерфейсы можно изображать с помощью нотации "чупа-чупс" ❹, так что отношение взаимодействия компонентов через некоторый интерфейс выглядит естественным и симметричным

Диаграмма компонентов ИС ОК, выполненная в нотации UML 2

Сказанное иллюстрирует рисунок, на котором указаны те же сущности и отношения, что и на рисунке выше.



ИЗМЕНЕНИЯ В ТЗ ДЛЯ ОК

Приведенный пример диаграммы компонентов достаточно тривиален, поэтому приведем еще один пример, связанный с информационной системой отдела кадров, в котором постараемся показать, что диаграммы компонентов являются достаточно выразительным средством проектирования архитектуры.

Допустим, что в проектируемой информационной системе отдела кадров требуется разграничить права на выполнение операций и доступ к данным для различных категорий пользователей.

У нашего приложения два действующих лица, т.е. две категории пользователей.

Допустим, что достаточно разграничить права на уровне категорий пользователей. Тогда можно поступить следующим образом: сделать просто два различных приложения (или, как обычно говорят в таких случаях, два автоматизированных рабочих места – АРМа). Пользователи, имеющие доступ к АРМу в целом, могут выполнять все операции АРМа и, таким образом, имеют те и только те права на доступ к данным, которые обеспечиваются операциями, реализованными в АРМе.

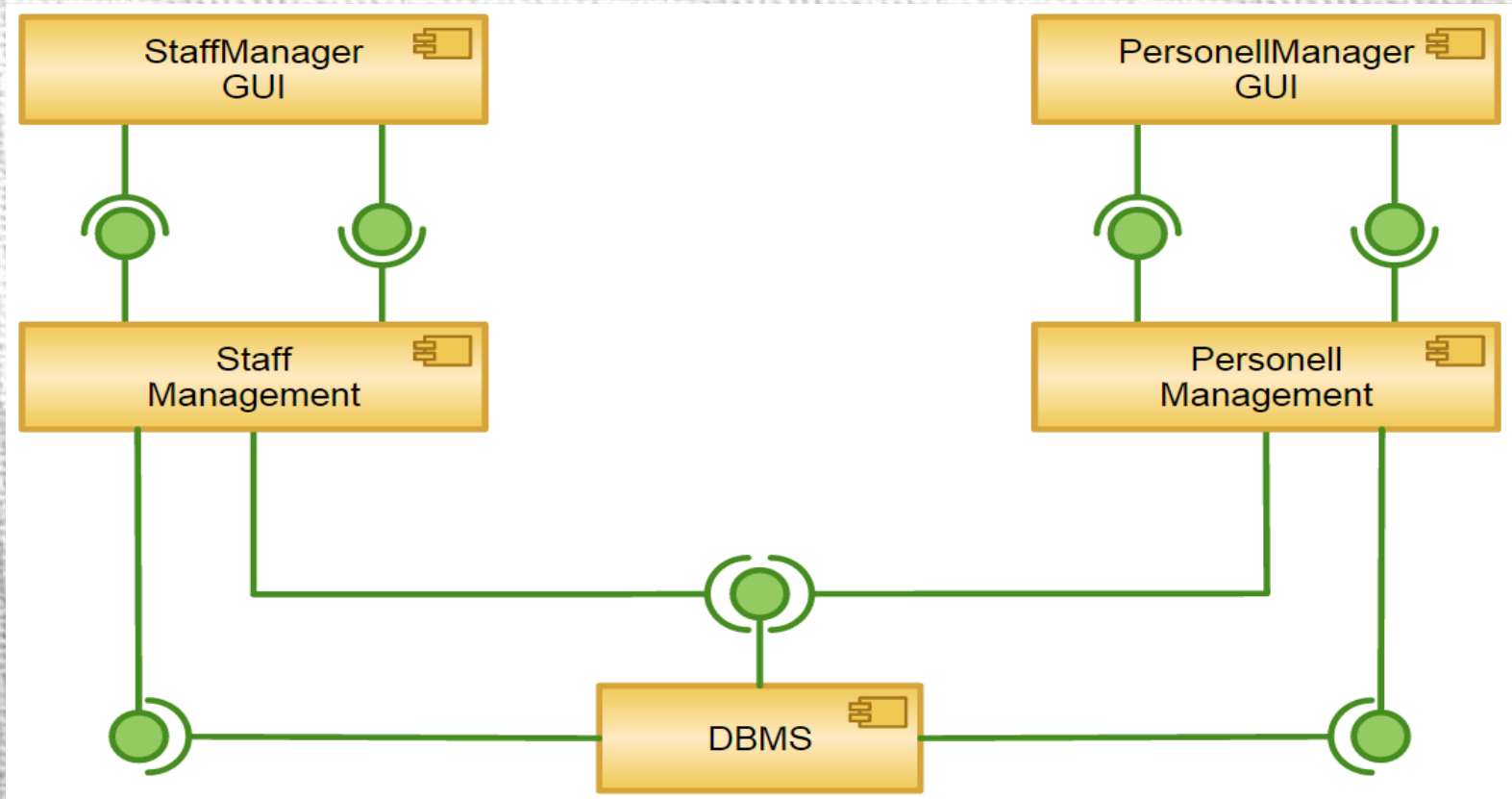
ИЗМЕНЕНИЯ В ТЗ ДЛЯ ОК

Для приложения типа информационной системы отдела кадров такого решения практически достаточно.

Таким образом, разграничение прав доступа к данным переносится на уровень доступа к компьютерам и установленным на них приложениям, а это уже проблемы операционных систем и служб безопасности предприятия, о которых в информационной системе отдела кадров можно не заботиться.

Принятое решение легко выражается на диаграмме компонентов.

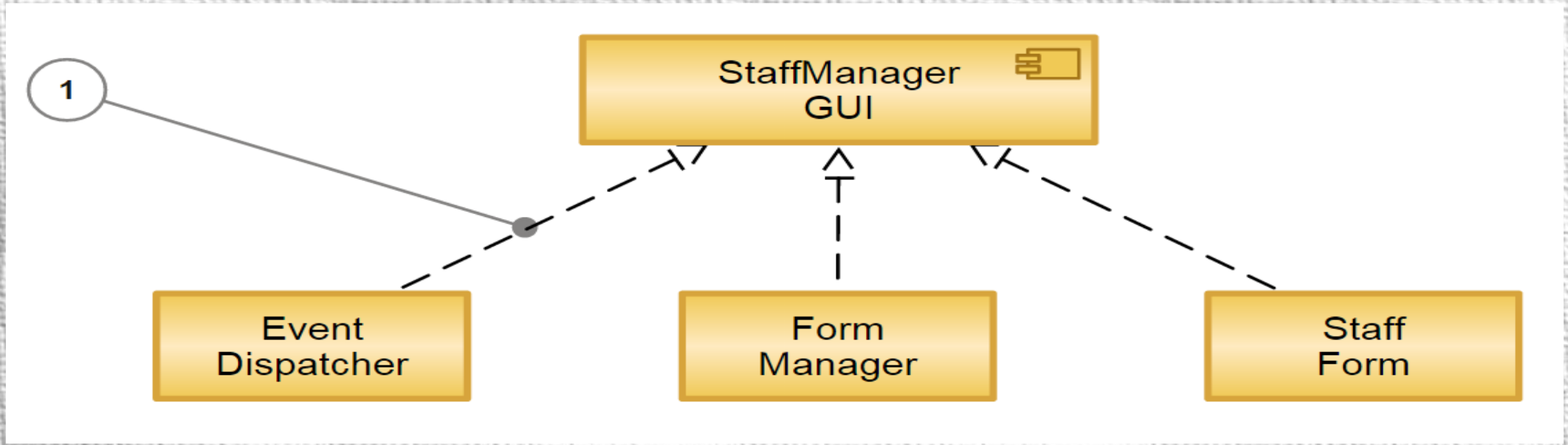
ДИАГРАММА КОМПОНЕНТОВ ИС ОК



Все что осталось сделать – это определить состав компонентов, т.е. показать какие классы в какие компоненты входят.

ДИАГРАММА КОМПОНЕНТОВ ИС ОК

Самый простой способ показать связь между компонентом и входящими в него классами – использовать отношение реализации 1, как это представлено ниже.



Другой способ определения состава компонента

Другой способ определения состава компонента – рассматривать его как структурированный классификатор и использовать диаграмму внутренней структуры (см. ниже).

Размещения артефактов относительно вычислительных ресурсов

Следующим структурным аспектом, который необходимо обсудить, является описание размещения артефактов относительно участвующих в работе вычислительных ресурсов.

Допустим, что мы приняли архитектуру, приведенную выше на рисунке "Диаграмма компонентов ИС ОК". Сколько компьютеров будет использоваться при работе данного приложения? На этот вопрос нужно отвечать также вопросом: а сколько пользователей будет у системы и сколько из них будут работать с приложением одновременно? Если имеется только один пользователь (или, хуже того, нашу систему установят "для галочки", а использовать не будут), то проблем нет – настольное приложение – один компьютер и диаграмма размещения не нужна.

Допустим, что у нашей системы должно быть много пользователей, и они могут работать одновременно. Тогда ответ очевиден: узлов должно быть не меньше, чем число одновременно работающих пользователей, потому что вдвоем за одним персональным компьютером обычным пользователям работать неудобно.

Скорее всего, узлов должно быть на единицу больше чем пользователей, т.к. в большинстве организаций есть специально выделенный компьютер (сервер) для хранения корпоративных данных. Там мы и поместим нашу базу данных, в расчете на то, что нужная СУБД, скорее всего, на сервере уже установлена.

Остается вопрос о размещении артефактов, реализующих бизнес-логику. Здесь возможны разные варианты:

- на компьютере пользователя,
- на промежуточной машине (сервере приложений),
- на корпоративном сервере баз данных.

Если мы остановимся на последнем варианте (который на жаргоне называется "архитектура клиент/сервер с тонким клиентом"), то получим диаграммы, приведенные на следующих двух рисунках.

Обе эти диаграммы являются диаграммами размещения, но каждая из них имеет свои особенности.

На первой диаграмме упор сделан на указание соответствия между компонентами и артефактами, выражающийся в наличии большого количества отношений зависимости со стереотипом «*manifest*» (см., например, ❶ на первой диаграмме).

Вторая диаграмма показывает отношения между артефактами, или, другими словами, определяет, какой артефакт от какого зависит, например, запрашивает данные (в качестве примера см. ❶ на второй диаграмме).

На обеих диаграммах показаны вычислительные узлы и отношения между ними (❷ на обоих диаграммах). Заметим, что на диаграмме появился дополнительный артефакт – **Help** (например, ❸ на второй диаграмме). Это документ, содержащий справочную информацию.

Диаграмма размещения ИС ОК (артефакты и компоненты)

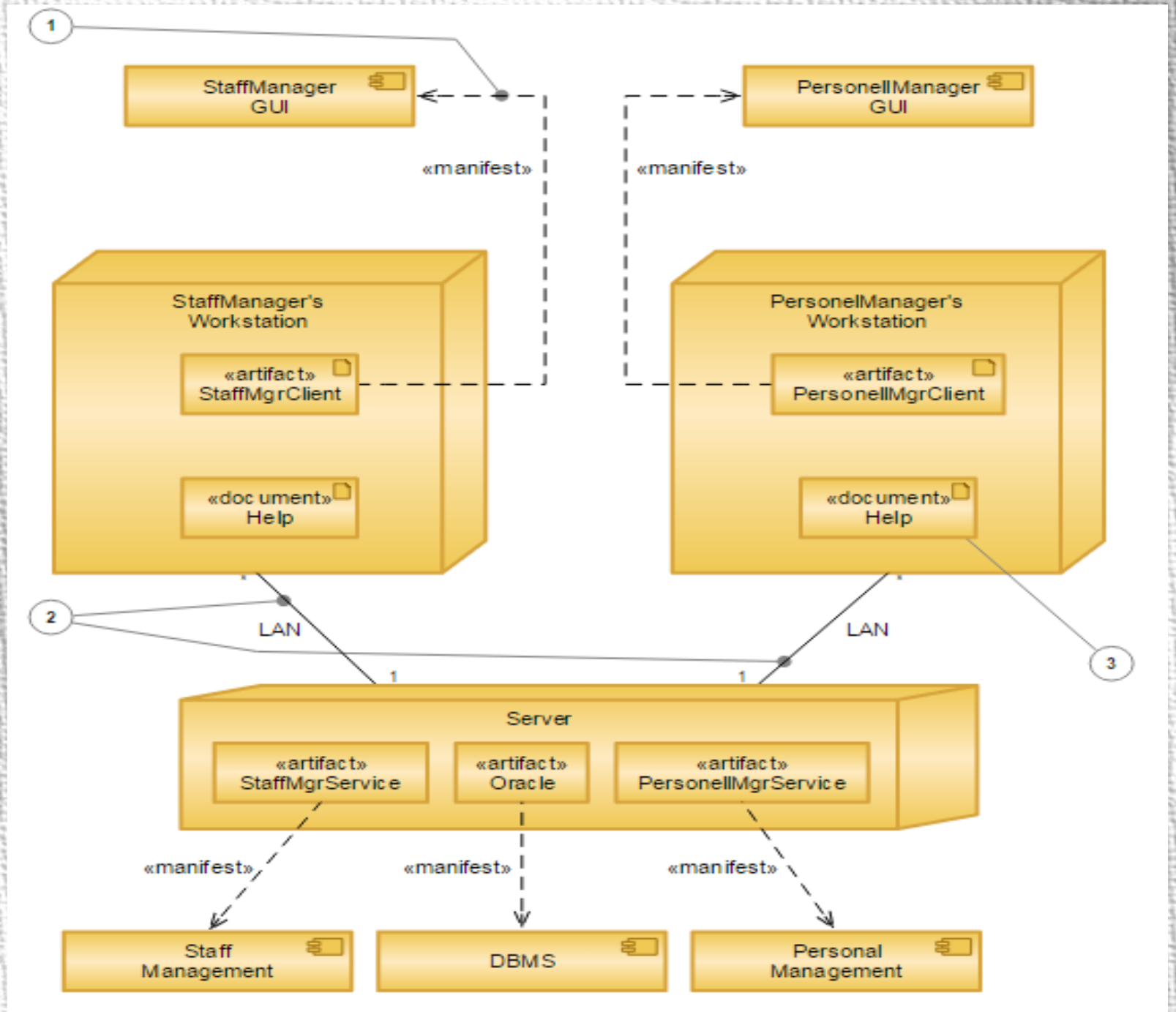
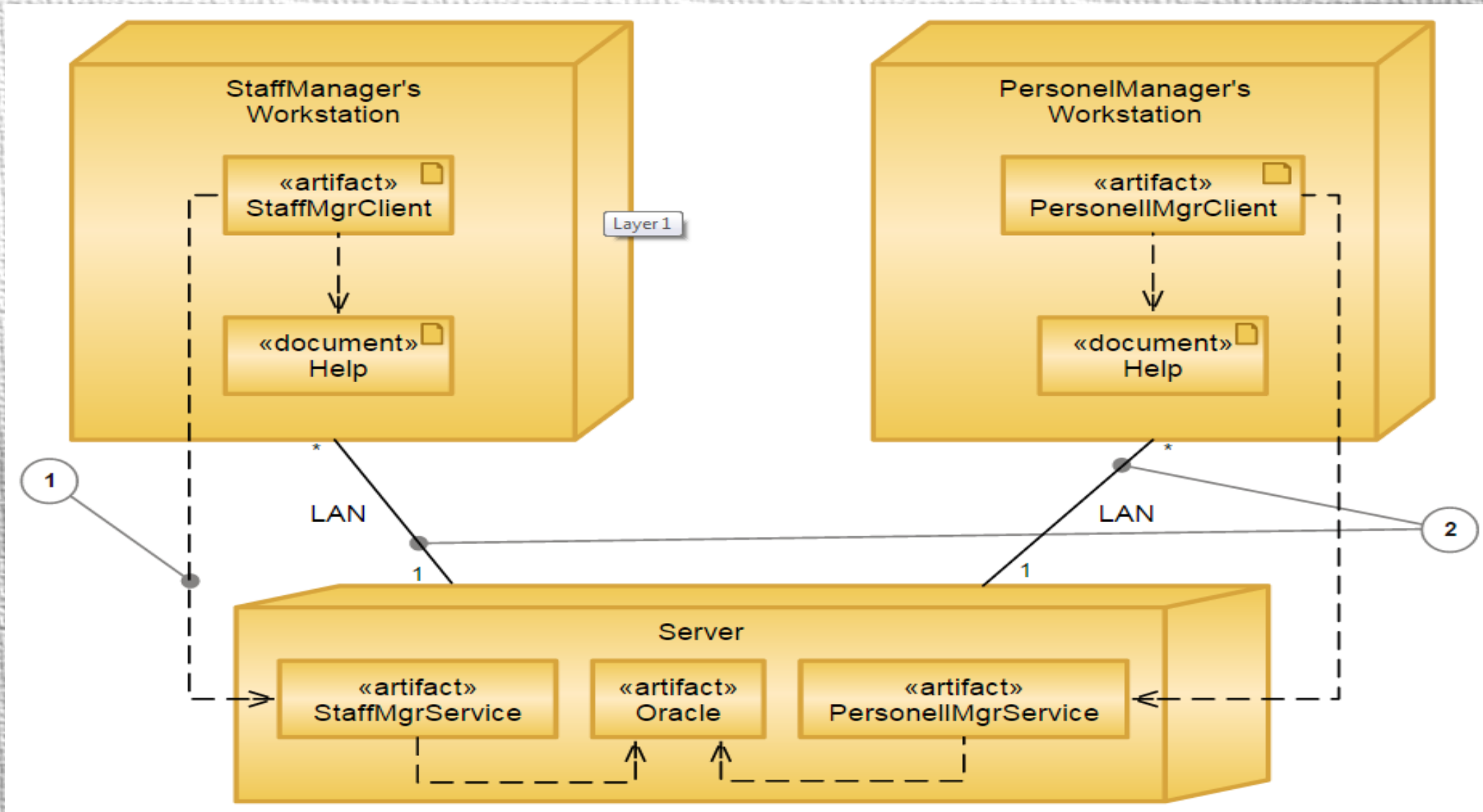


Диаграмма размещения ИС ОК (артефакты и зависимости между ними)



В каких случаях следует применять диаграммы компонентов и размещения

В случае разработки "монолитного" настольного приложения диаграммы размещения не нужны – они оказываются тривиальными и никакой полезной информации не содержат. При моделировании распределенных приложений значение диаграмм размещения резко возрастает: они являются описанием топологии развернутой системы.

Таким образом, диаграммы размещения применяются только при моделировании многокомпонентных приложений.

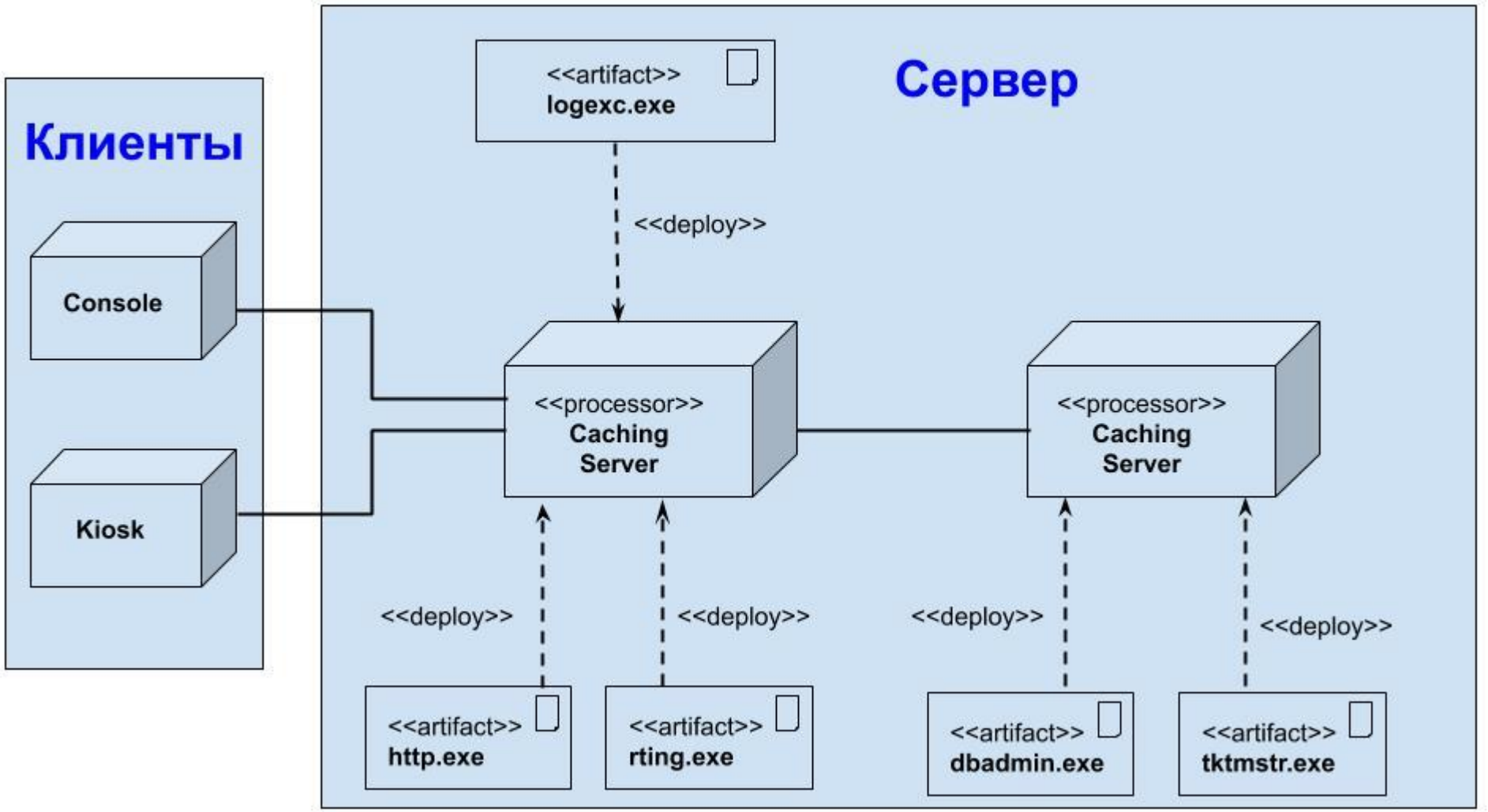
Если приложение поставляется в виде "конструктора" (набора "кубиков") из которого при установке собирается конкретный уникальный экземпляр приложения, то диаграммы размещения оказываются просто незаменимым средством. Действительно, многие современные приложения, особенно развитые системы автоматизации управления делопроизводством предприятия, поставляются в виде большого (десятки и сотни) набора артефактов, из которых "на месте" собирается нужная пользователю, часто уникальная, конфигурация.

Некоторые авторитетные источники рекомендуют использовать диаграммы размещения для управления конфигурацией не только на фазе поставки и установки программного обеспечения, но и в процессе разработки: для отслеживания версий компонентов, вариантов сборки и т.п.

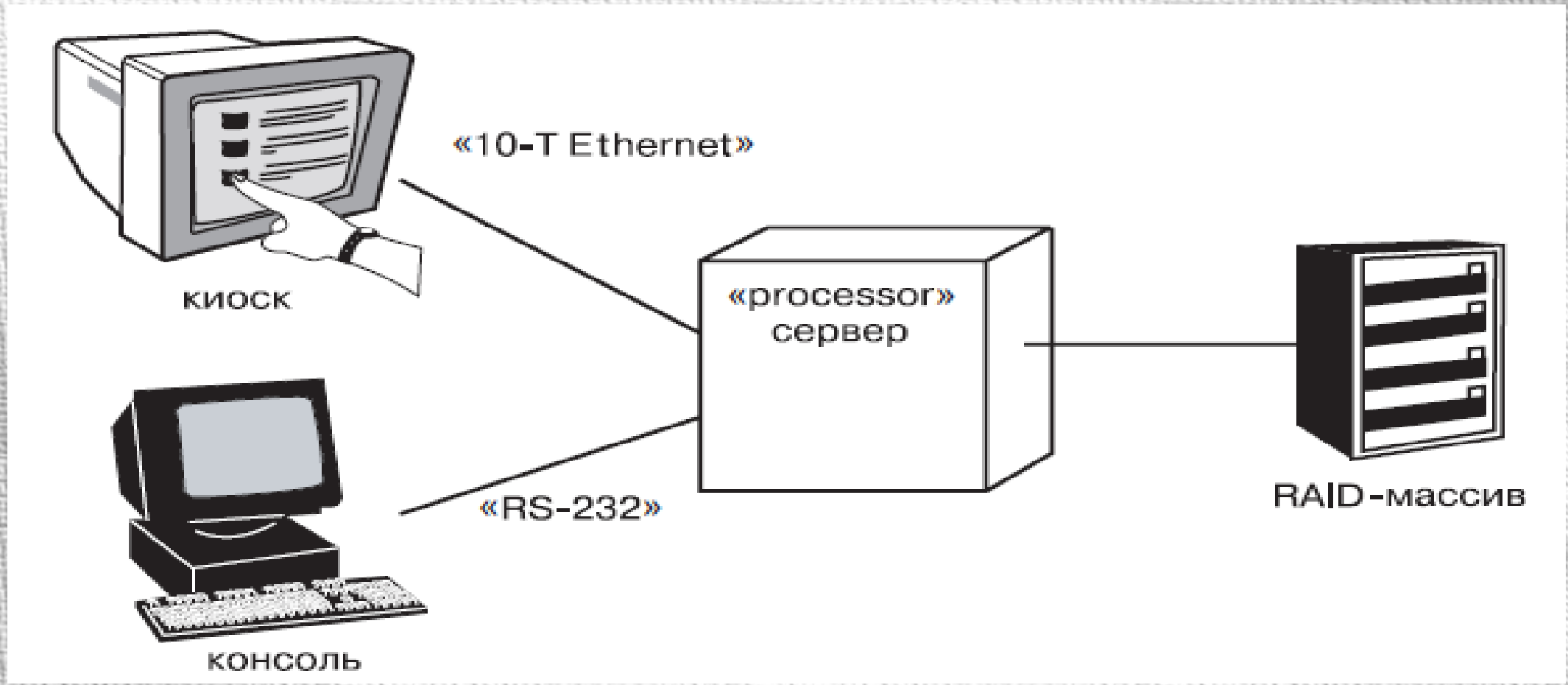
При разработке приложений, которые должны взаимодействовать с так называемыми унаследованными (legacy) приложениями и данными, без диаграмм компонентов трудно обойтись. Дело в том, что фактически единственным средством UML, позволяющим как-то описать и включить в модель унаследованные приложения и данные являются компоненты (и их интерфейсы). Сюда же относится случай моделирования доступа к данным из "неродной" СУБД.

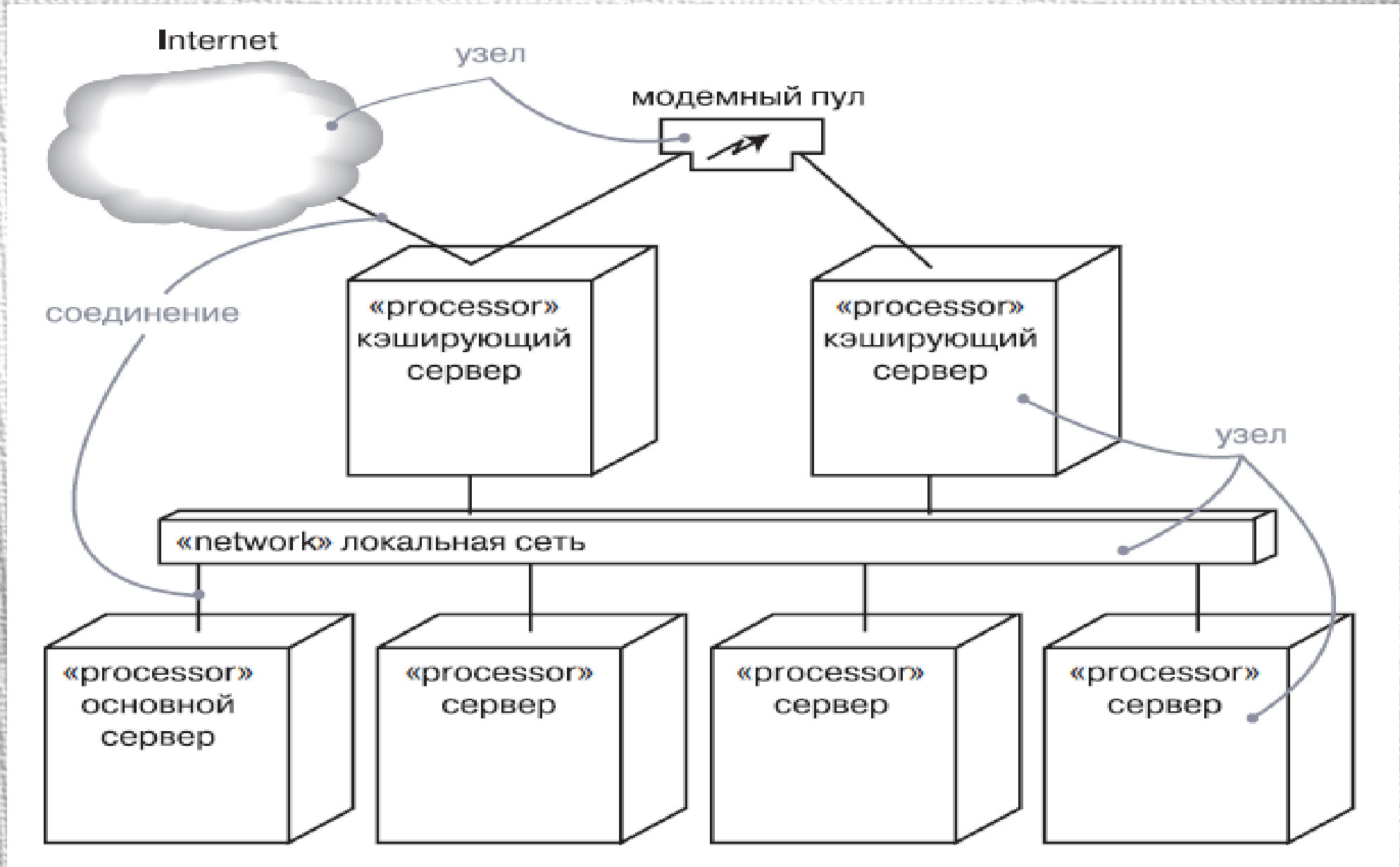
Последним примером применения диаграмм размещения является моделирование систем динамической архитектуры, то есть таких систем, которые меняют состав и количество экземпляров своих артефактов во время выполнения. Например, многие web-приложения меняют свою конфигурацию во время выполнения в зависимости от текущей нагрузки.

Диаграмма Развертывания

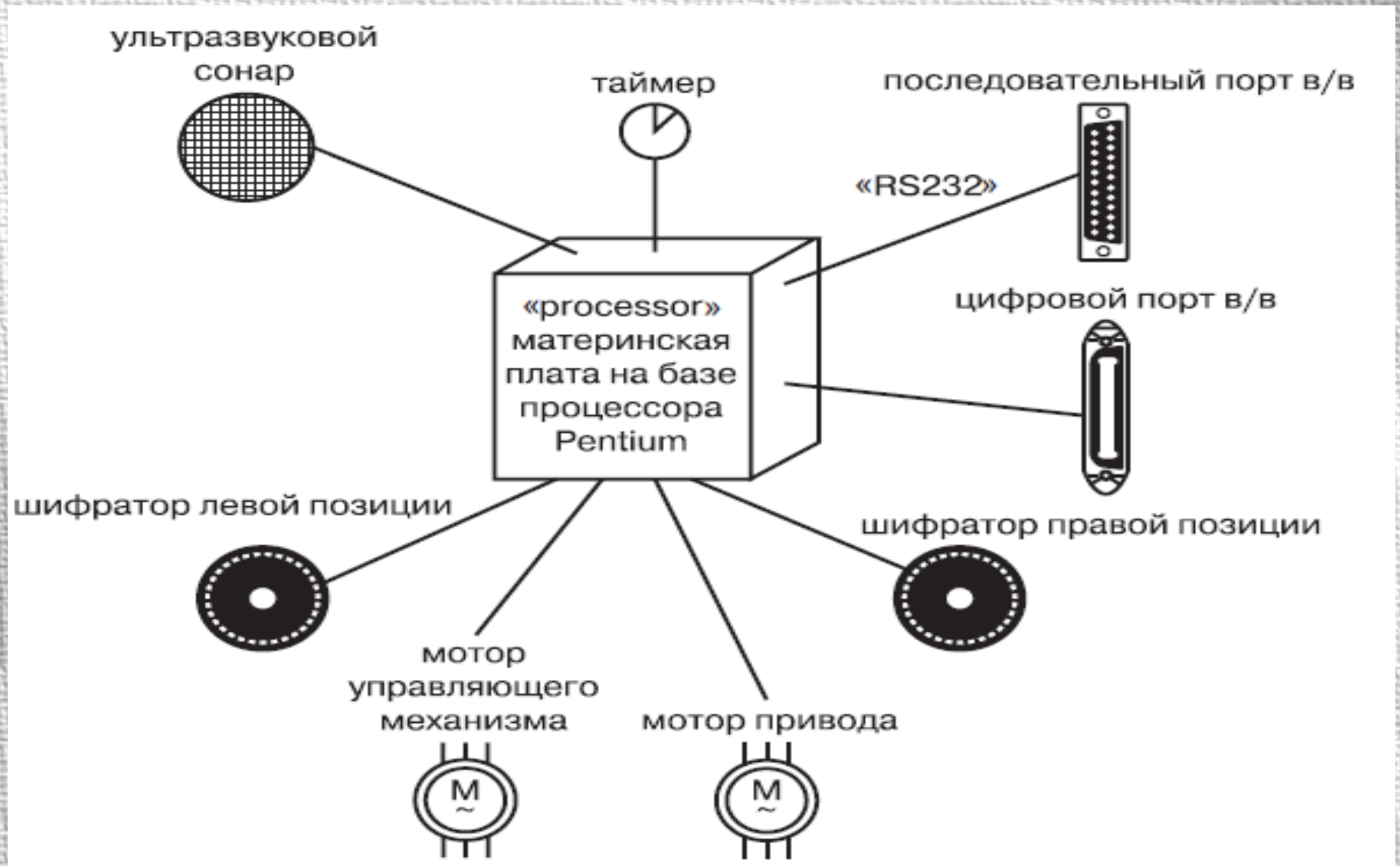


Моделирование процессоров и устройств

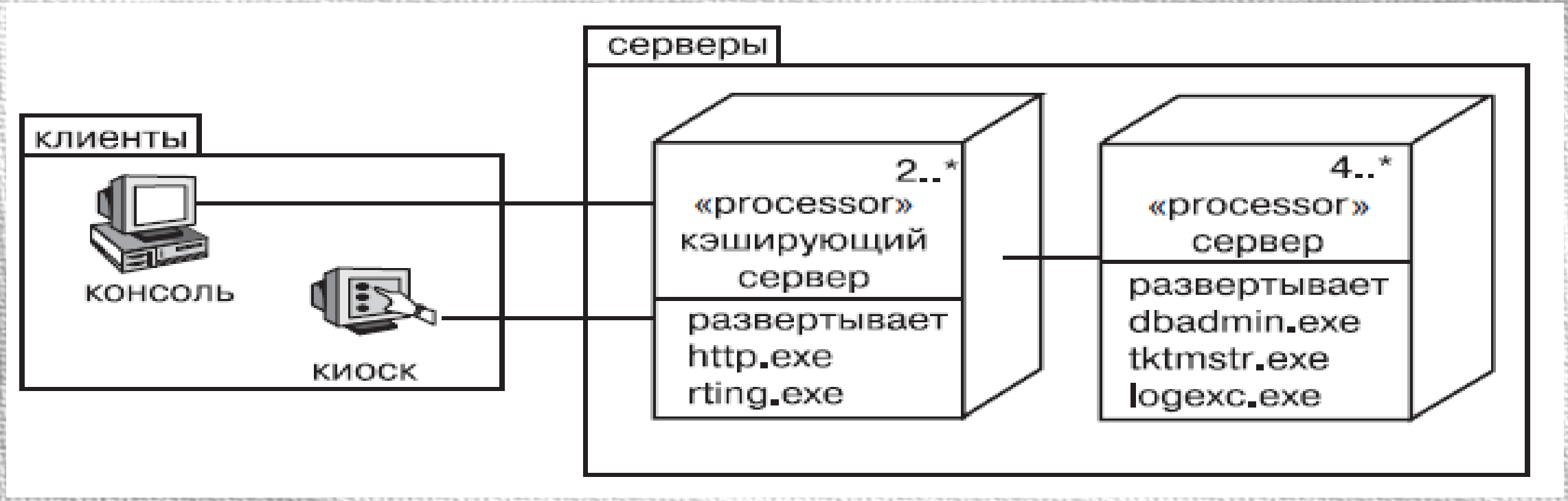



Internet
узел
модемный пул
соединение
«processor»
кэширующий
сервер
«processor»
кэширующий
сервер
узел
«network» локальная сеть
«processor»
основной
сервер
«processor»
сервер
«processor»
сервер
«processor»
сервер

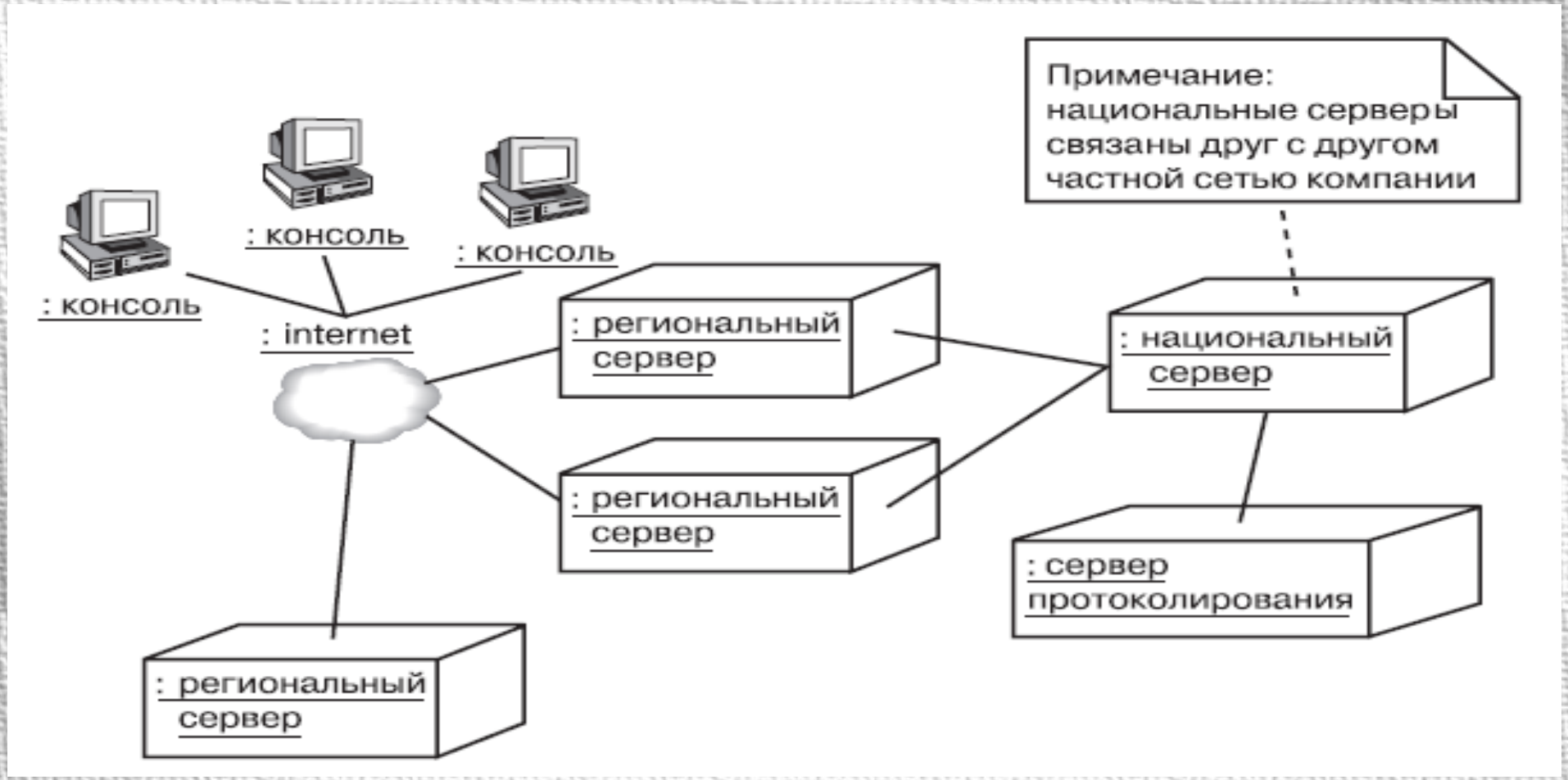
Моделирование встроенной системы



Моделирование клиент-серверной системы



Моделирование полностью распределенной системы



- ццццц

КООПЕРАЦИЯ

Уровни кооперации

Понятие кооперации (collaboration) является одним из фундаментальных понятий в языке UML. Оно служит для обозначения множества взаимодействующих с определенной целью объектов в общем контексте моделируемой системы. Цель самой кооперации состоит в том, чтобы специфицировать особенности реализации отдельных наиболее значимых операций в системе. Кооперация определяет структуру поведения системы в терминах взаимодействия участников этой кооперации.

Кооперация может быть представлена на двух уровнях:

1. **На уровне спецификации - показывает роли классификаторов и роли ассоциаций в рассматриваемом взаимодействии.**
2. **На уровне примеров - указывает экземпляры и связи, образующие отдельные роли в кооперации.**

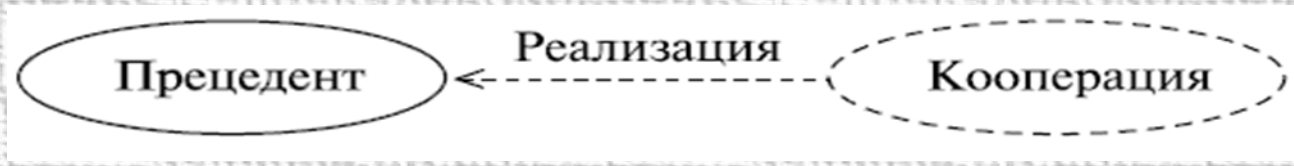
Диаграмма кооперации уровня спецификации

Диаграмма кооперации уровня спецификации показывает роли, которые играют участвующие во взаимодействии элементы. Элементами кооперации на этом уровне являются классы и ассоциации, которые обозначают отдельные роли классификаторов и ассоциации между участниками кооперации

ПРЕЦЕДЕНТ И КООПЕРАЦИЯ

О кооперации мы уже говорили ранее как о множестве ролей, работающих вместе, чтобы обеспечить некоторое поведение системы. Мы также упоминали о том, что прецеденты отвечают на вопрос "что делает система?", но не говорят, как именно она это делает. На этапе анализа понимать, как именно система реализует свое поведение, действительно не нужно. Но при переходе к реализации неплохо бы знать, какие именно классы (или другие элементы модели), совместно работая, обеспечивают нужное поведение.

То есть мы логично перешли от разговора о прецедентах к разговору о кооперации! Недаром обозначения кооперации и прецедента очень похожи (*кооперация* обозначается пунктирным эллипсом)



Так в каком же отношении находятся *прецедент* и *кооперация*?

Из предыдущего абзаца логично следует, что это *отношение* реализации. **Каждый *прецедент* реализуется одной или несколькими кооперациями.** Это, конечно, не означает, что классы жестко распределены *по* кооперациям: классы, принимающие участие в кооперации, реализующей определенный *прецедент*, будут участвовать и в других кооперациях.

РЕАЛИЗАЦИЯ ПРЕЦЕДЕНТА

Одно из назначений кооперацій состоит в моделировании прецедентов. Как правило, при анализе системы вы руководствуетесь тем, как она может использоваться. Переходя же к этапу реализации, вы должны реализовать идентифицированные прецеденты в виде конкретных структур и поведений. В общем случае каждый прецедент должен быть реализован одной или несколькими кооперациями. Если рассматривать систему в целом, то классификаторы, участвующие в кооперации, которая связана с некоторым прецедентом, будут принимать участие и в других кооперациях.

Моделирование реализации прецедента состоит из следующих этапов:

- Идентифицируйте те структурные элементы, которые необходимы и достаточны для осуществления семантики прецедента.
- Организуйте эти структурные элементы в диаграмму классов.
- Рассмотрите отдельные сценарии, которые представляют данный прецедент. Каждый сценарий описывает один из путей прохождения прецедента.
- Отобразите динамику этих сценариев на диаграммах взаимодействия. Воспользуйтесь диаграммами последовательности, если нужно подчеркнуть порядок сообщений, и диаграммами кооперации, если более важны структурные отношения между кооперирующимися объектами.
- Организуйте эти структурные и поведенческие элементы как кооперацию, которую вы можете соединить с прецедентом через реализацию.

Например, на рисунке изображены прецеденты, относящиеся к системе контроля кредитных карточек, в том числе основные из них - Разместить заказ и Создать счет, а также два вспомогательных - Распознать подлог и Проверить транзакцию. Хотя в большинстве случаев не возникает необходимости в явном моделировании этого отношения (такую задачу можно оставить инструментальным: средствам), на рисунке показана явная модель реализации. Разместить заказ с помощью кооперации Управление заказами. В свою очередь эта кооперация: может быть разложена на структурный и поведенческий аспекты, что в итоге дает диаграмму классов и диаграмму взаимодействия. Вот так, через отношение реализации, вы и связываете прецедент с его сценариями.

В большинстве случаев нет необходимости явно моделировать отношение между прецедентом и реализующей его кооперацией. Лучше оставить это на заднем: плане модели и позволить инструментальным средствам воспользоваться имеющейся связью для навигации между прецедентом и его реализацией.

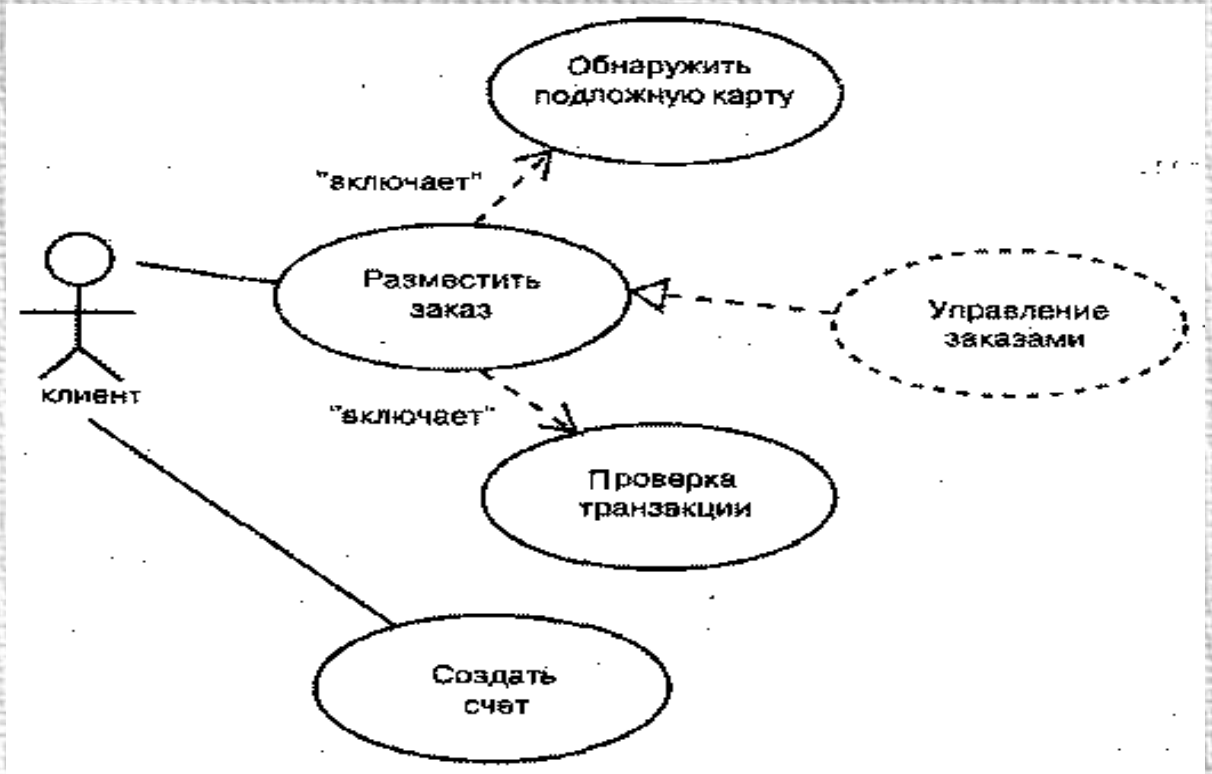


Диаграмма кооперации уровня спецификации

Кооперация на уровне спецификации изображается на диаграмме пунктирным эллипсом, внутри которого записывается имя этой кооперации. Такое представление кооперации относится к отдельному варианту использования и детализирует особенности его последующей реализации. Символ эллипса кооперации соединяется отрезками пунктирной линии с каждым из участников этой кооперации, в качестве которых могут выступать объекты или классы. **Каждая из этих пунктирных линий помечается ролью (role) участника.** Роли соответствуют именам элементов в контексте всей кооперации. Эти имена трактуются как параметры, которые ограничивают спецификацию элементов при любом их появлении в отдельных представлениях модели.

Диаграмма кооперации уровня примеров

Диаграмма кооперации уровня примеров представляется совокупностью объектов (экземпляры классов) и связей (экземпляры ассоциаций). При этом связи дополняются стрелками сообщений. На данном уровне показываются только релевантные объекты, т. е. имеющие непосредственное отношение к реализации операции или классификатора. В кооперации уровня примеров определяются свойства, которые должны иметь экземпляры для того, чтобы участвовать в кооперации. Кроме свойств объектов на диаграмме кооперации также указываются ассоциации, которые должны иметь место между объектами кооперации.

При этом вовсе не обязательно изображать все свойства или все ассоциации, поскольку на диаграмме кооперации присутствуют только роли классификаторов, но не сами классификаторы. Таким образом, в то время как классификатор требует полного описания всех своих экземпляров, роль классификатора требует описания только тех свойств и ассоциаций, которые необходимы для участия в отдельной кооперации.

Отсюда вытекает важное следствие. Одна и та же совокупность объектов может участвовать в различных кооперациях. При этом, в зависимости от рассматриваемой кооперации, могут изменяться как свойства отдельных объектов, так и связи между ними. Именно это отличает диаграмму кооперации от диаграммы классов, на которой должны быть указаны все свойства и ассоциации между элементами диаграммы.

СПЕЦИАЛЬНЫЕ ДИАГРАММЫ

- Диаграмма объектов
- Диаграмма внутренней структуры
- Обзорная диаграмма взаимодействия
- Диаграмма синхронизации
- Диаграмма пакетов

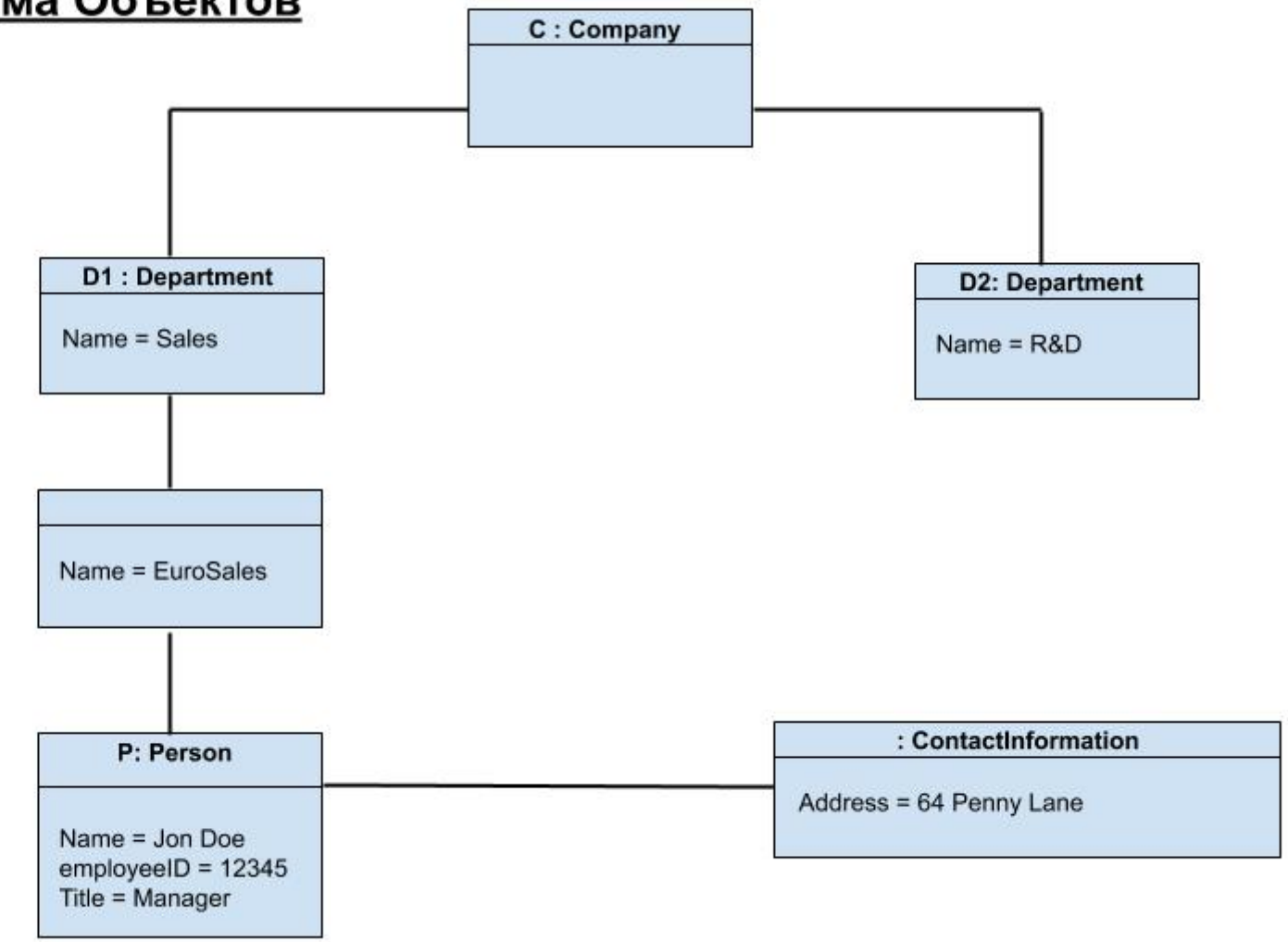
Диаграмма объектов

(object diagram)

Статическая диаграмма объектов является экземпляром диаграммы класса; она показывает снимок подробного состояния системы в определенный момент времени. Разница в том, что диаграмма классов представляет собой абстрактную модель, состоящую из классов и их отношений.

Тем не менее, диаграмма объекта представляет собой экземпляр в конкретный момент, который имеет конкретный характер. Использование диаграмм объектов довольно ограничено, а именно — чтобы показать примеры структуры данных.

Диаграмма Объектов



Нотация диаграммы объектов

Диаграмма объектов **является экземпляром диаграммы классов.**

На диаграмме объектов применяют один основной тип сущностей: объекты ❶ (экземпляры классов), между которыми указываются конкретные связи ❷ (чаще всего экземпляры ассоциаций).

Диаграммы объектов имеют вспомогательный характер – по сути это примеры (можно сказать, дампы памяти), показывающие, какие имеются объекты и связи между ними в некоторый конкретный момент функционирования системы.

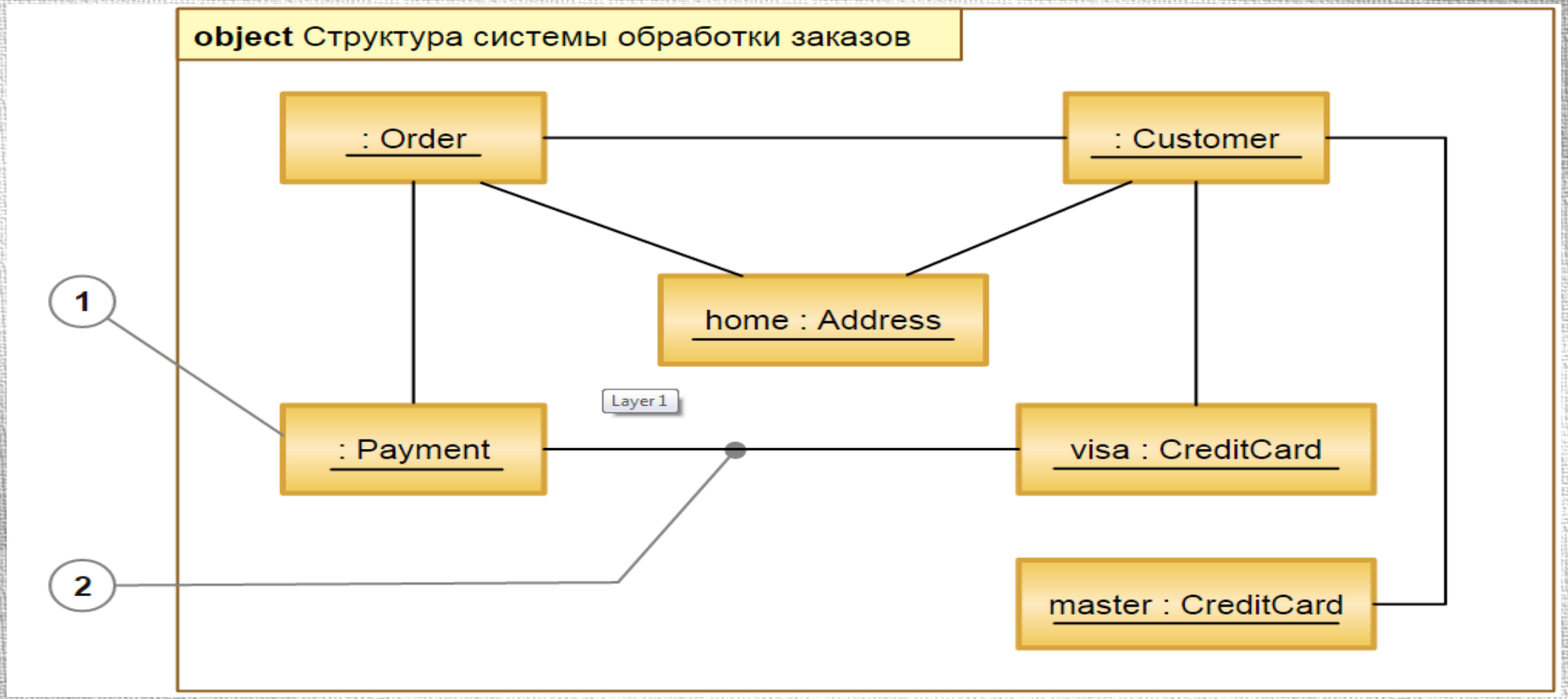

object Структура системы обработки заказов
: Order
: Customer
home : Address
1
Layer 1
: Payment
visa : CreditCard
2
master : CreditCard

Диаграмма внутренней структуры

(composite structure diagram)

Диаграмма составной структуры аналогична диаграмме классов и является своего рода диаграммой компонентов, используемой в основном при моделировании системы на микроуровне, но она изображает отдельные части вместо целых классов. Это тип статической структурной диаграммы, которая показывает внутреннюю структуру класса и взаимодействия, которые эта структура делает возможными.

Эта диаграмма может включать внутренние части, порты, через которые части взаимодействуют друг с другом или через которые экземпляры класса взаимодействуют с частями и с внешним миром, и соединители между частями или портами. Составная структура — это набор взаимосвязанных элементов, которые взаимодействуют во время выполнения для достижения какой-либо цели. Каждый элемент имеет определенную роль в сотрудничестве.

Диаграмма Составной Структуры

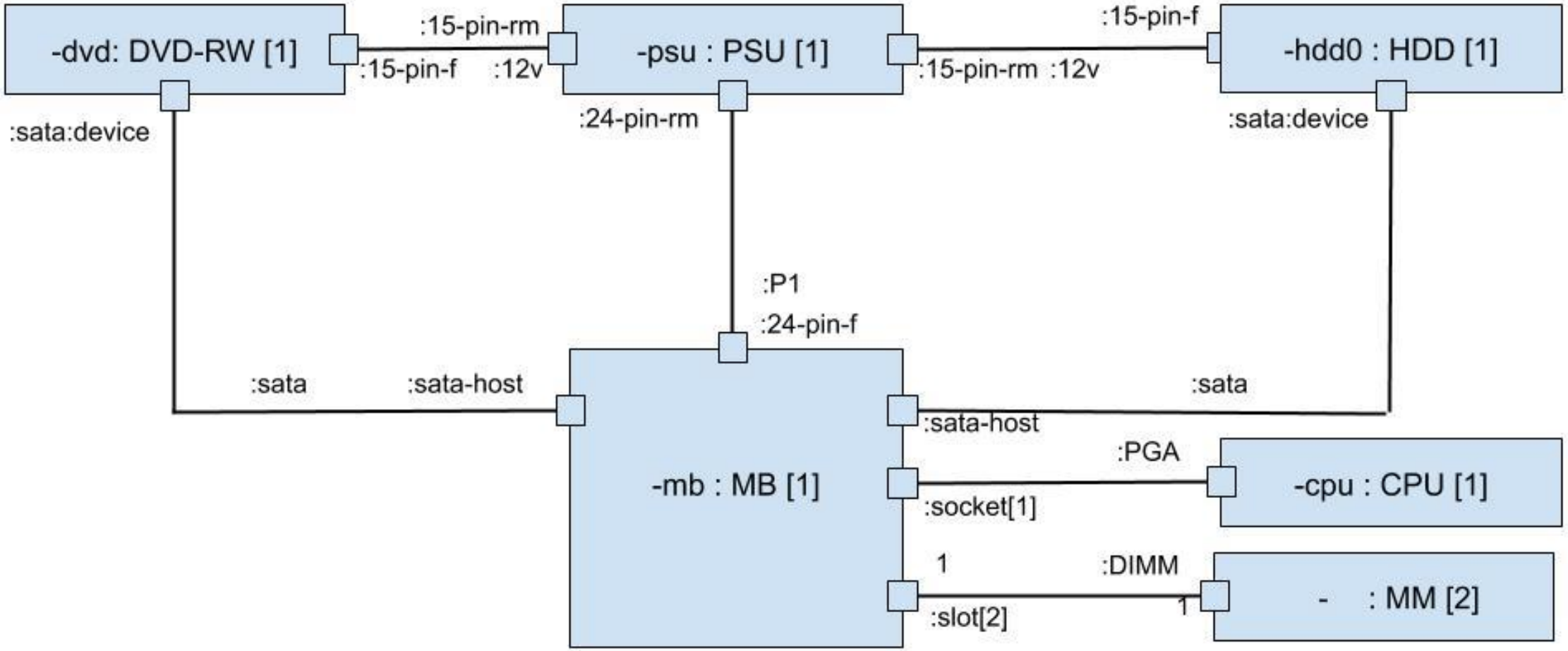


Диаграмма внутренней структуры **используется для более подробного представления структурных классификаторов, прежде всего классов и компонентов.**

Структурный классификатор изображается в виде прямоугольника ❶, в верхней части которого находится имя классификатора ❷. Внутри находятся части (parts) ❸. Частей может быть несколько. Каждая часть является экземпляром некоторого другого классификатора. Части могут взаимодействовать друг с другом. Это обозначается с помощью соединителей (connectors) ❹ различных видов. Место на внешней границе части, к которому присоединяется соединитель, называется портом (port) ❺. Порты располагаются также на внешней границе структурного классификатора ❻, обеспечивая ему связь с внешним миром.

Нотация диаграммы внутренней структуры

1. Структурный классификатор
2. Имя классификатора
3. Части
4. Соединитель
5. Порт

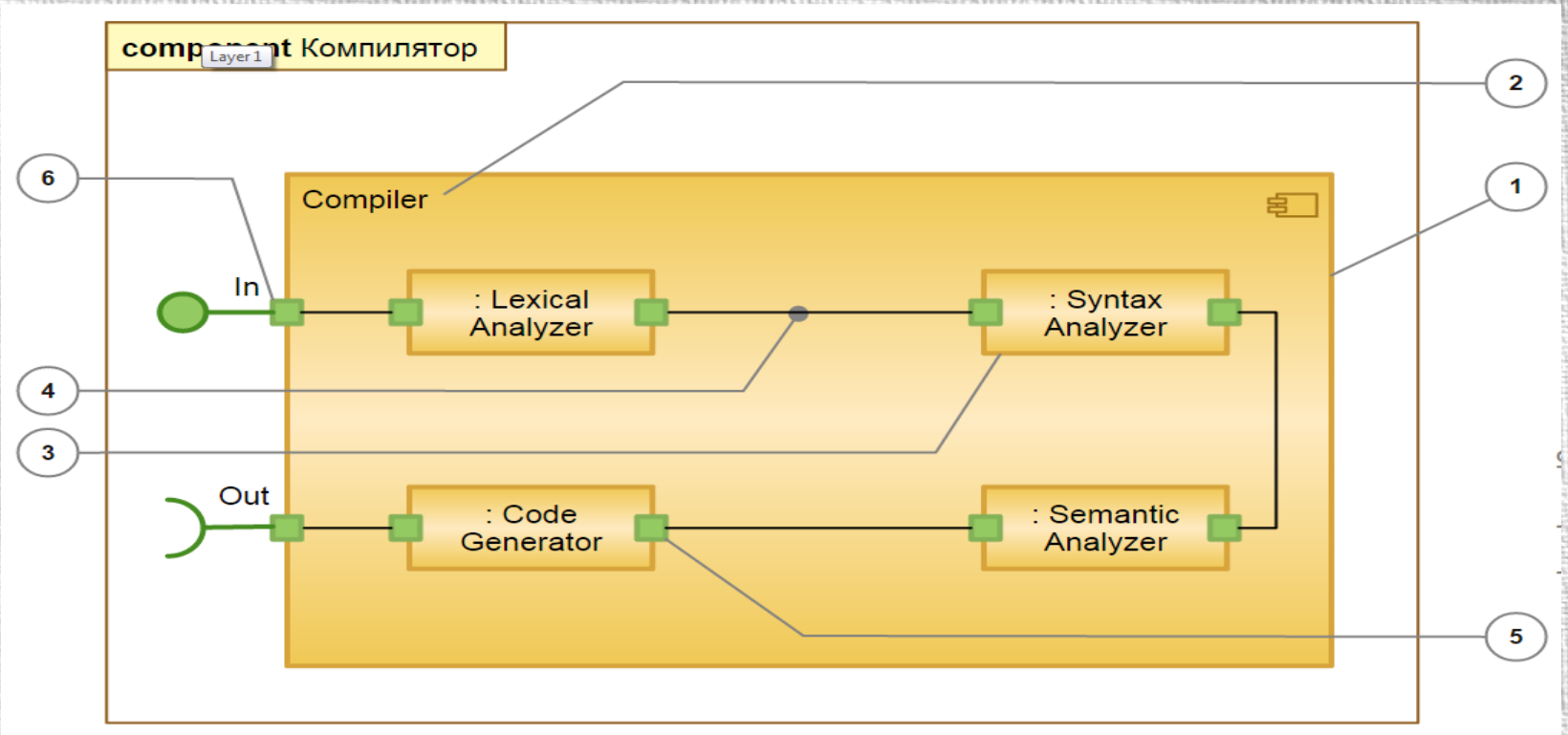
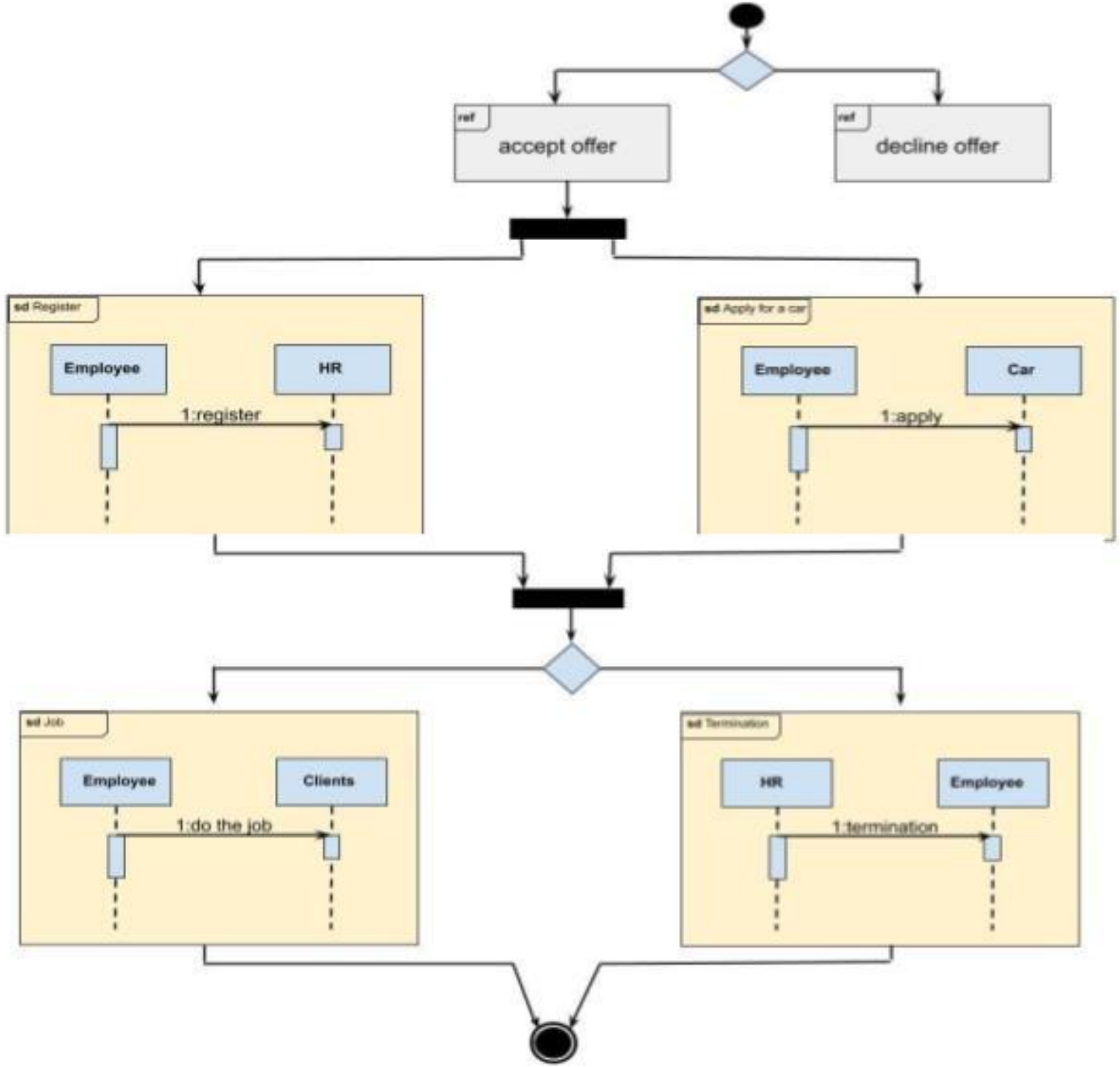


Диаграмма обзора взаимодействий фокусируется на обзоре потока управления взаимодействиями. Это вариант Диаграммы деятельности, где узлами являются взаимодействия или события взаимодействия. Диаграмма обзора взаимодействий описывает взаимодействия, в которых сообщения и линии жизни скрыты. Мы можем связать «реальные» диаграммы и добиться высокой степени навигации между диаграммами внутри диаграммы обзора взаимодействия.


ref
accept offer
ref
decline offer
sd Register
Employee
HR
1:register
sd Apply for a car
Employee
Car
1:apply
sd Job
Employee
Clients
1:do the job
sd Termination
HR
Employee
1:termination

Обзорная диаграмма взаимодействия **является разновидностью диаграммы деятельности с расширенным синтаксисом: в качестве элементов обзорной диаграммы взаимодействия могут выступать *ссылки на взаимодействия* (interaction use) ❶, определяемые диаграммами последовательности.**

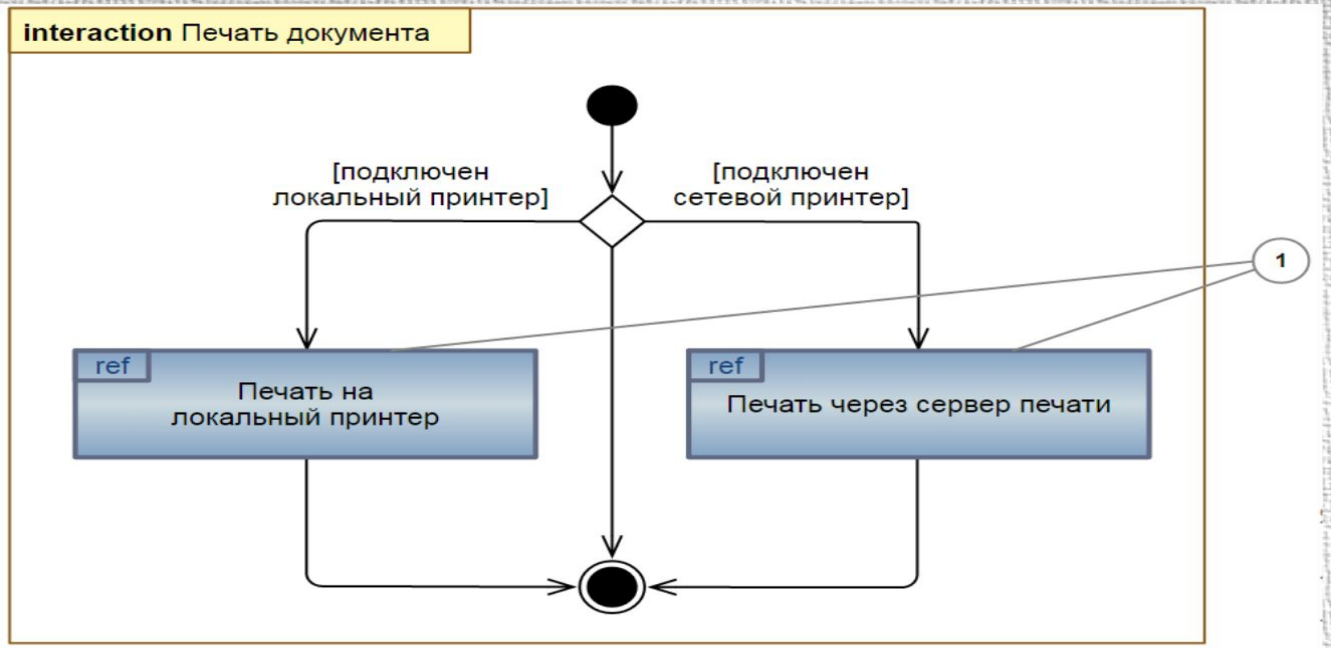
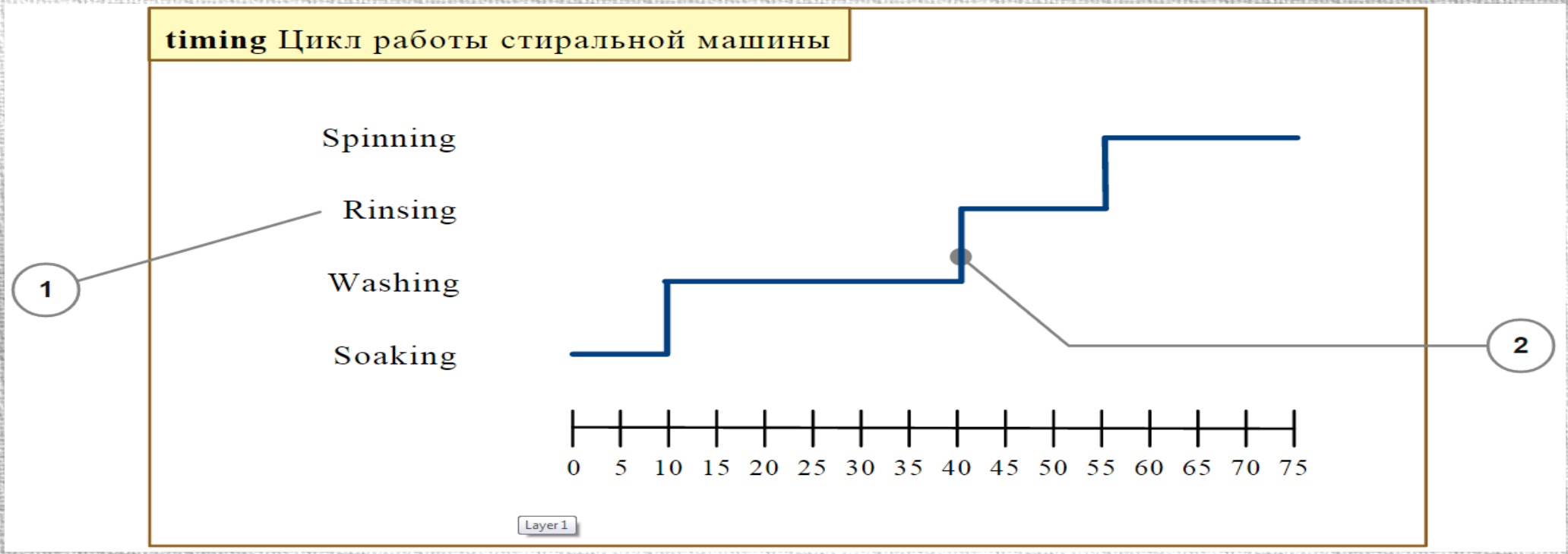
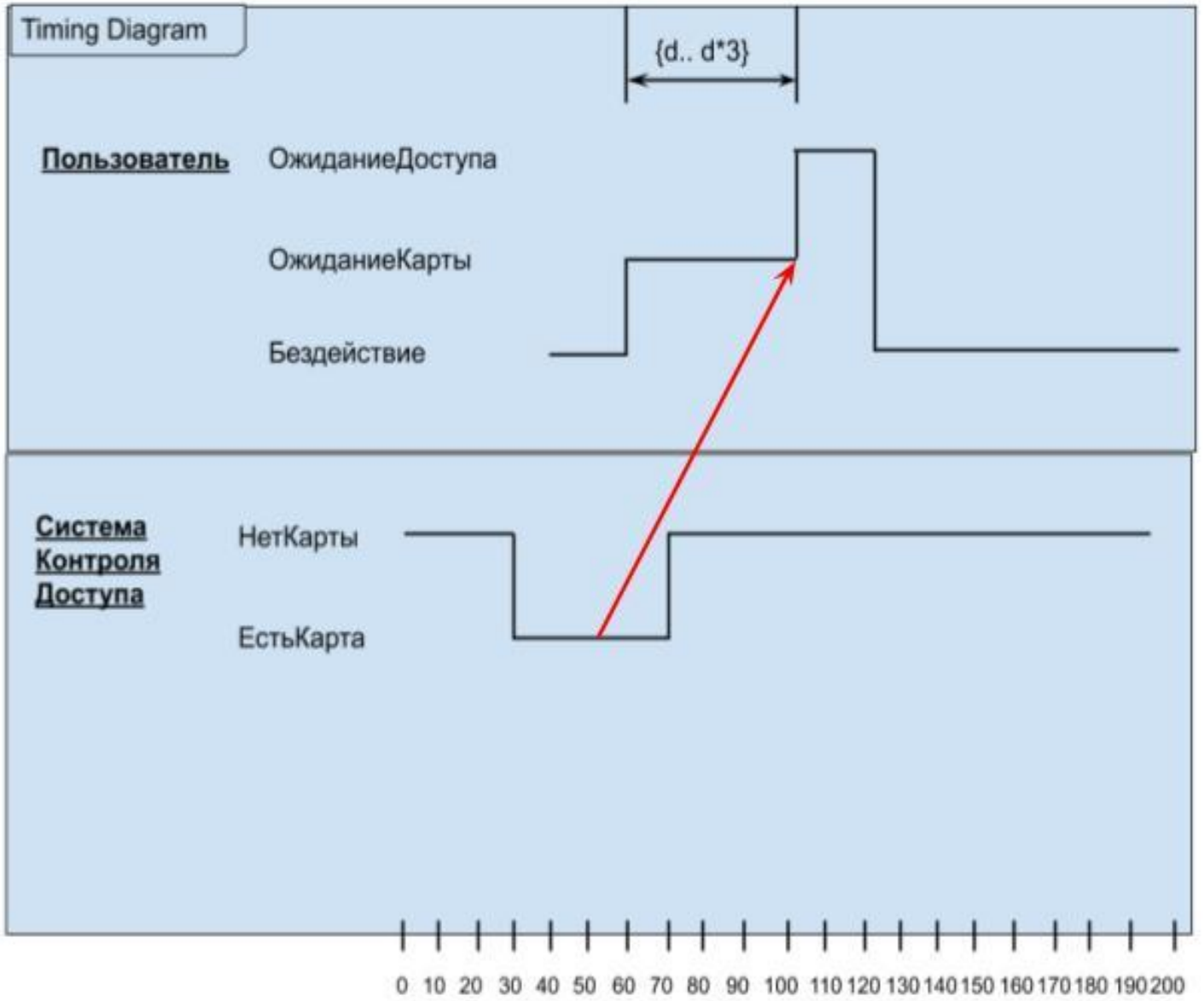


Диаграмма синхронизации **(timing diagram) представляет собой особую форму диаграммы последовательности, на которой особое внимание уделяется изменению состояний ❶ различных экземпляров классификаторов и их временной синхронизации ❷.**



Временная Диаграмма



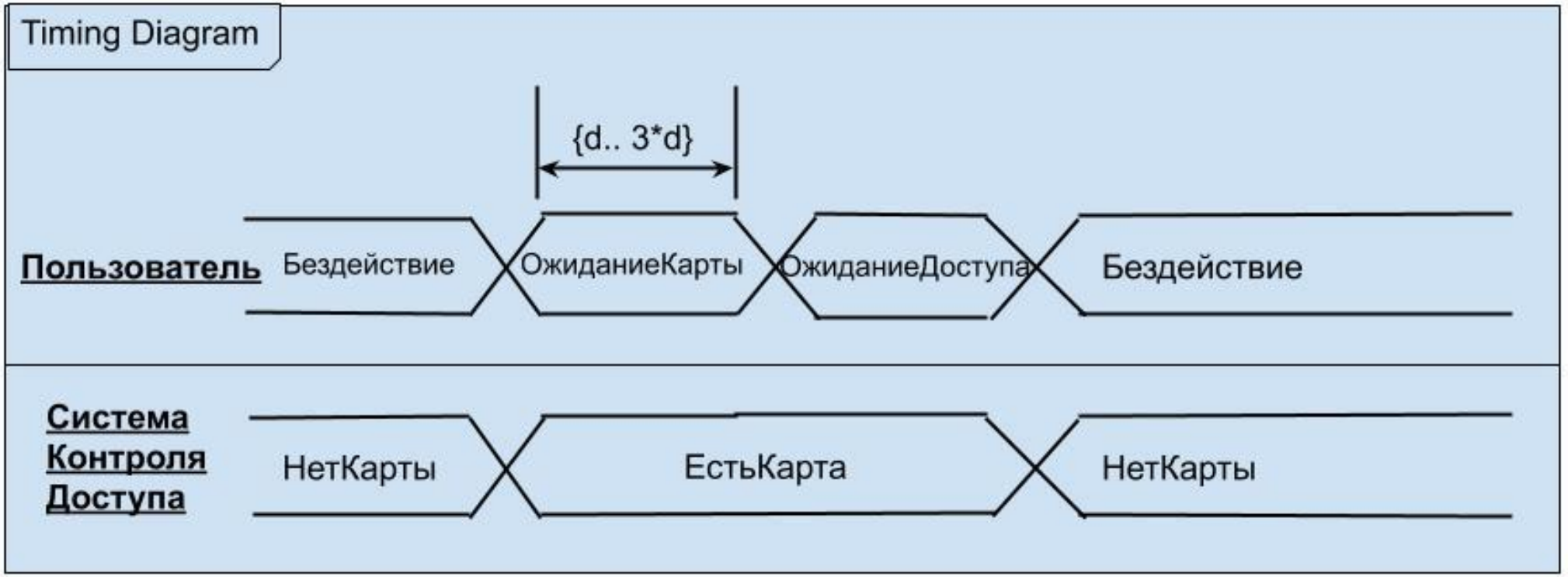

Timing Diagram
{d.. 3*d}
Пользователь
Бездействие
ОжиданиеКарты
ОжиданиеДоступа
Бездействие
Система Контроля Доступа
НетКарты
ЕстьКарта
НетКарты

Диаграмма профилей позволяет нам создавать специфичные для домена и платформы стереотипы и определять отношения между ними. Мы можем создавать стереотипы, рисуя формы стереотипов и связывая их с композицией или обобщением через интерфейс, ориентированный на ресурсы. Мы также можем определять и визуализировать значения стереотипов.

Диаграмма Профилей